

圆柱药材烘干的二维热湿传递建模与收缩一致性分析

摘要

本文针对圆柱形中药材热风烘干过程中的温度、水分迁移及尺寸收缩问题，建立**二维轴对称热湿传递模型**，显式考虑药材侧面与端面的热质交换。根据各物性关系和环境数据，采用有限体积法进行空间离散，并结合**非线性扩散积分通量与隐式 BDF 方法**求解温度场和含水率场；对于尺寸变化问题，则在移动坐标下同步更新扩散距离、控制体体积和交换面积。

对于问题一，建立预热阶段的**二维轴对称热湿传递模型**，计算 0 ~ 1800 s 内药材温度和含水率的空间分布。结果表明，1800 s 时药材中面中心温度为 33.5753 °C，侧面中部温度为 36.7855 °C；对应含水率分别为 2.5500 kg/kg 和 1.5103 kg/kg。初始热扩散率约为水分扩散系数的 34.2 倍，说明预热阶段热量已明显向内部传递，而失水仍主要集中在外层，**热响应显著快于水分迁移**。

对于问题二，将密度、比热、导热系数和水分扩散系数随温湿状态更新，建立**变物性热湿耦合模型**。得到 3 h 时中面中心与侧面中部温度分别为 49.8508 °C 和 49.9672 °C，温差仅为 0.1164 °C；含水率分别为 1.7662 kg/kg 和 1.0081 kg/kg，仍相差 0.7580 kg/kg。结果表明温度场已基本均匀，而水分场仍存在明显梯度，且外层失水导致扩散系数降低，使**内部水分迁移成为后续干燥的主要限制**。

对于问题三，利用与问题二相同的二维轴对称热湿传递模型，以药材全域含水率均低于 0.15 kg/kg 为干燥完成判据，对问题二的热湿过程继续积分。得到连续临界干燥时间为 **57.4722 h**，首个严格满足要求的整数时刻为 206900 s。干燥过程中表面明显早于内部达到要求，最终最湿位置位于中面轴心；说明仅依据表面含水率判断干燥终点会造成提前终止，**药材内部湿芯控制了干燥后期的完成时间**。

对于问题四，根据附件给出的半径变化构造二维移动区域，并采用**径向、轴向等比例收缩假设（长径比相等假设）**求解。得到连续临界干燥时间为 **51.0920 h**，首个严格达标整数时刻为 183932 s，此时药材半径为 1.2000 cm，等比例假设下长度为 15.0000 cm。

质量-几何一致性检验发现，若将题给密度解释为当前真实湿密度，则主模型的水分场、几何变化与干物质质量守恒不能同时满足。对于问题四，当半径为 1.20 cm 且全域严格达标时，可解析得到 $25.469 \text{ cm} \leq L < 28.778 \text{ cm}$ ，因此**等比例缩短至 15 cm 与质量-几何约束不相容，甚至 25 cm 初始长度也不在范围内**；进一步建立守恒变长度模型，得到达标长度约为 28.03 cm。

另一方面，真实汽化过程必然伴随潜热消耗，但题给条件不足以唯一确定表面平衡关系及相应参数。为判断这些缺失参数能否通过经验赋值处理，选取三组候选表面平衡条件配对计算，结果表明不同参数不仅改变绝对干燥时长，计入潜热后产生的时长增量也达到 8.29–12.91 h。这说明**潜热影响不可忽略，且缺失的表面参数对结果具有显著影响，在缺乏独立观测约束时不能任意拟值或选取**；因此现有条件尚不足以给出唯一可信的含潜热烘干时长。

关键词：二维轴对称；热湿耦合；有限体积法；尺寸收缩；质量-几何一致性

1 问题重述

1.1 问题背景

干燥是中药材加工的重要工序，其工艺条件直接关系到干燥效率、能量消耗和成品品质。热风烘干通过调节烘房的温湿环境，使药材经历预热平衡和恒温干燥两个阶段，逐步降低含水率。工艺参数选择不当，容易造成干燥时间过长或品质不稳定；单纯依靠反复试验调整参数，又存在成本高、周期长的问题。因此，借助数学模型分析烘干规律，可为工艺条件的选择提供依据。

药材烘干包含热量传递和水分迁移两个相互影响的过程：**热风向药材供热，内部水分逐渐向外迁移并排入环境**。由于热量和水的传递速度不同，表面与内部的温度、含水率会随时间形成差异，表面变干时内部仍可能保留较多水分。因此，若想判断烘干是否完成，就需要了解药材内部的含水率分布。

这一过程还受到材料物性和尺寸变化的影响。温度与含水率改变时，药材的导热、储热及水分扩散能力也会变化；失水引起的收缩则进一步改变内部迁移距离和表面交换面积。本文以圆柱形药材为对象，结合烘房环境记录、材料物性关系和半径观测数据，研究各阶段的温度与含水率变化，确定全域达到干燥要求的时间，并分析尺寸收缩对烘干过程的影响。

1.2 研究任务

药材初始半径为 $R_0 = 0.02\text{ m}$ ，整根长度为 $L_0 = 0.25\text{ m}$ ，初始温度为 $T_0 = 28^\circ\text{C}$ ，初始干基含水率为 $C_0 = 2.55\text{ kg/kg}$ 。附件 1 提供热风温度和空气水分指标，附件 2 提供半径随时间的观测，附件 3 给出结果表格式。四问的数据与输出关系见表 1。

表 1 四问的条件与输出对应关系

问题	几何与计算时段	需要回答的问题
一	初始圆柱，0—1800 s	预热阶段各位置的温 度、含水率
二	初始圆柱，0—3 h	变物性条件下的温湿 分布
三	初始圆柱，积分至全 域达标	全域含水率低于 0.15 的时间
四	附件 2 半径驱动的收 缩圆柱	当前尺寸下的含水率 及达标时间

四问使用题设给定的同一组初始状态和附件 1 环境。问题二从原始初态开始，问题三在问题二的初值问题上继续积分，以求得后续达标时间。**附件 2 的半径记录只用于第四问**，以及文中明确标注的第四问条件研究。

2 问题分析

2.1 预热过程分析

问题一要求分析半小时内药材各位置的温度和含水率变化。热量由表面向内部传导，水分向外迁移，**两者的传递速度不同**，表面与内部的响应也有差异。因此，需要结合题给初态、材料物性和随时间变化的烘房环境，分析**温度与含水率的时空分布**，判断预热阶段升温和失水对内部的影响程度。

2.2 变物性过程分析

问题二将研究时段延长至三小时，并考虑**材料物性随温湿状态变化**。温度影响水分扩散，失水又改变材料的储热和导热能力，二者相互影响。本问需要考虑**传热与水分迁移的耦合**，分析这种反馈如何改变温湿分布，以及表面与内部的差异如何随时间演化。

2.3 干燥终止条件分析

问题三要求确定药材**全域含水率低于 0.15 kg/kg** 的时间。各位置的干燥进度不同，表面达标时内部仍可能偏湿，因此应以**全域最后达标的时刻**作为干燥终点。分析需关注最湿位置及其含水率变化，并明确**环境记录结束后的条件**，才能确定后续烘干时间。

2.4 尺寸收缩影响分析

问题四需考虑失水过程中尺寸变化的影响。收缩同时改变**扩散距离、材料体积和交换面积**；附件只给出半径变化，**轴向尺寸及内部变形仍需补充几何假设**，并检查其与密度、水分场的相容性。此外，问题三、四的物性不同，应在**相同物性和环境条件下比较固定尺寸与收缩情况**，才能区分几何变化对烘干时间的作用。

3 总体建模思路

本文以**二维轴对称热湿传递模型**完成四问求解。

由于圆柱药材的特殊几何性质，一种自然的建模想法是将圆柱的侧面面积通过折合比例折合为侧面，以实现隐式 BDF 方法和非线性扩散基本通量，但是考虑到圆柱药材的侧面和端面均与热风接触，**内部温湿状态可能同时沿径向和轴向变化**，因此分别计算两类表面的交换，保留端部与中部的差异。同时设置一维模型，在相同条件下进行对照，以评估空间简化对温湿分布和烘干时间的影响。

四问在同一种二维框架下按题给条件展开：问题一计算预热阶段的温湿分布；问题二引入**随状态变化的物性**，联立求解传热与水分迁移；问题三延续问题二的初值问题，以**全域最大含水率低于阈值**确定烘干时间；问题四采用对应物性和附件半径，在等比例收缩假设下更新计算区域，分析收缩过程中的水分分布及达标时间。通过网格加密、时间精度收紧和离散收支检查，评估上述结果的数值可靠性。

得到四问结果后，进一步**物理一致性检验**。当题给密度按真实湿密度解释时，原有计算条件存在不相容之处。针对第四问，另行建立守恒变长度模型，由半径、水分场和密度关系约束轴向长度，并比较两种局部变形映射，考察结果对内部运动学假设的敏感程度。该部分用于分析几何约束与质量守恒之间的关系，**其结果作为条件解单独报告**。

主模型未显式计入蒸发潜热，因而还需考察这一省略对能量收支和烘干时间的影响。为此，设置**表面交换与蒸发耗能配套的潜热条件研究**，在同一表面平衡关系下配对比较开启、关闭相变能量

通道的结果。由于所需表面参数尚未得到独立数据约束，再比较不同候选参数下的响应，说明潜热影响及结论的适用范围。各部分的关系见图 1。

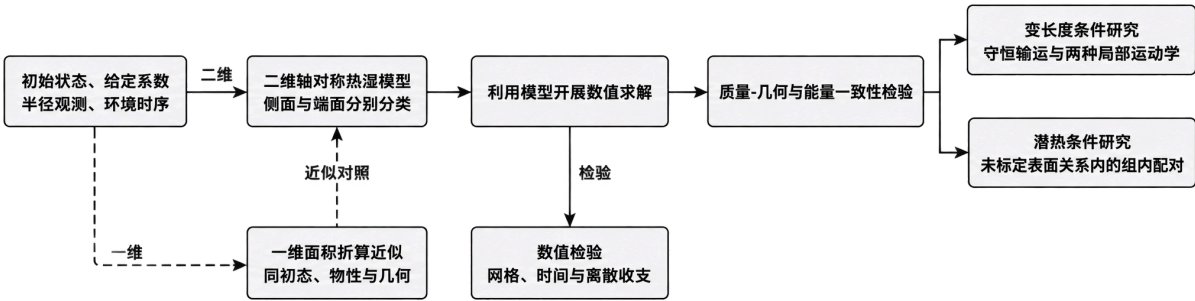


图 1 四问求解及其检验关系。二维主线给出题定结果；一维用于空间近似对照，变长度与潜热计算分别回答附加条件下的相容性和能量影响。

正文四问报告二维主模型的数值，完整一维计算列入附录 A。变长度时长对应所选密度解释与运动学，潜热时长对应未标定的表面参数。这两类条件研究用于检验主模型的适用范围，其结果单独报告，不填入题定结果表。

4 模型假设

模型采用下列近似，各项的作用范围在后续推导和对照中保持一致。

1. 药材视为**等效连续介质**，使用题给标量物性表示导热与水分扩散，暂不解析细胞、孔隙和不同组织。
2. 药材**外形保持圆柱**，温湿场满足轴对称与中面对称；计算中保留径向和轴向变化。
3. 侧面及两个端面处于**同一热风环境**，使用相同的题给换热、传质系数。
4. 主模型用**内部导热与表面对流**描述温度变化，不另设辐射、内部反应热或应力方程。
5. 水分采用**有效扩散及既有表面经验关系**描述，不显式区分液态水、水蒸气 and 结合水。
6. 问题一至三**保持初始尺寸**；第四问半径取附件值，长度跟随**长径比不变**约束缩放。
7. 观测点之间**分段线性插值**；超过环境观测时段后，以**末小时均值延拓**。半径只在附件覆盖区间内插值。
8. 二维主模型**不显式计入蒸发潜热与迁移水携带的显焓**；后文分别讨论这些取舍与质量、表面平衡条件的关系。

5 符号说明

表 2 列出主要符号。 C 为水质量与干物质质量之比； w_a 保留附件的空气水分指标含义，二者的质量基准不混用。 ρ 为题给密度关系， ρ_d 为单位当前材料体积的干质量。 L 一律表示整根长度，计算半长为 $L/2$ 。

表 2 主要符号

符号	含义	单位
T, T_a, T_K	材料温度、热风温度、材料绝对温度	$^{\circ}\text{C}, ^{\circ}\text{C}, \text{K}$
C, C_0, C_{eq}	干基含水率、初值、表面平衡含水率	kg/kg 干物质
w_a	附件空气水分指标	kg/kg
r, z	当前径向、距中面的轴向坐标	m
R, L, V	当前半径、整根长度、整根体积	$\text{m}, \text{m}, \text{m}^3$
ξ, η	等比例移动域的归一化坐标	1
ρ, ρ_d	湿密度、单位材料体积干质量	kg/m^3
c_p, k, D	比热、导热系数、水分扩散系数	$\text{J}/(\text{kg K}), \text{W}/(\text{m K}), \text{m}^2/\text{s}$
h, h_m	对流换热、题给传质系数	$\text{W}/(\text{m}^2\text{K}), \text{m}/\text{s}$
M_d, M_w	整根干物质质量、水质量	kg
F, q	标量扩散通量、导热通量	$\text{m}/\text{s}, \text{W}/\text{m}^2$
j, J_s	相对骨架的水质量通量、其外法向分量	$\text{kg}/(\text{m}^2\text{s})$
v, D_t	材料速度、材料导数	$\text{m}/\text{s}, \text{s}^{-1}$
$a, \zeta, \mathcal{J}$	参考径向、参考轴向坐标、体积变形比	$\text{m}, \text{m}, 1$
H, h_b, h_v	单位干质量显焓、水有效显焓、气相水焓	J/kg
下标 s, c, f	外表面、中面轴心、共享面	—

绝对温度按 $T_K = T + 273.15$ 换算；相同温差在摄氏与开尔文尺度上的数值相等。几何、时间和系数在程序内使用 SI 单位，表格和图中按需要换算为 cm 、 h 。

6 二维热湿耦合模型

6.1 端面效应分析

圆柱的侧面积为 $2\pi RL$ ，两个端面的总面积为 $2\pi R^2$ 。按题给初态计算，端面与侧面积之比为 0.08，端面占总交换面积的 7.41%。若一维模型以同一代表性表面状态描述侧面和端面，就可把**两类表面的总交换量折算到侧面**，得到

$$\gamma = \frac{2\pi RL + 2\pi R^2}{2\pi RL} = 1 + \frac{R}{L}, \quad h_{\text{eff}} = \gamma h, \quad h_{m,\text{eff}} = \gamma h_m. \quad (1)$$

初始面积折算因子为 $\gamma = 1.08$ 。前三问保持尺寸固定，第四问的一维对照采用等比例缩放，因此这些对照中的 γ 均不随时间改变。**折算保留了总交换面积，但同时假定各外露面可由同一个表面状态代表**。几何与计算域见图 2。

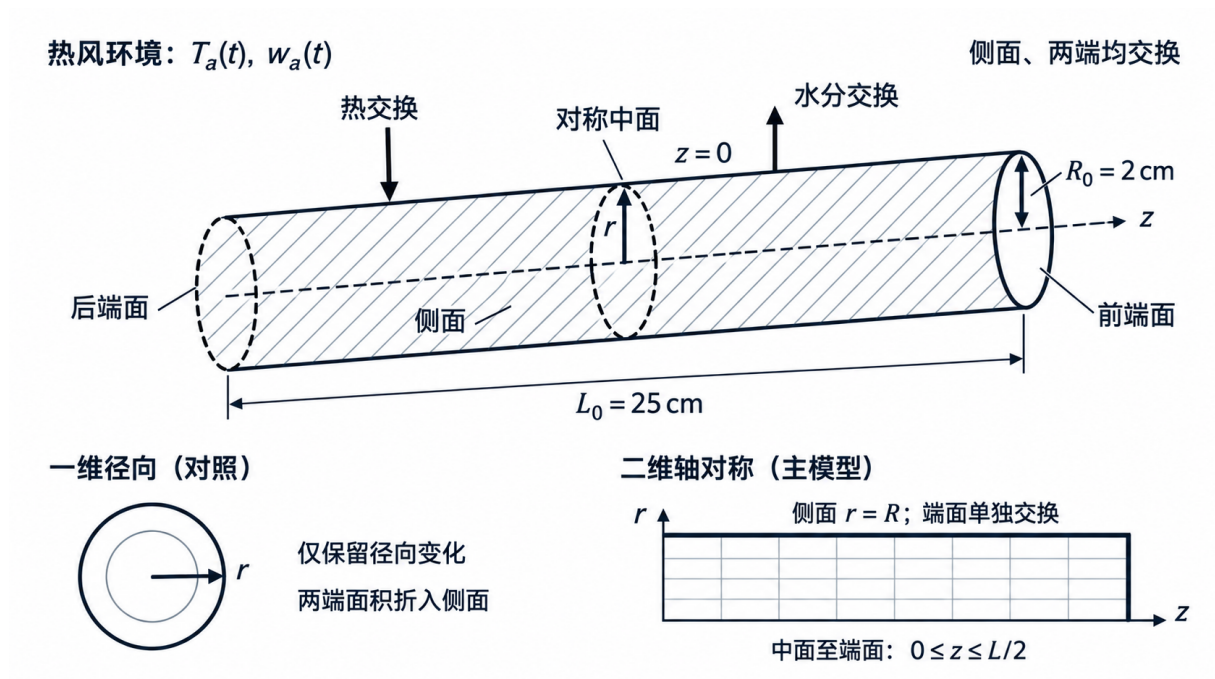


图2 圆柱外形、交换边界与计算区域。三维投影采用初始尺寸，半纵截面从中面延伸至端面；下部一维区域用于解释端面折算对照。

在同一物性、初态、环境、经验边界及几何条件下，将一维结果与二维中面场、轴向平均场比较，得到表3。轴向平均定义为

$$\bar{f}(r, t) = \frac{2}{L(t)} \int_0^{L(t)/2} f(r, z, t) dz, \quad f \in \{T, C\}. \quad (2)$$

表3 一维与二维在共同保存采样集上的最大绝对差

问题	中面温差 / °C	中面含水率差	轴向平均温差 / °C	轴向平均含水率差	二维减一维时差 / min
一	0.2674	0.0558	0.1338	0.0425	—
二、三	0.3697	0.0581	0.1351	0.0441	28.57
四	0.3300	0.0510	0.2305	0.0780	11.18

含水率差的单位为 kg/kg。问题三、四的时差分别约占一维时长的 0.84% 和 0.37%，表明面积折算能够近似本组计算的总体时间。局部分布仍有区别：端面附近可以沿轴向交换热量和水分，中部与端部因而难以用同一个径向场表示。正文据此采用二维模型，完整的一维推导、四问结果表及局部误差图列入附录 A。表内最大差仅指已保存的共同采样集。

6.2 热湿传递方程

令轴向原点位于药材中面，计算半纵截面为

$$\Omega_{rz}(t) = \{(r, z) : 0 \leq r \leq R(t), 0 \leq z \leq L(t)/2\}. \quad (3)$$

该截面绕轴旋转得到半根圆柱，另一半由中面对称确定。半根圆柱的体积元为 $2\pi r dr dz$ ；计算整根的质量或体积时，需要将其加倍，使用权重 $4\pi r dr dz$ 。圆柱坐标径向散度中的几何因子，以及后文整根质量积分的权重，都与这一体积表示对应。

对温度场，局部储热率写成热容量与温升速率的乘积，邻域导热通量的净输入决定这一储热变化。对含水率场，则采用题给有效扩散系数描述水分在空间中的迁移。两者组成下列方程：

$$\begin{aligned}\rho(C)c_p(C)D_t T &= \frac{1}{r} \frac{\partial}{\partial r} \left(rk(C) \frac{\partial T}{\partial r} \right) + \frac{\partial}{\partial z} \left(k(C) \frac{\partial T}{\partial z} \right), \\ D_t C &= \frac{1}{r} \frac{\partial}{\partial r} \left(rD(C, T) \frac{\partial C}{\partial r} \right) + \frac{\partial}{\partial z} \left(D(C, T) \frac{\partial C}{\partial z} \right).\end{aligned}\quad (4)$$

其中 $D_t = \partial_t + \mathbf{v} \cdot \nabla$ 为跟随材料的导数。固定圆柱中 $\mathbf{v} = 0$ ；等比例移动域中的材料速度由第 6.5 节给定。径向项中的 r 保留圆柱壳面积变化 [2]，轴向项则描述中部与端部之间的传递。

第二式直接推进干基含水率这一标量，**模型尚未联立干物质连续方程**。温度方程中的题给 $\rho(C)$ 用于计算局部热容量。若还要把它用于真实湿质量的计算，就需要检查同一密度关系是否与水分场及几何相容，第 9 章的质量诊断专门检验这一点。

6.3 物性关系

问题一采用题目附录 2：

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (5)$$

问题二、三采用题目附录 3：

$$\begin{aligned}\rho(C) &= 650 + 128C, & c_p(C) &= 1450 + \frac{2736C}{1+C}, \\ k(C) &= 0.21 + \frac{0.38C}{1+C}, & D(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C} - \frac{3850}{T_K}\right).\end{aligned}\quad (6)$$

问题四采用题目附录 4：

$$\begin{aligned}\rho(C) &= 760 + 90C, & c_p(C) &= 1850 + \frac{2150C}{1+C}, \\ k(C) &= 0.12 + \frac{0.20C}{1+C}, & D(C, T) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C} - \frac{3850}{T_K}\right).\end{aligned}\quad (7)$$

式中各量采用表 2 所列 SI 单位。问题二至四的反馈关系见图 3：**升温使扩散系数增大，失水又改变导热系数和热容量**，继而影响温度响应。问题一使用常数热物性，只保留其专属公式中的依赖关系，并不包含图中的全部反馈。

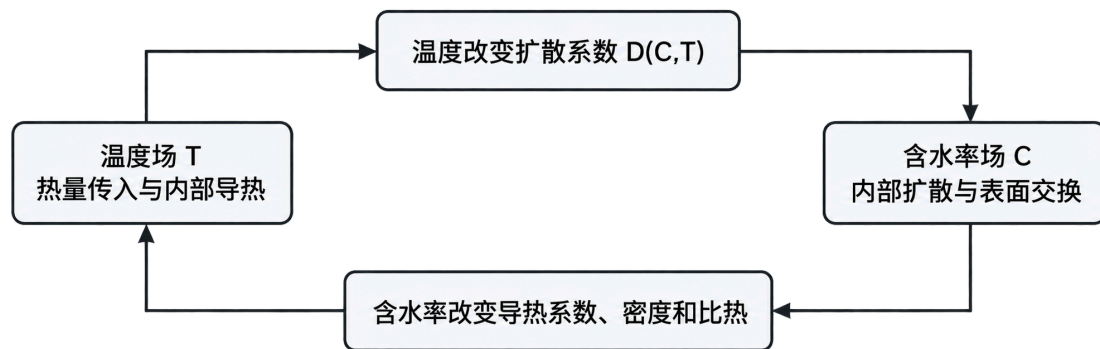


图 3 问题二至四的物性耦合。箭头表示系数对状态的依赖；隐式时间步内，两个场与其物性同时更新。

图 4 给出物性对状态变化的响应。温度一定时, $\exp(-a/C)$ 在低含水率区迅速下降, 较干外层的扩散能力随之减弱; 含水率一定时, 升温则提高扩散系数。表面先失水后, **外层形成的低扩散区域会逐渐限制内部水分向外迁移**。已有材料实验也考察了温度对干燥扩散的影响 [3], 本题计算仍使用各问附录给定的系数。右图比较的附录差别还说明, 解释跨问时长差时必须同时考虑物性变化。

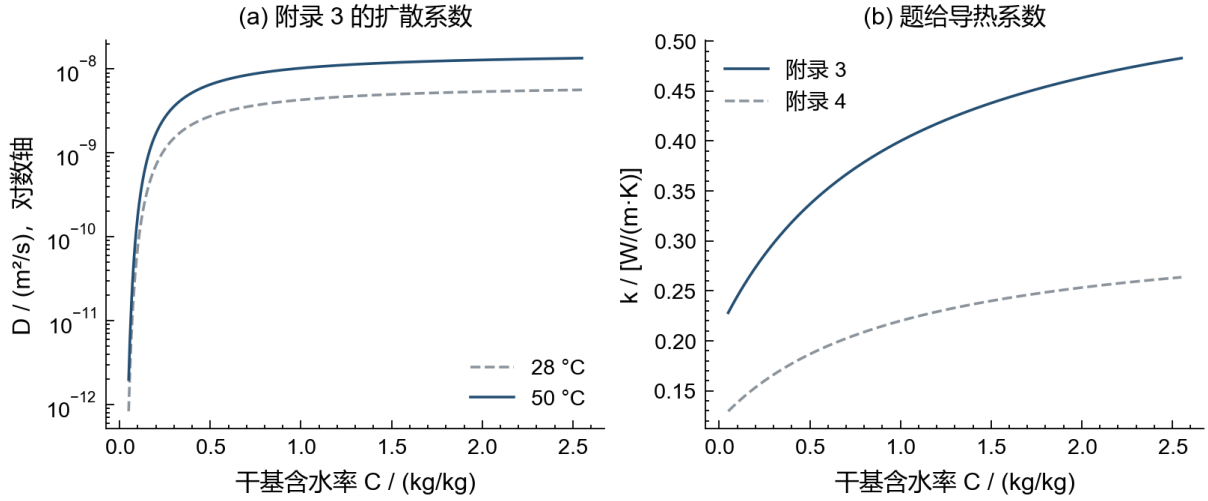


图 4 题给物性公式的状态曲线。左图采用附录 3 在 28°C 、 50°C 的两个温度切片, 扩散系数为对数纵轴; 右图比较附录 3、4 的导热系数, 均由公式直接计算。

6.4 初边值条件

各问从均匀初态出发:

$$T(r, z, 0) = 28, \quad C(r, z, 0) = 2.55. \quad (8)$$

轴线与中面采用对称条件,

$$\partial_r T|_{r=0} = \partial_r C|_{r=0} = 0, \quad \partial_z T|_{z=0} = \partial_z C|_{z=0} = 0. \quad (9)$$

对称条件允许轴心状态随时间变化。侧面 $r = R(t)$ 与端面 $z = L(t)/2$ 分别采用外法向交换条件,

$$-k\partial_n T = h[T_s - T_a(t)], \quad -D\partial_n C = h_m[C_s - w_a(t)], \quad h = 25, \quad h_m = 8 \times 10^{-7}. \quad (10)$$

h 、 h_m 的单位分别为 $\text{W}/(\text{m}^2\text{K})$ 、 m/s 。二维模型分别计算侧面和端面, 边界可直接使用原系数。 $C_s - w_a$ 保留既有经验交换关系的写法。虽然两量都以 kg/kg 表示, 其质量基准不同, **这个差值尚不能直接视为气相蒸发驱动力**。潜热实验所需的表面平衡关系在后文单独给出。

附件 1 含 0—4 h 内 241 组记录, 采样间隔 60 s。对于相邻观测时刻, 边界输入按

$$f(t) = \frac{t_{i+1} - t}{t_{i+1} - t_i} f_i + \frac{t - t_i}{t_{i+1} - t_i} f_{i+1}, \quad t_i \leq t \leq t_{i+1}, \quad f \in \{T_a, w_a\}. \quad (11)$$

超过 4 h 后, 环境输入取末小时均值 $T_a = 49.9989^\circ\text{C}$ 、 $w_a = 0.04998754 \text{ kg/kg}$, 用于预测后续时段。图 5 所示前期观测仍按实际变化进入边界。后期采用恒定延拓, 不改变预热阶段使用时变环境的处理。

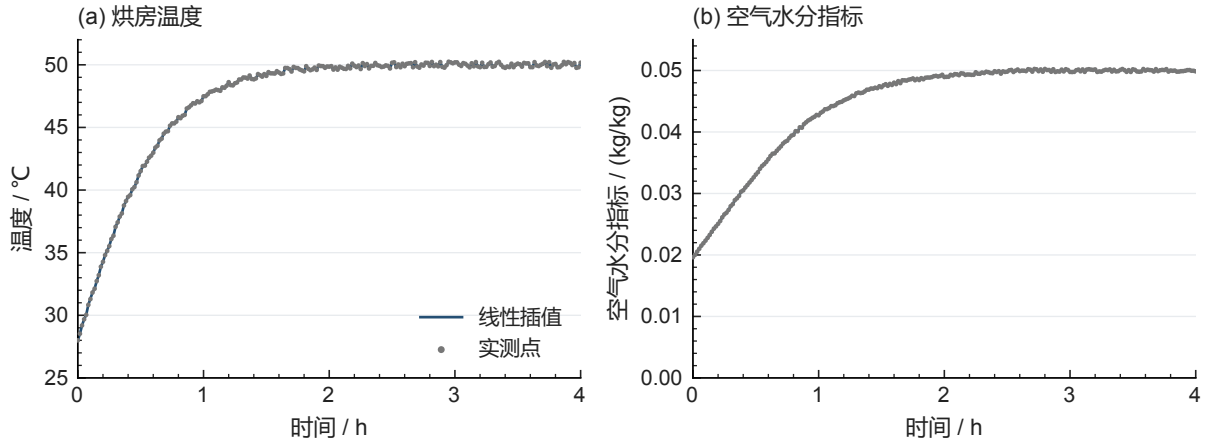


图5 附件1的环境记录及观测区间内的线性插值。两图分别为温度与空气水分指标；4 h以后的末小时均值延拓属于模型条件。

6.5 等比例收缩模型

附件2提供0—72 h的145个半径观测，间隔0.5 h。半径 $R_{\text{obs}}(t)$ 在相邻观测点之间线性插值，不向附件覆盖范围以外延拓。问题四采用

$$R(t) = R_{\text{obs}}(t), \quad L(t) = L_0 \frac{R(t)}{R_0}, \quad \xi = \frac{r}{R(t)}, \quad \eta = \frac{2z}{L(t)}. \quad (12)$$

记 $\alpha(t) = R(t)/R_0$ ，内部材料点按 $(r, z) = \alpha(t)(r_0, z_0)$ 运动。除插值结点外，其速度为 $\mathbf{v} = (\dot{R}r/R, \dot{L}z/L)$ 。固定 (ξ, η) 的时间导数即该运动下的材料导数，方程转为

$$\begin{aligned} \rho c_p \partial_t T|_{\xi, \eta} &= \frac{1}{R^2 \xi} \partial_\xi (\xi k \partial_\xi T) + \frac{4}{L^2} \partial_\eta (k \partial_\eta T), \\ \partial_t C|_{\xi, \eta} &= \frac{1}{R^2 \xi} \partial_\xi (\xi D \partial_\xi C) + \frac{4}{L^2} \partial_\eta (D \partial_\eta C). \end{aligned} \quad (13)$$

侧面的法向导数换为 $R^{-1} \partial_\xi$ ，端面换为 $2L^{-1} \partial_\eta$ ，控制体体积和面面积也随同一几何更新。由于时间导数已跟随材料运动，方程中不再重复加入网格平流。这一变换给出了选定运动学下的求解形式。

6.6 数值求解方法

采用节点中心有限体积离散。径向128个区间、轴向64个区间分别对应129、65个节点，节点包含对称边界和外表面。归一化节点按 $x_i = 1 - (1 - i/N)^p$ 布置，径向 $p = 2$ 、轴向 $p = 3$ ，靠近外露面加密。相邻节点的中点作为控制体界面，边界节点的控制体截止于真实边界。

对内部节点 (i, j) ，半根圆柱中的环形控制体体积和面面积为

$$\Delta V_{ij} = \pi(r_{i+1/2}^2 - r_{i-1/2}^2) \Delta z_j, \quad (14)$$

$$A_{i\pm 1/2, j}^r = 2\pi r_{i\pm 1/2} \Delta z_j, \quad A_{i, j\pm 1/2}^z = \pi(r_{i+1/2}^2 - r_{i-1/2}^2).$$

令 $\mathbf{F} = -DVC$ ，单元含水率方程由四个面的有向通量相加得到：

$$\Delta V_{ij} \dot{C}_{ij} = A_{i-1/2, j}^r F_{i-1/2, j}^r - A_{i+1/2, j}^r F_{i+1/2, j}^r + A_{i, j-1/2}^z F_{i, j-1/2}^z - A_{i, j+1/2}^z F_{i, j+1/2}^z. \quad (15)$$

图6显示控制体与两类共享面的对应关系。同一内部面上的通量以相反符号进入相邻单元，因此在全域汇总时抵消[4]。轴线处径向面面积为零，中面对称面通量为零，离散式由此处理对称边界，

无需在 $r = 0$ 处直接计算 $1/r$ 。温度方程采用相同的面收支结构：先求导热净输入功率，再除以当前控制体热容量 $\rho c_p \Delta V_{ij}$ ，得到温升速率。

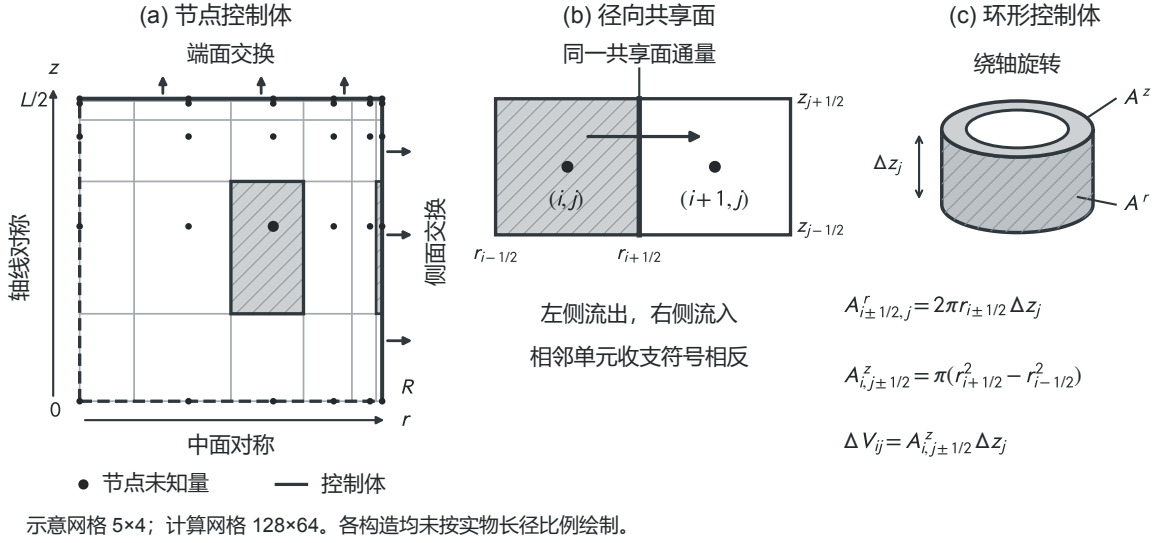


图6 二维轴对称有限体积表示：(a) 节点及边界截断控制体；(b) 径向共享面通量；(c) 旋转后的环形控制体与面面积。灰色排线标出控制体，同一面通量在相邻单元中以相反符号计入。

水分扩散系数可写成 $D(C, T) = b(T)e^{-a/C}$ ，其中 a 分问取 0.89、0.45、0.30。由于干燥后期该函数对含水率变化敏感，界面通量采用含水率积分势：

$$\Phi_a(C) = Ce^{-a/C} + a \text{Ei}(-a/C), \quad \Phi'_a(C) = e^{-a/C}, \quad F_f = -b(T_f) \frac{\Phi_a(C_+) - \Phi_a(C_-)}{\Delta s_f}. \quad (16)$$

Δs_f 表示相邻节点的实际距离， T_f 为两节点温度的算术平均。径向和轴向均按这一规则计算水分通量，导热系数则取调和平均。积分势的导数可直接核对含水率依赖项，面通量据此包含该项的积分；温度因子的离散误差仍通过网格检验判断。移动域中的距离、面积和体积均使用当前 R, L 。

温度场与含水率场采用 SciPy 的隐式 BDF 方法联立求解 [5]。每次迭代均根据当前湿温状态更新材料物性，并按当前时间更新环境条件和计算区域，使传热与水分迁移的相互影响参与求解。

主计算的相对容差设为 2×10^{-8} ，温度和含水率的绝对容差分别为 10^{-8} 和 10^{-10} ，最大时间步长为 120 s。求解器根据误差估计和迭代收敛情况自动调整步长与阶数；迭代未收敛时，更新雅可比矩阵或缩小步长后重新计算。计算采用稀疏雅可比矩阵以降低求解开销，全程保留双精度。题目要求的逐秒结果由已接受时间步内的连续插值获得，输出间隔与实际求解步长分别设置。

6.7 干燥终止判据

药材各位置的干燥进度不同，表面含水率降低时，内部仍可能保留较多水分。因此，本文以全域最大含水率低于 0.15 kg/kg 作为干燥完成的条件，定义

$$C_{\max}(t) = \max_{\mathbf{x} \in \Omega_{r_c}(t)} C(\mathbf{x}, t), \quad t_* = \inf\{t : C_{\max}(t) < 0.15\}. \quad (17)$$

其中， t_* 表示达到干燥要求的临界时间。最湿位置由整个二维含水率场确定，不预先限定在药材中心。采用分片双线性插值时，单元内部含水率不会超过其节点最大值，因此可通过全部节点确定离散场的最大含水率。

为区分临界状态与严格达标状态，先确定 $C_{\max} = 0.15$ 的时刻，再取其后首个满足严格不等式的整数秒作为结果表的终止时刻。**判定使用未经舍入的计算值**，表中显示的 0.1500 仅表示四位小数下的结果。

正文径向分布取药材中面 $z = 0$ ，用于展示由中心到侧面的变化；**干燥终点仍由全域判据确定**，轴向平均值仅用于第 6.1 节的一维近似对照。按题目要求，问题一、二每秒输出一次，问题三、四每 60 s 输出一次，径向间隔均为 0.1 cm，并补充严格达标时刻的结果。第四问随半径收缩，部分固定位置会移出材料范围；这些位置留空，另列当前表面含水率，避免将材料外部误记为零含水率。

7 四问求解与结果分析

本章采用前文建立的二维热湿传递模型及数值求解方法，结合各问给定条件，求解四个问题并分析结果。

7.1 问题一：预热阶段温湿分布

7.1.1 预热过程的计算与结果提取

问题一要确定预热半小时内，不同半径位置升温 and 失水的程度。取初始温度 28 °C、干基含水率 2.55 kg/kg，并保持初始圆柱尺寸；物性采用式 (5)，初边值条件采用式 (8)—(10)。附件 1 给出的热风条件随时间输入侧面和端面，内部温湿分布由这些表面交换逐步形成。

计算时，将半纵截面划分为前章所述的控制体。对每个控制体，利用与邻区之间的温度差计算导热通量，以当前含水率对应的扩散系数计算水分通量；处于侧面或端面的控制体还与热风交换，轴线和中面采用对称条件。由各面的净交换得到温度、含水率的变化，再用隐式 BDF 推进至下一时刻。本问的密度、比热和导热系数保持常数，**水分扩散系数仍随含水率更新**。

由此求得 0—1800 s 的二维温湿场。**题目所需的径向分布统一从中面提取**，即在同一时刻沿圆心到侧面读取数值。热风记录按分钟给出，求解器的内部步长由误差控制确定；输出时刻从已接受时间步的连续解中读取，二者不要求相同。

7.1.2 温度与含水率分布

表 4、表 5 分别列出题定时刻和半径处的温度、含水率。沿每一行可以比较同一时刻的内外差异，沿每一列可以追踪该位置的变化。先看预热结束时的结果：1800 s 时，中面中心温度为 33.5753 °C，侧面中部为 36.7855 °C，温差 3.2102 °C；相应含水率分别为 2.5500、1.5103 kg/kg。**此时中心已经升温，水分减少仍集中在外侧**，说明升温与失水尚未推进到相同的内部范围。

表 4 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0041	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3680	28.8488
600	28.4533	28.5361	28.8040	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2224	32.1125	33.4276
1500	31.9957	32.1868	32.7661	33.7463	35.1202
1800	33.5753	33.7721	34.3642	35.3621	36.7855

表 5 30 分钟内药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0509
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5046	1.7548
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

注：两表均取中面径向位置；温度单位为 $^{\circ}\text{C}$ ，水分浓度单位为 kg/kg 。

图 7 将表中的径向变化画成剖面曲线，并补充预热结束时的二维场。上排显示温升和含水率下降的影响逐步深入内部，下排用于检查径向表未展示的端面影响。1800 s 时，端面轴心温度为 37.2712°C ，高于中面轴心的 33.5753°C ；端面同样与热风接触，因此两个轴心位置具有不同响应。题定表选取中面，并不表示整根圆柱只沿径向变化。完整每秒、每 0.1 cm 的中面结果见 result1.xlsx。

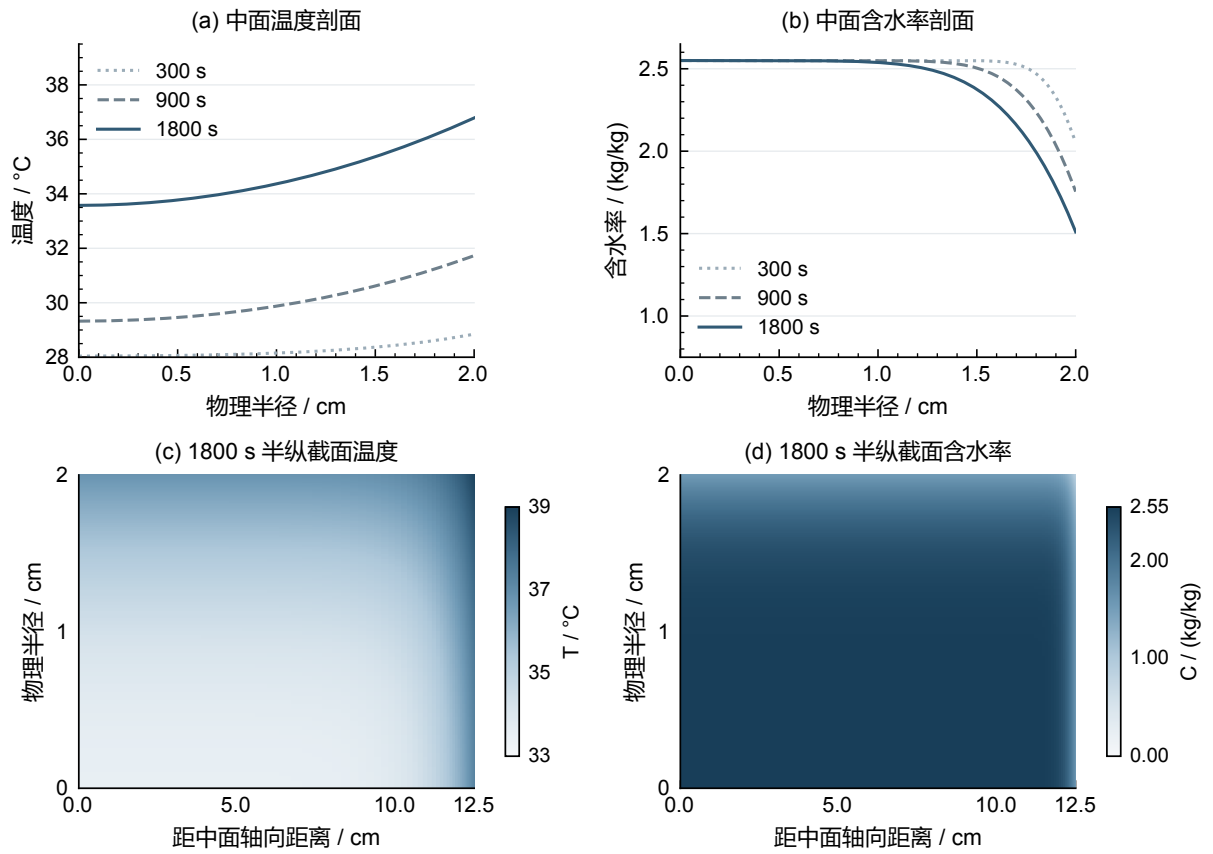


图 7 问题一二维主模型的预热响应。上排为 300、900、1800 s 的中面网格剖面，下排为 1800 s 二维场。空间两轴独立缩放，温度与含水率独立设色标。

7.1.3 热湿响应特征

为解释中心升温而含水率仍接近初值的现象，可比较两种传递过程的初始时间尺度。初始热扩散率为 $\alpha_0 = k/(\rho c_p) = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ ，初始水分扩散系数为 $D_0 = 4.9377 \times 10^{-9} \text{ m}^2/\text{s}$ ，前者约为后者的 34.2 倍。以初始半径估算，

$$\tau_T = \frac{R_0^2}{\alpha_0} = 0.6580 \text{ h}, \quad \tau_C = \frac{R_0^2}{D_0} = 22.5028 \text{ h}. \tag{18}$$

半小时已经接近径向导热的初始时间尺度，却远短于水分扩散的初始时间尺度，与图 7 的场分布一致。因此，问题一的主要结果是：**预热结束时内部已产生温升，显著失水区域仍靠近外表面**。失水后扩散系数还会降低，初始尺度只能解释早期响应，不能直接用作完成干燥的时间。下一问将进一步考虑热物性随含水率变化时的反馈。

7.2 问题二：变物性热湿演化

7.2.1 变物性反馈与联立求解

问题二将观察时间延长至 3 h，并采用附录 3 的变物性公式。按题设初态重新取 $T_0 = 28^\circ\text{C}$ 、 $C_0 = 2.55$ ，不接续问题一的末态；几何仍为初始圆柱，环境和表面交换沿用本章定义。这一问需要同时求解温度与含水率，因为**温度会改变水分扩散速度，失水又会改变导热系数、密度和比热**。

式 (6) 中的物性须由当前试算的温度和含水率计算。隐式时间步内，先用试算状态更新物性及各面通量，再由两条方程共同修正温湿场，直至满足迭代和误差要求。这样得到的下一时刻状态**在规定容差内同时满足两条离散方程**，温湿反馈也贯穿整个求解过程。空间离散和时间积分仍采用第 6 章的方法。

7.2.2 温度与含水率分布

计算得到各时刻的二维场后，按每 0.5 h 提取一次中面径向分布，分别列于表 6、表 7。比较两张表，可以判断温度和水分分布是否以相近的速度趋于均匀。3 h 时，中心温度为 49.8508°C ，侧面中部为 49.9672°C ，二者已十分接近；中心和侧面中部的含水率却仍分别为 1.7662、1.0081 kg/kg，均显著高于最终阈值。完整每秒、每 0.1 cm 结果见 result2.xlsx。

表 6 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1892	32.3822	32.9660	33.9608	35.4130
1.0	40.3816	40.5537	41.0606	41.8783	42.9977
1.5	45.8473	45.9354	46.1935	46.6056	47.1404
2.0	48.4517	48.4895	48.5998	48.7747	49.0042
2.5	49.4687	49.4809	49.5152	49.5666	49.6619
3.0	49.8508	49.8566	49.8757	49.9111	49.9672

表 7 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1167
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

注：两表均取中面径向位置；温度单位为 $^\circ\text{C}$ ，水分浓度单位为 kg/kg。

图 8 用于进一步辨认这两种变化发生的先后。上排将中面各半径的结果按时间展开，显示表面变化逐步传入内部的过程；下排直接比较侧面中部与中心，用同一组保存结果计算温差和含水率差。差值面板采用图中标注的半小时间隔输出，因此能够与表 6、表 7 逐项对应。

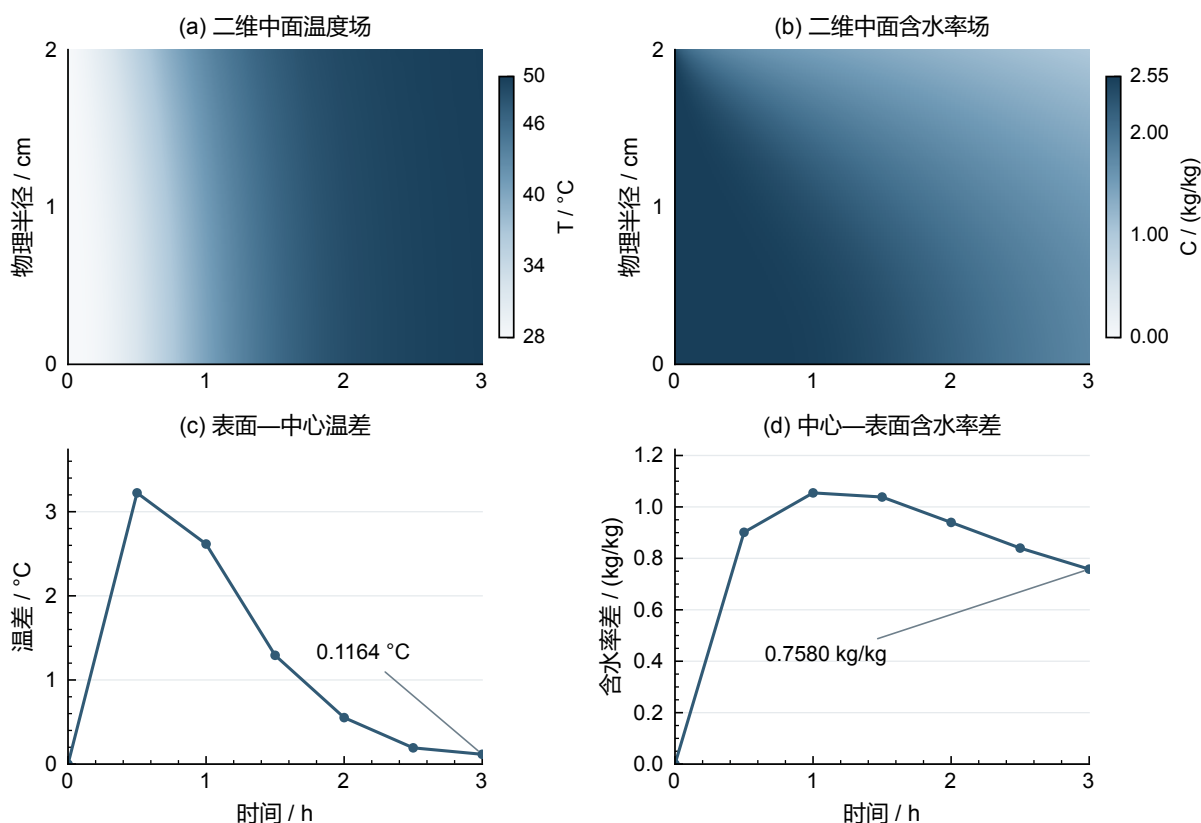


图8 问题二二维主模型的中面温湿演化。上排为中面时空分布，下排为同源表面—中心差值；温差取表面减中心，含水率差取中心减表面，单位与数值尺度各自独立。

7.2.3 热湿耦合特征

前期表面先随热风升温，表面与中心的温差增大；热量继续向内传递后，中心温度逐渐接近表面，温差随之减小。到3 h，中面两端点温差已降至 0.1164 °C，含水率差仍为 0.7580 kg/kg。因此，温度接近均匀时，药材内部仍保留明显的水分梯度。

这一区别还可由扩散系数的状态依赖解释： $\exp(-3850/T_K)$ 随升温增大， $\exp(-0.45/C)$ 则随含水率下降而减小。升温加快水分扩散，外层变干又会降低该处的扩散能力，两种作用同时存在。因此，即使温度已经升高，较干外层仍可能限制内部水分向外迁移。这里依据题给扩散函数和求得的含水率场解释结果，未单独追踪内部蒸发界面。问题二由此给出后续求解的依据：判断烘干完成仍须继续跟踪水分场，温度接近热风温度本身不足以确定达标。

7.3 问题三：固定尺寸干燥时长

7.3.1 从水分场确定终止时间

问题三所求的是整根药材何时达到含水率要求。问题二得到的3 h 温湿场尚未达标，因此沿用同一初值问题继续积分，物性、固定尺寸、经验边界和空间离散均保持相同。文中时间始终从题设初态起算。附件1 覆盖的前4 h 使用时变环境，超过4 h 后按末小时环境均值延拓。

每得到一个时刻的温湿场，就按式 (17) 检查全部二维节点，取其中最大的含水率作为整根药材的判定量。只要这个最大值仍高于阈值，便继续积分；当其接近 0.15 时，利用连续输出定位临界时刻，并检查其后的首个严格达标整数秒。由此，问题三将二维水分场转化为一条全域最大含水率随时间的曲线，再由这条曲线确定烘干时长。

7.3.2 含水率分布

表 8 按每 6 h 列出中面含水率，并补入严格终止行。图 9 同时画出代表位置与全域最大含水率，用来说明终止判据为何必须覆盖内部。12 h 时，中心含水率为 0.4561，侧面中部为 0.1642；18 h 时，侧面中部已降至 0.0845，中心仍为 0.2988。表面此时已经达标，内部仍需继续失水，若只检查表面便会提前结束干燥。

表 8 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0548
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
烘干结束时间 (57.4722)	0.1500	0.1477	0.1402	0.1249	0.0526

注：取中面径向位置；水分浓度单位为 kg/kg。

终止行按严格达标时刻提取，数值显示保留四位小数。

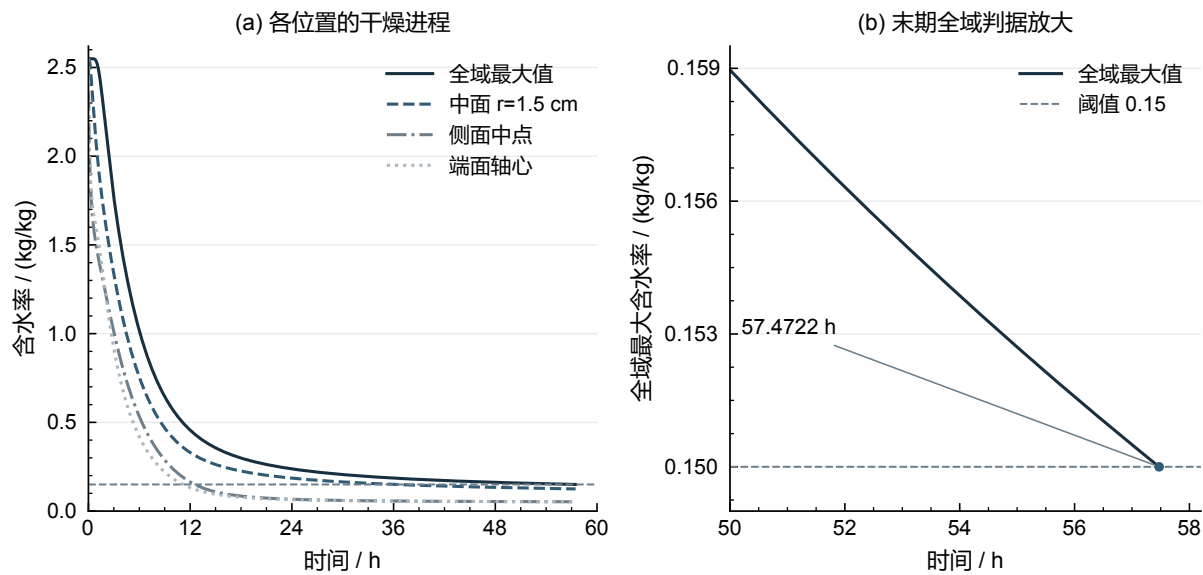


图 9 问题三维主模型的全域最大含水率、代表位置含水率及后期放大。虚线为 0.15 阈值，各曲线来自同一计算；终止时间由全域节点最大值确定。两面板纵轴范围不同。

进入后期后,含水率下降逐渐变慢。图 9 的放大图显示,54 h 时中心含水率为 0.1539,距离 0.15 已经很近,却仍需继续约 3.47 h 才满足全域要求。这与低含水率下扩散系数降低有关,也说明干燥时间不能按早期的下降速度线性外推。完整每 60 s 输出见 result3.xlsx。

7.3.3 全域达标时间

按上述过程求得的连续临界时刻为 206899.7714 s,即 57.4721587 h。在这一临界点,全域最大含水率等于阈值;题目要求严格小于阈值,故再检查其后的整数秒,得到

$$t_{\text{III}} = 206900 \text{ s}, \quad C_{\text{max}}(206900 \text{ s}) = 0.1499999334 < 0.15. \quad (19)$$

全场检查得到的最大值位于中面轴心 $(r, z) = (0, 0)$, 该位置由结果确定,程序未预先只检查中心。由此,本模型下问题三的烘干时长约为 57.47 h,题定严格终止时刻取 206900 s。终止行显示的 0.1500 来自四位小数舍入,内部判断仍使用双精度值。模型和网格之间比较连续临界时间,结果表记录严格达标整数秒;两者均通过事件定位获得,图中曲线用于展示趋近阈值的过程。

7.4 问题四: 收缩条件干燥时长

7.4.1 将半径记录引入移动域求解

问题四在求烘干时长的同时,还需处理失水过程中的半径变化。采用附录 4 物性和题设原始初态,从附件 2 读取各时刻半径,并在相邻记录间线性插值。计算限于附件半径覆盖的时段。**半径仅确定外部径向尺寸**;为确定轴向尺寸及内部材料点的位置,本节沿用式 (12) 的等比例收缩假设。

具体求解在归一化参考区域内进行,每次计算通量前,由当前半径更新径向坐标,并按同一比例更新轴向坐标。随后**重算相邻节点的实际距离、控制体体积和各面面积**,再计算内部扩散、导热及侧面和端面的交换。温度与含水率仍由隐式方法共同推进。由于方程的时间导数已经跟随材料运动,这一过程中不重复加入网格平流项。

每个时刻的场都对应当时的药材外形。求烘干时长时,继续检查全部节点的最大含水率;填写径向结果表时,则先判断题定物理位置是否仍在药材内,再读取对应含水率。这样,**收缩既参与方程求解,也参与结果位置的解释**。

7.4.2 含水率分布

表 9 列出题定物理半径处的中面含水率,同期尺寸在右侧并列给出。图 10 将水分场与移动外边界放在一起:外侧先变干,端面附近还存在轴向交换,中部湿芯在较长时间内保持较高含水率;半径缩小时,承载这些数值的材料区域也相应缩小。读图时需要同时看含水率和外边界的位置。

例如,表中 6 h 及之后的半径已小于 1.5 cm,原来位于 1.5 cm 和初始 2 cm 处的固定空间位置已经落到药材外部。**这些位置没有对应的内部含水率**,故用“—”表示,不能填成零。“药材表面”一列则始终取 $r = R(t)$,随收缩向内移动,用于记录当时真实边界上的数值。

表9 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	药材表面
6	1.7196	1.5377	1.0226	—	0.4207
12	0.7377	0.6546	0.4083	—	0.1669
18	0.4088	0.3686	0.2397	—	0.0892
24	0.2851	0.2611	0.1790	—	0.0673
30	0.2263	0.2094	0.1493	—	0.0595
36	0.1928	0.1797	0.1317	—	0.0560
42	0.1712	0.1604	0.1200	—	0.0541
48	0.1561	0.1468	0.1116	—	0.0530
烘干结束时间 (51.0922)	0.1500	0.1413	0.1081	—	0.0526

同期尺寸（等比例收缩假设）

时间/h	半径/cm	长度/cm
6	1.3740	17.1750
12	1.2480	15.6000
18	1.2140	15.1750
24	1.2040	15.0500
30	1.2010	15.0125
36	1.2000	15.0000
42	1.2000	15.0000
48	1.2000	15.0000
51.0922	1.2000	15.0000

注：含水率取中面径向位置，单位为 kg/kg；同期尺寸按等比例收缩假设计算。终止行按严格达标时刻提取，数值显示保留四位小数。

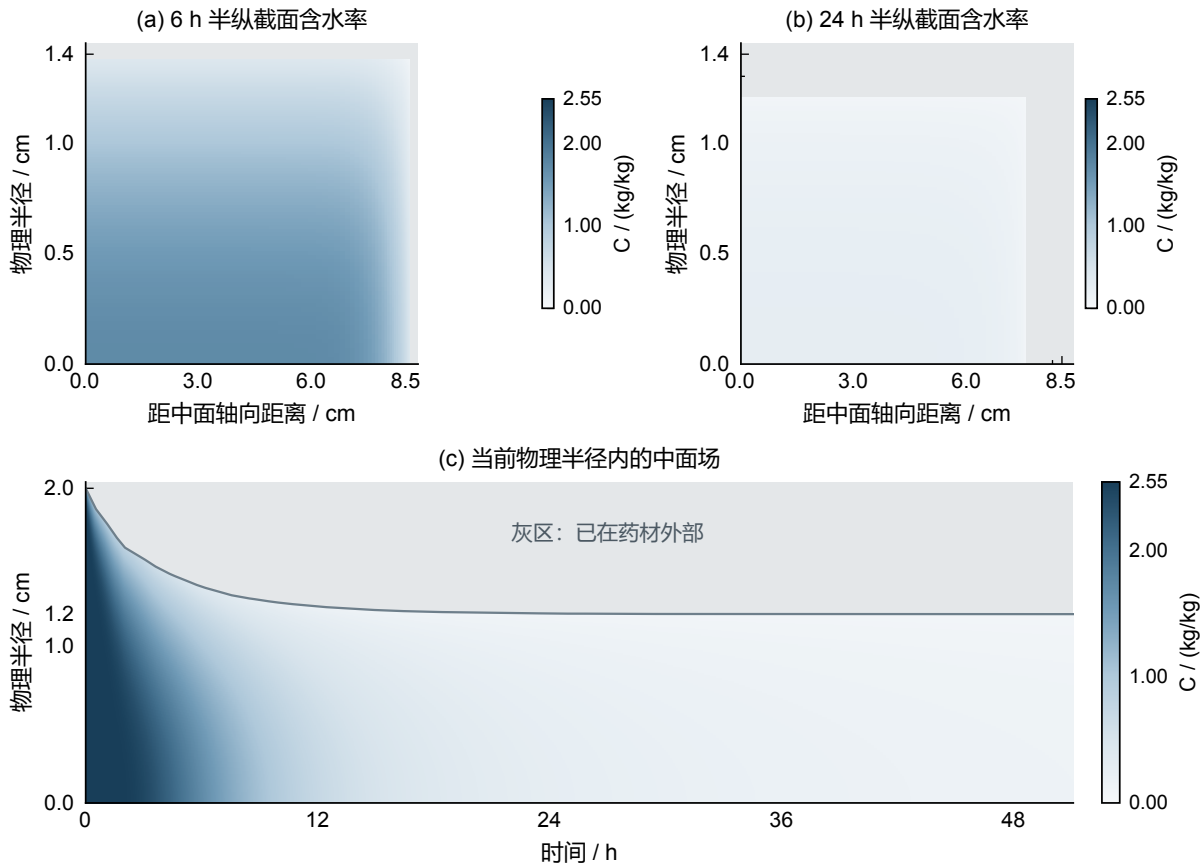


图10 问题四等比例收缩条件下的二维含水率与中面响应。上排为6 h、24 h 快照，采用共同物理坐标范围和色标；下排为当前物理半径内的中面时空场，灰区位于当时材料外部。空间两轴独立缩放。

完整每 60 s、每 0.1 cm 的结果见 result4.xlsx。该文件按题目模板保留末列“药材表面”，每个时刻填入移动表面值，其前各列表示固定物理位置。原始数组另保留初始 2 cm 位置，收缩后按域外规则记为缺失。上述区分使表中每个数都能对应一个明确的物理位置。

7.4.3 全域达标时间

对随收缩演化的全场采用与问题三相同的判据，求得连续临界时刻为 183931.2207 s，即 51.0920057 h。随后检查首个严格达标整数秒及其全场最大值，得到

$$t_{IV} = 183932 \text{ s}, \quad C_{\max}(183932 \text{ s}) = 0.1499995998 < 0.15. \quad (20)$$

全场最大值仍位于中面轴心。因此，在附件半径与等比例收缩条件下，问题四的烘干时长约为 51.09 h，题定严格终止时刻取 183932 s。达标半径为 1.2000 cm，按本节等比例假设计算的整根长度为 15.0000 cm。末行时间由整数秒换算为 51.0922 h，与连续临界时间显示的 51.0920 h 略有差异。

7.4.4 收缩效应分析

问题四的时长短于问题三，但两问还使用了不同的物性，直接相减无法单独判断收缩的作用。为作几何对照，另取附录 4 的相同物性、初态、二维边界和数值设置，只将几何改为初始固定圆柱，再积分至 72 h。该算例此时的全域最大含水率仍为 0.2247，尚未达标；等比例收缩算例已于约 51.09 h 达标。两条全过程曲线见图 11。

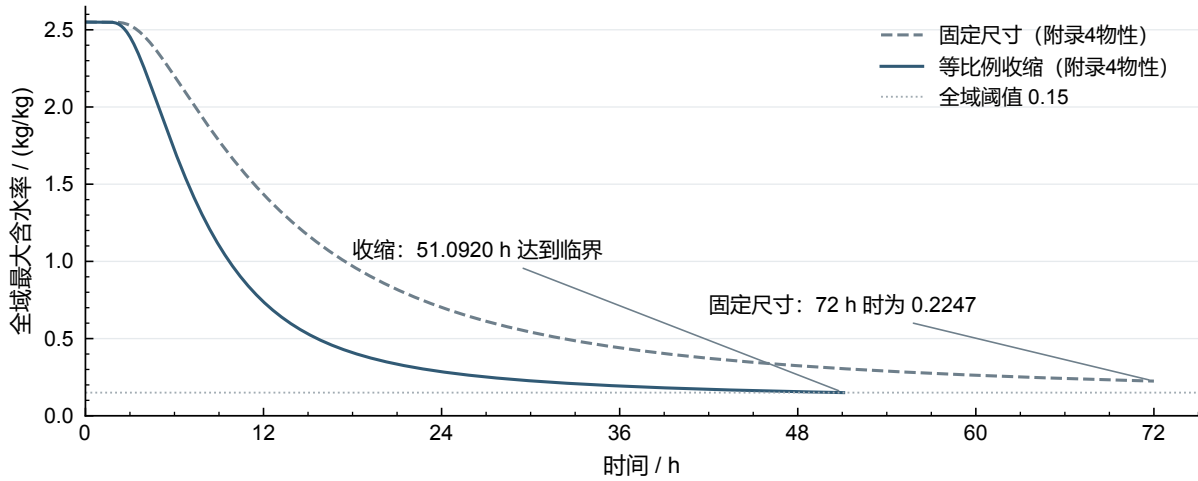


图 11 附录 4 同物性下的几何对照。两算例分别保持初始尺寸和采用等比例收缩，其他物性、初态、环境及经验交换关系相同；固定域在 72 h 截止时仍未达标。

在当前方程组中，收缩缩短了扩散路径，同时改变体积与交换面积的比例，这两项变化共同影响水分运输。由于固定域算例只积分到 72 h，其最终达标时间尚未确定，本对照能够支持的结论是：在附录 4 的同物性条件下，所选收缩几何使模型更早达到含水率要求。

8 数值检验

至此，四问所需的温湿分布与终止时间均已从同一求解框架得到。接下来先通过网格、时间精度和离散收支检验数值解，再利用密度积分检查几何与干质量是否相容。问题四的 15 cm 长度来自等比例假设，其材料合理性仍须接受后述检验。

8.1 空间收敛性

将二维网格从 64×32 区间加密到 128×64 区间，保持物性、边界、几何和时间精度设置相同。表 10 分别列出连续临界时差，以及 201 个共同归一化径向位置上的代表时刻中面场差。

表 10 二维空间加密检验 (时间变化取细网格减粗网格)

算例	代表时刻	最大温差 / °C	最大含水率差 /	
			kg/kg	临界时间变化 / s
问题一	1800 s	6.7630×10^{-4}	2.3592×10^{-4}	—
问题二、三	3 h	5.3924×10^{-5}	1.5469×10^{-4}	-2.51995
问题四	3 h	6.5390×10^{-5}	4.4565×10^{-4}	-3.34789

问题三、四的网格时差分别约 2.52 s、3.35 s, 明显小于第 6.1 节维度比较的 28.57 min、11.18 min。因此, 现有维度差异无法仅由本次空间离散变解释。两级网格比较反映加密后的变化量, 尚不足以给出连续解的严格误差界或可靠的渐近收敛阶。

8.2 时间积分精度

在 64×32 二维网格上, 将最大步长由 120 s 改为 60 s, 相对及绝对容差同时缩小十倍。问题三、四的临界时间变化分别为 -0.0001653 s、 $+0.0116350$ s; 问题一 1800 s 中面温度与含水率的最大变化分别为 5.3377×10^{-7} °C、 2.8391×10^{-10} kg/kg。

与本次空间加密相比, 时间精度设置收紧引起的输出变化更小, 表明当前设置下时间积分误差对这些输出的影响较小。由于容差和步长上限同时改变, 这一实验只评价两者共同作用, 不能拆分各自贡献。完整配置与字段对照见附录 C。

8.3 离散收支检验

主模型的水分方程直接推进 C , 故首先检查其实际离散的参考域标量收支。记半根体积为 $V_{1/2}(t)$, 在固定域或等比例移动域中, 权重 $w_{ij} = \Delta V_{ij}/V_{1/2}$ 均不随时间变化。汇总式 (15), 得到

$$\bar{C}_w(t) = \sum_{ij} w_{ij} C_{ij}(t), \quad \mathcal{R}_C(t) = \bar{C}_w(t) - \bar{C}_w(0) + \int_0^t \frac{\int_{\partial\Omega_{1/2}(\tau)} \mathbf{F} \cdot \mathbf{n} dA}{V_{1/2}(\tau)} d\tau. \quad (21)$$

累计收支按各时刻的当前体积计算: 先将该时刻的边界净流出除以体积, 再对时间积分。内部共享面的通量在汇总时抵消, 移动域各时刻的体积也分别进入计算, 不视为同一常数。 $\mathcal{R}_C$ 的单位为 kg/kg。

表 11 主模型实际离散方程的收支检查

检查量	问题二、三二维解	问题四二维解
平均含水率累计收支残差 / kg/kg	7.1579×10^{-9}	4.3195×10^{-8}
瞬时含水率方程残差 / (kg/kg)/s	5.4210×10^{-20}	5.4210×10^{-20}
瞬时温度方程汇总残差 / W/m ³	1.8190×10^{-12}	1.8190×10^{-12}

表 11 的小残差表明, 内部通量与边界输入在数值上满足所写方程的离散收支。温度残差检查的是瞬时方程, 未计入累计蒸发潜热和迁移水显焓。参考域的含水率收支也有其特定测度, 不能代替使用变化湿密度计算的真实水质量收支。

实际接受步的记录中未出现负含水率，全部快照的对称面均按零通量处理，外边界保留给定驱动力对应的交换方向。对称性由半域边界构造施加，本次未另用整根圆柱进行独立的对称性收敛实验。接下来的第 9 章按密度积分检验干物质质量，以检查离散残差尚未覆盖的材料相容性。

9 质量—几何一致性分析

9.1 干物质质量检验

若将题给 $\rho(C)$ 解释为当前真实湿密度，则干基含水率定义要求

$$\rho_d(C) = \frac{\rho(C)}{1+C}, \quad \rho_w^{\text{bulk}}(C) = \frac{\rho(C)C}{1+C}, \quad M_d^{\text{diag}}(t) = 4\pi \int_0^{L(t)/2} \int_0^{R(t)} \frac{\rho(C(r,z,t))}{1+C(r,z,t)} r dr dz. \quad (22)$$

ρ_w^{bulk} 是单位药材体积内的水质量。整根积分采用第 6.2 节的几何权重，先在每个位置计算 $\rho(C)/(1+C)$ ，再对空间求和。这一关系对 C 非线性，因此不能先取平均含水率再代入，否则会改变积分结果。

若烘干只带走水分、干物质不流失， $M_d^{\text{diag}}(t)$ 就应始终等于初值。上述定义已经使总湿质量等于干质量与水质量之和，这一相等关系属于代数结果，不能作为额外验证。**独立的检验应考察干质量是否随时间漂移。**

将二维主结果代入，问题三终止时的干质量比为 2.1584，问题四为 0.5357，均明显偏离 1。图 12 表明偏离发生在整个干燥过程中，远大于第 8 章的数值收支残差。

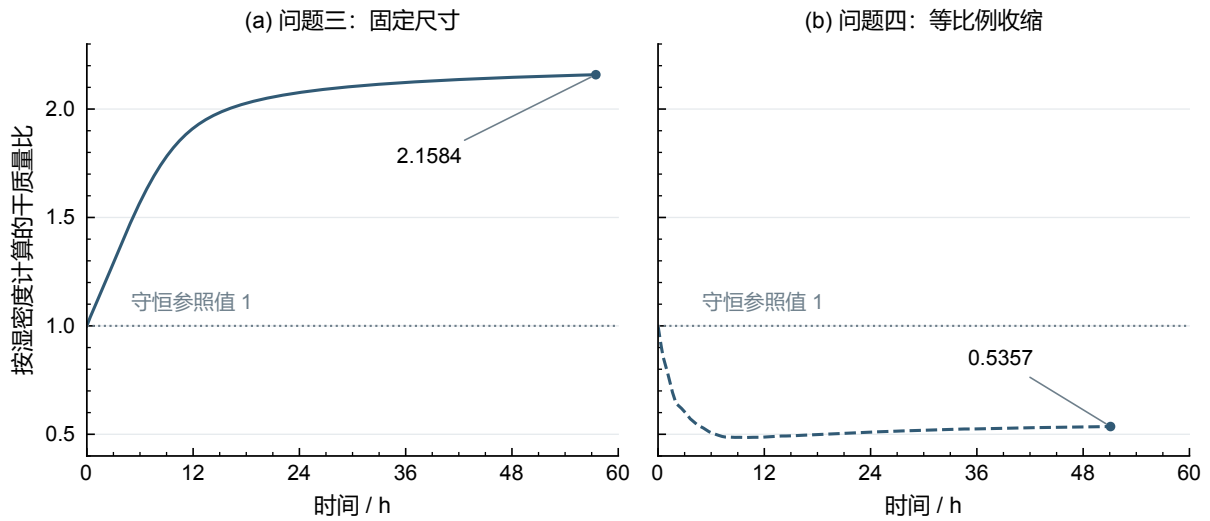


图 12 二维主结果在真实湿密度解释下的干质量诊断。两图分别采用问题三与第四问各自物性和几何，虚线 1 表示干物质不流失条件；跨问曲线不用于隔离收缩效应。

问题三保持体积固定。随着 C 减小，题给密度关系算出的单位体积干质量增大，整根干质量积分也随之上升。问题四还同时发生等比例体积收缩，积分干质量先下降、随后部分回升，但终点仍有明显净亏损。因此，**所得水分场、所选几何、真实湿密度解释和干质量守恒无法同时成立**。固定域中的问题三也已出现偏离，**这一矛盾不能全部归于第四问的轴向收缩，也不能归因于潜热省略。**

主方程直接描述 C 的扩散，尚未联立骨架连续方程。因此，即使第 8 章的参考域标量收支通过，式 (22) 也未必恒定。这项诊断指出上述条件组合不相容，**尚不足以单独判定题给密度、半径或某项观测有误**。以下保留第四问密度、半径和干质量守恒，进一步推导尺寸及输运方程需要满足的关系。

9.2 轴向尺寸约束

第四问单位干质量对应的当前体积为

$$v(C) = \frac{1+C}{760+90C}, \quad V(t) = \int v(C) dm_d, \quad L(t) = \frac{V(t)}{\pi R_{\text{obs}}(t)^2}. \quad (23)$$

这里的积分覆盖整根初始干物质。初始干质量由本问的尺寸、含水率及密度确定：

$$M_{d0} = \pi R_0^2 L_0 \frac{760+90C_0}{1+C_0} \approx 0.0875663642 \text{ kg}. \quad (24)$$

式 (23) 把总长度与当前水分场联系起来。截面积的相对减小幅度大于总体积的相对减小幅度时，长度就会增加；半径接近稳定后，继续失水又可能使长度回落。因此，**径向缩小和轴向增加可以伴随总体积减少**，长度随时间也不必单调变化。

这一关系还给出无需数值扩散解的必要界限。由

$$v'(C) = \frac{670}{(760+90C)^2} > 0 \quad (25)$$

可知，非负含水率满足 $v(C) \geq 1/760$ ；若全域严格达标，还满足 $v(C) < 1.15/773.5$ 。按干质量积分并除以截面积，即有

$$L_- := \frac{M_{d0}}{760\pi R_*^2} \leq L_* < \frac{1.15M_{d0}}{773.5\pi R_*^2} =: L_+. \quad (26)$$

取 $R_* = 1.20 \text{ cm}$ 时，两个端点近似为 25.46897 cm 和 28.77812 cm 。下界只要求 $C \geq 0$ ，尚未达标时同样适用；下端取等号要求水分在干质量意义下几乎处处为零。若将达标要求放宽为 $C \leq 0.15$ ，上端便可取等号，对应 $C = 0.15$ 几乎处处。文中显示的 25.469 cm 和 28.778 cm 经过舍入，判断严格界限仍应使用式 (26) 的精确表达。

图 13 将必要界限与不同长度取值作比较。保留上述湿密度解释、初始干质量、圆柱外形和 1.20 cm 半径，即使水已全部排尽，长度也至少比初始 25 cm 多约 0.47 cm ；等比例缩至 15 cm 及保持初长均低于下界。后文映射 A 的 28.0262 cm 位于区间内。**界限不依赖内部映射或扩散系数，但只提供必要检查**，材料的真实变形和水分场仍需独立观测。

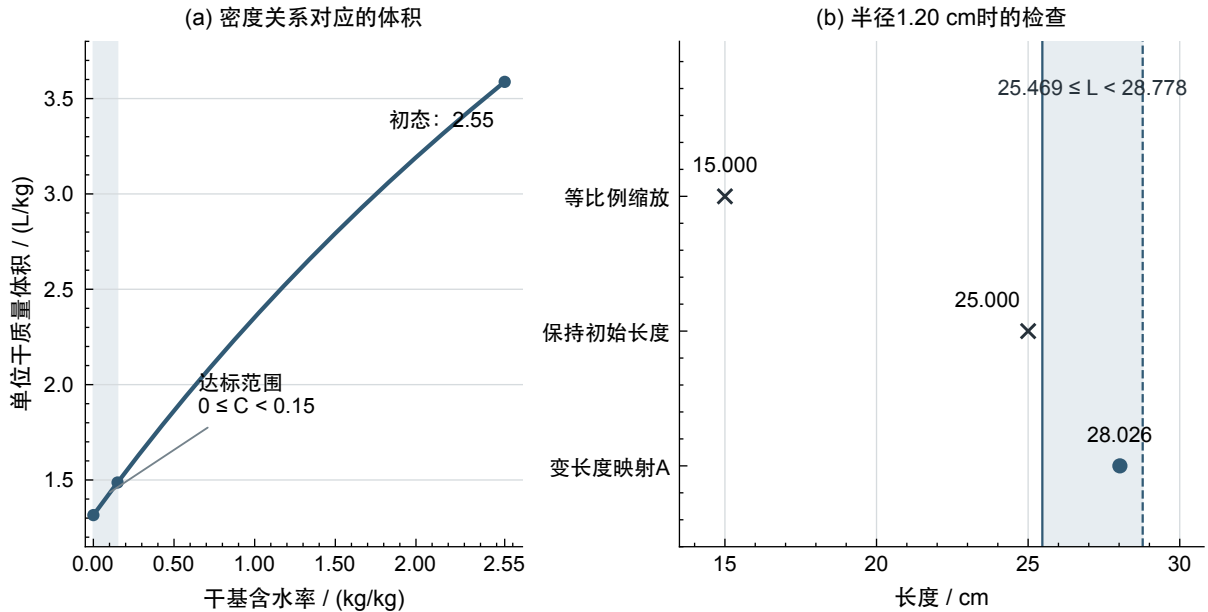


图 13 长度的解析必要条件与已有计算值。左图直接计算单位干质量体积；右图的阴影为式 (26) 的必要区间，端点显示经过舍入。叉号表示两种几何规则的长度，圆点为后文 32×16 映射 A 的条件结果；该区间不表示统计置信范围。

9.3 守恒运输与显焓方程

为使水量计量与干物质守恒配套，引入材料速度 $\mathbf{v}$ 及相对骨架的水质量通量 $\mathbf{j}$ ：

$$\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}) = 0, \quad \rho_d D_t C = -\nabla \cdot \mathbf{j}, \quad \mathbf{j} = -\rho_d D(C, T) \nabla C. \quad (27)$$

该条件模型沿用附录 4 的 D ，同时**选择将其用于真实水质量通量**。与主模型的标量扩散相比，展开后多出干密度梯度项：

$$\begin{aligned} D_t C &= \nabla \cdot (D \nabla C) + D \nabla \ln \rho_d \cdot \nabla C \\ &= \nabla \cdot (D \nabla C) - \frac{670D}{(1+C)(760+90C)} |\nabla C|^2. \end{aligned} \quad (28)$$

沿用相同数值的 D ，**输运算符仍因干密度梯度项而改变**。该项与材料运动速度、边界通量一起作用于水分场，仅凭它在某点的符号，无法单独判断干燥时间会如何变化。外表面暂时保留题给传质系数和经验驱动力，写为

$$J_s = \mathbf{j} \cdot \mathbf{n} = \rho_{d,s} h_m (C_s - w_a). \quad (29)$$

于是同一有向 J_s 同时用于水分方程和外部净失水核算。其表面平衡适用性仍需另行讨论。

热容量继续采用附录 4。将比热换算到单位干质量后， $(1+C)c_p = 1850 + 4000C$ 。以 $T_{\text{ref}} = 28^\circ\text{C}$ 为显焓零点，定义

$$H(C, T) = (1850 + 4000C)(T - T_{\text{ref}}), \quad h_b(T) = 4000(T - T_{\text{ref}}). \quad (30)$$

系数 4000 来自题给比热的代数展开。显焓零点取值及暂不计混合、脱附参考焓是该能量表示的附加条件。配套能量方程为

$$\rho_d D_t H = -\nabla \cdot (\mathbf{q} + h_b \mathbf{j}), \quad \mathbf{q} = -k \nabla T. \quad (31)$$

外边界的总显焓流出为 $h(T_s - T_a) + h_b(T_s)J_s$ 。利用水分方程消去 $D_t C$ ，可写成

$$\rho(C)c_p(C)D_t T = \nabla \cdot (k \nabla T) - 4000\mathbf{j} \cdot \nabla T. \quad (32)$$

上式保留题给 ρc_p ，新增项表示内部迁移水携带的显焓。**水的显焓输运即使在没有相变时也会影响能量收支**。本节据此配套了水量和显焓方程，蒸发相变所需能量在第 10 章进一步处理，完整推导与边界条件列于附录 B。

9.4 变长度计算结果

式 (23) 确定整根长度，**内部各点的位置还需指定运动学映射**。参考映射 A 保持内部圆柱壳为圆柱面，同时允许内部横截面弯曲，**使密度要求的局部体积比与附件外半径同时成立**。求解在参考材料域上进行，采用双线性有限元、干质量集中形式和三阶高斯积分，并保留非正交度量。输出物理位置的含水率时，先由当前映射反算参考坐标，再在参考单元内插值。

32×16 网格下，参考映射的连续临界时长为 47.9436 h，严格整数秒为 172598 s，达标半径 1.2000 cm、长度 28.0262 cm。该长度位于式 (26) 的必要区间内。6 h 保存时刻长度为 31.9256 cm，随后保存值回落，72 h 为 27.6977 cm；这些值均为所选条件下的计算输出。

图 14 用截面积比、长度比和体积比解释这一几何过程：

$$\frac{V(t)}{V_0} = \left[\frac{R(t)}{R_0} \right]^2 \frac{L(t)}{L_0}. \quad (33)$$

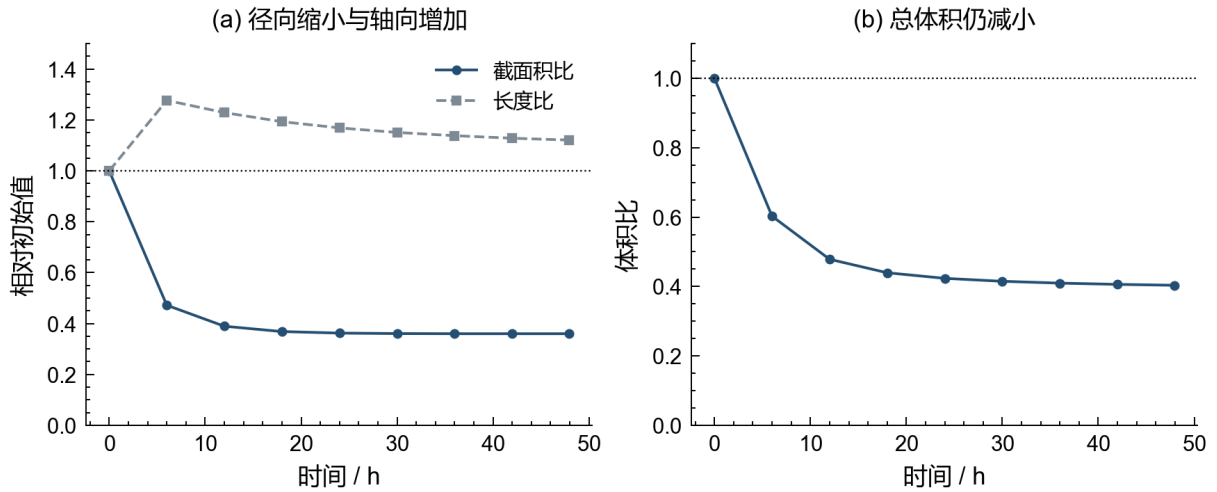


图 14 变长度条件解的几何分解。初态和所列输出时刻对应附件半径与参考映射长度；面积比与长度比的乘积给出体积比，长度和体积均为条件计算值。

所列非初始时刻的长度比均大于 1，体积比仍小于 1，表明轴向增加可与总体积减少同时发生。二维等比例模型约 51.09 h、变长度模型约 47.94 h，这一对照同时包含质量输运、几何、内部运动学和水显焓的变化，整体时差因而不能单独归于轴向伸长。完整含水率、长度历史和数值设置见附录 B、C。

9.5 局部运动学敏感性

整根长度由密度、半径和干质量守恒共同约束，内部材料点的位置仍需运动学关系确定。本节通过替换这一关系，检验前述烘干时长和长度演化是否依赖某一种局部变形安排。

映射 B 保持内部横截面为平面，允许圆柱壳面弯曲。给定相同参考含水率场时，A、B 具有相同外半径、总长度和体积；继续求解各自的输运方程后，含水率场可能不同，由此得到的长度历史也可能不同。这两种映射都满足所规定的几何约束，这些构造能否描述实际材料运动，尚无材料力学或内部变形观测加以识别。

配对计算保留相同的附录 4 物性、初态、附件环境及半径、经验边界、干质量和显焓关系，仅替换内部映射以及对应的度量、面积权重。32×16 网格下，A、B 的时长分别为 47.9436 h、47.9348 h，B 短 0.5298 min。从 24×12 加密至 32×16 时，两者各自的时长变化约 2.33 min，配对时差仅变化约 0.14 s。在本组条件下，两种构造给出的时间接近；但各自的空间离散变化更大，配对误差抵消不能作为任一模型的绝对误差界。

在 0—72 h 共同保存时刻，两种长度的最大绝对差约 0.0099 mm，出现在 15.150 h；101 个共同物理半径上的中面含水率采样最大差为 0.00179 kg/kg，出现在 4.617 h、 $r/R = 0.40$ 。完整曲线、采样方法和时空位置保留在附录 C。

这一对照的范围由两种映射的构造决定。当体积比场可分离为 $\mathcal{J}(a, \zeta) = p(a)q(\zeta)$ 时，两种积分映射完全相同，仅沿径向变化也属于这一特例。因而，现有结果尚不能代表任意局部变形，也不能直接外推到含潜热分支。附录 B 给出映射公式及其等价关系。

综合时长、长度历史与中面水分剖面的对照，在本组物性、经验边界及守恒条件下，变长度计算结果对 A、B 两种局部映射的选择并不敏感。细网格下的时长差仅占映射 A 时长的 0.0184%，两者均得到约 47.94 h；相较等比例模型约 51.09 h 的整体时长差，在两种映射下均得以保留。这说明前述差异未随本次映射替换而消失，但其成因仍包括守恒输运、几何和显焓等多项修正。与此同时，末

期长度超过初长的必要性已由式 (26) 给出，它来自所保留的密度、初始干质量、半径与含水率约束。由此，**保留映射 A 作为变长度条件结果的参考构造具有配对计算依据**，映射 B 则用于评价采用另一种局部变形安排时的输出变化。接下来进一步考察蒸发耗能与表面平衡条件对烘干过程的影响。

10 蒸发潜热讨论

10.1 相变能量与表面平衡

水进入气相需要能量，**真实蒸发通量应同时进入水量和能量边界**。主模型的 $C_s - w_a$ 经验关系未直接表示水蒸气平衡；若只按其排水量扣除汽化热，降温后的表面状态可能与交换方向不相容。相变边界因此需要同时关联材料表面含水率、水活度、表面温度和环境水蒸气状态 [6,7]。

在第四问变长度框架内，条件实验取

$$\begin{aligned} J_s &= \rho_{d,s} h_m [C_s - C_{eq}(T_s, w_a; \theta)], \\ \mathbf{q} \cdot \mathbf{n} &= h(T_s - T_a) + [h_v(T_{up}) - h_b(T_s)] J_s. \end{aligned} \quad (34)$$

C_{eq} 是指定表面温度和空气状态下的平衡含水率， θ 为尚未标定的参数。表面含水率高于平衡值时净排水，低于平衡值时允许反向交换。计算能量通量时，蒸发取表面温度作为上游温度，凝结取环境温度。**质量与能量核算采用同一个有向通量**，因而相变方向和相应热量收支保持配套。

这一条件实验还采用标准大气压，并按干空气基准解释空气水分指标。附录 D 逐项给出候选水活度、温度修正、蒸气状态关系及异常处理。所补充的表面关系仅用于该实验，其适用范围与主模型的经验边界分别说明。

10.2 潜热影响与适用范围

以末期环境为参考，取 0.03、0.075、0.12 kg/kg 三组平衡含水率。每组保留同一表面关系、附件半径、几何闭合与运动学规则及数值设置，**仅开关相变能量通道**。三组时长分别增加 8.2935、9.9388、12.9118 h，完整终值配对列于附录 D。

图 15 选取参考平衡点 0.075 kg/kg，展示延时对应的温湿过程。3 h 中心温度由关闭时的 49.8110 °C 降至开启时的 41.0205 °C，全域最大含水率则由 2.4594 升至 2.5436 kg/kg。开启组此时侧面中部为 40.9812 °C，略低于中心；**表面蒸发耗能改变了温度分布**，并通过温度相关扩散系数影响水分演化。

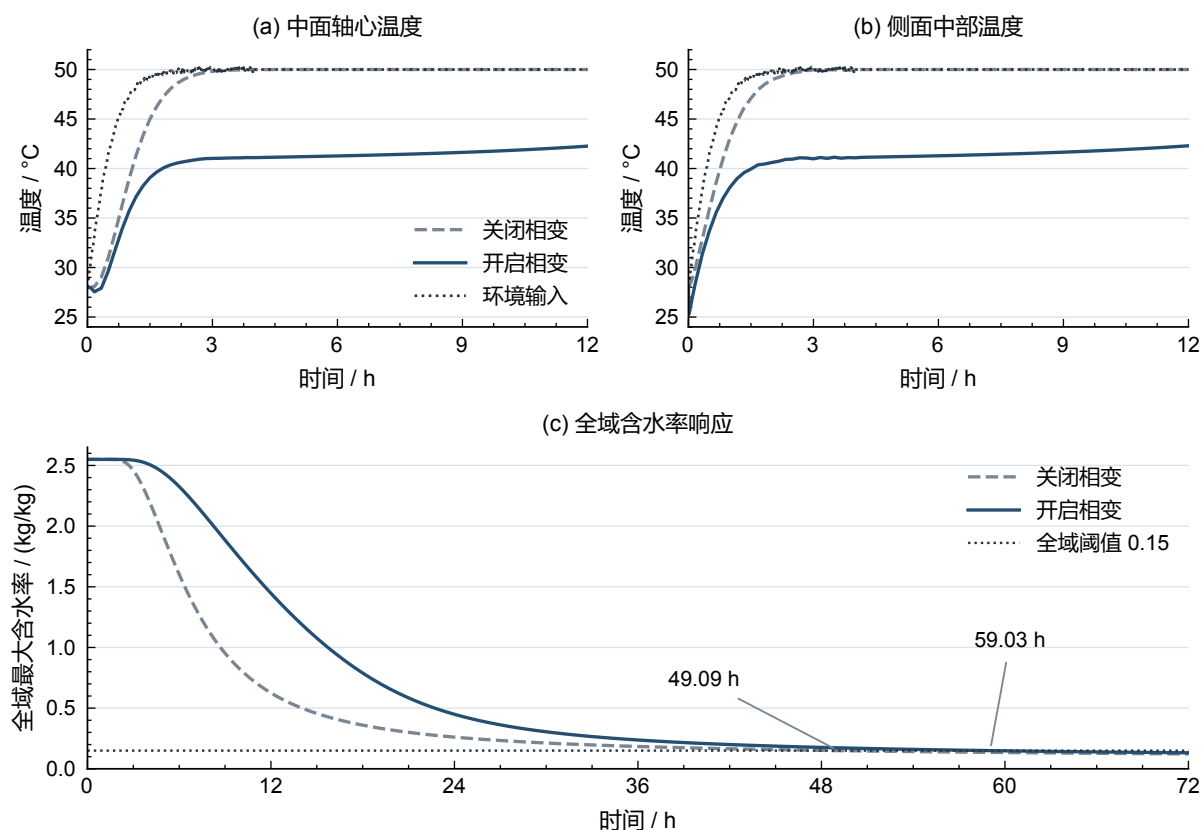


图 15 第四问同一参考平衡点 0.075 kg/kg 、 32×16 网格下的潜热开关对照。温度图区分中心与侧面中部，水分图比较全域最大值及阈值。材料曲线来自保存状态；环境 4 h 后按末小时均值延拓。两组保留同一表面平衡与运动学关系，各自长度随水分重算。

关闭相变的组内基准仍用新设平衡边界，不能直接与第 9 章原经验边界下的 47.9436 h 作单因素比较。三组参考点尚未标定；附件半径已经是共同输入，重复拟合它无法区分这些表面参数，还需独立失重、表面温度或平衡含水率观测。因此，四问题定答案沿用前述二维经验模型，潜热分支检验其能量处理的限制。现有证据既未确定唯一的含潜热实物时长，也未支持忽略潜热。

11 模型评价与推广

11.1 模型优点

二维模型在同一方程与数值框架中使用各问对应的数据，分别计算侧面和端面交换。题定表取中面结果，维度对照另取轴向平均，终止判据则检查全域，各项输出的用途明确。水分面通量采用积分势处理低含水率下的强非线性，并与共享面离散、隐式时间积分配合；空间加密、时间收紧和方程收支均保留了对照记录。

质量—几何诊断补充了数值残差以外的检验。先按含水率和密度定义计算干质量积分，再利用单位干质量体积的单调性推导尺寸界限，就能判断相关条件是否可以同时满足。这一判断依据质量和几何关系，不以计算时长较短作为选择模型的依据。

11.2 模型局限

主模型以经验关系描述表面水分交换， 4 h 后将环境延拓为末小时均值，第四问以等比例运动确定轴向和内部几何。方程未显式纳入潜热、水显焓及完整骨架连续方程；密度积分已显示，这组条

件与真实干质量守恒存在偏离。因此，四问结果应理解为所列近似下的计算值，**其数值精度不能直接视为对材料实测值的预测准确度。**

变长度模型进一步配套了守恒输运、密度体积关系和显焓，但局部运动学仍缺少力学或材料观测的约束。两种映射的对照只覆盖有限的构造，且各自网格变化大于配对时差。潜热实验还引入未标定的平衡关系，多组输出反映参数敏感性，尚不能据此给出实物时长的置信区间。

11.3 模型推广

该框架可用于其他柱状多孔材料的温湿传递计算，但需重新确定相应材料的物性、表面交换和变形规律。若外形或环境不再轴对称，空间表示及相应边界也须扩展，当前端面对称条件便不能直接沿用。

后续测量可针对尚未约束的量开展：失重曲线用于检验总水分迁移，轴向长度和内部标记位置用于检验运动学，表面温度及吸附或解吸平衡数据用于约束相变边界。这些观测须与已经作为输入的半径记录区分使用，才能独立检验模型预测，并减少条件参数的自由度。

12 结论

本文建立了二维轴对称热湿模型，按各问物性与几何计算温湿场。问题一 1800 s 中面中心与表面温度为 33.5753 °C、36.7855 °C，中心含水率仍接近初值；问题二 3 h 的相应温差为 0.1164 °C、含水率差为 0.7580 kg/kg，显示**温度均化先于水分均化**。问题三连续临界时间为 57.4722 h，严格达标秒为 206900 s；问题四在附件半径和等比例收缩下为 51.0920 h、183932 s，达标半径 1.2000 cm、整根长度 15.0000 cm。同附录 4 的固定尺寸对照在 72 h 内未达标，表明**所选几何变化对该扩散过程具有实质影响**。

将题给密度解释为当前真实湿密度后，主结果的干质量积分明显漂移。保留第四问密度、初始干质量、圆柱外形和 1.20 cm 半径时，严格达标要求长度位于约 25.469 cm 至 28.778 cm 之间，上端不包含在内。参考变长度模型给出约 47.94 h、28.03 cm 的条件结果，通过该必要检查；两种内部映射的时差约 0.53 min，长度历史与中面水分剖面也接近，**支持本组变长度结果对这两种映射选择不敏感**，并为保留映射 A 作为参考构造提供依据。该结论的适用范围限于当前共同条件，材料变形仍需观测验证。

潜热需要与表面平衡关系一起处理。三组未标定边界下，加入潜热使时长变化 8.29—12.91 h，表明当前近似在能量处理上存在限制。正文给出四问计算，数值检验检查求解实现，质量几何推论与潜热实验则检验相应条件的适用范围。进一步确定真实烘干时长，**仍需要独立的水量、尺寸和表面状态观测**。

人工智能工具使用声明

本论文使用了人工智能工具。用途摘要：用于辅助模型代码实现、python 图表代码编写、辅助查询文献资料详细使用情况见支撑材料《AI 工具使用详情.pdf》。

参考文献

- [1] Yang H, Sakai N. Shrinkage and Mechanical Characteristics of Potato Undergoing Air Convection Drying[J]. Japan Journal of Food Engineering, 2001, 2(2): 67–72. DOI: 10.11301/jsfe2000.2.67. 来源。
- [2] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.

[3] Srikiatden J, Roberts J S. Measuring moisture diffusivity of potato and carrot (core and cortex) during convective hot air and isothermal drying[J]. Journal of Food Engineering, 2006, 74(1): 143–152. DOI: 10.1016/j.jfoodeng.2005.02.026. 来源。

[4] Eymard R, Gallouët T, Herbin R. Finite volume methods[M]//Ciarlet P G, Lions J L. Handbook of Numerical Analysis. Vol.7. Elsevier, 2000: 713–1018. DOI: 10.1016/S1570-8659(00)07005-8. 来源。

[5] Shampine L F, Reichelt M W. The MATLAB ODE Suite[J]. SIAM Journal on Scientific Computing, 1997, 18(1): 1–22. DOI: 10.1137/S1064827594276424. 来源。

[6] COMSOL AB. Vapor Transport: COMSOL Multiphysics 6.4 Documentation[EB/OL]. [2026-09-12]. 来源。

[7] COMSOL AB. Theory for Moisture Transport in Porous Media: COMSOL Multiphysics 6.4 Documentation[EB/OL]. [2026-09-12]. 来源。

[8] Linstrom P J, Mallard W G, eds. NIST Chemistry WebBook, NIST Standard Reference Database Number 69: Water[DB/OL]. Gaithersburg, MD: National Institute of Standards and Technology. [2026-09-12]. DOI: 10.18434/T4D303. 来源。

附录 A 一维近似与维度对照

A.1 一维模型

一维对照将侧面和端面合并为一个代表性外表面，利用式 (1) 保留总交换面积，采用 $h_{\text{eff}} = 1.08h$ 、 $h_{m,\text{eff}} = 1.08h_m$ 。这个折算成立于本组固定长径比几何；若轴向独立变化，系数应随 R/L 更新。一维模型不表示端面与中部的独立状态，其误差由二维同条件计算评估。

在当前半径内采用径向方程

$$\rho c_p D_t T = \frac{1}{r} \partial_r (rk \partial_r T), \quad D_t C = \frac{1}{r} \partial_r (rD \partial_r C), \quad 0 < r < R(t). \quad (35)$$

初值、轴心及外侧边界为

$$\begin{aligned} T(r, 0) &= 28, & C(r, 0) &= 2.55, \\ \partial_r T(0, t) &= 0, & \partial_r C(0, t) &= 0, \\ -k \partial_r T(R, t) &= h_{\text{eff}}(T_s - T_a), & -D \partial_r C(R, t) &= h_{m,\text{eff}}(C_s - w_a). \end{aligned} \quad (36)$$

问题一、二和三、四分别使用式 (5)、(6)、(7) 的完整物性，环境插值及延拓与正文相同。问题二从题设初态重新计算，问题三延长问题二的初值问题；附件半径只进入第四问。

前三问固定 $R = R_0$ 。第四问采用 $R = R_{\text{obs}}$ 、 $L = L_0 R / R_0$ ，内部材料点按 $r = R(t)\xi$ 移动。在归一化坐标内有

$$\rho c_p \partial_t T|_{\xi} = \frac{1}{R^2 \xi} \partial_{\xi} (\xi k \partial_{\xi} T), \quad \partial_t C|_{\xi} = \frac{1}{R^2 \xi} \partial_{\xi} (\xi D \partial_{\xi} C). \quad (37)$$

式中的时间导数已跟随所选材料运动，外表面导数相应换为 $R^{-1} \partial_{\xi}$ ，面积折算因子保持 1.08。这里得到的是有效标量方程在移动坐标中的写法，真实干质量是否守恒仍需由质量关系另行检验。

空间采用 128 个径向区间、表面加密指数 2 的节点中心有限体积法。归一化界面为节点中点， $v_i = (\xi_{i+1/2}^2 - \xi_{i-1/2}^2)/2$ 。由相邻壳面的净通量构造

$$R^2 v_i \dot{C}_i = R(\xi_{i-1/2} F_{i-1/2} - \xi_{i+1/2} F_{i+1/2}), \quad F = -D \partial_r C. \quad (38)$$

导热方程采用对应热容量和导热通量；中心的面面积为零。水分界面采用式 (16) 的积分势，导热系数取调和平均。BDF 相对容差为 2×10^{-8} ， T, C 绝对容差为 $10^{-8}, 10^{-10}$ ，最大步长 120 s。每次右端项求值均更新物性和外部条件。以全部径向节点的最大含水率检查阈值，并单独验证首个严格整数秒。

A.2 一维计算结果

A.2.1 问题一

表 12、表 13 保留面积折算近似的题定结果。1800 s 时中心和表面温度分别为 33.7759、37.0529 °C，温差 3.2770 °C；含水率分别为 2.5500、1.4544 kg/kg。上述数字只对应本附录一维模型。

表 12 问题一——维近似的温度分布 / °C

时间 / s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0004	28.0043	28.0349	28.1919
300	28.0434	28.0675	28.1606	28.3896	28.8966
600	28.4784	28.5653	28.8463	29.3822	30.2692
900	29.3893	29.5290	29.9635	30.7333	31.8883
1200	30.6548	30.8278	31.3581	32.2777	33.6337
1500	32.1548	32.3515	32.9475	33.9542	35.3612
1800	33.7759	33.9771	34.5824	35.6014	37.0529

表 13 问题一——维近似的含水率 / kg/kg

时间 / s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2251
300	2.5500	2.5500	2.5500	2.5491	2.0170
600	2.5500	2.5500	2.5500	2.5343	1.8343
900	2.5500	2.5500	2.5497	2.5017	1.7068
1200	2.5500	2.5500	2.5480	2.4595	1.6071
1500	2.5500	2.5499	2.5442	2.4133	1.5248
1800	2.5500	2.5497	2.5376	2.3660	1.4544

图 16 连接 300、900、1800 s 时五个径向位置的输出，显示热量已传入中心，失水却仍集中于外层。题给初态下，热扩散率 $1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ 约为水分扩散系数 $4.9377 \times 10^{-9} \text{ m}^2/\text{s}$ 的 34.2 倍，径向时间尺度分别约 0.658 h 和 22.50 h，与图中的响应次序一致。上述尺度按初态常系数估算，只用于解释初期差别；后续含水率仍由变系数方程计算。

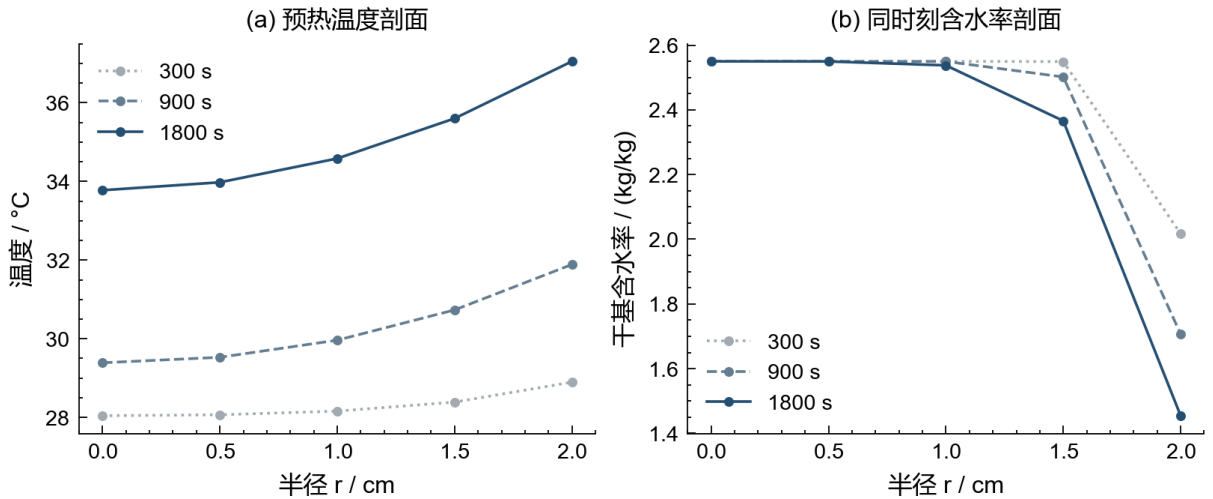


图 16 问题一面积折算一维模型的径向剖面。圆点为题定五个输出位置，线段连接计算值。

A.2.2 问题二

3 h 时，一维模型的中心和表面温度为 49.8812、49.9918 °C，相应含水率为 1.7315、0.9500 kg/kg。表 14、表 15 给出半小时时间间隔值，图 17 将空间分布与端点差值置于同一时间窗口。

表 14 问题二一维近似的温度分布 / °C

时间 / h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.3818	32.5815	33.1854	34.2132	35.7126
1.0000	40.7398	40.9124	41.4203	42.2379	43.3565
1.5000	46.1401	46.2252	46.4741	46.8708	47.3837
2.0000	48.6194	48.6546	48.7572	48.9196	49.1324
2.5000	49.5450	49.5557	49.5854	49.6293	49.7173
3.0000	49.8812	49.8865	49.9044	49.9378	49.9918

表 15 问题二一维近似的含水率 / kg/kg

时间 / h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5488	2.5236	2.3116	1.6018
1.0000	2.5233	2.4901	2.3450	1.9948	1.4215
1.5000	2.3735	2.3103	2.1111	1.7664	1.2950
2.0000	2.1482	2.0836	1.8927	1.5852	1.1757
2.5000	1.9269	1.8692	1.6997	1.4275	1.0593
3.0000	1.7315	1.6805	1.5302	1.2865	0.9500

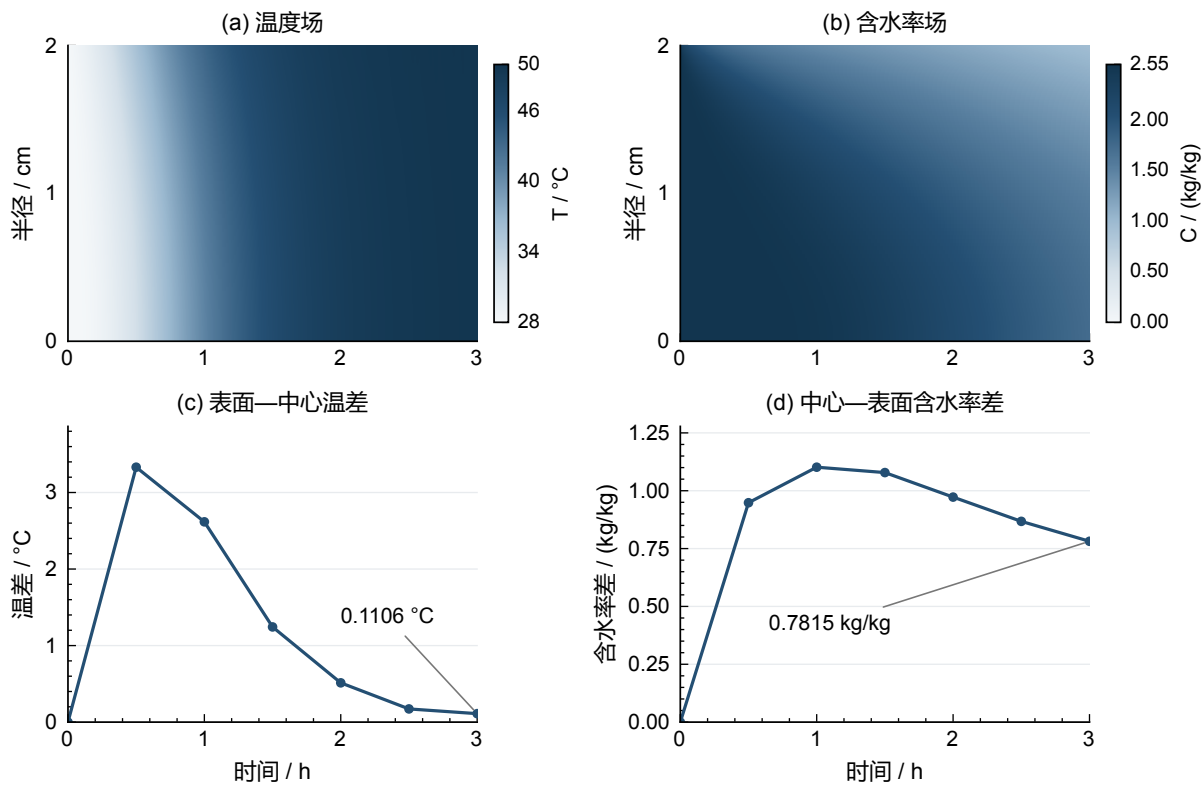


图 17 问题二一维模型的温湿时空分布和端点差值。上排使用逐秒保存场及原非均匀径向网格；下排取相同场的半小时采样点。温差为表面减中心，含水率差为中心减表面，各面板单位独立。

3 h 时温差仅 0.1106 °C，含水率差仍有 0.7815 kg/kg。外层变干使 $\exp(-0.45/C)$ 下降，低扩散区继续影响湿芯的排水。这一过程来自有效扩散方程，计算中没有单独设置内部蒸发界面。

A.2.3 问题三

表 16 给出每 6 h 分布和严格终止行。12 h 时表面已降至 0.1446，中心仍为 0.4382 kg/kg，说明表面达标明显早于全域达标。图 18 将这种次序与后期缓慢变化分别呈现。

表 16 问题三一维近似的含水率及严格终止行 / kg/kg

时间 / h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6.0000	0.9765	0.9476	0.8605	0.7126	0.4840
12.0000	0.4382	0.4256	0.3868	0.3155	0.1446
18.0000	0.2922	0.2851	0.2630	0.2209	0.0797
24.0000	0.2346	0.2296	0.2139	0.1834	0.0645
30.0000	0.2039	0.2000	0.1875	0.1628	0.0588
36.0000	0.1846	0.1813	0.1707	0.1495	0.0560
42.0000	0.1711	0.1682	0.1589	0.1400	0.0544
48.0000	0.1611	0.1585	0.1501	0.1329	0.0534
54.0000	0.1533	0.1509	0.1431	0.1273	0.0527
56.9961	0.1500	0.1477	0.1402	0.1249	0.0524

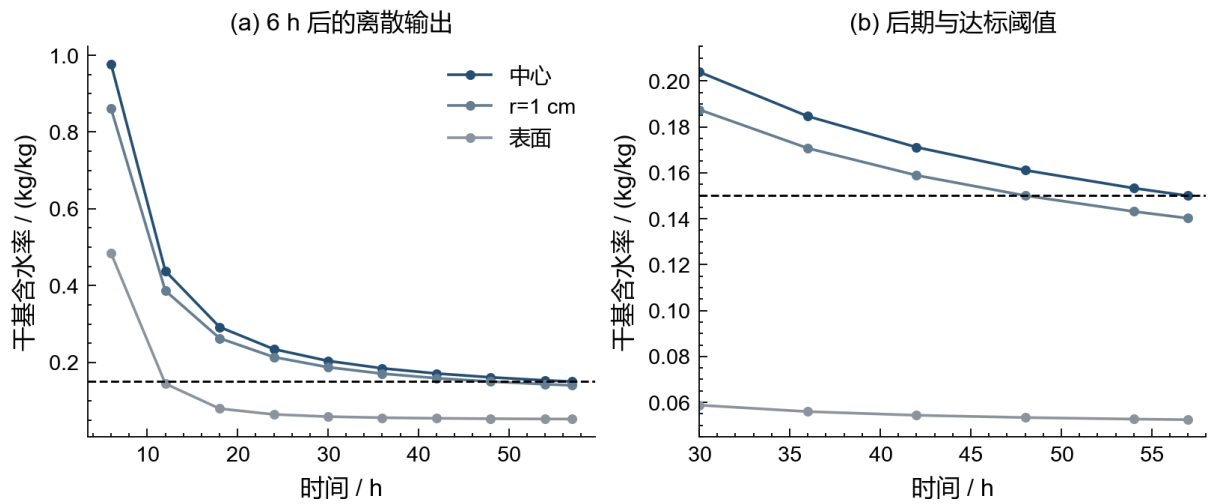


图 18 问题三一维模型的固定位置含水率与后期放大。离散点为已保存输出，虚线为 0.15 阈值；最终停止时间由全场事件定位，图中连线只解释趋势。

一维连续临界时间为 56.9959529 h，首个严格整数秒为 205186 s，此时 $C_{\max} = 0.1499998341 < 0.15$ 。终止行的 56.9961 h 来自整数秒换算，0.1500 来自四位小数显示。该时长与二维主结果的差异用于维度对照，不替换正文答案。

A.2.4 问题四

表 17 给出附件半径下的一维含水率。半径改变时，实际扩散距离和表面导数同时更新。表中“—”表示固定位置超出当前材料域，末列单独给出当前表面。

表 17 问题四等比例收缩的一维含水率 / kg/kg

时间 / h	0 cm	0.5 cm	1.0 cm	1.5 cm	当前表面
6.0000	1.7029	1.5195	1.0009	—	0.3902
12.0000	0.7252	0.6430	0.3994	—	0.1547
18.0000	0.4036	0.3640	0.2369	—	0.0852
24.0000	0.2828	0.2590	0.1778	—	0.0657
30.0000	0.2250	0.2082	0.1486	—	0.0586
36.0000	0.1920	0.1789	0.1312	—	0.0554
42.0000	0.1707	0.1599	0.1197	—	0.0538
48.0000	0.1558	0.1464	0.1113	—	0.0528
50.9058	0.1500	0.1413	0.1081	—	0.0524

一维连续临界时间为 50.9057341 h，严格达标整数秒为 183261 s。达标半径为 1.2000 cm，等比例假设给出长度 15.0000 cm。问题三、四采用不同物性，几何净作用仍由正文第 7.4.4 节的同物性对照判断。

图 19 说明为何表中 1.5 cm 位置在所列时刻均为空缺：这些位置位于当前半径之外。灰色区域只表示初始域中当前已经不再被材料占据的部分。

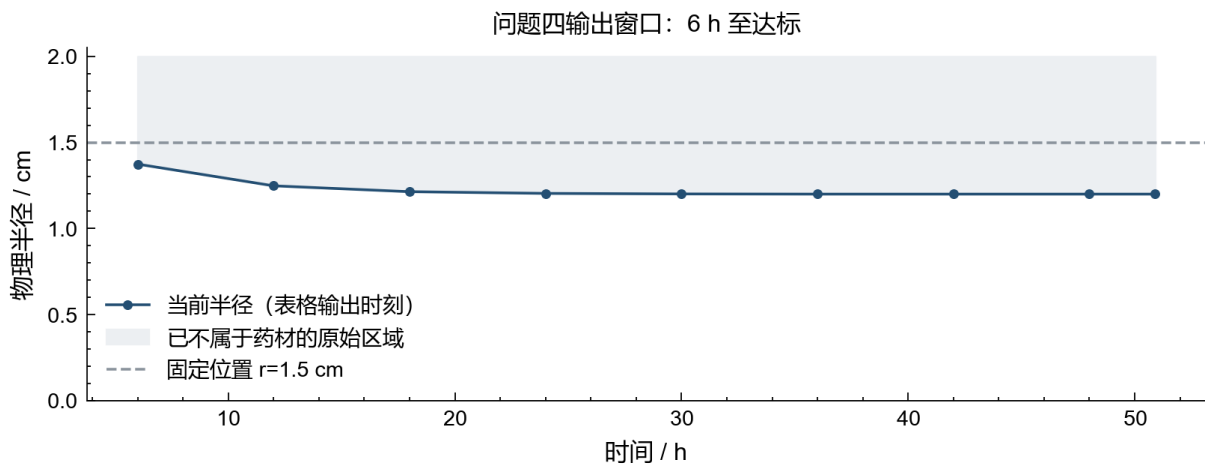


图 19 第四问一维输出时刻的当前半径与域外位置。图中只使用已有 6 h 之后的表格时刻，不据这些点重建早期收缩；灰色区域无对应材料含水率。

A.3 维度对照结果

二维参照分别采用中面 $f(r, 0, t)$ 和式 (2) 的轴向平均，以区分中面状态与整段轴向的平均状态。比较时，先将一维和二维结果取到相同保存时刻、相同物理半径，再计算绝对差。表 3 的最大差是在这些共同样本中得到的，不作为连续时空全部极值的界限。

图 20 给出第四问 6 h 的二维场及一维误差。表格统计最大绝对差，图中则保留“一维减二维”的符号，用于区分偏高和偏低。二维中面与端部仍有明显差异；分别以中面、轴向平均为参照，一维含水率的最大绝对差约为 0.0305、0.0715 kg/kg。面积折算虽能近似总体时间，但只能以一个代表状态概括轴向剖面。

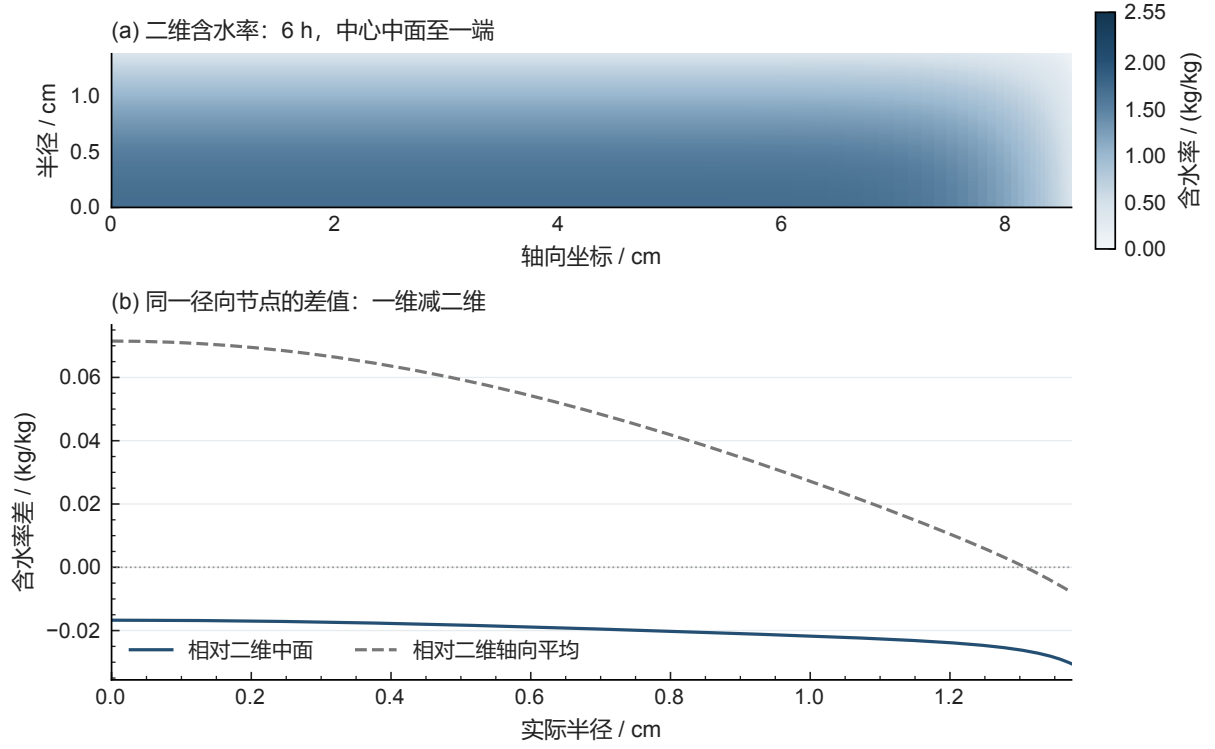


图 20 第四问 6 h 的二维含水率场及一维近似误差。二维场使用当时物理尺寸，径向差值均为一维减二维，分别以中面与轴向平均为参照，曲线只比较同一时刻。

问题三、四的二维连续临界时间分别比一维长 28.57、11.18 min，相当于一维时长的约 0.84%、0.37%。径向区间从 64 加密至 128 时，一维临界时间的变化仅为 -3.61196、-3.78695 s，问题一 1800 s 剖面的变化不超过 0.000695 °C 和 0.000249 kg/kg。网格加密变化远小于维度时差，二者应分别解释；但仅凭两级网格仍无法给出连续解误差的严格上界。完整比较配置见附录 C。

附录 B 变长度模型推导

B.1 守恒方程与显焓关系

本附录只研究第四问的一组附加条件：题给密度解释为真实湿密度，干物质不流失，圆柱外半径严格采用附件 2，局部体积由水分和密度关系确定；内部运动另由下述映射选择。所有热物性、初态与环境仍使用第四问定义，表面经验驱动力先保持不变。干基含水率是水与干物质的质量比，因而有

$$\rho_d = \frac{760 + 90C}{1 + C}, \quad \rho_w^{\text{bulk}} = \rho_d C. \quad (39)$$

跟随骨架运动的干物质连续方程及水连续方程分别为

$$\begin{aligned} \partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}) &= 0, \\ \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v} + \mathbf{j}) &= 0. \end{aligned} \quad (40)$$

将第二式展开，并利用第一式消去干物质变化，得到 $\rho_d D_t C = -\nabla \cdot \mathbf{j}$ 。取通量本构 $\mathbf{j} = -\rho_d D(C, T) \nabla C$ ，有

$$\begin{aligned} D_t C &= \rho_d^{-1} \nabla \cdot (\rho_d D \nabla C) \\ &= \nabla \cdot (D \nabla C) + D \nabla \ln \rho_d \cdot \nabla C, \\ \frac{d \ln \rho_d}{dC} &= \frac{90}{760 + 90C} - \frac{1}{1 + C} = -\frac{670}{(760 + 90C)(1 + C)}. \end{aligned} \quad (41)$$

正文式 (28) 的密度梯度项由上述保守质量通量推得。将题给 D 用于这一通量本构仍是一项模型选择：守恒方程规定水量如何收支，材料是否服从该通量关系还需另行验证。外边界取 $J_s = \rho_{d,s} h_m (C_s - w_a)$ ，与内部使用同一水质量计量，对称面取零通量。

由题给比热作代数换算，单位干质量对应的热容量为

$$(1 + C) c_p(C) = (1 + C) \left(1850 + \frac{2150C}{1 + C} \right) = 1850 + 4000C. \quad (42)$$

选择 28 °C 为显焓参考温度，并令不同含水率在该温度的额外混合、脱附参考焓暂为零，便得到 $H = (1850 + 4000C)(T - 28)$ 。迁移水对应的有效显焓取 $h_b = 4000(T - 28)$ 。它们满足 $\partial H / \partial T = (1 + C) c_p$ 、 $\partial H / \partial C = h_b$ ，构成同一套储存与输运定义。

显焓连续方程写为

$$\partial_t (\rho_d H) + \nabla \cdot (\rho_d H \mathbf{v}) = -\nabla \cdot (\mathbf{q} + h_b \mathbf{j}), \quad \mathbf{q} = -k \nabla T. \quad (43)$$

外表面以 $J_s > 0$ 表示净向外排水，其能量通量为

$$(\mathbf{q} + h_b \mathbf{j}) \cdot \mathbf{n} = h(T_s - T_a) + h_b(T_s) J_s. \quad (44)$$

对式 (43) 使用干质量连续方程，再展开 $D_t H = (1850 + 4000C) D_t T + h_b D_t C$ 。右侧水显焓散度中 $-h_b \nabla \cdot \mathbf{j}$ 与左侧 $\rho_d h_b D_t C$ 抵消，剩下

$$\rho c_p D_t T = \nabla \cdot (k \nabla T) - 4000 \mathbf{j} \cdot \nabla T. \quad (45)$$

这一项对应迁移水经过温度梯度时的显焓输运，由质量与能量方程联立推得，即使不设置气化热也仍然存在。当前方程配套的是所选显焓收支，水进入气相所需的边界焓差则在附录 D 中进一步处理。

B.2 局部运动学映射

B.2.1 共同体积条件

参考半圆柱坐标取 $0 \leq a \leq R_0$ 、 $0 \leq \zeta \leq \ell_0 = L_0/2$ 。初始干密度为 $\rho_{d0} = \rho(C_0)/(1 + C_0)$ ，当前体积与参考体积之比为

$$\mathcal{J}(a, \zeta, t) = \frac{\rho_{d0}}{\rho_d(C)} = \frac{\rho_{d0}(1 + C)}{760 + 90C}. \quad (46)$$

给定同一材料含水率场，两种映射都必须满足上述局部体积比、附件半径、圆柱外形，以及正文式 (23) 给出的总长度。这些条件尚未唯一确定内部点的位置：映射 A 选择保持圆柱壳，映射 B 选择保持横截面。两者没有引入应力平衡或弹性参数，均作为待检验的运动学闭合。

B.2.2 保持圆柱壳的映射 A

先对轴向积分，再对径向积分：

$$\begin{aligned} K(a, \zeta) &= \int_0^\zeta \mathcal{J}(a, s) ds, \quad \overline{\mathcal{J}}(a) = \frac{K(a, \ell_0)}{\ell_0}, \\ I(a) &= \int_0^a b \overline{\mathcal{J}}(b) db, \quad \lambda_z = \frac{2I(R_0)}{R_{\text{obs}}^2}. \end{aligned} \quad (47)$$

令

$$r(a) = \sqrt{\frac{2I(a)}{\lambda_z}}, \quad z(a, \zeta) = \lambda_z \frac{K(a, \zeta)}{\overline{\mathcal{J}}(a)}, \quad L = \lambda_z L_0. \quad (48)$$

因为 r 不随 ζ 变化，固定 a 的内部圆柱壳仍为圆柱面； z 可随 a 变化，内部横截面可以弯曲。外侧满足 $r(R_0) = R_{\text{obs}}$ ，中面 $z(a, 0) = 0$ ，端面 $z(a, \ell_0) = L/2$ 。轴对称体积雅可比为

$$\frac{r}{a} r_a z_\zeta = \frac{\overline{\mathcal{J}}}{\lambda_z} \frac{\lambda_z \mathcal{J}}{\overline{\mathcal{J}}} = \mathcal{J}. \quad (49)$$

参考平面内的变形梯度及梯度度量

$$F_A = \begin{pmatrix} r_a & 0 \\ z_a & z_\zeta \end{pmatrix}, \quad G_A = F_A^{-1} F_A^{-T} = \begin{pmatrix} \frac{1}{r_a^2} & -\frac{z_a}{r_a^2 z_\zeta} \\ -\frac{z_a}{r_a^2 z_\zeta} & \frac{1}{z_\zeta^2} + \frac{z_a^2}{r_a^2 z_\zeta^2} \end{pmatrix}. \quad (50)$$

B.2.3 保持横截面的映射 B

改为先对径向积分：

$$S(a, \zeta) = \int_0^a b \mathcal{J}(b, \zeta) db, \quad B(\zeta) = \frac{2S(R_0, \zeta)}{R_0^2}, \quad \alpha = \frac{R_{\text{obs}}}{R_0}. \quad (51)$$

构造

$$z(\zeta) = \alpha^{-2} \int_0^\zeta B(s) ds, \quad r(a, \zeta) = R_{\text{obs}} \sqrt{\frac{S(a, \zeta)}{S(R_0, \zeta)}}. \quad (52)$$

固定 ζ 的横截面保持平面，固定 a 的壳面可以弯曲。外侧 $r(R_0, \zeta) = R_{\text{obs}}$ ，中面 $z(0) = 0$ ，端面 $z(\ell_0) = L/2$ 。直接求导得 $(r/a)r_a = \alpha^2 \mathcal{J}/B$ ， $z_\zeta = B/\alpha^2$ ，其乘积恢复 $\mathcal{J}$ 。因而两种映射对于同一 C 给出相同总长度。

其梯度与度量为

$$F_B = \begin{pmatrix} r_a & r_\zeta \\ 0 & z_\zeta \end{pmatrix}, \quad G_B = F_B^{-1} F_B^{-T} = \begin{pmatrix} \frac{1}{r_a^2} + \frac{r_\zeta^2}{r_a^2 z_\zeta^2} & -\frac{r_\zeta}{r_a z_\zeta^2} \\ -\frac{r_\zeta}{r_a z_\zeta^2} & \frac{1}{z_\zeta^2} \end{pmatrix}. \quad (53)$$

G_A, G_B 中的交叉项不同，因此各自求解都使用对应的完整度量矩阵。图 21 将两种规则作用于同一个输入体积比场，显示保持圆柱壳与保持横截面分别对内部形状施加了怎样的约束。

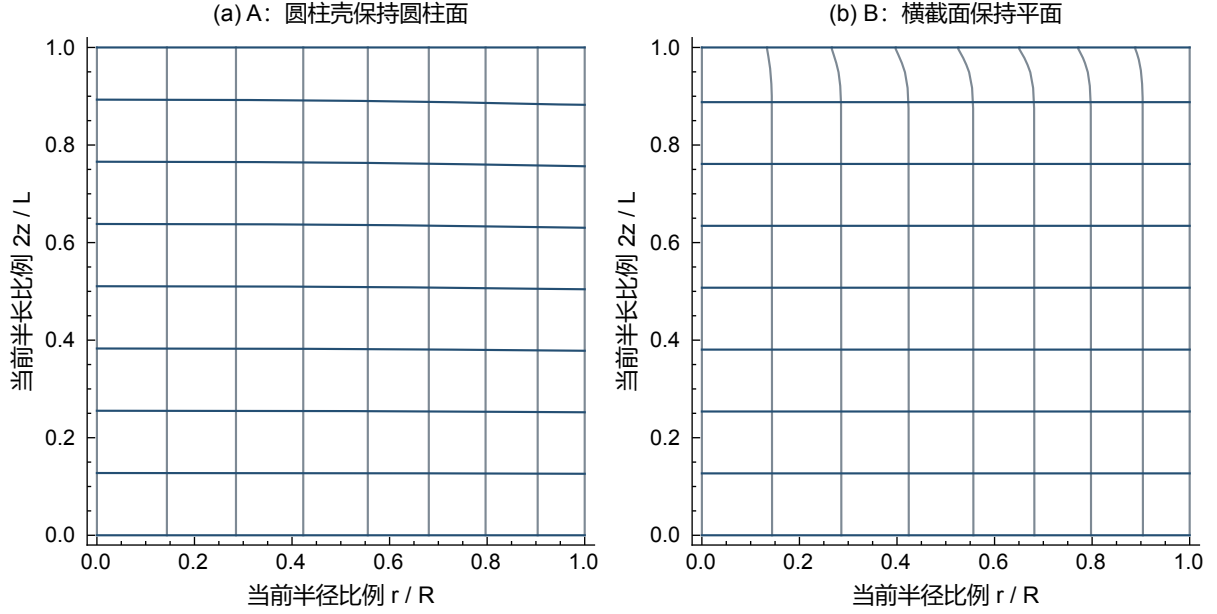


图 21 同一体积比场下的两种运动学映射。共同输入为映射 A 在 4 h 的场，图中只比较几何构造；两轴按当前半径及半长归一化，图形比例不代表药材实际长径比。

B.2.4 边界、轴线和等价情形

两种映射在初始均匀状态 $\mathcal{J} = 1$ 、 $R_{\text{obs}} = R_0$ 下恢复恒等映射。正 $\mathcal{J}$ 使 r_a, z_ζ 为正，连续映射保持方向。侧面与端面分别仍为直圆柱面和平面，中面对称复制后，整根外表面积权重为

$$dA_s = 4\pi R_{\text{obs}} z_\zeta d\zeta, \quad dA_e = 4\pi r r_a da. \quad (54)$$

两者分别积分为 $2\pi R_{\text{obs}} L$ 和 $2\pi R_{\text{obs}}^2$ 。轴线处不直接除以零：A 的 r/a 极限为 $\sqrt{\mathcal{J}(0)/\lambda_z}$ ，B 的极限为 $\alpha\sqrt{\mathcal{J}(0, \zeta)/B(\zeta)}$ ，由积分定义连续延拓。

若体积比可分离为 $\mathcal{J} = p(a)q(\zeta)$ ，记 $P(a) = \int_0^a b p(b) db$ 、 $Q(\zeta) = \int_0^\zeta q(s) ds$ 。将其分别代入 A、B，二者均化为

$$r = R_{\text{obs}} \sqrt{\frac{P(a)}{P(R_0)}}, \quad z = \frac{2P(R_0)}{R_{\text{obs}}^2} Q(\zeta). \quad (55)$$

仅沿径向变化也属于可分离情形。因此，两条几何规则不同，并不意味着对任何体积比场都会得到明显不同的映射。正文的敏感性结果只对应已经计算的两种构造及其共同条件。

B.3 有限元离散

参考域采用双线性四节点单元，主网格为 32×16 个单元，径向及轴向表面加密指数分别为 1.5 和 2.0。节点自由度为 C, H ，由 $T = 28 + H/(1850 + 4000C)$ 恢复温度。干质量采用集中形式，在整个积分过程中保持固定；单元及边界采用三阶高斯积分。

对任一参考试函数 N_i ，梯度按 $\nabla N_i = F^{-T} \nabla_0 N_i$ 转换。水分方程的扩散弱形式具有以下结构：

$$m_i \dot{C}_i = - \int_{\Omega_0} \rho_{a0} D (\nabla_0 N_i)^T G \nabla_0 C dV_0 - \int_{\partial\Omega} N_i J_s dA, \quad (56)$$

其中 m_i 为集中干质量， G 随 A、B 映射选择， dV_0 已包含轴对称权重。上式表达连续弱式的结构，实际离散还使用积分势：先在节点计算 $\Phi_{0.30}(C_i)$ ，再构造 $\Phi_h = \sum_i N_i \Phi_{0.30}(C_i)$ 。弱式中的 $DG \nabla_0 C_h$ 按 $b(T_q) G \nabla_0 \Phi_h$ 处理，对应参考水质量通量为 $-\rho_{a0} b(T_q) G \nabla_0 \Phi_h$ ，其中 $b(T_q) = 4.2 \times 10^{-4} e^{-3850/(T_q + 273.15)}$ 。

这一离散积分了含水率依赖项，与直接使用点值 $D(C_q, T_q)G\nabla_0 C_h$ 的通量不同。若在半根与整根权重之间切换，储存项和全部通量须同步采用相同倍数。能量方程的对应弱式为

$$m_i \dot{H}_i = \int_{\Omega} (\nabla N_i) \cdot (\mathbf{q} + h_b \mathbf{j}) dV - \int_{\partial\Omega} N_i [h(T_s - T_a) + h_b J_s] dA. \quad (57)$$

计算系数时，程序使用当前重建的几何及积分点状态，并保留非正交交叉项。双线性节点体积比场所需的径向、轴向构造积分可按分段多项式精确计算；场函数与非线性物性的空间积分仍有离散误差，其加密结果见附录 C。

每次隐式试算，程序先由 C 更新体积比、几何和度量，再由 H, C 恢复 T ，计算物性与通量。BDF 相对容差为 2×10^{-7} ， H, C 绝对容差分别为 10^{-3} J/kg、 10^{-9} ，最大步长 150 s。收敛困难由求解器通过调整步长处理，材料参数保持原设定。

输出指定物理位置时，首先反算其参考坐标。映射 A 先求解 $r(a) = r_{\text{sample}}$ ，再求对应的 z ；映射 B 先由 $z(\zeta)$ 确定 ζ ，再在该截面求解 $r(a, \zeta)$ 。随后在参考单元内插值。映射允许内部网格弯曲，因此节点编号和节点间直线距离都不能直接作为均匀物理坐标使用，否则会改变采样位置的含义。

B.4 完整数值结果

表 18 列出映射 A 的中面含水率、严格终止行及同期长度，均取当前物理半径。6 h 后所有列出时刻的 1.5 cm 位置已在域外。参考模型的连续临界时间为 47.9436179 h，严格达标秒为 172598 s， $C_{\max} = 0.1499994889$ ；表内时间与含水率只在显示时舍入。

表 18 第四问映射 A 的完整中面结果；含水率单位 kg/kg

时间 / h	0 cm	0.5 cm	1.0 cm	1.5 cm	当前表面	长度 / cm
6.0000	1.5734	1.3646	0.8395	—	0.3030	31.9256
12.0000	0.6099	0.5360	0.3291	—	0.1188	30.7238
18.0000	0.3520	0.3173	0.2082	—	0.0747	29.8325
24.0000	0.2564	0.2349	0.1626	—	0.0620	29.2198
30.0000	0.2093	0.1937	0.1391	—	0.0570	28.7750
36.0000	0.1815	0.1691	0.1246	—	0.0546	28.4571
42.0000	0.1630	0.1527	0.1147	—	0.0533	28.2073
47.9439	0.1500	0.1410	0.1075	—	0.0525	28.0262

达标时 $R = 1.2000$ cm、 $L = 28.0262$ cm。6 h 保存时刻的长度为 31.9256 cm，后续所列表值回落；延长至 72 h 的保存值为 27.6977 cm。上述保存值不用于断言 6 h 恰为连续长度曲线的极大点。

图 22 并列给出长度历史、收支及网格结果。长度曲线表示条件解的几何响应，收支与网格面板分别检查数值核算和离散变化。截面积比、长度比与体积比之间的乘积关系已由正文图 14 给出，可结合两图解释同一组输出。

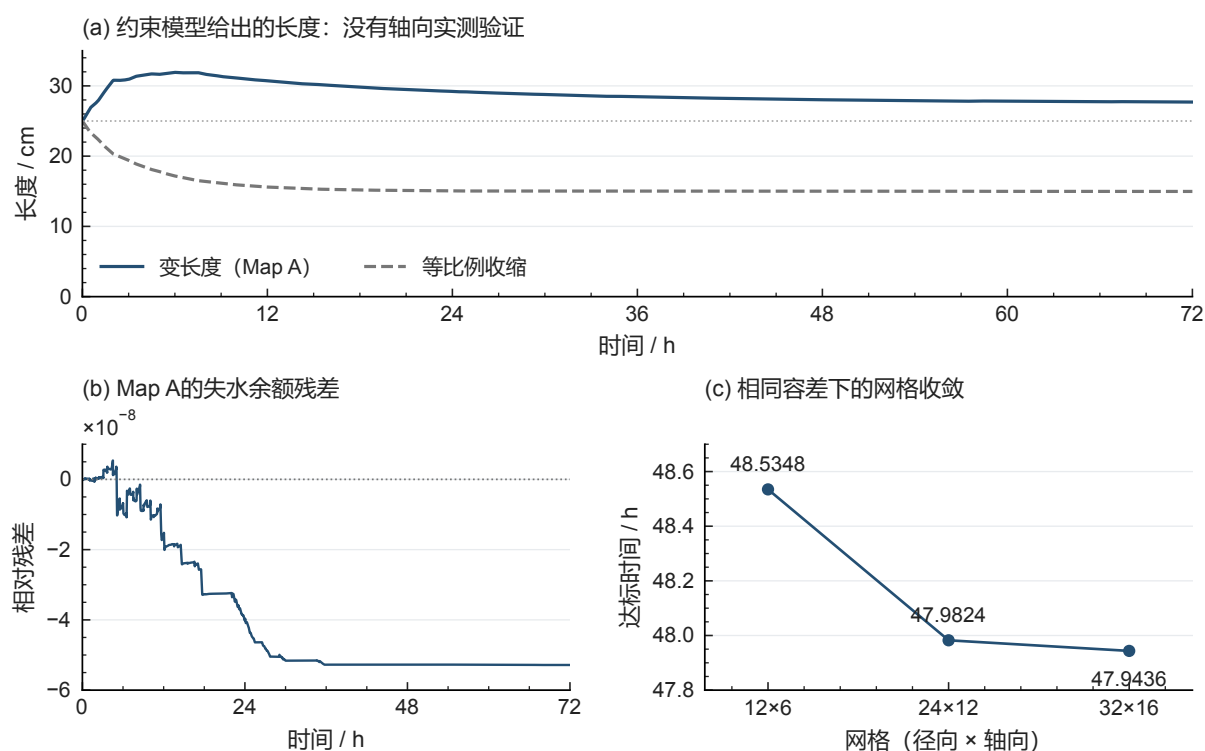


图 22 第四问映射 A 的长度历史、水量收支残差与网格检验。上排同时列出等比例长度作几何参照；变长度由密度、水分和附件半径共同确定，残差及网格差衡量所写方程的数值求解，不代表实物预测误差。

相较二维等比例模型，本模型同时改变了水质量输运、密度与体积的耦合、局部运动及水显焓。51.0920 h 与 47.9436 h 的差值对应这些变化的共同响应；没有单项配对证据时，不能将全部时差归于轴向伸长。

附录 C 数值检验补充

C.1 对照方案

各组对照均先确定改变对象，再固定其余条件，具体关系见表 19。解释结果时据此区分：只有匹配的单项变化才能用于判断该项影响；若方程与几何等多项同时改变，时差只能表示它们的组合响应。

表 19 各项对照的固定条件、改变项与解释范围

比较对象	固定条件	改变项	可支持的判断
一维与二维	同问物性、初态、环境、经验边界与几何	轴向场及端面表示	本组面积折算对局部场和总体时间的影响
第四问几何对照	附录 4、二维方程、初态和交换边界	固定尺寸或等比例收缩	所选扩散方程内的几何作用
变长度与等比例	附录 4、附件半径及初态	质量输运、局部运动、轴向长度、水显焓	多项修正的整体响应
局部映射 A、B	守恒输运、显焓、经验边界、附件及初态	局部映射及相应度量、面积权重	两种指定运动学的敏感性
潜热组内配对	同一平衡关系、映射 A、网格与精度	相变能量通道	指定表面参数下的能量影响
网格与时间精度	同一物理方程、边界和运动学	空间网格，或时间步上限与容差	数值离散及积分变化

C.2 空间与时间检验

C.2.1 基础一维与二维模型

基础模型的径向区间从 64 加密到 128；二维轴向区间同步从 32 增加到 64。全域事件均采用同一阈值和定位方式。表 20 保留全部临界时差，二维代表时刻的场差见表 10。

表 20 基础模型的连续临界时间空间加密变化

问题	维度	网格加密	细网格减粗网格 / s
三	一维	64 → 128	-3.61196
三	二维	64×32 → 128×64	-2.51995
四	一维	64 → 128	-3.78695
四	二维	64×32 → 128×64	-3.34789

在 201 个相同归一化中面位置上，问题一 1800 s 的二维温度、含水率加密差分别为 6.7630×10^{-4} °C、 2.3592×10^{-4} kg/kg。一维相应变化不超过 6.95×10^{-4} °C、 2.49×10^{-4} kg/kg。两组差值分别检验各自维度的离散，不能交叉代用。

二维时间精度检验保持 64×32 网格，将最大步长 120 s 降至 60 s；相对容差 2×10^{-8} 改为 2×10^{-9} ，温度和含水率绝对容差分别由 10^{-8} 、 10^{-10} 改为 10^{-9} 、 10^{-11} 。结果列于表 21。

表 21 二维固定空间网格下的时间精度配对检验

检验对象	收紧后减原设置
问题一 1800 s 中面最大温差	$5.3377 \times 10^{-7} \text{ }^{\circ}\text{C}$
问题一 1800 s 中面最大含水率差	$2.8391 \times 10^{-10} \text{ kg/kg}$
问题三连续临界时间变化	-0.0001653 s
问题四连续临界时间变化	+0.0116350 s

表中场差采用绝对值，时间差保留正负方向。时间精度检验同时收紧容差与步长上限，结果反映这一组设置的共同影响。现有两级空间网格和一组时间收紧尚不足以证明严格的渐近收敛阶，因此精度评价以实际观察到的变化量为依据。

C.2.2 变长度模型与局部映射

A、B 采用完全配对的物性、经验边界、固定干质量、显焓关系及数值参数。表 22 列出三组网格的连续临界时间。

表 22 同条件下两种局部映射的空间对照

径向 × 轴向单元	A / h	B / h	B 减 A / min
12×6	48.5348	48.4413	-5.6059
24×12	47.9824	47.9736	-0.5275
32×16	47.9436	47.9348	-0.5298

由 24×12 加密到 32×16 时，A、B 时长分别变化 -2.3293、-2.3316 min，配对时差仅变化 -0.1404 s。细网格中 B 比 A 短 0.5298 min，约占 A 时长的 0.0184%。稳定的配对差表明，这两条规则在当前条件下给出的总体时间接近；各自网格变化仍然较大，不能因配对中误差抵消，就认为任一模型的绝对误差小于 0.53 min。

变长度模型另在 12×6 网格上收紧时间精度，A、B 临界时间分别变化约 0.397、0.583 s。这些结果只检查各自粗网格上的时间积分，细网格仍需单独评价空间离散。容差和步长配置保存在结果文件中，定位方式见附录 E。秒级终止行便于复现判据，物理解释所用有效位数则受现有网格证据限制。

C.3 水量、显焓与几何检验

守恒模型中的总水量和总显焓分别为 $M_w = \int C dm_d$ 、 $E_H = \int H dm_d$ 。对于本附录不含气化焓差的 A、B 算例，累计残差定义为

$$\begin{aligned}\mathcal{R}_M(t) &= M_w(t) - M_w(0) + \int_0^t \int_{\partial\Omega(\tau)} J_s dA d\tau, \\ \mathcal{R}_H(t) &= E_H(t) - E_H(0) + \int_0^t \int_{\partial\Omega(\tau)} [h(T_s - T_a) + h_b(T_s)J_s] dA d\tau.\end{aligned}\tag{58}$$

程序将残差除以固定干质量，记录为单位干质量下的收支差。水量相对残差的尺度取初始含水率与累计水交换通道总变差中的较大者，本组为 2.55 kg/kg。显焓相对残差使用外部各能量交换通道

的累计总变差作为尺度，以免净交换接近零时放大误差；具体尺度保存在摘要文件中。表 23 按这些不同定义分别列出指标。

表 23 水量、显焓和几何核查：浮点零仅针对所检查的保存量

检查量 (32×16)	映射 A	映射 B
累计相对水量残差	5.28×10^{-8}	5.56×10^{-8}
累计相对显焓残差	1.05×10^{-7}	1.01×10^{-7}
积分点密度—体积最大相对差	0.3866%	0.3866%
保存外半径最大误差 / m	3.47×10^{-18}	0
保存端平面最大误差 / m	2.78×10^{-17}	1.67×10^{-16}
独立边界面积积分最大相对差	6.66×10^{-16}	8.88×10^{-16}
长度与干质量体积积分最大差 / m	2.22×10^{-16}	1.67×10^{-16}

节点处 $\mathcal{J} = \rho_{d0}/\rho_d(C)$ 按定义成立，连续积分映射也满足雅可比恒等式；在积分点记插值得到的体积比与含水率为 $\mathcal{J}_q, C_q$ ，从固定干质量和体积反算湿密度，再与题给密度函数比较，定义

$$\delta_\rho = \frac{|\rho_{d0}(1 + C_q)/\mathcal{J}_q - \rho(C_q)|}{\rho(C_q)}. \tag{59}$$

该指标的最大值为 0.3866%。若用 $|\mathcal{J}_q - \mathcal{J}(C_q)|/\mathcal{J}(C_q)$ 衡量体积比差，结果约为 0.3851%；由于归一化定义不同，两个数值应分别报告。该差异随加密减小，反映空间表示与非线性物性插值之间的误差。它与累计约 10^{-8} — 10^{-7} 的收支残差检查不同环节，不能相互替代，也不能合并解释为“全域精确守恒”。

几何核查还检查了保存的中面、轴线位置，以及通过连续映射反算的物理采样。保存值中的中面与轴线误差为零，独立求根与程序采样的最大差为 10^{-14} 量级。这些结果验证当前几何和采样算法的实现，内部材料点是否按该方式运动仍需观测确认。

基础二维主模型按式 (21) 核算参考域含水率收支。问题一的累计最大残差为 7.5570×10^{-10} kg/kg，问题二、三及第四问见表 11。边界在观测间线性插值，累计通量在环境及半径插值结点处分段积分。这些残差对应标量含水率方程，并未按真实湿密度核算水质量；温度瞬时残差也未包含主模型省略的能量项。

C.4 局部运动学对照

图 23 比较两种映射的长度历史、差值和中面水分剖面。两套结果都先通过附录 B 的连续映射反算参考坐标，再在共同的实际半径上插值。这样比较的是同一物理位置，避免将位置不同的参考节点直接配对。

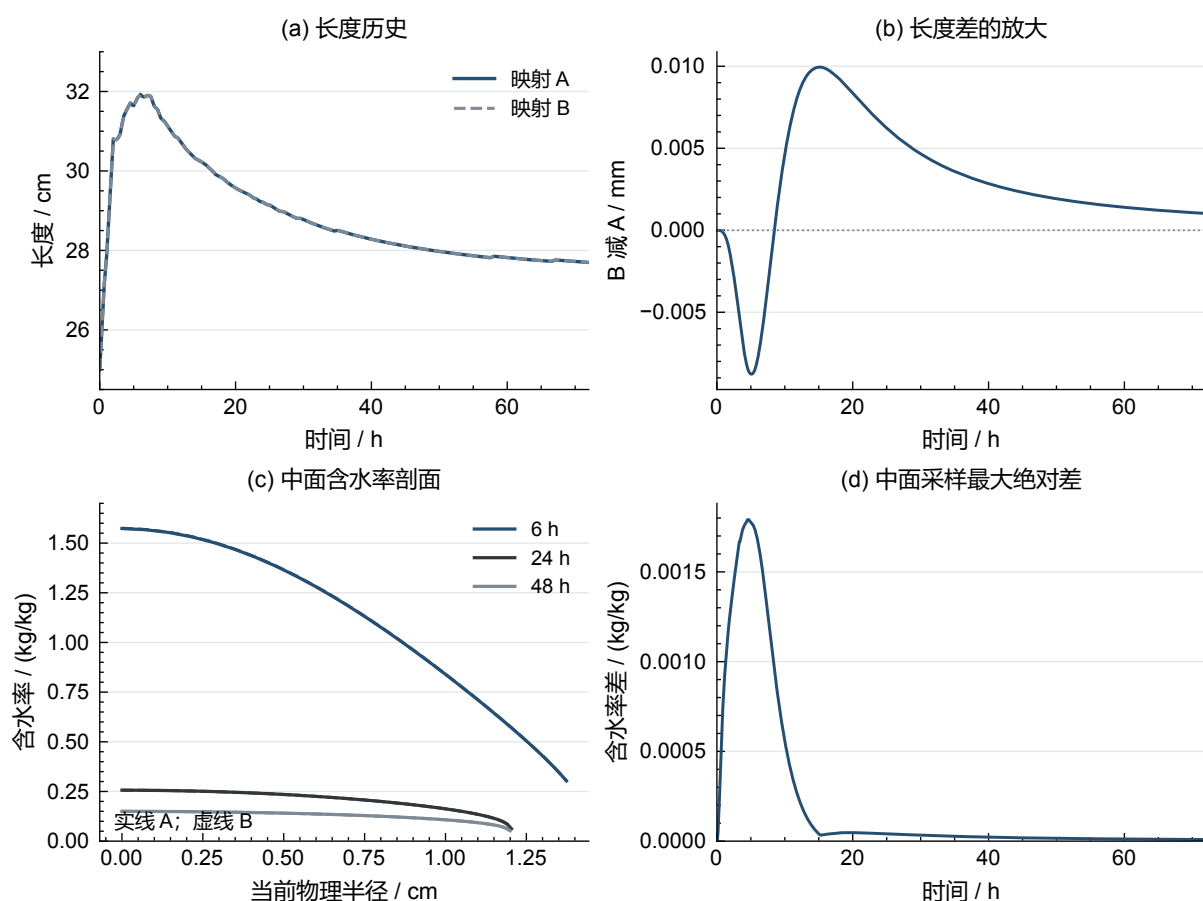


图 23 映射 A、B 的长度及中面含水率对照，均为 32×16 网格。长度差单独放大，中面剖面取 6、24、48 h；末面板在每个保存时刻比较 101 个共同物理半径。曲线为条件解，采样最大差不构成连续时空误差界。

0—72 h 保存时刻中的长度最大绝对差为 0.0099498 mm，出现在 15.150 h。含水率比较在各时刻取 $r/R(t) = 0, 0.01, \dots, 1$ 共 101 点，最大绝对差为 0.0017920 kg/kg，出现在 4.6167 h、 $r/R = 0.40$ 。表 24 列出部分保存时刻，帮助判断大差值出现的阶段。

表 24 共同保存时刻的长度与中面含水率差；差值单位 kg/kg

时间 / h	A 长度 / cm	B 长度 / cm	B 减 A / mm	101 点最大含水率差
1.0	27.8209	27.8208	-0.000185	0.0008397
3.0	30.9269	30.9264	-0.004282	0.0015972
6.0	31.9256	31.9248	-0.007852	0.0016745
12.0	30.7238	30.7246	0.008339	0.0002421
24.0	29.2198	29.2205	0.006616	0.0000421
36.0	28.4571	28.4575	0.003416	0.0000258
48.0	28.0247	28.0249	0.002063	0.0000167
72.0	27.6977	27.6978	0.001024	0.0000089

差异主要出现在前期，后期逐渐减小。两组计算还共同保存了达标后的输出，因此 48 h 和 72 h 的场仍可相互比较。两种映射对可分离体积比场给出相同几何，本次又均未加入潜热，故检验范围

限于当前两种构造。结果不能外推到任意内部变形或潜热边界，也不能据此将主模型与变长度模型的整体时差单独归于长度变化。

附录 D 蒸发边界参数化

D.1 水活度与平衡含水率

潜热实验采用第四问的守恒输运与映射 A，附件半径、题给 h, h_m, D 及初态均保持不变。为使表面降温能够反馈到净排水方向，补充候选水活度函数，并反解相应的平衡含水率：

$$a_w(C, T) = \frac{C}{C + s} \exp[g(T)], \quad C_{eq}(T, w_a) = \frac{s p_a(w_a)}{e^{g(T)} p_{sat}(T) - p_a(w_a)}. \quad (60)$$

表面平衡要求 $a_w(C_{eq}, T) p_{sat}(T) = p_a(w_a)$ 。s 控制该候选材料关系，需独立平衡数据才能标定 [7]。本研究采用有限的参考点对照，未将其识别为真实药材的解吸等温线。

令 $\vartheta = T + 273.15$ 、 $\vartheta_0 = 301.15$ K，温度修正和空气水蒸气分压换算取

$$g(T) = \frac{4186 - 4000}{461.52} \left[\ln \frac{\vartheta}{\vartheta_0} + \frac{\vartheta_0}{\vartheta} - 1 \right], \quad (61)$$

$$p_a(w_a) = 101325 \frac{w_a}{0.62198 + w_a} \quad (\text{Pa}).$$

这里将总压额外设为 101325 Pa，空气水分指标按 kg 水/kg 干空气换算。温度修正的零点取 28 °C，即 $g(28) = 0$ ，未选在约 50 °C 的末期环境。各系数的含义分别为：4186 是候选纯液态水比热，4000 是由题给材料比热换算的迁移水有效比热，461.52 是水蒸气比气体常数，单位均为 J/(kg K)。

D.2 蒸气状态、水焓与边界方向

采用 28 °C 的纯水饱和压 3783.1 Pa 与汽化焓 2434600 J/kg 为参考值，温度范围内以常比热关系展开。参考取值可与水热物性资料核对 [8]，常比热近似及材料水活度修正属于本条件模型。令 $\Delta c = 1860 - 4186 = -2326$ J/(kg K)，则

$$\ln \frac{p_{sat}(T)}{3783.1 \text{ Pa}} = \frac{(2434600 - \Delta c \vartheta_0)(1/\vartheta_0 - 1/\vartheta) + \Delta c \ln(\vartheta/\vartheta_0)}{461.52}, \quad (62)$$

$$h_v(T) = 2434600 + 1860(\vartheta - \vartheta_0) \quad (\text{J/kg}).$$

内部水分与显焓仍采用附录 B。外部净水质量通量改为 $J_s = \rho_{d,s} h_m (C_s - C_{eq})$ ；同一有向通量进入能量边界。以 $J_s > 0$ 为蒸发，取上游 $T_{up} = T_s$ ；以 $J_s < 0$ 为凝结，取 $T_{up} = T_a$ 。总向外能量通量为

$$(q + h_b j) \cdot n = h(T_s - T_a) + h_{out} J_s, \quad (63)$$

$$h_{out} = \begin{cases} h_v(T_{up}), & \text{开启相变通道,} \\ h_b(T_s), & \text{关闭相变通道.} \end{cases}$$

开启能量通道后，边界即为正文式 (34)。蒸气排出焓与内部有效水显焓的差额，由导热、对流和储热共同负担 [6]。关闭该通道仅用于同一平衡关系下的消融对照，不对应另一种已经标定的材料。

表面降温会改变 p_{sat} 和 C_{eq} ，继而改变净排水通量，求解器在试算中同步更新这些量。因此，计算中的通量会随温度反馈调整，未采用固定原排水量再扣热的处理。若式 (60) 的反解分母非正，说明该参数化不再适用于当前状态，程序会抛出异常并终止该次计算；不截断驱动力，也不强制维持正蒸发。

D.3 参考参数与求解设置

参考环境 (T_b, w_b) 为附件 1 末小时均值。分别指定 $C_b = 0.03, 0.075, 0.12 \text{ kg/kg}$ ，通过

$$s = C_b \left[\frac{e^{g(T_b)} p_{\text{sat}}(T_b)}{p_a(w_b)} - 1 \right] \quad (64)$$

由上式得到三组 s ，只是将指定参考点换算为函数参数。这一步没有利用附件进行材料回归，也未把其他模型的预期时长当作拟合目标。

每组均保留同一表面关系、附件半径、映射 A 规则、守恒输运和显焓定义，仅切换式 (63) 中的能量通道。空间采用 32×16 单元和三阶积分，BDF 相对容差为 2×10^{-7} ， H, C 绝对容差为 10^{-3} J/kg 、 10^{-9} ，最大步长 300 s 。两算例使用同一几何闭合规则，但各自的水分场不同，实际长度须分别重算，长度历史因此可以不同。

关闭相变通道时以 h_b 核算边界水焓，开启时改用上游气相焓，残差检查也随所选通道更新。本组未另拟合传质系数， h_m 仍取题给数值；将它与新平衡驱动力相乘，仍属于需要说明的条件建模选择。

D.4 完整配对结果

表 25 列出连续临界时间。图 24 在每一参考平衡关系内部连接两种能量设置，线段长度直接对应组内差值。

表 25 相同参考平衡关系与运动学规则下的潜热配对结果

参考 $C_b / \text{kg/kg}$	关闭相变 / h	开启相变 / h	组内时差 / h
0.030	47.6772	55.9708	8.2935
0.075	49.0904	59.0292	9.9388
0.120	57.4266	70.3384	12.9118

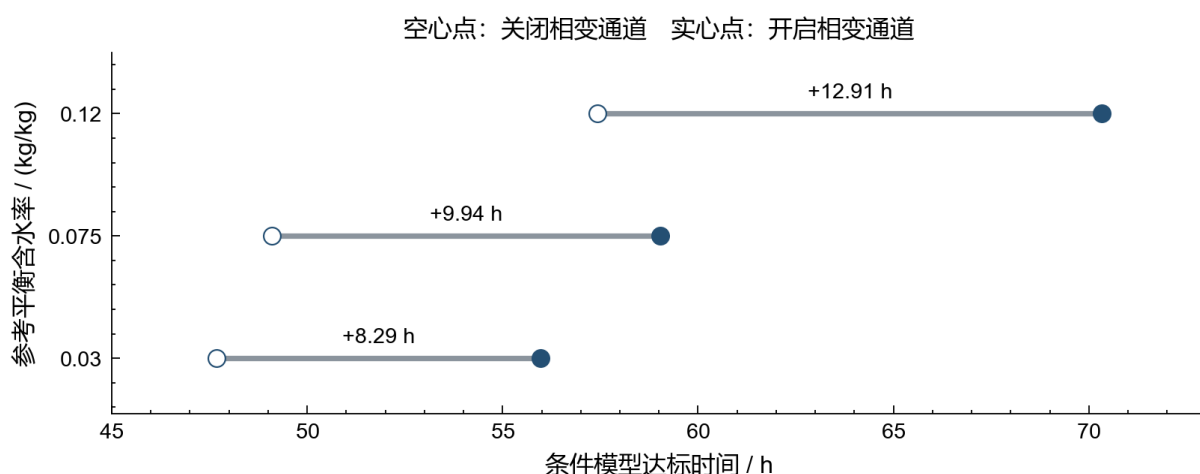


图 24 第四间潜热条件实验的配对点图，采用 32×16 网格。每条横线连接同一参考平衡点的两种能量设置；横线和三组点均无统计误差条或置信区间含义。

开启相变后，三组时长分别增加 8.2935、9.9388、12.9118 h，反映各自闭合规则内温度、水分和长度反馈的共同作用。不同参考点给出不同绝对时长，仍需材料数据约束。组内关闭相变的结果也保留平衡边界，不能与第 9 章 47.9436 h 作单因素比较。

图 25 进一步列出 0.075 kg/kg 参考组的长度和能量。开启、关闭组的保存长度峰值分别为 45.1229 cm (7.5 h) 和 32.3320 cm (6 h)。这些长度由密度、附件半径和水分场共同反推，尚无轴向实测验证；明显伸长也是该条件模型需要检验的输出。

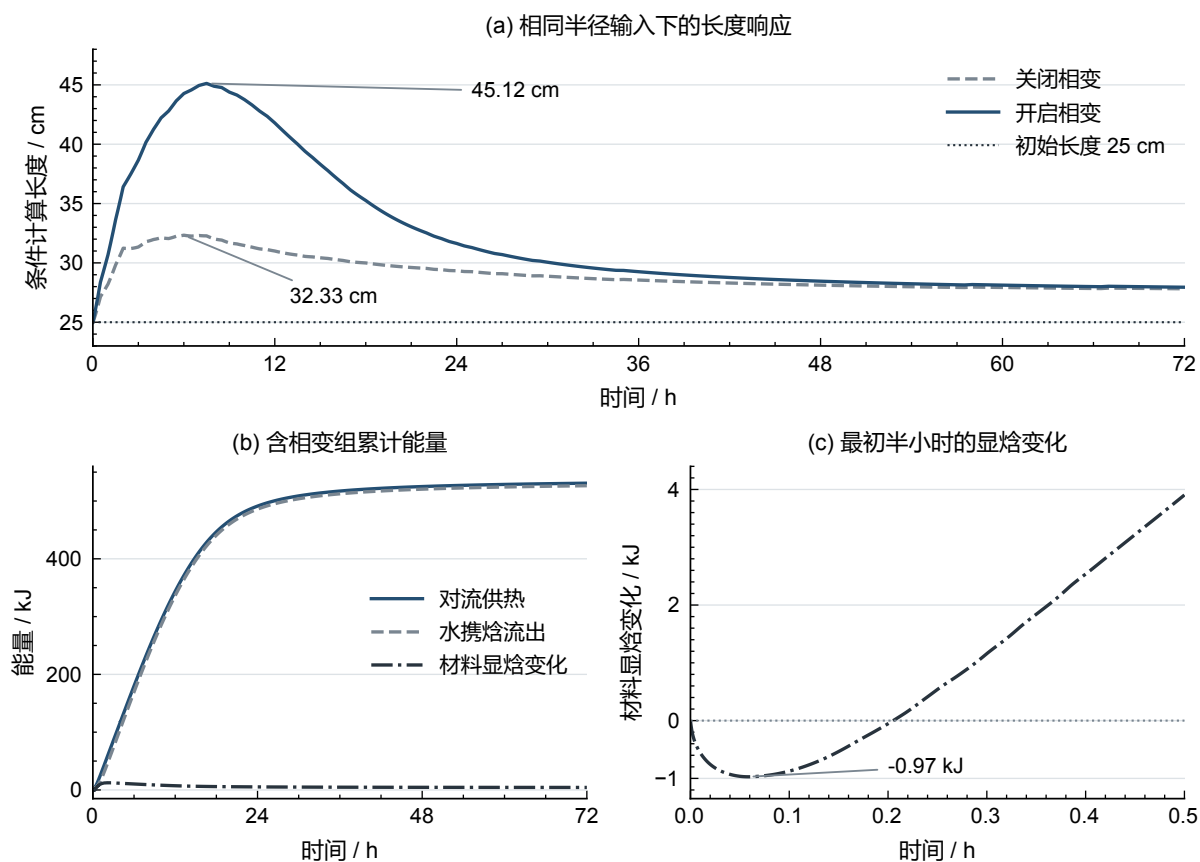


图 25 潜热参考组的几何与能量响应。长度图使用 0.075 kg/kg 下的开、关配对；能量图仅对应开启组，均换算为整根药材的 kJ。累计对流供热与水携焓流出、材料显焓增量配套比较，储热放大图保留相对初态的负值。水携焓流出包含显焓，长度属于条件计算。

72 h 时，对流累计供热 530.8686 kJ，水携焓流出 526.4898 kJ，材料显焓增加 4.3788 kJ；前期相对初态的储热曾降至约 -0.9722 kJ。图中收支检查配套方程的数值核算，不能代替材料验证。

图 26 比较三组参考点各自的开、关算例。24×12 加密到 32×16 时，六条绝对时长变化约 -2.28 至 -2.56 min，组内增时变化分别为 -1.4651、+3.2368、+8.7232 s。配对中的空间变化存在抵消，不能把这几个秒级差值当作绝对时长误差界。另在 0.075 kg/kg、12×6 开启组同时收紧容差及步长上限，临界时刻仅变化约 +0.090 s；该检查只覆盖这一粗网格算例。

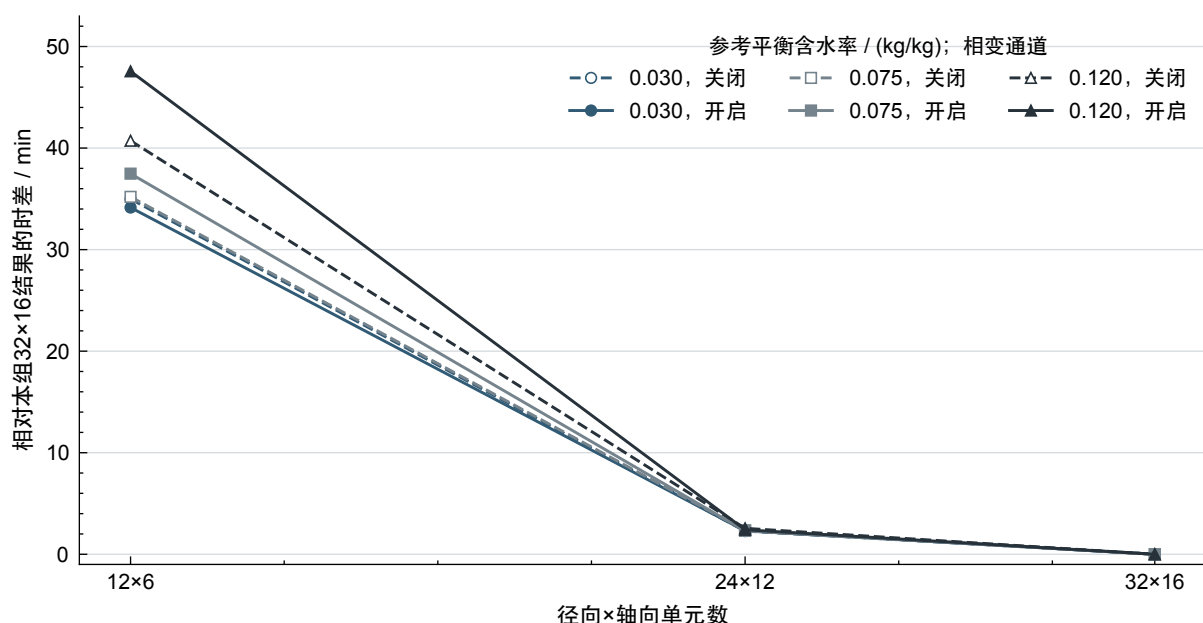


图 26 潜热条件算例的三网格对照。纵轴为每条算例相对自身 32×16 结果的时差；曲线只连接 12×6、24×12、32×16 三个已运行网格。颜色和标记区分参考平衡点，实虚线及填充区分相变开关。最细网格的零值由参考相减得到，不表示精确解误差为零。

附件半径作为所有算例的外部输入，能够重现它并不能独立约束 s 。目前缺少独立失重、表面温度或平衡含水率观测，三组参数因而无法依据本题材料证据排序。这些结果用于判断给定条件下的潜热影响，并揭示主模型省略相变的物理限制；尚无一组能够作为唯一的含潜热题定答案。

附录 E 程序与计算资料

E.1 算法流程与源程序

源程序分为二维主模型、一维近似对照、变长度研究和潜热配对实验四部分，主要程序及用途见表 26，完整代码列于附录 F。主求解程序依次计算当前物性、共享面通量和温湿场变化率，再通过隐式 BDF 方法推进，并检查全域干燥终止条件。问题二、三沿用同一初值问题；第四问单独读取附件半径。变长度研究分别实现两种局部映射，并在相同物理位置比较结果。潜热实验在各自相同的表面平衡关系下开关潜热项，保持组内配对条件一致。

表 26 主要源程序及用途

模型类别	程序文件	主要功能
二维主模型及一维对照	run_model.py、solve.py	读取各问条件，求解温湿场并确定达标时间
结果导出	export_and_audit.py	按题目模板生成结果表并核对数值
变长度映射 A	run_case.py、solve_case.py	守恒输运与变长度求解
变长度映射 B	record_compare.py、mapping_b.py	不同局部运动学下的配对计算
潜热配对实验	replay_case.py、run_experiment.py	三组表面平衡条件下的潜热开关对照
解析界限	check_bounds.py	计算末期长度的必要范围

E.2 输入与结果文件

附件 1 提供 241 组环境记录，附件 2 提供 145 组半径记录。问题一至三使用各自题给物性及初始几何，第四问使用收缩附件。四问结果分别保存于 result1.xlsx 至 result4.xlsx；表 27 列出数据规模与输出时刻，数据行数不含表头。

表 27 正文四问二维题定工作簿

工作簿	数值表	每张表数据行数	最后一行时刻/s	半径及表面口径
result1.xlsx	温度、水分浓度	1800	1800	0—2 cm，间隔 0.1 cm
result2.xlsx	温度、水分浓度	10800	10800	0—2 cm，间隔 0.1 cm
result3.xlsx	水分浓度	3449	206900	0—2 cm，间隔 0.1 cm
result4.xlsx	水分浓度	3066	183932	0—1.9 cm 固定位置，末列为当前表面

各数值表包含时间列与 21 个结果列。问题一、二逐秒输出，问题三、四逐分钟输出并补入严格达标整数秒。第四问固定位置超出当前药材半径时留空，末列单列当前表面值。全部数值及空值位置均已与保存的求解数组逐项核对。内部计算使用双精度，仅在表格显示时舍入。

E.3 计算环境与图形资料

计算使用 Python 3.13.9、NumPy 2.4.4 与 SciPy 1.17.1，题定工作簿由 openpyxl 3.1.5 导出。各案例的网格、容差、步长上限和边界参数随源码提供，数值检验见正文及附录 C。图形由保存的二维场、剖面及配对数据生成，绘图过程不改变求解结果和材料参数。

附录 F 完整程序源码

本附录列出模型求解、对照实验、结果导出及绘图所用源程序。121 个源文件中，相同内容合并列示，共 72 份代码。源文件及必要配置随支撑材料一并提供；本机路径已匿名化，算法与数值参数保持不变。

核心计算伪代码

统一记号。 T 为温度， C 为干基含水率， $R(t)$ 、 $L(t)$ 为当前半径和整根长度， $\Omega_{rz}(t)$ 为中面至端面的二维轴对称计算域。每个问题均以题设原始温湿状态为初值；问题一至三保持初始尺寸，问题四由附件 2 的半径记录更新几何。主模型分别计算侧面和端面交换，**不显式加入蒸发潜热**。

算法 1 四问条件装配

输入：问题编号 $q \in \{1, 2, 3, 4\}$ ；附件 1 环境序列 $\{T_a(t), w_a(t)\}$ ；附件 2 半径序列 $R_{\text{obs}}(t)$ ；题给物性关系；初始条件 (T_0, C_0) ；输出时刻与位置。

输出：本问配置 $\mathcal{P}_q$ ，包括物性、几何、终止规则和输出规则。

1. 读取本问指定的附录物性关系：问题一使用附录 2，问题二、三使用附录 3，问题四使用附录 4。
2. 令 $T(r, z, 0) \leftarrow T_0$ ， $C(r, z, 0) \leftarrow C_0$ 。问题二、三均从题设初态开始，问题三只是在问题二的同一初值问题上延长积分时间。
3. 若 $q \leq 3$ ，设 $R(t) \leftarrow R_0$ ， $L(t) \leftarrow L_0$ ；若 $q = 4$ ，在附件记录点间线性插值得到 $R(t)$ ，并按主模型的等比例几何关系令 $L(t) \leftarrow L_0 R(t)/R_0$ 。
4. 对附件 1 中的 T_a, w_a 作时间插值。计算超过环境记录后，采用末小时均值延拓；问题四不将半径外推到附件覆盖范围外。
5. 设定输出：问题一、二每 1 s 保存一次；问题三、四每 60 s 保存一次，并额外保存严格达标时刻。

算法 2 二维轴对称热湿主求解器

输入：配置 $\mathcal{P}_q$ ，径向-轴向有限体积网格，初始状态 $y^0 = (T^0, C^0)$ 。

输出：已接受时间点的二维场 $\{T^n, C^n\}$ 、表面通量、几何与事件记录。

1. 在半纵截面 $0 \leq r \leq R(t)$ 、 $0 \leq z \leq L(t)/2$ 上建立控制体。轴线 $r = 0$ 与中面 $z = 0$ 采用对称边界；侧面 $r = R(t)$ 与端面 $z = L(t)/2$ 分别施加热质交换边界。
2. 将所有节点温度和含水率拼接为同一状态向量 $y = (T_{i,j}, C_{i,j})$ ，设初始时刻 $t \leftarrow 0$ 。
3. 当 t 未达到本问结束条件时，重复：
 - a) 从当前试算状态 y 计算 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 和 $D(C, T)$ ；问题一按其附录的常数热物性处理。
 - b) 从当前时刻读取 $T_a(t), w_a(t)$ ；问题四同时读取 $R(t), L(t)$ ，更新节点实际坐标、相邻节点距离、控制体体积及侧面和端面面积。
 - c) 在每个内部共享面计算导热通量和水分扩散通量；在外表面计算对流换热和题给经验水分交换通量。
 - d) 将各控制体的净通量写成联立半离散方程 $\dot{y} = F(t, y)$ 。共享面通量在相邻控制体中以相反符号计入。
 - e) 用隐式 BDF 推进到下一已接受时间点；每次试算右端项时均重复上述物性、几何和通量更新。若非线性迭代未收敛或局部误差超限，缩小时间步后重试；否则接受新状态并更新 t 。
 - f) 若到达保存时刻，则由已接受解的连续输出插值并保存 T, C 、当前几何和边界通量。

算法 3 全域达标时间与题定结果提取

输入：已保存或连续输出的二维含水率场 $C(r, z, t)$ ，阈值 $C_{\text{crit}} = 0.15 \text{ kg/kg}$ 。

输出：连续临界时间、首个严格达标整数秒及题定表格。

1. 对每个候选时刻，计算所有二维节点中的最大含水率：

$$C_{\text{max}}(t) \leftarrow \max_{(i,j)} C_{i,j}(t).$$

2. 当 C_{max} 首次接近 C_{crit} 时，利用连续输出定位 $C_{\text{max}}(t) = C_{\text{crit}}$ 的临界时刻。
3. 从临界时刻之后逐秒检查，取第一个满足 $C_{\text{max}}(t) < C_{\text{crit}}$ 的整数秒作为题定终止时刻。判定使用未舍入的双精度数值。
4. 问题一、二：在中面 $z = 0$ 读取题定半径位置的温度和含水率。问题三：除中面表外，报告全域最大含水率及最后达标位置。
5. 问题四：先判断固定物理位置是否满足 $r \leq R(t)$ ；域外位置输出空值，当前表面单独取 $r = R(t)$ 的数值。不得将域外位置记为零含水率。

算法 4 第四问的收缩几何更新

输入：附件 2 半径记录，初始尺寸 (R_0, L_0) ，参考域坐标 (ξ, η) 。

输出：各时刻移动域中的节点坐标、控制体几何量和边界面积。

1. 在半径记录的相邻时刻间对 $R_{\text{obs}}(t)$ 作线性插值，得到当前半径 $R(t)$ 。
2. 按等比例收缩主假设，计算 $L(t) = L_0 R(t) / R_0$ 。
3. 将固定参考坐标映射为当前物理坐标：

$$r \leftarrow R(t)\xi, \quad z \leftarrow \frac{L(t)}{2}\eta.$$

4. 用当前坐标重算每个控制体的体积、径向共享面面积、轴向共享面面积以及侧面和端面交换面积。
5. 将更新后的几何量传入算法 2 的通量计算。时间导数已按材料随几何变化的形式处理，不再额外叠加网格平流项。
6. 保存时，固定空间位置仅在当前域内插值；同时记录当前半径和按本假设得到的长度。

算法 5 数值检验与一维对照

输入：二维主计算、面积折算一维对照、粗细两级空间网格及两组时间精度设置。

输出：网格与时间敏感性、离散收支残差和维度对照结果。

1. 在相同物性、初态、环境、几何和经验边界下运行一维面积折算模型。初始折算系数为 $\gamma = 1 + R_0/L_0 = 1.08$ 。
2. 将一维结果分别与二维中面结果、二维轴向平均结果在共同保存时刻和共同物理半径处比较；记录局部温湿差和连续临界时间差。
3. 将二维网格由 64×32 加密为 128×64 ，比较代表时刻的场差与临界时间变化；在固定网格下收紧最大步长和容差，比较时间积分结果。
4. 对标量含水率方程累计内部储量变化与边界净通量，计算离散水分收支残差；对温度方程检查瞬时离散热量收支残差。
5. 仅当网格、时间精度和离散收支的变化均明显小于所讨论的模型差异时，将该差异解释为模型条件带来的响应，而非数值步长造成的假象。

说明。以上伪代码概括二维主模型及数值检验。一维面积折算作为对照；变长度和潜热条件研究的适用范围见正文，其程序与主模型源码一并列于后文。

编号	源文件	起页
F.1	run_model.py	52
F.2	verify_package.py	54
F.3	assemble_data.py	54
F.4	build_workbooks.mjs	58
F.5	export_workbooks.py	59
F.6	package_model.py	60
F.7	plot_system.py	62
F.8	write_report.py	64
F.9	baseline_engine.py	70
F.10	check_identities.py	72
F.11	export_and_audit.py	73
F.12	frozen_empirical_engine.py	77
F.13	replay.py	84
F.14	run_cases.py	85
F.15	run_record.py	86
F.16	solve.py	92
F.17	assemble.py	96
F.18	build_delivery.py	98
F.19	__init__.py	100
F.20	__init__.py	100
F.21	boundary_model.py	100
F.22	thermo_reference.py	102
F.23	volume_fem.py	108
F.24	elongation_model.py	116
F.25	experiment_model.py	125
F.26	minimal_model.py	126
F.27	preflight.py	128
F.28	run_case.py	129
F.29	solve_case.py	130
F.30	summarize_results.py	135
F.31	compare_run.py	136
F.32	mapping_b.py	136
F.33	mapping_preflight.py	138
F.34	record_compare.py	140
F.35	analyze_comparison.py	140

编号	源文件 (续)	起页
F.36	document_results.py	143
F.37	plot_comparison.py	145
F.38	assemble_report.py	148
F.39	audit_branch.py	150
F.40	import_evidence.py	153
F.41	launch_refinement.py	155
F.42	plot_branch.py	156
F.43	replay_case.py	160
F.44	run_experiment.py	161
F.45	verify_package.py	163
F.46	write_report.py	164
F.47	run_queue.py	174
F.48	audit.py	175
F.49	old-q123-solve.py	177
F.50	check_bounds.py	181
F.51	build_reproduction_index.py	184
F.52	extract_two_d.py	189
F.53	audit_latent_evidence.py	196
F.54	audit_new_figures.py	200
F.55	build_main_results.py	202
F.56	build_structure_figures.py	209
F.57	build_comparisons.py	215
F.58	audit_figures.py	221
F.59	build_figures.py	223
F.60	prepare_data.py	228
F.61	build_figures.py	233
F.62	build_merged.py	238
F.63	build_figures.py	241
F.64	validate_figures.py	250
F.65	extract_paper_data.py	251
F.66	enrich.py	252
F.67	round2-figures.py	256
F.68	check_and_package.py	258
F.69	figure_library.py	259
F.70	figure_typography.py	265

编号	源文件（续）	起页
F.71	_common.py	268
F.72	figure_export.py	271

F.1 run_model.py

类别：统一入口与结果导出。原始代码共 67 行。

```
1  """One entry point for the frozen, explicitly separated model families.
2
3  Every replay writes to a new directory; existing numerical evidence is untouched.
4  This launcher does not change any physical parameter or combine model equations.
5  """
6  from pathlib import Path
7  import argparse
8  import json
9  import shutil
10 import subprocess
11 import sys
12
13 ROOT = Path(__file__).resolve().parent
14
15
16 def main():
17     parser = argparse.ArgumentParser(description=__doc__)
18     parser.add_argument('--model', choices=['1d', '2d', 'fixed-control', 'variable-length',
19     'latent'], required=True)
19     parser.add_argument('--question', type=int, choices=[1, 2, 3, 4], required=True)
20     parser.add_argument('--output-dir', type=Path, required=True, help='A directory that does not
21     exist yet')
22     parser.add_argument('--latent-case', help='For latent only; an existing case in
23     latent-branch/designs/all-main-cases.json')
24     parser.add_argument('--dry-run', action='store_true', help='Print the frozen selection and
25     commands without writing files or integrating PDEs')
26     args = parser.parse_args()
27     destination = args.output_dir.resolve()
28     if destination.exists():
29         parser.error('Output directory already exists; use a new directory to preserve the
30         evidence.')
31     if args.model not in ['1d', '2d'] and args.question != 4:
32         parser.error('This family is a question-4 branch; no question-4 data may be imported into
33         questions 1-3.')
34     if args.latent_case and args.model != 'latent':
35         parser.error('--latent-case applies only to the latent branch.')
36
37     commands = []
38     if args.model in ['1d', '2d', 'fixed-control']:
39         case = 'q4-2d-fixed-control' if args.model == 'fixed-control' else f'q{"23" if
40         args.question in [2, 3] else args.question}-{args.model}-main'
41         commands.append([sys.executable, '-B', str(ROOT/'baseline/src/replay.py'), case,
42         '--output-dir', str(destination)])
43         settings = json.loads((ROOT/'baseline/configs'/f'{case}.json').read_text())
44     elif args.model == 'latent':
```

```

38     case = args.latent_case or 'q4-latent1-ce075-32x16'
39     matches = [c for c in
40                 json.loads((ROOT/'latent-branch/designs/all-main-cases.json').read_text())['cases'] if
41                 c['name'] == case]
42     if len(matches) != 1:
43         parser.error('Unknown frozen latent case.')
44     settings = matches[0]
45     commands.append([sys.executable, '-B', str(ROOT/'latent-branch/src/replay_case.py'),
46                     '--case', case, '--destination', str(destination)])
47 else:
48     case = 'q4-robin-latent0-32x16'
49     settings =
50     json.loads((ROOT/'variable-length/results'/case/'summary.json').read_text())['config']
51     commands.append([sys.executable, '-B', str(destination/'src/run_case.py'), '--nr',
52                     str(settings['nr']), '--nz', str(settings['nz']), '--out', 'results/replay', '--rtol',
53                     str(settings['rtol']), '--atol-H', str(settings['atol_H']), '--atol-C',
54                     str(settings['atol_C']), '--max-step', str(settings['max_step'])])
55     commands.append([sys.executable, '-B', str(destination/'src/build_delivery.py'),
56                     '--cases', 'replay'])
57
58 selection = {'model': args.model, 'question': args.question, 'frozen_case': case,
59             'settings': settings, 'commands': commands, 'output_directory': str(destination)}
60 print(json.dumps(selection, ensure_ascii=False, indent=2), flush=True)
61 if args.dry_run:
62     return
63 if args.model == 'variable-length':
64     destination.mkdir(parents=True)
65     for name in ['src', 'input', 'designs']:
66         shutil.copytree(ROOT/'variable-length'/name, destination/name,
67                         ignore=shutil.ignore_patterns('__pycache__', '*.pyc'))
68     (destination/'analysis').mkdir()
69 for command in commands:
70     subprocess.run(command, cwd=destination if args.model == 'variable-length' else ROOT,
71                   check=True)
72 (destination/'model-selection.json').write_text(json.dumps(selection, ensure_ascii=False,
73                   indent=2)+'\n')
74
75 if __name__ == '__main__':
76     main()

```

F.2 verify_package.py

类别：统一入口与结果导出。原始代码共 32 行。

```
1 """Verify packaged files against their recorded SHA-256 values (not model truth)."""
2 from pathlib import Path
3 import hashlib
4 import json
5
6 ROOT = Path(__file__).resolve().parent
7
8
9 def digest(path):
10     h = hashlib.sha256()
11     with path.open('rb') as handle:
12         for block in iter(lambda: handle.read(1024 * 1024), b''):
13             h.update(block)
14     return h.hexdigest()
15
16
17 def main():
18     manifest = json.loads((ROOT/'package-manifest.json').read_text())
19     errors = []
20     for record in manifest['files']:
21         path = ROOT/record['path']
22         if not path.is_file():
23             errors.append({'path': record['path'], 'error': 'missing'})
24         elif path.stat().st_size != record['bytes'] or digest(path) != record['sha256']:
25             errors.append({'path': record['path'], 'error': 'size or hash mismatch'})
26     print(json.dumps({'checked': len(manifest['files']), 'errors': errors,
27                      'scope': 'File integrity only; not physical validation'}, ensure_ascii=False,
28                      indent=2))
29
30     raise SystemExit(bool(errors))
31
32 if __name__ == '__main__':
33     main()
```

F.3 assemble_data.py

类别：统一入口与结果导出。原始代码共 117 行。

```
1 """Assemble computed fields into nine template workbooks and comparison data.
2
3 No PDE integration, refitting, extrapolation beyond strict stops, or rounding.
4 """
5 from pathlib import Path
6 import csv,json,hashlib,sys
7 import numpy as np
8
```

```

9  ROOT=Path(__file__).resolve().parents[1]
10 OUT=ROOT/'integration'
11
12 def load_json(p):return json.loads(p.read_text())
13 def clean(value):
14     if isinstance(value,np.ndarray):return clean(value.tolist())
15     if isinstance(value,(np.floating,float)):return None if not np.isfinite(value) else
        float(value)
16     if isinstance(value,np.integer):return int(value)
17     if isinstance(value,dict):return {k:clean(v) for k,v in value.items()}
18     if isinstance(value,(list,tuple)):return [clean(v) for v in value]
19     return value
20 def write_csv(p,rows):
21     p.parent.mkdir(parents=True,exist_ok=True)
22     with p.open('w',newline='',encoding='utf-8-sig') as f:csv.writer(f).writerows(clean(rows))
23
24 def main():
25     books=[];summary=[];series=[];bindings=[];tables={}
26     for dim in ['1d','2d']:
27         for q in [1,2,3,4]:
28             sourceq='23' if q in [2,3] else str(q)
29             case=f'q{sourceq}-{dim}-main';prefix=ROOT/'baseline/results'/case
30             needed=['times_s','radii_cm','temperature_mid_C','moisture_mid','surface_mid_TC','R_m',
                'L_m','Cmax','density_dry_mass_kg']
31             with np.load(prefix.with_suffix('.npz')) as archive:a={k:archive[k] for k in needed}
32             d=load_json(prefix.with_suffix('.json'))
33             t=a['times_s'];mask=t>0
34             if q==1:mask&=t<=1800
35             if q==2:mask&=t<=10800
36             if q>=3:mask&=((np.mod(t,60)==0)|(t==d['strict_stop_s']))
37             ts=t[mask];radii=a['radii_cm']
38             channels=[('温度','temperature_mid_C',0),('水分浓度','moisture_mid',1)] if q<=2 else
                [('Sheet1','moisture_mid',1)]
39             sheets=[]
40             for title,key,index in channels:
41                 vals=a[key][mask]
42                 if q==4:vals=np.column_stack((vals[:, :20],a['surface_mid_TC'][mask,index]));head_e_
                    r=['时间\\到药材中心的距离',*radii[:20].tolist(), '药材表面']
43                 else:header=['时间\\到药材中心的距离',*radii.tolist()]
44                 rows=[header,*np.column_stack((ts,vals)).tolist()]
45                 sheets.append({'name':title,'rows':rows,'kind':'data'})
46                 write_csv(ROOT/f'workbooks/{dim}/result{q}-{title}.csv',rows)
47             geom=np.column_stack((ts,a['R_m'][mask]*100,a['L_m'][mask]*100,a['Cmax'][mask]))
48             sheets.append({'name':'几何与判据','kind':'data','rows':[['时间/s','当前半径/cm','当
                前长度/cm','全域最大干基含水率'],*geom.tolist()]})
49             label='一维端面面积折算近似' if dim=='1d' else '二维轴对称显式端面'

```

```

50     notes=[['项目','说明'],['模型',label],['取样','一维径向近似值' if dim=='1d' else
    '二维长度中截面径向值; 达标检查完整二维场'],['潜热','不计;
    原经验含水率差边界'],['单位','时间s; 距离cm; 温度°C;
    含水率kg水/kg干物质'],['显示精度','显示四位小数;
    底层数据保持双精度'],['空白','对应物理位置已在当前药材外,
    不表示含水率为0'],['物性','附录2' if q==1 else '附录3' if q in [2,3] else
    '附录4'],['几何','固定原始尺寸' if q<4 else
    '附件半径与等长径比仿射收缩'],['数据来源','题干及对应附件; 数值为模型计算,
    非实测'],['环境延拓','4小时后按末一小时均值, 属于模型假设']]
51     if q>=3:notes.extend(['严格停止时间/s',d['strict_stop_s']],['停止时全域最大C (未舍入)
    ',d['strict_stop_Cmax']],['达标规则','全部计算节点C<0.15; 不是平均值达标']))
52     sheets.append({'name':'模型说明','kind':'note','rows':notes})
53     books.append({'id':f'{dim}-q{q}','path':f'workbooks/{dim}/result{q}.xlsx','sheets':sh
    eets})
54     # Paper table follows required times/radii, with full precision retained.
55     paper_times=[100,300,600,900,1200,1500,1800] if q==1 else
    [1800,3600,5400,7200,9000,10800] if q==2 else
    [*np.arange(21600,d['strict_stop_s'],21600),d['strict_stop_s']]
56     pts=[]
57     for tt in paper_times:
58         idx=np.flatnonzero(np.isclose(t,tt,rtol=0,atol=1e-7));assert len(idx)==1,(case,tt)
59         i=idx[0];idxr=[0,5,10,15,20] if q<4 else [0,5,10,15]
60         pt={'time_s':float(tt),'radius_cm':float(a['R_m'][i]*100),'T':a['temperature_mid_
    C'][i,idxr].tolist(),'C':a['moisture_mid'][i,idxr].tolist()}
61         if q==4:pt['T'].append(float(a['surface_mid_TC'][i,0]));pt['C'].append(float(a['s
    urface_mid_TC'][i,1]))
62         pts.append(pt)
63     tables[f'{dim}-q{q}']=pts
64     if q!=2:
65         summary.append({'model':dim,'case':case,'question':f'2/3' if q==3 else str(q),'cri
    tical_h':d['critical_h'],'strict_stop_s':d['strict_stop_s'],'scalar_balance':d['
    scalar_mean_balance_max_8'],'dry_mass_relative_drift':d['density_dry_mass_final_
    relative_change'],'source':str(prefix.with_suffix('.json').relative_to(ROOT))})
66     for key in [prefix.with_suffix('.json'),prefix.with_suffix('.npz')]:bindings.appe
    nd({'path':str(key.relative_to(ROOT)),'sha256':hashlib.sha256(key.read_bytes()).
    hexdigest()})
67     sel=np.unique(np.r_[np.arange(0,len(t),max(1,len(t)//1600)),len(t)-1])
68     for i in sel:series.append([case,t[i]/3600,a['Cmax'][i],a['R_m'][i]*100,a['L_m'][
    i]*100,a['density_dry_mass_kg'][i]/a['density_dry_mass_kg'][0]])
69     if '--baseline-only' in sys.argv:
70         (OUT/'workbook-payload.json').write_text(json.dumps(clean({'workbooks':books}),ensure_asc
    ii=False,allow_nan=False))
71         (OUT/'paper-tables.json').write_text(json.dumps(clean(tables),ensure_ascii=False,indent=2
    ,allow_nan=False))
72         print(json.dumps({'baseline_workbooks':len(books)},ensure_ascii=False));return
73     control=ROOT/'baseline/results/q4-2d-fixed-control'
74     cd=load_json(control.with_suffix('.json'))

```

```

75     with np.load(control.with_suffix('.npz')) as archive:ca={k:archive[k] for k in
    ['times_s','Cmax','R_m','L_m','density_dry_mass_kg']}
76     summary.append({'model':'2d-fixed-control','case':'q4-2d-fixed-control','question':'4-control',
    'critical_h':cd['critical_h'],'strict_stop_s':cd['strict_stop_s'],'Cmax_end':float(ca['Cmax_
    x'][-1]),'horizon_h':float(ca['times_s'][-1]/3600),'source':str(control.with_suffix('.json')
    .relative_to(ROOT))})
77     for p in [control.with_suffix('.json'),control.with_suffix('.npz')]:bindings.append({'path':s,
    tr(p.relative_to(ROOT)),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()})
78     for i in range(len(ca['times_s'])):series.append(['q4-2d-fixed-control',ca['times_s'][i]/3600,
    ,ca['Cmax'][i],ca['R_m'][i]*100,ca['L_m'][i]*100,ca['density_dry_mass_kg'][i]/ca['density_dr
    y_mass_kg'][0]])
79     # Free-length result selection is numerical resolution, never agreement with an expected time.
80     candidates=[]
81     for p in (ROOT/'variable-length/results').glob('*summary.json'):
82         d=load_json(p)
83         if d.get('strict_stop_verified'):candidates.append((d['config']['nr'],d['config']['nz'],-
            d['config']['rtol'],p,d))
84     if not candidates:raise ValueError('No completed variable-length result')
85     _,_,vp,vd=max(candidates,key=lambda v:v[:3]);vdir=vp.parent
86     assert vd['config']['nr']>=32,'Wait for final fine grid'
87     va=np.load(vdir/'fields.npz');cols=list(va['record_columns']);rec=va['records'];stop=vd['stri
    ct_stop_s']
88     radii=np.arange(21)/10;vrows=[];vgeom=[]
89     delivery=ROOT/'variable-length/delivery'/vdir.name
90     # Nonaffine geometry needs exact inversion to the finite-element coordinates.
91     # The worker's audited export performs that inversion, including the strict stop.
92     with (delivery/'midplane.csv').open(encoding='utf-8-sig') as f:mid=list(csv.DictReader(f))
93     with (delivery/'surface.csv').open(encoding='utf-8-sig') as f:sur={float(x['time_s']):x for x
    in csv.DictReader(f)}
94     bytime={}
95     for row in
        mid:bytime.setdefault(float(row['time_s']),{})[round(float(row['radius_cm']),6)]=row
96     for t,spatial in sorted(bytime.items()):
97         if t<=0 or t>stop or not (t%60==0 or t==stop):continue
98         s=sur[t];cv=[None if spatial[round(float(r),6)]['C_kg_per_kgdry']==' ' else
            float(spatial[round(float(r),6)]['C_kg_per_kgdry']) for r in radii[:20]]
99         vrows.append([t,*cv,float(s['side_midplane_C'])])
100        where=np.flatnonzero(np.isclose(va['times'],t,rtol=0,atol=1e-8))
101        maxc=vd['strict_stop_Cmax'] if t==stop else float(rec[where[0],cols.index('Cmax')])
102        vgeom.append([t,float(s['current_radius_cm']),float(s['length_cm']),maxc])
103    assert vrows[-1][0]==stop

```

```

104     vnotes=[['项目','说明'],['模型','二维、题给湿密度、干质量守恒、Map
A变长度'],['取样','长度中截面；全域最大含水率判定达标'],['潜热','不计；原经验边界；
保留水迁移显焓'],['附加假设','内部圆柱壳保持圆柱面，轴向材料重分布'],['密度与热容','采用附录4；
局部热容量仍为题给pcp'],['单位','时间s；距离cm；
含水率kg水/kg干物质'],['空白','该固定位置在药材外；
当前表面另列'],['严格停止时间/s',stop],['停止全域最大C（未舍入）
',vd['strict_stop_Cmax']],['长度解释','由密度、含水率和附件半径计算；
不是实测验证'],['物理限制','未解决真实蒸发能量与材料力学闭合；条件计算']]
105     books.append({'id':'variable-q4','path':'workbooks/variable-length/result4.xlsx','sheets':[{
name':'Sheet1','kind':'data','rows':[['时间\\到药材中心的距离',*radii[:20], '药材表面'],*vrows
]],{'name':'几何与判据','kind':'data','rows':[['时间/s','当前半径/cm','当前长度/cm','全域最大
干基含水率'],*vgeom]],{'name':'模型说明','kind':'note','rows':vnotes}}])
106     summary.append({'model':'variable-length','case':vdir.name,'question':'4','critical_h':vd['cr
itical_h'],'strict_stop_s':stop,'mass_balance':vd['mass_balance_relative'],'energy_balance':
vd['enthalpy_balance_relative'],'dry_mass_relative_drift':vd['checks_max']['nodal_dry_mass_r
elative_error'],'source':str(vp.relative_to(ROOT))})
107     mdratio=float(va['lumped_dry_mass'].sum()/vd['dry_mass_kg'])
108     for x in rec:series.append([vdir.name,x[0]/3600,x[cols.index('Cmax')],x[1]*100,x[cols.index('
length_m')]*100,mdratio])
109     bindings.extend([{'path':str(p.relative_to(ROOT)),'sha256':hashlib.sha256(p.read_bytes()).hex
digest()} for p in [vp,vdir/'fields.npz',delivery/'midplane.csv',delivery/'surface.csv']])
110     (OUT/'selected-variable.json').write_text(json.dumps({'directory':str(vdir.relative_to(ROOT)),
'summary':vd},ensure_ascii=False,indent=2))
111     (OUT/'workbook-payload.json').write_text(json.dumps(clean({'workbooks':books}),ensure_ascii=F
alse,allow_nan=False))
112     (OUT/'computed-summary.json').write_text(json.dumps(clean({'models':summary,'sources':binding
s}),ensure_ascii=False,indent=2,allow_nan=False))
113     (OUT/'paper-tables.json').write_text(json.dumps(clean(tables),ensure_ascii=False,indent=2,all
ow_nan=False))
114     write_csv(OUT/'comparison-series.csv',[['case','time_h','Cmax','radius_cm','length_cm','Md_ra
tio'],*series])
115     print(json.dumps({'workbooks':len(books),'selected_variable':vdir.name,'source_files':len(bin
dings}),ensure_ascii=False))
116
117 if __name__=='__main__':main()

```

F.4 build_workbooks.mjs

类别：统一入口与结果导出。原始代码共 43 行。

```

1 import fs from 'node:fs/promises';
2 import path from 'node:path';
3 import {fileURLToPath} from 'node:url';
4 import {Workbook,SpreadsheetFile} from '@oai/artifact-tool';
5
6 const here=path.dirname(fileURLToPath(import.meta.url));
7 const root=path.dirname(here);
8 const input=JSON.parse(await fs.readFile(path.join(here,'workbook-payload.json'),'utf8'));
9 const only=process.argv[2];

```

```

10 for(const book of input.workbooks.filter(x=>!only||x.id===only)){
11   const out=path.join(root,book.path);await fs.mkdir(path.dirname(out),{recursive:true});
12   const wb=Workbook.create();
13   for(const sheet of book.sheets){
14     const sh=wb.worksheets.add(sheet.name), rows=sheet.rows, nc=rows[0].length;
15     const all=sh.getRangeByIndexes(0,0,rows.length,nc);all.values=rows;
16     all.format.font={name:'Arial',size:10};all.format.columnWidth=12;all.format.rowHeight=19;
17     sh.showGridLines=false;
18     if(sheet.kind==='note'){
19       sh.getRangeByIndexes(0,0,rows.length,1).format.columnWidth=24;
20       sh.getRangeByIndexes(0,1,rows.length,nc-1).format.columnWidth=100;
21     }else{
22       sh.getRangeByIndexes(1,1,rows.length-1,nc-1).setNumberFormat('0.0000');
23       sh.getRangeByIndexes(1,0,rows.length-1,1).setNumberFormat('0');
24       sh.getRangeByIndexes(0,0,1,1).format.columnWidth=32;
25       sh.freezePanels.freezeRows(1);sh.freezePanels.freezeColumns(1);
26     }
27     const header=sh.getRangeByIndexes(0,0,1,nc);
28     header.format.fill='#E9EDF2';header.format.font={name:'Arial',size:10,bold:true};header.format.rowHeight=27;
29   }
30   wb.recalculate();
31   const checks=[];
32   for(const sheet of book.sheets){
33     const range=sheet.kind==='note'? 'A1:B8': 'A1:F8';
34     checks.push({sheet:sheet.name,inspection:(await wb.inspect({kind:'table',range:`'${sheet.name}'!${range}`,tableMaxRows:8,tableMaxCols:6,maxChars:1000})).ndjson});
35     const blob=await wb.render({sheetName:sheet.name,range,range,format:'png'});
36     const prev=path.join(here,'workbook-previews','${book.id}-${sheet.name}.png');await
37       fs.mkdir(path.dirname(prev),{recursive:true});
38     await fs.writeFile(prev,new Uint8Array(await blob.arrayBuffer()));
39   }
40   const scan=await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#N/A|#NUM!|#NULL!',options:{useRegex:true,maxResults:20},maxChars:1000});
41   const xlsx=await SpreadsheetFile.exportXlsx(wb);await xlsx.save(out);
42   await fs.writeFile(out+'.inspect.json',JSON.stringify({checks,errorScan:scan.ndjson,null,2}));
43   console.log(JSON.stringify({id:book.id,path:book.path,sheets:book.sheets.map(s=>({name:s.name,rows:s.rows.length-1})))));
44 }

```

F.5 export_workbooks.py

类别：统一入口与结果导出。原始代码共 17 行。

```

1 """Export requested template books in isolated Node processes."""
2 from pathlib import Path
3 import subprocess,sys,json,concurrent.futures,shutil
4 ROOT=Path(__file__).resolve().parents[1]
5 NODE=Path('./local/.cache/codex-runtimes/codex-primary-runtime/dependencies/node/bin/node')

```

```

6 if not NODE.exists():NODE=Path(shutil.which('node') or 'node')
7 payload=json.loads((ROOT/'integration/workbook-payload.json').read_text())
8 cases=[b for b in payload['workbooks'] if not (ROOT/b['path']).exists()]
9 def run(book):
10     p=subprocess.run([str(NODE),'build_workbooks.mjs',book['id']],cwd=ROOT/'integration',capture_
        output=True,text=True)
11     log=ROOT/'integration/workbook-build-logs';log.mkdir(exist_ok=True)
12     (log/f"{book['id']}.log").write_text(p.stdout+'\n'+p.stderr)
13     if p.returncode:raise RuntimeError(book['id']+'\n'+p.stderr[-2000:])
14     return {'id':book['id'],'path':book['path'],'status':'exported'}
15 with concurrent.futures.ThreadPoolExecutor(max_workers=2) as pool:
16     jobs=[pool.submit(run,b) for b in cases]
17     for job in concurrent.futures.as_completed(jobs):print(json.dumps(job.result(),ensure_ascii=F,
        else),flush=True)

```

F.6 package_model.py

类别：统一入口与结果导出。原始代码共 68 行。

```

1 """Package complete model evidence and a separate compact reading/table bundle."""
2 from pathlib import Path
3 from datetime import datetime, timezone
4 import hashlib
5 import json
6 import os
7 import zipfile
8
9 ROOT = Path(__file__).resolve().parents[1]
10
11
12 def digest(path):
13     h = hashlib.sha256()
14     with path.open('rb') as f:
15         for block in iter(lambda: f.read(1024*1024), b''):
16             h.update(block)
17     return h.hexdigest()
18
19
20 def main():
21     files = []
22     excluded = []
23     for base, directories, names in os.walk(ROOT, followlinks=False):
24         for name in list(directories):
25             path = Path(base)/name
26             if name in {'node_modules', '__pycache__', '.cache'} or path ==
                ROOT/'integration/build' or path.is_symlink():
27                 directories.remove(name)
28                 excluded.append(str(path.relative_to(ROOT))+ '/')
29         for name in names:

```

```

30         path = Path(base)/name
31         if name in {'package-manifest.json', '.DS_Store'} or name.endswith('.inspect.ndjson')
           or path.is_symlink():
32             excluded.append(str(path.relative_to(ROOT)))
33             continue
34         files.append(path)
35     files.sort()
36     records = [{'path':str(p.relative_to(ROOT)), 'bytes':p.stat().st_size, 'sha256':digest(p)} for
           p in files]
37     manifest = {'created_utc': datetime.now(timezone.utc).isoformat(), 'scope':'All scientific
           source, inputs, cases and delivery files; no claim of material validation',
38               'excluded_reason':'Runtime caches, authoring-only Excel cell dumps and duplicate
           LaTeX intermediates are omitted; numerical evidence is retained.',
39               'excluded': sorted(excluded), 'files': records}
40     (ROOT/'package-manifest.json').write_text(json.dumps(manifest, ensure_ascii=False,
           indent=2)+'\n')
41     complete = ROOT.parent/'模型系统 _ 完整代码与数据 _20260912.zip'
42     compact = ROOT.parent/'模型系统 _ 结果表与说明 _20260912.zip'
43     compressed_extensions = {'.npz', '.xlsx', '.pdf', '.png', '.jpg', '.zip'}
44     for target, selected in [
45         (complete, files+[ROOT/'package-manifest.json']),
46         (compact, [p for p in files if (p.parent == ROOT and p.suffix in {'.md','.pdf'}) or
           (p.parent.parent.name == 'workbooks' and p.suffix == '.xlsx')]),
47     ]:
48         with zipfile.ZipFile(target, 'w', allowZip64=True) as z:
49             for path in selected:
50                 typ=zipfile.ZIP_STORED if path.suffix.lower() in compressed_extensions else
                    zipfile.ZIP_DEFLATED
51                 z.write(path, arcname=str(Path(ROOT.name)/path.relative_to(ROOT)),
                    compress_type=typ, compresslevel=None if typ == zipfile.ZIP_STORED else 6)
52             if target == compact:
53                 z.writestr(str(Path(ROOT.name)/'先读我.txt'),'这个小包包含模型说明和9本结果Excel,
                    供直接阅读。完整代码、CSV、NPZ计算场与审查证据在
                    “模型系统_完整代码与数据_20260912.zip”；README中的复算命令应在完整包中执行。\\n')
54         with zipfile.ZipFile(target) as z:
55             bad=z.testzip()
56             if bad:raise RuntimeError(f'ZIP CRC failed: {bad}')
57             if target == complete:
58                 for record in records:
59                     arc=str(Path(ROOT.name)/record['path'])
60                     h=hashlib.sha256()
61                     with z.open(arc) as f:
62                         for block in iter(lambda:f.read(1024*1024),b''):h.update(block)
63                     if h.hexdigest()!=record['sha256']:raise RuntimeError(f'ZIP SHA-256 mismatch:
                        {arc}')
64     print(json.dumps({'path':str(target), 'bytes':target.stat().st_size, 'files':len(selected)+(,
           target==compact), 'sha256':digest(target), 'verified':True}, ensure_ascii=False), flush=True)

```

```

65
66
67 if __name__ == '__main__':
68     main()

```

F.7 plot_system.py

类别：统一入口与结果导出。原始代码共 74 行。

```

1  """Quantitative figures with explicit model and data scopes."""
2  import os,sys,json,hashlib,csv
3  from pathlib import Path
4  ROOT=Path(__file__).resolve().parents[1]
5  os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'integration/.cache/mpl'))
6  os.environ.setdefault('XDG_CACHE_HOME',str(ROOT/'integration/.cache'))
7  SUPPORT=ROOT/'latent-branch/support'
8  sys.path[:0]=[str(SUPPORT/'lab-ultra-scientific-figure-maker/scripts'),str(SUPPORT/'lab-ultra-sc_
   scientific-visualization/scripts')]
9  from figure_library import publication_style
10 from figure_export import export_figure
11 from figure_typography import inspect_figure_typography
12 import numpy as np
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16
17 DEST=ROOT/'figures';DEST.mkdir(exist_ok=True)
18 def h(p):return hashlib.sha256(p.read_bytes()).hexdigest()
19 def export(fig,fid,claim,materials,paths,encodings):
20     sources=[{'path':str(p.relative_to(ROOT)), 'sha256':h(p)} for p in paths]
21     req={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'audience':'Modeler comparing
   conditional drying models','medium':'A4 model system report','final_width_mm':160,'language':
   'zh-CN','data_sources':sources,'variables': [{'name':'time','unit':'h','role':'independent
   variable'},{'name':'C','unit':'kg/kg dry matter','role':'computed
   state'}], 'uncertainty':{'kind':'deterministic numerical cases; no confidence
   intervals'}, 'editable_format':'svg', 'claim_locked':True}
22     ir={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'route':'quantitative_code','recip_
   e':'physical-coordinate fields and computed
   trajectories','panels':[{'panel_id':str(i+1),'purpose':ax.get_title()} for i,ax in enumerate(
   fig.axes)], 'encodings':encodings,'annotations':[], 'style_tokens':{'palette':'okabe-ito and
   viridis','base_font_pt':8.5}, 'exports':['svg','pdf','png'], 'invariants':['data','units','cla_
   im','transform','uncertainty'],'provenance':{'renderer':'matplotlib','materials':materials,'
   data_sources':sources}}
23     for ext,obj in
24         [ ('request',req), ('ir',ir), ('typography',inspect_figure_typography(fig,final_width_mm=160))]:
25         (DEST/f'{fid}.{ext}.json').write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=
   str))

```

```

25     export_figure(fig,DEST/fid,formats=['svg','pdf','png'],mkdir=True,overwrite=True,dpi=220,prov_
    enance={'data_sources':sources,'materials':materials},write_manifest=True,accessibility={'ti_
    tle':fid,'desc':claim})
26     plt.close(fig)
27
28     def main():
29         p=R00T/'integration/comparison-series.csv'
30         with p.open(encoding='utf-8-sig') as f:rows=list(csv.DictReader(f))
31         groups={}
32         for r in rows:groups.setdefault(r['case'],[]).append(r)
33         def xy(case,key):
34             return np.array([float(r['time_h']) for r in groups[case]],np.array([float(r[key]) for r
    in groups[case]])
35         blue='#0072B2';orange='#D55E00';green='#009E73'
36         with publication_style('nature',language='zh-CN',width_mm=160,height_mm=116) as mat:
37             fig,axes=plt.subplots(2,1,layout='constrained')
38             for ax,q,title in [(axes[0],'23','问题三：固定尺寸、相同附录3物性'),(axes[1],'4','问题四：
    相同附录4物性')]:
39                 for dim,color,ls,label in
                    [(('1d',blue,'--','一维端面折算'),('2d',orange,'-','二维显式端面'))]:
40                     ax.plot(*xy(f'q{q}-{dim}-main','Cmax'),color=color,ls=ls,label=label)
41                     if q=='4':ax.plot(*xy('q4-2d-fixed-control','Cmax'),color=green,ls='-',label='二维固
    定尺寸对照')
42                     ax.axhline(.15,color='#666666',lw=.8,ls='-.');ax.set(xlabel='时间 /
    h',ylabel='全域最大含水率 / (kg/kg)',ylim=(0,2.65),title=title)
43                     ax.legend(frameon=False,fontsize=7.5);ax.grid(alpha=.18)
44             export(fig,'dimension-comparison','Dimensional comparisons share empirical physics; Q4
    fixed-size control shares Appendix4 properties. Curves stop at recorded horizons, with no
    extrapolation.',mat,[p],[{'field':'time_h','channel':'x','scale':'linear'},{'field':'Cma_
    x','channel':'y','scale':'linear'},{'field':'case','channel':'color','scale':'categorical'
    }])
45
46     data=R00T/'baseline/results/q4-2d-main.npz';a=np.load(data)
47     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=90) as mat:
48         fig,axes=plt.subplots(2,1,layout='constrained')
49         first=np.flatnonzero(np.isclose(a['times_s'],21600,rtol=0,atol=1e-7))[0]
50         limits=(a['L_m'][first]*50,a['R_m'][first]*100)
51         for ax,t in zip(axes,[21600,86400]):
52             ii=np.flatnonzero(np.isclose(a['snapshot_times_s'],t,rtol=0,atol=1e-7));assert
    len(ii)==1
53             jj=np.flatnonzero(np.isclose(a['times_s'],t,rtol=0,atol=1e-7));assert len(jj)==1
54             field=a['snapshot_fields_TC'][ii[0],:,:,1]
55             r=a['x']*a['R_m'][jj[0]]*100;z=a['eta']*a['L_m'][jj[0]]*50
56             im=ax.contourf(z,r,field,levels=np.linspace(0,2.55,129),vmin=0,vmax=2.55,cmap='viridis')
57             im.set_edgecolor('face');im.set_linewidth(.1)
58             if field.min()<.15<field.max():ax.contour(z,r,field,levels=[.15],colors='white',linew_
    idths=.7)

```

```

59         ax.set(xlabel='距长度中面 / cm',ylabel='距中心轴 / cm',title=f'{t/3600:g} h:
           中面至一端、轴线至侧面',xlim=(0,limits[0]),ylim=(0,limits[1]));ax.set_aspect('equal',
           adjustable='box')
60     fig.colorbar(im,ax=list(axes),shrink=.8,label='干基含水率 /
           (kg/kg)',ticks=[0,.5,1,1.5,2,2.55])
61     export(fig,'q4-spatial-fields','Physical-coordinate Q4 affine 2D fields show axial/radial
           gradients; plotted body subset is half-length and positive radius. Same color scale at
           both times.',mat,[data],[{'field':'z_cm','channel':'x','scale':'linear'},{'field':'r_cm',
           'channel':'y','scale':'linear'},{'field':'C','channel':'color','scale':'linear'}])
62
63     vp=json.loads((ROOT/'integration/selected-variable.json').read_text());case=Path(vp['director',
           y]).name
64     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=111) as mat:
65         fig,axes=plt.subplots(2,1,layout='constrained')
66         for c,color,label in [('q4-2d-main',orange,'等长径比经验模型'),(case,blue,'质量—
           体积约束变长度模型')]:
67             axes[0].plot(*xy(c,'length_cm'),color=color,label=label)
68             axes[1].plot(*xy(c,'Md_ratio'),color=color,label=label)
69             axes[0].axhline(25,color='#777777',ls=':',lw=.8,label='初始长度');axes[0].set(ylabel='模
           型长度 / cm',title='长度变化由所选约束决定, 均非长度实测')
70             axes[1].axhline(1,color='#777777',ls=':',lw=.8);axes[1].set(xlabel='时间 /
           h',ylabel='干质量 / 初始干质量',title='经验模型按密度积分的诊断, 与守恒模型对照')
71             for ax in axes:ax.legend(frameon=False,fontsize=7.3);ax.grid(alpha=.18);ax.set_xlim(0,72)
72             export(fig,'q4-geometry-mass','Map A enforces dry-mass/density/volume relations; affine
           empirical model does not. Difference includes mass operator, internal kinematics and
           sensible enthalpy, not an isolated length
           effect.',mat,[p,ROOT/'integration/selected-variable.json'],[{'field':'time_h','channel':
           'x','scale':'linear'},{'field':'length_cm,Md_ratio','channel':'y','scale':'linear'},{'fi
           eld':'case','channel':'color','scale':'categorical'}])
73
74     if __name__=='__main__':main()

```

F.8 write_report.py

类别: 统一入口与结果导出。原始代码共 193 行。

```

1  """Write a compact model-system report from completed numerical outputs."""
2  from pathlib import Path
3  import json, csv, math
4  ROOT=Path(__file__).resolve().parents[1]
5  def load(p):return json.loads(p.read_text())
6  def fmt(x,n=4):return '—' if x is None else f'{x:.{n}f}'
7  def table(rows,head):return '| '+' | '.join(head)+' |\n| '+' | '.join(['---']*len(head))+
           |\n|'+'\n'.join('| '+' | '.join(map(str,row))+ ' |' for row in rows)+'\n'
8  def main():
9       summary=load(ROOT/'integration/computed-summary.json')['models'];by={x['case']:x for x in
           summary}
10      vd=load(ROOT/'integration/selected-variable.json')['summary'];tabs=load(ROOT/'integration/pap
           er-tables.json')

```

```

11     audit=load(R00T/'baseline/numerical-summary.json')
12     errtable=table([['q1':'问题1','q23':'问题2/3','q4':'问题4'][x['problem']],fmt(x['sampled_midplane_max_T_error_C']),fmt(x['sampled_midplane_max_C_error']),fmt(x['sampled_axialmean_max_C_error'])]) for x in
    audit['dimensions']],['对照','中面温差/°C','中面含水率差','轴向平均含水率差'])
13     gridmax=max(abs(x['critical_change_s']) for x in audit['grid_refinement'] if
    x['critical_change_s'] is not None)
14     timemax=max(abs(x['critical_change_s']) for x in audit['time_refinement'] if
    x['critical_change_s'] is not None)
15     base=R00T/'baseline/results'
16     ds={c:load(base/(c+'.json')) for c in ['q1-1d-main','q1-2d-main','q23-1d-main','q23-2d-main','q4-1d-main','q4-2d-main','q4-2d-fixed-control']}
17     c=ds['q4-2d-fixed-control']
18     control=f"{c['critical_h']:.4f} h" if c['critical_h'] is not None else
    f"在{c['sampled_until_s']/3600:g} h内未达标"
19     paired=[]
20     for q in ['23','4']:
21         a=ds[f'q{q}-1d-main']['critical_h'];b=ds[f'q{q}-2d-main']['critical_h']
22         paired.append(['第三问' if q=='23' else
            '第四问',fmt(a),fmt(b),fmt((a-b)*60,2),f'{{(a/b-1)*100:+.3f}}%'])
23     text=r'''---
24 title: 圆柱药材烘干模型系统与计算结果
25 subtitle: 一维近似、二维对照、质量—体积修正与潜热支线
26 date: 2026 年 9 月 12 日
27 lang: zh-CN
28 documentclass: ctexart
29 classoption:
30     - fontset=fandol
31     - zihao=5
32 geometry:
33     - margin=20mm
34 colorlinks: true
35 linkcolor: black
36 urlcolor: blue
37 ---
38
39 # 1. 模型层次与主要结果
40
41 一维与二维经验模型分别完成问题1—4，输出两套题目要求的Excel文件。二维结果用于检验降维近似；
    第四问另有保持干质量与密度—体积关系的变长度计算。潜热只保留为条件试验，不替换四问的经验计算结果。
42
43 | 模型 | 覆盖范围 | 主要作用 |
44 |---|---|---|
45 | 一维端面折算 | 问题 1—4 | 径向近似；把端面交换按面积并入侧面 |
46 | 二维显式端面 | 问题 1—4 | 直接求径向与轴向场，检验一维近似 |
47 | 二维变长度 | 问题 4 | 题给密度、干质量、附件半径共同约束体积与长度 |
48 | 二维潜热试验 | 问题 4 支线 | 检验汽化耗能与未知表面关系的影响 |

```

49

50 以下是全域含水率首次降至阈值的连续时间；Excel 未行使用首次满足严格不等式的整数秒。

51

52 '''

53 text+=table(paired,['问题','一维 / h','二维 / h','一维 - 二维 / min','相对差'])

54 text+=f'''

55 第四问变长度分支的阈值时间为 $\{vd['critical_h']:.4f\}$ h**；达标时半径

$\{100*vd['R_at_strict_stop_m']:.4f\}$ cm**、长度 $\{100*vd['L_at_strict_stop_m']:.4f\}$ cm**。

全程峰值长度为 $\{100*vd['L_max_m']:.3f\}$ cm。这是所选运动学与守恒关系下的计算，不是轴向伸长实测。

56

57 为分离尺寸与物性的作用，另用附录4做二维固定尺寸对照，结果为 $\{control\}$ **。

第三问和第四问物性公式不同，不能把两问时长差全部归因于收缩。

58

59 **精度与物理含义分开报告。** 基础模型的标量方程数值收支通过，不意味着真实干物质质量守恒；

变长度模型修复相应质量—体积约束，但不因此成为已验证的完整蒸发与力学模型。

60

61 \\newpage

62

63 # 2. 方程、假设与数值方法

64

65 温度为 T ，材料干基含水率为 C 。 $D_t=\partial_t+\mathbf{v}\cdot\nabla$ 表示随材料运动的导数；

固定域中为 ∂_t 。基础经验系统为

66

67 \$\$

68 $\rho\ c_p D_t T = \nabla\cdot(k\nabla T), \quad$

69 $D_t C = \nabla\cdot(D\nabla C). \quad \tag{1}$

70 \$\$

71

72 一维采用圆柱径向算子 $r^{-1}\partial_r(rf_r)$ ；二维增加 $\partial_z f_z$ 。

中心轴与长度中面取对称边界；表面使用

73

74 \$\$

75 $-k\partial_n T = h(T_s - T_a), \quad$

76 $-D\partial_n C = h_m(C_s - w_a). \quad \tag{2}$

77 \$\$

78

79 $h=25\text{ W/(m}^2\cdot\text{K)}$ 、 $h_m=8\times 10^{-7}\text{ m/s}$ ；式（2）的含水率差是声明的经验近似，题目仅给系数，

没有完整写出这条边界。这里 w_a 称附件空气水分指标。

80

81 一维把两端面积折到侧面，令 $\gamma=1+R/L$ ，两种交换系数同时乘 γ ；二维显式处理端面，

不重复折算。固定几何和等长径比候选均有 $\gamma=1.08$ 。一维近似并不强制长径比不变。

82

83 各问初态均匀，温度 28°C 、 $C=2.55$ 、 $R_0=2\text{ cm}$ 、 $L_0=25\text{ cm}$ ；采用轴对称和中面对称，

各暴露面受同一环境及交换系数作用。问题1用附录2，问题2、3用附录3并共用同一初态轨迹；

问题4用附录4及附件2。第四问基础收缩候选采用

84

85 \$\$

86 $R(t)=R_{\{\text{obs}\}}(t), \quad L(t)=L_0 R(t)/R_0,$

87 $r=xR(t), \quad z=\eta L(t)/2.$ $\tag{3}$

88 \$\$

89

90 内部计算采用SI单位与双精度。附件1在0—4 h线性插值，之后延拓为末一小时均值；这不是后续实测环境。前三问不读取附件2，第二问不接第一问末态后重新计时。

91

92 一维与二维基础空间离散采用圆柱控制体积，温湿方程用隐式BDF联立推进；
 $D(C,T)$ 和其他物性在每次非线性求值中更新。水分通量采用Kirchhoff原函数以处理低含水率处强变系数。主要径向分辨率128，二维轴向64，并做64/128及时间收紧检验。

93

94 变长度模型采用材料坐标有限元；主要分辨率32×16，另有12×6、24×12及时间精度对照。其均温极限下水分变化与显焓储存的抵消也经过检验。各模型均检查全场含水率、严格达标状态、越界位置 and 对应数值收支。

95

96 $\backslash\backslash\text{newpage}$

97

98 # 3. 一维近似究竟改动了什么

99

100 ! [同一经验物理条件下的维度对照。第四问固定尺寸控制使用相同附录4物性；各曲线只画到实际保存时段。
 $(\text{figures/dimension-comparison.png})\{\{\text{fig-dim width}=160\text{mm}\}\}$

101

102 面积折算把端面的交换分摊到整个侧面。它保留均匀状态下的总交换量，却不能保留所有非均匀场细节。端面干燥首先影响端部，一维折算也会影响整根的径向中心，因此必须分别检查平均量、题定中截面和全域最大值。

103

104 本组参数下，一维阈值时间比二维早，差值见第1页。这个差值是所选降维近似的误差指标，二维也只是同一经验模型的数值参照，不是实测真值。各版Excel明确标识模型来源；没有将两种剖面的代表性混写。

105

106 所比较共同时刻和采样位置上的最大绝对差如下。第四问的轴向平均含水率差并不比中面差更小，不能声称面积折算对所有平均指标都更准确。

107

108 {errtable}

109

110 基础模型64至128径向分辨率的阈值变化最大{gridmax:.3f} s；
 二维64网格上时间收紧的变化最大{timemax:.4f} s，均小于上述维度差。这些检验不是实测精度或置信区间。

111

112 $\backslash\backslash\text{newpage}$

113

114 # 4. 二维场说明端面为何不能简单忽略

115

116 ! [第四问等长径比二维场。坐标为当前物理尺寸，画出长度中面至一端、中心轴至侧面的半剖面；
 两图使用同一颜色范围，白线为C=0.15。](figures/q4-spatial-fields.png){\{\text{fig-field width}=160\text{mm}\}}

117

118 侧面与端面均交换水分，靠近外边界的区域先干，内部水分向外迁移。温度和含水率通过当前物性相互影响；两条PDE的形式不同不表示独立求解。

119

120 题目表格采用二维长度中截面的径向剖面，全域达标检查包含端部和内部所有计算节点。
另存的轴向平均剖面用于同口径评估一维面积折算；不能以它替代中截面后仍称为同一输出。

121

122 第四问每个时刻都更新半径、长度、空间导数尺度、端面和侧面交换。 $r > R(t)$ 的固定位置不在药材内，
输出留空，并另列当前表面值。不存在把移出材料的位置填零再参与最大值判断的操作。

123

124 `\\newpage`

125

126 # 5. 质量—体积诊断与变长度计算

127

128 ![两种第四问模型的长度和干质量诊断。变长度曲线是守恒约束下的条件计算；基础曲线止于达标，未向后外推。
](figures/q4-geometry-mass.png){#fig-geom width=160mm}}

129

130 基础模型把 $\rho(C)c_p(C)$ 用于储热，却没有同时强制 $M_d = \int \rho(C)/(1+C) dV$ 恒定。二维第三问，
末态的这个积分约为初始的`{1+ds['q23-2d-main']]['density_dry_mass_final_relative_change']:.4f}`倍；
第四问等长径比末态约为`{1+ds['q4-2d-main']]['density_dry_mass_final_relative_change']:.4f}`倍。
这是精确湿密度、所选几何与旧经验水分方程不能同时满足干质量守恒的诊断，不证明密度或半径单独错误。

131

132 第四问变长度分支令 $\rho_d = \rho/(1+C)$ 、 $\mathbf{j} = -\rho_d \nabla C$ ，
联立 $\rho_d \mathbf{t} C = -\nabla \cdot \mathbf{j}$ ，采用已批准Map A内部变形。总长度由

133

134
$$L(t) = \frac{1}{\pi R(t)^2} \int \frac{1+C}{760+90C} dm_d. \quad \text{tag}{4}$$

135

136
$$H = (1850 + 4000C)(T - 28) + h_b = 4000(T - 28)$$
配套记账，局部热容量仍等于题给 ρc_p 。
相对基础二维，这个分支同时改变守恒水量算子、内部运动学及水迁移显焓项；
不能把全部时长差单独归因于长度变化。

137

138

139

140 变长度网格由24×12加密至32×16，阈值变化2.329分钟；
水量与显焓预算相对残差为 5.28×10^{-8} 和 1.05×10^{-7} 。
节点密度关系精确满足所定约束；积分点几何插值与非线性密度关系的最大差为0.3866%，较粗网格继续下降。
预算误差与这项空间插值误差不可混同。

141

142 `\\newpage`

143

144 # 6. 二维题定结果：问题 1

145

146 下列数值为无潜热二维经验模型的中截面结果。温度单位°C，含水率单位kg/kg干物质，径向位置单位cm。
完整逐秒表在二维result1.xlsx与result2.xlsx。

147

148 '''

149 for q in [1,2]:

150 if q==2: text += '\\newpage\\n\\n# 7. 二维题定结果：问题2\\n\\n温度单位°C，含水率单位kg/kg干物质；
五个径向位置单位cm。\\n\\n'

151 for channel, label in [('T', '温度'), ('C', '含水率')]:

```

152         rows=[]
153         for row in tabs[f'2d-q{q}']:
154             time=row['time_s'] if q==1 else row['time_s']/3600
155             rows.append([f'{time:g}',*[fmt(x) for x in row[channel]]])
156             text+=f'## 问题{q}: {label}\n\n'+table(rows,['时间/s' if q==1 else
                '时间/h','0','0.5','1','1.5','2'])+'\n'
157     text+=f'\n\nnewpage\n\n# 8. 二维题定结果: 问题 3、4\n\n'
158     for q in [3,4]:
159         rows=[f'x{time_s}/3600',*[f'x{c} for c in x['C']] for x in tabs[f'2d-q{q}']]
160         text+=f'## 问题{q}: 含水率\n\n'+table(rows,['时间/h','0 cm','0.5 cm','1 cm','1.5 cm','2
                cm' if q==3 else '当前表面'])+'\n'
161     text+=r'''
162 最后一行是严格达标整数秒换算的小时数，底层判据使用未舍入的全域最大含水率。表格显示0.1500不表示按
    “等于0.15”判定达标。第四问破折号对应当前材料之外的固定坐标。
163
164 一维近似和变长度第四问的完整结果分别放在其独立Excel目录，文件名均按题定result编号；
    不混在同一张表中冒充同一个模型。
165
166 \n\nnewpage
167
168 # 9. 潜热支线与适用范围
169
170 在二维守恒框架上进一步进行的潜热试验，让同一水分通量同时进入质量和能量边界。
    在三个已声明但未标定的表面参数条件下，成对开关汽化相关焓差使第四问时长增加8.29—12.91 h。
    此范围是三个条件算例的结果，不是实测区间或置信区间。
171
172 这组配对具有自己的材料表面平衡关系，其关闭版本不等于前述原始经验边界模型。
    原经验传质边界直接接潜热曾出现表面低于露点仍向外蒸发的方向不相容；
    简单把题给系数换成气相浓度差解释也没有解决问题。故潜热试验用于说明适用限制，不作为确定主答案。
    资料不足不证明潜热可以物理上忽略。
173
174 本模型体系的数值方程、结果与对照已分别保存；基础经验模型、质量—体积条件修正和真实材料验证仍需区分。
    没有通过强制匹配附件半径来声称表面参数已拟合，也没有将计算伸长称为观测事实。
175
176 ## 文件与复现
177
178 - `workbooks/1d/`与`workbooks/2d/`: 各含四问 Excel 及全精度 CSV。
179 - `workbooks/variable-length/`: 第四问变长度结果；不替换前两套文件。
180 - `baseline/`,`variable-length/`: 独立求解器、配置、全精度场、数值诊断及逐次执行记录。
181 - `latent-branch/`,`surface-audit/`: 已完成的潜热支线和表面边界核查，作为复用证据保留。
182 - `模型架构.md`: 完整方程与模型层次；`README.md`: 统一入口、文件说明与复现方法。
183
184 ## 来源
185

```

186 题干和附件由用户提供，各问按对应附录使用。潜热和表面关系讨论参照COMSOL的[气液界面交换说明](https://doc.comsol.com/6.4/doc/com.comsol.help.chem/chem_ug_chemsptrans.08.264.html)与[多孔介质水分理论](https://doc.comsol.com/6.4/doc/com.comsol.help.heat/heat_ug_theory.07.080.html);
水蒸气压独立复核采用[NIST水物性数据](<https://webbook.nist.gov/cgi/cbook.cgi?ID=C7732185&Mask=4&Type=ANTOINE&Plot=on>)。这些来源支持方程解释，不提供本题药材的实测表面参数。

187

188 AI参与模型整理、代码、实验和写作。独立子智能体审查与作者核查均不替代真实材料试验或竞赛团队人工审阅。

```
189 '''
190     (ROOT/'模型系统与计算结果.md').write_text(text)
191     (ROOT/'integration/report-print.md').write_text(text.replace('figures/dimension-comparison.png',
192     'figures/dimension-comparison.pdf').replace('figures/q4-spatial-fields.png', 'figures/q4-spatial-fields.pdf').replace('figures/q4-geometry-mass.png', 'figures/q4-geometry-mass.pdf'))
192     print('Report written from completed arrays; no handwritten experimental numbers except frozen latent summary.')
193 if __name__=='__main__':main()
```

F.9 baseline_engine.py

类别：一维与二维主计算。原始代码共 81 行。

```
1  """Specified no-latent empirical baseline; no density-mass closure is implied.
2
3  The original nonlinear FV equations are frozen in frozen_empirical_engine.py.
4  This subclass updates the axial geometry for Q4 and exposes diagnostics/samples.
5  """
6  from dataclasses import dataclass
7  import numpy as np
8  from frozen_empirical_engine import Config as FrozenConfig, DryingModel
9
10
11  @dataclass(frozen=True)
12  class Config(FrozenConfig):
13      dimension: int = 2
14      geometry: str = 'fixed'
15
16
17  class BaselineModel(DryingModel):
18      def __init__(self, cfg, env, radius):
19          super().__init__(cfg, env, radius)
20          if cfg.dimension not in (1, 2):
21              raise ValueError('dimension must be 1 or 2')
22          if (cfg.dimension == 1) != (cfg.nz == 0):
23              raise ValueError('1D requires nz=0; 2D requires nz>0')
24          if cfg.question != 4 and (cfg.geometry != 'fixed' or len(radius)):
25              raise ValueError('Q1-3 may not read radius observations or shrink')
26          if cfg.question == 4 and cfg.geometry not in ('constant_aspect', 'fixed'):
27              raise ValueError('Q4 permits aspect-ratio branch or fixed-geometry control')
28          self.initial_volume = np.pi * cfg.R0**2 * cfg.length
```

```

29     self.initial_density = float(self.density(np.array(cfg.C0)))
30     self.initial_dry_mass = self.initial_volume*self.initial_density/(1+cfg.C0)
31     self.initial_wet_mass = self.initial_volume*self.initial_density
32
33     def length(self, t, y=None):
34         c = self.cfg
35         return c.length if c.geometry == 'fixed' else c.length*self.radius(t,y)/c.R0
36
37     def area_factor(self,t,y=None):
38         return 1+self.radius(t,y)/self.length(t,y)
39
40     def density(self,C):
41         q=self.cfg.question
42         if q == 1: return np.full_like(C,820.,dtype=float)
43         if q in (2,3): return 650+128*C
44         return 760+90*C
45
46     def fluxes(self,t,y,R=None):
47         # This is a material affine mapping, not a bare rescaling of the last node.
48         # Both axial metric H^-2 and axial boundary H*h must use the current H.
49         self.H=self.length(t,y)/2
50         return super().fluxes(t,y,R)
51
52     def water_out(self,t,y):
53         self.H=self.length(t,y)/2
54         return super().water_out(t,y)
55
56     def state_summary(self,t,y):
57         a=self.physical_state(y); T=a[:, :, 0]; C=a[:, :, 1]
58         R=self.radius(t,y); L=self.length(t,y); V=np.pi*R*R*L
59         rho=self.density(C); rho_d=rho/(1+C)
60         Md=V*float(np.sum(self.weights*rho_d))
61         Mw=V*float(np.sum(self.weights*rho_d*C))
62         return {
63             'R_m':R, 'L_m':L, 'volume_m3':V, 'area_factor':1+R/L,
64             'Tmean':float(np.sum(self.weights*T)),
65             'Cmean':float(np.sum(self.weights*C)),
66             'Cmax':float(C.max()), 'Cmin':float(C.min()),
67             'density_dry_mass_kg':Md, 'density_water_mass_kg':Mw,
68             'density_wet_mass_kg':V*float(np.sum(self.weights*rho)),
69             'density_dry_mass_relative_change':Md/self.initial_dry_mass-1,
70         }
71
72     def differential_balance(self,t,y):
73         cap,hr,hz,mr,mz,R=self.fluxes(t,y)
74         dT=self.divergence(hr,hz,R)/cap
75         dC=self.divergence(mr,mz,R)
76         scalar_rate=float(np.sum(self.weights*dC))

```

```

77         scalar_out=float(self.water_out(t,y).sum())
78         heat_rate=float(np.sum(self.weights*cap*dT))
79         heat_boundary=float(2*hr[-1]@self.vz/R**2+2*self.vr@hz[:,-1]/self.H**2)
80         return np.array([t,scalar_rate,scalar_out,scalar_rate+scalar_out,
81                         heat_rate,heat_boundary,heat_rate-heat_boundary])

```

F.10 check_identities.py

类别：一维与二维主计算。原始代码共 50 行。

```

1  """Synthetic algebra/geometry checks; not an empirical material test."""
2  from pathlib import Path
3  import json
4  from dataclasses import replace
5  import numpy as np
6  from baseline_engine import Config,BaselineModel
7  from solve import load_sheet
8  ROOT=Path(__file__).resolve().parents[1]
9
10 def main():
11     fixture=json.loads((ROOT/'input/verification-fixtures.json').read_text())
12     env=load_sheet(ROOT/'input/附件 1.xlsx');radius=load_sheet(ROOT/'input/附件 2.xlsx')
13     closed=[];rates=[];fold=[]
14     for d in (1,2):
15         gamma=1.08 if d==1 else 1
16         c=Config(question=4,nr=16,nz=8 if d==2 else 0,dimension=d,
17                 geometry='constant_aspect',radius_mode='observed',h=0,hm=0,
18                 end_heat=d==2,end_mass=d==2,radial_power=2,axial_power=3)
19         m=BaselineModel(c,env,radius)
20         y=np.tile([fixture['uniform_state']['T_C'],fixture['uniform_state']['C']],m.size//2)
21         for t in fixture['test_times_s']:
22             rhs=m.rhs(t,y);e=float(np.max(np.abs(rhs)))
23             closed.append({'dimension':d,'t_s':t,'max_abs_uniform_noexchange_rhs':e,
24                           'weight_sum':float(m.weights.sum()),'area_factor':m.area_factor(t,y),
25                           'R_m':m.radius(t,y),'L_m':m.length(t,y)})
26             assert e<1e-13 and abs(m.weights.sum()-1)<1e-14
27             c=replace(c,h=25*gamma,hm=8e-7*gamma);m=BaselineModel(c,env,radius)
28             a=m.initial_state().reshape(m.shape)
29             a[:,0]=31+3*m.x[:,None]**2+2*m.z[None,:]**2
30             a[:,1]=1.7-.2*m.x[:,None]**2-.1*m.z[None,:]**2
31             for t in fixture['test_times_s']:
32                 v=m.differential_balance(t,a.ravel())
33                 rates.append({'dimension':d,'t_s':t,'scalar_rate_residual':float(v[3]),'heat_rate_res',
34                               'idual_W_m3':float(v[6])})
35                 assert abs(v[3])<1e-12 and abs(v[6])<1e-6
36             y=np.tile([fixture['uniform_state']['T_C'],fixture['uniform_state']['C']],m.size//2)
37             for t in (0.,7200.,21600.):
38                 v=m.differential_balance(t,y)

```

```

38         fold.append({'dimension':d,'t_s':t,'scalar_out':float(v[2]),'heat_boundary':float(v[5,
        ])}))
39     uniform_errors=[]
40     for t in (0.,7200.,21600.):
41         p=[a for a in fold if a['t_s']==t]
42         e={'t_s':t,'scalar_out_difference':p[0]['scalar_out']-p[1]['scalar_out'],
43           'heat_boundary_difference_W_m3':p[0]['heat_boundary']-p[1]['heat_boundary']}
44         uniform_errors.append(e);assert abs(e['scalar_out_difference'])<1e-14 and
            abs(e['heat_boundary_difference_W_m3'])<1e-9
45     result={'data_kind':'synthetic','noexchange_uniform_state':closed,'differential_balance':rate,
        s,
46         'fold_matches_uniform_total_exchange':uniform_errors,
47         'scope':'Verifies discrete algebra, factor at uniform fields and affine geometry update
            only; does not prove nonuniform field accuracy or material closure'}
48     with (ROOT/'evidence/identity-checks.json').open('x') as
        f:json.dump(result,f,ensure_ascii=False,indent=2)
49     print(json.dumps({'algebra_checks':'passed','cases':len(closed)+len(rates)+len(uniform_errors,
        )}))
50 if __name__=='__main__':main()

```

F.11 export_and_audit.py

类别：一维与二维主计算。原始代码共 145 行。

```

1  """Audit stored fields and produce template-ready CSV and numeric comparisons."""
2  from pathlib import Path
3  import json, csv
4  import numpy as np
5  ROOT=Path(__file__).resolve().parents[1]
6
7  def load(case,old=False):
8      folder=ROOT/('evidence' if old else 'results')
9      with np.load(folder/f'{case}.npz') as z:a={k:z[k] for k in z.files}
10     return a,json.loads((folder/f'{case}.json').read_text())
11
12 def write_csv(path,headers,rows):
13     path.parent.mkdir(parents=True,exist_ok=True)
14     with path.open('w',newline='') as f:
15         w=csv.writer(f);w.writerow(headers)
16         for row in rows:w.writerow(['' if isinstance(x,(float,np.floating)) and np.isnan(x) else x
            for x in row])
17
18 def old_field(a,t,field='middle'):
19     idx=np.where(np.abs(a['times_s']-t)<1e-7)[0]
20     if not len(idx):raise ValueError((t,'not saved in old evidence'))
21     return a[field][idx[0]]
22
23 def main():
24     cases=json.loads((ROOT/'configs/batch.json').read_text())['cases']

```

```

25     all_results={};audit=[]
26     for case in cases:
27         a,d=load(case);all_results[case]=(a,d)
28         t=a['times_s'];w=2*a['radial_volume_weights'][:,None]*a['axial_volume_weights'][None,:]
29         assert abs(w.sum()-1)<1e-13 and np.all(np.diff(t)>0)
30         fields=a['snapshot_fields_TC'];assert np.isfinite(fields).all() and
           fields[:, :, :, 1].min()>=0
31         assert np.isfinite(a['Cmax']).all() and np.isfinite(a['Cmean']).all()
32         recomputed=np.einsum('sij,ij->s',fields[:, :, :, 1],w)
33         samples=np.asarray([np.where(np.isclose(t,s,atol=1e-8,rtol=0))[0][0] for s in
           a['snapshot_times_s']])
34         mean_error=float(np.max(np.abs(a['Cmean'][samples]-recomputed)))
35         max_error=float(np.max(np.abs(a['Cmax'][samples]-fields[:, :, :, 1].max(axis=(1,2)))))
36         assert mean_error<1e-12 and max_error<1e-12
37         q=d['config']['question'];rho=(np.full_like(fields[:, :, :, 1],820.) if q==1 else
38             650+128*fields[:, :, :, 1] if q in (2,3) else 760+90*fields[:, :, :, 1])
39         md=np.einsum('sij,ij->s',rho/(1+fields[:, :, :, 1]),w)*a['volume_m3'][samples]
40         mw=np.einsum('sij,ij->s',rho*fields[:, :, :, 1]/(1+fields[:, :, :, 1]),w)*a['volume_m3'][samples]
41         md_err=float(np.max(np.abs(md-a['density_dry_mass_kg'][samples])))
42         mw_err=float(np.max(np.abs(mw-a['density_water_mass_kg'][samples])))
43         partition_error=float(np.max(np.abs(a['density_wet_mass_kg']-a['density_dry_mass_kg']-a['
           density_water_mass_kg'])))
44         assert md_err<1e-12 and mw_err<1e-12 and partition_error<1e-12
45         mask=a['radii_cm'][None,:]/100>a['R_m'][:,None]+1e-12
46         for key in
           ('temperature_mid_C','moisture_mid','temperature_axialmean_C','moisture_axialmean'):
47             assert np.array_equal(np.isnan(a[key]),mask)
48         if d['strict_stop_s'] is not None:
49             assert t[-1]==d['strict_stop_s'] and a['strict_stop_field_TC'][:, :, 1].max()<.15
50         if q!=4:assert d['radius_data_loaded'] is False
51         audit.append({'case_id':case,'mean_reconstruction_error':mean_error,'max_reconstruction_e
           rror':max_error,
52             'dry_mass_reconstruction_error_kg':md_err,'water_mass_reconstruction_error_kg':mw_err,
53             'wet_equals_dry_plus_water_max_error_kg':partition_error,'nan_mask_correct':True,
54             'strict_fullfield_check':d['strict_stop_s'] is not None or q==1 or
           d['threshold_status']=='not_reached',
55             'numerical_diagnostics':{k:d[k] for k in
           ('critical_h','strict_stop_s','strict_stop_Cmax','scalar_mean_balance_max_8',
56             'differential_heat_residual_max_W_m3','density_dry_mass_max_abs_relative_change',
           'density_dry_mass_final_relative_change')}})
57     headers=['time_s']+['r_{r:.1f}_cm' for r in np.arange(0,2.001,.1)]
58     for dim in (1,2):
59         out=R00T/'tables'/f'{dim}d'
60         for q,label in ((1,'q1'),(2,'q23'),(3,'q23'),(4,'q4')):
61             a,d=all_results[f'{label}-{dim}d-main'];t=a['times_s']

```

```

62     mask=t<=10800 if q==2 else
        (np.isclose(np.mod(t,60),0)|np.isclose(t,d['strict_stop_s'])) if q==3 else
        np.ones(len(t),bool)
63     for var in (['temperature','moisture'] if q in (1,2) else ['moisture']):
64         key='temperature_mid_C' if var=='temperature' else 'moisture_mid'
65         write_csv(out/f'result{q}-{var}.csv',headers,np.c_[t[mask],a[key][mask]])
66     if q==4:
67         write_csv(out/'result4-current-surface.csv',['time_s','radius_m','length_m','surf_
            ace_T_C','surface_C'],
68                 np.c_[t,a['R_m'],a['L_m'],a['surface_mid_TC']])
69     comparisons=[];old_reuse=[];grid=[];timecheck=[]
70     common_r=np.linspace(0,1,201)
71     for q,label in ((1,'q1'),(3,'q23')):
72         for dim,mode in ((1,'area_fold'),(2,'explicit_ends')):
73             a,d=all_results[f'{label}-{dim}d-main']
74             old128,old_d=load(f'q{q}-n128-{mode}',True)
75             matching=[]
76             for t in old128['times_s']:
77                 ids=np.where(np.isclose(a['times_s'],t,atol=1e-7,rtol=0))[0]
78                 if not len(ids):continue
79                 idx=ids[0];j=np.where(np.isclose(old128['times_s'],t,atol=1e-7,rtol=0))[0][0]
80                 matching.append(float(np.max(np.abs(a['middle_field_TC'][idx]-old128['middle'][j]))))
81             old_reuse.append({'case':f'{label}-{dim}d-main','old_receipt':f'geometry-n128','max_m_
                atching_field_abs_error':max(matching),
82                 'critical_time_difference_s':None if q==1 else d['critical_s']-old_d['critical_s'],
83                 'meaning':'same PDE/solver input states, new dense sampling only'})
84             assert max(matching)<1e-10
85             lo,ld=load(f'q{q}-n64-{mode}',True)
86             row={'question':q,'dimension':dim,'coarse_nr':64,'fine_nr':128,
87                 'critical_change_s':None if q==1 else d['critical_s']-ld['critical_s']}
88             tt=1800. if q==1 else 10800.
89             f=old_field(old128,tt);g=old_field(lo,tt)
90             row.update({'profile_{v}_max_change_at_{int(tt)}s':float(np.max(np.abs(np.interp(common_
                mon_r,old128['x'],f[:,j])-np.interp(common_r,lo['x'],g[:,j])))) for j,v in
                enumerate(('T','C'))})
91             grid.append(row)
92     for dim in (1,2):
93         hi,hd=all_results[f'q4-{dim}d-main'];lo,ld=all_results[f'q4-{dim}d-grid64']
94         row={'question':4,'dimension':dim,'coarse_nr':64,'fine_nr':128,'critical_change_s':hd['cr_
            itical_s']-ld['critical_s']}
95         for tt in (1800.,7200.,21600.):
96             for j,v in enumerate(('T','C')):
97                 f=hi['middle_field_TC'][np.where(np.isclose(hi['times_s'],tt))[0][0],:,j]
98                 g=lo['middle_field_TC'][np.where(np.isclose(lo['times_s'],tt))[0][0],:,j]
99                 row[f'profile_{v}_max_change_at_{int(tt)}s']=float(np.max(np.abs(np.interp(common_
                _r,hi['x'],f)-np.interp(common_r,lo['x'],g))))
100            grid.append(row)

```

```

101     for q,label in ((1,'q1'),(3,'q23'),(4,'q4')):
102         hi,hd=all_results[f'{label}-2d-time64']
103         lo,ld=(load(f'q{q}-n64-explicit_ends',True) if q in (1,3) else all_results['q4-2d-grid64'])
104         row={'question':q,'dimension':2,'nr':64,'max_step_from_s':120,'max_step_to_s':60,'tolerance_factor':.1,
105             'critical_change_s':None if q==1 else hd['critical_s']-ld['critical_s']}
106         tt=1800. if q==1 else 10800.
107         f=hi['middle_field_TC'][np.where(np.isclose(hi['times_s'],tt))[0][0]]
108         g=old_field(lo,tt) if q in (1,3) else
109         lo['middle_field_TC'][np.where(np.isclose(lo['times_s'],tt))[0][0]]
110         row['max_T_difference_at_keytime']=float(np.max(np.abs(f[:,0]-g[:,0])))
111         row['max_C_difference_at_keytime']=float(np.max(np.abs(f[:,1]-g[:,1])))
112         timecheck.append(row)
113     for label in ('q1','q23','q4'):
114         a,ad=all_results[f'{label}-1d-main'];b,bd=all_results[f'{label}-2d-main']
115         common=np.intersect1d(a['times_s'],b['times_s'])
116         # Simulated times, not interpolation across missing intervals.
117         i=np.searchsorted(a['times_s'],common);j=np.searchsorted(b['times_s'],common)
118         finite=np.isfinite(a['moisture_mid'][i])&np.isfinite(b['moisture_mid'][j])
119         row={'problem':label,'one_d_critical_h':ad['critical_h'],'two_d_critical_h':bd['critical_h'],
120             'two_d_minus_one_d_time_minutes':None if ad['critical_h'] is None else
121             (bd['critical_h']-ad['critical_h'])*60,
122             'sampled_midplane_max_T_error_C':float(np.max(np.abs(a['temperature_mid_C'][i]-b['temperature_mid_C'][j])[finite])),
123             'sampled_midplane_max_C_error':float(np.max(np.abs(a['moisture_mid'][i]-b['moisture_mid'][j])[finite])),
124             'sampled_axialmean_max_T_error_C':float(np.max(np.abs(a['temperature_axialmean_C'][i]-b['temperature_axialmean_C'][j])[finite])),
125             'sampled_axialmean_max_C_error':float(np.max(np.abs(a['moisture_axialmean'][i]-b['moisture_axialmean'][j])[finite]))}
126         comparisons.append(row)
127         rows=[]
128         for tt in (100.,300.,600.,900.,1200.,1500.,1800.,3600.,5400.,7200.,9000.,10800.,21600.,43200.,86400.,172800.):
129             if tt not in common:continue
130             ii=np.searchsorted(a['times_s'],tt);jj=np.searchsorted(b['times_s'],tt)
131             rows.append([tt,a['Cmax'][ii],b['Cmax'][jj],a['Cmean'][ii],b['Cmean'][jj],a['Tmean'][ii],b['Tmean'][jj],
132                 a['surface_mid_TC'][ii,1],b['surface_mid_TC'][jj,1],b['end_center_TC'][jj,1]])
133             write_csv(ROOT/'comparison'/f'{label}-dimensions.csv',[f'time_s','1d_Cmax','2d_Cmax','1d_Cmean','2d_Cmean','1d_Tmean','2d_Tmean','1d_Csurface','2d_Csurface_mid','2d_Cend_axis'],rows)
134         af,afd=all_results['q4-2d-main'];fx,fxd=all_results['q4-2d-fixed-control']
135         geometry={'changing_geometry_critical_h':afd['critical_h'],'fixed_geometry_critical_h':fxd['critical_h'],
136             'fixed_horizon_h':float(fx['times_s'][-1]/3600),'fixed_horizon_Cmax':float(fx['Cmax'][-1]),

```

```

135         'comparison_scope':'Appendix4, same 2D mesh/initial/boundary/properties; only geometry
           differs',
136         'delay_lower_bound_h':None if fxd['critical_h'] is not None else
           float(fx['times_s'][-1]/3600-afd['critical_h'])}
137     summary={'main_cases':{k:d for k,(a,d) in all_results.items() if k.endswith('-main') or
           k.endswith('-control')},
138             'dimensions':comparisons,'geometry_ablation':geometry,'grid_refinement':grid,'time_refine_
           ment':timecheck,
139             'reuse_matching_checks':old_reuse,'field_audits':audit,
140             'claim_scope':'No-latent empirical answers, with geometry and numerical diagnostics.
           Scalar-balance pass does not establish dry-mass or full energy closure.'}
141     (ROOT/'comparison').mkdir(exist_ok=True)
142     (ROOT/'numerical-summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2,all_
           ow_nan=False))
143     print(json.dumps({'cases_audited':len(audit),'main_cases':len(summary['main_cases']),
144                     'all_valid':True,'geometry_ablation':geometry,'dimensions':comparisons},ensure_ascii=Fals
           e))
145 if __name__=='__main__':main()

```

F.12 frozen_empirical_engine.py

类别：一维与二维主计算。原始代码共 291 行。

```

1  """Shared empirical drying equations, with explicit geometry and transport inputs.
2
3  The zero-axial-interval case is the radial model. Closed-loop geometry depends
4  only on simulated C, never on observed radius. Inputs are passed explicitly.
5  """
6  from dataclasses import dataclass, asdict
7  from pathlib import Path
8  import json, time
9  import numpy as np
10 from openpyxl import load_workbook
11 from scipy.integrate import BDF
12 from scipy.optimize import brentq
13 from scipy.optimize._numdiff import approx_derivative, group_columns
14 from scipy.sparse import diags, eye, kron, csc_matrix
15 from scipy.special import expi
16 from numpy.polynomial.legendre import leggauss
17
18
19 @dataclass(frozen=True)
20 class Config:
21     question: int = 4
22     nr: int = 128
23     nz: int = 64
24     radial_power: float = 1.5
25     axial_power: float = 2.0
26     R0: float = 0.02

```

```

27     length: float = 0.25
28     T0: float = 28.0
29     C0: float = 2.55
30     h: float = 25.0
31     hm: float = 8e-7
32     D_scale: float = 1.0
33     end_heat: bool = True
34     end_mass: bool = True
35     radius_mode: str = 'observed'
36     Rstar_cm: float = 1.2
37     shrink_p: float = 2.0
38     shrink_q: float = 0.0
39     tail_temp_shift: float = 0.0
40     tail_air_shift: float = 0.0
41     equilibrium_offset: float = 0.0
42     rtol: float = 2e-8
43     atol_T: float = 1e-8
44     atol_C: float = 1e-10
45     max_step: float = 120.0
46     inverse_jacobian: str = 'full'
47
48
49 def load_data(directory):
50     out=[]
51     for name in ('附件 1.xlsx','附件 2.xlsx'):
52         w=load_workbook(Path(directory)/name,read_only=True,data_only=True)
53         a=np.asarray(list(w.active.values)[1:],float);w.close()
54         if not np.isfinite(a).all() or not np.all(np.diff(a[:,0])>0):
55             raise ValueError('Input has nonfinite values or nonincreasing time.')
56         out.append(a)
57     return tuple(out)
58
59
60 class DryingModel:
61     def __init__(self,cfg,env_data,radius_data):
62         if cfg.question not in (1,2,3,4) or cfg.nr<2 or cfg.nz<0:
63             raise ValueError('Invalid question or grid.')
64         if cfg.D_scale<0 or cfg.hm<0 or cfg.h<0 or cfg.length<=0:
65             raise ValueError('Transport and geometry parameters out of range.')
66         if cfg.radius_mode not in ('fixed','observed','constitutive'):
67             raise ValueError('Unknown radius mode.')
68         self.cfg=cfg;self.env_data=env_data;self.radius_data=radius_data
69         self.tail=env_data[env_data[:,0]>=10800,1:].mean(axis=0)
70         self.x=1-(1-np.linspace(0,1,cfg.nr+1))**cfg.radial_power
71         self.rf=np.r_[0,(self.x[1:]+self.x[:-1])/2,1.]
72         self.vr=np.diff(self.rf**2)/2
73         self.z=1-(1-np.linspace(0,1,cfg.nz+1))**cfg.axial_power if cfg.nz else np.array([0.])
74         self.zf=np.r_[0,(self.z[1:]+self.z[:-1])/2,1.]

```

```

75     self.vz=np.diff(self.zf);self.H=cfg.length/2
76     self.weights=2*self.vr[:,None]*self.vz[None,:]
77     self.shape=(cfg.nr+1,cfg.nz+1,2);self.size=int(np.prod(self.shape))
78     ar=diags([np.ones(cfg.nr),np.ones(cfg.nr)],[-1,1],shape=(cfg.nr+1,cfg.nr+1))
79     adjacency=eye((cfg.nr+1)*(cfg.nz+1))+kron(ar,eye(cfg.nz+1))
80     if cfg.nz:
81         az=diags([np.ones(cfg.nz),np.ones(cfg.nz)],[-1,1],shape=(cfg.nz+1,cfg.nz+1))
82         adjacency+=kron(eye(cfg.nr+1),az)
83     self.sparsity=kron(adjacency,np.ones((2,2))).tocsc()
84     self.groups=group_columns(self.sparsity)
85     self.guard_rhs_calls=0
86
87     def env(self,t):
88         if t<=self.env_data[-1,0]:
89             return tuple(float(np.interp(t,self.env_data[:,0],self.env_data[:,i])) for i in (1,2))
90         return self.tail[0]+self.cfg.tail_temp_shift,self.tail[1]+self.cfg.tail_air_shift
91
92     def radius(self,t,y=None):
93         c=self.cfg
94         if c.radius_mode=='fixed':return c.R0
95         if c.radius_mode=='observed':
96             if t>self.radius_data[-1,0]+1e-7:raise ValueError('Observed radius may not be
97                 extrapolated beyond supplied data.')
98             return float(np.interp(t,self.radius_data[:,0],self.radius_data[:,1]))/100
99         if y is None:raise ValueError('Closed-loop radius requires the simulated state.')
100         M=max(float(np.sum(self.weights*y.reshape(self.shape)[:,:,:1]))/c.C0,1e-10)
101         v=(c.Rstar_cm/(100*c.R0))**2
102         return c.R0*np.sqrt(v+(1-v)*M**c.shrink_p*np.exp(c.shrink_q*(M-1)))
103
104     def properties(self,T,C):
105         cc=np.maximum(C,1e-8);q=self.cfg.question
106         if q==1:
107             cap=np.full_like(C,820*2600.);k=np.full_like(C,.36)
108         elif q in (2,3):
109             cap=(650+128*cc)*(1450+2736*cc/(1+cc));k=.21+.38*cc/(1+cc)
110         else:
111             cap=(760+90*cc)*(1850+2150*cc/(1+cc));k=.12+.20*cc/(1+cc)
112         return cap,k
113
114     def transport(self,T,C):
115         a,pref,E={1:(.89,7e-9,0.),2:(.45,2.4e-3,3850.),
116             3:(.45,2.4e-3,3850.),4:(.30,4.2e-4,3850.)}[self.cfg.question]
117         cc=np.maximum(C,1e-8)
118         phi=cc*np.exp(-a/cc)+a*expi(-a/cc)
119         def factor(t):return pref*self.cfg.D_scale*np.exp(-E/(t+273.15))
120         return phi,factor
121
122     def fluxes(self,t,y,R=None):

```

```

122     c=self.cfg;state=y.reshape(self.shape);T=state[:,0];C=state[:,1]
123     R=self.radius(t,y) if R is None else R
124     Ta,wa=self.env(t);eq=wa+c.equilibrium_offset
125     cap,k=self.properties(T,C);phi,factor=self.transport(T,C)
126     hr=np.zeros((c.nr+2,c.nz+1));hz=np.zeros((c.nr+1,c.nz+2))
127     mr=np.zeros_like(hr);mz=np.zeros_like(hz)
128     kr=2*k[:,1]*k[1:]/(k[:,1]+k[1:])
129     hr[1:-1]=self.rf[1:-1,None]*kr*np.diff(T,axis=0)/np.diff(self.x[:,None]
130     hr[-1]=R*c.h*(Ta-T[-1])
131     mr[1:-1]=self.rf[1:-1,None]*factor((T[-1]+T[1:])/2)*np.diff(phi,axis=0)/np.diff(self.x[:,None]
132     mr[-1]=R*c.hm*(eq-C[-1])
133     if c.nz:
134         kz=2*k[:,1]*k[:,1:]/(k[:,1]+k[:,1:])
135         hz[:,1:-1]=kz*np.diff(T,axis=1)/np.diff(self.z[None,:])
136         mz[:,1:-1]=factor((T[:,1:-1]+T[:,1:])/2)*np.diff(phi,axis=1)/np.diff(self.z[None,:])
137         if c.end_heat:hz[:,1:-1]=self.H*c.h*(Ta-T[:,1:-1])
138         if c.end_mass:mz[:,1:-1]=self.H*c.hm*(eq-C[:,1:-1])
139     return cap,hr,hz,mr,mz,R
140
141 def divergence(self,radial,axial,R):
142     return np.diff(radial,axis=0)/(self.vr[:,None]*R**2)+np.diff(axial,axis=1)/(self.vz[None,
143     :]*self.H**2)
144
145 def rhs_fixed_radius(self,t,y,R):
146     if np.min(y[1:-1])<1e-8:self.guard_rhs_calls+=1
147     cap,hr,hz,mr,mz,R=self.fluxes(t,y,R)
148     out=np.empty(self.shape);out[:,0]=self.divergence(hr,hz,R)/cap
149     out[:,1]=self.divergence(mr,mz,R)
150     return out.ravel()
151
152 def rhs(self,t,y):return self.rhs_fixed_radius(t,y,self.radius(t,y))
153
154 def jac(self,t,y):
155     R=self.radius(t,y)
156     local=csc_matrix(approx_derivative(lambda u:self.rhs_fixed_radius(t,u,R),y,
157     method='2-point',sparsity=(self.sparsity,self.groups)))
158     if self.cfg.inverse_jacobian=='local':return local
159     if self.cfg.inverse_jacobian!='full':raise ValueError('Unknown Jacobian policy.')
160     c=self.cfg;mean=float(np.sum(self.weights*y.reshape(self.shape)[::-1]));M=mean/c.C0
161     if M<=1e-10:return local
162     v=(c.Rstar_cm/(100*c.R0))**2;g=M*c.shrink_p*np.exp(c.shrink_q*(M-1))
163     Ry=np.zeros_like(y)
164     Ry[1:-1]=c.R0**2*(1-v)*g*(c.shrink_p/M+c.shrink_q)/(2*R*c.C0)*self.weights.ravel()
165     e=R*1e-5
166     fR=(self.rhs_fixed_radius(t,y,R+e)-self.rhs_fixed_radius(t,y,R-e))/(2*e)
167     return local+csc_matrix(np.outer(fR,Ry))

```

```

168     def water_out(self,t,y):
169         a=y.reshape(self.shape);C=a[:, :, 1];c=self.cfg;R=self.radius(t,y)
170         eq=self.env(t)[1]+c.equilibrium_offset
171         side=2*c.hm/R*(C[-1]@self.vz-eq)
172         end=c.hm/self.H*(2*self.vr@C[:, -1]-eq) if c.nz and c.end_mass else 0.
173         return np.array([side,end])
174
175     def initial_state(self):return np.tile([self.cfg.T0,self.cfg.C0],self.size//2)
176     def physical_state(self,y):return y.reshape(self.shape)
177
178
179 def run_model(model,end_s,times,snapshot_times=(),quadrature=False,stop_on_threshold=False):
180     """Stream accepted BDF steps and local dense output, not full dense history.
181
182     Gauss postintegration, when enabled, samples accepted-step interpolants
183     independently of the ODE state; both 4/8 rules are retained as diagnostics.
184     """
185     c=model.cfg;times=np.unique(np.asarray(times,float));snap=np.asarray(snapshot_times,float)
186     if len(times)==0 or times[0]!=0 or times[-1]>end_s:raise ValueError('Invalid sampling times.')
187     y0=model.initial_state();started=time.time();i=0
188     kwargs={'jac':model.jac} if c.radius_mode=='constitutive' else {'jac_sparsity':model.sparsity}
189     atol=model.component_atol() if hasattr(model,'component_atol') else [c.atol_T,c.atol_C]
190     solver=BDF(model.rhs,0.,y0,end_s,rtol=c.rtol,atol=np.tile(atol,model.size//2),max_step=c.max_
191         step,**kwargs)
192     saved={k:[] for k in ('times_s','radius_m','Cmax','Cmean','Tmean','middle','end','axis','side
193         ','side_out','end_out')}
194     balance=[];energy_balance=[];has_energy=hasattr(model,'energy_out')
195     energy0=float(np.sum(model.weights*y0.reshape(model.shape)[: :, 0])) if has_energy else 0.
196     ecum4=0.;ecum8=0.;eres4=0.;eres8=0.;eqdiff=0.
197     snapshots=[];snapshot_actual=[];critical=None;critical_state=None;strict_stop=None;stop_state
198         =None
199     accepted=0;amin=np.array([np.inf,np.inf]);amax=-amin;cum4=0.;cum8=0.;qdiff_max=0.;residmax4=0
200         .;residmax8=0.
201     initialmean=float(np.sum(model.weights*model.physical_state(y0)[: :, 1]))
202     initialmax=float(model.physical_state(y0)[: :, 1].max())
203     mincurve_change=-np.inf;lastmax=initialmax;old_dense=None;center_gap_max=max(0.,initialmax-fl
204         oat(model.physical_state(y0)[0,0,1]))
205     g4,w4=leggauss(4);g8,w8=leggauss(8)
206
207     def append(t,y):
208         a=model.physical_state(y);C=a[:, :, 1];T=a[:, :, 0];R=model.radius(t,y);flow=model.water_out(
209             t,y)
210         vals=(t,R,float(C.max()),float(np.sum(model.weights*C)),float(np.sum(model.weights*T)),a[
211             :, 0].copy(),a[:, -1].copy(),a[0].copy(),a[-1].copy(),flow[0],flow[1])
212         for k,v in zip(saved,vals):saved[k].append(v)
213         if np.any(np.isclose(snap,t,atol=1e-7,rtol=0)):
214             snapshots.append(a.copy());snapshot_actual.append(t)

```

```

208
209     append(0.,y0);i=1
210     a0=model.physical_state(y0);amin=a0.min(axis=(0,1));amax=a0.max(axis=(0,1))
211     while solver.status=='running':
212         t0=solver.t;y_prev=solver.y.copy();solver.step()
213         if solver.status=='failed':raise RuntimeError('BDF integration failed.')
214         accepted+=1;t1=solver.t;dense=solver.dense_output();a=model.physical_state(solver.y)
215         if not np.isfinite(solver.y).all() or not np.isfinite(a).all():raise
            RuntimeError('Nonfinite accepted state.')
216         amin=np.minimum(amin,a.min(axis=(0,1)));amax=np.maximum(amax,a.max(axis=(0,1)))
217         currentmax=float(a[:, :, 1].max());mincurve_change=max(mincurve_change,currentmax-lastmax)
218         center_gap_max=max(center_gap_max,currentmax-float(a[0,0,1]))
219         if critical is None and lastmax>.15 and currentmax<=.15:
220             critical=float(brentq(lambda
                t:float(model.physical_state(dense(t))[:, :, 1].max())-.15),t0,t1,xtol=1e-6))
221             critical_state=model.physical_state(dense(critical)).copy();strict_stop=int(np.floor(
                critical))+1
222         if strict_stop is not None and stop_state is None and t0<=strict_stop<=t1:
223             stop_state=model.physical_state(dense(strict_stop)).copy()
224         if quadrature:
225             # Splitting at supplied forcing knots prevents a polynomial quadrature
226             # error estimate from hiding the known piecewise input changes.
227             knots=np.r_[model.env_data[:,0],model.radius_data[:,0]]
228             cuts=np.unique(np.r_[t0,knots[(knots>t0)&(knots<t1)],t1])
229             for left,right in zip(cuts[:-1],cuts[1:]):
230                 mid=(left+right)/2;half=(right-left)/2
231                 cum4+=half*sum(w*model.water_out(mid+half*g,dense(mid+half*g)).sum() for g,w in
                    zip(g4,w4))
232                 cum8+=half*sum(w*model.water_out(mid+half*g,dense(mid+half*g)).sum() for g,w in
                    zip(g8,w8))
233                 if has_energy:
234                     ecum4+=half*sum(w*model.energy_out(mid+half*g,dense(mid+half*g)).sum() for g,w
                        in zip(g4,w4))
235                     ecum8+=half*sum(w*model.energy_out(mid+half*g,dense(mid+half*g)).sum() for g,w
                        in zip(g8,w8))
236             cv=float(np.sum(model.weights*a[:, :, 1]));residmax4=max(residmax4,abs(cv-initialmean+cum4));
                residmax8=max(residmax8,abs(cv-initialmean+cum8));qdiff_max=max(qdiff_max,abs(
                    cum4-cum8))
237             balance.append((t1,cv,cum4,cum8,cv-initialmean+cum4,cv-initialmean+cum8))
238             if has_energy:
239                 en=float(np.sum(model.weights*solver.y.reshape(model.shape)[:, :, 0]))
240                 eres4=max(eres4,abs(en-energy0+ecum4));eres8=max(eres8,abs(en-energy0+ecum8));eqd,
                    iff=max(eqdiff,abs(ecum4-ecum8))
241                 energy_balance.append((t1,en,ecum4,ecum8,en-energy0+ecum4,en-energy0+ecum8))
242             stop_t=strict_stop if stop_on_threshold and strict_stop is not None else end_s
243             while i<len(times) and times[i]<=min(t1,stop_t)+1e-8:
244                 append(float(times[i]),dense(times[i]));i+=1

```

```

245         lastmax=currentmax;old_dense=dense
246         if stop_on_threshold and stop_state is not None:
247             if saved['times_s'][-1]!=strict_stop:
248                 # Stored physical fields suffice for the empirical T/C state.
249                 append(float(strict_stop),dense(strict_stop))
250             break
251     if stop_state is not None and float(stop_state[:, :, 1].max())>=.15:
252         raise RuntimeError('Strict-stop verification failed.')
253     arr={k:np.asarray(v) for k,v in saved.items()}
254     arr.update(x=model.x,z=model.z,vr=model.vr,vz=model.vz,balance_accepted_steps=np.asarray(balanc
nce).reshape(-1,6),
255               energy_balance_accepted_steps=np.asarray(energy_balance).reshape(-1,6),
256               snapshots=np.asarray(snapshots),snapshot_times_s=np.asarray(snapshot_actual),
257               critical_state=np.asarray([]) if critical_state is None else critical_state,
258               stop_state=np.asarray([]) if stop_state is None else stop_state)
259     d={'config':asdict(c),'elapsed_s':time.time()-started,'accepted_steps':accepted,'nfev':solver
.nfev,'njev':solver.njev,'nlu':solver.nlu,
260       'critical_s':critical,'strict_stop_s':strict_stop if stop_state is not None else None,
261       'stop_Cmax':None if stop_state is None else float(stop_state[:, :, 1].max()),
262       'accepted_T_C_min':amin.tolist(),'accepted_T_C_max':amax.tolist(),'max_accepted_Cmax_incre
ase':mincurve_change,
263       'initial_Cmean':initialmean,'initial_Cmax':initialmax,
264       'max_accepted_Cmax_minus_center':center_gap_max,
265       'guard_trial_rhs_calls':model.guard_rhs_calls,'quadrature_enabled':quadrature,
266       'cumulative_residual_max_4':residmax4 if quadrature else
None,'cumulative_residual_max_8':residmax8 if quadrature else None,
267       'max_4_vs_8_quadrature_difference':qdiff_max if quadrature else None,
268       'integration_status':'stopped_at_strict_threshold' if stop_on_threshold and stop_state is
not None else solver.status,
269       'threshold_status':'reached' if critical is not None else 'not_reached_within_horizon',
270       'integrated_until_s':float(solver.t),'sampled_rows':len(arr['times_s'])}
271     if has_energy:
272         d.update(energy_residual_max_4=eres4 if quadrature else None,energy_residual_max_8=eres8
if quadrature else None,
273               energy_residual_relative_initial=eres8/max(abs(energy0),1.) if quadrature else
None,
274               energy_4_vs_8_difference=eqdiff if quadrature else None)
275     return arr,d
276
277
278 def physical_samples(arr,radii_cm):
279     points=np.asarray(radii_cm,float)/100;result=np.full((len(arr['times_s']),len(points)+1,2),np
.nan)
280     for i,R in enumerate(arr['radius_m']):
281         mask=points<=R+1e-12
282         for j in range(2):result[i,:len(points),j][mask]=np.interp(points[mask]/R,arr['x'],arr['m
iddle'][i,:,j])

```

```

283         result[i,-1]=arr['middle'][i,-1]
284     return result
285
286
287 def save_run(prefix,arr,diag):
288     prefix=Path(prefix);prefix.parent.mkdir(parents=True,exist_ok=True)
289     if prefix.with_suffix('.json').exists():raise FileExistsError(prefix)
290     np.savez_compressed(prefix.with_suffix('.npz'),**arr)
291     prefix.with_suffix('.json').write_text(json.dumps(diag,ensure_ascii=False,indent=2,allow_nan=False))

```

F.13 replay.py

类别：一维与二维主计算。原始代码共 21 行。

```

1  """Replay one frozen case into a new portable directory, never overwriting evidence."""
2  from pathlib import Path
3  import argparse,json,shutil,subprocess,sys
4  ROOT=Path(__file__).resolve().parents[1]
5
6  def main():
7      p=argparse.ArgumentParser();p.add_argument('case_id');p.add_argument('--output-dir',type=Path,
8          ,required=True);a=p.parse_args()
9      setting=ROOT/'configs'/f'{a.case_id}.json'
10     if not setting.is_file():raise SystemExit(f'Unknown case: {a.case_id}')
11     dst=a.output_dir.resolve()
12     if dst.exists() and any(dst.iterdir()):raise SystemExit('Output directory must be new or
13         empty')
14     for name in
15         ('src','input','configs','results','evidence'):(dst/name).mkdir(parents=True,exist_ok=True)
16     for name in ('frozen_empirical_engine.py','baseline_engine.py','solve.py','run_record.py','run_
17         cases.py'):
18         shutil.copy2(ROOT/'src'/name,dst/'src'/name)
19     shutil.copy2(setting,dst/'configs'/setting.name)
20     required=['附件 1.xlsx','题千文本.txt']
21     if
22         json.loads(setting.read_text())['model']['radius_mode']=='observed':required+=['附件2.xlsx']
23     for name in required:shutil.copy2(ROOT/'input'/name,dst/'input'/name)
24     result=subprocess.run([sys.executable,'-B','src/run_cases.py',a.case_id,'--workers','1'],cwd=
25         dst)
26     raise SystemExit(result.returncode)
27 if __name__=='__main__':main()

```

F.14 run_cases.py

类别：一维与二维主计算。原始代码共 37 行。

```
1 """Replay selected baseline cases into fresh named outputs, with execution bindings."""
2 from pathlib import Path
3 from concurrent.futures import ThreadPoolExecutor,as_completed
4 import argparse,json,subprocess,sys
5 ROOT=Path(__file__).resolve().parents[1]
6
7 def one(case):
8     setting=json.loads((ROOT/'configs'/f'{case}.json').read_text())
9     command=[sys.executable,str(ROOT/'src/run_record.py'),'record','--case',str(ROOT),'--run-id',
10             case]
11     for code in ('src/frozen_empirical_engine.py','src/baseline_engine.py','src/solve.py'):
12         command+=['--code',code]
13     for item in (f'configs/{case}.json','input/附件 1.xlsx','input/题干文本.txt'):
14         command+=['--input',item]
15     if setting['model']['radius_mode']=='observed':command+=['--input','input/附件 2.xlsx']
16     command+=['--output',f'results/{case}.npz','--output',f'results/{case}.json',
17             '--data-kind','user-provided','--timeout','3600','--',sys.executable,'-B','src/solve.py',
18             f'configs/{case}.json']
19     print(json.dumps({'start':case}),flush=True)
20     p=subprocess.run(command,cwd=ROOT,capture_output=True,text=True)
21     if p.returncode:
22         print(p.stdout,p.stderr,flush=True);raise RuntimeError(case)
23     verify=subprocess.run([sys.executable,str(ROOT/'src/run_record.py'),'verify','--case',str(ROOT),
24                             T),'--run-id',case],capture_output=True,text=True)
25     result={'case':case,'execution_verification':json.loads(verify.stdout)}
26     if verify.returncode or result['execution_verification']['execution_binding']!='valid':raise
27     RuntimeError(result)
28     d=json.loads((ROOT/'results'/f'{case}.json').read_text())
29     result.update({k:d[k] for k in
30                     ('critical_h','elapsed_s','sample_count','scalar_mean_balance_max_8')})
31     print(json.dumps(result),flush=True)
32     return result
33
34 def main():
35     p=argparse.ArgumentParser();p.add_argument('cases',nargs='*');p.add_argument('--workers',type=
36     =int,default=2);a=p.parse_args()
37     cases=a.cases or json.loads((ROOT/'configs/batch.json').read_text())['cases']
38     records=[]
39     with ThreadPoolExecutor(max_workers=a.workers) as pool:
40         for task in as_completed([pool.submit(one,c) for c in cases]):records.append(task.result())
41     name='execution-'+('-' if len(cases)>2 else 'batch')+'.json'
42     (ROOT/'evidence'/name).write_text(json.dumps(records,ensure_ascii=False,indent=2))
43 if __name__=='__main__':main()
```

F.15 run_record.py

类别：一维与二维主计算。原始代码共 267 行。

```
1  """Record authorized local computations; verify file bindings, not scientific truth."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import json
8  import math
9  import os
10 import platform
11 import re
12 import subprocess
13 import sys
14 import time
15 from datetime import datetime, timezone
16 from pathlib import Path
17
18 VERSION = 'cumcm-execution/0.1'
19
20
21 def digest(path: Path) -> str:
22     value = hashlib.sha256()
23     with path.open('rb') as handle:
24         for chunk in iter(lambda: handle.read(1024 * 1024), b''):
25             value.update(chunk)
26     return value.hexdigest()
27
28
29 def local_path(root: Path, value: object) -> Path:
30     if not isinstance(value, str) or not value.strip() or '\\' in value:
31         raise ValueError('Expected a nonempty case-relative path')
32     relative = Path(value)
33     if relative.is_absolute() or '..' in relative.parts or not relative.parts:
34         raise ValueError(f'Unsafe relative path: {value!r}')
35     current = root
36     for part in relative.parts:
37         current = current / part
38         if current.is_symlink():
39             raise ValueError(f'Symlink is not a bound local artifact: {value!r}')
40     current.resolve().relative_to(root.resolve())
41     return current
42
43
44 def artifact(root: Path, value: str, *, nonempty: bool = False) -> dict:
45     path = local_path(root, value)
```

```

46     if not path.is_file():
47         raise ValueError(f'Missing regular file: {value}')
48     size = path.stat().st_size
49     if nonempty and size == 0:
50         raise ValueError(f'Empty result file: {value}')
51     return {'path': path.relative_to(root).as_posix(), 'bytes': size, 'sha256': digest(path)}
52
53
54 def run_directory(case: Path, run_id: str) -> Path:
55     if not isinstance(run_id, str) or not re.fullmatch(r'[A-Za-z0-9][A-Za-z0-9_.-]{0,63}',
56         run_id):
57         raise ValueError('run_id must be a simple 1-64 character identifier')
58     return local_path(case, f'runs/{run_id}')
59
60 def now() -> str:
61     return datetime.now(timezone.utc).isoformat()
62
63
64 def record(case: Path, run_id: str, argv: list[str], code: list[str], inputs: list[str],
65     outputs: list[str], *, data_kind: str = 'synthetic', timeout: float = 30,
66     no_input_reason: str = '') -> dict:
67     case = case.resolve()
68     if not case.is_dir():
69         raise ValueError('case must be an existing directory')
70     if not isinstance(argv, list) or not argv or not all(isinstance(v, str) and v for v in argv):
71         raise ValueError('An explicit nonempty argv is required; shell strings are not accepted')
72     if not code or not outputs:
73         raise ValueError('Declare code and at least one expected computed result')
74     if not inputs and not no_input_reason.strip():
75         raise ValueError('A no-input calculation requires an explicit reason')
76     if data_kind not in {'synthetic', 'user-provided', 'official'}:
77         raise ValueError('Unknown data_kind')
78     if isinstance(timeout, bool) or not math.isfinite(timeout) or timeout <= 0:
79         raise ValueError('timeout must be a positive finite number')
80     roles = code + inputs + outputs
81     canonical = [local_path(case, value).resolve() for value in roles]
82     if len(set(canonical)) != len(canonical):
83         raise ValueError('Code, input and output paths must be distinct')
84     for value in roles:
85         if Path(value).parts[0] == 'runs':
86             raise ValueError('runs/ is reserved for execution records')
87     code_before = [artifact(case, value) for value in code]
88     inputs_before = [artifact(case, value) for value in inputs]
89     for value in outputs:
90         path = local_path(case, value)
91         if path.exists():
92             raise ValueError(f'Refusing to reuse or overwrite output: {value}')

```

```

93     directory = run_directory(case, run_id)
94     directory.mkdir(parents=True, exist_ok=False)
95     errors = []
96     returncode = None
97     timed_out = False
98     started = now()
99     start_clock = time.monotonic()
100    stdout = directory / 'stdout.log'
101    stderr = directory / 'stderr.log'
102    with stdout.open('xb') as out, stderr.open('xb') as err:
103        try:
104            completed = subprocess.run(argv, cwd=case, stdout=out, stderr=err,
105                                      timeout=timeout, check=False, shell=False,
106                                      env={**os.environ, 'PYTHONDONTWRITEBYTECODE': '1'})
107            returncode = completed.returncode
108        except subprocess.TimeoutExpired:
109            timed_out = True
110            errors.append('Command timed out; numerical result is not accepted')
111        except OSError as exc:
112            errors.append(f'Command could not start: {exc}')
113    if returncode != 0:
114        errors.append(f'Command did not exit successfully: {returncode}')
115    for item in code_before + inputs_before:
116        try:
117            if artifact(case, item['path']) != item:
118                errors.append(f'Declared source changed during execution: {item["path"]}')
119        except (OSError, ValueError) as exc:
120            errors.append(str(exc))
121    actual_outputs = []
122    for value in outputs:
123        try:
124            actual_outputs.append(artifact(case, value, nonempty=True))
125        except (OSError, ValueError) as exc:
126            errors.append(str(exc))
127    receipt = {
128        'schema_version': VERSION, 'run_id': run_id,
129        'status': 'executed' if not errors else 'failed',
130        'argv': argv, 'started_at': started, 'finished_at': now(),
131        'elapsed_seconds': time.monotonic() - start_clock,
132        'environment': {'python': sys.version, 'platform': platform.platform()},
133        'data_kind': data_kind, 'no_input_reason': no_input_reason,
134        'returncode': returncode, 'timed_out': timed_out, 'timeout_seconds': timeout,
135        'code': code_before, 'inputs': inputs_before,
136        'expected_outputs': [local_path(case, v).relative_to(case).as_posix() for v in outputs],
137        'outputs': actual_outputs,
138        'logs': [artifact(case, p.relative_to(case).as_posix()) for p in (stdout, stderr)],
139        'errors': errors, 'scientific_review': 'not_checked',
140    }

```

```

141     with (directory / 'receipt.json').open('x', encoding='utf-8') as handle:
142         json.dump(receipt, handle, ensure_ascii=False, indent=2)
143         handle.write('\n')
144     return receipt
145
146
147 def verify(case: Path, run_id: str) -> dict:
148     case = case.resolve()
149     errors = []
150     try:
151         directory = run_directory(case, run_id)
152         receipt = json.loads((directory / 'receipt.json').read_text(encoding='utf-8'))
153         if not isinstance(receipt, dict):
154             raise ValueError('Receipt root must be an object')
155         if receipt.get('schema_version') != VERSION or receipt.get('run_id') != run_id:
156             raise ValueError('Unsupported or mismatched execution record')
157     except (OSError, ValueError) as exc:
158         return {'execution_binding': 'failed', 'errors': [str(exc)], 'scientific_review':
159             'not_checked'}
160
161     if receipt.get('status') != 'executed':
162         errors.append('Run was not recorded as executed successfully')
163     if type(receipt.get('returncode')) is not int or receipt['returncode'] != 0:
164         errors.append('Expected integer returncode 0')
165     if receipt.get('timed_out') is not False or receipt.get('errors') != []:
166         errors.append('Timeout or recorded execution errors remain')
167     if receipt.get('scientific_review') != 'not_checked':
168         errors.append('Execution receipt must not self-certify scientific review')
169     if receipt.get('data_kind') not in {'synthetic', 'user-provided', 'official'}:
170         errors.append('Unknown data provenance category')
171     argv = receipt.get('argv')
172     if not isinstance(argv, list) or not argv or not all(isinstance(v, str) and v for v in argv):
173         errors.append('Missing actual command argv')
174     for field in ('started_at', 'finished_at'):
175         try:
176             stamp = datetime.fromisoformat(receipt[field])
177             if stamp.tzinfo is None:
178                 raise ValueError('Timezone is required')
179         except (KeyError, TypeError, ValueError):
180             errors.append(f'Invalid {field}')
181     environment = receipt.get('environment')
182     if not isinstance(environment, dict) or not all(isinstance(environment.get(k), str) and
183         environment[k]
184             for k in ('python', 'platform')):
185         errors.append('Missing environment record')
186     for field in ('elapsed_seconds', 'timeout_seconds'):
187         value = receipt.get(field)
188         if type(value) not in (int, float) or not math.isfinite(value) or value < 0 or (field ==
189             'timeout_seconds' and value == 0):

```

```

186         errors.append(f'Invalid {field}')
187     paths_by_role = {}
188     all_paths = set()
189     for role in ('code', 'inputs', 'outputs', 'logs'):
190         collection = receipt.get(role)
191         if not isinstance(collection, list) or (role != 'inputs' and not collection):
192             errors.append(f'Missing/non-list {role}')
193             continue
194         paths_by_role[role] = set()
195         if role == 'inputs' and not collection:
196             if not isinstance(receipt.get('no_input_reason'), str) or not
                receipt['no_input_reason'].strip():
197                 errors.append('Empty inputs require an explanation')
198         for item in collection:
199             try:
200                 if not isinstance(item, dict):
201                     raise ValueError(f'{role} item is not an object')
202                 size = item.get('bytes')
203                 if type(size) is not int or size < 0:
204                     raise ValueError(f'{role} item has invalid byte count')
205                 if not isinstance(item.get('sha256'), str) or not re.fullmatch(r'[a-f0-9]{64}',
                    item['sha256']):
206                     raise ValueError(f'{role} item has invalid SHA-256')
207                 current = artifact(case, item.get('path'), nonempty=role == 'outputs')
208                 key = current['path']
209                 if key in all_paths:
210                     raise ValueError(f'Duplicate/overlapping artifact: {key}')
211                 all_paths.add(key)
212                 paths_by_role[role].add(key)
213                 if current != item:
214                     raise ValueError(f'Stale or changed {role} artifact: {key}')
215             except (OSError, ValueError, TypeError) as exc:
216                 errors.append(str(exc))
217     expected = receipt.get('expected_outputs')
218     if not isinstance(expected, list) or not expected or not all(isinstance(v, str) and v for v in
        expected):
219         errors.append('Missing expected result list')
220     else:
221         try:
222             expected_paths = [local_path(case, v).relative_to(case).as_posix() for v in expected]
223             if len(set(expected_paths)) != len(expected_paths) or set(expected_paths) !=
                paths_by_role.get('outputs'):
224                 errors.append('Expected outputs do not match bound outputs')
225             except (ValueError, OSError) as exc:
226                 errors.append(str(exc))
227     wanted_logs = {f'runs/{run_id}/stdout.log', f'runs/{run_id}/stderr.log'}
228     if paths_by_role.get('logs') != wanted_logs:
229         errors.append('Captured stdout/stderr logs are missing or mismatched')

```

```

230     return {'execution_binding': 'valid' if not errors else 'failed', 'errors': errors,
231            'scientific_review': 'not_checked', 'data_kind': receipt.get('data_kind')}
232
233
234 def main() -> int:
235     parser = argparse.ArgumentParser(description=__doc__)
236     sub = parser.add_subparsers(dest='mode', required=True)
237     capture = sub.add_parser('record')
238     check = sub.add_parser('verify')
239     for command in (capture, check):
240         command.add_argument('--case', type=Path, required=True)
241         command.add_argument('--run-id', required=True)
242     capture.add_argument('--code', action='append', required=True)
243     capture.add_argument('--input', action='append', default=[])
244     capture.add_argument('--output', action='append', required=True)
245     capture.add_argument('--no-input-reason', default='')
246     capture.add_argument('--data-kind', choices=['synthetic', 'user-provided', 'official'],
247                          default='synthetic')
248     capture.add_argument('--timeout', type=float, default=30)
249     capture.add_argument('argv', nargs=argparse.REMAINDER)
250     args = parser.parse_args()
251     try:
252         if args.mode == 'record':
253             argv = args.argv[1:] if args.argv[:1] == ['--'] else args.argv
254             result = record(args.case, args.run_id, argv, args.code, args.input, args.output,
255                            data_kind=args.data_kind, timeout=args.timeout,
256                            no_input_reason=args.no_input_reason)
257             print(json.dumps({'status': result['status'], 'errors': result['errors'],
258                             'scientific_review': 'not_checked'}, ensure_ascii=False, indent=2))
259             return 0 if result['status'] == 'executed' else 2
260         result = verify(args.case, args.run_id)
261         print(json.dumps(result, ensure_ascii=False, indent=2))
262         return 0 if result['execution_binding'] == 'valid' else 2
263     except (OSError, ValueError) as exc:
264         print(json.dumps({'status': 'failed', 'error': str(exc)}, ensure_ascii=False))
265         return 2
266
267 if __name__ == '__main__':
268     raise SystemExit(main())

```

F.16 solve.py

类别：一维与二维主计算。原始代码共 155 行。

```
1  """Complete full-precision Q1, Q2/3, Q4 samples using accepted BDF dense output."""
2  import os
3  for key in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS'):
4      os.environ[key]='1'
5  from pathlib import Path
6  from dataclasses import asdict
7  import argparse, json, time
8  import numpy as np
9  from numpy.polynomial.legendre import leggauss
10 from scipy.integrate import BDF
11 from scipy.optimize import brentq
12 from openpyxl import load_workbook
13 from baseline_engine import Config, BaselineModel
14
15 ROOT=Path(__file__).resolve().parents[1]
16
17 def load_sheet(path):
18     w=load_workbook(path, read_only=True, data_only=True)
19     x=np.asarray(list(w.active.values)[1:], dtype=float); w.close()
20     if not np.isfinite(x).all() or np.any(np.diff(x[:,0])<=0):
21         raise ValueError('Invalid observations')
22     return x
23
24 def run(settings):
25     cfg=Config(**settings['model'])
26     env=load_sheet(ROOT/'input/附件 1.xlsx')
27     # Explicitly isolate Q1-3 from Q4 data.
28     radius=load_sheet(ROOT/'input/附件2.xlsx') if cfg.radius_mode=='observed' else np.empty((0,2))
29     model=BaselineModel(cfg, env, radius)
30     complete=settings.get('complete_output', True)
31     end=1800. if cfg.question==1 else 72*3600.
32     snaps=np.asarray([0,100,300,600,900,1200,1500,1800] if cfg.question==1 else
33                      [0,1800,3600,5400,7200,9000,10800,14400,21600,43200,64800,
34                      86400,108000,129600,151200,172800,194400,216000,237600,259200], float)
35     if complete:
36         times=np.arange(0,1801,dtype=float) if cfg.question==1 else (
37             np.r_[np.arange(0,10801), np.arange(10860, end+1, 60)].astype(float)
38             if cfg.question in (2,3) else np.arange(0, end+1, 60, dtype=float))
39     else:
40         times=snaps
41     times=np.unique(np.r_[times, snaps]); times=times[times<=end]
42     sample_r=np.arange(21, dtype=float)/1000
43     saved={k:[] for k in
44           ('times_s', 'R_m', 'L_m', 'volume_m3', 'area_factor', 'Cmax', 'Cmin', 'Cmean', 'Tmean',
45            'temperature_mid_C', 'moisture_mid', 'temperature_axialmean_C', 'moisture_axialmean',
```

```

45     'surface_mid_TC', 'surface_axialmean_TC', 'end_center_TC', 'side_mean_TC', 'end_mean_TC',
46     'middle_field_TC', 'axialmean_field_TC', 'side_field_TC', 'axis_field_TC', 'end_field_TC',
47     'density_dry_mass_kg', 'density_water_mass_kg', 'density_wet_mass_kg', 'density_dry_mass_rel',
    'ative_change',
48     'scalar_out_side_per_s', 'scalar_out_end_per_s'})
49 snapshots=[];snap_actual=[];balance=[];rate_checks=[]
50 y0=model.initial_state();mean0=float(np.sum(model.weights*model.physical_state(y0)[:,: ,1]))
51 cum4=0.;cum8=0.;maxres4=0.;maxres8=0.;maxqdiff=0.;critical=None;strict=None;stopstate=None;cr_
    iticalstate=None
52 positive_min=np.array([cfg.T0,cfg.C0]);positive_max=positive_min.copy()
53 fullmax_minus_center=0.;max_increase=0.;lastmax=cfg.C0
54 g4,w4=leggauss(4);g8,w8=leggauss(8)
55 knots=np.unique(np.r_[env[:,0],radius[:,0]])
56 def append(t,y):
57     a=model.physical_state(y);s=model.state_summary(t,y)
58     middle=a[:,0];axmean=np.einsum('ijk,j->ik',a,model.vz)
59     mids=np.full((21,2),np.nan);means=mids.copy();mask=sample_r<=s['R_m']+1e-12
60     for j in range(2):
61         mids[mask,j]=np.interp(sample_r[mask]/s['R_m'],model.x,middle[:,j])
62         means[mask,j]=np.interp(sample_r[mask]/s['R_m'],model.x,axmean[:,j])
63     flow=model.water_out(t,y)
64     values=dict(times_s=t,**s,temperature_mid_C=mids[:,0],moisture_mid=mids[:,1],
65         temperature_axialmean_C=means[:,0],moisture_axialmean=means[:,1],
66         surface_mid_TC=middle[-1].copy(),surface_axialmean_TC=axmean[-1].copy(),
67         end_center_TC=a[0,-1].copy(),side_mean_TC=np.einsum('ij,i->j',a[-1],model.vz),
68         end_mean_TC=np.einsum('ij,i->j',a[:,-1],2*model.vr),
69         middle_field_TC=middle.copy(),axialmean_field_TC=axmean.copy(),
70         side_field_TC=a[-1].copy(),axis_field_TC=a[0].copy(),end_field_TC=a[:,-1].copy(),
71         scalar_out_side_per_s=flow[0],scalar_out_end_per_s=flow[1])
72     for key in saved:saved[key].append(values[key])
73     if np.any(np.isclose(snaps,t,atol=1e-7,rtol=0)):
74         snapshots.append(a.copy());snap_actual.append(t)
75         rate_checks.append(model.differential_balance(t,y))
76 started=time.time();append(0.,y0);i=1;accepted=0
77 bdf=BDF(model.rhs,0.,y0,end,rtol=cfg.rtol,
78     atol=np.tile([cfg.atol_T,cfg.atol_C],model.size//2),
79     max_step=cfg.max_step,jac_sparsity=model.sparsity)
80 while bdf.status=='running':
81     t0=bdf.t;bdf.step()
82     if bdf.status=='failed':raise RuntimeError('BDF failed')
83     t1=bdf.t;dense=bdf.dense_output();a=model.physical_state(bdf.y)
84     if not np.isfinite(a).all():raise RuntimeError('Nonfinite accepted field')
85     accepted+=1;positive_min=np.minimum(positive_min,a.min(axis=(0,1)));positive_max=np.maxim_
        um(positive_max,a.max(axis=(0,1)))
86     currentmax=float(a[:, :,1].max());max_increase=max(max_increase,currentmax-lastmax)
87     fullmax_minus_center=max(fullmax_minus_center,currentmax-float(a[0,0,1]))
88     if cfg.question!=1 and critical is None and lastmax>.15 and currentmax<=.15:

```

```

89         critical=float(brentq(lambda
        t:float(model.physical_state(dense(t))[:, :, 1].max()- .15), t0, t1, xtol=1e-7))
90         criticalstate=model.physical_state(dense(critical)).copy(); strict=int(np.floor(critic
        al))+1
91         audit_end=min(t1, strict) if strict is not None else t1
92         if strict is not None and t0<=strict<=t1:
93             stopstate=model.physical_state(dense(strict)).copy()
94             cuts=np.unique(np.r_[t0, knots[(knots>t0)&(knots<audit_end)], audit_end])
95             for left, right in zip(cuts[:-1], cuts[1:]):
96                 mid=(left+right)/2; half=(right-left)/2
97                 cum4+=half*sum(w*model.water_out(mid+half*g, dense(mid+half*g)).sum() for g, w in
                    zip(g4, w4))
98                 cum8+=half*sum(w*model.water_out(mid+half*g, dense(mid+half*g)).sum() for g, w in
                    zip(g8, w8))
99             am=model.physical_state(dense(audit_end)); currentmean=float(np.sum(model.weights*am[:, :, 1
        ]))
100         res4=currentmean-mean0+cum4; res8=currentmean-mean0+cum8
101         maxres4=max(maxres4, abs(res4)); maxres8=max(maxres8, abs(res8)); maxqdiff=max(maxqdiff, abs(c
        um4- cum8))
102         balance.append((audit_end, currentmean, cum4, cum8, res4, res8))
103         while i<len(times) and times[i]<=audit_end+1e-8:
104             append(float(times[i]), dense(times[i])); i+=1
105         lastmax=currentmax
106         if stopstate is not None:
107             if saved['times_s'][-1]!=strict:append(float(strict), dense(strict))
108             snapshots.append(stopstate.copy()); snap_actual.append(float(strict))
109             rate_checks.append(model.differential_balance(float(strict), dense(strict)))
110             break
111         arrays={k:np.asarray(v) for k, v in saved.items()}
112         arrays.update(x=model.x, eta=model.z, radial_volume_weights=model.vr, axial_volume_weights=model
        .vz,
113             radii_cm=sample_r*100, snapshot_fields_TC=np.asarray(snapshots), snapshot_times_s=np.asarra
        y(snap_actual),
114             scalar_balance=np.asarray(balance), differential_balance=np.asarray(rate_checks),
115             critical_field_TC=np.asarray([]) if criticalstate is None else criticalstate,
116             strict_stop_field_TC=np.asarray([]) if stopstate is None else stopstate)
117         diag={'config':asdict(cfg), 'case_id':settings['case_id'], 'elapsed_s':time.time()-started,
118             'accepted_steps':accepted, 'nfev':bdf.nfev, 'njev':bdf.njev, 'nlu':bdf.nlu, 'solver_status':b
        df.status,
119             'threshold_status':'not_required' if cfg.question==1 else ('reached' if critical else
        'not_reached'),
120             'critical_s':critical, 'critical_h':None if critical is None else critical/3600,
121             'strict_stop_s':strict, 'strict_stop_Cmax':None if stopstate is None else
        float(stopstate[:, :, 1].max()),
122             'sampled_until_s':float(arrays['times_s'][-1]), 'integrated_until_s':float(bdf.t),
123             'sample_count':len(arrays['times_s']), 'accepted_T_C_min':positive_min.tolist(),

```

```

124     'accepted_T_C_max':positive_max.tolist(),'max_accepted_Cmax_minus_center':fullmax_minus_center,
125     'max_accepted_Cmax_increase':max_increase,'guard_trial_rhs_calls':model.guard_rhs_calls,
126     'scalar_mean_balance_max_4':maxres4,'scalar_mean_balance_max_8':maxres8,
127     'scalar_quadrature_difference_max':maxqdiff,
128     'differential_moisture_residual_max':float(np.abs(arrays['differential_balance'][:,3]).max()),
129     'differential_heat_residual_max_W_m3':float(np.abs(arrays['differential_balance'][:,6]).max()),
130     'density_dry_mass_initial_kg':model.initial_dry_mass,
131     'density_dry_mass_max_abs_relative_change':float(np.abs(arrays['density_dry_mass_relative_change']).max()),
132     'density_dry_mass_final_relative_change':float(arrays['density_dry_mass_relative_change'][-1]),
133     'radius_data_loaded':bool(len(radius)),
134     'observed_environment_end_s':float(env[-1,0]),'environment_tail_T_C_wa':model.tail.tolist(),
135     'boundary_model':'empirical Cs-wa; not a calibrated vapor boundary',
136     'latent_heat':False,'water_enthalpy':False,
137     'conservation_scope':'reference-coordinate scalar C balance; not physical dry-mass or complete energy closure',
138     'sampling_scope':'1D is an axial-average approximation; 2D template primary samples use axial midplane',
139     'point_outside_moving_body':'NaN, never zero',
140     'properties_update':'every nonlinear RHS/Jacobian trial; not frozen per macro step'}
141 if positive_min[1]<0 or model.guard_rhs_calls:raise RuntimeError('Moisture positivity guard failed')
142 if maxres8>1e-5:raise RuntimeError('Scalar balance failed')
143 if cfg.question!=1 and strict is not None and diag['strict_stop_Cmax']>=.15:raise
RuntimeError('No valid full-field strict stop')
144 if cfg.question!=1 and strict is None and not settings.get('allow_not_reached',False):raise
RuntimeError('No threshold reached')
145 return arrays,diag
146
147 def main():
148     p=argparse.ArgumentParser();p.add_argument('config',type=Path);p.add_argument('--output-prefix',type=Path);a=p.parse_args()
149     settings=json.loads(a.config.read_text());prefix=a.output_prefix or
ROOT/'results'/settings['case_id']
150     if prefix.with_suffix('.json').exists() or prefix.with_suffix('.npz').exists():raise
FileExistsError(prefix)
151     prefix.parent.mkdir(parents=True,exist_ok=True)
152     arr,diag=run(settings);np.savez_compressed(prefix.with_suffix('.npz'),**arr)
153     prefix.with_suffix('.json').write_text(json.dumps(diag,ensure_ascii=False,indent=2,allow_nan=False))

```

```

154     print(json.dumps({k:diag[k] for k in
        ('case_id','elapsed_s','critical_h','strict_stop_s','sample_count','scalar_mean_balance_max_',
        8,'density_dry_mass_max_abs_relative_change')),ensure_ascii=False),flush=True)
155 if __name__=='__main__':main()

```

F.17 assemble.py

类别：守恒变长度模型 A。原始代码共 108 行。

```

1  """Independent saved-array audit and event-inclusive tables; no raw output edits."""
2  import os
3  for k in ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS'):os.environ[k]='1'
4  from pathlib import Path
5  import argparse, csv, json, subprocess, sys
6  import numpy as np
7  from minimal_model import Config, load_input
8  from experiment_model import Model
9  ROOT=Path(__file__).resolve().parents[1]
10
11
12 def readcsv(path):
13     with path.open(encoding='utf-8-sig', newline='') as f:
14         rd=csv.reader(f); return next(rd), list(rd)
15
16 def writecsv(path, head, rows):
17     with path.open('w', encoding='utf-8-sig', newline='') as f:
18         w=csv.writer(f); w.writerow(head); w.writerows(rows)
19
20
21 def run_case(case):
22     directory=ROOT/'results'/case; dest=ROOT/'delivery'/case
23     dest.mkdir(parents=True, exist_ok=False)
24     summary=json.loads((directory/'summary.json').read_text())
25     z=np.load(directory/'fields.npz'); times=z['times']; state=z['states_H_C']; mass=z['lumped_dry_m',
        'ass']; Md=mass.sum()
26     H=state[:, :, 0]; C=state[:, :, 1]; T=28+H/(1850+4000*C)
27     record=z['records']; b=z['budget_steps']; J=summary['rhod0']*(1+C)/(760+90*C)
28     volume_state=((1+C)/(760+90*C))@mass
29     volume_geo=np.pi*record[:, 1]**2*record[:, 8]
30     r=z['r_nodes_m']; zp=z['z_nodes_m']
31     positive=np.isfinite(state).all() and C.min()>0 and T.min()>0
32     wm=C@mass; wet_mass=(1+C)@mass
33     mass_res=b[:, 2]-summary['config']['C0']+b[:, 3]
34     energy_res=b[:, 1]+b[:, 4]
35     finetime=float(times[-1]); finetarget=(summary['config']['C0']-.15)
36     stop=summary['strict_stop_s']; threshold_state=z['strict_stop_state_H_C']
37     strict=bool(len(threshold_state)>0 and threshold_state[:, 1].max()<.15)
38     if summary['strict_stop_verified']!=strict: raise AssertionError('Threshold flag differs from
        arrays')

```

```

39     cfg={k:v for k,v in summary['config'].items() if k!='equilibrium_C50'}
40     env,rad=load_input(ROOT/'input');m=Model(Config(**cfg),env,rad)
41     mid_head,mid_rows=readcsv(directory/'midplane.csv');surf_head,surf_rows=readcsv(directory/'surface.csv')
42     snapshot_head,snapshots=readcsv(directory/'snapshots.csv')
43     if strict and not np.any(abs(times-stop)<1e-9):
44         hs,cs,ts=m.unpack(threshold_state);gd=m.geometry_data(stop,cs);rn,zn=m.nodal_positions(gd)
45         cs=cs.reshape(m.shape);ts=ts.reshape(m.shape)
46         for rr in np.arange(21)/10:
47             if rr/100<=gd['R']+1e-12:
48                 tt=float(np.interp(min(rr/100,rn[-1,0]),rn[:,0],ts[:,0]));cc=float(np.interp(min(rr/100,rn[-1,0]),rn[:,0],cs[:,0]))
49             else:tt=cc=''
50             mid_rows.append([stop,stop/3600,rr,tt,cc,gd['R']*100])
51             surf_rows.append([stop,stop/3600,gd['R']*100,ts[-1,0],cs[-1,0],ts[0,-1],cs[0,-1],ts[-1,-1],cs[-1,-1],gd['L']*100])
52     mid_rows.sort(key=lambda row:(float(row[0]),float(row[2])));surf_rows.sort(key=lambda row:float(row[0]))
53     writecsv(dest/'midplane.csv',mid_head,mid_rows);writecsv(dest/'surface.csv',surf_head,surf_rows)
54     writecsv(dest/'snapshots.csv',snapshot_head,snapshots)
55     trace_head,trace_rows=readcsv(directory/'trajectory.csv')
56     if strict and not
57         np.any(abs(times-stop)<1e-9):trace_rows.append(m.record(stop,threshold_state))
58     trace_rows.sort(key=lambda
59         row:float(row[0]));writecsv(dest/'trajectory.csv',trace_head,trace_rows)
60     # Every physically exterior point is blank, including early and final rows.
61     outside_bad=0;inside_blank=0
62     for row in mid_rows:
63         outside=float(row[2])>float(row[5])+1e-10
64         if outside and (row[3]!='' or row[4]!=''):outside_bad+=1
65         if not outside and (row[3]=='' or row[4]==''):inside_blank+=1
66     audit={
67         'case':case,'positive_finite_state':bool(positive),'completed_until_s':finetime,
68         'strict_threshold_verified_from_saved_state':strict,
69         'strict_stop_s':stop,'critical_h':summary['critical_h'],
70         'dry_mass_kg':float(Md),'dry_mass_initial_geometry_relative_error':float(abs(Md-summary['rhod0'])*np.pi*.02**2*.25)/Md),
71         'dry_mean_C_array_record_max_abs':float(abs(wm/Md-record[:,2]).max()),
72         'dry_mean_T_array_record_max_abs':float(abs((T@mass)/Md-record[:,3]).max()),
73         'full_field_max_record_max_abs':float(abs(C.max(axis=1)-record[:,4]).max()),
74         'wet_mass_minus_dry_and_water_max_abs_kg':float(abs(wet_mass-Md-wm).max()),
75         'density_state_volume_vs_cylinder_max_relative':float(abs(volume_state-volume_geo).max()/volume_geo.min()),
76         'boundary_radius_saved_vs_attachment_max_abs_m':float(abs(record[:,1]-np.interp(times,rad[:,0],rad[:,1])/100).max()),
77         'saved_surface_radius_vs_record_max_abs_m':float(abs(r[:,1]-record[:,1,None]).max()),

```

```

76     'saved_end_z_vs_half_length_max_abs_m':float(abs(zp[:, :, -1]-record[:, 8, None]/2).max()),
77     'saved_midplane_z_max_abs_m':float(abs(zp[:, :, 0]).max()),
78     'mass_residual_recomputed_max_kg_perkgdry':float(abs(mass_res).max()),
79     'energy_residual_recomputed_max_J_perkgdry':float(abs(energy_res).max()),
80     'budget_channel_sum_max_abs_J_perkgdry':float(abs(b[:, 4]-b[:, 5:8].sum(axis=1)).max()),
81     'end_budget_C_vs_field_max_abs':float(abs(b[-1, 2]-wm[-1]/Md)),
82     'end_budget_H_vs_field_max_abs':float(abs(b[-1, 1]-H[-1]*mass/Md)),
83     'exterior_nonblank_rows':outside_bad, 'interior_blank_rows':inside_blank,
84     'event_midplane_rows':sum(float(row[0])==stop for row in mid_rows) if strict else 0,
85     'event_surface_rows':sum(float(row[0])==stop for row in surf_rows) if strict else 0,
86     'mass_balance_relative':summary['mass_balance_relative'], 'enthalpy_balance_relative':summary['enthalpy_balance_relative'],
87     'density_interpolation_relative_error':summary['checks_max']['rho_mixture_relative_error'],
88     'length_peak_cm':100*summary['L_max_m'], 'length_at_stop_cm':None if not strict else
        100*summary['L_at_strict_stop_m'],
89     'physical_vapor_energy_included':False, 'new_surface_equilibrium_parameters':False,
90     'numerical_consistency_is_material_validation':False,
91 }
92 assert positive and finetime==72*3600 and strict and outside_bad==inside_blank==0
93 assert audit['event_midplane_rows']==21 and audit['event_surface_rows']==1
94 assert audit['density_state_volume_vs_cylinder_max_relative']<1e-12
95 assert audit['full_field_max_record_max_abs']<1e-12
96 assert audit['budget_channel_sum_max_abs_J_perkgdry']<1e-7
97 (dest/'audit.json').write_text(json.dumps(audit, ensure_ascii=False, indent=2, allow_nan=False)+
    '\n')
98 return audit
99
100
101 def main():
102     p=argparse.ArgumentParser();p.add_argument('--cases', nargs='+', required=True);a=p.parse_args()
103     audits=[run_case(c) for c in a.cases]
104     writcsv(ROOT/'delivery/case-summary.csv', list(audits[0]), [[x[k] for k in audits[0]] for x in
        audits])
105     (ROOT/'analysis/saved-array-audit.json').write_text(json.dumps(audits, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
106     print(json.dumps(audits, ensure_ascii=False, indent=2))
107
108 if __name__=='__main__':main()

```

F.18 build_delivery.py

类别：守恒变长度模型 A。原始代码共 64 行。

```

1  """Assemble all cases, sample fixed physical radii in the exact mapped element.
2
3  The inherited raw CSV uses linear physical-node interpolation. This final export
4  instead inverts the actual monotone radial Map A within each element, then uses
5  its reference-coordinate Q1 field. Both raw and final data remain traceable.
6  """

```

```

7 import os
8 for k in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS'): os.environ[k]='1'
9 import argparse, json
10 from pathlib import Path
11 import numpy as np
12 from scipy.optimize import brentq
13 from assemble import run_case, writecsv, ROOT
14 from minimal_model import Config, load_input
15 from experiment_model import Model
16
17
18 def sample_midplane(m, t, y, radii):
19     _, C, T = m.unpack(y); C = C.reshape(m.shape); T = T.reshape(m.shape)
20     gd = m.geometry_data(t, C); rn = np.sqrt(2*gd['I']/gd['lambda_z'])
21     out = []; linear_error = np.zeros(2); reconstruction_error = 0.
22     for radius in radii:
23         r = radius/100
24         if r > gd['R'] + 1e-12: out.append([t, t/3600, radius, '', '', gd['R']*100]); continue
25         r = min(r, gd['R'])
26         i = min(int(np.searchsorted(rn, r, side='right') - 1), m.c.nr - 1); i = max(0, i)
27         a0 = m.A[i]; da = m.A[i+1] - a0; b0 = gd['bar'][i]; ba = (gd['bar'][i+1] - b0)/da
28         target = gd['lambda_z']*r*r/2 - gd['I'][i]
29         def poly(u): return b0*(a0*u + u*u/2) + ba*(a0*u + u*u/2 + u*u*u/3)
30         if target <= 0.: u = 0.
31         elif target >= poly(da): u = da
32         else: u = brentq(lambda u: poly(u) - target, 0., da, xtol=1e-15, rtol=1e-14)
33         x = u/da; ct = (1-x)*C[i, 0] + x*C[i+1, 0]; tt = (1-x)*T[i, 0] + x*T[i+1, 0]
34         rr = np.sqrt(2*(gd['I'][i] + poly(u))/gd['lambda_z'])
35         reconstruction_error = max(reconstruction_error, abs(rr - r))
36         linear_error = np.maximum(linear_error, abs(np.array([np.interp(r, rn, T[:, 0]) - tt, np.interp(r, rn, C[:, 0]) - ct])))
37     out.append([t, t/3600, radius, float(tt), float(ct), gd['R']*100])
38     return out, linear_error, reconstruction_error
39
40
41 def main():
42     p = argparse.ArgumentParser(); p.add_argument('--cases', nargs='+', required=True); a = p.parse_args()
43     env, rad = load_input(ROOT/'input')
44     for name in a.cases:
45         audit = run_case(name); raw = ROOT/'results'/name; dest = ROOT/'delivery'/name
46         summary = json.loads((raw/'summary.json').read_text()); z = np.load(raw/'fields.npz')
47         cfg = {k: v for k, v in summary['config'].items() if k != 'equilibrium_C50'}; m = Model(Config(**cfg), env, rad)
48         rows = []; error = np.zeros(2); rerror = 0.
49         states = list(zip(z['times'], z['states_H_C']))
50         if summary['strict_stop_verified'] and not np.any(abs(z['times'] - summary['strict_stop_s']) < 1e-9):
            states.append((summary['strict_stop_s'], z['strict_stop_state_H_C']))

```

```

51         for t,y in sorted(states,key=lambda x:x[0]):
52             out,err,rerr=sample_midplane(m,float(t),y,np.arange(21)/10);rows.extend(out);error=np.
                .maximum(error,err);rerror=max(rerror,rerr)
53         writcsv(dest/'midplane.csv',['time_s','time_h','radius_cm','T_C','C_kg_per_kgdry','curren
                nt_radius_cm'],rows)
54         assert rerror<1e-12
55         audit.update({'point_sampling_method':'Invert exact monotone radial Map A, then
                interpolate Q1 field in reference radial coordinate',
56                     'point_sampling_radius_reconstruction_error_m':rerror,
57                     'raw_physical_node_interpolation_max_difference_T_K':float(error[0]),
58                     'raw_physical_node_interpolation_max_difference_C':float(error[1])})
59         (dest/'audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2,allow_nan=False),
                se)+'\n');audits.append(audit)
60         writcsv(ROOT/'delivery/case-summary.csv',list(audits[0]),[[x[k] for k in audits[0]] for x in
                audits])
61         (ROOT/'analysis/saved-array-audit.json').write_text(json.dumps(audits,ensure_ascii=False,inde
                nt=2,allow_nan=False)+'\n')
62         print(json.dumps(audits,ensure_ascii=False,indent=2))
63
64 if __name__=='__main__':main()

```

F.19 __init__.py

类别：守恒变长度模型 A。原始代码共 0 行。
该文件为空，仅承担 Python 包标识作用。

F.20 __init__.py

类别：守恒变长度模型 A。原始代码共 1 行。

```

1 """Frozen reviewed Q4 thermal/FEM dependencies; not a new material model."""

```

F.21 boundary_model.py

类别：守恒变长度模型 A。原始代码共 62 行。

```

1 """Explicit Q4 boundary substitution; reused Q1 FEM/geometry are not rebuilt.
2
3 The empirical hm is interpreted as a material dry-basis effective coefficient.
4 The sorption relation is a declared conditional closure, not a supplied fit.
5 latent=0 is a numerical energy-channel ablation, NOT a physical vapor model.
6 """
7 from dataclasses import dataclass
8 import types
9 import numpy as np
10 from .volume_fem import Config as FEMConfig, Model as FEMModel, load_input, solve
11 from .thermo_reference import RV, TR, CPL, CPV, LREF, SIGMA, psat, hv
12
13
14 @dataclass(frozen=True)
15 class Config(FEMConfig):

```

```

16     question: int = 4
17     volume_mode: str = 'given_density'
18     desorption_B: float = 0.
19     equilibrium_C50: float = .075
20     emissivity: float = 0.
21     hm: float = 8e-7
22     latent: float = 1.
23
24
25     class Model(FEMModel):
26         def __init__(self, cfg, env, radius):
27             if cfg.question != 4 or cfg.volume_mode != 'given_density':
28                 raise ValueError('This bounded experiment only evaluates Q4 given-density closure')
29             if cfg.latent not in (0., 1.) or cfg.hm <= 0:
30                 raise ValueError('Invalid paired experiment setting')
31             super().__init__(cfg, env, radius)
32             # Replace every boundary evaluation before any solve. The reused Lewis
33             # surface function is never called by this experiment.
34             self.th.surface = types.MethodType(self._new_surface, self.th)
35
36         def _new_surface(self, th, t, T, C):
37             c=self.c
38             T=np.asarray(T);C=np.asarray(C)
39             if np.any(C<=0):raise ValueError('Nonpositive surface moisture; no clipping')
40             Ta,wa=th.env(t);pa=th.vapor_pressure(wa)
41             thermal=np.exp(th.thermal_log_aw(T))
42             den=thermal*psat(T)-pa
43             if np.any(den<=0):
44                 raise ValueError(f'No finite equilibrium moisture: denominator minimum {den.min()} Pa')
45             Ceq=th.s*pa/den
46             rhod=(760.+90.*C)/(1.+C)
47             J=rhod*c.hm*(C-Ceq)
48             aw=th.activity(T,C);pw=aw*psat(T)
49             upstream=np.where(J>=0,T,Ta)
50             hv_up=hv(upstream)+c.energy_gauge
51             carry=hv_up if c.latent==1 else th.bound_h(T)
52             convection=c.h*(T-Ta)
53             radiation=c.emissivity*SIGMA*((T+273.15)**4-(Ta+th.c.wall_offset+273.15)**4)
54             energy=convection+radiation+carry*J
55             # Full physical film expression is recorded with the new signed J.
56             # For latent=0 it is only a counterfactual diagnostic: the ablated
57             # energy equation does not have the physical vapor entropy flux.
58             Ks=T+273.15;Ka=Ta+273.15
59             integral=(LREF-CPV*TR)*(1/Ks-1/Ka)+CPV*np.log(Ka/Ks)
60             sigma=convection*(1/Ka-1/Ks)+J*(RV*np.log(pw/pa)+integral+hv(upstream)*(1/Ka-1/Ks))
61             sigma+=radiation*(1/(Ta+th.c.wall_offset+273.15)-1/Ks)
62             return J,energy,convection,radiation,carry*J,aw,sigma

```

F.22 thermo_reference.py

类别：守恒变长度模型 A。原始代码共 274 行。

```
1  """Conditional dry-mass and enthalpy conserving axisymmetric drying model.
2
3  Not a calibrated material predictor. rho(C) is explicitly NOT retained as a
4  current wet density: the supplied density is used at the initial state only.
5  All unidentified closures remain declared parameters. See model report.
6  """
7  from dataclasses import dataclass, asdict
8  from pathlib import Path
9  import json, time
10 import numpy as np
11 from openpyxl import load_workbook
12 from scipy.integrate import BDF
13 from scipy.optimize import brentq
14 from scipy.sparse import diags, eye, kron
15 from scipy.special import expi
16 from numpy.polynomial.legendre import leggauss
17
18 TR=301.15
19 RV=461.52
20 RDA=287.055
21 CPL=4186.
22 CPV=1860.
23 LREF=2434600.
24 PREF=3783.1
25 SIGMA=5.670374419e-8
26
27
28 def psat(T):
29     """Ideal-vapor Clausius/constant-cp closure, anchored to NIST at28C."""
30     K=np.asarray(T)+273.15
31     dc=CPV-CPL
32     return PREF*np.exp(((LREF-dc*TR)*(1/TR-1/K)+dc*np.log(K/TR))/RV)
33
34
35 def hv(T):return LREF+CPV*(np.asarray(T)+273.15-TR)
36
37
38 @dataclass(frozen=True)
39 class Config:
40     question:int=4
41     nr:int=32
42     nz:int=16
43     radial_power:float=2.
44     axial_power:float=3.
45     R0:float=.02
```

```

46     length0:float=.25
47     T0:float=28.
48     C0:float=2.55
49     radius_mode:str='observed'
50     axial_exponent:float=0.
51     h:float=25.
52     end_heat:bool=True
53     end_mass:bool=True
54     pressure:float=101325.
55     Lewis:float=.845
56     gas_transfer_scale:float=1.
57     equilibrium_C50:float=.075
58     desorption_B:float=0.
59     emissivity:float=0.
60     wall_offset:float=0.
61     tail_T_shift:float=0.
62     tail_w_shift:float=0.
63     D_scale:float=1.
64     latent_fraction:float=1. # !=1 is an explicitly nonphysical algorithmic ablation
65     energy_gauge:float=0.
66     rtol:float=2e-7
67     atol_H:float=1e-3
68     atol_C:float=1e-9
69     max_step:float=120.
70
71
72 def load_input(path):
73     a=[]
74     for name in ('附件 1.xlsx','附件 2.xlsx'):
75         w=load_workbook(Path(path)/name,read_only=True,data_only=True)
76         x=np.array(list(w.active.values)[1:],float);w.close()
77         if not np.isfinite(x).all() or not np.all(np.diff(x[:,0])>0):raise ValueError('Invalid
            supplied data')
78         a.append(x)
79     return a
80
81
82 class Model:
83     def __init__(self,cfg,env,rad):
84         self.c=cfg;self.env_data=env;self.rad_data=rad
85         if cfg.question not in (1,2,3,4) or cfg.nr<2 or cfg.nz<0:raise ValueError('Invalid
            configuration')
86         if not 0<=cfg.emissivity<=1 or cfg.desorption_B<0 or cfg.equilibrium_C50<=0:raise
            ValueError('Unphysical closure parameter')
87         self.tail=env[env[:,0]>=10800,1:].mean(axis=0)
88         self.x=1-(1-np.linspace(0,1,cfg.nr+1))*cfg.radial_power
89         self.z=1-(1-np.linspace(0,1,cfg.nz+1))*cfg.axial_power if cfg.nz else np.array([0.])
90         self.rf=np.r_[0,(self.x[:-1]+self.x[1:])/2,1.]

```

```

91     zf=np.r_[0,(self.z[:-1]+self.z[1:])/2,1.]
92     self.vr=np.diff(self.rf**2)/2;self.vz=np.diff(zf)
93     self.weights=2*self.vr[:,None]*self.vz[None,:]
94     self.shape=(cfg.nr+1,cfg.nz+1,2);self.size=int(np.prod(self.shape))
95     self.cs,self.cw={1:(2600.,2600.),2:(1450.,4186.),3:(1450.,4186.),4:(1850.,4000.)}[cfg.question]
96     rho0={1:820.,2:650+128*cfg.C0,3:650+128*cfg.C0,4:760+90*cfg.C0}[cfg.question]
97     self.rhod0=rho0/(1+cfg.C0)
98     self.Mdry=self.rhod0*np.pi*cfg.R0**2*cfg.length0
99     # Parameterize at the actual declared late ambient temperature; no fit.
100    Ta=self.tail[0];wa=self.tail[1]
101    rh=self.vapor_pressure(wa)/psat(Ta)
102    self.s=cfg.equilibrium_C50*(np.exp(self.thermal_log_aw(Ta))/rh-1)
103    if self.s<=0:raise ValueError('Selected reference equilibrium incompatible with activity closure')
104    ar=diags([np.ones(cfg.nr),np.ones(cfg.nr)],[-1,1],shape=(cfg.nr+1,cfg.nr+1))
105    adj=eye((cfg.nr+1)*(cfg.nz+1))+kron(ar,eye(cfg.nz+1))
106    if cfg.nz:
107        az=diags([np.ones(cfg.nz),np.ones(cfg.nz)],[-1,1],shape=(cfg.nz+1,cfg.nz+1))
108        adj+=kron(eye(cfg.nr+1),az)
109    self.sparsity=kron(adj,np.ones((2,2))).tocsc()
110    self.guard_calls=0
111
112    def thermal_log_aw(self,T):
113        K=np.asarray(T)+273.15
114        return (CPL-self.cw)/RV*(np.log(K/TR)+TR/K-1)+self.c.desorption_B/RV*(1/TR-1/K)
115
116    def activity(self,T,C):
117        return np.asarray(C)/(np.asarray(C)+self.s)*np.exp(self.thermal_log_aw(T))
118
119    def vapor_pressure(self,w):return self.c.pressure*w/(.62198+w)
120
121    def env(self,t):
122        if t<=self.env_data[-1,0]:return
123        tuple(float(np.interp(t,self.env_data[:,0],self.env_data[:,j])) for j in (1,2))
124        return self.tail[0]+self.c.tail_T_shift,self.tail[1]+self.c.tail_w_shift
125
126    def geometry(self,t):
127        c=self.c
128        if c.radius_mode=='fixed':R=c.R0
129        elif c.radius_mode=='observed':
130            if t>self.rad_data[-1,0]+1e-6:raise ValueError('No radius extrapolation beyond supplied72h')
131            R=float(np.interp(t,self.rad_data[:,0],self.rad_data[:,1]))/100
132        else:raise ValueError('Unknown geometry mode')
133        length=c.length0*(R/c.R0)**c.axial_exponent
134        rhod=self.rhod0*(c.R0/R)**2*c.length0/length
135        return R,length/2,rhod

```

```

135
136 def enthalpy(self,T,C):return (self.cs+self.cw*np.asarray(C))*(np.asarray(T)-28)+(self.c.energy_gauge-self.c.desorption_B)*np.asarray(C)
137 def bound_h(self,T):return self.cw*(np.asarray(T)-28)+self.c.energy_gauge-self.c.desorption_B
138 def physical(self,y):
139     u=np.asarray(y).reshape(self.shape);C=u[:, :, 1]
140     T=28+(u[:, :, 0]-(self.c.energy_gauge-self.c.desorption_B)*C)/(self.cs+self.cw*C)
141     return T,C
142 def initial(self):return
143     np.tile([float(self.enthalpy(self.c.T0,self.c.C0)),self.c.C0],self.size//2)
144
145 def transport(self,T,C):
146     cc=np.maximum(C,1e-12)
147     a,b,E={1:(.89,7e-9,0.),2:(.45,2.4e-3,3850.),3:(.45,2.4e-3,3850.),4:(.30,4.2e-4,3850.)}[self.c.question]
148     return cc*np.exp(-a/cc)+a*exp(-a/cc),lambda tf:b*self.c.D_scale*np.exp(-E/(tf+273.15))
149
150 def conductivity(self,C):
151     q=self.c.question
152     if q==1:return np.full_like(C,.36)
153     b,a=(.21,.38) if q in (2,3) else (.12,.20)
154     return b+a*C/(1+C)
155
156 def surface(self,t,T,C):
157     """Outward physical mass J and total enthalpy flux, upstream vapor h."""
158     c=self.c;Ta,wa=self.env(t);Tf=(T+Ta)/2+273.15
159     pa=self.vapor_pressure(wa);aw=self.activity(T,np.maximum(C,1e-12));pw=aw*psat(T)
160     rhoair=c.pressure/(RDA*Tf)*(1-.378*pa/c.pressure)
161     cpair=(1006+CPV*wa)/(1+wa)
162     kg=c.gas_transfer_scale*c.h/(rhoair*cpair*c.Lewis**(2/3))
163     beta=kg/(RV*Tf)
164     J=beta*(pw-pa)
165     source_T=np.where(J>=0,T,Ta)
166     vapor_h=hv(source_T)+c.energy_gauge
167     hb=self.bound_h(T)
168     carry=hb+c.latent_fraction*(vapor_h-hb)
169     convection=c.h*(T-Ta)
170     radiation=c.emissivity*SIGMA*((T+273.15)**4-(Ta+c.wall_offset+273.15)**4)
171     energy=convection+radiation+carry*J
172     # Full film entropy from ideal-vapor chemical potential; gauge cancels.
173     Ks=T+273.15;Ka=Ta+273.15
174     hv_integral=(LREF-CPV*TR)*(1/Ks-1/Ka)+CPV*np.log(Ka/Ks)
175     sigma=convection*(1/Ka-1/Ks)+J*(RV*np.log(np.maximum(pw,1e-100)/pa)+hv_integral+hv(source_T)*(1/Ka-1/Ks))
176     sigma+=radiation*(1/(Ta+c.wall_offset+273.15)-1/Ks)
177     return J,energy,convection,radiation,carry*J,aw,sigma
178
179 def fluxes(self,t,y):

```

```

179     T,C=self.physical(y);c=self.c;R,H,rhod=self.geometry(t)
180     if C.min()<1e-12:self.guard_calls+=1
181     phi,b=self.transport(T,C);k=self.conductivity(np.maximum(C,0))
182     fr=np.zeros((c.nr+2,c.nz+1));fz=np.zeros((c.nr+1,c.nz+2))
183     qr=np.zeros_like(fr);qz=np.zeros_like(fz)
184     Tr=(T[:-1]+T[1:])/2
185     fr[1:-1]=self.rf[1:-1,None]*b(Tr)*np.diff(phi,axis=0)/np.diff(self.x)[: ,None]
186     kr=2*k[:-1]*k[1:]/(k[:-1]+k[1:])
187     qr[1:-1]=self.rf[1:-1,None]*kr*np.diff(T,axis=0)/np.diff(self.x)[: ,None]/rhod+self.bound_
188     h(Tr)*fr[1:-1]
189     side=self.surface(t,T[-1],C[-1]);end=self.surface(t,T[: ,-1],C[: ,-1])
190     fr[-1]=-R*side[0]/rhod;qr[-1]=-R*side[1]/rhod
191     if c.nz:
192         Tz=(T[:, :-1]+T[:, 1:])/2
193         fz[:, 1:-1]=b(Tz)*np.diff(phi,axis=1)/np.diff(self.z)[None, :]
194         kz=2*k[:, :-1]*k[:, 1:]/(k[:, :-1]+k[:, 1:])
195         qz[:, 1:-1]=kz*np.diff(T,axis=1)/np.diff(self.z)[None, :]/rhod+self.bound_h(Tz)*fz[:, 1:-1]
196         if c.end_mass:fz[:, -1]=-H*end[0]/rhod
197         qout=(end[2]+end[3] if c.end_heat else 0)+(end[4] if c.end_mass else 0)
198         qz[:, -1]=-H*qout/rhod
199     return fr,fz,qr,qz,R,H,rhod,side,end
200
201 def divergence(self,fr,fz,R,H):return
202     np.diff(fr,axis=0)/(self.vr[:,None]*R**2)+np.diff(fz,axis=1)/(self.vz[None,:]*H**2)
203
204 def rhs(self,t,y):
205     fr,fz,qr,qz,R,H,*_=self.fluxes(t,y)
206     out=np.empty(self.shape);out[:, :,0]=self.divergence(qr,qz,R,H);out[:, :,1]=self.divergence_
207     (fr,fz,R,H)
208     return out.ravel()
209
210 def boundary_budget(self,t,y):
211     T,C=self.physical(y);R,H,rho=self.geometry(t);c=self.c
212     side=self.surface(t,T[-1],C[-1]);end=self.surface(t,T[: ,-1],C[: ,-1])
213     # Independently integrate physical surface fluxes, normalized by dry mass.
214     As=4*np.pi*R*H;Ae=2*np.pi*R**2
215     avs=lambda v:float(v@self.vz)
216     ave=lambda v:float(2*self.vr@v)
217     vals=[]
218     for idx in (0,1,2,3,4):
219         se=avs(side[idx]);ee=ave(end[idx]) if c.nz else 0.
220         if idx==0 and not c.end_mass:ee=0.
221         if idx in (2,3) and not c.end_heat:ee=0.
222         if idx==4 and not c.end_mass:ee=0.
223         if idx==1 and c.nz:ee=(ave(end[2]+end[3]) if c.end_heat else 0)+(ave(end[4]) if
224             c.end_mass else 0)
225         vals.append((As*se+Ae*ee)/self.Mdry)
226     return np.array(vals)

```

```

223
224 def solve(model,end_s,save_step=600.,full_times=(),threshold=False):
225     c=model.c;y0=model.initial();beg=time.time()
226     solver=BDF(model.rhs,0.,y0,end_s,rtol=c.rtol,atol=np.tile([c.atol_H,c.atol_C],model.size//2),,
        jac_sparsity=model.sparsity,max_step=c.max_step)
227     times=np.unique(np.r_[np.arange(0,end_s+1e-6,save_step),end_s,np.array(full_times)[np.array(f,
        ull_times)<=end_s]])
228     records=[];states=[];full=[];full_t=[];cum=np.zeros(5);cum4=np.zeros(5);budgets=[]
229     g,w=leggauss(8);g4,w4=leggauss(4);initialmean=np.sum(model.weights[:, :,None]*y0.reshape(model,
        .shape),axis=(0,1))
230     max_res=np.zeros(2);maxquad=0.;minC=c.C0;Tmin=c.T0;Tmax=c.T0;maxaw=0.;maxaw_all=0.;minsigma=0,
        .;critical=None;stop=None;stopstate=None
231     def save(t,y):
232         T,C=model.physical(y);R,H,_=model.geometry(t)
233         records.append([t,R,H,float((model.weights*C).sum()),float((model.weights*T).sum()),float(
        (C.max()),float(T.min()),float(T.max()))])
234         states.append(np.stack((T[:,0],C[:,0]),axis=-1))
235         if np.any(np.isclose(t,full_times,atol=1e-7,rtol=0)):
236             full.append(np.stack((T,C),axis=-1));full_t.append(t)
237     save(0.,y0);nexti=1;previousmax=c.C0;steps=0
238     while solver.status=='running':
239         t0=solver.t;solver.step();steps+=1
240         if solver.status=='failed':raise RuntimeError('BDF failed')
241         t1=solver.t;dense=solver.dense_output();T,C=model.physical(solver.y)
242         if not np.isfinite(solver.y).all():raise RuntimeError('Nonfinite state')
243         minC=min(minC,float(C.min()));Tmin=min(Tmin,float(T.min()));Tmax=max(Tmax,float(T.max()))
244         maxaw_all=max(maxaw_all,float(np.max(model.activity(T,C))))
245         for ts,cs in [(T[-1],C[-1]),(T[:,-1],C[:,-1])]:
246             surf=model.surface(t1,ts,cs);maxaw=max(maxaw,float(np.max(surf[5])));minsigma=min(min,
                sigma,float(np.min(surf[6])))
247         currentmax=float(C.max())
248         if previousmax>.15 and currentmax<=.15 and critical is None:
249             critical=float(brentq(lambda t:model.physical(dense(t))[1].max()- .15,t0,t1,xtol=1e-6),
                );stop=int(np.floor(critical))+1
250         if stop is not None and t0<=stop<=t1 and stopstate is None:
251             stopstate=np.stack(model.physical(dense(stop)),axis=-1)
252             if not float(stopstate[:, :,1].max())<.15:raise RuntimeError("Detected crossing failed
                strict integer threshold check")
253         cuts=np.unique(np.r_[t0,model.env_data[:,0][(model.env_data[:,0]>t0)&(model.env_data[:,0],
                <t1)],model.rad_data[:,0][(model.rad_data[:,0]>t0)&(model.rad_data[:,0]<t1)],t1))
254         for left,right in zip(cuts[:-1],cuts[1:]):
255             mid=(left+right)/2;half=(right-left)/2
256             cum+=half*sum(ww*model.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
                zip(g,w))
257             cum4+=half*sum(ww*model.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
                zip(g4,w4))
258         mean=np.sum(model.weights[:, :,None]*solver.y.reshape(model.shape),axis=(0,1))

```

```

259     residual=np.array([mean[1]-initialmean[1]+cum[0],mean[0]-initialmean[0]+cum[1]])
260     max_res=np.maximum(max_res,np.abs(residual));maxquad=max(maxquad,float(np.max(np.abs(cum-
    cum4)/np.array([1,1e6,1e6,1e6,1e6]))))
261     budgets.append(np.r_[t1,mean,cum,residual])
262     while nexti<len(times) and times[nexti]<=t1+1e-7:
263         if threshold and stop is not None and times[nexti]>stop:break
264         save(float(times[nexti]),dense(float(times[nexti])));nexti+=1
265     if threshold and stopstate is not None:
266         if records[-1][0]!=stop:save(float(stop),dense(float(stop)))
267         break
268     previousmax=currentmax
269 arr={'records':np.array(records),'middle':np.array(states),'full_times':np.array(full_t),'ful
    l_states':np.array(full),'x':model.x,'z':model.z,'weights':model.weights,'budget_steps':np.a
    rray(budgets),'stop_state':np.array([]) if stopstate is None else stopstate}
270 diag={'config':asdict(c),'conditional_parameters_not_calibrated':True,'closure_s':float(model
    .s),'rhod0':model.rhod0,'dry_mass_kg':model.Mdry,'elapsed_s':time.time()-beg,'steps':steps,'
    nfev':solver.nfev,'njev':solver.njev,'nlu':solver.nlu,'critical_h':None if critical is None
    else critical/3600,'strict_stop_s':stop,'stop_Cmax':None if stopstate is None else
    float(stopstate[:, :, 1].max()),'strict_stop_verified':bool(stopstate is not None and
    stopstate[:, :, 1].max())<.15),'saved_Cmax_end':records[-1][5],'Cmean_end':records[-1][3],'T_ra
    nge_C':[Tmin,Tmax],'min_C':minC,'max_surface_activity':maxaw,'max_all_activity':maxaw_all,'m
    in_boundary_entropy_W_m2K':minsigma if c.latent_fraction==1 else
    None,'mass_residual_max':float(max_res[0]),'enthalpy_residual_max_J_perkgdry':float(max_res[
    1]),'quadrature_scaled_difference':maxquad,'guard_calls':model.guard_calls,'integrated_until
    _s':solver.t,'budget_channels':['time','mean_H','mean_C','cum_mass_out','cum_total_energy_ou
    t','cum_convection_out','cum_radiation_out','cum_water_enthalpy_out','mass_residual','enthal
    py_residual'],'scope':'Matched conditional thermodynamic model if latent_fraction=1; other
    values are nonphysical energy-coupling ablations, not alternative calibrated materials.')}
271 diag['mathematical_admissible'] = bool(minC>=-1e-8 and maxaw_all<=1+1e-8 and Tmin>=.01 and
    Tmax<=55 and (c.latent_fraction!=1 or minsigma>=-1e-8))
272 diag['physical_family_member'] = bool(diag['mathematical_admissible'] and
    c.latent_fraction==1)
273 diag['net_desorption_min_J_kg'] =
    float(c.desorption_B+min((CPL-model.cw)*(Tmin-28),(CPL-model.cw)*(Tmax-28)))
274 return arr,diag

```

F.23 volume_fem.py

类别：守恒变长度模型 A。原始代码共 317 行。

```

1 """Conditional constituent-volume closure, axisymmetric material-coordinate Q1 FEM.
2
3 The copied thermo_reference is used only for thermodynamic and boundary functions.
4 No former FVM geometry or flux implementation is used. Observed radius is validation
5 data, never a prescribed geometry. No nodal moisture clipping is performed.
6 """
7 from dataclasses import dataclass, asdict
8 import time
9 import numpy as np

```

```

10 from numpy.polynomial.legendre import leggauss
11 from scipy.integrate import BDF
12 from scipy.optimize import brentq
13 from scipy.special import expi
14 from .thermo_reference import Config as ThermalConfig, Model as ThermalModel, load_input, psat,
    RV, TR
15
16
17 @dataclass(frozen=True)
18 class Config:
19     nr: int = 12
20     nz: int = 6
21     radial_power: float = 1.5
22     axial_power: float = 2.0
23     shrink_beta: float = .5
24     volume_mode: str = 'linear_beta'
25     water_density: float = 1000.
26     quadrature_order: int = 3
27     question: int = 4
28     R0: float = .02
29     length0: float = .25
30     C0: float = 2.55
31     T0: float = 28.
32     desorption_B: float = 20000.
33     equilibrium_C50: float = .075
34     emissivity: float = 0.
35     h: float = 25.
36     D_scale: float = 1.
37     gas_transfer_scale: float = 1.
38     energy_gauge: float = 0.
39     rtol: float = 2e-6
40     atol_H: float = .01
41     atol_C: float = 1e-8
42     max_step: float = 300.
43     boundary_mode: str = 'ambient'
44
45
46 class Model:
47     def __init__(self, cfg, env, radius):
48         self.c=cfg
49         if not 0<=cfg.shrink_beta<=1 or cfg.water_density<=0:
50             raise ValueError('Declared no-initial-pore-collapse family requires beta in [0,1]')
51         if cfg.volume_mode not in ('linear_beta','given_density'):raise ValueError('Unknown volume
            closure')
52         if cfg.nr<2 or cfg.nz<2 or cfg.quadrature_order<2: raise ValueError('Invalid mesh')
53         tc=ThermalConfig(question=cfg.question,nr=cfg.nr,nz=cfg.nz,R0=cfg.R0,length0=cfg.length0,
54             C0=cfg.C0,T0=cfg.T0,desorption_B=cfg.desorption_B,
55             equilibrium_C50=cfg.equilibrium_C50,emissivity=cfg.emissivity,h=cfg.h,

```

```

56         D_scale=cfg.D_scale,gas_transfer_scale=cfg.gas_transfer_scale,
57         energy_gauge=cfg.energy_gauge,radius_mode='fixed')
58     self.th=ThermalModel(tc,env,radius)
59     if cfg.boundary_mode=='equilibrium_initial':
60         p=float(self.th.activity(cfg.T0,cfg.C0)*psat(cfg.T0))
61         w=.62198*p/(tc.pressure-p)
62         self.th.env=lambda t,te=cfg.T0,we=w:(te,we)
63     elif cfg.boundary_mode!='ambient': raise ValueError('Unknown boundary mode')
64     self.rhod0=self.th.rhod0;self.Mdry=self.th.Mdry;self.H0=cfg.length0/2
65     self.rho_a,self.rho_b={1:(820.,0.),2:(650.,128.),3:(650.,128.),4:(760.,90.)}[cfg.question]
66     self.A=cfg.R0*(1-(1-np.linspace(0,1,cfg.nr+1))**cfg.radial_power)
67     self.Z=self.H0*(1-(1-np.linspace(0,1,cfg.nz+1))**cfg.axial_power)
68     self.shape=(cfg.nr+1,cfg.nz+1);self.nnode=int(np.prod(self.shape));self.size=2*self.nnode
69     self.mesh_index=np.arange(self.nnode).reshape(self.shape)
70     i,j=np.meshgrid(np.arange(cfg.nr),np.arange(cfg.nz),indexing='ij')
71     self.ei=i.ravel();self.ej=j.ravel()
72     self.conn=np.stack([self.mesh_index[i,j],self.mesh_index[i+1,j],self.mesh_index[i+1,j+1],
73         self.mesh_index[i,j+1]],axis=-1).reshape(-1,4)
74     g,w=leggauss(cfg.quadrature_order);g=(g+1)/2;w=w/2
75     xi,eta=np.meshgrid(g,g,indexing='ij');wx,wy=np.meshgrid(w,w,indexing='ij')
76     self.xi=xi.ravel();self.eta=eta.ravel();self.qw=(wx*wy).ravel()
77     x=self.xi;y=self.eta
78     self.N=np.stack([(1-x)*(1-y),x*(1-y),x*y,(1-x)*y],axis=-1)
79     dNx=np.stack([- (1-y),1-y,y,-y],axis=-1)
80     dNy=np.stack([- (1-x),-x,x,1-x],axis=-1)
81     da=np.diff(self.A)[self.ei];dz=np.diff(self.Z)[self.ej]
82     self.gradN=np.stack([dNx[None,:],:/da[:,None,None],dNy[None,:],:/dz[:,None,None]],axis=-1)
83     self.aq=self.A[self.ei,None]+da[:,None]*self.xi
84     self.zq=self.Z[self.ej,None]+dz[:,None]*self.eta
85     # 2 pi A times factor 2 for the mirrored half-cylinder.
86     self.weights=4*np.pi*self.aq*da[:,None]*dz[:,None]*self.qw
87     local_mass=self.rhod0*np.einsum('eq,qk->ek',self.weights,self.N)
88     self.mass=np.bincount(self.conn.ravel(),weights=local_mass.ravel(),minlength=self.nnode)
89     assert np.isclose(self.mass.sum(),self.Mdry,rtol=5e-14)
90     self.side_conn=np.stack([self.mesh_index[-1,:-1],self.mesh_index[-1,1:]],axis=-1)
91     self.end_conn=np.stack([self.mesh_index[:-1,-1],self.mesh_index[1:,-1]],axis=-1)
92     self.bN=np.stack([1-g,g],axis=-1);self.bg=g;self.bw=w
93     self.radius_data=radius
94     self.call_count=0
95     self.a0=self.rhod0*cfg.C0/cfg.water_density
96     if self.a0>=1:raise ValueError('Initial pure water volume already fills sample')
97     if cfg.volume_mode=='given_density':
98         f0=1/self.rho_a
99         fc0=(1+cfg.C0)/(self.rho_a+self.rho_b*cfg.C0)-cfg.C0/cfg.water_density
100         self.solid_specific_volume_max=min(f0,fc0)
101     else:self.solid_specific_volume_max=(1-self.a0)/self.rhod0
102     if self.solid_specific_volume_max<=0:raise ValueError('No common finite solid-density
    volume family')

```

```

102     self.solid_reference_fraction=self.rhod0*self.solid_specific_volume_max
103     self.phi0_min=1-self.solid_reference_fraction-self.a0
104
105     def unpack(self,y):
106         u=np.asarray(y).reshape(-1,2);H=u[:,0];C=u[:,1]
107         T=28+(H-(self.c.energy_gauge-self.c.desorption_B)*C)/(self.th.cs+self.th.cw*C)
108         return H,C,T
109
110     def initial(self):
111         return np.tile([float(self.th.enthalpy(self.c.T0,self.c.C0)),self.c.C0],self.nnode)
112
113     def geometry_data(self,C):
114         c=self.c;C=np.asarray(C).reshape(self.shape)
115         J=self.volume_law(C)
116         if np.min(J)<=0:raise ValueError('Nonpositive constituent volume')
117         axial_int=np.cumsum((J[:, :-1]+J[:, 1:])/2*np.diff(self.Z),axis=1)
118         K=np.c_[np.zeros(c.nr+1),axial_int]
119         bar=K[:, -1]/self.H0
120         da=np.diff(self.A);slope=np.diff(bar)/da;a=self.A[-1]
121         pieces=bar[: -1]*(a*da+da**2/2)+slope*(a*da**2/2+da**3/3)
122         I=np.r_[0.,np.cumsum(pieces)]
123         return {'J':J,'K':K,'bar':bar,'I':I,'R':float(np.sqrt(2*I[-1]))}
124
125     def volume_law(self,C):
126         if self.c.volume_mode=='given_density':return self.rhod0*(1+C)/(self.rho_a+self.rho_b*C)
127         return 1-self.c.shrink_beta*self.rhod0/self.c.water_density*(self.c.C0-C)
128
129     def map_points(self, gd, i, j, xi, eta):
130         # i,j shape (elements,1); xi,eta broadcast against them.
131         a0=self.A[i];z0=self.Z[j];da=self.A[i+1]-a0;dz=self.Z[j+1]-z0
132         a=a0+da*xi;dza=dz*eta;du=da*xi
133         Jn=gd['J'];K=gd['K'];bn=gd['bar'];In=gd['I']
134         b0=bn[i];ba=(bn[i+1]-b0)/da;b=b0+ba*du
135         I=In[i]+b0*(a0*du+du**2/2)+ba*(a0*du**2/2+du**3/3)
136         r=np.sqrt(2*I)
137         ra=np.divide(a*b,r,out=np.sqrt(b).copy()+np.zeros_like(r),where=r>0)
138         Jlo=(1-xi)*Jn[i,j]+xi*Jn[i+1,j]
139         Jhi=(1-xi)*Jn[i,j+1]+xi*Jn[i+1,j+1]
140         Jq=Jlo+(Jhi-Jlo)*eta
141         Iz=(1-xi)*K[i,j]+xi*K[i+1,j]+Jlo*dza+.5*(Jhi-Jlo)/dz*dza**2
142         Ja0=(Jn[i+1,j]-Jn[i,j])/da;Ja1=(Jn[i+1,j+1]-Jn[i,j+1])/da
143         Iza=(K[i+1,j]-K[i,j])/da+Ja0*dza+.5*(Ja1-Ja0)/dz*dza**2
144         z=Iz/b;za=Iza/b-Iz*ba/b**2;zz=Jq/b
145         m11=1/ra**2;m12=-za/(ra**2*zz);m22=(za/(ra*zz))**2+1/zz**2
146         metric=np.stack([m11,m12,m12,m22],axis=-1).reshape(Jq.shape+(2,2))
147         hoop=np.divide(r,a,out=np.sqrt(b).copy()+np.zeros_like(r),where=a>0)
148         det=hoop*ra*zz
149         return {'J':Jq,'bar':b,'r':r,'z':z,'ra':ra,'za':za,'zz':zz,'metric':metric,'det':det}

```

```

150
151 def material_fields(self,y,strict=True):
152     H,C,T=self.unpack(y)
153     if not np.isfinite(y).all():raise ValueError('Nonfinite state')
154     if strict and np.min(C)<=0:raise ValueError(f'Nonpositive moisture trial state: {C.min()}')
155     if strict and np.max(C)>self.c.C0+1e-7:raise ValueError(f'Outside declared drying volume
        law C<=C0: {C.max()}')
156     if np.min(T)<=-273.15:raise ValueError('Nonpositive absolute temperature')
157     if np.min(T)<.01 or np.max(T)>55:raise ValueError(f'Outside verified thermal domain
        [.01,55]C: {T.min()}, {T.max()}')
158     return H,C,T
159
160 def boundary(self,t,C,T,gd):
161     c=self.c
162     si=np.full((c.nz,1),c.nr-1);sj=np.arange(c.nz)[: ,None]
163     sidegeo=self.map_points(gd,si,sj,np.ones((1,len(self.bg))),self.bg[None,:])
164     ei=np.arange(c.nr)[: ,None];ej=np.full((c.nr,1),c.nz-1)
165     endgeo=self.map_points(gd,ei,ej,self.bg[None,:],np.ones((1,len(self.bg))))
166     out=[]
167     for conn,geo,dL,kind in [(self.side_conn,sidegeo,np.diff(self.Z),'side'),(self.end_conn,ej,
        ndgeo,np.diff(self.A),'end')]:
168         Cq=C[conn]@self.bN.T;Tq=T[conn]@self.bN.T
169         vals=self.th.surface(t,Tq,Cq)
170         if kind=='side':W=4*np.pi*gd['R']*geo['zz']*dL[: ,None]*self.bw
171         else:W=4*np.pi*(self.A[: -1,None]+dL[: ,None]*self.bg)*geo['bar']*dL[: ,None]*self.bw
172         out.append((conn,W,vals))
173     return out
174
175 def terms(self,t,y,need_checks=False):
176     H,C,T=self.material_fields(y);gd=self.geometry_data(C)
177     geo=self.map_points(gd,self.ei[: ,None],self.ej[: ,None],self.xi[None,:],self.eta[None,:])
178     Cn=C[self.conn];Tn=T[self.conn];Cq=Cn@self.N.T;Tq=Tn@self.N.T
179     a,b,E=[1:(.89,7e-9,0.),2:(.45,2.4e-3,3850.),3:(.45,2.4e-3,3850.),4:(.30,4.2e-4,3850.)][self.c.question]
180     phi=Cn*np.exp(-a/Cn)+a*exp(-a/Cn)
181     gradphi=np.einsum('eqkd,ek->eqd',self.gradN,phi)
182     gradT=np.einsum('eqkd,ek->eqd',self.gradN,Tn)
183     pref=b*self.c.D_scale*np.exp(-E/(Tq+273.15))
184     mass_flux=-self.rhod0*pref[: ,: ,None]*np.einsum('eqab,eqb->eqa',geo['metric'],gradphi)
185     heat_flux=-geo['J'][: ,: ,None]*self.th.conductivity(Cq)[: ,: ,None]*np.einsum('eqab,eqb->eqa',
        geo['metric'],gradT)
186     total_flux=heat_flux+self.th.bound_h(Tq)[: ,: ,None]*mass_flux
187     lm=np.einsum('eqkd,eqd,eq->ek',self.gradN,mass_flux,self.weights)
188     lh=np.einsum('eqkd,eqd,eq->ek',self.gradN,total_flux,self.weights)
189     fm=np.bincount(self.conn.ravel(),weights=lm.ravel(),minlength=self.nnnode)
190     fh=np.bincount(self.conn.ravel(),weights=lh.ravel(),minlength=self.nnnode)
191     kelvin=T+273.15
192     mu_over_T=(self.th.cw*((T-28)/kelvin-np.log(kelvin/TR))

```

```

193         +self.c.desorption_B*(1/TR-1/kelvin)+RV*np.log(C/(C+self.th.s))
194         +self.c.energy_gauge/kelvin)
195     internal_entropy=float(np.sum(fh/kelvin-mu_over_T*fm))
196     budgets=np.zeros(5);boundary_min_sigma=np.inf;boundary_maxaw=0.
197     for conn,W,vals in self.boundary(t,C,T,gd):
198         for f,idx in [(fm,0),(fh,1)]:
199             local=np.einsum('eq,eq,qk->ek',W,vals[idx],self.bN)
200             f-=np.bincount(conn.ravel(),weights=local.ravel(),minlength=self.nnode)
201             for k,idx in enumerate((0,1,2,3,4)):budgets[k]+=np.sum(W*vals[idx])
202             boundary_min_sigma=min(boundary_min_sigma,float(np.min(vals[6])))
203             boundary_maxaw=max(boundary_maxaw,float(np.max(vals[5])))
204     out=np.stack([fh/self.mass,fm/self.mass],axis=-1).ravel()
205     check=None
206     if need_checks:
207         Jn=gd['J'].ravel();amin=np.min(self.th.activity(T,C));amax=np.max(self.th.activity(T,C))
208         alpha_w=self.rhod0*C/(self.c.water_density*Jn)
209         phi_numer=Jn-(1-self.a0)-self.rhod0*C/self.c.water_density
210         phi_q=geo['J']-(1-self.a0)-self.rhod0*Cq/self.c.water_density
211         feasible_air=Jn-self.solid_reference_fraction-self.rhod0*C/self.c.water_density
212         feasible_air_q=geo['J']-self.solid_reference_fraction-self.rhod0*Cq/self.c.water_density
213         desired_J=self.volume_law(Cq)
214         constituent_error=float(np.max(abs(geo['J']-desired_J)/desired_J))
215         rho_given=self.rho_a+self.rho_b*Cq
216         rho_error=float(np.max(abs(self.rhod0*(1+Cq)/geo['J']-rho_given)/rho_given)) if
217             self.c.volume_mode=='given_density' else None
218         volume_quad=float(np.sum(self.weights*geo['J']))
219         volume_cyl=np.pi*gd['R']**2*self.c.length0
220         check={'Cmin':float(C.min()),'Cmax':float(C.max()),'Tmin':float(T.min()),'Tmax':float(
221             T.max()),
222             'min_J':float(geo['J'].min()),'max_det_error':float(np.max(abs(geo['J']-geo['d
223             et']))),
224             'volume_quad_m3':volume_quad,'volume_cylinder_m3':volume_cyl,
225             'relative_volume_error':float(abs(volume_quad-volume_cyl)/volume_cyl),
226             'max_water_volume_fraction':float(alpha_w.max()),
227             'min_air_fraction_for_phi0_zero':float(np.min(phi_numer/Jn)),
228             'min_air_fraction_q_for_phi0_zero':float(np.min(phi_q/geo['J'])),
229             'min_air_fraction_for_feasible_phi0_min':float(np.min(feasible_air/Jn)),
230             'min_air_fraction_q_for_feasible_phi0_min':float(np.min(feasible_air_q/geo['J'])),
231             'constitutive_volume_relative_error':constituent_error,
232             'rho_mixture_relative_error':0. if rho_error is None else rho_error,
233             'max_volume_quadrature_activity':float(np.max(self.th.activity(Tq,Cq))),
234             'internal_entropy_W_K':internal_entropy,
235             'min_activity':float(amin),'max_activity':float(amax),
236             'min_boundary_entropy':boundary_min_sigma,'max_boundary_activity':boundary_maxaw,
237             'instant_mass_balance_kg_s':float(fm.sum()+budgets[0]),
238             'instant_enthalpy_balance_W':float(fh.sum()+budgets[1]),
239             'max_metric_cross_relative':float(np.max(abs(geo['metric'][:, :, 0, 1])/np.sqrt(g,
240             eo['metric'][:, :, 0, 0]*geo['metric'][:, :, 1, 1])))}

```

```

237         return out,budgets/self.Mdry,gd,check
238
239     def rhs(self,t,y):
240         self.call_count+=1
241         return self.terms(t,y)[0]
242
243     def boundary_budget(self,t,y):
244         _,C,T=self.material_fields(y);gd=self.geometry_data(C);b=np.zeros(5)
245         for _,W,v in self.boundary(t,C,T,gd):
246             b+=np.array([np.sum(W*v[k]) for k in (0,1,2,3,4)])
247         return b/self.Mdry
248
249     def record(self,t,y):
250         _,C,T=self.material_fields(y);gd=self.geometry_data(C)
251         return [t,gd['R'],np.dot(self.mass,C)/self.Mdry,np.dot(self.mass,T)/self.Mdry,C.max(),C.min(),T.min(),T.max()]
252
253
254     def solve(model,end_s,save_step=600.):
255         c=model.c;beg=time.time();y0=model.initial()
256         solver=BDF(model.rhs,0.,y0,end_s,rtol=c.rtol,atol=np.tile([c.atol_H,c.atol_C],model.nnode),max_step=c.max_step)
257         extra=np.array([0,1,10,60,300,1800,3600,10800,14400,21600,43200,86400,172800,259200])
258         targets=np.unique(np.r_[np.arange(0,end_s,save_step),end_s,extra[extra<=end_s]])
259         ts=[];ys=[];records=[];budgets=[];checks=[];cursor=0;steps=0
260         cum=np.zeros(5);cum4=np.zeros(5);g,w=leggauss(8);g4,w4=leggauss(4)
261         mean0=np.sum(model.mass[:,None]*y0.reshape(-1,2),axis=0)/model.Mdry
262         critical=None;stop_s=None;stop_state=None;stop_Cmax=None
263         max_res=np.zeros(2);maxquad=0.;last_Cmax=c.C0
264         knots=model.th.env_data[:,0]
265         while solver.status=='running':
266             told=solver.t;msg=solver.step();steps+=1
267             if solver.status=='failed':raise RuntimeError(msg)
268             t=solver.t;dense=solver.dense_output()
269             _,_,_,check=model.terms(t,solver.y,True)
270             if max(check['max_activity'],check['max_boundary_activity'],check['max_volume_quadrature_activity'])>1+1e-10:raise ValueError(f'Activity exceeds 1: {check}')
271             if check['min_boundary_entropy'] < -1e-8:raise ValueError(f'Negative boundary entropy: {check}')
272             if check['max_water_volume_fraction']>1+1e-10 or min(check['min_air_fraction_for_feasible_phi0_min'],check['min_air_fraction_q_for_feasible_phi0_min'])<-1e-9:raise ValueError('Constituent volume violation')
273             if check['relative_volume_error']>1e-11 or check['max_det_error']>1e-11:raise ValueError('Geometry integration failure')
274             check['time_s']=t;checks.append(check)
275             if critical is None and last_Cmax>=.15 and check['Cmax']<.15:
276                 critical=brentq(lambda tt:model.unpack(dense(tt))[1].max()-.15,told,t)

```

```

277         stop_s=int(np.floor(critical))+1
278     if stop_s is not None and stop_state is None and told<=stop_s<=t:
279         stop_state=dense(stop_s).reshape(-1,2).copy()
280         stop_Cmax=float(stop_state[:,1].max())
281     last_Cmax=check['Cmax']
282     cuts=np.unique(np.r_[told,knots[(knots>told)&(knots<t)],t])
283     for lo,hi in zip(cuts[:-1],cuts[1:]):
284         mid=(lo+hi)/2;half=(hi-lo)/2
285         cum+=half*sum(ww*model.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
286             zip(g,w))
287         cum4+=half*sum(ww*model.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
288             zip(g4,w4))
289     mean=np.sum(model.mass[:,None]*solver.y.reshape(-1,2),axis=0)/model.Mdry
290     residual=np.array([mean[1]-mean0[1]+cum[0],mean[0]-mean0[0]+cum[1]])
291     max_res=np.maximum(max_res,abs(residual));maxquad=max(maxquad,np.max(abs(cum-cum4)/np.array([1,1e6,1e6,1e6,1e6])))
292     budgets.append(np.r_[t,mean,cum,residual])
293     while cursor<len(targets) and targets[cursor]<=t+1e-9:
294         tt=targets[cursor];yy=y0 if tt==0 else dense(tt)
295         ts.append(tt);ys.append(yy.reshape(-1,2).copy());records.append(model.record(tt,yy));
296         cursor+=1
297     rr=np.array(records)
298     obs=np.interp(rr[:,0],model.radius_data[:,0],model.radius_data[:,1])/100 if c.question==4 else
299         np.full(len(rr),np.nan)
300     checksmax={k:max(q[k] for q in checks) for k in
301         ['max_det_error','relative_volume_error','max_water_volume_fraction','max_activity','max_bou
302         ndary_activity','max_volume_quadrature_activity','max_metric_cross_relative','constitutive_v
303         olume_relative_error','rho_mixture_relative_error']}
304     checksmin={k:min(q[k] for q in checks) for k in ['min_J','min_air_fraction_for_phi0_zero','mi
305         n_air_fraction_q_for_phi0_zero','min_air_fraction_for_feasible_phi0_min','min_air_fraction_q
306         _for_feasible_phi0_min','min_boundary_entropy','internal_entropy_W_K','Cmin','Tmin']}
307     diag={'config':asdict(c),'elapsed_s':time.time()-beg,'steps':steps,'nfev':solver.nfev,'njev':
308         solver.njev,'nlu':solver.nlu,'rhs_actual_calls':model.call_count,
309         'integrated_until_s':solver.t,'critical_h':None if critical is None else critical/3600,
310         'strict_stop_s':stop_s,'strict_stop_Cmax':stop_Cmax,'strict_stop_verified':bool(stop_Cmax
311             is not None and stop_Cmax<.15),
312         'Cmax_end':rr[-1,4],'Cmean_end':rr[-1,2],'Tmean_end':rr[-1,3],'R_end_m':rr[-1,1],
313         'dry_mass_kg':model.Mdry,'rhod0':model.rhod0,'initial_water_volume_fraction':model.a0,
314         'initial_phi0_feasible_interval':[model.phi0_min,1-model.a0],'phi0_upper_open':True,
315         'rho_s_min_kg_m3':1/model.solid_specific_volume_max,'phi0_rho_s_identified':False,
316         'mass_residual_max_kg_perkgdry':max_res[0],'enthalpy_residual_max_J_perkgdry':max_res[1],
317         'quadrature_scaled_difference':maxquad,'checks_max':checksmax,'checks_min':checksmin,
318         'radius_reference_applies_to_question':bool(c.question==4),
319         'radius_rmse_m':float(np.sqrt(np.mean((rr[:,1]-obs)**2))) if c.question==4 else None,
320         'radius_max_abs_error_m':float(np.max(abs(rr[:,1]-obs))) if c.question==4 else None,
321         'constitutive_and_kinematic_parameters_calibrated':False,

```

```

311         'scope': 'Conditional local volume law and cylindrical-shell kinematic mapping. Nodal
            volumes are Q1-interpolated; nonlinear given-density constitutive error is reported at
            quadrature points. Constituent capacity is enforced; solid mechanics is not identified.
            No fit to observed radius.'}
312     arrays={'times':np.array(ts),'states_H_C':np.array(ys),'records':rr,'observed_radius_m':obs,'
            A':model.A,'Z':model.Z,'lumped_dry_mass':model.mass,
313             'budget_steps':np.array(budgets),'check_times':np.array([q['time_s'] for q in checks]),
314             'strict_stop_state_H_C':np.array([]) if stop_state is None else stop_state}
315     for k in checks[0]:
316         if k!='time_s':arrays['check_'+k]=np.array([q[k] for q in checks])
317     return arrays,diag

```

F.24 elongation_model.py

类别：守恒变长度模型 A。原始代码共 383 行。

```

1  """Conditional Q4 latent-heat trial with observed radius and variable length.
2
3  This module ALWAYS uses rho(C) as current true wet density through local J.
4  Only reviewed thermal functions/fluxes and Q1 FEM algebra are reused from the
5  isolated frozen dependencies. Their historical module-level scope descriptions
6  do not describe this class. Mapping A is conditional kinematics, NOT identified
7  solid mechanics. Radius is prescribed input, not a validated model prediction.
8  No material integration occurs on import or model construction.
9  """
10 from dataclasses import dataclass, asdict
11 import time
12 import numpy as np
13 from numpy.polynomial.legendre import leggauss
14 from scipy.integrate import BDF
15 from scipy.optimize import brentq
16 from scipy.special import expi
17
18 if __package__:
19     from ._elongation_frozen.boundary_model import Config as BoundaryConfig, Model as BoundaryModel
20     from ._elongation_frozen.volume_fem import Model as FrozenFEMModel
21     from ._elongation_frozen.thermo_reference import load_input, RV, TR
22 else:
23     from _elongation_frozen.boundary_model import Config as BoundaryConfig, Model as BoundaryModel
24     from _elongation_frozen.volume_fem import Model as FrozenFEMModel
25     from _elongation_frozen.thermo_reference import load_input, RV, TR
26
27
28 RECORD_COLUMNS = (
29     'time_s', 'radius_m', 'Cmean_dry', 'Tmean_dry_C', 'Cmax', 'Cmin',
30     'Tmin_C', 'Tmax_C', 'length_m', 'lambda_z', 'prescribed_radius_m',
31     'radius_input_error_m', 'global_J',
32 )
33

```

```

34
35 @dataclass(frozen=True)
36 class Config(BoundaryConfig):
37     """Defaults are a pending conditional trial, not calibrated material data."""
38     question: int = 4
39     volume_mode: str = 'given_density'
40     latent: float = 1.
41
42
43 class Model(BoundaryModel):
44     def __init__(self, cfg, env, radius):
45         radius = np.array(radius, dtype=float, copy=True)
46         env = np.array(env, dtype=float, copy=True)
47         if radius.ndim != 2 or radius.shape[1] != 2 or len(radius) < 2:
48             raise ValueError('Radius data must be [time_s, radius_cm] with at least two rows')
49         if not np.isfinite(radius).all() or np.any(np.diff(radius[:, 0]) <= 0) or np.any(radius[:,
50             1] <= 0):
51             raise ValueError('Invalid radius observations')
52         if radius[0, 0] != 0. or not np.isclose(radius[0, 1] / 100, cfg.R0, rtol=0, atol=1e-14):
53             raise ValueError('Initial radius data must match the declared initial radius at t=0')
54         if cfg.D_scale != 1.:
55             raise ValueError('This trial retains the given diffusion coefficient without a fitted
56                 multiplier')
57
58         # The frozen initializer still performs its historical compatibility
59         # checks with the water-density reference; defaults pass them. This is
60         # disclosed legacy initialization behavior, not identified phase data.
61         super().__init__(cfg, env, radius)
62         self.radius_data = radius
63         self.radius_data.setflags(write=False)
64         self.radius_last_s = min(float(radius[-1, 0]), 72. * 3600.)
65         # Frozen superclass derived an existence family only over 0<=C<=C0.
66         # These are NOT material parameters here and must not leak to diagnostics.
67         for name in ('solid_specific_volume_max', 'solid_reference_fraction', 'phi0_min'):
68             if hasattr(self, name):
69                 delattr(self, name)
70
71     def observed_radius(self, t):
72         t = float(t)
73         if not np.isfinite(t) or t < self.radius_data[0, 0] or t > self.radius_last_s:
74             raise ValueError(f'No radius extrapolation: t={t}, allowed [0,{self.radius_last_s}] s')
75         return float(np.interp(t, self.radius_data[:, 0], self.radius_data[:, 1])) / 100.
76
77     def geometry_data(self, t, C):
78         """All geometry callers pass time explicitly; no mutable self.t exists."""
79         R = self.observed_radius(t)
80         gd = FrozenFEMModel.geometry_data(self, C)
81         global_J = 2. * gd['I'][-1] / self.c.R0**2
82         lambda_z = global_J / (R / self.c.R0)**2

```

```

80         if not np.isfinite(lambda_z) or lambda_z <= 0:
81             raise ValueError('Nonpositive or nonfinite axial stretch')
82         gd.update(time_s=float(t), R_fixed_length=float(gd['R']), R=R,
83                 prescribed_R=R, global_J=float(global_J), lambda_z=float(lambda_z),
84                 L=float(self.c.length0 * lambda_z))
85         return gd
86
87     def map_points(self, gd, i, j, xi, eta):
88         geo = FrozenFEMModel.map_points(self, gd, i, j, xi, eta)
89         lz = gd['lambda_z']
90         root_lz = np.sqrt(lz)
91         geo['r'] = geo['r'] / root_lz
92         geo['ra'] = geo['ra'] / root_lz
93         for key in ('z', 'za', 'zz'):
94             geo[key] = geo[key] * lz
95         ra, za, zz = geo['ra'], geo['za'], geo['zz']
96         m11 = 1. / ra**2
97         m12 = -za / (ra**2 * zz)
98         m22 = (za / (ra * zz))**2 + 1. / zz**2
99         geo['metric'] = np.stack((m11, m12, m12, m22), axis=-1).reshape(geo['J'].shape + (2, 2))
100        a = self.A[i] + (self.A[i+1] - self.A[i]) * xi
101        hoop = np.divide(geo['r'], a,
102                        out=np.sqrt(geo['bar'] / lz).copy() + np.zeros_like(geo['r']), where=a > 0)
103        geo['det'] = hoop * ra * zz
104        return geo
105
106    def nodal_positions(self, gd):
107        r = np.sqrt(2. * gd['I'] / gd['lambda_z'])
108        z = gd['lambda_z'] * gd['K'] / gd['bar'][:, None]
109        return np.broadcast_to(r[:, None], self.shape).copy(), z
110
111    def material_fields(self, y, strict=True):
112        H, C, T = self.unpack(y)
113        if not np.isfinite(y).all():
114            raise ValueError('Nonfinite state')
115        if strict and np.min(C) <= 0:
116            raise ValueError(f'Nonpositive moisture trial state: {C.min()}; no clipping')
117        # No C<=C0 assumption: signed surface transfer may permit condensation.
118        if np.min(T) <= -273.15:
119            raise ValueError('Nonpositive absolute temperature')
120        if np.min(T) < .01 or np.max(T) > 55:
121            raise ValueError(f'Outside reviewed thermal domain [.01,55] C: {T.min()}, {T.max()}')
122        return H, C, T
123
124    def boundary(self, t, C, T, gd):
125        if float(t) != gd['time_s']:
126            raise ValueError('Boundary time differs from explicitly evaluated geometry time')
127        c = self.c

```

```

128     si = np.full((c.nz, 1), c.nr - 1)
129     sj = np.arange(c.nz)[: , None]
130     sidegeo = self.map_points(gd, si, sj, np.ones((1, len(self.bg))), self.bg[None, :])
131     ei = np.arange(c.nr)[: , None]
132     ej = np.full((c.nr, 1), c.nz - 1)
133     endgeo = self.map_points(gd, ei, ej, self.bg[None, :], np.ones((1, len(self.bg))))
134     out = []
135     for conn, geo, dl, kind in (
136         (self.side_conn, sidegeo, np.diff(self.Z), 'side'),
137         (self.end_conn, endgeo, np.diff(self.A), 'end'),
138     ):
139         Cq = C[conn] @ self.bN.T
140         Tq = T[conn] @ self.bN.T
141         vals = self.th.surface(t, Tq, Cq)
142         if kind == 'side':
143             W = 4. * np.pi * gd['R'] * geo['zz'] * dl[:, None] * self.bw
144         else:
145             # End plane area changes by 1/lambda_z compared with frozen map.
146             aq = self.A[:-1, None] + dl[:, None] * self.bg
147             W = 4. * np.pi * aq * geo['bar'] / gd['lambda_z'] * dl[:, None] * self.bw
148         out.append((conn, W, vals))
149     return out
150
151 def terms(self, t, y, need_checks=False):
152     H, C, T = self.material_fields(y); gd = self.geometry_data(t, C)
153     geo = self.map_points(gd, self.ei[:, None], self.ej[:, None], self.xi[None, :], self.eta[None, :])
154     Cn = C[self.conn]; Tn = T[self.conn]; Cq = Cn @ self.N.T; Tq = Tn @ self.N.T
155     a, b, E = {1: (.89, 7e-9, 0.), 2: (.45, 2.4e-3, 3850.), 3: (.45, 2.4e-3, 3850.), 4: (.30, 4.2e-4, 3850.)}[self.c.question]
156     phi = Cn * np.exp(-a/Cn) + a * expi(-a/Cn)
157     gradphi = np.einsum('eqkd, ek->eqd', self.gradN, phi)
158     gradT = np.einsum('eqkd, ek->eqd', self.gradN, Tn)
159     pref = b * self.c.D_scale * np.exp(-E/(Tq + 273.15))
160     mass_flux = -self.rhod0 * pref[:, :, None] * np.einsum('eqab, eqb->eqa', geo['metric'], gradphi)
161     heat_flux = -geo['J'][:, :, None] * self.th.conductivity(Cq)[:, :, None] * np.einsum('eqab, eqb->eqa', geo['metric'], gradT)
162     total_flux = heat_flux + self.th.bound_h(Tq)[:, :, None] * mass_flux
163     lm = np.einsum('eqkd, eqd, eq->ek', self.gradN, mass_flux, self.weights)
164     lh = np.einsum('eqkd, eqd, eq->ek', self.gradN, total_flux, self.weights)
165     fm = np.bincount(self.conn.ravel(), weights=lm.ravel(), minlength=self.nnode)
166     fh = np.bincount(self.conn.ravel(), weights=lh.ravel(), minlength=self.nnode)
167     kelvin = T + 273.15
168     mu_over_T = (self.th.cw * ((T - 28) / kelvin - np.log(kelvin / TR))
169                 + self.c.desorption_B * (1 / TR - 1 / kelvin) + RV * np.log(C / (C + self.th.s))
170                 + self.c.energy_gauge / kelvin)
171     internal_entropy = float(np.sum(fh / kelvin - mu_over_T * fm))
172     budgets = np.zeros(5); boundary_min_sigma = np.inf; boundary_maxaw = 0.
173     for conn, W, vals in self.boundary(t, C, T, gd):

```

```

174         for f,idx in [(fm,0),(fh,1)]:
175             local=np.einsum('eq,eq,qk->ek',W,vals[idx],self.bN)
176             f-=np.bincount(conn.ravel(),weights=local.ravel(),minlength=self.nnnode)
177             for k,idx in enumerate((0,1,2,3,4)):budgets[k]+=np.sum(W*vals[idx])
178             boundary_min_sigma=min(boundary_min_sigma,float(np.min(vals[6])))
179             boundary_maxaw=max(boundary_maxaw,float(np.max(vals[5])))
180 out=np.stack([fh/self.mass,fm/self.mass],axis=-1).ravel()
181 check=None
182 if need_checks:
183     Jn=gd['J'].ravel()
184     amin=np.min(self.th.activity(T,C));amax=np.max(self.th.activity(T,C))
185     alpha_n=self.rhod0*C/(self.c.water_density*Jn)
186     alpha_q=self.rhod0*Cq/(self.c.water_density*geo['J'])
187     available_n=Jn/self.rhod0-C/self.c.water_density
188     available_q=geo['J']/self.rhod0-Cq/self.c.water_density
189     desired_J=self.volume_law(Cq)
190     constituent_error=float(np.max(abs(geo['J']-desired_J)/desired_J))
191     rho_given=self.rho_a+self.rho_b*Cq
192     rho_error=float(np.max(abs(self.rhod0*(1+Cq)/geo['J']-rho_given)/rho_given))
193     volume_quad=float(np.sum(self.weights*geo['J']))
194     volume_cyl=np.pi*gd['R']**2*gd['L']
195     mapped_radius=float(np.sqrt(2*gd['I'][-1]/gd['lambda_z']))
196     check={'Cmin':float(C.min()),'Cmax':float(C.max()),'Tmin':float(T.min()),'Tmax':float(
197         (T.max()),
198         'min_J':float(geo['J'].min()),'max_det_error':float(np.max(abs(geo['J']-geo['d
199         et'])))},
200     'volume_quad_m3':volume_quad,'volume_cylinder_m3':volume_cyl,
201     'relative_volume_error':float(abs(volume_quad-volume_cyl)/volume_cyl),
202     'length_m':gd['L'],'lambda_z':gd['lambda_z'],'global_J':gd['global_J'],
203     'radius_input_abs_error_m':abs(mapped_radius-gd['prescribed_R']),
204     'max_water_volume_fraction':float(max(alpha_n.max(),alpha_q.max())),
205     'min_available_solid_specific_volume_m3_kgdry':float(min(available_n.min(),ava
206     ilable_q.min())),
207     'constitutive_volume_relative_error':constituent_error,
208     'rho_mixture_relative_error':rho_error,
209     'max_volume_quadrature_activity':float(np.max(self.th.activity(Tq,Cq))),
210     'internal_entropy_W_K':internal_entropy,
211     'min_activity':float(amin),'max_activity':float(amax),
212     'min_boundary_entropy':boundary_min_sigma,'max_boundary_activity':boundary_maxaw,
213     'instant_mass_balance_kg_s':float(fm.sum()+budgets[0]),
214     'instant_enthalpy_balance_W':float(fh.sum()+budgets[1]),
215     'max_metric_cross_relative':float(np.max(abs(geo['metric'][:, :, 0, 1])/np.sqrt(g
216     eo['metric'][:, :, 0, 0]*geo['metric'][:, :, 1, 1])))}
217     return out,budgets/self.Mdry,gd,check
218
219 def boundary_budget(self, t, y):
220     _, C, T = self.material_fields(y)

```

```

218         gd = self.geometry_data(t, C)
219         b = np.zeros(5)
220         for _, W, v in self.boundary(t, C, T, gd):
221             b += np.array([np.sum(W * v[k]) for k in (0, 1, 2, 3, 4)])
222         return b / self.Mdry
223
224     def record(self, t, y):
225         _, C, T = self.material_fields(y)
226         gd = self.geometry_data(t, C)
227         return [t, gd['R'], np.dot(self.mass, C) / self.Mdry,
228                 np.dot(self.mass, T) / self.Mdry, C.max(), C.min(), T.min(), T.max(),
229                 gd['L'], gd['lambda_z'], gd['prescribed_R'], gd['R'] - gd['prescribed_R'],
230                 gd['global_J']]
231
232 def solve(model, end_s, save_step=600.):
233     """Run only after external approval; this function does not grant approval.
234
235     Both BDF time steps and independent flux quadrature respect environment and
236     radius data knots. No length monotonicity condition or geometric clamp.
237     """
238     c = model.c
239     end_s = float(end_s)
240     if not 0. < end_s <= model.radius_last_s or save_step <= 0:
241         raise ValueError('Positive horizon within supplied radius domain and positive save step
242             required')
243     beg = time.time()
244     y0 = model.initial()
245     solver = BDF(model.rhs, 0., y0, end_s, rtol=c.rtol,
246                 atol=np.tile([c.atol_H, c.atol_C], model.nnode), max_step=c.max_step)
247     all_knots = np.unique(np.r_[model.th.env_data[:, 0], model.radius_data[:, 0]])
248     integration_cuts = np.unique(np.r_[0., all_knots[(all_knots > 0) & (all_knots < end_s)],
249                                     end_s])
250     cut_index = 1
251     extra = np.array([0., 1., 10., 60., 300., 1800., 3600., 10800., 14400., 21600., 43200.,
252                     86400., 172800., 259200.])
253     targets = np.unique(np.r_[np.arange(0., end_s, save_step), end_s, extra[extra <= end_s]])
254     ts, ys, records, budgets, checks, rnodes, znodes = [], [], [], [], [], [], []
255     cursor = steps = 0
256     cum = np.zeros(5)
257     cum4 = np.zeros(5)
258     g, w = leggauss(8)
259     g4, w4 = leggauss(4)
260     mean0 = np.sum(model.mass[:, None] * y0.reshape(-1, 2), axis=0) / model.Mdry
261     critical = stop_s = stop_state = stop_Cmax = None
262     max_res = np.zeros(2)
263     maxquad = 0.
264     last_Cmax = float(model.unpack(y0)[1].max())

```

```

262 # Existence bound only: no density or porosity is selected as a material parameter.
263 initial_available = 1. / model.rhod0 - c.C0 / c.water_density
264 common_solid_specific_capacity = float(initial_available)
265
266 while solver.status == 'running':
267     told = solver.t
268     tol_t = 8 * np.finfo(float).eps * max(1., abs(told))
269     while cut_index < len(integration_cuts) - 1 and integration_cuts[cut_index] <= told +
270         tol_t:
271         cut_index += 1
272         distance_to_cut = integration_cuts[cut_index] - told
273         if distance_to_cut <= 0:
274             raise RuntimeError('Invalid integration cut progression')
275         solver.max_step = min(c.max_step, distance_to_cut)
276         msg = solver.step()
277         steps += 1
278         if solver.status == 'failed':
279             raise RuntimeError(msg)
280         t = solver.t
281         dense = solver.dense_output()
282         _, _, _, check = model.terms(t, solver.y, True)
283         if max(check['max_activity'], check['max_boundary_activity'],
284             check['max_volume_quadrature_activity']) > 1 + 1e-10:
285             raise ValueError(f'Activity exceeds 1: {check}')
286         if c.latent == 1. and check['min_boundary_entropy'] < -1e-8:
287             raise ValueError(f'Negative boundary entropy: {check}')
288         common_solid_specific_capacity = min(common_solid_specific_capacity,
289             check['min_available_solid_specific_volume_m3_kgdry'])
290         # Water-density/phase capacity is a conditional diagnostic only. It has
291         # not been approved as an extra constitutive admissibility constraint.
292         if check['relative_volume_error'] > 1e-11 or check['max_det_error'] > 1e-11:
293             raise ValueError('Geometry integration failure')
294         check['time_s'] = t
295         checks.append(check)
296         if critical is None and last_Cmax >= .15 and check['Cmax'] < .15:
297             critical = brentq(lambda tt: model.unpack(dense(tt))[1].max() - .15, told, t)
298             stop_s = int(np.floor(critical)) + 1
299         if stop_s is not None and stop_state is None and told <= stop_s <= t:
300             stop_state = dense(stop_s).reshape(-1, 2).copy()
301             stop_Cmax = float(stop_state[:, 1].max())
302             last_Cmax = check['Cmax']
303         cuts = np.unique(np.r_[told, all_knots[(all_knots > told) & (all_knots < t)], t])
304         for lo, hi in zip(cuts[:-1], cuts[1:]):
305             mid, half = (lo + hi) / 2., (hi - lo) / 2.
306             cum += half * sum(wv * model.boundary_budget(mid + half * gg, dense(mid + half * gg))
307                 for gg, wv in zip(g, w))
308             cum4 += half * sum(wv * model.boundary_budget(mid + half * gg, dense(mid + half * gg))
309                 for gg, wv in zip(g4, w4))

```

```

305     mean = np.sum(model.mass[:, None] * solver.y.reshape(-1, 2), axis=0) / model.Mdry
306     residual = np.array([mean[1] - mean0[1] + cum[0], mean[0] - mean0[0] + cum[1]])
307     max_res = np.maximum(max_res, abs(residual))
308     maxquad = max(maxquad, float(np.max(abs(cum - cum4) / np.array([1., 1e6, 1e6, 1e6, 1e6]))))
309     budgets.append(np.r_[t, mean, cum, residual])
310     while cursor < len(targets) and targets[cursor] <= t + 1e-9:
311         tt = float(targets[cursor])
312         yy = y0 if tt == 0 else dense(tt)
313         ts.append(tt)
314         ys.append(yy.reshape(-1, 2).copy())
315         records.append(model.record(tt, yy))
316         gd = model.geometry_data(tt, model.unpack(yy)[1])
317         rn, zn = model.nodal_positions(gd)
318         rnodes.append(rn)
319         znodes.append(zn)
320         cursor += 1
321
322     rr = np.array(records)
323     obs = np.array([model.observed_radius(tt) for tt in rr[:, 0]])
324     max_keys = ['max_det_error', 'relative_volume_error', 'max_water_volume_fraction',
325               'max_activity',
326               'max_boundary_activity', 'max_volume_quadrature_activity',
327               'max_metric_cross_relative',
328               'constitutive_volume_relative_error', 'rho_mixture_relative_error',
329               'radius_input_abs_error_m']
330     min_keys = ['min_J', 'min_boundary_entropy', 'internal_entropy_W_K', 'Cmin', 'Tmin',
331               'min_available_solid_specific_volume_m3_kgdry']
332     checksmax = {k: max(q[k] for q in checks) for k in max_keys}
333     checksmin = {k: min(q[k] for q in checks) for k in min_keys}
334     diag = {
335         'config': asdict(c), 'elapsed_s': time.time() - beg, 'steps': steps,
336         'nfew': solver.nfew, 'njev': solver.njev, 'nlu': solver.nlu, 'rhs_actual_calls':
337         model.call_count,
338         'integrated_until_s': solver.t, 'critical_h': None if critical is None else critical /
339         3600.,
340         'strict_stop_s': stop_s, 'strict_stop_Cmax': stop_Cmax,
341         'strict_stop_verified': bool(stop_Cmax is not None and stop_Cmax < .15),
342         'Cmax_end': rr[-1, 4], 'Cmean_end': rr[-1, 2], 'Tmean_end': rr[-1, 3],
343         'R_end_m': rr[-1, 1], 'L_end_m': rr[-1, 8],
344         'L_min_m': float(min(c.length0, min(q['length_m'] for q in checks))),
345         'L_max_m': float(max(c.length0, max(q['length_m'] for q in checks))),
346         'length_was_constrained_monotone': False,
347         'dry_mass_kg': model.Mdry, 'rhod0': model.rhod0,
348         'water_density_reference_kg_m3': c.water_density,
349         'initial_water_volume_fraction': model.a0,
350         'common_solid_specific_capacity_min_m3_kgdry': common_solid_specific_capacity,
351         'solid_density_lower_bound_for_sampled_capacity_kg_m3': (
352             1. / common_solid_specific_capacity if common_solid_specific_capacity > 0 else None),

```

```

348     'phase_capacity_diagnostic_positive': bool(common_solid_specific_capacity > 0),
349     'capacity_diagnostic_controls_solver_acceptance': False,
350     'capacity_bound_is_material_parameter': False, 'porosity_identified': False,
351     'boundary_physical_entropy_check_enforced': bool(c.latent == 1.),
352     'boundary_entropy_is_counterfactual_for_this_run': bool(c.latent == 0.),
353     'mass_residual_max_kg_perkgdry': max_res[0], 'enthalpy_residual_max_J_perkgdry':
max_res[1],
354     'quadrature_scaled_difference': maxquad, 'checks_max': checksmax, 'checks_min': checksmin,
355     'radius_is_prescribed_input': True, 'radius_match_is_validation': False,
356     'radius_input_match_max_abs_error_m': float(np.max(abs(rr[:, 1] - obs))),
357     'radius_interpolation': 'piecewise linear within supplied [0,72h] only',
358     'integration_cut_count': len(integration_cuts),
359     'constitutive_and_kinematic_parameters_calibrated': False, 'main_model_promoted': False,
360     'record_columns': list(RECORD_COLUMNS),
361     'budget_columns': ['time_s', 'mean_H', 'mean_C', 'cum_mass_out', 'cum_total_energy_out',
362                        'cum_convection_out', 'cum_radiation_out', 'cum_water_enthalpy_out',
363                        'mass_residual', 'enthalpy_residual'],
364     'inherited_unused_config_fields': ['shrink_beta', 'gas_transfer_scale'],
365     'scope': 'Conditional mapping A with current true wet density, observed radius input, and
free total length. '
366         'No solid-mechanical force balance or mechanical energy is identified. Reviewed
latent boundary is reused. '
367         'Quadrature-point nonlinear constitutive interpolation error is reported.
Capacity uses a stated water-density '
368         'reference and is diagnostic only, not a solver admissibility rule or identified
solid density/porosity. '
369         'Latent=0 physical vapor entropy is counterfactual and is not an acceptance gate.
No C<=C0 constraint.',
370 }
371 arrays = {
372     'times': np.array(ts), 'states_H_C': np.array(ys), 'records': rr,
373     'record_columns': np.array(RECORD_COLUMNS), 'observed_radius_m': obs,
374     'A': model.A, 'Z': model.Z, 'lumped_dry_mass': model.mass,
375     'r_nodes_m': np.array(rnodes), 'z_nodes_m': np.array(znodes),
376     'budget_steps': np.array(budgets), 'check_times': np.array([q['time_s'] for q in checks]),
377     'integration_cuts_s': integration_cuts,
378     'strict_stop_state_H_C': np.array([]) if stop_state is None else stop_state,
379 }
380 for k in checks[0]:
381     if k != 'time_s':
382         arrays['check_' + k] = np.array([q[k] for q in checks])
383 return arrays, diag

```

F.25 experiment_model.py

类别：守恒变长度模型 A。原始代码共 51 行。

```
1 """Q4 free-length, original empirical Robin, no latent heat.
2
3 The frozen minimal_model supplies the exact pre-existing numerical RHS. This
4 subclass only adds diagnostics at accepted states; it never calls an isotherm.
5 """
6 import numpy as np
7 from minimal_model import Model as MinimalModel
8
9
10 class Model(MinimalModel):
11     def __init__(self, cfg, env, radius):
12         if cfg.latent != 0 or cfg.desorption_B != 0 or cfg.emissivity != 0:
13             raise ValueError('This branch fixes latent=0, desorption_B=0 and emissivity=0')
14         super().__init__(cfg, env, radius)
15
16     def terms(self, t, y, need_checks=False):
17         out, budgets, gd, checks = super().terms(t, y, need_checks)
18         if not need_checks:
19             return out, budgets, gd, checks
20         H,C,T=self.material_fields(y)
21         geo=self.map_points(gd,self.ei[:,None],self.ej[:,None],self.xi[None,:],self.eta[None,:])
22         Cq=C[self.conn]@self.N.T
23         desired_J=self.volume_law(Cq)
24         rho_given=760.+90.*Cq
25         rho_mapped=self.rhod0*(1+Cq)/geo['J']
26         volume_quad=float(np.sum(self.weights*geo['J']))
27         volume_cylinder=float(np.pi*gd['R']**2*gd['L'])
28         solid_available_node=gd['J'].ravel()/self.rhod0-C/self.c.water_density
29         solid_available_q=geo['J']/self.rhod0-Cq/self.c.water_density
30         nodal_dry_density=(760.+90.*C)/(1+C)
31         nodal_mass=np.sum(self.mass*gd['J'].ravel()*nodal_dry_density/self.rhod0)
32         direction=self.direction(t,y)
33         checks.update({
34             'Cmax':float(C.max()), 'Tmin':float(T.min()), 'Tmax':float(T.max()),
35             'min_J':float(geo['J'].min()),
36             'max_det_error':float(np.max(abs(geo['J']-geo['det']))),
37             'volume_quad_m3':volume_quad,'volume_cylinder_m3':volume_cylinder,
38             'relative_volume_error':abs(volume_quad-volume_cylinder)/volume_cylinder,
39             'length_m':float(gd['L']), 'lambda_z':float(gd['lambda_z']),
40             'global_J':float(gd['global_J']),
41             'constitutive_volume_relative_error':float(np.max(abs(geo['J']-desired_J)/desired_J)),
42             'rho_mixture_relative_error':float(np.max(abs(rho_mapped-rho_given)/rho_given)),
43             'nodal_dry_mass_relative_error':float(abs(nodal_mass-self.Mdry)/self.Mdry),
44             'radius_input_abs_error_m':float(abs(np.sqrt(2*gd['I'][-1]/gd['lambda_z'])-gd['R'])),
```

```

45         'max_metric_cross_relative':float(np.max(abs(geo['metric'][:, :, 0, 1])/np.sqrt(geo['metric'][:, :, 0, 0]*geo['metric'][:, :, 1, 1]))),
46         'max_water_volume_fraction':float(max(np.max(self.rhod0*C/(self.c.water_density*gd['J_'].ravel()), np.max(self.rhod0*Cq/(self.c.water_density*geo['J']))),),
47         'min_available_solid_specific_volume_m3_kgdry':float(min(solid_available_node.min(), solid_available_q.min())),
48         'boundary_bad_surface_area_fraction':direction['bad_surface_area_fraction'],
49         'minimum_evaporating_psat_minus_pa_Pa':direction['minimum_evaporating_psat_minus_pa_Pa'],
50     })
51     return out, budgets, gd, checks

```

F.26 minimal_model.py

类别：守恒变长度模型 A。原始代码共 104 行。

```

1  """Q4 diagnostic: existing Map A + existing enthalpy + original Robin flux.
2
3  No sorption/equilibrium curve enters the RHS or acceptance checks. This is an
4  audit of an already-used phenomenological boundary, not a new main model.
5  """
6  from dataclasses import replace
7  import types
8  import numpy as np
9  from scipy.special import expi
10 from core.elongation_model import Config, Model as CoreModel, load_input
11 from core._elongation_frozen.thermo_reference import hv, psat
12
13
14 def forbidden_isotherm(*args, **kwargs):
15     raise RuntimeError('No isotherm is allowed in this diagnostic')
16
17
18 class Model(CoreModel):
19     def __init__(self, cfg, env, radius):
20         # The frozen geometric initializer expects its historical config fields.
21         # Its temporary equilibrium scalar is removed below. Preflight varies
22         # that initializer scalar and checks exact RHS/budget invariance.
23         super().__init__(cfg, env, radius)
24         self.c = replace(cfg, equilibrium_C50=None)
25         self.th.c = replace(self.th.c, equilibrium_C50=None)
26         del self.th.s
27         self.th.activity = forbidden_isotherm
28         self.th.thermal_log_aw = forbidden_isotherm
29         self.th.surface = types.MethodType(self._surface, self.th)
30
31     def _surface(self, th, t, T, C):
32         T, C = np.asarray(T), np.asarray(C)
33         if np.any(C <= 0):

```

```

34         raise ValueError('Nonpositive moisture; no clipping')
35     Ta, wa = th.env(t)
36     rhod = (760 + 90*C)/(1+C)
37     J = rhod*self.c.hm*(C-wa)
38     carry = hv(np.where(J >= 0, T, Ta)) + self.c.energy_gauge
39     if self.c.latent == 0:
40         carry = th.bound_h(T)
41     convection = self.c.h*(T-Ta)
42     radiation = np.zeros_like(T) # Already-approved emissivity=0 only.
43     return J, convection+carry*J, convection, radiation, carry*J
44
45 def terms(self, t, y, need_checks=False):
46     # Same constitutive and geometric flux expressions as frozen Q4 core.
47     H, C, T = self.material_fields(y)
48     gd = self.geometry_data(t, C)
49     geo = self.map_points(gd, self.ei[:,None], self.ej[:,None], self.xi[None,:],
50                          self.eta[None,:])
51     Cn, Tn = C[self.conn], T[self.conn]
52     Cq, Tq = Cn@self.N.T, Tn@self.N.T
53     a, b, E = .30, 4.2e-4, 3850.
54     phi = Cn*np.exp(-a/Cn)+a*expi(-a/Cn)
55     gradphi = np.einsum('eqkd,ek->eqd', self.gradN, phi)
56     gradT = np.einsum('eqkd,ek->eqd', self.gradN, Tn)
57     pref = b*np.exp(-E/(Tq+273.15))
58     mass_flux = -self.rhod0*pref[:, :, None]*np.einsum('eqab,eqb->eqa', geo['metric'], gradphi)
59     heat_flux = -geo['J'][:, :, None]*self.th.conductivity(Cq)[:, :, None]*np.einsum('eqab,eqb->eqa', geo['metric'], gradT)
60     total_flux = heat_flux+self.th.bound_h(Tq)[:, :, None]*mass_flux
61     lm = np.einsum('eqkd,eqd,eq->ek', self.gradN, mass_flux, self.weights)
62     lh = np.einsum('eqkd,eqd,eq->ek', self.gradN, total_flux, self.weights)
63     fm = np.bincount(self.conn.ravel(), weights=lm.ravel(), minlength=self.nnnode)
64     fh = np.bincount(self.conn.ravel(), weights=lh.ravel(), minlength=self.nnnode)
65     budgets = np.zeros(5)
66     for conn, W, vals in self.boundary(t, C, T, gd):
67         for f, idx in ((fm, 0), (fh, 1)):
68             local = np.einsum('eq,eq,qk->ek', W, vals[idx], self.bN)
69             f -= np.bincount(conn.ravel(), weights=local.ravel(), minlength=self.nnnode)
70             budgets += [np.sum(W*vals[i]) for i in range(5)]
71     out = np.stack([fh/self.mass, fm/self.mass], axis=-1).ravel()
72     checks = {
73         'instant_mass_balance_kg_s': float(fm.sum()+budgets[0]),
74         'instant_enthalpy_balance_W': float(fh.sum()+budgets[1]),
75         'Cmin': float(C.min()), 'Tmin_C': float(T.min()),
76     } if need_checks else None
77     return out, budgets/self.Mdry, gd, checks
78
79 def direction(self, t, y):
80     _, C, T = self.material_fields(y)

```

```

80     gd = self.geometry_data(t,C)
81     Ta, wa = self.th.env(t)
82     pa = float(self.th.vapor_pressure(wa))
83     worst = None
84     bad_area = total_area = 0.
85     for kind, (conn,W,vals) in zip(('side','end'),self.boundary(t,C,T,gd)):
86         Cq, Tq = C[conn]@self.bN.T, T[conn]@self.bN.T
87         pp = psat(Tq)
88         gap = pp-pa
89         active = vals[0] > 1e-12
90         total_area += float(W.sum())
91         bad_area += float(W[active & (gap < 0)].sum())
92         if np.any(active):
93             score = np.where(active,gap,np.inf)
94             idx = np.unravel_index(np.argmin(score),score.shape)
95             item = {'surface':kind, 'gap_Pa':float(gap[idx]),
96                   'surface_T_C':float(Tq[idx]), 'surface_C':float(Cq[idx]),
97                   'J_out_kg_m2_s':float(vals[0][idx]),
98                   'psat_surface_Pa':float(pp[idx]), 'air_pv_Pa':pa,
99                   'air_T_C':float(Ta), 'air_w':float(wa)}
100             if worst is None or item['gap_Pa'] < worst['gap_Pa']:
101                 worst = item
102     return {'time_s':float(t),'worst':worst,
103           'bad_surface_area_fraction':bad_area/total_area,
104           'minimum_evaporating_psat_minus_pa_Pa':worst['gap_Pa'] if worst else None}

```

F.27 preflight.py

类别：守恒变长度模型 A。原始代码共 43 行。

```

1  """Synthetic algebra and code-substitution identities, not material data."""
2  import os
3  for k in ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS'):os.environ[k]='1'
4  import json
5  from pathlib import Path
6  from dataclasses import replace
7  import numpy as np
8  from minimal_model import Model as FrozenMinimal,Config,load_input
9  from experiment_model import Model
10 root=Path(__file__).resolve().parents[1]
11 env,rad=load_input(root/'input')
12 c=Config(nr=6,nz=3,latent=0.,desorption_B=0.,emissivity=0.)
13 m=Model(c,env,rad);base=FrozenMinimal(c,env,rad)
14 nodes=np.linspace(0,1,m.nnode)
15 C=.4+2.*nodes;T=28.+15.*nodes[:-1]
16 y=np.stack([m.th.enthalpy(T,C),C],axis=-1).ravel()
17 a=m.terms(3600.,y);b=base.terms(3600.,y)
18 exact_rhs=np.array_equal(a[0],b[0]);exact_budget=np.array_equal(a[1],b[1])
19 legacy=[]

```

```

20 for ce in (.03,.075,.12):
21     alt=Model(replace(c,equilibrium_C50=ce),env,rad)
22     legacy.append(alt.terms(3600.,y)[:2])
23 invariant=all(np.array_equal(legacy[0][0],x[0]) and np.array_equal(legacy[0][1],x[1]) for x in
    legacy[1:])
24 cp=1850.+2150.*C/(1+C)
25 capacity_error=float(np.max(abs((1+C)*cp-(1850+4000*C))))
26 # Uniform-temperature water transport must not alter T when all incoming heat
27 # is zero and transported water carries the same sensible enthalpy throughout.
28 Tstar=42.;m.th.env=lambda t:(Tstar,.05)
29 yis=np.stack([m.th.enthalpy(Tstar,C),C],axis=-1).ravel()
30 rhs=m.terms(3600.,yis)[0].reshape(-1,2)
31 Td=(rhs[:,0]-4000*(Tstar-28.)*rhs[:,1])/(1850+4000*C)
32 cancel_error=float(abs(Td).max())
33 assert exact_rhs and exact_budget and invariant
34 assert capacity_error<5e-12 and cancel_error<1e-12
35 out={'data_kind':'synthetic','purpose':'Algebra/code identity only; not a fitted material
    observation',
36     'diagnostic_subclass_rhs_exactly_matches_frozen_minimal':exact_rhs,
37     'diagnostic_subclass_budget_exactly_matches_frozen_minimal':exact_budget,
38     'legacy_equilibrium_parameter_rhs_and_budget_invariance_nonuniform_state':invariant,
39     'supplied_cp_to_H_coefficient_max_abs_error_J_kgdryK':capacity_error,
40     'uniform_T_energy_mass_cancellation_max_abs_K_s':cancel_error,
41     'tests_pass':True}
42 (root/'analysis/preflight.json').write_text(json.dumps(out,indent=2)+'\n')
43 print(json.dumps(out))

```

F.28 run_case.py

类别：守恒变长度模型 A。原始代码共 21 行。

```

1 """Replay CLI that records code, raw-input, scope and full output hashes."""
2 import argparse,json,subprocess,sys
3 from pathlib import Path
4 ROOT=Path(__file__).resolve().parents[1]
5 OUTPUTS=('fields.npz','summary.json','trajectory.csv','midplane.csv','surface.csv','snapshots.csv',
    v')
6 p=argparse.ArgumentParser();p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',
    type=int,required=True);p.add_argument('--out',required=True)
7 p.add_argument('--end-h',type=float,default=72);p.add_argument('--save-step',type=float,default=
    60)
8 p.add_argument('--rtol',type=float,default=2e-6);p.add_argument('--atol-H',type=float,default=.0
    1);p.add_argument('--atol-C',type=float,default=1e-8);p.add_argument('--max-step',type=float,def
    ault=300)
9 a=p.parse_args();dest=Path(a.out)
10 if dest.is_absolute():dest=dest.resolve().relative_to(ROOT)
11 if '..' in dest.parts:raise ValueError('Output must stay within the case directory')
12 run_id=dest.name
13 cmd=[sys.executable,'-B','src/run_record.py','record','--case',str(ROOT),'--run-id',run_id]

```

```

14 for f in sorted((ROOT/'src').rglob('*.py')):cmd+=['--code',str(f.relative_to(ROOT))]
15 for f in [ROOT/'input/附件1.xlsx',ROOT/'input/附件2.xlsx',ROOT/'designs/scope.json']:cmd+=['--inp,
ut',str(f.relative_to(ROOT))]
16 for name in OUTPUTS:cmd+=['--output',str(dest/name)]
17 cmd+=['--data-kind','user-provided','--timeout','7200','--',sys.executable,'-B','src/solve_case.
py','--nr',str(a.nr),'--nz',str(a.nz),'--out',str(dest),'--end-h',str(a.end_h),'--save-step',str
(a.save_step),'--rtol',str(a.rtol),'--atol-H',str(a.atol_H),'--atol-C',str(a.atol_C),'--max-step
',str(a.max_step)]
18 r=subprocess.run(cmd,cwd=ROOT)
19 if r.returncode:raise SystemExit(r.returncode)
20 r=subprocess.run([sys.executable,'-B','src/run_record.py','verify','--case',str(ROOT),'--run-id',
run_id],cwd=ROOT)
21 raise SystemExit(r.returncode)

```

F.29 solve_case.py

类别：守恒变长度模型 A。原始代码共 166 行。

```

1 """Reproduce one no-latent Q4 free-length case; no result is overwritten."""
2 import os
3 for name in
    ('OPENBLAS_NUM_THREADS','OMP_NUM_THREADS','MKL_NUM_THREADS','VECLIB_MAXIMUM_THREADS'):
4     os.environ[name]='1'
5 import argparse, csv, json, time, sys, platform
6 from dataclasses import asdict, replace
7 from pathlib import Path
8 import numpy as np
9 import scipy
10 from numpy.polynomial.legendre import leggauss
11 from scipy.integrate import BDF
12 from scipy.optimize import brentq
13 from minimal_model import Config, load_input
14 from experiment_model import Model
15 from core.elongation_model import RECORD_COLUMNS
16
17 ROOT=Path(__file__).resolve().parents[1]
18 BUDGET_COLUMNS=['time_s','mean_H','mean_C','cum_mass_out','cum_total_energy_out','cum_convection,
_out','cum_radiation_out','cum_water_enthalpy_out','mass_residual','enthalpy_residual']
19
20
21 def solve(m, end_s, save_step):
22     cfg=m.c
23     if not 0<end_s<=m.radius_last_s or save_step<=0:raise ValueError('Invalid output horizon')
24     start=time.monotonic();y0=m.initial()
25     mean0=np.sum(m.mass[:,None]*y0.reshape(-1,2),axis=0)/m.Mdry
26     solver=BDF(m.rhs,0.,y0,end_s,rtol=cfg.rtol,atol=np.tile([cfg.atol_H, cfg.atol_C],m.nnode),max_,
step=cfg.max_step)
27     knots=np.unique(np.r_[m.th.env_data[:,0],m.radius_data[:,0]])
28     cuts=np.unique(np.r_[0.,knots[(knots>0)&(knots<end_s)],end_s]);cut_index=1

```

```

29 extra=np.array([0.,1.,10.,60.,300.,1800.,3600.,10800.,14400.,21600.,43200.,86400.,172800.,259,
200.])
30 targets=np.unique(np.r_[np.arange(0,end_s,save_step),end_s,extra[extra<=end_s]])
31 times=[];states=[];records=[];budgets=[];checks=[];rnodes=[];znodes=[]
32 cum=np.zeros(5);cum4=np.zeros(5);max_res=np.zeros(2);quad_error=0.
33 g,w=leggauss(8);g4,w4=leggauss(4)
34 critical=stop_s=stop_Cmax=stop_state=None
35 critical_state=None;cursor=steps=0;last_Cmax=cfg.C0;last_print=time.monotonic()
36 while solver.status=='running':
37     told=solver.t;tol=8*np.finfo(float).eps*max(1.,abs(told))
38     while cut_index<len(cuts)-1 and cuts[cut_index]<=told+tol:cut_index+=1
39     solver.max_step=min(cfg.max_step,float(cuts[cut_index]-told))
40     msg=solver.step();steps+=1
41     if solver.status=='failed':raise RuntimeError(msg)
42     t=float(solver.t);dense=solver.dense_output()
43     _,_,ck=m.terms(t,solver.y,True)
44     if ck['relative_volume_error']>1e-11 or ck['max_det_error']>1e-11:raise
ValueError('Geometry integration failure')
45     ck['time_s']=t;checks.append(ck)
46     if critical is None and last_Cmax>=.15 and ck['Cmax']<.15:
47         critical=float(brentq(lambda tt:m.unpack(dense(tt))[1].max()-0.15,told,t))
48         critical_state=dense(critical).reshape(-1,2).copy();stop_s=int(np.floor(critical))+1
49     if stop_s is not None and stop_state is None and told<=stop_s<=t:
50         stop_state=dense(stop_s).reshape(-1,2).copy();stop_Cmax=float(stop_state[:,1].max())
51     last_Cmax=ck['Cmax']
52     lohi=np.unique(np.r_[told,knots[(knots>told)&(knots<t)],t])
53     for lo,hi in zip(lohi[:-1],lohi[1:]):
54         mid,half=(lo+hi)/2,(hi-lo)/2
55         cum+=half*sum(ww*m.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
zip(g,w))
56         cum4+=half*sum(ww*m.boundary_budget(mid+half*gg,dense(mid+half*gg)) for gg,ww in
zip(g4,w4))
57     mean=np.sum(m.mass[:,None]*solver.y.reshape(-1,2),axis=0)/m.Mdry
58     residual=np.array([mean[1]-mean0[1]+cum[0],mean[0]-mean0[0]+cum[1]])
59     max_res=np.maximum(max_res,abs(residual))
60     quad_error=max(quad_error,float(np.max(abs(cum-cum4)/np.array([1.,1e6,1e6,1e6,1e6]))))
61     budgets.append(np.r_[t,mean,cum,residual])
62     while cursor<len(targets) and targets[cursor]<=t+1e-9:
63         tt=float(targets[cursor]);yy=y0 if tt==0 else dense(tt)
64         times.append(tt);states.append(yy.reshape(-1,2).copy());records.append(m.record(tt,yy))
65         gd=m.geometry_data(tt,m.unpack(yy)[1]);rn,zn=m.nodal_positions(gd);rnodes.append(rn);
znodes.append(zn)
66         cursor+=1
67     if time.monotonic()-last_print>=30:
68         print(json.dumps({'progress_h':t/3600,'Cmax':ck['Cmax'],'length_cm':100*ck['length_m'],
, 'elapsed_s':time.monotonic()-start}),flush=True);last_print=time.monotonic()
69     arr={'times':np.array(times),'states_H_C':np.array(states),'records':np.array(records),

```

```

70     'record_columns':np.array(RECORD_COLUMNS),'A':m.A,'Z':m.Z,'lumped_dry_mass':m.mass,
71     'r_nodes_m':np.array(rnodes),'z_nodes_m':np.array(znodes),'budget_steps':np.array(budget_
    s),
72     'budget_columns':np.array(BUDGET_COLUMNS),'check_times':np.array([c['time_s'] for c in
    checks]),
73     'integration_cuts_s':cuts,'initial_state_H_C':y0.reshape(-1,2),
74     'strict_stop_state_H_C':np.array([]) if stop_state is None else stop_state,
75     'critical_state_H_C':np.array([]) if critical_state is None else critical_state}
76 for k in checks[0]:
77     if k!='time_s':arr['check_'+k]=np.array([c[k] for c in checks])
78 min_keys=['Cmin','Tmin','min_J','minimum_evaporating_psat_minus_pa_Pa','min_available_solid_s_
    pecific_volume_m3_kgdry']
79 max_keys=['Cmax','Tmax','max_det_error','relative_volume_error','length_m','constitutive_volu_
    me_relative_error','rho_mixture_relative_error','nodal_dry_mass_relative_error','radius_inpu_
    t_abs_error_m','max_metric_cross_relative','max_water_volume_fraction','boundary_bad_surface_
    _area_fraction']
80 tv=np.abs(np.diff(np.vstack([np.zeros(5),arr['budget_steps'][:,3:8]]),axis=0)).sum(axis=0)
81 heat_scale=max(m.th.cs+m.th.cw*cfg.C0,float(tv[2:5].sum()))
82 mass_scale=max(cfg.C0,float(tv[0]))
83 diag={'config':asdict(cfg),'scope':'Q4 no-latent empirical Robin; current true wet density,
    dry-mass conservation, observed radius, approved conditional Map A free length',
84     'elapsed_s':time.monotonic()-start,'steps':steps,'nfev':solver.nfev,'njev':solver.njev,
    'nlu':solver.nlu,'rhs_actual_calls':m.call_count,
85     'integrated_until_s':float(solver.t),'critical_h':None if critical is None else
    critical/3600,'strict_stop_s':stop_s,
86     'strict_stop_Cmax':stop_Cmax,'strict_stop_verified':bool(stop_Cmax is not None and
    stop_Cmax<.15),
87     'dry_mass_kg':m.Mdry,'rhod0':m.rhod0,'L_min_m':float(min(cfg.length0,min(q['length_m']
    for q in checks))),
88     'L_max_m':float(max(cfg.length0,max(q['length_m'] for q in checks))),
89     'L_at_strict_stop_m':None if stop_state is None else
    m.geometry_data(stop_s,stop_state[:,1])['L'],
90     'R_at_strict_stop_m':None if stop_state is None else m.observed_radius(stop_s),
91     'Cmax_end':float(arr['records'][-1,4]),'Cmean_end':float(arr['records'][-1,2]),'Tmean_e_
    nd':float(arr['records'][-1,3]),
92     'L_end_m':float(arr['records'][-1,8]),'R_end_m':float(arr['records'][-1,1]),
93     'mass_residual_max_kg_perkgdry':float(max_res[0]),'enthalpy_residual_max_J_perkgdry':fl_
    oat(max_res[1]),
94     'mass_balance_relative':float(max_res[0]/mass_scale),'enthalpy_balance_relative':float(
    max_res[1]/heat_scale),
95     'budget_scales':{'mass_kg_perkgdry':mass_scale,'enthalpy_J_perkgdry':heat_scale,'channe_
    l_total_variations':tv.tolist()},
96     'quadrature_scaled_difference':quad_error,'checks_min':{k:min(q[k] for q in checks) for
    k in min_keys},
97     'checks_max':{k:max(q[k] for q in checks) for k in max_keys},
98     'equilibrium_parameter_used':False,'isotherm_calls_forbidden':True,'new_boundary_fitted_
    _parameters':[],

```

```

99     'physical_vapor_energy_included':False,'phase_direction_is_acceptance_gate':False,
100     'radius_is_prescribed_input':True,'radius_match_is_validation':False,
101     'length_is_observed':False,'length_is_constrained_monotone':False,'solid_mechanics_solve':False,
102     'mechanical_energy_included':False,'constitutive_parameters_fitted':False,
103     'latent_heat_branch_promoted':False,'full_material_thermodynamic_closure_claimed':False,
104     'capacity_is_only_a_diagnostic_using_1000kg_m3_reference':True,
105     'integration_cut_count':len(cuts),'record_columns':list(RECORD_COLUMNS),'budget_columns':BUDGET_COLUMNS,
106     'runtime':{'python':sys.version,'numpy':np.__version__,'scipy':scipy.__version__,'platform':platform.platform()},
107     'inherited_unused_fields':['shrink_beta','gas_transfer_scale','equilibrium_C50 (removed and prohibited)'],
108     'known_limits':['Empirical C_s-w_a driver equates unlike dry-basis references as a modeling approximation, not a supplied full law.',
109                     'Latent heat is absent; water sensible enthalpy is retained. No real evaporation-energy closure claimed.',
110                     'Length and local Map A deformation have not been measured or mechanically identified.',
111                     'Local heat capacity equals the supplied rho*cp. Changes versus the empirical base are conservative moisture operator, water sensible-enthalpy transport, and geometry, not only length.']]
112     return arr,diag
113
114
115     def csvfile(path,columns,rows):
116         with path.open('w',newline='',encoding='utf-8-sig') as f:
117             w=csv.writer(f);w.writerow(columns)
118             for row in rows:w.writerow(['' if v is None or isinstance(v,(float,np.floating)) and not np.isfinite(v) else v for v in row])
119
120
121     def export(m,arr,diag,out):
122         np.savez_compressed(out/'fields.npz',**arr)
123         (out/'summary.json').write_text(json.dumps(diag,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
124         csvfile(out/'trajectory.csv',RECORD_COLUMNS,arr['records'])
125         radii_cm=np.arange(21)/10
126         rows=[];surface=[];snaps=[]
127         selected={0.,1.,3.,4.,6.,12.,24.,36.,48.,60.,72.}
128         for n,(t,y) in enumerate(zip(arr['times'],arr['states_H_C'])):
129             H,C,T=m.unpack(y);C=C.reshape(m.shape);T=T.reshape(m.shape)
130             r=arr['r_nodes_m'][n,:,0];rn=arr['r_nodes_m'][n];zn=arr['z_nodes_m'][n]
131             query=radii_cm/100
132             inside=query<=arr['records'][n,1]+1e-12
133             midT=np.where(inside,np.interp(np.minimum(query,r[-1]),r,T[:,0]),np.nan)
134             midC=np.where(inside,np.interp(np.minimum(query,r[-1]),r,C[:,0]),np.nan)

```

```

135         for radius,tv,cv in
            zip(radii_cm,midT,midC):rows.append([t,t/3600,radius,tv,cv,arr['records'][n,1]*100])
136         surface.append([t,t/3600,arr['records'][n,1]*100,T[-1,0],C[-1,0],T[0,-1],C[0,-1],T[-1,-1],
            ,C[-1,-1],arr['records'][n,8]*100])
137         if t/3600 in selected:
138             for i in range(m.shape[0]):
139                 for j in range(m.shape[1]):snaps.append([t,t/3600,i,j,rn[i,j]*100,zn[i,j]*100,T[i,
                    ,j],C[i,j]])
140         if diag['strict_stop_verified']:
141             t=diag['strict_stop_s'];y=arr['strict_stop_state_H_C'];H,C,T=m.unpack(y)
142             gd=m.geometry_data(t,C);rn,zn=m.nodal_positions(gd);C=C.reshape(m.shape);T=T.reshape(m.sh
                ape)
143             for i in range(m.shape[0]):
144                 for j in
                    range(m.shape[1]):snaps.append([t,t/3600,i,j,rn[i,j]*100,zn[i,j]*100,T[i,j],C[i,j]])
145         csvfile(out/'midplane.csv',['time_s','time_h','radius_cm','T_C','C_kg_per_kgdry','current_rad
            ius_cm'],rows)
146         csvfile(out/'surface.csv',['time_s','time_h','current_radius_cm','side_midplane_T_C','side_mi
            dplane_C','end_axis_T_C','end_axis_C','corner_T_C','corner_C','length_cm'],surface)
147         csvfile(out/'snapshots.csv',['time_s','time_h','i','j','radius_cm','axial_from_midplane_cm','
            T_C','C_kg_per_kgdry'],snaps)
148
149
150 def main():
151     p=argparse.ArgumentParser();p.add_argument('--nr',type=int,required=True);p.add_argument('--nz
        ,type=int,required=True);p.add_argument('--out',type=Path,required=True)
152     p.add_argument('--end-h',type=float,default=72);p.add_argument('--save-step',type=float,default
        ,lt=60)
153     p.add_argument('--rtol',type=float,default=2e-6);p.add_argument('--atol-H',type=float,default
        ,=.01);p.add_argument('--atol-C',type=float,default=1e-8);p.add_argument('--max-step',type=fl
        oat,default=300)
154     args=p.parse_args();out=args.out.resolve();out.mkdir(parents=True,exist_ok=False)
155     cfg=Config(nr=args.nr,nz=args.nz,latent=0.,rtol=args.rtol,atol_H=args.atol_H,atol_C=args.atol
        _C,max_step=args.max_step,emissivity=0.,desorption_B=0.)
156     env,rad=load_input(R00T/'input');m=Model(cfg,env,rad)
157     refs=[]
158     for c0 in (.03,.075,.12):
159         alt=Model(replace(cfg,equilibrium_C50=c0),env,rad);refs.append(alt.terms(0.,alt.initial()
            )[:2])
160     invariant=all(np.array_equal(refs[0][0],r[0]) and np.array_equal(refs[0][1],r[1]) for r in
        refs[1:])
161     if not invariant:raise RuntimeError('Legacy initialization parameter contaminated RHS')
162     arr,diag=solve(m,args.end_h*3600,args.save_step);diag['legacy_initializer_parameter_invarianc
        e_exact']=invariant
163     export(m,arr,diag,out)
164     print(json.dumps({k:diag[k] for k in ('critical_h','strict_stop_s','L_max_m','mass_balance_re
        lative','enthalpy_balance_relative','elapsed_s')},ensure_ascii=False),flush=True)

```

165

```
166 if __name__=='__main__':main()
```

F.30 summarize_results.py

类别：守恒变长度模型 A。原始代码共 26 行。

```
1  """Summarize already computed cases; no PDE reruns or imported external values."""
2  from pathlib import Path
3  import json, csv
4  import numpy as np
5  ROOT=Path(__file__).resolve().parents[1]
6  cases=['q4-robin-latent0-12x6', 'q4-robin-latent0-12x6-tight', 'q4-robin-latent0-24x12', 'q4-robin-
    latent0-32x16']
7  rows=[]
8  for case in cases:
9      d=json.loads((ROOT/'results'/case/'summary.json').read_text());z=np.load(ROOT/'results'/case/'
    fields.npz')
10     ti=z['check_times'];ll=z['check_length_m'];peak=int(np.argmax(ll))
11     rows.append({'case':case, 'nr':d['config']['nr'], 'nz':d['config']['nz'], 'rtol':d['config']['rtol'],
        'max_step_s':d['config']['max_step'],
12                 'critical_h':d['critical_h'], 'strict_stop_s':d['strict_stop_s'], 'strict_stop_Cmax',
        x':d['strict_stop_Cmax'],
13                 'length_peak_cm':100*d['L_max_m'], 'length_peak_h':float(ti[peak]/3600),
14                 'length_at_stop_cm':100*d['L_at_strict_stop_m'], 'radius_at_stop_cm':100*d['R_at_
        strict_stop_m'],
15                 'length_72h_cm':100*d['L_end_m'], 'mass_residual_relative':d['mass_balance_relati
        ve'],
16                 'enthalpy_residual_relative':d['enthalpy_balance_relative'],
17                 'density_interpolation_relative_error':d['checks_max']['rho_mixture_relative_err
        or'], 'elapsed_s':d['elapsed_s']})
18 checks={'time_refinement_12x6_difference_s':abs(rows[1]['critical_h']-rows[0]['critical_h'])*360
    0,
19         'grid12_to24_difference_min':abs(rows[2]['critical_h']-rows[1]['critical_h'])*60,
20         'grid24_to32_difference_min':abs(rows[3]['critical_h']-rows[2]['critical_h'])*60,
21         'selected_case':cases[-1],
22         'scope':'Numerical convergence of this conditional no-latent Q4 equation system; not
        real-material validation'}
23 (ROOT/'analysis/final-convergence.json').write_text(json.dumps({'cases':rows, 'comparisons':check
    s}, ensure_ascii=False, indent=2)+'\n')
24 with (ROOT/'delivery/convergence.csv').open('w', newline='', encoding='utf-8-sig') as f:
25     w=csv.DictWriter(f, fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
26 print(json.dumps(checks, ensure_ascii=False))
```

F.31 compare_run.py

类别：局部运动学模型 B。原始代码共 29 行。

```
1 """Execute one matched mapping case, preserving the frozen solver/export logic."""
2 import os
3 for name in
    ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS'):
4     os.environ[name]='1'
5 import argparse,json
6 from pathlib import Path
7 from minimal_model import Config,load_input
8 from experiment_model import Model as ModelA
9 from mapping_b import Model as ModelB
10 from solve_case import solve,export
11
12 p=argparse.ArgumentParser()
13 p.add_argument('--mapping',choices=['A','B'],required=True)
14 p.add_argument('--nr',type=int,required=True);p.add_argument('--nz',type=int,required=True)
15 p.add_argument('--out',type=Path,required=True);p.add_argument('--end-h',type=float,default=72.)
16 p.add_argument('--rtol',type=float,default=2e-7);p.add_argument('--max-step',type=float,default=
    150.)
17 p.add_argument('--atol-H',type=float,default=.001);p.add_argument('--atol-C',type=float,default=
    1e-9)
18 a=p.parse_args();root=Path(__file__).resolve().parents[1]
19 a.out.mkdir(parents=True,exist_ok=False)
20 cfg=Config(nr=a.nr,nz=a.nz,latent=0.,rtol=a.rtol,atol_H=a.atol_H,
21           atol_C=a.atol_C,max_step=a.max_step,emissivity=0.,desorption_B=0.)
22 env,rad=load_input(root/'input');m=({'A':ModelA,'B':ModelB}[a.mapping])(cfg,env,rad)
23 arr,diag=solve(m,a.end_h*3600,60.)
24 diag['mapping']=a.mapping
25 diag['scope']=f'Q4 paired local kinematics Map {a.mapping}; no latent; unchanged empirical Robin
    and conservative mass/enthalpy laws'
26 diag['known_limits']=[s.replace('Map A',f'Map {a.mapping}') for s in diag['known_limits']]
27 diag['kinematic_geometry_only_changed']=True
28 export(m,arr,diag,a.out)
29 print(json.dumps({k:diag[k] for k in ('mapping','critical_h','strict_stop_s','L_max_m','mass_bala
    nce_relative','enthalpy_balance_relative','elapsed_s')},ensure_ascii=False))
```

F.32 mapping_b.py

类别：局部运动学模型 B。原始代码共 78 行。

```
1 """Map B: planar cross-sections, curved interior cylindrical shells.
2
3 Only geometry and boundary area transforms differ from the frozen Map A.
4 All field equations and physical coefficients are inherited unchanged.
5 """
6 import numpy as np
7 from experiment_model import Model as ModelA
```

```

8
9
10 class Model(ModelA):
11     def geometry_data(self, t, C):
12         gd = super().geometry_data(t, C)
13         J = gd['J']
14         a = self.A[:,-1, None]
15         da = np.diff(self.A)[:,-1, None]
16         slope = np.diff(J, axis=0)/da
17         pieces = J[:-1]*(a*da+da**2/2)+slope*(a*da**2/2+da**3/3)
18         G = np.vstack((np.zeros((1, self.c.nz+1)), np.cumsum(pieces, axis=0)))
19         K = G[-1]
20         P = np.r_[0., np.cumsum((K[:-1]+K[1:])/2*np.diff(self.Z))]
21         if np.any(K <= 0):
22             raise ValueError('Nonpositive cross-sectional volume integral')
23         gd.update(radial_G=G, radial_K=K, axial_P=P)
24         return gd
25
26     def map_points(self, gd, i, j, xi, eta):
27         a0=self.A[i];da=self.A[i+1]-a0;dz=self.Z[j+1]-self.Z[j]
28         u=da*xi;v=dz*eta;a=a0+u
29         J=gd['J'];G=gd['radial_G'];K=gd['radial_K'];P=gd['axial_P'];R=gd['R']
30         # Integrate the SAME bilinear nodal J field used by Map A exactly.
31         J0=(1-eta)*J[i,j]+eta*J[i,j+1]
32         J1=(1-eta)*J[i+1,j]+eta*J[i+1,j+1]
33         Ja=(J1-J0)/da
34         Gq=(1-eta)*G[i,j]+eta*G[i,j+1]+J0*(a0*u+u*u/2)+Ja*(a0*u*u/2+u**3/3)
35         Jq=J0+Ja*u
36         J0z=(J[i,j+1]-J[i,j])/dz
37         Jaz=((J[i+1,j+1]-J[i+1,j])/dz-J0z)/da
38         Gz=(G[i,j+1]-G[i,j])/dz+J0z*(a0*u+u*u/2)+Jaz*(a0*u*u/2+u**3/3)
39         Kq=(1-eta)*K[j]+eta*K[j+1]
40         Kz=(K[j+1]-K[j])/dz
41         z=2/R**2*(P[j]+K[j]*v+.5*Kz*v*v)
42         zz=2*Kq/R**2
43         r=R*np.sqrt(Gq/Kq)
44         ra=np.divide(R**2*a*Jq,2*r*Kq,out=R*np.sqrt(Jq/(2*Kq)),where=r>0)
45         rz=np.divide(R**2*(Gz-Gq*Kz/Kq),2*r*Kq,out=np.zeros_like(r),where=r>0)
46         za=np.zeros_like(r)
47         m11=1/ra**2+(rz/(ra*zz))**2
48         m12=-rz/(ra*zz**2)
49         m22=1/zz**2
50         metric=np.stack([m11,m12,m12,m22],axis=-1).reshape(Jq.shape+(2,2))
51         hoop=np.divide(r,a,out=R*np.sqrt(Jq/(2*Kq)),where=a>0)
52         det=hoop*ra*zz
53         return {'J':Jq,'r':r,'z':z,'ra':ra,'rz':rz,'za':za,'zz':zz,
54                 'metric':metric,'det':det,'bar':2*Kq/self.c.R0**2}
55

```

```

56     def nodal_positions(self, gd):
57         r=gd['R']*np.sqrt(gd['radial_G']/gd['radial_K'][None,:])
58         z=2/gd['R']**2*gd['axial_P']
59         return r,np.broadcast_to(z[None,:],self.shape).copy()
60
61     def boundary(self, t, C, T, gd):
62         if float(t)!=gd['time_s']:
63             raise ValueError('Boundary/geometry time mismatch')
64         c=self.c
65         side=self.map_points(gd,np.full((c.nz,1),c.nr-1),np.arange(c.nz)[:None],
66                               np.ones((1,len(self.bg))),self.bg[None,:])
67         end=self.map_points(gd,np.arange(c.nr)[:None],np.full((c.nr,1),c.nz-1),
68                             self.bg[None,:],np.ones((1,len(self.bg))))
69         out=[]
70         for conn,geo,dL,kind in ((self.side_conn,side,np.diff(self.Z),'side'),
71                                   (self.end_conn,end,np.diff(self.A),'end')):
72             vals=self.th.surface(t,T[conn]@self.bN.T,C[conn]@self.bN.T)
73             if kind=='side':
74                 W=4*np.pi*gd['R']*geo['zz']*dL[:None]*self.bw
75             else:
76                 W=4*np.pi*geo['r']*geo['ra']*dL[:None]*self.bw
77             out.append((conn,W,vals))
78         return out

```

F.33 mapping_preflight.py

类别：局部运动学模型 B。原始代码共 70 行。

```

1  """Synthetic geometry identities and independent finite-difference checks."""
2  import json
3  from pathlib import Path
4  import numpy as np
5  from minimal_model import Config,load_input
6  from experiment_model import Model as ModelA
7  from mapping_b import Model as ModelB
8
9  root=Path(__file__).resolve().parents[1]
10 cfg=Config(nr=12,nz=6,latent=0.,emissivity=0.,desorption_B=0.)
11 env,rad=load_input(root/'input');models=[ModelA(cfg,env,rad),ModelB(cfg,env,rad)]
12 m=models[1];x=m.A[:None]/cfg.R0;y=m.Z[None,:]/m.H0
13 results={'data_kind':'synthetic','checks':[]}
14 def from_J(J):return (m.rhod0-760*J)/(90*J-m.rhod0)
15 for name,J in [(['uniform',np.ones(m.shape)),('separable',(.95-.15*x*x)*(0.95-.2*y*y)),
16                ('nonseparable',.90-.18*x*x-.12*y*y+.06*np.sin(np.pi*x)*np.sin(np.pi*y))]:
17     C=from_J(J).ravel();t=0. if name=='uniform' else 7200.
18     geo=[]
19     for model in models:
20         gd=model.geometry_data(t,C)
21         q=model.map_points(gd,model.ei[:None],model.ej[:None],model.xi[None,:],model.eta[None,:])

```

```

22 rn,zn=model.nodal_positions(gd)
23 weights=[float(W.sum()) for _,W,_ in model.boundary(t,C,np.full_like(C,28.),gd)]
24 areas=[2*np.pi*gd['R']*gd['L'],2*np.pi*gd['R']]
25 errors={'det':float(np.max(abs(q['J']-q['det']))),
26         'radius':float(np.max(abs(rn[-1]-gd['R']))),
27         'end_plane':float(np.max(abs(zn[:,-1]-gd['L']/2))),
28         'area_rel':float(np.max(abs(np.array(weights)/areas-1))),
29         'volume_rel':float(abs(np.sum(model.weights*q['J'])/(np.pi*gd['R']**2*gd['L']-1)))}
30 assert max(errors.values())<2e-12,errors
31 geo.append((rn,zn,gd,q))
32 results['checks'].append({'case':name,'map':'A' if model is models[0] else 'B',**errors})
33 diff=max(float(np.max(abs(geo[0][0]-geo[1][0])),float(np.max(abs(geo[0][1]-geo[1][1]))))
34 if name in ('uniform','separable'):assert diff<3e-15,diff
35 else:assert diff>1e-6,diff
36 results[name+'_map_position_difference_m']=diff
37
38 # Derivatives from central finite differences away from nodes and the axis.
39 C=from_J(.9-.18*x*-0.12*y*y+.06*np.sin(np.pi*x)*np.sin(np.pi*y)).ravel()
40 gd=m.geometry_data(7200.,C);i=np.array([3],[8]);j=np.array([2],[4])
41 xi=np.array([.23,.67]);eta=np.array([.31,.71]);h=1e-5
42 q=m.map_points(gd,i,j,xi,eta)
43 px=m.map_points(gd,i,j,xi+h,eta);mx=m.map_points(gd,i,j,xi-h,eta)
44 py=m.map_points(gd,i,j,xi,eta+h);my=m.map_points(gd,i,j,xi,eta-h)
45 derivs={
46     'ra':(px['r']-mx['r'])/(2*h*(m.A[i+1]-m.A[i])),
47     'za':(px['z']-mx['z'])/(2*h*(m.A[i+1]-m.A[i])),
48     'rz':(py['r']-my['r'])/(2*h*(m.Z[j+1]-m.Z[j])),
49     'zz':(py['z']-my['z'])/(2*h*(m.Z[j+1]-m.Z[j]))}
50 errors={k:float(np.max(abs(v-q[k]))) for k,v in derivs.items()}
51 assert max(errors.values())<2e-8,errors
52 results['finite_difference_derivative_absolute_error']=errors
53 F=np.stack([q['ra'],q['rz'],q['za'],q['zz']],axis=-1).reshape(q['r'].shape+(2,2))
54 Finv=np.linalg.inv(F);metric=Finv@np.swapaxes(Finv,-1,-2)
55 results['metric_matrix_inverse_error']=float(np.max(abs(metric-q['metric'])))
56 assert results['metric_matrix_inverse_error']<1e-12
57 initial=[model.rhs(0.,model.initial()) for model in models]
58 results['initial_rhs_A_B_max_abs_difference']=float(np.max(abs(initial[0]-initial[1])))
59 assert np.allclose(initial[0],initial[1],rtol=2e-12,atol=2e-10)
60 for label,model in zip('AB',models):
61     H=np.full_like(C,model.th.enthalpy(31.,1.)) # any positive admissible trial H
62     state=np.stack([H,C],axis=-1).ravel()
63     _,_,ck=model.terms(7200.,state,True)
64     results['instant_balance_'+label]={k:ck[k] for k in
65         ('instant_mass_balance_kg_s','instant_enthalpy_balance_W')}
66     assert abs(ck['instant_mass_balance_kg_s'])<1e-12
67     assert abs(ck['instant_enthalpy_balance_W'])<1e-8
68 results['status']='passed';results['material_validation']=False
69 (root/'analysis').mkdir(exist_ok=True)

```

```

69 (root/'analysis/mapping-preflight.json').write_text(json.dumps(results,ensure_ascii=False,indent,
    =2))
70 print(json.dumps(results,ensure_ascii=False,indent=2))

```

F.34 record_compare.py

类别：局部运动学模型 B。原始代码共 15 行。

```

1 """Hash declared sources/input and retain execution receipts for each new case."""
2 import argparse,subprocess,sys
3 from pathlib import Path
4 p=argparse.ArgumentParser();p.add_argument('mapping',choices=['A','B']);p.add_argument('nr',type,
    =int);p.add_argument('nz',type=int)
5 p.add_argument('--rtol',type=float,default=2e-7);p.add_argument('--max-step',type=float,default=,
    150.)
6 p.add_argument('--atol-H',type=float,default=.001);p.add_argument('--atol-C',type=float,default=,
    1e-9)
7 p.add_argument('--suffix',default='');a=p.parse_args()
8 root=Path(__file__).resolve().parents[1];run=f'map-{a.mapping}-{a.nr}x{a.nz}'+a.suffix;dest=f're,
    sults/{run}'
9 cmd=[sys.executable,'src/run_record.py','record','--case',str(root),'--run-id',run]
10 for f in sorted((root/'src').rglob('*.py')):cmd+=['--code',str(f.relative_to(root))]
11 for f in ('input/附件 1.xlsx','input/附件 2.xlsx','designs/scope.json'):cmd+=['--input',f]
12 for f in ('fields.npz','summary.json','trajectory.csv','midplane.csv','surface.csv','snapshots.c,
    sv'):cmd+=['--output',f'{dest}/{f}']
13 cmd+=['--data-kind','user-provided','--timeout','14400','--',sys.executable,'-B','src/compare_ru,
    n.py','--mapping',a.mapping,'--nr',str(a.nr),'--nz',str(a.nz),'--out',dest,'--rtol',str(a.rtol),,
    '--max-step',str(a.max_step),'--atol-H',str(a.atol_H),'--atol-C',str(a.atol_C)]
14 subprocess.run(cmd,cwd=root,check=True)
15 subprocess.run([sys.executable,'src/run_record.py','verify','--case',str(root),'--run-id',run],c,
    wd=root,check=True)

```

F.35 analyze_comparison.py

类别：局部运动学模型 B。原始代码共 117 行。

```

1 """Compare matched runs, with exact inversion of their Q1 volume-map geometry."""
2 from pathlib import Path
3 import sys,json,hashlib,csv
4 import numpy as np
5 from scipy.optimize import brentq
6 ROOT=Path(__file__).resolve().parents[1]
7 sys.path.insert(0,str(ROOT/'src'))
8 from minimal_model import Config,load_input
9 from experiment_model import Model as ModelA
10 from mapping_b import Model as ModelB
11 BASE=ROOT.parent/'model-system-20260912/variable-length/results'
12 env,rad=load_input(ROOT/'input')
13
14 def sample(m,gd,C,query,kind):

```

```

15     """Invert  $r^2 = \text{factor} \times \text{integral}(a)$  before interpolating C in reference a."""
16     C=C.reshape(m.shape)
17     values=gd['I'] if kind=='A' else gd['radial_G'][:,0]
18     density=gd['bar'] if kind=='A' else gd['J'][:,0]
19     factor=2/gd['lambda_z'] if kind=='A' else gd['R']**2/gd['radial_K'][0]
20     target=np.asarray(query)**2/factor
21     ids=np.clip(np.searchsorted(values,target,side='right')-1,0,m.c.nr-1)
22     a0=m.A[ids];da=m.A[ids+1]-a0;b=density[ids];slope=(density[ids+1]-b)/da
23     lo=np.zeros_like(target);hi=da.copy()
24     for _ in range(42):
25         u=(lo+hi)/2
26         integ=values[ids]+b*(a0*u+u*u/2)+slope*(a0*u*u/2+u**3/3)
27         low=integ<target;lo=np.where(low,u,lo);hi=np.where(low,hi,u)
28     xi=(lo+hi)/(2*da)
29     return (1-xi)*C[ids,0]+xi*C[ids+1,0]
30
31 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
32 pairs=[];provenance=[]
33 for nr,nz,suffix in ((12,6,'-tight'),(24,12,''),(32,16,'')):
34     ad=BASE/f'q4-robin-latent0-{nr}x{nz}{suffix}';bd=R00T/f'results/map-B-{nr}x{nz}'
35     sa=json.loads((ad/'summary.json').read_text());sb=json.loads((bd/'summary.json').read_text())
36     assert sa['config']==sb['config'],'Unmatched case parameters'
37     pairs.append({'nr':nr,'nz':nz,'A_h':sa['critical_h'],'B_h':sb['critical_h'],
38                 'B_minus_A_min':60*(sb['critical_h']-sa['critical_h']),
39                 'B_minus_A_percent':100*(sb['critical_h']/sa['critical_h']-1)})
40     for d in (ad,bd):
41         provenance.extend({'path':str(f.resolve()),'sha256':sha(f)} for f in
42                             (d/'summary.json',d/'fields.npz'))
43
44 ad=BASE/'q4-robin-latent0-32x16';bd=R00T/'results/map-B-32x16'
45 models=[];fields=[];mids=[];checks={};fractions=np.linspace(0,1,101)
46 for name,cls,d in (('A',ModelA,ad),('B',ModelB,bd)):
47     info=json.loads((d/'summary.json').read_text());cfgdata=info['config'].copy();cfgdata['equilibrium_C50']=.075
48     m=cls(Config(**cfgdata),env,rad);arr=np.load(d/'fields.npz');models.append(m);fields.append(arr)
49     ts=arr['times'];records=arr['records'];rn=arr['r_nodes_m'];zn=arr['z_nodes_m']
50     check={'saved_radius_abs_error_m':float(np.max(abs(rn[:,-1,:]-records[:,1,None]))),
51           'saved_end_plane_abs_error_m':float(np.max(abs(zn[:,:,:-1]-records[:,8,None]/2))),
52           'midplane_abs_error_m':float(np.max(abs(zn[:,:,:0]))),
53           'axis_abs_error_m':float(np.max(abs(rn[:,:,:0])))}
54     assert max(check.values())<2e-14
55     cm=[];legacy_diff=0.;inverse_diff=0.;area_error=0.;length_closure_error=0.
56     for n,(t,state) in enumerate(zip(ts,arr['states_H_C'])):
57         H,C,T=m.unpack(state);gd=m.geometry_data(t,C);query=gd['R']*fractions
58         cp=sample(m,gd,C,query,name);cm.append(cp)
59         legacy=np.interp(query,rn[n,:,:0],C.reshape(m.shape)[:,:0]);legacy_diff=max(legacy_diff,floor(
60             at(np.max(abs(cp-legacy))))

```

```

59         if n%60==0 or n==len(ts)-1:
60             weights=[float(w.sum()) for _,w,_ in m.boundary(t,C,T,gd)]
61             target=[2*np.pi*gd['R']*gd['L'],2*np.pi*gd['R']**2]
62             area_error=max(area_error,float(np.max(abs(np.array(weights)/target-1))))
63             length_by_mass=float(np.dot(m.mass,m.volume_law(C)/m.rhod0)/(np.pi*gd['R']**2))
64             length_closure_error=max(length_closure_error,abs(length_by_mass-gd['L']))
65         if n in (60,180,360,720):
66             for q in (gd['R']*0.25,gd['R']*0.5,gd['R']*0.75):
67                 def rr(a):
68                     i=min(int(np.searchsorted(m.A,a,side='right')-1),m.c.nr-1);i=max(i,0)
69                     return float(m.map_points(gd,np.array([[i]]),np.array([[0]]),np.array([(a-m.A[i])/(m.A[i+1]-m.A[i])])),np.array([[0.]])['r'])[0,0])
70                 aa=brentq(lambda a:rr(a)-q,0,m.c.R0,xtol=1e-15)
71                 ref=np.interp(aa,m.A,C.reshape(m.shape)[:0])
72                 inverse_diff=max(inverse_diff,abs(ref-float(sample(m,gd,C,np.array([q]),name)[0])))
73             check.update(max_legacy_midplane_sampling_difference=legacy_diff,independent_brentq_sample_difference=inverse_diff,
74                          sampled_boundary_area_relative_error=area_error,length_mass_integral_absolute_error=length_closure_error)
75         assert inverse_diff<1e-10 and area_error<1e-12 and length_closure_error<1e-12
76         checks[name]=check;mids.append(np.array(cm))
77
78     a,b=fields;assert np.array_equal(a['times'],b['times'])
79     times=a['times'];ra=a['records'];rb=b['records'];dc=mids[1]-mids[0];dl=rb[:,8]-ra[:,8]
80     loc=np.unravel_index(np.argmax(abs(dc)),dc.shape);li=int(np.argmax(abs(dl)))
81     selected=[]
82     for h in (1,3,6,12,24,36,48,72):
83         n=int(np.where(abs(times-h*3600)<1e-6)[0][0]);cfs=[]
84         for kind,m,arr in zip('AB',models,fields):
85             _,C,_=m.unpack(arr['states_H_C'][n]);gd=m.geometry_data(times[n],C)
86             cfs.append(sample(m,gd,C,np.array([0,.005,.01]),kind))
87         selected.append({'time_h':h,'A_length_cm':float(100*ra[n,8]),'B_length_cm':float(100*rb[n,8]),
88                        'delta_length_mm':float(1000*dl[n]),
89                        'A_mid_C_r0_r05_r10':cfs[0].tolist(),'B_mid_C_r0_r05_r10':cfs[1].tolist(),
90                        'max_midplane_abs_C_diff_101pts':float(np.max(abs(dc[n])))})
91     fine=pairs[-1];middle=pairs[-2]
92     sb_fine=json.loads((bd/'summary.json').read_text())
93     btight=json.loads((R00T/'results/map-B-12x6/summary.json').read_text())
94     bloose=json.loads((R00T/'results/map-B-12x6-loose/summary.json').read_text())
95     result={'data_sources':provenance,'matched_configs_exact':True,'grid_pairs':pairs,'fine_time_difference_min':fine['B_minus_A_min'],
96            'fine_time_relative_difference_percent':fine['B_minus_A_percent'],
97            'grid24_to32_A_min':60*(fine['A_h']-middle['A_h']),'grid24_to32_B_min':60*(fine['B_h']-middle['B_h']),
98            'paired_difference_change24_to32_seconds':60*(fine['B_minus_A_min']-middle['B_minus_A_min']),
99            'max_length_abs_difference_mm':float(abs(dl[li])*1000),'max_length_difference_time_h':float(times[li]/3600),

```

```

100     'max_midplane_abs_C_difference':float(abs(dc[loc])), 'max_midplane_difference_time_h':float(
        times[loc[0]]/3600),
101     'max_midplane_difference_radius_fraction':float(fractions[loc[1]]),
102     'sampling':'All saved times (60 s plus early times), 101 common physical radii
        r=R(t)*linspace(0,1,101); invert continuous map then reference Q1 interpolation.',
103     'B_time_refinement_12x6_seconds':3600*(btight['critical_h']-bloose['critical_h']),
104     'B_fine_summary':sb_fine,
105     'independent_saved_geometry_and_sampling_checks':checks, 'selected_history':selected,
106     'scope':'Matched A/B conditional kinematics with unchanged conservative transport and
        nonlatent enthalpy. Not all kinematics and not latent-model robustness.']}
107 (ROOT/'analysis/comparison.json').write_text(json.dumps(result,ensure_ascii=False,indent=2))
108 np.savez_compressed(ROOT/'analysis/comparison-data.npz',time_h=times/3600,length_A_cm=ra[:,8]*10,
        0,length_B_cm=rb[:,8]*100,
109         delta_length_mm=dl*1000,radial_fraction=fractions,midplane_A=mids[0],midplane_
        _B=mids[1],delta_midplane_C=dc,
110         radius_cm=ra[:,1]*100,grid_nr=np.array([p['nr'] for p in pairs]),
111         grid_A_h=np.array([p['A_h'] for p in pairs]),grid_B_h=np.array([p['B_h'] for p
        in pairs]))
112 with (ROOT/'analysis/selected-history.csv').open('w',newline='',encoding='utf-8-sig') as f:
113     w=csv.writer(f);w.writerow(['time_h','A_length_cm','B_length_cm','delta_length_mm','A_C0','B_
        C0','A_C05','B_C05','A_C10','B_C10','max_mid_abs_diff'])
114     for row in selected:
115         aa=row['A_mid_C_r0_r05_r10'];bb=row['B_mid_C_r0_r05_r10']
116         w.writerow([row['time_h'],row['A_length_cm'],row['B_length_cm'],row['delta_length_mm'],aa,
        [0],bb[0],aa[1],bb[1],aa[2],bb[2],row['max_midplane_abs_C_diff_101pts']])
117 print(json.dumps({k:v for k,v in result.items() if k not in
        ('data_sources','selected_history')},ensure_ascii=False,indent=2))

```

F.36 document_results.py

类别：局部运动学模型 B。原始代码共 70 行。

```

1 from pathlib import Path
2 import json,hashlib
3 import numpy as np
4 ROOT=Path(__file__).resolve().parents[1]
5 d=json.loads((ROOT/'analysis/comparison.json').read_text());fine=d['grid_pairs'][-1]
6 rows='\n'.join(f" | {p['nr']}\t\t{p['nz']}\t\t | {p['A_h']:.7f}\t\t | {p['B_h']:.7f}\t\t |
        {p['B_minus_A_min']:.5f}\t\t |" for p in d['grid_pairs'])
7 text=f'''# 第四问局部运动学对照：已执行结果
8
9 本轮仅比较两种无潜热变长度条件模型。A 保持内部圆柱壳为圆柱面；B 保持横截面为平面。
        两者沿用同一附录4物性、初态、附件环境及半径、真实质量通量、显焓关系和经验表面边界。
        没有拟合新材料参数，潜热分支未变动。
10
11 ## 核心结果
12
13 | 网格 | A / h | B / h | B 减 A / min |
14 |---|---:|---:|---:|

```

```

15 {rows}
16
17 细网格的配对差为 {60*fine['B_minus_A_min']:.5f} 秒，相对差 {fine['B_minus_A_percent']:.5f}%。中、
    细网格的配对差只改变 {abs(d['paired_difference_change24_to32_seconds']:.5f) 秒，
    但单个模型的网格变化仍约2.33分钟。前者支持这两种指定构造的总体时间接近；
    后者提醒不能将秒级时差当作材料预测精度。
18
19 - 0—72 h 保存时刻的最大长度差：{d['max_length_abs_difference_mm']:.7f} mm，发生在
    {d['max_length_difference_time_h']:.3f} h。
20 - 同时刻、同一物理中面101点的最大含水率差：{d['max_midplane_abs_C_difference']:.8f} kg/kg，发生在
    {d['max_midplane_difference_time_h']:.4f} h、
    r/R={d['max_midplane_difference_radius_fraction']:.2f}。这不是连续时空上界。
21 - B 的粗网格时间设置收紧后，达标时间改变 {d['B_time_refinement_12x6_seconds']:.6f} 秒；
    该检查不能代表任意网格的时间误差。
22 - B 细网格水量与显焓相对收支残差分别为 {d['B_fine_summary']['mass_balance_relative']:.4e}、
    {d['B_fine_summary']['enthalpy_balance_relative']:.4e}。
23 - 连续映射的局部体积恒等式和节点关系成立。非线性密度在积分点与几何插值仍有约0.3866%的最大相对差；
    没有把离散约束声称处处精确。
24
25 ## 物理解释
26
27 展开质量方程后存在密度梯度项；保持 D 数值不变并不意味着保持原输运方程不变。因此，约51.09 h至47.94
    h仍是守恒输运、几何、密度和水显焓等多项变化的综合差异。本轮A/B只替换局部映射，才是运动学的匹配对照。
28
29 给定同一个材料体积比场时，A和B具有相同外形、总长度及体积；分别耦合求解后，
    含水率不同才可能产生长度差。若体积比可分离，两映射完全重合。因此结论不能推广为任何运动学都不重要，
    也不能推广到未计算的含潜热A/B对照。
30
31 ## 实际执行与复现
32
33 新算 B 的12×6、24×12、32×16网格及12×6较松时间设置，共4次PDE求解。A的3次配对结果复用冻结来源，
    经执行记录和文件哈希核验；未为本轮重新运行A。
34
35 从本目录执行：
36
37 ```sh
38 /opt/anaconda3/bin/python src/record_compare.py B 32 16 --suffix=-recheck
39 /opt/anaconda3/bin/python analysis/analyze_comparison.py
40 /opt/anaconda3/bin/python analysis/plot_comparison.py
41 /opt/anaconda3/bin/python paper/revise_paper.py
42 ```
43
44 `--suffix` 为复算另建目录，保留本轮证据。精确环境与输入、代码、输出哈希见 runs/ 下 receipt.json；
    输入来源和冻结哈希见 frozen-input-code.json。数值分析目前引用同项目的冻结A结果目录，
    不应把本文件夹单独搬走后称作全项目独立运行包。
45
46 ## 文件导航

```

47

```
48 - 完整论文源稿: paper/manuscript.md; 新增公式位于第 6 节与附录 A, 配对结果位于 6.6 节。
49 - 机器可读结果: analysis/comparison.json; 精选时间表: analysis/selected-history.csv。
50 - 全部中面采样、长度曲线: analysis/comparison-data.npz; 真实B场: results/map-B-32x16/fields.npz。
51 - 映射实现: src/mapping_b.py; 独立推导: independent-derivation.md。
52 - 几何和响应图: figures/; 含 SVG、PDF、PNG、数据绑定及图形检查。
53 - 新版完整 PDF 和排版记录: paper/typeset-v3/, 最终交付以 delivery 中的文件及清单为准。
54
55 本目录是新条件实验与论文修订, 未覆盖原模型、原文稿或原结果。
56 '''
57 (ROOT/'README.md').write_text(text)
58 a=np.load(ROOT/'analysis/comparison-data.npz')
59 profile={'data_kind':'paired deterministic
        simulations','source':str(ROOT/'analysis/comparison-data.npz'),
60         'rows':len(a['time_h']),'radial_samples':len(a['radial_fraction']),
61         'variables':{k:{'shape':list(a[k].shape),'min':float(a[k].min()),'max':float(a[k].max()),
        'finite':bool(np.isfinite(a[k]).all())} for k in a.files},
62         'uncertainty':'No statistical intervals. Three paired grids and coarse-grid time
        refinement are reported separately.',
63         'sampling':d['sampling'],'transform':'No smoothing or fitting; geometry inversion before
        Q1 state interpolation.'}
64 (ROOT/'figures/data-profile.json').write_text(json.dumps(profile,ensure_ascii=False,indent=2))
65 files=[ROOT/'analysis/comparison.json',ROOT/'analysis/comparison-data.npz',ROOT/'analysis/select_
        ed-history.csv',ROOT/'analysis/analyze_comparison.py',ROOT/'analysis/plot_comparison.py']
66 record={'inputs':d['data_sources'],'outputs_and_scripts':[{ 'path':str(f.relative_to(ROOT)),'sha2_
        56':hashlib.sha256(f.read_bytes()).hexdigest()} for f in files],
67         'PDE_executions':'See four B run receipts; A was reused after verification.',
68         'postprocess_commands':['python analysis/analyze_comparison.py','python
        analysis/plot_comparison.py'],'postprocess_exit_status':0}
69 (ROOT/'analysis/postprocess-binding.json').write_text(json.dumps(record,ensure_ascii=False,inden_
        t=2))
70 print('Wrote README, DataProfile and postprocess binding')
```

F.37 plot_comparison.py

类别: 局部运动学模型 B。原始代码共 89 行。

```
1 from pathlib import Path
2 import sys,json,hashlib
3 import numpy as np
4 import matplotlib
5 matplotlib.use('Agg')
6 import matplotlib.pyplot as plt
7 ROOT=Path(__file__).resolve().parents[1];OUT=ROOT/'figures'
8 sys.path[:0]=[str(ROOT/'src'),'./support/skills/lab-ultra-scientific-figure-maker/scripts','./su_
        pport/skills/lab-ultra-scientific-visualization/scripts']
9 from figure_library import publication_style
10 from figure_export import export_figure
11 from figure_typography import inspect_figure_typography
```

```

12 from minimal_model import Config,load_input
13 from experiment_model import Model as ModelA
14 from mapping_b import Model as ModelB
15
16 BLUE='#244F73';GRAY='#7D8994';BLACK='#30363A'
17 source=R00T/'analysis/comparison-data.npz';D=np.load(source);S=json.loads((R00T/'analysis/compari
    ison.json').read_text())
18 provenance=[{'path':str(source),'sha256':hashlib.sha256(source.read_bytes()).hexdigest()}]
19 def save(fig,name,claim,desc,variables,source_records=None):
20     src=source_records or provenance
21     req={'schema_version':'1.0.0','figure_id':name,'claim':claim,'audience':'CUMCM paper
    readers','medium':'A4 paper',
22         'final_width_mm':158,'language':'zh-CN','data_sources':src,'variables':variables,
23         'uncertainty':{'kind':'none; paired deterministic conditional-model outputs, not
    confidence intervals'},'editable_format':'svg','claim_locked':True}
24     ir={'schema_version':'1.0.0','figure_id':name,'claim':claim,'route':'quantitative_code','reci
    pe':'paired_lines_and_geometry',
25         'panels':[{'panel_id':str(i),'purpose':ax.get_title()} for i,ax in enumerate(fig.axes)],
26         'encodings':[{'field':v['name'],'channel':'position','scale':'linear'} for v in variables],
27         'annotations':[],'style_tokens':{'palette':'blue-gray','base_font_pt':8.5},'exports':['sv
    g','pdf','png'],
28         'invariants':['data','units','claim','transform','uncertainty'],
29         'provenance':{'renderer':'matplotlib','source_script':str(Path(__file__)), 'materials':sty
    le}}
30     for suffix,value in
31         [ ('request',req), ('ir',ir), ('typography',inspect_figure_typography(fig,final_width_mm=158))]:
32         (OUT/f'{name}.{suffix}.json').write_text(json.dumps(value,ensure_ascii=False,indent=2,def
    ault=str))
33     export_figure(fig,OUT/name,formats=['svg','pdf','png'],dpi=280,overwrite=True,mkdir=True,
34         provenance={'sources':src,'materials':style},write_manifest=True,accessibility=
35         {'title':claim,'desc':desc})
36     plt.close(fig)
37
38 with publication_style('nature',language='zh-CN',width_mm=158,height_mm=118) as style:
39     plt.rcParams.update({'font.size':8.5,'axes.labelsize':8.5,'axes.titlesize':8.5,'legend.fontsi
    ze':7.5,
40
41         'axes.spines.top':False,'axes.spines.right':False,'svg.fonttype':'none'})
42     f,axes=plt.subplots(2,2,figsize=(158/25.4,118/25.4),layout='constrained')
43     t=D['time_h'];ax=axes[0,0]
44     ax.plot(t,D['length_A_cm'],color=BLUE,lw=1.25,label='映射 A')
45     ax.plot(t,D['length_B_cm'],color=GRAY,lw=1.15,ls='--',label='映射 B')
46     ax.set(title='(a) 长度历史',xlabel='时间 / h',ylabel='长度 /
    cm',xlim=(0,72),ylim=(24.5,33));ax.legend()
47     ax=axes[0,1];ax.plot(t,D['delta_length_mm'],color=BLUE,lw=1.1);ax.axhline(0,color=GRAY,lw=.7,
    ls=':')
48     ax.set(title='(b) 长度差的放大',xlabel='时间 / h',ylabel='B 减 A / mm',xlim=(0,72))
49     ax=axes[1,0]

```

```

47     for h,color in ((6,BLUE),(24,BLACK),(48,GRAY)):
48         i=int(np.where(abs(t-h)<1e-8)[0][0]);r=D['radial_fraction']*D['radius_cm'][i]
49         ax.plot(r,D['midplane_A'][i],color=color,lw=1.2,label=f'{h} h')
50         ax.plot(r,D['midplane_B'][i],color=color,lw=1.1,ls='--')
51     ax.set(title='(c) 中面含水率剖面',xlabel='当前物理半径 / cm',ylabel='含水率 / (kg/kg)',ylim=(0,1.7));ax.legend()
52     ax.text(.03,.04,'实线 A; 虚线 B',transform=ax.transAxes,fontsize=7.5)
53     ax=axes[1,1];ax.plot(t,np.max(abs(D['delta_midplane_C'])),axis=1,color=BLUE,lw=1.1)
54     ax.set(title='(d) 中面采样最大绝对差',xlabel='时间 / h',ylabel='含水率差 / (kg/kg)',xlim=(0,72));ax.set_ylim(bottom=0)
55     for ax in axes.ravel():ax.grid(axis='y',color='#E3E7EA',lw=.5)
56     save(f,'fig-kinematics-response','Compare time-resolved length and exact midplane moisture under two local maps',
57         'A/B at 32 by 16, same physics. Fine time difference and grid resolution are reported in the accompanying table. Panel d uses saved times and 101 common physical radial positions, not a continuous error bound.',
58         [{'name':'time_h','unit':'h','role':'x'},{'name':'length_A_cm,length_B_cm','unit':'cm','role':'response'},
59         {'name':'delta_length_mm','unit':'mm','role':'paired difference'},{'name':'midplane_A,midplane_B,delta_midplane_C','unit':'kg/kg','role':'moisture'}])
60
61     # A shared computed J field isolates geometry, without pretending to be B's evolved field.
62     datafile=R00T.parent/'model-system-20260912/variable-length/results/q4-robin-latent0-32x16/fields.npz'
63     data=np.load(datafile);i=int(np.where(data['times']==14400.)[0][0]);C=data['states_H_C'][i,:,:1]
64     env,rad=load_input(R00T/'input');cfg=Config(nr=32,nz=16,latent=0.,emissivity=0.,desorption_B=0.)
65     f,axes=plt.subplots(1,2,figsize=(158/25.4,83/25.4),layout='constrained')
66     geometry_data={}
67     for name,cls,ax in zip('AB',(ModelA,ModelB),axes):
68         m=cls(cfg,env,rad);gd=m.geometry_data(14400.,C)
69         def position(a,z):
70             ii=np.clip(np.searchsorted(m.A,a,side='right')-1,0,m.c.nr-1)
71             jj=np.clip(np.searchsorted(m.Z,z,side='right')-1,0,m.c.nz-1)
72             geo=m.map_points(gd,ii,jj,(a-m.A[ii])/(m.A[ii+1]-m.A[ii]),(z-m.Z[jj])/(m.Z[jj+1]-m.Z[jj]))
73             return geo['r']/gd['R'],2*geo['z']/gd['L']
74         curves=[]
75         for alpha in np.linspace(0,1,9):
76             z=np.linspace(0,m.H0,241);r,zz=position(np.full_like(z,alpha*cfg.R0),z)
77             ax.plot(r,zz,color=GRAY,lw=.8);curves.append(np.stack([r,zz]))
78         for beta in np.linspace(0,1,9):
79             a=np.linspace(0,cfg.R0,241);r,z=position(a,np.full_like(a,beta*m.H0))
80             ax.plot(r,z,color=BLUE,lw=.8);curves.append(np.stack([r,z]))
81     ax.set(xlim=(-.02,1.02),ylim=(-.02,1.02),xlabel='当前半径比例 r / R',ylabel='当前半长比例 2z / L',

```

```

82         title='(a) A: 圆柱壳保持圆柱面' if name=='A' else '(b) B: 横截面保持平面')
83         ax.set_aspect('equal');geometry_data[name]=np.array(curves)
84     np.savez_compressed(OUT/'fig-kinematics-maps.data.npz',**geometry_data)
85     src=[{'path':str(datafile),'sha256':hashlib.sha256(datafile.read_bytes()).hexdigest()},
86          {'path':str(R00T/'src/mapping_b.py'),'sha256':hashlib.sha256((R00T/'src/mapping_b.py').r_
            ead_bytes()).hexdigest()}]
87     save(f,'fig-kinematics-maps','A and B satisfy the same exterior and local volume but place
            interior material differently',
88          'Common J field from Map A at 4 h; blue curves are initial cross sections, gray curves are
            initial cylindrical shells. Both axes normalized by current exterior dimensions. This is
            a shared-state mapping illustration, not B evolved moisture evidence.',
89          [{'name':'A,B radial/axial material curves','unit':'1','role':'normalized
            positions'}],src)

```

F.38 assemble_report.py

类别：蒸发潜热条件实验。原始代码共 63 行。

```

1  """Build the delivery only after all four new PDE runs have completed and verified."""
2  import csv
3  import json
4  from pathlib import Path
5  import shutil
6  import subprocess
7  import sys
8
9  R00T=Path(__file__).resolve().parents[1]
10
11
12  def run(args, **kwargs):
13      return subprocess.run(args,cwd=R00T,check=True,**kwargs)
14
15
16  def main():
17      design=json.loads((R00T/'designs/endpoint-grid-refinement.json').read_text())
18      for c in design['cases']:
19          run([sys.executable,'-B','src/run_record.py','verify','--case','.','--run-id',c['name']])
20      output=['analysis/final/'+name for name in ['scientific-checks.json','cases.csv','pairs.csv',
            'grids.csv','trajectories.csv','key-times.csv']]
21      if not (R00T/'runs/final-field-audit/receipt.json').exists():
22          command=[sys.executable,'-B','src/run_record.py','record','--case','.','
23                  '--run-id','final-field-audit','--code','src/audit_branch.py',
24                  '--data-kind','user-provided','--timeout','180']
25          inputs=['designs/endpoint-grid-refinement.json','input/附件 1.xlsx','input/附件 2.xlsx']
26          paths=list((R00T/'evidence/reused').glob('*/*'))+list((R00T/'evidence/time-refinement').g
            lob('*/*'))
27          paths+=list((R00T/'results').glob('q4-*/*'))
28          inputs+=sorted(str(p.relative_to(R00T)) for p in paths if p.name in
            ('summary.json','fields.npz'))

```

```

29     for f in inputs:command+=['--input',f]
30     for f in output:command+=['--output',f]
31     command+=['--',sys.executable,'-B','src/audit_branch.py']
32     run(command)
33 run([sys.executable,'-B','src/run_record.py','verify','--case','.','--run-id','final-field-au_
dit'])
34 run([sys.executable,'-B','src/plot_branch.py'])
35 run([sys.executable,'-B','src/write_report.py'])
36 with (ROOT/'analysis/final/cases.csv').open() as f:cases=list(csv.DictReader(f))
37 maincases=[r for r in cases if int(r['nr'])==32 and float(r['shape_exponent'])==1.]
38 specs=[]
39 for r in maincases:
40     s=json.loads((ROOT/r['source']).read_text())
41     specs.append({'name':r['case'],'config':s['config'],'end_h':72,'save_step_s':600})
42 (ROOT/'designs/all-main-cases.json').write_text(json.dumps({'cases':specs},ensure_ascii=False,
,indent=2))
43 pandoc=shutil.which('pandoc')
44 if not pandoc:raise RuntimeError('Pandoc is required for report compilation; numerical replay
does not need it.')
45 run([pandoc,'report-print.md','--standalone','--to=latex',
46     '--include-in-header=latex-header.tex','-o','report.tex'])
47 compiler=Path('./local/.codex/plugins/cache/openai-bundled/latex/0.2.6/scripts/compile_latex.
py')
48 if compiler.exists():
49     result=run([sys.executable,str(compiler),str(ROOT/'report.tex'),'--compiler','texlive',
50         '--engine','xelatex','--output-directory',str(ROOT/'build'),'--json'],capture_
_output=True,text=True)
51     (ROOT/'analysis/latex-compile.json').write_text(result.stdout)
52     (ROOT/'analysis/latex-compile.stderr.txt').write_text(result.stderr)
53 else:
54     xelatex=shutil.which('xelatex')
55     if not xelatex:raise RuntimeError('XeLaTeX with ctex/Fandol is required for report
compilation.')
56     (ROOT/'build').mkdir(exist_ok=True)
57     for _ in
58         range(2):run([xelatex,'-no-shell-escape','-halt-on-error','-interaction=nonstopmode',
59             '-output-directory=build','report.tex'],capture_output=True,text=True)
59     shutil.copy2(ROOT/'build/report.pdf',ROOT/'蒸发潜热实验分支.pdf')
60     print('Built report, PDF and all six main-case replay specifications.',flush=True)
61
62
63 if __name__=='__main__':main()

```

F.39 audit_branch.py

类别：蒸发潜热条件实验。原始代码共 124 行。

```
1  """Recompute field, accounting, parameter-pair and mesh diagnostics from saved arrays."""
2  import argparse
3  import csv
4  import hashlib
5  import json
6  from pathlib import Path
7  import numpy as np
8
9  ROOT=Path(__file__).resolve().parents[1]
10
11
12  def csv_write(path, rows):
13      with path.open('w', newline='') as f:
14          writer=csv.DictWriter(f,fieldnames=list(rows[0]))
15          writer.writeheader();writer.writerows(rows)
16
17
18  def collect(include_new):
19      files=list((ROOT/'evidence/reused').glob('*/*.summary.json'))
20      files.append(ROOT/'evidence/time-refinement/summary.json')
21      if include_new:
22          files+=list((ROOT/'results').glob('q4-*/summary.json'))
23          expected=json.loads((ROOT/'designs/endpoint-grid-refinement.json').read_text())['cases']
24          for c in expected:
25              if not (ROOT/'results'/c['name']/'summary.json').exists():
26                  raise ValueError('Refinement incomplete: '+c['name'])
27      summaries={}; audit=[]; trajectories=[]; keytimes=[]
28      for p in sorted(files):
29          s=json.loads(p.read_text()); name=p.parent.name;cfg=s['config']
30          with np.load(p.parent/'fields.npz') as a:
31              u=a['states_H_C']; times=a['times'];rec=a['records']; mass=a['lumped_dry_mass']
32              C=u[:, :, 1];H=u[:, :, 0];T=28+H/(1850+4000*C)
33              Md=float(mass.sum());meanC=C@mass/Md;meanH=H@mass/Md
34              volumes=((1+C)/(760+90*C))@mass
35              cylinder=np.pi*rec[:, 1]**2*rec[:, 8]
36              b=a['budget_steps']
37              rm=b[:, 2]-cfg['C0']+b[:, 3]
38              re=b[:, 1]-meanH[0]+b[:, 4]
39              stop=a['strict_stop_state_H_C']
40              finite=all(np.isfinite(a[k]).all() for k in a.files if a[k].dtype.kind in 'fi')
41              tests={
42                  'finite':bool(finite),
43                  'positive_saved_moisture':bool(C.min(>0),
44                  'all_saved_Cmax_matches_record':bool(np.allclose(C.max(axis=1), rec[:, 4], rtol=0, atol=2e-12)),
```

```

45     'dry_mass_weighted_C_matches_record':bool(np.allclose(meanC,rec[:,2],rtol=0,atol=,
46     2e-12)),
47     'dry_mass_matches_initial_given_density':bool(np.isclose(Md,(760+90*2.55)/(1+2.55,
48     )*np.pi*.02**2*.25,rtol=2e-13)),
49     'mass_budget_residual_recomputed':bool(np.allclose(rm,b[:,8],atol=2e-12,rtol=0)),
50     'energy_budget_residual_recomputed':bool(np.allclose(re,b[:,9],atol=2e-8,rtol=0)),
51     'energy_channels_sum_to_total':bool(np.allclose(b[:,4],b[:,5]+b[:,6]+b[:,7],atol=,
52     2e-7,rtol=0)),
53     'last_saved_state_matches_budget':bool(np.allclose([meanH[-1],meanC[-1]],b[-1,1:3,
54     ],atol=2e-8,rtol=1e-13)),
55     'constitutive_total_volume_matches_geometry':bool(np.allclose(volumes,cylinder,rt,
56     ol=3e-12,atol=1e-16)),
57     'strict_stop_checks_all_nodes':bool(stop.size and stop[:,1].max()<.15 and
58     abs(stop[:,1].max()-s['strict_stop_Cmax'])<1e-12),
59     'given_h_hm_D_retained':bool(cfg['h']==25 and cfg['hm']==8e-7 and
60     cfg['D_scale']==1 and cfg['desorption_B']==0),
61     'accepted_activity_within_bound':bool(max(s['checks_max'][k] for k in ['max_activ,
62     ity','max_boundary_activity','max_volume_quadrature_activity'])<=1+1e-10),
63     'positive_deformation_determinant':bool(a['check_min_J'].min(>0),
64     'budget_relative_residuals_below_1e_6':bool(s['mass_balance_relative']<=1e-6 and
65     s['enthalpy_balance_relative']<=1e-6),
66 }
67 if cfg['latent']==1:
68     tests['physical_boundary_entropy_nonnegative_to_tolerance']=bool(a['check_min_bou,
69     ndary_entropy'].min(>=-1e-8)
70     tests['recorded_internal_entropy_nonnegative_to_tolerance']=bool(a['check_interna,
71     l_entropy_W_K'].min(>=-1e-8)
72 if not all(tests.values()):raise ValueError((name,{k:v for k,v in tests.items() if not
73     v}))
74 row={'case':name,'nr':cfg['nr'],'nz':cfg['nz'],'Ceq_reference':cfg['equilibrium_C50'],
75     'shape_exponent':cfg.get('oswin_b',1.),'latent':cfg['latent'],
76     'critical_h':s['critical_h'],'strict_stop_s':s['strict_stop_s'],
77     'mass_relative_residual':s['mass_balance_relative'],'energy_relative_residual':s,
78     ['enthalpy_balance_relative'],
79     'maximum_density_interpolation_error':s['checks_max']['rho_mixture_relative_error'],
80     'Lmax_cm':100*s['L_max_m'],'Lend_72h_cm':100*s['L_end_m'],
81     'observed_radius_fit_is_not_validation':True,
82     'source':str(p.relative_to(R00T)),
83     'source_sha256':hashlib.sha256(p.read_bytes()).hexdigest()}
84 audit.append({'case':name,'tests':tests,'metrics':row})
85 summaries[name]=(s,row)
86 # Keep time series exactly as saved; no smoothing or synthetic samples.
87 for k,t in enumerate(times):
88     trajectories.append({'case':name,'t_h':float(t/3600),'Cmax':float(C[k].max()),
89     'Cmean_dry':float(meanC[k]),'Tmean_dry_C':float(T[k]@mass/Md),
90     'Tcenter_C':float(T[k,0]),'Ccenter':float(C[k,0]),
91     'L_cm':float(100*rec[k,8]),'R_cm':float(100*rec[k,1])})

```

```

79         for t in [1800.,10800.,21600.,43200.,86400.,172800.,259200.]:
80             where=np.flatnonzero(times==t)
81             if len(where)!=1:raise ValueError((name,t,'required saved diagnostic time missing'))
82             k=int(where[0]);idx=-cfg['nz']-1
83             Dc=.00042*np.exp(-.30/C[k,0]-3850/(T[k,0]+273.15))
84             keytimes.append({'case':name,'t_h':t/3600,'Cmax':float(C[k].max()),'Cmean_dry':float(meanC[k]),
85                             'Tcenter_C':float(T[k,0]),'Tside_mid_C':float(T[k,idx]),
86                             'Ccenter':float(C[k,0]),'Cside_mid':float(C[k,idx]),
87                             'Dcenter_m2_s':float(Dc),'L_cm':float(100*rec[k,8])})
88     pairs=[];grids=[]
89     for ce in (30,75,120):
90         for n in (12,24,32):
91             names=[f'q4-latent{a}-ce{ce:03d}-{n}x{n//2}' for a in (0,1)]
92             if not all(x in summaries for x in names):continue
93             off,on=[summaries[x][0] for x in names]
94             changed=[k for k in on['config'] if on['config'][k]!=off['config'][k]]
95             if changed!=['latent']:raise ValueError((names,'pair differs beyond latent',changed))
96             pairs.append({'Ceq_reference':ce/1000,'nr':n,'nz':n//2,
97                           'off_h':off['critical_h'],'on_h':on['critical_h'],
98                           'delay_h':on['critical_h']-off['critical_h'],
99                           'delay_percent':100*(on['critical_h']/off['critical_h']-1),
100                          'only_config_difference':'latent'})
101         for a in (0,1):
102             names=[f'q4-latent{a}-ce{ce:03d}-{n}x{n//2}' for n in (12,24,32)]
103             if all(x in summaries for x in names):
104                 ds=[summaries[x][0] for x in names];cfgs=[d['config'] for d in ds]
105                 for c in cfgs[1:]:
106                     if {k for k in c if c[k]!=cfgs[0][k]}!=['nr','nz']:
107                         raise ValueError('Grid comparison changes physics')
108                 grids.append({'Ceq_reference':ce/1000,'latent':a,
109                              't12_h':ds[0]['critical_h'],'t24_h':ds[1]['critical_h'],'t32_h':ds[2]
110                               [['critical_h'],
111                               'change_24_to_32_minutes':60*(ds[2]['critical_h']-ds[1]['critical_h']),
112                               'meaning':'grid difference, not material prediction error'})
113     return audit,pairs,grids,trajectories,keytimes
114
115 if __name__=='__main__':
116     p=argparse.ArgumentParser();p.add_argument('--reused-only',action='store_true');args=p.parse_
117     args()
118     audit,pairs,grids,trajectories,keytimes=collect(not args.reused_only)
119     folder=ROOT/'analysis/'('reused-audit' if args.reused_only else 'final')
120     folder.mkdir(parents=True,exist_ok=True)
121     (folder/'scientific-checks.json').write_text(json.dumps({'scope':'Independent
122     saved-field/accounting reanalysis; not independent material validation',
123     'cases':audit,'all_declared_tests_pass':True,'new_grid_runs_included':not
124     args.reused_only},ensure_ascii=False,indent=2))

```

```

122     for name,rows in [(('cases',[x['metrics'] for x in audit]),('pairs',pairs),('grids',grids),('trajectories',trajectories),('key-times',keytimes))]:
123         csv_write(folder/(name+'.csv'),rows)
124     print(json.dumps({'cases_audited':len(audit),'pairs':len(pairs),'three_grid_comparisons':len(grids)},ensure_ascii=False))

```

F.40 import_evidence.py

类别：蒸发潜热条件实验。原始代码共 82 行。

```

1  """Verify existing execution receipts before reusing their numerical evidence."""
2  import hashlib
3  import importlib.util
4  import json
5  from pathlib import Path
6  import shutil
7
8  ROOT = Path(__file__).resolve().parents[1]
9  SOURCE = ROOT.parent/'surface-equilibrium-20260912'
10
11
12 def sha(path):
13     return hashlib.sha256(path.read_bytes()).hexdigest()
14
15
16 def main():
17     spec = importlib.util.spec_from_file_location('execution_records', ROOT/'src/run_record.py')
18     runner = importlib.util.module_from_spec(spec)
19     spec.loader.exec_module(runner)
20     imported = {r['case']: r for r in
21                 json.loads((SOURCE/'input/imported-results-manifest.json').read_text())}
22     receipts = list((SOURCE/'runs').glob('*/*receipt.json'))
23     names = [f'q4-latent{a}-ce{ce:03d}-{n}x{n//2}'
24              for a in (0,1) for ce in (30,75,120) for n in (12,24)]
25     names += [f'q4-latent{a}-ce075-32x16' for a in (0,1)]
26     names += [f'q4-oswin075-latent{a}-ce075-24x12' for a in (0,1)]
27     evidence=[]
28     for name in names:
29         source_dir = SOURCE/'results'/name
30         if (source_dir/'summary.json').exists():
31             matches=[]
32             for p in receipts:
33                 r=json.loads(p.read_text())
34                 if any(x['path']==f'results/{name}/summary.json' for x in r['outputs']):
35                     matches.append(p)
36             if len(matches)!=1: raise ValueError((name, 'ambiguous receipt', matches))
37             receipt=matches[0]
38         else:
39             source_dir=SOURCE/'input/imported-results'/name

```

```

39         receipt=Path(imported[name]['original_receipt'])
40         verification=runner.verify(receipt.parent.parent.parent, receipt.parent.name)
41         if verification['execution_binding']!='valid': raise ValueError((name,verification))
42         destination=R00T/'evidence/reused'/name
43         destination.mkdir(parents=True, exist_ok=False)
44         r=json.loads(receipt.read_text())
45         files=[]
46         for filename in ('summary.json','fields.npz'):
47             source=source_dir/filename
48             stamps=[x for x in r['outputs'] if x['path'].endswith('/'+filename)]
49             if len(stamps)!=1 or sha(source)!=stamps[0]['sha256']:
50                 raise ValueError((name,filename,'hash mismatch with verified original output'))
51             shutil.copy2(source,destination/filename)
52             files.append({'path':str((destination/filename).relative_to(R00T)),
53                           'sha256':sha(destination/filename),'original_file':str(source.resolve())})
54             shutil.copy2(receipt,destination/'original-receipt.json')
55             evidence.append({'case':name,'original_receipt':str(receipt),
56                             'original_execution_verified_now':verification,'files':files})
57 # Failure evidence is excluded from the valid drying-time matrix.
58 minimum=R00T.parent/'minimal-latent-boundary-20260912'
59 for n in (24,48):
60     name=f'q4-robin-latent1-10s-{n}x{n//2}'
61     verification=runner.verify(minimum,name)
62     if verification['execution_binding']!='valid':raise ValueError(verification)
63     destination=R00T/'evidence/failure'/name
64     destination.mkdir(parents=True,exist_ok=False)
65     files=[]
66     for filename in ('summary.json','trajectory.npz'):
67         source=minimum/'results'/name/filename
68         shutil.copy2(source,destination/filename)
69         files.append({'path':str((destination/filename).relative_to(R00T)),
70                       'sha256':sha(destination/filename),'original_file':str(source)})
71         shutil.copy2(minimum/'runs'/name/'receipt.json',destination/'original-receipt.json')
72         evidence.append({'case':name,'physical_status':'rejected specific closure combination',
73                         'original_execution_verified_now':verification,'files':files})
74 # Preserve statements on failed curve shapes and literature provenance.
75 for path in ['analysis/failure-diagnosis.json','表面边界来源补查.md','实验结论与模型选择.md']:
76     target=R00T/'evidence'/Path(path).name
77     shutil.copy2(SOURCE/path,target)
78 (R00T/'input/reused-evidence-manifest.json').write_text(json.dumps(evidence,ensure_ascii=False,
79                             e,indent=2))
80 print(json.dumps({'verified_imports':len(evidence),'time_matrix_cases':len(names)},ensure_ascii=False))
81
82 if __name__=='__main__':main()

```

F.41 launch_refinement.py

类别：蒸发潜热条件实验。原始代码共 59 行。

```
1 """Run the four approved-physics grid refinements, with separate execution records."""
2 import concurrent.futures
3 import json
4 from pathlib import Path
5 import subprocess
6 import sys
7 import time
8
9 ROOT = Path(__file__).resolve().parents[1]
10
11
12 def execute(case):
13     name = case['name']
14     command = [sys.executable, '-B', 'src/run_record.py', 'record', '--case', '.',
15               '--run-id', name, '--data-kind', 'user-provided', '--timeout', '7200']
16     for path in ['src/run_experiment.py', 'src/elongation_model.py',
17                 'src/elongation_frozen/__init__.py',
18                 'src/elongation_frozen/volume_fem.py',
19                 'src/elongation_frozen/boundary_model.py',
20                 'src/elongation_frozen/thermo_reference.py']:
21         command += ['--code', path]
22     for path in ['input/附件 1.xlsx', 'input/附件 2.xlsx',
23                 'input/approval-20260912-conditional.json', '试验方案.md',
24                 'input/current-scope.json', 'input/frozen-source-manifest.json',
25                 'designs/endpoint-grid-refinement.json']:
26         command += ['--input', path]
27     for path in ['summary.json', 'fields.npz']:
28         command += ['--output', f'results/{name}/{path}']
29     command += ['--', sys.executable, '-B', 'src/run_experiment.py', '--root', '.',
30               '--design', 'designs/endpoint-grid-refinement.json',
31               '--approval', 'input/approval-20260912-conditional.json', '--name', name]
32     start = time.time()
33     done = subprocess.run(command, cwd=ROOT, capture_output=True, text=True)
34     row = {'case': name, 'returncode': done.returncode, 'wall_seconds': time.time()-start,
35           'runner_stdout': done.stdout, 'runner_stderr': done.stderr}
36     if done.returncode == 0:
37         v = subprocess.run([sys.executable, '-B', 'src/run_record.py', 'verify',
38                             '--case', '.', '--run-id', name], cwd=ROOT,
39                             capture_output=True, text=True)
40         row['verification_returncode'] = v.returncode
41         row['verification_stdout'] = v.stdout
42     return row
43
44
45 if __name__ == '__main__':
```

```

46     cases = json.loads((ROOT/'designs/endpoint-grid-refinement.json').read_text())['cases']
47     status = [{'case': c['name'], 'state': 'queued'} for c in cases]
48     target = ROOT/'analysis/refinement-execution.json'
49     target.write_text(json.dumps(status, indent=2))
50     print('Starting four independent 32x16 grid refinements; no new physical parameters.',
          flush=True)
51     with concurrent.futures.ThreadPoolExecutor(max_workers=4) as pool:
52         futures = {pool.submit(execute, c): c['name'] for c in cases}
53         for future in concurrent.futures.as_completed(futures):
54             result = future.result()
55             status = [r for r in status if r['case'] != result['case']] + [result]
56             target.write_text(json.dumps(status, ensure_ascii=False, indent=2))
57             print(json.dumps(result, ensure_ascii=False), flush=True)
58     if any(r.get('returncode') != 0 or r.get('verification_returncode') != 0 for r in status):
59         raise SystemExit('At least one refinement did not finish with a valid execution binding.')

```

F.42 plot_branch.py

类别：蒸发潜热条件实验。原始代码共 133 行。

```

1  """Deterministic figures: matched times, thermal feedback, geometry and energy accounts."""
2  import os
3  from pathlib import Path
4  ROOT=Path(__file__).resolve().parents[1]
5  os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'cache/matplotlib'))
6  os.environ.setdefault('XDG_CACHE_HOME',str(ROOT/'cache'))
7  import argparse
8  import csv
9  import hashlib
10 import json
11 import sys
12 import numpy as np
13
14 SKILL=ROOT/'support'
15 sys.path.insert(0,str(SKILL/'lab-ultra-scientific-figure-maker/scripts'))
16 sys.path.insert(0,str(SKILL/'lab-ultra-scientific-visualization/scripts'))
17 from figure_library import publication_style
18 from figure_export import export_figure
19 from figure_typography import inspect_figure_typography
20 import matplotlib
21 matplotlib.use('Agg')
22 import matplotlib.pyplot as plt
23
24
25 def read_csv(p):
26     with p.open() as f: return list(csv.DictReader(f))
27
28
29 def hashfile(p): return hashlib.sha256(p.read_bytes()).hexdigest()

```

```

30
31
32 def main():
33     parser=argparse.ArgumentParser();parser.add_argument('--preview',action='store_true');args=pa
rser.parse_args()
34     analysis=ROOT/'analysis'/('reused-audit' if args.preview else 'final')
35     dest=ROOT/'figures'/('preview' if args.preview else
'final');dest.mkdir(parents=True,exist_ok=True)
36     pairs=read_csv(analysis/'pairs.csv'); trajectories=read_csv(analysis/'trajectories.csv')
37     n=24 if args.preview else 32
38     pairs=sorted([p for p in pairs if int(p['nr'])==n,key=lambda p:float(p['Ceq_reference'])]
39 if len(pairs)!=3:raise ValueError('All three parameter cases must be present before rendering')
40     groups={}
41     for alpha in (0,1):
42         name=f'q4-latent{alpha}-ce075-32x16'
43         groups[alpha]=[x for x in trajectories if x['case']==name]
44     def values(alpha,key):return np.array([float(x[key]) for x in groups[alpha]])
45     def export(fig,fid,claim,materials,data_paths,variables,encodings):
46         data=[{'path':str(p.relative_to(ROOT)),'sha256':hashfile(p)} for p in data_paths]
47         request={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'audience':'User reviewing
a conditional drying experiment',
48                 'medium':'A4 experiment
report','final_width_mm':160,'language':'zh-CN','data_sources':data,
49                 'variables':variables,'uncertainty':{'kind':'none; conditional parameter cases,
not confidence bounds'},
50                 'editable_format':'svg','claim_locked':True}
51         ir={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'route':'quantitative_code','r
ecipe':'matched points and physical time series',
52             'panels': [{'panel_id':chr(97+i),'purpose':ax.get_title()} for i,ax in
enumerate(fig.axes)],
53             'encodings':encodings,'annotations':[],
54             'style_tokens':{'palette':'okabe-ito','base_font_pt':8.5},'exports':['svg','pdf','png'],
55             'invariants':['data','units','claim','transform','uncertainty'],
56             'provenance':{'renderer':'matplotlib','materials':materials,'data_sources':data}}
57         (dest/(fid+'.request.json')).write_text(json.dumps(request,ensure_ascii=False,indent=2))
58         (dest/(fid+'.ir.json')).write_text(json.dumps(ir,ensure_ascii=False,indent=2))
59         profile={'source_kind':'deterministic simulation','paired':True,'missing_data_policy':'no
imputation; require complete matrix',
60                 'uncertainty':'No statistical uncertainty is encoded','data_sources':data,
61                 'rows':len(pairs) if fid=='paired-times' else len(groups[0])+len(groups[1]),
62                 'group_count':3 if fid=='paired-times' else 2}
63         (dest/(fid+'.profile.json')).write_text(json.dumps(profile,ensure_ascii=False,indent=2))
64         typography=inspect_figure_typography(fig,final_width_mm=160)
65         (dest/(fid+'.typography.json')).write_text(json.dumps(typography,ensure_ascii=False,inden
t=2,default=str))
66         export_figure(fig,dest/fid,formats=['svg','pdf','png'],mkdir=True,overwrite=True,dpi=220,
67                 provenance={'materials':materials,'data_sources':data},write_manifest=True,

```

```

68         accessibility={'title':fid,'desc':claim})
69     plt.close(fig)
70
71     blue='#0072B2';orange='#D55E00';gray='#666666'
72     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=66) as materials:
73         fig,ax=plt.subplots(layout='constrained')
74         for y,row in enumerate(pairs):
75             off=float(row['off_h']);on=float(row['on_h']);delta=float(row['delay_h'])
76             ax.plot([off,on],[y,y],color='#999999',lw=1.3,zorder=1)
77             ax.scatter([off],[y],color=blue,marker='o',s=30,label='关闭汽化相关焓差' if y==0 else
78                 None,zorder=3)
79             ax.scatter([on],[y],color=orange,marker='^',s=34,label='计入汽化相关焓差' if y==0 else
80                 None,zorder=3)
81             ax.annotate(f'+{delta:.2f} h',((off+on)/2,y),xytext=(0,8),textcoords='offset
82                 points',ha='center',fontsize=8)
83             ax.set(yticks=range(3),yticklabels=[f"{float(p['Ceq_reference']):.3f}" for p in pairs],
84                 xlabel='全域含水率达标时间 / h',ylabel='参考平衡含水率 / (kg/kg
85                 干物质)',xlim=(0,80),ylim=(-.6,2.6))
86             ax.grid(axis='x',alpha=.2);ax.legend(loc='upper left',frameon=False,fontsize=7.5)
87             ax.set_title(f'相同参数、相同 {n}x{n//2} 网格下的配对结果',loc='left')
88             export(fig,'paired-times','Three paired parameter scenarios; connectors show latent-switch
89                 effects, not confidence intervals.',materials,
90                 [analysis/'pairs.csv'],[{'name':'critical_h','unit':'h','role':'outcome'}],
91                 [{'field':'off_h,on_h','channel':'x','scale':'linear'},{'field':'Ceq_reference','c
92                 hannel':'y','scale':'categorical'}])
93
94     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=109) as materials:
95         fig,axes=plt.subplots(2,1,layout='constrained')
96         for a,color,label in [(0,blue,'关闭汽化相关焓差'),(1,orange,'计入汽化相关焓差')]:
97             axes[0].plot(values(a,'t_h'),values(a,'Tcenter_C'),color=color,linestyle='--' if a==0
98                 else '-',label=label)
99             axes[1].plot(values(a,'t_h'),values(a,'Cmax'),color=color,linestyle='--' if a==0 else
100                 '-',label=label)
101
102         from openpyxl import load_workbook
103         w=load_workbook(R00T/'input/附件 1.xlsx',read_only=True,data_only=True)
104         env=np.array(list(w.active.values)[1:],float);w.close()
105         axes[0].plot(env[:,0]/3600,env[:,1],color=gray,lw=1,label='题给烘房温度')
106         tail=env[env[:,0]>=10800,1].mean()
107         axes[0].plot([4,12],[tail,tail],color=gray,ls=':',lw=1,label='4 h 后环境延拓')
108         axes[0].set(xlim=(0,12),ylim=(20,55),xlabel='时间 / h',ylabel='温度 / °C',title='(a)
109             前期中心温度与烘房温度')
110         axes[0].legend(ncol=2,loc='lower right',frameon=False,fontsize=7.4)
111         axes[1].axhline(.15,color=gray,lw=1,ls=':');axes[1].annotate('达标阈值
112             0.15',(2,.15),xytext=(0,4),textcoords='offset points',fontsize=7.5)
113         axes[1].set(xlim=(0,72),ylim=(0,2.65),xlabel='时间 / h',ylabel='全域最大含水率 / (kg/kg
114             干物质)',title='(b) 水分场达到阈值的时间')
115         for ax in axes:ax.grid(alpha=.18)

```

```

104     export(fig, 'thermal-feedback', 'Reference equilibrium 0.075, 32x16 mesh: latent coupling
        lowers early center temperature and delays moisture threshold.', materials,
105         [analysis/'trajectories.csv', ROOT/'input/附件 1.xlsx'],
106         [{'name': 't_h', 'unit': 'h', 'role': 'independent'}, {'name': 'Tcenter_C', 'unit': 'degC',
            'role': 'outcome'}, {'name': 'Cmax', 'unit': 'kg/kg dry matter', 'role': 'outcome'}],
107         [{'field': 't_h', 'channel': 'x', 'scale': 'linear'}, {'field': 'Tcenter_C, Cmax', 'channel':
            'y', 'scale': 'linear'}])

108
109     with publication_style('nature', language='zh-CN', width_mm=160, height_mm=109) as materials:
110         fig, axes = plt.subplots(2, 1, layout='constrained')
111         for a, color, label in [(0, blue, '关闭汽化相关焓差'), (1, orange, '计入汽化相关焓差')]:
112             axes[0].plot(values(a, 't_h'), values(a, 'L_cm'), color=color, ls='--' if a==0 else
                '-', label=label)
113             axes[0].axhline(25, color=gray, lw=.8, ls='-')
114             axes[0].set(xlim=(0, 72), ylim=(20, 50), xlabel='时间 / h', ylabel='模型反推长度 /
                cm', title='(a) 长度是条件计算结果, 未经尺寸测量验证')
115             axes[0].legend(loc='upper right', frameon=False, fontsize=7.5)
116             source = ROOT/'evidence/reused/q4-latent1-ce075-32x16'
117             s = json.loads((source/'summary.json').read_text())
118             with np.load(source/'fields.npz') as arr: b = arr['budget_steps']
119             scale = s['dry_mass_kg']/1000
120             axes[1].plot(b[:, 0]/3600, -b[:, 5]*scale, color=blue, label='累计对流供热')
121             axes[1].plot(b[:, 0]/3600, b[:, 7]*scale, color=orange, ls='--', label='累计水分携焓流出')
122             axes[1].plot(b[:, 0]/3600, b[:, 1]*scale, color='#009E73', ls='-.', label='材料焓增加')
123             axes[1].set(xlim=(0, 72), ylim=(0, None), xlabel='时间 / h', ylabel='能量 / kJ', title='(b)
                含潜热参考算例的能量收支')
124             axes[1].legend(loc='upper left', frameon=False, fontsize=7.5)
125             for ax in axes: ax.grid(alpha=.18)
126             export(fig, 'geometry-energy', 'Reference parameter case: mass-volume closure implies
                unmeasured elongation; cumulative heat balances water enthalpy export and
                storage.', materials,
127                 [analysis/'trajectories.csv', source/'fields.npz', source/'summary.json'],
128                 [{'name': 'length', 'unit': 'cm', 'role': 'conditional
                    outcome'}, {'name': 'energy', 'unit': 'kJ', 'role': 'cumulative flux or storage'}],
129                 [{'field': 't_h, time_s/3600', 'channel': 'x', 'scale': 'linear'}, {'field': 'L_cm, -cum_co,
                    nvection_out*Md/1000, cum_water_enthalpy_out*Md/1000, mean_H*Md/1000', 'channel': 'y',
                    'scale': 'linear'}])

130     print(str(dest))
131
132
133 if __name__ == '__main__': main()

```

F.43 replay_case.py

类别：蒸发潜热条件实验。原始代码共 38 行。

```
1  """Replay one packaged Q4 case into a fresh directory without overwriting evidence."""
2  import argparse
3  import json
4  from pathlib import Path
5  import shutil
6  import subprocess
7  import sys
8  from datetime import datetime, timezone
9
10 ROOT=Path(__file__).resolve().parents[1]
11 CODE=['src/run_experiment.py','src/elongation_model.py','src/_elongation_frozen/__init__.py',
12       'src/_elongation_frozen/boundary_model.py','src/_elongation_frozen/thermo_reference.py',
13       'src/_elongation_frozen/volume_fem.py']
14 INPUT=['input/附件1.xlsx','input/附件2.xlsx','input/approval-20260912-conditional.json','试验方
    案.md',
15        'input/current-scope.json','input/frozen-source-manifest.json']
16
17
18 if __name__=='__main__':
19     p=argparse.ArgumentParser();p.add_argument('--case',required=True);p.add_argument('--destinat
    ion',type=Path);a=p.parse_args()
20     cases=json.loads((ROOT/'designs/all-main-cases.json').read_text())['cases']
21     matches=[c for c in cases if c['name']==a.case]
22     if len(matches)!=1:raise SystemExit('Unknown case; choose from designs/all-main-cases.json')
23     target=(a.destination or
    ROOT/'replays'/datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')).resolve()
24     target.mkdir(parents=True,exist_ok=False)
25     for rel in CODE+INPUT+['src/run_record.py']:
26         out=target/rel;out.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(ROOT/rel,out)
27     (target/'designs').mkdir()
28     (target/'designs/replay.json').write_text(json.dumps({'cases':matches},ensure_ascii=False,ind
    ent=2))
29     command=[sys.executable,'-B','src/run_record.py','record','--case','.','--run-id','replay',
30              '--data-kind','user-provided','--timeout','7200']
31     for rel in CODE:command+=['--code',rel]
32     for rel in INPUT+['designs/replay.json']:command+=['--input',rel]
33     for filename in
        ['summary.json','fields.npz']:command+=['--output',f'results/{a.case}/{filename}']
34     command+=['--',sys.executable,'-B','src/run_experiment.py','--root','.','.',
35               '--design','designs/replay.json','--approval','input/approval-20260912-conditional.
        json','--name',a.case]
36     print('Fresh replay directory:',target,flush=True)
37     subprocess.run(command,cwd=target,check=True)
38     subprocess.run([sys.executable,'-B','src/run_record.py','verify','--case','.','--run-id','rep
        lay'],cwd=target,check=True)
```

F.44 run_experiment.py

类别：蒸发潜热条件实验。原始代码共 126 行。

```
1 """Run one explicitly approved conditional trial; never overwrite old results.
2
3 This entry point does not turn a successful numerical run into physical validation.
4 The approval snapshot records the user's requested boundary between a proposal and
5 a permitted material experiment. Synthetic geometry tests use a separate entry.
6 """
7 import os
8 for thread_variable in ('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS'):
9     os.environ[thread_variable] = '1'
10
11 import argparse
12 from dataclasses import asdict
13 import hashlib
14 import json
15 from pathlib import Path
16 import platform
17 import sys
18 import traceback
19 import numpy as np
20 import scipy
21
22
23 def file_digest(path):
24     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
25
26
27 def check_approval(root, approval_path):
28     approval = json.loads(approval_path.read_text())
29     required = ('geometry_and_initial', 'surface_and_phase', 'environment_and_energy')
30     if approval.get('scope') != 'conditional_q4_elongation_latent_trial':
31         raise ValueError('Wrong approval scope')
32     if any(approval.get('groups', {}).get(key) is not True for key in required):
33         raise ValueError('Physical assumptions still await the user response; no material solve
34                             started')
35     if not approval.get('user_reply'):
36         raise ValueError('Approval must include the actual user reply')
37     if approval.get('proposal_sha256') != file_digest(root/'试验方案.md'):
38         raise ValueError('Proposal differs from the approved snapshot')
39     return approval
40
41 def enrich(arr, diag, model):
42     b = arr['budget_steps']
43     cumulative = np.vstack([np.zeros(5), b[:, 3:8]])
44     total_variations = np.abs(np.diff(cumulative, axis=0)).sum(axis=0)
```

```

45     mass_scale = max(model.c.C0, float(total_variations[0]))
46     heat_scale = max(model.th.cs + model.th.cw*model.c.C0,
47                       float(total_variations[2:5].sum()))
48     initial_mean_H = float(model.mass @ model.unpack(model.initial())[0] / model.Mdry)
49     diag.update({
50         'experiment': 'Q4 observed radius, variable length, conditional map A, coupled latent
                    heat',
51         'config': asdict(model.c),
52         'mass_balance_relative': float(diag['mass_residual_max_kg_perkgdry']/mass_scale),
53         'enthalpy_balance_relative': float(diag['enthalpy_residual_max_J_perkgdry']/heat_scale),
54         'budget_scales': {'mass_kg_perkgdry': mass_scale, 'enthalpy_J_perkgdry': heat_scale,
55                           'channel_total_variations': total_variations.tolist()},
56         'physical_vapor_energy_included': bool(model.c.latent == 1),
57         'latent_zero_is_only_diagnostic': bool(model.c.latent == 0),
58         'radius_role': 'prescribed input, not held-out validation',
59         'length_role': 'conditional computed output, not an observation',
60         'constitutive_parameters_fitted': False,
61         'solid_mechanics_solved': False,
62         'accepted_as_main_model': False,
63         'equilibrium_reference': {'C_kg_perkgdry': model.c.equilibrium_C50,
64                                   'T_degC': float(model.th.tail[0]),
65                                   'air_humidity_kg_perkgdryair': float(model.th.tail[1]),
66                                   's_parameter': float(model.th.s)},
67         'integrated_channels_actual_units': {
68             'mass_out_kg': float(b[-1, 3]*model.Mdry),
69             'total_energy_out_J': float(b[-1, 4]*model.Mdry),
70             'convection_heat_in_J': float(-b[-1, 5]*model.Mdry),
71             'radiation_heat_in_J': float(-b[-1, 6]*model.Mdry),
72             'carried_water_enthalpy_out_J': float(b[-1, 7]*model.Mdry),
73             'material_enthalpy_change_J': float((b[-1, 1]-initial_mean_H)*model.Mdry),
74             'max_mass_residual_kg': float(diag['mass_residual_max_kg_perkgdry']*model.Mdry),
75             'max_enthalpy_residual_J': float(diag['enthalpy_residual_max_J_perkgdry']*model.Mdry)},
76         'runtime': {'python': sys.version, 'numpy': np.__version__,
77                     'scipy': scipy.__version__, 'platform': platform.platform()},
78     })
79     diag['conservation_audit_pass_at_1e_6'] = bool(
80         diag['mass_balance_relative'] <= 1e-6 and diag['enthalpy_balance_relative'] <= 1e-6)
81     return diag
82
83
84 def main():
85     parser = argparse.ArgumentParser()
86     parser.add_argument('--root', type=Path, required=True)
87     parser.add_argument('--design', type=Path, required=True)
88     parser.add_argument('--approval', type=Path, required=True)
89     parser.add_argument('--name', required=True)
90     args = parser.parse_args()
91     root = args.root.resolve()

```

```

92     check_approval(root, args.approval.resolve())
93     design = json.loads(args.design.read_text())
94     matches = [case for case in design['cases'] if case['name'] == args.name]
95     if len(matches) != 1:
96         raise ValueError('Case name must match exactly one design entry')
97     spec = matches[0]
98     destination = root/'results'/spec['name']
99     if destination.exists():
100         raise ValueError('Existing case output is never overwritten')
101     destination.mkdir(parents=True)
102     (destination/'running.json').write_text(json.dumps({'case': spec, 'status': 'starting'},
103                                                    indent=2))
104     try:
105         # Imported only after the consent check, to keep proposal preparation
106         # distinct from approved material execution.
107         from elongation_model import Config, Model, load_input, solve
108         env, radius = load_input(root/'input')
109         model = Model(Config(**spec['config']), env, radius)
110         arrays, diagnostics = solve(model, spec['end_h']*3600, spec.get('save_step_s', 600.))
111         diagnostics = enrich(arrays, diagnostics, model)
112         diagnostics['case_name'] = spec['name']
113         diagnostics['design_sha256'] = file_digest(args.design)
114         diagnostics['approval_sha256'] = file_digest(args.approval)
115         np.savez_compressed(destination/'fields.npz', **arrays)
116         (destination/'summary.json').write_text(json.dumps(diagnostics, ensure_ascii=False,
117                                                         indent=2, allow_nan=False)+'\n')
118         print(json.dumps({'name': spec['name'], 'elapsed_s': diagnostics['elapsed_s'],
119                         'critical_h': diagnostics['critical_h'], 'Cmax_end':
120                         diagnostics['Cmax_end'],
121                         'mass_balance_relative': diagnostics['mass_balance_relative'],
122                         'enthalpy_balance_relative': diagnostics['enthalpy_balance_relative']},
123                        ensure_ascii=False), flush=True)
124     except Exception:
125         (destination/'failure.txt').write_text(traceback.format_exc())
126         raise
127
128 if __name__ == '__main__':
129     main()

```

F.45 verify_package.py

类别：蒸发潜热条件实验。原始代码共 20 行。

```

1  """Check packaged file integrity. This does not certify material-model correctness."""
2  import hashlib
3  import json
4  from pathlib import Path
5

```

```

6 ROOT=Path(__file__).resolve().parents[1]
7
8
9 if __name__=='__main__':
10     manifest=json.loads((ROOT/'package-manifest.json').read_text())
11     errors=[]
12     for row in manifest['files']:
13         file=(ROOT/row['path']).resolve()
14         if not file.is_relative_to(ROOT) or not file.is_file():
15             errors.append({'path':row['path'],'error':'missing or outside package'});continue
16         if hashlib.sha256(file.read_bytes()).hexdigest()!=row['sha256']:
17             errors.append({'path':row['path'],'error':'hash changed'})
18     print(json.dumps({'files_checked':len(manifest['files']),'integrity_valid':not errors,
19                     'scientific_validation':'not certified by file
20                     hashes','errors':errors},ensure_ascii=False,indent=2))
21     raise SystemExit(1 if errors else 0)

```

F.46 write_report.py

类别：蒸发潜热条件实验。原始代码共 336 行。

```

1 """Produce a bounded experiment report from audited results, never hand-entered times."""
2 import csv
3 import json
4 from pathlib import Path
5
6 ROOT=Path(__file__).resolve().parents[1]
7
8
9 def rows(name):
10     with (ROOT/'analysis/final'/name).open() as f:return list(csv.DictReader(f))
11
12
13 def table(header, data):
14     return '\n'.join(['| '+' | '.join(header)+' |','| '+' | '.join(['---']*len(header))+ ' |']+
15                     ['| '+' | '.join(map(str,r))+ ' | ' for r in data])
16
17
18 def main():
19     audit=json.loads((ROOT/'analysis/final/scientific-checks.json').read_text())
20     if not audit['all_declared_tests_pass'] or len(audit['cases'])!=21:
21         raise ValueError('Complete audited evidence required before report generation')
22     pairs=[r for r in rows('pairs.csv') if int(r['nr'])==32]
23     grids=rows('grids.csv'); cases=rows('cases.csv'); key=rows('key-times.csv')
24     maincases=[r for r in cases if int(r['nr'])==32 and float(r['shape_exponent'])==1.]
25     time_refinement=json.loads((ROOT/'analysis/time-refinement.json').read_text())
26     pairs.sort(key=lambda r:float(r['Ceq_reference']))
27     pair_table=table(['参考平衡含水率','关闭汽化相关焓差 / h','计入汽化相关焓差 / h','配对增量 /
h'],

```

```

28         [[f"{float(r['Ceq_reference']):.3f}",f"{float(r['off_h']):.2f}",f"{float(r['',
on_h']):.2f}",f"+{float(r['delay_h']):.2f}"] for r in pairs])
29 grid_table=table(['参考平衡含水率','焓差开关','12×6 / h','24×12 / h','32×16 / h','最后一次变化
/ min'],
30                 [[f"{float(r['Ceq_reference']):.3f}",'计入' if float(r['latent']) else '关闭',
31                  * [f"{float(r[k]):.4f}" for k in
                      ['t12_h','t24_h','t32_h']],f"{float(r['change_24_to_32_minutes']):+.3f}"
                      for r in grids])
32 length_table=table(['参考平衡含水率','焓差开关','最大模型长度 / cm','72 h 时模型长度 / cm'],
33                    [[f"{float(r['Ceq_reference']):.3f}",'计入' if float(r['latent']) else '关闭',
34                     f"{float(r['Lmax_cm']):.2f}",f"{float(r['Lend_72h_cm']):.2f}"] for r in
                      sorted(maincases,key=lambda
                          r:(float(r['Ceq_reference']),float(r['latent'])))])
35 tc=[]
36 for a in (0,1):
37     k=next(r for r in key if r['case']==f'q4-latent{a}-ce075-32x16' and float(r['t_h'])==3)
38     tc.append(k)
39 temp_table=table(['参考算例: 3 h','中心温度 / °C','中心含水率 / kg/kg','中心扩散系数 / m²/s'],
40                 [['关闭汽化相关焓差',f"{float(tc[0]['Tcenter_C']):.3f}",f"{float(tc[0]['Ccente',
r']):.4f}",f"{float(tc[0]['Dcenter_m2_s']):.5e}"],
41                 ['计入汽化相关焓差',f"{float(tc[1]['Tcenter_C']):.3f}",f"{float(tc[1]['Ccente',
r']):.4f}",f"{float(tc[1]['Dcenter_m2_s']):.5e}"]])
42 failure=[]
43 for n in (24,48):
44     s=json.loads((R00T/f'evidence/failure/q4-robin-latent1-10s-{n}x{n//2}/summary.json').read_
_text())
45     d=s['direction']['worst']
46     failure.append([f'{n}x{n//2}',f"{d['surface_T_C']:.3f}",f"{d['psat_surface_Pa']:.1f}",f"{
d['air_pv_Pa']:.1f}",
47                    f"{d['J_out_kg_m2_s']:.3e}",f"{100*s['finite_margin_area_check']['gap_belo,
w_minus_100Pa_fraction']:.0f}%")])
48 failure_table=table(['网格','最不利表面温度 / °C','饱和压 / Pa','空气水汽压 / Pa','向外通量 /
kg/(m²s)','压差低于 -100 Pa 的面积比例'],failure)
49 report=r'''---
50 title: 蒸发潜热耦合实验分支
51 subtitle: 第四问的守恒配对、边界筛选与参数敏感性
52 date: 2026 年 9 月 12 日
53 lang: zh-CN
54 documentclass: ctexart
55 classoption:
56     - fontset=fandol
57     - zihao=5
58 geometry:
59     - margin=22mm
60 colorlinks: true
61 linkcolor: black
62 urlcolor: blue

```

63 ---

64

65 # 1. 实验回答了什么

66

67 **在已经声明的表面关系和变形假设下，含潜热模型能够完成第四问的全程计算，并通过本轮质量、能量、几何和边界检查；但表面参数尚未标定，绝对烘干时间不能当作题给数据唯一确定的答案。**

68

69 本分支检验三件事：同一模型只开关汽化相关焓差，结果改变多少；这种改变是否可能只是网格误差；直接把原经验排水边界接上潜热，会不会出现物理方向错误。它不替换正式论文主结果，也不把第四问的数据用于前三问。

70

71 下面每一行都使用相同的 32×16 网格、题给物性、传质系数及同一表面关系。两列之间只改变水排出时携带的汽化相关焓差。三个参考平衡含水率均为事先获准的条件值，不是拟合值。

72

73 @@PAIR_TABLE@@

74

75 这些结果支持“在本模型的三组条件下，汽化耗能明显延长达标时间”。它们不支持“真实时长必然落在这三个算例之间”，也不构成统计置信区间。

76

77 ![相同网格下的成对结果。连接线连接同一参数的两个计算，不表示误差条或置信区间。
](figures/final/paired-times.png){#fig-paired width=160mm}

78

79 # 2. 模型具体计算什么

80

81 ## 2.1 状态、物性与守恒

82

83 计算对象是二维轴对称的完整圆柱，利用中面对称只离散半根长度，积分权重恢复整根质量和表面积。温度为 T (°C)，材料干基含水率为 C (kg 水/kg 干物质)，当前干物质体积密度为 ρ_d 。第四问采用附录4：

84

85
$$\begin{aligned} \rho(C) &= 760 + 90C, \quad \& \quad \rho_d = \frac{\rho(C)}{1+C}, \quad \& \quad \\ c_p(C) &= 1850 + \frac{2150C}{1+C}, \quad \& \quad k(C) = 0.12 + \frac{0.20C}{1+C}, \quad \& \quad \\ D(C, T) &= 0.00042 \exp\left(-\frac{0.30}{C} - \frac{3850}{T+273.15}\right). \end{aligned} \tag{1}$$

86

87

88

89

90

91

92

93 式 (1) 按题给单位使用，内部统一为 SI；温度指数使用 K。密度为 kg/m³，比热为 J/(kg·K)，导热系数为 W/(m·K)，扩散系数为 m²/s。密度在本分支中用于当前质量—体积关系，没有另换成一个只用于热量计算的同名参数。

94

95 以单位干物质质量计的材料焓及水分有效携焓为

96

97
$$H = (1850 + 4000C)(T - 28), \quad \& \quad h_b(T) = 4000(T - 28). \tag{2}$$

98

99

100

101 因为 $(1+C)c_p(C)=1850+4000C$ ，式（2）保留了题给比热随失水变化的作用。记骨架速度为 $\mathbf{v}$ 、
水相对骨架的质量通量为 $\mathbf{j}$ ，则采用

102

103
$$\begin{aligned} & \frac{\partial}{\partial t}(\rho_d + \nabla \cdot (\rho_d \mathbf{v})) = 0, \\ & \frac{\partial}{\partial t}(\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v} + \mathbf{j}) = 0, \\ & \frac{\partial}{\partial t}(\rho_d H) + \nabla \cdot (\rho_d H \mathbf{v} + \mathbf{q} + h_b \mathbf{j}) = 0, \\ & \mathbf{j} = -\rho_d \nabla C, \quad \mathbf{q} = -k \nabla T. \end{aligned}$$

104
$$\tag{3}$$

105
$$\tag{3}$$

106 这里的轴对称散度为 $r^{-1}(\partial_r(r f_r) + \partial_z f_z)$ ；不是直角坐标下的一维二阶导数。式（3）
暂不单独求机械储能、辐射、内部气液两相和额外脱附热，相关限制列于第3节。

107

108 初始条件是全域 $T=28^\circ\text{C}$ 、 $C=2.55$ ；中心轴和中面对称，无穿越它们的热、水分通量。
侧面和两个端面均参与交换，使用当前几何的真实积分面积。

109

110 ## 2.2 表面交换：同一通量用于失水和耗能

111

112 环境湿含量 w_a 的分母是干空气质量；材料含水率 C 的分母是干物质质量。
边界先把空气湿含量换成水汽分压，再计算材料表面的平衡状态：

113

114
$$\begin{aligned} p_a &= P \frac{w_a}{0.62198 + w_a}, \\ a_w(T, C) &= g(T) \frac{C}{C + s}, \\ C_{\text{eq}}(T_s, p_a) &= \frac{s p_a}{g(T_s) p_{\text{sat}}(T_s) - p_a}, \\ J_s &= \rho_d (C_s) h_m, [C_s - C_{\text{eq}}(T_s, p_a)]. \end{aligned}$$

115
$$\tag{4}$$

116
$$\tag{4}$$

117 $J_s > 0$ 表示水向外流出，单位 $\text{kg}/(\text{m}^2 \cdot \text{s})$ ； $h_m = 8 \times 10^{-7} \text{ m/s}$ 保留题给值。
本分支将它解释为材料干基含水率差对应的有效系数，乘 ρ_d 转成质量通量。
保留数值不等于题目已经给出了这一边界解释。

118

119 s 是由参考平衡含水率换算得到的曲线参数；三个参考值为0.03、0.075、0.12，参考环境为附件末小时均值
(约50 °C、 $w_a \approx 0.05$)。没有通过半径数据拟合 s 。温度修正沿用与式（2）配套的条件关系：

120

121
$$g(T) = \exp\left\{\frac{4186 - 4000}{461.52}\right\}$$

122
$$\left[\log\frac{T+273.15}{301.15} + \frac{301.15}{T+273.15} - 1\right]\right\}.\tag{5}$$

123
$$\tag{5}$$

124 活动度关系是有文献形式依据的候选近似，并非本药材的实测等温线[1–3]。式（4）反解要求分母为正；
执行中检查此定义域和活动度范围。它不是可覆盖任意凝结工况的通用公式。

125

126

127

128

129

140 饱和压沿用常定比热近似。令 $\theta = T + 273.15$ K、 $\theta_0 = 301.15$ K、 $\Delta c = 1860 - 4186 = -2326$ J/(kg · K)，则

141

142
$$\log \frac{p_{\text{sat}}(T)}{3783.1 \text{ Pa}} = \frac{(2434600 - \Delta c \cdot \theta_0)(1/\theta_0 - 1/\theta) + \Delta c \log(\theta/\theta_0)}{461.52} \tag{5a}$$

143

144

145

146

147

148 式 (5a) 中 2434600 的单位为 J/kg，461.52 的单位为 J/(kg · K)。由参考状态代入式 (4) 得到 $s = C_{\text{ref}}[g(T_{\text{ref}})p_{\text{sat}}(T_{\text{ref}})/p_a - 1]$ ，不是回归拟合。

149

150 这条关系的一个可直接检验的优点是交换方向一致。记 $A = g(T_s)p_{\text{sat}}(T_s)$ 、 $p_s = a_w(T_s, C_s)p_{\text{sat}}(T_s)$ ，在 $A > p_a$ 的定义域内有

151

152
$$p_s - p_a = \frac{A - p_a}{C_s + s}(C_s - C_{\text{eq}}), \quad J_s(p_s - p_a) = \rho_d h_m \frac{A - p_a}{C_s + s}(C_s - C_{\text{eq}})^2 \geq 0 \tag{5b}$$

153

154

155

156

157 式 (5b) 保证该定义域内质量通量与模型自身的水汽压驱动力同向；配合活动度上界检查，可以排除第5节的逆向蒸发情形。但它不能证明参数 s 属于真实药材，也不能单独代替跨温度的完整能量和熵检查。

158

159 表面能量条件写成

160

161
$$\begin{aligned} \mathbf{j} \cdot \mathbf{n} &= J_s, \\ (\mathbf{q} + h_b \mathbf{j}) \cdot \mathbf{n} &= h(T_s - T_a) + h_{\text{carry}} J_s, \\ h_{\text{carry}} &= \begin{cases} h_v(T_{\text{up}}), & \text{计入汽化相关焓差,} \\ h_b(T_s), & \text{关闭该焓差, 仅作算法消融,} \end{cases} \end{aligned}$$

162

163

164

165

166

167

168

169

170

171 $h_v(T) = 2434600 + 1860(T - 28)$.

172

173

174

175 其中 $h = 25$ W/(m² · K)， T_{up} 在蒸发时取表面温度、凝结时取环境温度。式 (6) 的两种设置使用同一表面传质关系，温度和水分联立重算，不把旧通量锁住后再扣热量。关闭分支仍保留内部与表面的有效水焓；它不是另一种真实水的物理过程。

176

177 **反馈链条：排水带走汽化相关能量 → 表面降温 → 平衡关系和当前扩散系数改变 → 蒸发与内部迁移重新分配。
** 这里改变的是同一物性公式在新状态下的数值，不是把题给扩散系数乘一个调参因子。

178

179 ## 2.3 收缩与长度

180

181 外半径 $R(t)$ 由附件2逐段线性插值, 限于0–72 h内。干质量 M_d 由初始几何、
初始含水率及附录4密度得到, 约87.566 g, 并保持不变。总长度由

182

183
$$L(t) = \frac{1}{\pi R(t)^2} \int \frac{1+C}{760+90C} \mathrm{d}m_d \tag{7}$$

184

185
$$L(t) = \frac{1}{\pi R(t)^2} \int \frac{1+C}{760+90C} \mathrm{d}m_d \tag{7}$$

186

187 确定。式 (7) 只确定总体积与长度, 内部变形还需要运动学。沿用已经批准的 Map A: 内部圆柱壳保持圆柱面, 轴向材料允许重新分布, 完整保留非正交映射的交叉项。该运动学不是通过材料力学测量确定的。

188

189 为使算法可核查, 记参考坐标为 (a, ζ) , $0 \leq a \leq R_0$, $0 \leq \zeta \leq L_0/2$, 体积雅可比为
 $J = \rho_{d0}(1+C)/\rho(C)$, 则映射为

190

191
$$\begin{aligned} \overline{J}(a) &= \frac{2}{L_0} \int_0^{L_0/2} J(a, \zeta) \mathrm{d}\zeta, \\ I(a) &= \int_0^a b \overline{J}(b) \mathrm{d}b, \\ \lambda_z &= \frac{2I(R_0)}{R(t)^2}, \\ r &= \sqrt{\frac{2I(a)}{\lambda_z}}, \\ z &= \frac{\lambda_z}{\overline{J}(a)} \int_0^{\zeta} J(a, \xi) \mathrm{d}\xi. \end{aligned} \tag{8}$$

192

193 式 (8) 使 $(r/a) r_{a z} \zeta = J$, 并同时满足给定外半径和总长度。有限元中先插值雅可比, 非线性密度关系在积分点仍有离散误差, 本报告保留该误差诊断, 不宣称有限网格上处处精确。

194

195

196

197

198

199

200

201

202

203 # 3. 哪些是数据, 哪些仍是假设

204

205 | 项目 | 本分支采用的内容 | 依据及限制 |

206 |---|---|---|

207 |题给输入|初始尺寸、28 °C、2.55 kg/kg; 附件1、附件2; 附录4物性; h 、 h_m |不跨问替换,
不缩放题给系数|

208 |初始和空间近似|初始温湿均匀、轴对称、中面对称、各向同性、端侧面同一环境|沿用的条件假设;
未做局部材料测量|

209 |质量与几何|干质量守恒; 题给密度作为当前湿密度; 自由总长度及 Map A|已批准的条件运动学;
不等于力学本构已确定|

210 |表面关系|式 (4) 的一参数曲线; 另保留 Oswin 指数0.75的形状对照|形式候选有文献支持;
本药材参数未标定|

211 | 参数参考值 |0.03、0.075、0.12| 事先获准的敏感性算例, 不是数据估计的区间 |

212 | 环境延拓 |4 h 后取附件 1 末小时均值 | 外推假设, 不是 4 h 后的实测; 积分在拼接点断开 |

213 |气相热物理|常压101325 Pa、理想水汽近似、原分支定比热饱和压关系|沿用常数和近似;
未新增本材料热物性测量|

214 |能量省略项|辐射、机械储能、额外脱附参数均不单独计入|沿用批准范围;
不能据此说这些项已被证明处处很小|

215 |内部蒸发|有效水分迁移加表面能量交换，不分开求液体、蒸气及移动干湿界面|等效近似；
未确定真实蒸发位置|

216 |容量辅助检查|水密度参考1000 kg/m³，
检查剩余可容纳固体的体积是否为正|仅用于条件诊断和继承初始化检查，未识别固体真密度或孔隙率|

217

218 **本轮新增的只有数值网格加密，没有新增材料参数、表面假设或变形方式。** 历史“无相变焓”与
“含相变焓”采用相同的这些条件。

219

220 # 4. 实验设计与数值检验

221

222 采用材料坐标上的双线性四边形有限元与集总干质量矩阵；径向和轴向网格分别向表面按1.5、2.0幂加密。
空间体积分使用3阶高斯点。对含水率相关的指数扩散项采用 Kirchhoff 原函数构造通量，
在非线性求解中按当前温湿状态更新系数和几何。

223

224 时间积分使用 SciPy BDF，温度通过 \$H,C\$ 换算。相对容差 2×10^{-7} ，焓绝对容差 10^{-3} J/kg干物质，含水率绝对容差 10^{-9} ，最大时间步300 s；在环境和半径数据节点处强制分段。
每个接受步都记录状态范围、几何和边界检查，另外用8点时间高斯积分累计表面通量，并与4点积分比较。
失败不通过截断含水率、活动度或替换系数掩盖。

225

226 达标条件为离散全域 $\max C < 0.15$ 。双线性有限元的单元内部值为节点值的凸组合，
因此对本离散场检查全部节点覆盖全单元最大值。用时间插值定位临界时刻，再验证其后第一个整秒的全场；
不是仅凭中心或表面单点停止。

227

228 实验按以下次序组织：

229

230 1. 复核原有三组参数、两种焓差设置的 12×6 和 24×12 结果。

231 2. 复用参数 0.075 已经完成的 32×16 成对结果。

232 3. 本轮新增参数 0.03、0.12 各自有无焓差的 32×16 计算，共 4 次，均积分至 72 h。

233 4. 从保存的场重新计算质量加权含水率、全域最大值、总体积及质量/能量账，并自动比较配对配置，
确认每对只差焓差开关。

234 5. 汇入已有的表面曲线形状对照和不含平衡关系的失败实验，分别保留，不混成一组时长。

235

236 ## 4.1 网格加密是否改变结论

237

238 @@GRID_TABLE@@

239

240 最后一次加密的最大时长变化为 **@@MAX_GRID_MIN@@ min**。该数是这几组网格之间的数值差，
不是对真实药材预测误差的承诺。参数造成的时长差异和每行潜热配对差均另行报告；
不能拿一个中心参数的网格检查替代其他参数的核验。

241

242 已核验的同一代码时间收紧试验，在参考参数0.075、12×6网格下将相对/绝对容差收紧10倍，最大时间步从300
s减为150 s，含潜热时长变化 **@@TIME_CHANGE@@ s**。这是该参考算例的时间精度检查，不宣称三个参数、
两个开关都做过独立时间加密。

243

244 6个32×16核心算例中，最大相对水量残差为 **@@@MASS_RES@@@**，最大相对焓收支残差为 **@@@ENERGY_RES@@@**。
 水量按初始含水量与累计通量变化量较大者归一化；
 能量按累计各边界能量通道总变化量与初始储热系数较大者归一化。
 因此这里不能把小残差读成相对于净储热变化的同样小误差。

245

246 同一批细网格算例中，积分点的非线性密度关系最大相对插值误差为 **@@@DENSITY_INTERP@@@**。这是离散误差，
 不能与全局质量残差混为一谈；节点状态与总体积关系另按式（7）核验。

247

248 **## 4.2 温度反馈及物性响应**

249

250 参考平衡含水率 0.075、32×16 网格在 3 h 时的状态如下。两行的扩散公式完全相同。

251

252 **@@TEMP_TABLE@@**

253

254 **![参考参数0.075下的温度与含水率反馈。上图展示前12 h，环境实测与4 h后的延拓分别标识；下图展示72 h内的全域最大含水率。](figures/final/thermal-feedback.png){#fig-feedback width=160mm}**

255

256 温度响应比含水率趋于达标更快；汽化耗能使前期温度降低，温度与含水率共同改变当前扩散系数。
 后期两条温度轨迹即使接近，之前形成的水分场差异仍会影响达标时间。
 因此不能用某一时刻的温差直接按比例换算整段烘干时间。

257

258 **## 4.3 必须同时报告的几何与能量结果**

259

260 **@@LENGTH_TABLE@@**

261

262 长度最大值覆盖0–72 h，最后一列明确是72 h而非各自达标时刻。明显伸长是式（7）
 与所选内部运动学的条件推论，不是附件中的观测，更不能解释为已经证明药材存在这种真实力学行为。

263

264 两组计算使用相同的长度计算规则，但潜热改变水分场后会进一步改变长度和扩散路径。这是同一模型中的反馈，
 配对时长差包含这项反馈。本轮没有额外固定某条长度轨迹来分离纯热效应与几何效应。

265

266 **![参考参数0.075下的几何和能量账。上图显示模型反推的长度；下图显示累计对流供热、
 水分携焓流出及材料焓增加。水分携焓包含有效显热和汽化相关贡献，不能把整条曲线称为“纯潜热”。
](figures/final/geometry-energy.png){#fig-geometry width=160mm}**

267

268 在本分支的能量省略范围内，对流供热等于材料焓增加与水分携焓流出的和。该账目通过说明数值方程相互配套；
 不能反过来验证表面参数或内部变形是真实的。

269

270 **# 5. 被否决的简单边界，以及否决的范围**

271

272 另一个已经实算的对照，保持第四问物性、Map A 和同一焓收支，只把表面关系改为

273

274 **\$\$J_s=\rho_d h_m(C_s-w_a).\tag{9}\$\$**

275

276 该对照不调用材料等温线，也没有使用0.075等参考平衡值。它在10 s时仍给出向外通量，
 但表面纯水饱和压已低于环境水汽分压：

277

278 @@FAILURE_TABLE@@

279

280 表中温度、压力、通量对应最不利表面积分点；面积比例是按边界高斯积分权重计算，
并非每一个连续表面点都被观测。两个网格均在全部边界积分权重上出现低于 -100 Pa 的压差，
因而不是单个极小角点的微弱超限。

281

282 在该分支采用的单温度、局部平衡、无额外压力驱动的被动气相交换解释下，材料活动度不大于1，
持续向外蒸发与上述状态不相容。**否决的是式 (9) 与现有潜热、几何和参数的组合，不是所有含潜热模型。**
最早越界时刻随网格变化，未将它当作已收敛的物理事件时间。该实验在越界后继续至10 s仅为了检查数值裕量，
没有继续生成一个“烘干时长”。

283

284 另一项历史形状对照中，Oswin指数0.75在已检验状态范围内完成积分；指数0.5与所选参考点、温度关系及4
h环境拼接的组合发生定义域/活动度问题。失败记录单列，不能用“某个公式有文献”覆盖本次实际失败，
也不能据此否决该公式在其他材料和条件下的全部用途。

285

286 # 6. 怎样在论文中使用这条分支

287

288 可以采用的结论是：在本分支指定的物性、运动学和表面平衡关系下，同一通量的质量—能量配对可以实现；
汽化相关焓差通过温度反馈改变水分迁移与达标时间，这一作用在三组参考参数和所比较网格上均存在；
直接给原经验边界增加汽化耗能并不自动获得物理一致性。

289

290 尚不能采用的结论是：某个参考参数已被题给数据拟合；附件半径证明了内部运动学；长度伸长已被实测支持；
三个时长构成真实范围；此分支已完成四问的全部物理闭合。

291

292 半径在这里是模型强制采用的输入，不同表面参数均能复现它，不能再把半径误差当作表面参数的拟合依据。
若要确定这些参数，应增加独立的材料平衡观测，或联合失重与表面温度观测，并先检查可辨识性；
不能以仿真输出冒充这些测量。

293

294 **本次定位：有完整方程、可追溯实算和反例支持的第四问修正分支。**
前三问的既有潜热算例采用过不同的密度和气相传质口径，本报告不将它们拼接成同一套结果，
也不重新利用附件2去修改第三问几何。

295

296 # 7. 复现与文件说明

297

298 计算入口：`src/run\_experiment.py`。

299

300 核心模型：`src/elongation\_model.py`，依赖于 `\_elongation\_frozen`。依赖文件保留历史说明，
但当前子类覆盖了密度、几何和表面实现；实际作用范围以本报告、配置和执行调用为准。

301

302 配置中的历史字段 `shrink\_beta`、`gas\_transfer\_scale` 不参与本分支输运方程；继承对象中的 Lewis
气相交换入口已由式 (4) 覆盖。本轮没有通过这些字段暗中调整扩散、收缩或传质速度。

303

304 `designs/endpoint-grid-refinement.json` 定义本轮4次新计算；`input` 保存题给附件、
原批准方案绑定和本轮范围记录；`results` 保存新增全场；`evidence/reused` 保存核验后的历史完整算例；
`evidence/failure` 保存简单边界的两组10 s反例。

305

```

306 `analysis/final` 提供全精度配对表、网格表、关键时刻与时间轨迹，以及逐例检查。`runs`
    中的记录绑定实际代码、输入、输出和日志的哈希；这种执行绑定不等于科学结论认证。图表保留代码、SVG、
    PDF、PNG和数据来源。没有独立实验测量，也没有进行新的参数拟合。
307
308 环境：Python数值栈使用 NumPy 2.4.4、SciPy 1.17.1、openpyxl 3.1.5；图表使用 Matplotlib 3.10.6。
    各运行记录另存实际 Python 和平台信息。复现命令和文件哈希检查见包内 `README.md`；应在新结果目录运行，
    避免覆盖原始证据。
309
310 # 参考资料与支持范围
311
312 1. Wang H, Lu T, Zhang Q. *Drying characteristics of biological porous media during convective
    drying*. 2016. [DOI:10.3965/j.ijabe.20160905.2057](https://doi.org/10.3965/j.ijabe.20160905.2057)。
    原文表面传质与潜热配对使用水活动度，且结合材料实验；不提供本题未知药材参数。
    本分支没有移植其中湿区/干区模型。
313 2. Soysal Y, Öztekin S. 药用及芳香植物的平衡水分研究，2001。
    [DOI:10.1006/jaer.2000.0636](https://doi.org/10.1006/jaer.2000.0636)。已核查出版方摘要，
    用于支持候选关系具有材料差异；未据此认定本药材参数。
314 3. Saad 等，2014。[DOI:10.1155/2014/427842](https://doi.org/10.1155/2014/427842)。
    已有多温度材料吸附/脱附试验，用于支持 Oswin 形式对照；其叶片参数没有当作本题圆柱药材的实测参数。
315
316 本报告以数值实验支持条件模型判断，不作为真实药材过程已经验证的声明。
317 '''
318     replacements={'PAIR_TABLE':pair_table,'GRID_TABLE':grid_table,'TEMP_TABLE':temp_table,
319                   'LENGTH_TABLE':length_table,'FAILURE_TABLE':failure_table,
320                   'MAX_GRID_MIN':f"{max(abs(float(r['change_24_to_32_minutes'])) for r in
321                                     grids):.3f}",
322                   'TIME_CHANGE':f"{time_refinement['difference_s']:.3f}",
323                   'MASS_RES':f"{max(float(r['mass_relative_residual']) for r in maincases):.3e}",
324                   'ENERGY_RES':f"{max(float(r['energy_relative_residual']) for r in
325                                     maincases):.3e}"}
326     replacements['DENSITY_INTERP']=f"{100*max(float(r['maximum_density_interpolation_error']) for
327                                     r in maincases):.3f}"
328     for k,v in replacements.items():report=report.replace('@'+k+'@',v)
329     if '@' in report:raise ValueError('Unresolved report value')
330     (ROOT/'蒸发潜热实验分支.md').write_text(report)
331     # Same content, vector figure targets for TeX generation.
332     tex_input=report
333     for name in ['paired-times','thermal-feedback','geometry-energy']:
334         tex_input=tex_input.replace(f'figures/final/{name}.png',f'figures/final/{name}.pdf')
335     (ROOT/'report-print.md').write_text(tex_input)
336     print('Report generated from final audited results.')
337
338 if __name__=='__main__':main()

```

F.47 run_queue.py

类别：蒸发潜热条件实验。原始代码共 57 行。

用途限定：潜热条件试验的历史批次调度入口。

```
1 """Execute independent approved parameter cases; serial grid follow-up per queue."""
2 import argparse
3 import json
4 from pathlib import Path
5 import subprocess
6 import sys
7 from datetime import datetime, timezone
8
9
10 def main():
11     p = argparse.ArgumentParser()
12     p.add_argument('--latent', type=int, choices=(0, 1), required=True)
13     args = p.parse_args()
14     root = Path(__file__).resolve().parents[1]
15     design = root / 'designs/01-approved-refinement.json'
16     cases = [c for c in json.loads(design.read_text())['cases']
17              if c['config']['latent'] == args.latent]
18     code = sorted(str(f.relative_to(root)) for f in (root / 'src').rglob('*.py')
19                  if f.name != 'run_record.py')
20     inputs = ['input/附件 1.xlsx', 'input/附件 2.xlsx',
21              'input/approval-20260912-conditional.json', '试验方案.md',
22              'designs/01-approved-refinement.json', 'input/current-user-scope.json',
23              'input/frozen-source-hashes.json']
24     statuses = []
25     for case in cases:
26         name = case['name']
27         row = {'name': name, 'status': 'running',
28               'started_utc': datetime.now(timezone.utc).isoformat()}
29         statuses.append(row)
30         status_path = root / f'analysis/queue-latent{args.latent}.json'
31         status_path.write_text(json.dumps(statuses, indent=2))
32         print(json.dumps(row), flush=True)
33         cmd = [sys.executable, '-B', 'src/run_record.py', 'record', '--case', '.',
34               '--run-id', name, '--data-kind', 'user-provided', '--timeout', '5400']
35         for f in code:
36             cmd += ['--code', f]
37         for f in inputs:
38             cmd += ['--input', f]
39         for f in ('summary.json', 'fields.npz'):
40             cmd += ['--output', f'results/{name}/{f}']
41         cmd += ['--', sys.executable, '-B', 'src/run_experiment.py', '--root', '.',
42               '--design', str(design.relative_to(root)), '--approval',
43               'input/approval-20260912-conditional.json', '--name', name]
44         run = subprocess.run(cmd, cwd=root)
```

```

45         row['returncode'] = run.returncode
46         row['status'] = 'executed' if run.returncode == 0 else 'failed'
47         if run.returncode == 0:
48             verify = subprocess.run([sys.executable, '-B', 'src/run_record.py', 'verify',
49                                     '--case', '.', '--run-id', name], cwd=root)
50             row['receipt_verification_returncode'] = verify.returncode
51         row['finished_utc'] = datetime.now(timezone.utc).isoformat()
52         status_path.write_text(json.dumps(statuses, indent=2))
53         print(json.dumps(row), flush=True)
54
55
56 if __name__ == '__main__':
57     main()

```

F.48 audit.py

类别：表面边界核查。原始代码共 75 行。

```

1  """Source/units audit and algebraic recheck of saved diagnostics; no new PDE model."""
2  from pathlib import Path
3  import json, math, hashlib
4  import numpy as np
5  import openpyxl
6  import fitz
7
8  ROOT=Path(__file__).resolve().parents[1]
9  cfg=json.loads((ROOT/'input/thermo.json').read_text())
10
11 def psat(T):
12     K=T+273.15
13     for row in cfg['segments']:
14         if row['Tmin_K']<=K<=row['Tmax_K']:
15             return 1e5*10**(row['A']-row['B']/(K+row['C']))
16     raise ValueError('Outside verified NIST coefficient intervals')
17
18 P=cfg['pressure_Pa']; eps=cfg['molar_mass_ratio']
19 def pressure(w, basis):
20     ratio=w if basis=='dry_air' else w/(1-w)
21     return P*ratio/(eps+ratio)
22
23 doc=fitz.open(ROOT/'input/problem.pdf')
24 text='\n'.join(f'PAGE {i+1}\n'+p.get_text() for i,p in enumerate(doc))
25 assert len(doc)==4 and '对流传质' in text and '干基含水率' in text
26 wb=openpyxl.load_workbook(ROOT/'input/environment.xlsx', read_only=True, data_only=True)
27 rows=list(wb.active.values);wb.close()
28 env=np.array(rows[1:],float);assert np.isfinite(env).all()
29 assert np.all(np.diff(env[:,0])>0)
30 old=(ROOT/'input/old-q123-solve.py').read_text()
31 line=[(i+1,s.strip()) for i,s in enumerate(old.splitlines()) if 'R*self.hm*(Ca-C[-1])' in s]

```

```

32 assert len(line)==1
33 rechecks=[]
34 for mesh in ['24x12','48x24']:
35     d=json.loads((R00T/f'input/diagnostic-{mesh}.json').read_text())
36     a=d['direction']['worst']; t=d['direction']['time_s']
37     wa=float(np.interp(t,env[:,0],env[:,2]))
38     assert abs(wa-a['air_w'])<1e-12 and a['J_out_kg_m2_s']>0
39     q={'mesh':mesh,'time_s':t,'surface_C':a['surface_C'],'surface_T_C':a['surface_T_C'],
40       'recorded_J_out_kg_m2_s':a['J_out_kg_m2_s'],'psat_NIST_Pa':psat(a['surface_T_C']),
41       'psat_old_Pa':a['psat_surface_Pa'],
42       'recorded_bad_area_fraction':d['direction']['bad_surface_area_fraction']}
43     for basis in ['dry_air','moist_air']:
44         pa=pressure(wa,basis)
45         q[basis]={ 'pa_Pa':pa, 'psat_minus_pa_Pa':q['psat_NIST_Pa']-pa,
46                   'wrong_direction_at_saved_state':q['psat_NIST_Pa']<pa}
47         assert q[basis]['wrong_direction_at_saved_state']
48     rechecks.append(q)
49
50 # Favorable flux upper bound, not a new full drying simulation:
51 # reuse existing fixed-cylinder scope, take pure-water saturated surface at
52 # observed maximum air temperature, zero ambient vapor and unlimited supply.
53 # All these maximize the tested J=k_g*(rho_vs-rho_va); no material fit enters.
54 # It applies only when surface T stays below this envelope and geometry is fixed.
55 Tmax=float(env[:,1].max());
56 rhovmax=psat(Tmax)/(cfg['water_vapor_gas_constant_J_kgK']*(Tmax+273.15))
57 hm=8e-7; R=.02; L=.25; C0=2.55; Ctarget=.15
58 area=2*math.pi*R*L+2*math.pi*R**2; vol=math.pi*R**2*L
59 md_q23=(650+128*C0)*vol/(1+C0)
60 need=md_q23*(C0-Ctarget); maxJ=hm*rhovmax; maxrate=area*maxJ
61 bound={'scope':'Diagnostic of reinterpreting hm as gas-concentration coefficient. Fixed original
62 cylinder, dry-mass conservation initialized ONLY with Appendix 3, T_surface<=maximum observed
63 T_air, zero ambient vapor, saturated surface and unlimited internal supply. Does not claim
64 fixed-volume Appendix 3 density consistency or actual Q3 drying time.',
65       'max_observed_air_T_C':Tmax,'gas_density_upper_kg_m3':rhovmax,'hm_m_s':hm,'area_m2':area,
66       'max_evaporation_g_day':maxrate*86400*1000,'initial_dry_mass_q23_g':md_q23*1000,
67       'necessary_water_removal_g':need*1000,'conditional_time_lower_bound_days':need/maxrate/86400,
68       'coefficient_needed_at_least_for_3_days_m_s':need/(3*86400*area*rhovmax),
69       'not_a_prediction':True}
70 out={'scope':'No newly simulated PDE; source audit, independent NIST check of existing saved
71 states, favorable-limit algebra.',
72     'source_pdf_sha256':hashlib.sha256((R00T/'input/problem.pdf').read_bytes()).hexdigest(),
73     'environment_headers':rows[0], 'environment_span_s':[float(env[0,0]),float(env[-1,0])],
74     'old_implemented_boundary_line':line,'direction_rechecks':rechecks,'gas_coefficient_bound':bound,
75     'limitations':['Moist-air alternative rechecks the saved old state, not its full
76 re-integrated trajectory.'],

```

```

70         'NIST pressure replacement rechecks the worst saved quadrature state, not every
           boundary point.',
71         'Bounds reject the stated interpretation under stated envelope, not the problem
           or all latent models.']}
72 (ROOT/'results').mkdir(exist_ok=True)
73 (ROOT/'results/audit.json').write_text(json.dumps(out,ensure_ascii=False,indent=2))
74 (ROOT/'results/problem-text.txt').write_text(text)
75 print(json.dumps(out,ensure_ascii=False,indent=2))

```

F.49 old-q123-solve.py

类别：表面边界核查。原始代码共 174 行。

用途限定：历史源码文本，仅供诊断读取，不作为当前主模型入口。

```

1  """Radial, material-coordinate drying model. Inputs are user-provided attachments.
2  Run: python src/solve.py --out results/production --n 320
3  Numerical values are model predictions, not measured product quality.
4  """
5  from pathlib import Path
6  import argparse, json, time, hashlib, platform
7  import numpy as np
8  import scipy
9  from scipy.integrate import solve_ivp
10 from scipy.optimize import brentq
11 from scipy.interpolate import PchipInterpolator
12 from scipy.sparse import lil_matrix
13 from scipy.special import expi
14 import openpyxl
15
16 ROOT = Path(__file__).resolve().parents[1]
17
18 def read_input():
19     # Q1-Q3 inherit only the common environment observations.
20     path = ROOT/'input'/'附件'/'附件 1.xlsx'
21     w = openpyxl.load_workbook(path, data_only=True)
22     a = np.array(list(w.active.values)[1:], float)
23     w.close()
24     assert np.isfinite(a).all() and np.all(np.diff(a[:, 0]) > 0)
25     return a
26
27 class Model:
28     def __init__(self, n=160, question=3, env_mode='tailmean', radius_mode='fixed',
29                 temp_shift=0., humidity_shift=0., hm_scale=1., diffusion_scale=1.):
30         if question not in (1, 2, 3):
31             raise ValueError('Q4 requires the separate Q4 entry point')
32         if radius_mode != 'fixed':
33             raise ValueError('Q1-Q3 reject shrinkage or Attachment 2 radius configuration')
34         self.n, self.question = n, question
35         self.x = 1.-(1.-np.linspace(0., 1., n+1))*1.5

```

```

36     self.faces = np.r_[0., (self.x[1:]+self.x[:-1])/2., 1.]
37     self.vol = np.diff(self.faces**2)/2.
38     self.env_data = read_input()
39     self.tail = self.env_data[self.env_data[:, 0] >= 10800, 1:].mean(axis=0)
40     self.env_mode, self.radius_mode = env_mode, radius_mode
41     self.temp_shift, self.humidity_shift = temp_shift, humidity_shift
42     self.hm = 8e-7*hm_scale
43     self.diffusion_scale = diffusion_scale
44     sp = lil_matrix((2*(n+1), 2*(n+1)), dtype=int)
45     for i in range(n+1):
46         sp[2*i:2*i+2, 2*max(0,i-1):2*min(n+1,i+2)] = 1
47     self.sparsity = sp.tocsc()
48
49     def env(self,t):
50         if t <= self.env_data[-1,0]:
51             return tuple(np.interp(t,self.env_data[:,0], self.env_data[:,j]) for j in (1,2))
52         e = self.tail.copy()
53         if self.env_mode == 'last': e = self.env_data[-1,1:].copy()
54         if self.env_mode == 'nominal': e = np.array([50.,.05])
55         return e[0]+self.temp_shift, e[1]+self.humidity_shift
56
57     def radius(self,t):
58         return np.ones_like(t,dtype=float)*.02
59
60     def props(self,T,C):
61         # Guard only solver trial evaluations; final physical ranges are audited.
62         cc = np.maximum(C,1e-8)
63         if self.question == 1:
64             rho = np.full_like(C,820.); cp=np.full_like(C,2600.)
65             k=np.full_like(C,.36); D=7e-9*np.exp(-.89/cc)
66         elif self.question in (2,3):
67             rho=650+128*cc; cp=1450+2736*cc/(cc+1)
68             k=.21+.38*cc/(cc+1); D=2.4e-3*np.exp(-.45/cc-3850/(T+273.15))
69         else:
70             rho=760+90*cc; cp=1850+2150*cc/(cc+1)
71             k=.12+.20*cc/(cc+1); D=4.2e-4*np.exp(-.30/cc-3850/(T+273.15))
72         return rho*cp, k, D*self.diffusion_scale
73
74     def divergence(self,u,a,R,exchange,external):
75         ah = 2*a[:-1]*a[1:]/(a[:-1]+a[1:])
76         flux=np.zeros(self.n+2)
77         flux[1:-1]=self.faces[1:-1]*ah*np.diff(u)/np.diff(self.x)
78         flux[-1]=R*exchange*(external-u[-1])
79         return np.diff(flux)/(self.vol*R*R)
80
81     def rhs(self,t,y):
82         z=y.reshape(-1,2); T,C=z[:,0],z[:,1]
83         cap,k,D=self.props(T,C); R=float(self.radius(t)); Ta,Ca=self.env(t)

```

```

84     out=np.empty_like(z)
85     out[:,0]=self.divergence(T,k,R,25.,Ta)/cap
86     # Kirchhoff flux integrates the strongly C-dependent diffusivity exactly
87     # across each edge at its mean temperature. This avoids a dry-side
88     # harmonic coefficient introducing an artificial transport bottleneck.
89     cc=np.maximum(C,1e-8)
90     a={1:.89,2:.45,3:.45,4:.30}[self.question]
91     primitive=cc*np.exp(-a/cc)+a*exp(-a/cc)
92     pref={1:7e-9,2:2.4e-3,3:2.4e-3,4:4.2e-4}[self.question]
93     factor=np.full(self.n,pref*self.diffusion_scale)
94     if self.question!=1:
95         factor*=np.exp(-3850/((T[:-1]+T[1:])/2+273.15))
96     flux=np.zeros(self.n+2)
97     flux[1:-1]=self.faces[1:-1]*factor*np.diff(primitive)/np.diff(self.x)
98     flux[-1]=R*self.hm*(Ca-C[-1])
99     out[:,1]=np.diff(flux)/(self.vol*R*R)
100    return out.ravel()
101
102    def solve(self,end,rtol=2e-8,max_step=60.):
103        y0=np.tile([28.,2.55],self.n+1)
104        start=time.time()
105        sol=solve_ivp(self.rhs,(0,end),y0,method='BDF',jac_sparsity=self.sparsity,
106                      rtol=rtol,atol=np.tile([1e-8,1e-10],self.n+1),
107                      dense_output=True,max_step=max_step)
108        assert sol.success, sol.message
109        return sol, {'seconds':time.time()-start,'nfev':sol.nfev,'njev':sol.njev,
110                    'nlu':sol.nlu,'steps':len(sol.t)-1,'rtol':rtol,'max_step':max_step}
111
112    def at(self,sol,times,radii_cm):
113        times=np.atleast_1d(times).astype(float)
114        raw=sol.sol(times).T.reshape(len(times),self.n+1,2)
115        R=self.radius(times)*100
116        out=np.full((len(times),len(radii_cm),2),np.nan)
117        for j,t in enumerate(times):
118            for p,r in enumerate(radii_cm):
119                if r<=R[j]+1e-10:
120                    out[j,p]=[np.interp(r/R[j],self.x,raw[j,:,k]) for k in (0,1)]
121        return out
122
123    def stopping(model,sol):
124        f=lambda t: float(np.max(sol.sol(t).reshape(-1,2)[:,-1])-.15)
125        if f(sol.t[-1]) >= 0: raise RuntimeError('Drying threshold not reached in simulated interval')
126        critical=brentq(f,0.,sol.t[-1],xtol=1e-6)
127        operational=float(np.floor(critical)+1)
128        assert f(operational)<0
129        return critical,operational
130
131    def run(args):

```

```

132 out=R00T/args.out; out.mkdir(parents=True,exist_ok=False)
133 records={}; metadata={'n':args.n,'environment':'linear interpolation through 14400 s; mean of
134                                'radius':'Q1-Q3 fixed 0.02 m; no Attachment 2 read',
135                                'mesh':'x_i=1-(1-i/N)^1.5; boundary-clustered','outputs':{},'python':plat
                                form.python_version(),'numpy':np.__version__,'scipy':scipy.__version__}
136 for q in [1,2,3]:
137     m=Model(args.n,q); end={1:1800,2:10800,3:259200,4:259200}[q]
138     sol,diag=m.solve(end)
139     rec={'diagnostics':diag,'tail_environment':m.tail.tolist()}
140     if q>=3:
141         tc,ts=stopping(m,sol);
142         rec.update(critical_s=tc,stop_s=ts,critical_h=tc/3600,stop_h=ts/3600)
143         times=np.r_[np.arange(60,ts,60),ts]
144         table_times=np.r_[np.arange(21600,ts,21600),ts]
145     else:
146         times=np.arange(1,end+1, dtype=float)
147         table_times=np.array([100,300,600,900,1200,1500,1800] if q==1 else
148                               [1800,3600,5400,7200,9000,10800])
149     radii=np.arange(21)/10
150     values=m.at(sol,times,radii)
151     tables=m.at(sol,table_times,[0,.5,1,1.5,2])
152     surface=sol.sol(times).T.reshape(-1,args.n+1,2)[:,-1,:]
153     table_surface=sol.sol(table_times).T.reshape(-1,args.n+1,2)[:,-1,:]
154     # Save full precision before spreadsheet display rounding.
155     np.savez_compressed(out/f'q{q}.npz',times=times,radii_cm=radii,values=values,
156                        surface=surface,radius_cm=m.radius(times)*100,
157                        table_times=table_times,table_values=tables,table_surface=table_surface,
158                        table_radius_cm=m.radius(table_times)*100)
159     sampled=sol.sol(np.linspace(0,end,721)).T.reshape(-1,args.n+1,2)
160     rec.update(T_range=[float(sampled[:,:,:0].min()),float(sampled[:,:,:0].max())],
161               C_range=[float(sampled[:,:,:1].min()),float(sampled[:,:,:1].max())],
162               max_radial_C_increased=float(np.diff(sampled[:,:,:1],axis=1).max()),
163               output_rows=len(times),table_times_s=table_times.tolist(),
164               table_temperature=tables[:,:,:0].tolist(),table_moisture=tables[:,:,:1].tolist(),
165               table_surface=table_surface.tolist(),table_radius_cm=(m.radius(table_times)*10,
166                               0).tolist())
167     records[f'q{q}']=rec
168     print('DONE',q,json.dumps({k:v for k,v in rec.items() if k in
169                                ['stop_h','critical_h','diagnostics','C_range']},ensure_ascii=False),flush=True)
170 metadata['outputs']=records
171 (out/'summary.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2,allow_nan=True),
172 e))
173 paths=[Path(__file__),*sorted((R00T/'input').rglob('*xlsx')),*sorted(out.glob('*npz')),out/'
174 summary.json']
175 receipt={'created_utc':time.strftime('%Y-%m-%dT%H:%M:%SZ',time.gmtime()),'files':{str(p.relative
176     to(R00T)):hashlib.sha256(p.read_bytes()).hexdigest() for p in paths}}

```

```

170     (out/'receipt.json').write_text(json.dumps(receipt,indent=2,ensure_ascii=False))
171
172 if __name__=='__main__':
173     p=argparse.ArgumentParser();p.add_argument('--n',type=int,default=320);p.add_argument('--out',
        ,default='results/production')
174     run(p.parse_args())

```

F.50 check_bounds.py

类别：解析与复现核查。原始代码共 131 行。

```

1  """Independent algebra and offline necessary-bound check; no PDE is solved."""
2  from __future__ import annotations
3
4  from decimal import Decimal, getcontext
5  from pathlib import Path
6  import hashlib
7  import json
8  import math
9  import platform
10 import sys
11
12 import numpy as np
13
14 ROOT = Path(__file__).resolve().parent
15 BASE = ROOT.parent
16 getcontext().prec = 50
17 D = Decimal
18
19
20 def digest(path: Path) -> str:
21     h = hashlib.sha256()
22     with path.open('rb') as f:
23         for block in iter(lambda: f.read(1024 * 1024), b''):
24             h.update(block)
25     return h.hexdigest()
26
27
28 def main() -> None:
29     R0, L0, C0, Rs, Cs = map(D, ['0.02', '0.25', '2.55', '0.012', '0.15'])
30     a, b = D(760), D(90)
31     rd0 = (a + b * C0) / (1 + C0)
32     # Pi cancels in the length endpoints; these Decimal values avoid pi rounding.
33     lower = L0 * (R0 / Rs)**2 * rd0 / a
34     upper = L0 * (R0 / Rs)**2 * rd0 * (1 + Cs) / (a + b * Cs)
35     Md = math.pi * float(R0)**2 * float(L0) * float(rd0)
36     result = {
37         'scope': 'Analytic necessary conditions plus offline checks of existing A/B saved states;
            no new PDE run.',

```

```

38     'assumptions': [
39         'C is dry-basis moisture and rho=760+90*C is actual wet bulk density.',
40         'The initial uniform state is R0=.02 m, L0=.25 m, C0=2.55.',
41         'Dry mass is conserved and the exterior is a circular cylinder.',
42         'Numerical constant endpoints below are conditional on current R=.012 m.',
43         'The lower bound needs C>=0; the upper bound additionally needs C<=.15 everywhere.',
44     ],
45     'algebra': {
46         'v': '(1+C)/(760+90*C)',
47         'derivative_numerator': str(a - b),
48         'derivative': '670/(760+90*C)^2 > 0 for C>=0',
49         'general_bound': 'Md*v(Cmin)/(pi*R^2) <= L <= Md*v(Cmax)/(pi*R^2)',
50         'dry_density_initial_kg_m3': str(rd0),
51         'dry_mass_initial_kg': Md,
52         'lower_cm_decimal': str(100 * lower),
53         'upper_cm_decimal': str(100 * upper),
54         'minimum_extension_percent': str(100 * (lower / L0 - 1)),
55         'radius_below_which_L_exceeds_L0_cm': str(100 * R0 * (rd0 / a).sqrt()),
56         'strict_criterion': 'For max(C)<.15 and C>=0, L belongs to [lower,upper).',
57         'endpoint_equality': 'Lower: C=0 dry-mass-almost-everywhere; upper: C=.15
dry-mass-almost-everywhere. This does not establish dynamic reachability.',
58         'fixed_25_cm_below_lower': bool(D('0.25') < lower),
59         'proportional_15_cm_below_lower': bool(D('0.15') < lower),
60         'necessary_not_dynamic_sufficiency': True,
61     },
62     'saved_field_checks': {},
63     'runtime': {'python': sys.version, 'numpy': np.__version__, 'platform':
platform.platform()},
64     'script_sha256': digest(Path(__file__)),
65 }
66 paths = {
67     'A': BASE / 'model-system-20260912/variable-length/results/q4-robin-latent0-32x16',
68     'B': BASE / 'kinematics-comparison-20260913/results/map-B-32x16',
69 }
70 for name, folder in paths.items():
71     summary_file, field_file = folder / 'summary.json', folder / 'fields.npz'
72     s = json.loads(summary_file.read_text())
73     with np.load(field_file, allow_pickle=False) as f:
74         w = f['lumped_dry_mass']
75         C = f['states_H_C'][:, :, 1]
76         stopC = f['strict_stop_state_H_C'][:, 1]
77         records = f['records']
78         names = list(f['record_columns'])
79         radii = records[:, names.index('radius_m')]
80         lengths = records[:, names.index('length_m')]
81     assert np.all(w > 0)
82     assert abs(float(w.sum()) - Md) < 1e-14
83     assert np.all(C >= 0)

```

```

84     v = lambda c: (1 + c) / (760 + 90 * c)
85     lo_actual = Md * v(C.min(axis=1)) / (math.pi * radii**2)
86     hi_actual = Md * v(C.max(axis=1)) / (math.pi * radii**2)
87     L_from_nodes = (v(C) @ w) / (math.pi * radii**2)
88     tolerance = 1e-12 # absolute m, only for floating point checks
89     assert np.all(lengths >= lo_actual - tolerance)
90     assert np.all(lengths <= hi_actual + tolerance)
91     post = C.max(axis=1) < .15
92     assert np.any(post)
93     lo_post = Md / (760 * math.pi * radii[post]**2)
94     hi_post = Md * 1.15 / (773.5 * math.pi * radii[post]**2)
95     assert np.all(lengths[post] >= lo_post - tolerance)
96     assert np.all(lengths[post] < hi_post + tolerance)
97     stop_R = s['R_at_strict_stop_m']
98     stop_L = s['L_at_strict_stop_m']
99     assert stop_R == .012
100    assert 0 <= float(stopC.min()) <= float(stopC.max()) < .15
101    assert float(lower) <= stop_L < float(upper)
102    reconstructed_stop_L = float(v(stopC) @ w / (math.pi * stop_R**2))
103    result['saved_field_checks'][name] = {
104        'inputs': [{ 'path': str(p), 'sha256': digest(p)} for p in [summary_file, field_file]],
105        'saved_state_count': len(C),
106        'post_threshold_saved_state_count': int(post.sum()),
107        'dry_mass_sum_kg': float(w.sum()),
108        'saved_C_min': float(C.min()),
109        'all_saved_states_within_bounds_from_own_C_extrema': True,
110        'post_threshold_states_within_radius_specific_0_to_point15_bounds': True,
111        'max_length_minus_lumped_v_integral_abs_m': float(np.max(np.abs(lengths -
112            L_from_nodes))),
113        'strict_stop_s': s['strict_stop_s'],
114        'strict_stop_C_min': float(stopC.min()),
115        'strict_stop_C_max': float(stopC.max()),
116        'strict_stop_R_m': stop_R,
117        'strict_stop_L_cm': 100 * stop_L,
118        'strict_stop_length_from_lumped_v_cm': 100 * reconstructed_stop_L,
119        'distance_above_lower_cm': 100 * (stop_L - float(lower)),
120        'distance_below_upper_cm': 100 * (float(upper) - stop_L),
121        'within_analytic_necessary_interval': True,
122    }
123    result['passed'] = True
124    (ROOT / 'bounds-check.json').write_text(json.dumps(result, ensure_ascii=False, indent=2) +
125        '\n')
126    print(json.dumps({'algebra': result['algebra'], 'checks': {
127        k: {q: v[q] for q in ['strict_stop_L_cm', 'max_length_minus_lumped_v_integral_abs_m',
128            'within_analytic_necessary_interval']}
129        for k, v in result['saved_field_checks'].items()
130    }, 'passed': True}, ensure_ascii=False, indent=2))
131

```

129

```
130 if __name__ == '__main__':
```

```
131     main()
```

F.51 build_reproduction_index.py

类别：解析与复现核查。原始代码共 178 行。

```
1  """Read-only index of existing drying experiments. Never executes a PDE runner."""
2  from pathlib import Path
3  import datetime,hashlib,json,platform,sys
4  import numpy as np, scipy, openpyxl
5  O=Path(__file__).resolve().parent
6  B=O.parents[1]
7  S=B/'model-system-20260912';K=B/'kinematics-comparison-20260913'
8  FILES={}
9  def entry(p):
10     p=Path(p).resolve()
11     if not p.is_file():raise FileNotFoundError(p)
12     rel=str(p.relative_to(B.resolve()))
13     if rel not in FILES:FILES[rel]={ 'path':rel,'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),',
    bytes':p.stat().st_size}
14     return FILES[rel]
15 def load(p):entry(p);return json.loads(Path(p).read_text())
16 def receipt(p,root,result_root=None,original_root=None):
17     r=load(p);checks=[]
18     for role in ['code','inputs','outputs']:
19         for a in r[role]:
20             actual=root/a['path']
21             if result_root is not None and role=='outputs':actual=result_root/Path(a['path']).name
22             via_original=False
23             if not actual.is_file() and original_root is not None:
24                 candidate=original_root/a['path']
25                 if candidate.is_file():actual=candidate;via_original=True
26             if actual.is_file():
27                 f=entry(actual);checks.append({'role':role,'recorded_path':a['path'],'current_path':f['path'],
28                     'matches_recorded_sha256':f['sha256']==a['sha256'],'located_via_original_import_source':via_
29                     _original})
30             else:checks.append({'role':role,'recorded_path':a['path'],'current_path':None,'matches_record_
31                 ed_sha256':None})
32     return {'file':entry(p),'run_id':r['run_id'],'started_at':r['started_at'],'finished_at':r['fini
33         shed_at'],
34         'status':r['status'],'returncode':r['returncode'],'timed_out':r['timed_out'],'historical_argv',
35         :r['argv'],
36         'historical_environment':r['environment'],'relocated_file_checks':checks,
37         'interpretation':'Original execution evidence; this indexing pass did not rerun it. Missing
38         original paths after import remain explicitly unresolved.'}
39
40 env=load(S/'baseline/evidence/environment.json')
```

```

36 settings=load(S/'latent-branch/designs/all-main-cases.json')
37 imports={x['case']:x for x in load(S/'latent-branch/input/reused-evidence-manifest.json')}
38 families={
39     'baseline': {'root': 'model-system-20260912/baseline', 'scope': 'Q1 to Q4, no latent heat or water
enthalpy, 1D/2D empirical baseline',
40     'entrypoints': [entry(S/'run_model.py'), entry(S/'baseline/src/replay.py'), entry(S/'baseline/src/
/run_cases.py'), entry(S/'baseline/src/solve.py')],
41     'core_code': [entry(S/'baseline/src/baseline_engine.py'), entry(S/'baseline/src/frozen_empirical_
_engine.py')],
42     'inputs': [entry(S/'baseline/input/附件1.xlsx'), entry(S/'baseline/input/题千文本.txt'), entry(S/'
baseline/input/附件2.xlsx')],
43     'input_rule': 'Q1 and Q23 do not load radius attachment; Q4 fixed-control also does not load it.',
44     'postprocess': [entry(S/'baseline/src/export_and_audit.py'), entry(S/'baseline/numerical-summary_
.json')],
45     'recorded_software': {'record': entry(S/'baseline/evidence/environment.json'), 'values': env, 'requ_
irements': entry(S/'baseline/requirements.txt')},
46     'replay_examples_not_executed': ['python -B run_model.py --model 2d --question 1 --output-dir
/absolute/new/q1-2d',
47     'python -B run_model.py --model 2d --question 2 --output-dir /absolute/new/q23-2d',
48     'python -B run_model.py --model 2d --question 4 --output-dir /absolute/new/q4-2d',
49     'python -B run_model.py --model 1d --question 3 --output-dir /absolute/new/q23-1d'],
50     'replay_working_directory': str(S.resolve())},
51     'variable_length_A': {'root': 'model-system-20260912/variable-length', 'scope': 'Q4 Map A;
conservative water and water sensible enthalpy; empirical Robin boundary; latent=0',
52     'entrypoints': [entry(S/'run_model.py'), entry(S/'variable-length/src/run_case.py'), entry(S/'var_
iable-length/src/solve_case.py')],
53     'core_code': [entry(S/'variable-length/src/experiment_model.py'), entry(S/'variable-length/src/m_
inimal_model.py'),
54     entry(S/'variable-length/src/core/elongation_model.py'), entry(S/'variable-length/src/core/_el_
ongation_frozen/volume_fem.py'),
55     entry(S/'variable-length/src/core/_elongation_frozen/boundary_model.py'), entry(S/'variable-le_
ngth/src/core/_elongation_frozen/thermo_reference.py')],
56     'inputs': [entry(S/'variable-length/input/附件1.xlsx'), entry(S/'variable-length/input/附件2.xls_
x')],
57     'design': entry(S/'variable-length/designs/scope.json'),
58     'postprocess': [entry(S/'variable-length/src/build_delivery.py'), entry(S/'variable-length/analy_
sis/final-convergence.json'), entry(S/'variable-length/delivery/case-summary.csv')],
59     'requirements': entry(S/'variable-length/requirements.txt'),
60     'sampling_rule': 'Final midplane values are exact Map A inverse-coordinate FE samples in
delivery/, rather than the legacy node-coordinate interpolation in results/ CSV.',
61     'replay_examples_not_executed': ['python -B run_model.py --model variable-length --question 4
--output-dir /absolute/new/q4-map-a'],
62     'replay_working_directory': str(S.resolve())},
63     'variable_length_B': {'root': 'kinematics-comparison-20260913', 'scope': 'Q4 Map B versus frozen Map
A, all nonkinematic conditions matched',
64     'entrypoints': [entry(K/'src/record_compare.py'), entry(K/'src/compare_run.py')],

```

```

65 'core_code':[entry(K/'src/mapping_b.py'),entry(K/'src/experiment_model.py'),entry(K/'src/minim_
al_model.py'),
66 entry(K/'src/core/elongation_model.py'),entry(K/'src/core/_elongation_frozen/volume_fem.py')],
67 'inputs':[entry(K/'input/附件 1.xlsx'),entry(K/'input/附件 2.xlsx')],
68 'design':entry(K/'designs/scope.json'),'frozen_input_code':entry(K/'frozen-input-code.json'),
69 'postprocess':[entry(K/'analysis/analyze_comparison.py'),entry(K/'analysis/comparison.json'),e
ntry(K/'analysis/comparison-data.npz'),entry(K/'analysis/selected-history.csv')],
70 'replay_examples_not_executed':['python -B src/record_compare.py B 32 16 --suffix=-replay'],
71 'replay_working_directory':str(K.resolve()),
72 'portability_limit':'analysis/analyze_comparison.py reads
../model-system-20260912/variable-length/results; the A/B comparison directory alone is not a
complete standalone replay package.',
73 'package_version_limit':'Python/NumPy/SciPy are recorded per summary; an independent historical
openpyxl version is not recorded for B.'},
74 'latent_pairs':{'root':'model-system-20260912/latent-branch','scope':'Six Q4 Map A conditional
cases: ce=0.03/0.075/0.12, latent=0/1, main comparison grid32x16',
75 'entrypoints':[entry(S/'run_model.py'),entry(S/'latent-branch/src/replay_case.py'),entry(S/'la
tent-branch/src/run_experiment.py')],
76 'core_code':[entry(S/'latent-branch/src/elongation_model.py'),entry(S/'latent-branch/src/_elon
gation_frozen/volume_fem.py'),
77 entry(S/'latent-branch/src/_elongation_frozen/boundary_model.py'),entry(S/'latent-branch/src/_
elongation_frozen/thermo_reference.py')],
78 'inputs':[entry(S/'latent-branch/input/附件1.xlsx'),entry(S/'latent-branch/input/附件2.xlsx'),
79 entry(S/'latent-branch/input/approval-20260912-conditional.json'),entry(S/'latent-branch/inpu
t/current-scope.json')],
80 'design':[entry(S/'latent-branch/designs/all-main-cases.json'),entry(S/'latent-branch/designs/
endpoint-grid-refinement.json')],
81 'postprocess':[entry(S/'latent-branch/src/audit_branch.py'),entry(S/'latent-branch/analysis/fi
nal/pairs.csv'),entry(S/'latent-branch/analysis/final/grids.csv')],
82 'requirements':entry(S/'latent-branch/requirements-numerical.txt'),
83 'replay_examples_not_executed':['python -B run_model.py --model latent --question 4
--latent-case q4-latent1-ce075-32x16 --output-dir /absolute/new/q4-latent075-on'],
84 'replay_working_directory':str(S.resolve()),
85 'placement_rule':'ce075 main pair is evidence/reused/<case>; other four are results/<case>.
The current replay design consolidates all six.']}
86
87 cases=[]
88 for dim in ['1d','2d']:
89     for label,questions in [(('q1',[1]),('q23',[2,3]),('q4',[4]))]:
90         case=f'{label}-{dim}-main';root=S/'baseline';d=load(root/'results'/f'{case}.json');cfg=load(ro
ot/'configs'/f'{case}.json')
91         assert cfg['model']==d['config']
92         cases.append({'id':case,'family':'baseline','questions':questions,'role':'main_2d' if dim=='2d'
else '1d_comparison',
93         'config_file':entry(root/'configs'/f'{case}.json'),'effective_config':d['config'],
94         'main_results':[entry(root/'results'/f'{case}.npz'),entry(root/'results'/f'{case}.json')],
95         'critical_h':d['critical_h'],'strict_stop_s':d['strict_stop_s'],

```

```

96     'receipt':receipt(root/'runs'/case/'receipt.json',root))
97 root=S/'baseline'
98 for case in ['q4-2d-fixed-control','q4-2d-grid64','q4-1d-grid64','q1-2d-time64','q23-2d-time64',,
    'q4-2d-time64']:
99     d=load(root/'results'/f'{case}.json')
100     cases.append({'id':case,'family':'baseline','role':'geometry_control' if 'control' in case else
        'numerical_validation',
101         'config_file':entry(root/'configs'/f'{case}.json'),'effective_config':d['config'],
102         'main_results':[entry(root/'results'/f'{case}.npz'),entry(root/'results'/f'{case}.json')],
103         'critical_h':d['critical_h'],'strict_stop_s':d['strict_stop_s'],'receipt':receipt(root/'runs'/
            case/'receipt.json',root))
104 for family,root,ids in [( 'variable_length_A',S/'variable-length',[
105     'q4-robin-latent0-12x6','q4-robin-latent0-12x6-tight','q4-robin-latent0-24x12','q4-robin-latent
        0-32x16'])),
106     ('variable_length_B',K,['map-B-12x6-loose','map-B-12x6','map-B-24x12','map-B-32x16']))]:
107     for case in ids:
108         d=load(root/'results'/case/'summary.json')
109         r={'id':case,'family':family,'role':'main32' if case.endswith('32x16') else
            'numerical_validation',
110             'config_source':'summary.json/config, checked against historical invocation in
                receipt','effective_config':d['config'],
111             'recorded_software':d.get('runtime'),'software_source':entry(root/'results'/case/'summary.js
                on'),
112             'main_results':[entry(root/'results'/case/'fields.npz'),entry(root/'results'/case/'summary.js
                on')],
113             'critical_h':d['critical_h'],'strict_stop_s':d['strict_stop_s'],'receipt':receipt(root/'runs'
                /case/'receipt.json',root)}
114         if family=='variable_length_A':
115             r['authoritative_final_samples']=[entry(root/'delivery'/case/f) for f in
                ['midplane.csv','surface.csv','trajectory.csv','snapshots.csv','audit.json']]
116         cases.append(r)
117 root=S/'latent-branch'
118 for conf in settings['cases']:
119     case=conf['name'];reuse='ce075' in case;rr=root/('evidence/reused' if reuse else
        'results')/case;d=load(rr/'summary.json')
120     assert d['config']==conf['config']
121     rp=rr/'original-receipt.json' if reuse else root/'runs'/case/'receipt.json'
122     cases.append({'id':case,'family':'latent_pairs','role':'pair_main32','reused_into_branch':reuse,
123         'config_file':entry(root/'designs/all-main-cases.json'),'config_selector':case,'effective_conf
            ig':d['config'],
124         'save_step_s':conf['save_step_s'],'end_h':conf['end_h'],'recorded_software':d.get('runtime'),
125         'main_results':[entry(rr/'fields.npz'),entry(rr/'summary.json')],'critical_h':d['critical_h'],
            'strict_stop_s':d['strict_stop_s'],
126         'receipt':receipt(rp,root,rr if reuse else
            None,Path(imports[case]['original_receipt']).parents[2] if reuse else None)})
127
128 validation=load(0/'two-d-validation.json');oldbind=load(0/'two-d-input-bindings.json')

```

```

129 workbooks=[]
130 for q in range(1,5):
131     f=S/'workbooks/2d'/f'result{q}.xlsx';fe=entry(f)
132     assert fe['sha256']==oldbind[str(f.resolve())]['sha256']
133     checks=validation['workbook_rechecks'][q-1]
134     # Fresh row count and header verification; numeric content remains bound to the completed
        all-cell check.
135     w=openpyxl.load_workbook(f,read_only=True,data_only=True)
136     counts={}
137     for sn,chk in checks['sheets'].items():
138         rows=list(w[sn].values);assert len(rows)-1==chk['rows'];assert rows[-1][0]==chk['last_time_s']
139         assert rows[0][-1]==('药材表面' if q==4 else 2)
140         counts[sn]={ 'data_rows_excluding_header':len(rows)-1, 'total_rows_including_header':len(rows), '
            first_time_s':rows[1][0],
141             'last_time_s':rows[-1][0], 'total_columns':len(rows[0]), 'last_column_header':rows[0][-1],
142             'all_value_previous_check_max_difference':chk['max_abs_excel_roundtrip_difference'], 'same_fil
                e_sha_as_all_value_check':True}
143     geom=list(w['几何与判据'].values);counts['几何与判据']={ 'data_rows_excluding_header':len(geom)-1,
        'last_time_s':geom[-1][0]}
144     w.close()
145     workbooks.append({'question':q, 'file':fe, 'sheets':counts, 'source_validation':entry(0/'two-d-val
        idation.json'),
146         'coordinate_scope':'2D axial midplane z=0; global threshold uses all 2D nodes',
147         'surface_rule':'0-1.9cm fixed positions plus current surface column' if q==4 else '0-2cm fixed
            positions at0.1cm steps'})
148
149 bad=[];missing=[]
150 for c in cases:
151     for chk in c['receipt']['relocated_file_checks']:
152         if chk['matches_recorded_sha256'] is False:bad.append({'case':c['id'],**chk})
153         if chk['matches_recorded_sha256'] is None:missing.append({'case':c['id'],**chk})
154     record={'scope':'Existing numerical evidence index for paper Appendix E; this pass reads/hashes
        files, verifies source workbooks, and does not run any PDE.',
155         'created_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(), 'base_path':str(B.resolve()),
156         'current_inspection_environment':{'python':sys.version, 'numpy':np.__version__, 'scipy':scipy.__v
            ersion__,
157         'openpyxl':openpyxl.__version__, 'platform':platform.platform(), 'use':'Indexing and reading
            only; not substituted for historical execution environment.'},
158         'families':families, 'cases':cases, 'workbooks_2d':workbooks, 'files':FILES,
159         'receipt_check_exceptions':{'mismatches':bad, 'unresolved_original_paths':missing},
160         'claims_not_made':['No independent full replay of every model family in this rewriting round.',
161             'Present software versions do not fill absent historical package records.',
162             'File hashes and execution exit codes do not establish material-model correctness.',
163             'No claim that the B comparison folder by itself is an independently portable package.']}
164     (0/'reproduction-index.json').write_text(json.dumps(record,ensure_ascii=False,indent=2))
165

```

```

166 def table(h,rs):return
    '\n'.join(['|'+ '|' '.join(h)+'|','|'+ '|' '.join(['---']*len(h))+'|']+['|'+ '|' '.join(str(x) for x in
    r)+'|' for r in rs])
167 md='# 论文附录E复现索引\n\n本轮只读取和复核已运行结果；没有重新求解PDE。路径相对
    `output/cumcm2026-a-labpro/`。完整SHA和逐案例配置见同名JSON。\\n\\n'
168 md+=## 家族入口与输出\\n\\n'+table(['模型','入口','配置及结果'],[
169 ['二维四问/一维对照','model-system-20260912/run_model.py','baseline/configs/<case>.json;
    baseline/results/<case>.npz和.json'],
170 ['变长度A','model-system-20260912/variable-length/src/run_case.py','results/<case>/summary.json,
    中的config; fields.npz; 最终采样用delivery/'],
171 ['变长度B','kinematics-comparison-20260913/src/record_compare.py','results/map-B-<grid>/summary.
    .json、fields.npz; analysis/comparison-data.npz'],
172 ['潜热配对','model-system-20260912/latent-branch/src/replay_case.py','designs/all-main-cases.js
    on; ce075两条在evidence/reused, 其余在results']]+'\\n\\n'
173 md+=## 主要案例与SHA\\n\\n'+table(['案例','临界/h','严格整数秒','主要场SHA256'],[[c['id'],c['criti
    cal_h'],c['strict_stop_s'],c['main_results'][0]['sha256']] for c in cases if c['role'] in
    ['main_2d','ld_comparison','main32','pair_main32','geometry_control']])+'\\n\\n'
174 md+=## 二维工作簿\\n\\n'+table(['文件','数值数据行（每张温度/水分表）
    ','末时刻/s','末列口径'],[[f"result{w['question']}.xlsx",next(iter(w['sheets'].values()))['data_
    rows_excluding_header'],next(iter(w['sheets'].values()))['last_time_s'],w['surface_rule']] for w
    in workbooks]])+'\\n\\n上述数据行不含表头；每个数值表均22列。Q1/Q2另各有温度和水分两张同长表。
    全部数值及域外空值已和原二维NPZ逐值核对，最大差0，文件SHA在本次重查保持一致。Q4末列是当前药材表面，
    原NPZ中固定2cm位置另保留NaN。\\n\\n'
175 md+=## 软件记录与迁移限制\\n\\n基线evidence/environment.json记录Python3.13.9、NumPy2.4.4、
    SciPy1.17.1、openpyxl3.1.5。A/B及潜热个案summary/runtime记录Python、NumPy、SciPy及平台；
    openpyxl版本在A和潜热requirements中列出，B没有独立的历史openpyxl版本记录。当前索引环境另列在JSON，
    不用于补写旧记录。\\n\\nB的对照分析仍从同级model-system-20260912/variable-length/results加载A，
    不应单独搬走B目录后宣称全流程可复现。潜热ce075的原回执路径保留导入前结构，
    现存结果依文件名与SHA定位。\\n\\n'
176 md+=f## 回执散列复核\\n\\n已索引{len(cases)}个既有案例、{len(FILES)}个明确引用文件。
    存在{len(bad)}项现存文件与原回执SHA不同、{len(missing)}项原回执路径在本分支无法直接定位；
    详情见JSON的receipt_check_exceptions。这两类均如实保留，未通过修写回执消除。\\n'
177 (O/'reproduction-index.md').write_text(md)
178 print(json.dumps({'cases':len(cases),'files':len(FILES),'receipt_mismatches':bad,'missing_origin_
    al_paths':len(missing),'index_sha256':hashlib.sha256((O/'reproduction-index.json').read_bytes()),
    .hexdigest()},ensure_ascii=False))

```

F.52 extract_two_d.py

类别：解析与复现核查。原始代码共 251 行。

```

1 """Extract existing 2D outputs for outline rewrite; never run a PDE solver."""
2 from pathlib import Path
3 import datetime, hashlib, json, platform, sys, time
4 import numpy as np
5 from openpyxl import load_workbook
6
7 OUT=Path(__file__).resolve().parent
8 BASE=OUT.parents[1]/'model-system-20260912'

```

```

9 SRC=BASE/'baseline'
10 inputs={}
11 outputs=[]
12 start=datetime.datetime.now(datetime.timezone.utc).isoformat()
13 def sha(p): return hashlib.sha256(p.read_bytes()).hexdigest()
14 def bind(p):
15     p=p.resolve(); inputs[str(p)]={'sha256':sha(p),'bytes':p.stat().st_size}; return p
16 def clean(x):
17     if isinstance(x,np.ndarray): return clean(x.tolist())
18     if isinstance(x,np.generic): return clean(x.item())
19     if isinstance(x,float) and not np.isfinite(x): return None
20     if isinstance(x,dict): return {str(k):clean(v) for k,v in x.items()}
21     if isinstance(x,(list,tuple)): return [clean(v) for v in x]
22     return x
23 def js(name,obj):
24     p=OUT/name; p.write_text(json.dumps(clean(obj),ensure_ascii=False,indent=2,allow_nan=False));ou
        tputs.append(p);return p
25 def text(name,s):
26     p=OUT/name;p.write_text(s);outputs.append(p);return p
27 def load(case,old=False):
28     root=SRC/('evidence' if old else 'results')
29     with np.load(bind(root/f'{case}.npz'),allow_pickle=False) as z:a={k:z[k] for k in z.files}
30     d=json.loads(bind(root/f'{case}.json').read_text())
31     return a,d
32 def indexes(a,ts):
33     idx=np.searchsorted(a['times_s'],ts)
34     assert np.array_equal(a['times_s'][idx],np.asarray(ts,float)),ts
35     return idx
36 def md_table(head,rows):
37     return '\n'.join(['|'+'|'.join(head)+'|','|'+'|'.join(['---']+['---:']*(len(head)-1))+'|'+|'|'
        +'|'.join(map(str,r))+|' for r in rows])
38 def fmt(x):return '—' if not np.isfinite(x) else f'{x:.4f}'
39
40 for f in ['baseline_engine.py','frozen_empirical_engine.py','solve.py','export_and_audit.py','ch
    eck_identities.py']:
41     bind(SRC/'src'/f)
42 for f in ['附件 1.xlsx','附件 2.xlsx','题干文本.txt']:
43     bind(SRC/'input'/f)
44 summary=json.loads(bind(SRC/'numerical-summary.json').read_text())
45 identity=json.loads(bind(SRC/'evidence/identity-checks.json').read_text())
46 requirements=bind(SRC/'requirements.txt').read_text()
47 A={};D={};array_audits={}
48 for label in ('q1','q23','q4'):
49     case=f'{label}-2d-main';a,d=load(case);A[label]=a;D[label]=d
50     config=json.loads(bind(SRC/'configs'/f'{case}.json').read_text())
51     assert config['model']==d['config']
52     c=d['config'];assert c['dimension']==2 and (c['nr'],c['nz'])==(128,64)
53     assert not d['latent_heat'] and not d['water_enthalpy']

```

```

54 w=2*a['radial_volume_weights'][:,None]*a['axial_volume_weights'][None,:]
55 assert abs(w.sum()-1)<1e-13
56 sf=a['snapshot_fields_TC'];si=indexes(a,a['snapshot_times_s'])
57 assert sf.shape[1:]==(129,65,2)
58 assert np.array_equal(sf[:, :, 0, :], a['middle_field_TC'][si])
59 mw=np.einsum('sij,ij->s',sf[:, :, :, 1],w)
60 assert np.max(np.abs(mw-a['Cmean'][si]))<1e-12
61 assert np.array_equal(sf[:, :, :, 1].max(axis=(1,2)),a['Cmax'][si])
62 assert np.isfinite(sf).all() and sf[:, :, :, 1].min()>=0
63 outside=a['radii_cm'][None,:]/100>a['R_m'][:,None]+1e-12
64 assert np.array_equal(np.isnan(a['moisture_mid']),outside)
65 # Verify exported midplane samples against the actual nonuniform grid for every time.
66 errs=[]
67 for i,t in enumerate(a['times_s']):
68     valid=~outside[i]
69     v=np.interp(a['radii_cm'][valid]/100/a['R_m'][i],a['x'],a['middle_field_TC'][i,:,1])
70     errs.append(np.max(np.abs(v-a['moisture_mid'][i,valid])))
71 assert max(errs)<1e-14
72 if d['strict_stop_s'] is not None:
73     assert a['times_s'][-1]==d['strict_stop_s']==int(np.floor(d['critical_s']))+1
74     assert a['strict_stop_field_TC'][:, :, 1].max()==d['strict_stop_Cmax']<.15
75     assert np.array_equal(a['strict_stop_field_TC'],sf[-1])
76 array_audits[label]={ 'config_matches':True, 'two_dimensional_shape':sf.shape, 'saved_fullfield_timesteps_s':a['snapshot_times_s'],
77     'saved_midplane_rows':len(a['times_s']), 'volume_weights_sum':w.sum(),
78     'snapshot_midplane_exact_match':True, 'snapshot_Cmean_max_recompute_difference':max(abs(mw-a['Cmean'][si])),
79     'snapshot_Cmax_exact_match':True, 'all_template_midplane_reconstruction_max_error':max(errs), 'outside_NaN_mask_correct':True,
80     'strict_stop_fullfield_verified':d['strict_stop_s'] is not None, 'critical_field_Cmax':None if not a['critical_field_TC'].size else a['critical_field_TC'][:, :, 1].max()}
81
82 # The source Excel outputs use 1 s / 60 s rows, excluding t=0; Q4 replaces the last column with
    the current surface.
83 wb_audits=[]
84 for q,label in ((1,'q1'),(2,'q23'),(3,'q23'),(4,'q4')):
85     a=A[label]; d=D[label]
86     wb=load_workbook(bind(BASE/'workbooks/2d'/f'result{q}.xlsx'), read_only=True, data_only=True)
87     rec={'question':q, 'sheets':{}}
88     for sn,key in (('温度','temperature_mid_C'),('水分浓度','moisture_mid')) if q<3 else
        [('Sheet1','moisture_mid')]:
89         rows=list(wb[sn].values); times=np.asarray([r[0] for r in rows[1:]],float);ix=indexes(a,times)
90         expected=a[key][ix].copy()
91         if q==4:
92             assert rows[0][-1]=='药材表面';expected[:,-1]=a['surface_mid_TC'][ix,1]
93         values=np.array([[np.nan if v is None else v for v in r[1:]] for r in rows[1:]],float)
94         assert values.shape==expected.shape and np.array_equal(np.isnan(values),np.isnan(expected))

```

```

95     err=float(np.nanmax(abs(values-expected)));assert err<1e-12
96     rec['sheets'][sn]={ 'rows':len(times),'first_time_s':times[0],'last_time_s':times[-1],
97         'max_abs_excel_roundtrip_difference':err,'mask_matches':True,'last_column':'current_surface'
          if q==4 else 'r_2_cm'}
98     wb.close();wb_audits.append(rec)
99
100 # Tables use saved times and the same midplane physical positions as the task template.
101 tables={}; sections=[]
102 for q,label,ts in
    [(1,'q1',[100,300,600,900,1200,1500,1800]),(2,'q23',list(range(1800,10801,1800))),
103     (3,'q23',list(range(21600,194401,21600))+[D['q23']['strict_stop_s']]),
104     (4,'q4',list(range(21600,172801,21600))+[D['q4']['strict_stop_s']])]:
105     a=A[label]; ix=indexes(a,ts);ri=np.array([0,5,10,15,20]);unit='s' if q==1 else 'h'
106     for v,key in (('temperature','temperature_mid_C'),('moisture','moisture_mid')) if q<3 else
        (('moisture','moisture_mid')):
107         vals=a[key][ix][:,ri]
108         extra={'R_cm':a['R_m'][ix]*100,'L_cm':a['L_m'][ix]*100,'Cmax':a['Cmax'][ix]}
109         tables[f'q{q}_{v}']={'source_case':f'{label}-2d-main','times_s':np.asarray(ts),'radii_cm':a['r',
            adii_cm][ri],
110             'values':vals,'unit':'degree_C' if v=='temperature' else
                'kg_water/kg_dry_solid','surface_mid_TC':a['surface_mid_TC'][ix]**extra}
111         title=f'问题 {q} 二维中面'+('温度 (°C)' if v=='temperature' else '干基含水率 (kg/kg) ')
112         head=[f'时间/{unit}'+[f'r={r:g} cm' for r in a['radii_cm'][ri]]
113             rows=[f'{t:g}' if q==1 else (f'{t/3600:.4f}' if t%21600 and q>=3 else f'{t/3600:g}'))+[fmt(x)
                for x in row] for t,row in zip(ts,vals)]
114         if q==4:
115             head+=['当前表面','R/cm','L/cm']
116             rows=[row+[fmt(a['surface_mid_TC'][ii,1]),fmt(a['R_m'][ii]*100),fmt(a['L_m'][ii]*100)] for
                row,ii in zip(rows,ix)]
117         sec='## '+title+'\n\n'+md_table(head,rows)+'\n\n'
118         if q>=3:sec+=f'末行是首个严格达标整数秒 {ts[-1]} s; 含水率显示四位小数,
            严格判定使用完整双精度节点场。 \n\n'
119         if q==4:sec+=f' “—” 表示固定物理位置已在当前药材外; 当前表面列独立取 r=R(t), 不沿用初始2 cm位置。
            \n\n'
120         sections.append(sec)
121     js('two-d-tables.json',tables);text('two-d-tables.md','#
        二维主模型题定表\n\n数据来自已保存的二维中面, 无新增PDE求解。 \n\n'+'.join(sections))
122
123 # Quantities for prose. Difference statistics below are over the retained temporal samples.
124 metrics={}; endpointintrows=[]
125 for q,label,upper in ((1,'q1',1800),(2,'q23',10800),(3,'q23',np.inf),(4,'q4',np.inf)):
126     a=A[label];d=D[label];mask=a['times_s']<=upper;ts=a['times_s'][mask];mid=a['middle_field_TC'][m,
        ask]
127     dt=mid[:, -1,0]-mid[:,0,0];dc=mid[:,0,1]-mid[:, -1,1]
128     last=np.where(mask)[0][-1]
129     m={'source_case':f'{label}-2d-main','sample_count':len(ts),'sampled_until_s':ts[-1],
130         'center_final_TC':mid[-1,0],'surface_mid_final_TC':mid[-1,-1],

```

```

131 'end_axis_final_TC':a['end_center_TC'][-1], 'Cmax_final':a['Cmax'][-1], 'Cmean_final':a['Cmean'],
    'an'][-1],
132 'R_final_cm':a['R_m'][-1]*100, 'L_final_cm':a['L_m'][-1]*100,
133 'deltaT_surface_minus_center_final_C':dt[-1], 'deltaC_center_minus_surface_final':dc[-1],
134 'sampled_deltaT_peak_C':dt.max(), 'sampled_deltaT_peak_time_s':ts[np.argmax(dt)],
135 'sampled_deltaC_peak':dc.max(), 'sampled_deltaC_peak_time_s':ts[np.argmax(dc)],
136 'dry_mass_ratio_final':a['density_dry_mass_kg'][-1]/a['density_dry_mass_kg'][0]
137 if q>=3:
138     state=a['strict_stop_field_TC'];pos=np.unravel_index(np.argmax(state[:, :, 1]), state[:, :, 1].shape)
139     m.update(critical_s=d['critical_s'], critical_h=d['critical_h'], strict_stop_s=d['strict_stop_s'],
        strict_stop_Cmax=d['strict_stop_Cmax'],
140     strict_stop_max_grid_indices=pos, strict_stop_max_physical_r_z_m=[a['R_m'][-1]*a['x'][pos[0]],
        a['L_m'][-1]/2*a['eta'][pos[1]]])
141     for name, curve in
        (('mid_surface', a['surface_mid_TC'][:, 1]), ('end_axis', a['end_center_TC'][:, 1])):
142         below=np.flatnonzero(curve<.15)
143         if len(below):
144             j=below[0];m[name+'_first_saved_strict_below_threshold_bracket_s']=[a['times_s'][j-1], a['time_s_s'][j]]
145     metrics[f'q{q}']=m
146     for t in ([100, 300, 600, 900, 1200, 1500, 1800] if q==1 else list(range(1800, 10801, 1800)) if q==2 else
        []):
147         i=indexes(a, [t])[0];f=a['middle_field_TC'][i]
148         endpointrows.append({'question':q, 'time_s':t, 'time_h':t/3600, 'center_T_C':f[0, 0], 'surface_T_C':
            f[-1, 0],
149         'center_C':f[0, 1], 'surface_C':f[-1, 1], 'deltaT_surface_minus_center_C':f[-1, 0]-f[0, 0],
150         'deltaC_center_minus_surface':f[0, 1]-f[-1, 1]})
151     alpha=.36/(820*2600);D0=7e-9*np.exp(-.89/2.55)
152     metrics['q1_initial_scales']={'alpha_m2_s':alpha, 'D0_m2_s':D0, 'alpha_over_D0':alpha/D0,
153     'R0_squared_over_alpha_s':.02**2/alpha, 'R0_squared_over_D0_s':.02**2/D0,
154     'scope':'initial radius-squared transport scales, not measured or predicted drying times'}
155     js('two-d-metrics.json', metrics);js('two-d-endpoint-differences.json', endpointrows)
156     text('two-d-endpoint-differences.md', '# 二维中面温湿差\n\nΔT=表面温度-中心温度;
        ΔC=中心含水率-表面含水率。两者单位不同。
        \n\n'+md_table(['问题', 't/h', 'ΔT/°C', 'ΔC/(kg/kg)'], [[r['question'], f"{r['time_h']:.4f}", f"{r['deltaT_surface_minus_center_C']:.4f}", f"{r['deltaC_center_minus_surface']:.4f}"] for r in
        endpointrows]))+'\n')
157
158 # Figure extraction: retain source time stamps; never interpolate to manufacture additional 2D
    snapshots.
159 figurebindings=[]
160 for q, label in ((1, 'q1'), (2, 'q23'), (3, 'q23'), (4, 'q4')):
161     a=A[label];d=D[label];ts=a['times_s']
162     mask=(ts<=1800) if q==1 else (ts<=10800) if q==2 else
        ((np.mod(ts, 60)==0)|(ts==d['strict_stop_s']))
163     smask=(a['snapshot_times_s']<=1800) if q==1 else (a['snapshot_times_s']<=10800) if q==2 else
        np.ones(len(a['snapshot_times_s']), bool)

```

```

164 si=indexes(a,a['snapshot_times_s'][smask]); sf=a['snapshot_fields_TC'][smask]
165 data={k:a[k][mask] for k in
    ['times_s','R_m','L_m','Cmax','Cmean','Tmean','surface_mid_TC','end_center_TC',
166  'middle_field_TC','axialmean_field_TC','temperature_mid_C','moisture_mid','density_dry_mass_re,
    lative_change',
167  'scalar_out_side_per_s','scalar_out_end_per_s']}]
168 data.update(x=a['x'],eta=a['eta'],radii_cm=a['radii_cm'],
169  snapshot_times_s=a['snapshot_times_s'][smask],snapshot_fields_TC=sf,snapshot_R_m=a['R_m'][si],
    snapshot_L_m=a['L_m'][si],
170  radial_volume_weights=a['radial_volume_weights'],axial_volume_weights=a['axial_volume_weights'],
171  center_TC=a['middle_field_TC'][mask,0,:],
172  deltaT_surface_minus_center_C=a['middle_field_TC'][mask,-1,0]-a['middle_field_TC'][mask,0,0],
173  deltaC_center_minus_surface=a['middle_field_TC'][mask,0,1]-a['middle_field_TC'][mask,-1,1])
174 if q>=3:data.update(critical_field_TC=a['critical_field_TC'],strict_stop_field_TC=a['strict_sto,
    p_field_TC'],critical_s=d['critical_s'],strict_stop_s=d['strict_stop_s'])
175 p=OUT/f'q{q}-2d-plot-data.npz';np.savez_compressed(p,**data);outputs.append(p)
176 figurebindings.append({'file':p.name,'sha256':sha(p),'source':str(SRC/'results'/f'{label}-2d-ma,
    in.npz'),
177  'source_sha256':inputs[str((SRC/'results'/f'{label}-2d-main.npz').resolve())]['sha256'],
178  'selection':'saved source times only; Q3 uses minute samples plus strict terminal second; all
    snapshot fields are exact copies',
179  'variable_units':{'times_s':'s','R_m':'m','L_m':'m','temperature':'degree_C','C':'kg_water/kg_,
    dry_solid'},
180  'axis_order':'time,radius_node,axial_half_domain_node,component(T,C); middle_field is z=0
    plane',
181  'fields':{'k':{'shape':v.shape if isinstance(v,np.ndarray) else
    [],'dtype':str(np.asarray(v).dtype)} for k,v in data.items()})
182
183 # Recompute existing 2D mesh and time comparisons; the old Q1/Q3 n64 saved fields are inspected,
    not re-solved.
184 convergence=[]
185 for label,q in [(('q1',1),('q23',3),('q4',4))]:
186  hi,hd=A[label],D[label]
187  lo,ld=load(f'q{q}-n64-explicit_ends',True) if q<4 else load('q4-2d-grid64')
188  tm,td=load(f'{label}-2d-time64')
189  if q<4:
190    # Provenance already contains an old n128 state; verify matching snapshots against current main
    data.
191    old,od=load(f'q{q}-n128-explicit_ends',True)
192    delta=[]
193    for t in old['times_s']:
194      if t in hi['times_s']:
195        i=indexes(hi,[t])[0];j=indexes(old,[t])[0];delta.append(np.max(abs(hi['middle_field_TC'][i]-o,
            ld['middle'][j])))
196    assert max(delta)<1e-12
197  def field(a,t):return a['middle_field_TC' if 'middle_field_TC' in a else
    'middle'][indexes(a,[t])[0]]

```

```

198 keytime=1800 if q==1 else 10800
199 rcommon=np.linspace(0,1,201)
200 g=field(lo,keytime);f=field(hi,keytime)
201 e=np.array([max(abs(np.interp(rcommon,hi['x'],f[:,j])-np.interp(rcommon,lo['x'],g[:,j]))) for j
    in range(2)])
202 timediff=np.max(abs(field(tm,keytime)-field(lo,keytime)),axis=0)
203 row={'question':q,'coarse_grid':[ld['config']['nr'],ld['config']['nz']],
    'fine_grid':[hd['config']['nr'],hd['config']['nz']],
204     'coarse_critical_s':ld['critical_s'],'fine_critical_s':hd['critical_s'],
205     'space_critical_change_s':None if q==1 else hd['critical_s']-ld['critical_s'],
206     'space_common_normalized_r_count':201,'space_midplane_keytime_s':keytime,'space_midplane_max_T_
    _C_change':e,
207     'time_comparison_grid':[td['config']['nr'],td['config']['nz']],
208     'time_control_max_step_s':[ld['config']['max_step'],td['config']['max_step']],
209     'time_control_rtol':[ld['config']['rtol'],td['config']['rtol']],
210     'time_critical_change_s':None if q==1 else td['critical_s']-ld['critical_s'],
211     'time_keytime_s':keytime,'time_midplane_max_T_C_change':timediff}
212 convergence.append(row)
213 fixed,fd=load('q4-2d-fixed-control');p=OUT/'q4-2d-geometry-control.npz'
214 np.savez_compressed(p,times_s=fixed['times_s'],Cmax=fixed['Cmax'],Cmean=fixed['Cmean'],middle_fi_
    eld_TC=fixed['middle_field_TC'],x=fixed['x'],R_m=fixed['R_m'],L_m=fixed['L_m']);outputs.append(p)
215 figurebindings.append({'file':p.name,'sha256':sha(p),'source':str(SRC/'results/q4-2d-fixed-contr_
    ol.npz'),
216     'source_sha256':inputs[str((SRC/'results/q4-2d-fixed-control.npz').resolve())['sha256'],'selec_
    tion':'all saved source rows; no temporal interpolation'})
217
218 # Recompute dimensions with the exact common recorded physical times and template radii.
219 dimensions=[]
220 for label in ('q1','q23','q4'):
221     one,od=load(f'{label}-1d-main');two,td=A[label],D[label]
222     common=np.intersect1d(one['times_s'],two['times_s']);i=indexes(one,common);j=indexes(two,common)
223     finite=np.isfinite(one['moisture_mid'][i])&np.isfinite(two['moisture_mid'][j]);row={'problem':l_
    abel,'common_time_count':len(common),'common_radius_count':21,'sampled_until_s':common[-1]}
224     for v,ks in [(('T',('temperature_mid_C','temperature_axialmean_C')),('C',('moisture_mid','moistur_
    e_axialmean'))):
225         for family,key in zip(['midplane','axialmean'],ks):
226             err=np.abs(one[key][i]-two[key][j]);flat=np.where(finite,err,-np.inf);it,ir=np.unravel_index(
                np.argmax(flat),flat.shape)
227             row[f'{family}_{v}_max_abs_difference']=flat[it,ir];row[f'{family}_{v}_max_location_time_s_ra_
                dius_cm']=[common[it],two['radii_cm'][ir]]
228     row['two_d_minus_one_d_critical_time_min']=None if od['critical_s'] is None else
        (td['critical_s']-od['critical_s'])/60
229     dimensions.append(row)
230 js('two-d-validation.json',{'array_rechecks':array_audits,'workbook_rechecks':wb_audits,'space_t_
    ime_recomputed':convergence,
231     'dimensions_recomputed':dimensions,'source_recorded_grid_summary':summary['grid_refinement'],
232     'source_recorded_time_summary':summary['time_refinement'],'existing_identity_checks':identity,

```

```

233 'recorded_main_diagnostics':D,'fixed_geometry_Cmax_at_72h':fixed['Cmax'][-1],
234 'limitations':['No new PDE integration was performed.',
235 'A two-level grid comparison does not estimate a verified asymptotic order or a rigorous error
bound.',
236 'Time study changes maximum step and tolerances together; it cannot separate their individual
effects.',
237 'Only stored full-field snapshots can be rechecked in this extraction; accepted-step field
extrema rely on the existing execution record.',
238 'Main temperature check is an instantaneous discrete equation residual; a physical cumulative
energy balance including latent heat and water enthalpy is absent.',
239 'Reflection symmetry is imposed as zero boundary flux; no separate full-cylinder
symmetry-convergence experiment exists.',
240 'Material accuracy and missing constitutive assumptions are not resolved by the numerical
residual checks.']]
241 js('two-d-figure-bindings.json',figurebindings)
242 js('two-d-input-bindings.json',inputs)
243 # Verify all declared sources stayed unchanged during extraction.
244 assert all(sha(Path(p))==v['sha256'] for p,v in inputs.items())
245 js('two-d-extraction-receipt.json',{'type':'existing_computed_data_extraction','started_utc':sta
rt,
246 'finished_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),'python':sys.version,'p
latform':platform.platform(),
247 'numpy':np.__version__,'script':str(Path(__file__).resolve()),'script_sha256':sha(Path(__file__
)),
248 'source_files_unchanged':True,'new_PDE_runs':0,'status':'passed',
249 'outputs':{p.name:{'sha256':sha(p),'bytes':p.stat().st_size} for p in outputs}})
250 print(json.dumps({'status':'passed','output_files':len(outputs),'q1_final':clean(metrics['q1']),
251 'q2_final':clean(metrics['q2']),'space_time':clean(convergence)},ensure_ascii=False))

```

F.53 audit_latent_evidence.py

类别：解析与复现核查。原始代码共 143 行。

```

1 """Read-only extraction of previously executed latent-branch comparisons; no PDE."""
2 from pathlib import Path
3 import csv, datetime, difflib, hashlib, importlib.util, json
4 from concurrent.futures import ThreadPoolExecutor
5
6 OUT = Path(__file__).resolve().parent
7 BASE = OUT.parents[1]
8 L = BASE / 'model-system-20260912/latent-branch'
9 M = BASE / 'prose-refinement-20260913/paper/manuscript.md'
10 sha = lambda p: hashlib.sha256(p.read_bytes()).hexdigest()
11 read = lambda p: json.loads(p.read_text())
12 def csvread(p):
13     with p.open() as f: return list(csv.DictReader(f))
14 def configdiff(a,b):
15     return {k:[a.get(k,'ABSENT'),b.get(k,'ABSENT')] for k in sorted(a.keys()+b.keys()) if
a.get(k)!=b.get(k)}

```

```

16 def binding(p,field=None):
17     d={'path':str(p),'sha256':sha(p)}
18     if field:d['field']=field
19     return d
20 rows=csvread(L/'analysis/final/cases.csv')
21 summary={r['case']:read(L/r['source']) for r in rows}
22 manifest={x['case']:x for x in read(L/'input/reused-evidence-manifest.json')}
23 cases=[]; tasks=[]; errors=[]
24 for row in rows:
25     name=row['case'];path=L/row['source'];s=summary[name]
26     receipt=path.parent/'original-receipt.json'
27     if not receipt.exists():receipt=L/'runs'/name/'receipt.json'
28     r=read(receipt)
29     if name in manifest:
30         original=Path(manifest[name]['original_receipt'])
31     elif name=='time-refinement':
32         original=BASE/'elongation-latent-trial-20260912/runs'/r['run_id']/'receipt.json'
33     else:original=receipt
34     output_hashes={Path(x['path']).name:x['sha256'] for x in r['outputs']}
35     checked={f:sha(path.parent/f)==output_hashes[f] for f in ['summary.json','fields.npz']}
36     case={'case':name,'summary':binding(path),'receipt':binding(receipt),'original_receipt':str(o,
        riginal),
37         'fields_npz':binding(path.parent/'fields.npz'),'csv_summary_sha_matches':sha(path)==row,
        ['source_sha256'],
38         'receipt_local_output_hashes_match':checked,'critical_h':s['critical_h'],'strict_stop_s,
        ':s['strict_stop_s'],
39         'config':s['config'],'core_code':{Path(x['path']).name:x['sha256'] for x in r['code']}}
40     cases.append(case);tasks.append((name,original))
41     if not all(checked.values()) or not
42         case['csv_summary_sha_matches']:errors.append({'case':name,'kind':'binding_mismatch'})
43 spec=importlib.util.spec_from_file_location('lab_ultra_run_record','./support/skills/lab-ultra-c,
        ompute/scripts/run_record.py')
44 runner=importlib.util.module_from_spec(spec);spec.loader.exec_module(runner)
45 def verify(item):
46     name,path=item
47     return {'case':name,'original_receipt':str(path),'verification':runner.verify(path.parent.par,
        ent.parent,path.parent.name)}
48 with ThreadPoolExecutor(max_workers=4) as pool:verifications=list(pool.map(verify,tasks))
49
50 pair_checks=[];grid_rows=[];small=[]
51 name=lambda a,c,n:f'q4-latent{a}-ce{c:03d}-{n}x{n//2}'
52 for c in [30,75,120]:
53     for n in [12,24,32]:
54         off,on=[summary[name(a,c,n)] for a in [0,1]]
55         d=configdiff(off['config'],on['config']);pair_checks.append({'Ceq_reference':c/1000,'nr':,
        n,'nz':n//2,'differences':d})

```

```

56         if set(d)!={'latent'}:errors.append({'kind':'pair_difference','Ceq_reference':c/1000,'nr':n,
        :n,'differences':d})
57     vals=[]
58     for a in [0,1]:
59         ss=[summary[name(a,c,n)] for n in [12,24,32]]
60         diffs=[configdiff(ss[0]['config'],x['config']) for x in ss[1:]]
61         if any(set(d)!={'nr','nz'} for d in diffs):errors.append({'kind':'grid_difference','Ceq_r,
        eference':c/1000,'latent':a,'differences':diffs})
62         ts=[x['critical_h'] for x in ss]
63         shift=(ts[2]-ts[1])*60;vals.append(shift)
64         grid_rows.append({'Ceq_reference':c/1000,'latent':a,'t12_h':ts[0],'t24_h':ts[1],'t32_h':t,
        s[2],
65                             'change_24_to_32_minutes':shift,'coarse_minus_fineest_minutes':[(t-ts[2])*60
        for t in ts],
66                             'config_differences':diffs,'source_cases':[name(a,c,n) for n in
        [12,24,32]]})
67     small.append({'comparison':'spatial_refinement','Ceq_reference':c/1000,'mesh':'24x12 ->
        32x16',
68                 'changed_settings':'nr,nz','off_change_minutes':vals[0],'on_change_minutes':vals[1],
69                 'on_off_delay_change_seconds':(vals[1]-vals[0])*60,
70                 'source_cases':[name(a,c,n) for a in [0,1] for n in [24,32]],
71                 'limit':'Observed mesh difference; not a true-solution error bound or material
        prediction error.'])
72     base=summary['q4-latent1-ce075-12x6'];tight=summary['time-refinement']
73     time_diff=configdiff(base['config'],tight['config'])
74     small.append({'comparison':'joint_time_accuracy_refinement','Ceq_reference':.075,'mesh':'12x6
        fixed',
75                 'changed_settings':time_diff,'off_change_minutes':None,'on_change_minutes':(tight['critical_h']-base['critical_h'])*60,
76                 'change_seconds':(tight['critical_h']-base['critical_h'])*3600,
77                 'source_cases':['q4-latent1-ce075-12x6','time-refinement'],
78                 'limit':'Only one coarse-mesh latent-on case; tolerances and maximum step changed
        together.'])
79     shape=[]
80     for a in [0,1]:
81         n0=name(a,75,24);n1=f'q4-oswin075-latent{a}-ce075-24x12';s0,s1=summary[n0],summary[n1]
82         d=configdiff(s0['config'],s1['config'])
83         if set(d)!={'oswin_b'}:errors.append({'kind':'shape_config_difference','latent':a,'difference,
        s':d})
84         shape.append({'latent':a,'baseline_h':s0['critical_h'],'shape075_h':s1['critical_h'],
85                     'change_minutes':(s1['critical_h']-s0['critical_h'])*60,'config_differences':d,
        'source_cases':[n0,n1]})
86     small.append({'comparison':'matched_reference_shape_change','Ceq_reference':.075,'mesh':'24x12
        fixed',
87                 'changed_settings':'0swin exponent b=1 (rational activity) -> b=0.75; A recalculated
        to keep the same reference point; reference entropy updated consistently',

```

```

88         'off_change_minutes':shape[0]['change_minutes'],'on_change_minutes':shape[1]['change_
        e_minutes'],
89         'source_cases':sum((x['source_cases'] for x in shape),[]),
90         'limit':'Conditional shape sensitivity on 24x12 only; b=0.75 has no own
        mesh/time-refinement series.')}
91
92 csv_errors=[]
93 for raw,derived in zip(csvread(L/'analysis/final/grids.csv'),grid_rows):
94     for key in ['Ceq_reference','latent','t12_h','t24_h','t32_h','change_24_to_32_minutes']:
95         if abs(float(raw[key])-derived[key])>1e-12:csv_errors.append({'source':'grids.csv','key':key,
        'raw':raw[key],'derived':derived[key]})
96 for r in csvread(L/'analysis/final/pairs.csv'):
97     c=round(float(r['Ceq_reference'])*1000);n=int(r['nr']);off,on=[summary[name(a,c,n)]['critical_
        _h'] for a in [0,1]]
98     for key,v in [(('off_h',off),('on_h',on),('delay_h',on-off),('delay_percent',100*(on/off-1)))]:
99         if abs(float(r[key])-v)>1e-12:csv_errors.append({'source':'pairs.csv','key':key,'raw':r[k
        ey],'derived':v})
100 errors+=csv_errors
101
102 failing=[]
103 for n in [24,48]:
104     p=L/f'evidence/failure/q4-robin-latent1-10s-{n}x{n//2}/summary.json';s=read(p)
105     failing.append({'case':p.parent.name,'source':binding(p),'config':s['config'],'time_s':s['int_
        egrated_until_s'],
106                   'status':s['status'],'drying_time_computed':s['drying_time_computed'],'direct_
        ion':s['direction'],
107                   'vs_current_main_config_differences':configdiff(summary[name(1,75,32)]['confi_
        g'],s['config'])})
108 shape_failures=read(L/'evidence/failure-diagnosis.json')
109 core_hashes={k:sorted({c['core_code'][k] for c in cases if k in c['core_code']})
110              for k in
111                  ['elongation_model.py','volume_fem.py','boundary_model.py','thermo_reference.py']}
112 shape_src=BASE/'rebuild-fixed-scope-20260912/surface-equilibrium-20260912/src'
113 base_src=BASE/'elongation-latent-trial-20260912/src/elongation_model.py'
114 shape_source_review={'baseline':binding(base_src),'shape_base':binding(shape_src/'shape_base.py'),
115                      'shape_model':binding(shape_src/'shape_model_v2.py'),
116                      'base_unified_diff':''.join(difflib.unified_diff(base_src.read_text().splitl_
        ines(True),(shape_src/'shape_base.py').read_text().splitlines(True),n=2)),
117                      'meaning':'The base change replaces rational reference activity with the
        selected reference-activity callable only in the internal entropy check. The
        shape subclass changes reference activity, inverse Ceq, and their consistent
        entropy evaluation while keeping caloric and transport coefficients fixed.'}
118 for f in failing:
119     original=Path(manifest[f['case']]['files'][0]['original_file'])
120     f['execution_verification']=runner.verify(original.parents[2],f['case'])
121     rp=Path(f['source']['path']).parent/'original-receipt.json';r=read(rp)

```

```

121     f['original_receipt_sha256']=sha(rp)
122     f['local_outputs_match_original_receipt']={Path(x['path']).name:sha(rp.parent/Path(x['path'])).name==x['sha256'] for x in r['outputs']}
123 out={'generated_utc':datetime.datetime.now(datetime.timezone.utc).isoformat(),
124       'scope':'Read-only review of archived experiment evidence. No PDE run; original receipt
125       verification invokes only run_record.verify.',
126       'manuscript':binding(M),'source_tables':[binding(L/'analysis/final'/f) for f in
127       ['cases.csv','pairs.csv','grids.csv']],
128       'time_refinement_record':binding(L/'analysis/time-refinement.json'),
129       'failure_diagnosis_record':binding(L/'evidence/failure-diagnosis.json'),
130       'cases':cases,'execution_binding_checks':verifications,'pair_config_checks':pair_checks,
131       'core_source_hash_sets':core_hashes,'shape_source_review':shape_source_review,
132       'grids':grid_rows,'time_config_difference':time_diff,'shape_comparison':shape,
133       'proposed_table_rows':small,'short_robin_failures':failing,'oswin05_failure_diagnosis':shape,
134       '_failures',
135       'errors':errors,'numerical_table_extraction_status':'pass' if not errors else
136       'needs_correction',
137       'unchecked':['No rerun of PDE or event root reconstruction.','No fresh all-state
138       conservation/material/thermodynamic audit.','No new literature validity review or
139       surface-parameter calibration.','No figure generated or final-size visual assessment.']}
140 (OUT/'latent-experiment-evidence.json').write_text(json.dumps(out,ensure_ascii=False,indent=2))
141 with (OUT/'latent-compact-comparison.csv').open('w',newline='') as f:
142     w=csv.DictWriter(f,fieldnames=['comparison','Ceq_reference','mesh','off_change_minutes','on_c,
143     hange_minutes']);w.writeheader()
144     w.writerows({k:r[k] for k in w.fieldnames} for r in small)
145 with (OUT/'latent-grid-comparison.csv').open('w',newline='') as f:
146     keys=['Ceq_reference','latent','t12_h','t24_h','t32_h','change_24_to_32_minutes'];w=csv.DictW,
147     riter(f,fieldnames=keys);w.writeheader()
148     w.writerows({k:r[k] for k in keys} for r in grid_rows)
149 print(json.dumps({'cases':len(cases),'pair_config_checks':len(pair_checks),'grid_rows':len(grid_,
150     rows),'compact_table_rows':len(small),
151     'binding_status':[(v['case'],v['verification'])['execution_binding']] for v in
152     verifications],
153     'shape':shape,'errors':errors},ensure_ascii=False))

```

F.54 audit_new_figures.py

类别：数据图形与示意图。原始代码共 53 行。

```

1 """Mechanical evidence for all eight new figures. Visual review is separate."""
2 from pathlib import Path
3 import base64, hashlib, io, json, subprocess, sys, xml.etree.ElementTree as ET
4 from PIL import Image
5 import fitz
6 P=Path(__file__).resolve().parent
7 IDS=['fig-rewrite-flow','fig-fvm2d','fig-q1-main','fig-q2-main','fig-q3-main','fig-q4-main','fig-
8 geometry-only','fig-mass-diagnostic']
9 S=P/'support/lab-ultra-scientific-figure-maker/scripts'
10 sha=lambda p:hashlib.sha256(Path(p).read_bytes()).hexdigest()

```

```

10 def dump(p,o):p.write_text(json.dumps(o,ensure_ascii=False,indent=2)+'\n')
11 reports={}
12 for fid in IDS:
13     contract=subprocess.run([sys.executable,str(S/'validate_figure_contract.py'),str(P/f'{fid}.re_
14     quest.json'),str(P/f'{fid}.ir.json')],capture_output=True,text=True)
15     (P/'review'/f'{fid}.contract.txt').write_text(contract.stdout+contract.stderr)
16     svg=subprocess.run([sys.executable,str(S/'audit_svg.py'),str(P/f'{fid}.svg')],capture_output=,
17     True,text=True)
18     (P/'review'/f'{fid}.svg-audit.json').write_text(svg.stdout)
19     assert contract.returncode==svg.returncode==0,(fid,contract.stdout,contract.stderr,svg.stderr)
20     sa=json.loads(svg.stdout);sa=sa[0] if isinstance(sa,list) else sa
21     fonts=subprocess.run(['pdffonts',str(P/f'{fid}.pdf')],capture_output=True,text=True)
22     (P/'review'/f'{fid}.pdffonts.txt').write_text(fonts.stdout)
23     assert 'MicrosoftYaHei' in fonts.stdout and 'ArialMT' in fonts.stdout
24     assert 'Type 3' not in fonts.stdout and 'Black' not in fonts.stdout
25     request=json.loads((P/f'{fid}.request.json').read_text())
26     for source in request['data_sources']:assert sha(P/source['path'])==source['sha256']
27     doc=fitz.open(P/f'{fid}.pdf');page=doc[0]
28     assert len(doc)==1
29     assert abs(page.rect.width*25.4/72-160)<.01
30     assert abs(page.rect.height*25.4/72-request['final_height_mm'])<.01
31     escaped=[]
32     for block in page.get_text('dict')['blocks']:
33         for line in block.get('lines',[]):
34             for span in line['spans']:
35                 box=fitz.Rect(span['bbox'])
36                 if box.x0 < -0.1 or box.y0 < -0.1 or box.x1 > page.rect.width+.1 or box.y1 >
37                 page.rect.height+.1:
38                     escaped.append({'text':span['text'],'box':list(box)})
39     rasters=[]
40     for im in page.get_images(full=True):
41         xref,_,w,h,*_im
42         for rect in page.get_image_rects(xref):
43             rasters.append({'pixels':[w,h],'display_rect_pt':list(rect),
44             'dpi_x':w/rect.width*72,'dpi_y':h/rect.height*72})
45     assert all(min(x['dpi_x'],x['dpi_y'])>=599 for x in rasters),(fid,rasters)
46     assert not escaped,(fid,escaped)
47     reports[fid]={'contract_pass':True,'svg_audit':sa,'pdf_sha256':sha(P/f'{fid}.pdf'),
48     'png_sha256':sha(P/f'{fid}.png'),'svg_sha256':sha(P/f'{fid}.svg'),
49     'figure_size_mm':[page.rect.width*25.4/72,page.rect.height*25.4/72],
50     'font_names_verified':['ArialMT','MicrosoftYaHei'],'rasters':rasters,'escaped_pdf_text':e,
51     scaped,
52     'sources_hash_matched':True,'visual_review':'separate from this mechanical audit'}
53 dump(P/'review/all-figure-audits.json',reports)
54 support={str(f.relative_to(P/'support')):sha(f) for f in (P/'support').rglob('*') if f.is_file()
55     and '__pycache__' not in str(f) and f.suffix!='.pyc'}
56 dump(P/'review/support-sha256.json',support)
57 print(json.dumps({k:{'contract':True,'svg':v['svg_audit']['valid'],'rasters':len(v['rasters']),

```

```

53         'minimum_raster_dpi':min([min(x['dpi_x'],x['dpi_y']) for x in
v['rasters']],default=None)} for k,v in reports.items()),indent=2))

```

F.55 build_main_results.py

类别：数据图形与示意图。原始代码共 253 行。

```

1  """Four primary 2D result figures from hash-locked saved outputs only."""
2  from pathlib import Path
3  import hashlib, json, os, sys
4  ROOT=Path(__file__).resolve().parent
5  os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'.mpl-cache'))
6  os.environ.setdefault('XDG_CACHE_HOME',str(ROOT/'.cache'))
7  sys.path[:0]=[str(ROOT/'support/lab-ultra-scientific-figure-maker/scripts'),
8               str(ROOT/'support/lab-ultra-scientific-visualization/scripts')]
9  from figure_library import publication_style
10 from figure_typography import inspect_figure_typography
11 from figure_export import export_figure
12 import numpy as np
13 import matplotlib
14 matplotlib.use('Agg')
15 import matplotlib.pyplot as plt
16 from matplotlib.colors import LinearSegmentedColormap,Normalize
17
18 sha=lambda p:hashlib.sha256(Path(p).read_bytes()).hexdigest()
19 def dump(p,o):Path(p).write_text(json.dumps(o,ensure_ascii=False,indent=2,default=str,allow_nan=
False)+'\n')
20 DATA=ROOT/'source-data'
21 BINDINGS={x['file']:x for x in json.loads((DATA/'two-d-figure-bindings.json').read_text())}
22 BLUE,DARK,GRAY='#315A75','#182F40','#6E7F8B'
23 CMAP=LinearSegmentedColormap.from_list('restrained_blue',['#F6F8FA','#D8E4EC','#AEC8DA','#719BB7',
,'#426F8D','#183E59'])
24 STYLE={'font.size':8.5,'axes.labelsize':8.5,'axes.titlesize':8.5,'xtick.labelsize':8,'ytick.labe
lsize':8,
25        'legend.fontsize':7.8,'axes.spines.top':False,'axes.spines.right':False,
26        'axes.linewidth':.65,'lines.linewidth':1.15,'grid.color':'#DCE2E6','grid.linewidth':.5,
27        'grid.alpha':.7,'svg.fonttype':'none','pdf.fonttype':42,
28        'figure.constrained_layout.h_pad':.055,'figure.constrained_layout.w_pad':.055}
29 COMMON=['Only supplied 2D saved outputs are used; no PDE rerun, data fitting, smoothing or unsaved
field reconstruction.',
30         'All center/surface profiles refer to the z=0 middle plane unless an end-axis series is
explicitly named.',
31         'Heatmaps use nearest control-volume faces constructed as adjacent-node midpoints; no
smooth shading.',
32         'Heatmap/colorbar data layers are rasterized at 600 dpi; axes, labels and curves are
vectors.',
33         'Blue is an ordered field or named curve encoding. Line styles and labels also distinguish
series.']
34 MANIFEST={}

```

```

35 def load(name):
36     p=DATA/name;assert sha(p)==BINDINGS[name]['sha256'],name
37     with np.load(p) as z:d={k:z[k] for k in z.files}
38     assert np.all(np.diff(d['times_s'])>0)
39     return d
40 def faces(a):return np.r_[a[0],(a[1:]+a[:-1])/2,a[-1]]
41 def idx(a,t):
42     ix=np.flatnonzero(a==t);assert len(ix)==1,(t,a);return int(ix[0])
43 def decorate(ax):ax.grid(axis='y');ax.tick_params(length=3,width=.6)
44 def cb(fig,ax,mesh,label,ticks=None):
45     c=fig.colorbar(mesh,ax=ax,pad=.025,fraction=.035,aspect=18,ticks=ticks)
46     c.set_label(label,fontsize=8);c.ax.tick_params(labelsize=7.5,length=2.5,width=.6)
47     return c
48 def finish(fid,fig,claim,panels,transforms,files,receipt,height,annotations,used):
49     req={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'audience':'中文数学建模论文读者',
50         'medium':'A4 paper','final_width_mm':160,'final_height_mm':height,'language':'zh-CN',
51         'data_sources':[{ 'path':str((DATA/f).relative_to(ROOT)), 'sha256':sha(DATA/f)} for f in
52             files],
53         'variables':[{ 'name':n, 'unit':u, 'role':r} for n,u,r in [( 'time','s or h as explicitly
54             labeled','time'),
55             ( 'r','cm','physical radial position'),( 'z','cm','physical axial
56             position'),( 'T','degree_C','temperature'),
57             ( 'C','kg_water/kg_dry_solid','dry-basis moisture')]],
58         'uncertainty':{'kind':'none; deterministic saved numerical outputs, not experimental
59             uncertainty'},
60         'editable_format':'svg','claim_locked':True}
61     ir={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'route':'quantitative_code',
62         'recipe':'Matplotlib paired field and curve panels','panels':[{ 'panel_id':a, 'purpose':b}
63             for a,b in panels],
64         'encodings':[{ 'field':a, 'channel':b, 'scale':'linear', 'source_layer':'deterministic'} for
65             a,b in used],
66         'annotations':annotations,'style_tokens':{'palette':'restrained sequential blue with
67             neutral line styles',
68             'base_font_pt':8.5,'layout_owner':'Matplotlib constrained
69             layout'},'exports':['svg','pdf','png'],
70         'invariants':['data','units','claim','transform','uncertainty','physical domain'],
71         'provenance':{'renderer':'Matplotlib','version':matplotlib.__version__,'materials':receipt,
72             'script_sha256':sha(__file__),'bindings':[BINDINGS[f] for f in files],
73             'transformations':COMMON+transforms}}
74     dump(ROOT/f'{fid}.request.json',req);dump(ROOT/f'{fid}.ir.json',ir)
75     profile={'source_shapes':{},'transformations':COMMON+transforms,'new_pde_runs':0}
76     for f in files:
77         d=load(f)
78         profile['source_shapes'][f]={ 'shape':list(a.shape), 'dtype':str(a.dtype),
79             'missing_count':int(np.isnan(a).sum()), 'finite_min':float(np.nanmin(a)), 'finite_max':,
80             float(np.nanmax(a))} for k,a in d.items()}
81     dump(ROOT/f'{fid}.profile.json',profile)

```

```

73     fig.canvas.draw()
74     dump(ROOT/f'{fid}.typography.json',inspect_figure_typography(fig,final_width_mm=160))
75     export_figure(fig,ROOT/fid,formats=['svg','pdf','png'],dpi=600,bbox_inches=None,mkdir=True,ov
erwrite=True,
76         write_manifest=True,provenance=ir['provenance'],accessibility={'title':claim,'desc':';
'.join(transforms)})
77     MANIFEST[fid]={ 'claim':claim,'width_mm':160,'height_mm':height,'source_files':files,
78         'outputs':{ext:sha(ROOT/f'{fid}.{ext}') for ext in ['pdf','svg','png']}}
79     plt.close(fig)
80
81 def q1():
82     f='q1-2d-plot-data.npz';d=load(f);fid='fig-q1-main'
83     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=114,overrides=STYLE)
as receipt:
84         fig,axs=plt.subplots(2,2,layout='constrained')
85         rr=100*d['R_m'][0]*d['x']
86         for t,color,ls in [(300,'#9BACB8',':'),(900,GRAY,'--'),(1800,BLUE,'-')]:
87             j=idx(d['times_s'],t)
88             for k,ax in enumerate(axs[0]):ax.plot(rr,d['middle_field_TC'][j,:,k],color=color,ls=l
s,label=f'{t} s')
89     axs[0,0].set(title='(a) 中面温度剖面',xlabel='物理半径 / cm',ylabel='温度 /
°C',xlim=(0,2),ylim=(28,39.5))
90     axs[0,1].set(title='(b) 中面含水率剖面',xlabel='物理半径 / cm',ylabel='含水率 /
(kg/kg)',xlim=(0,2),ylim=(.75,2.65))
91     axs[0,0].legend(loc='upper left',frameon=False,ncols=1)
92     axs[0,1].legend(loc='lower left',frameon=False,ncols=1)
93     for ax in axs[0]:decorate(ax);ax.set_xticks([0,.5,1,1.5,2])
94     j=idx(d['snapshot_times_s'],1800);r=100*d['snapshot_R_m'][j]*d['x'];z=50*d['snapshot_L_m']
[j]*d['eta']
95     for k,title,lims,label,ticks in [(0,'(c) 1800 s 半纵截面温度',(33,39),'T /
°C',[33,35,37,39]),
96         (1,'(d) 1800 s 半纵截面含水率',(0,2.55),'C /
(kg/kg)',[0,1,2,2.55])]:
97         ax=axs[1,k];mesh=ax.pcolormesh(faces(z),faces(r),d['snapshot_fields_TC'][j,:,k],
98             norm=Normalize(*lims),cmap=CMAP,shading='flat',rasterized=True)
99         ax.set(title=title,xlabel='距中面轴向距离 / cm',ylabel='物理半径 /
cm',xlim=(0,12.5),ylim=(0,2))
100         ax.set_xticks([0,5,10,12.5]);ax.set_yticks([0,1,2]);cb(fig,ax,mesh,label,ticks)
101     finish(fid,fig,'预热期间的中面剖面显示传热与失水进程;
同一末态的二维场显示端面与侧面的独立交换。',
102         [ ('a','temperature profiles at three saved times'),('b','moisture profiles at the same
times'),
103         ('c','actual 2D temperature at 1800 s'),('d','actual 2D moisture at 1800 s')],
104         ['Profile times are exactly 300, 900 and 1800 s. Both 2D fields are the saved 1800 s
snapshot.',
105         'Physical z ranges from middle plane 0 to end plane 12.5 cm; physical r ranges from axis
0 to side 2 cm.',

```

```

106         'The two spatial axes have independent display scales; the plots do not claim geometric
            aspect preservation.',
107         'Temperature color limits 33..39 degree_C contain the complete 1800 s field; moisture
            limits 0..2.55 cover the initial and final ranges.'],
108         [f],receipt,114,[],[('middle_field_TC','y'),('snapshot_fields_TC','color')])
109
110 def q2():
111     f='q2-2d-plot-data.npz';d=load(f);fid='fig-q2-main';h=d['times_s']/3600;r=100*d['R_m'][0]*d[1,
        x']
112     ti=np.array([idx(d['times_s'],s) for s in np.arange(7)*1800]);dt=d['deltaT_surface_minus_center_C'][ti];dc=d['deltaC_center_minus_surface'][ti]
113     np.savetxt(R00T/'source-data/q2-two-d-difference-points.csv',np.c_[h[ti],dt,dc],delimiter=',',
114         header='time_h,surface_minus_center_T_C,center_minus_surface_C_kgkg',comments='',fmt='%1
        7g')
115     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=110,overrides=STYLE)
        as receipt:
116         fig,axs=plt.subplots(2,2,layout='constrained')
117         for k,title,lims,label,ticks in [(0,'(a) 二维中面温度场',(28,50),'T /
            °C',[28,34,40,46,50]),
118             (1,'(b) 二维中面含水率场',(0,2.55),'C /
            (kg/kg)',[0,1,2,2.55])]:
119             ax=axs[0,k];im=ax.pcolormesh(faces(h),faces(r),d['middle_field_TC'][:, :, k].T,cmap=CMAP,
120                 norm=Normalize(*lims),shading='flat',rasterized=True)
121             ax.set(title=title,xlim=(0,3),ylim=(0,2),ylabel='物理半径 /
            cm');ax.set_yticks([0,1,2]);cb(fig,ax,im,label,ticks)
122         for k,a,title,label in [(0,dt,'(c) 表面—中心温差','温差 / °C'),(1,dc,'(d) 中心—
            表面含水率差','含水率差 / (kg/kg)')]:
123             ax=axs[1,k];ax.plot(h[ti],a,color=BLUE,marker='o',markersize=3)
124             ax.set(title=title,xlim=(0,3),ylim=(0,float(a.max())*1.16),xlabel='时间 /
            h',ylabel=label);decorate(ax)
125             axs[1,0].annotate(f'{dt[-1]:.4f} °C',xy=(3,dt[-1]),xytext=(1.8,1.25),fontSize=8,
126                 arrowprops={'arrowstyle':'-','lw':.6,'color':GRAY})
127             axs[1,1].annotate(f'{dc[-1]:.4f} kg/kg',xy=(3,dc[-1]),xytext=(.65,.4),fontSize=8,
128                 arrowprops={'arrowstyle':'-','lw':.6,'color':GRAY})
129             for ax in axs.flat:ax.set_xticks([0,1,2,3])
130         finish(fid,fig,'问题二中面温度在3 h内接近均匀, 含水率仍保留明显径向差异;
            两种差值由同一二维场计算。',
131             [('a','temperature time-radius field at middle plane'),('b','moisture time-radius field
            at middle plane'),
132             ('c','temperature endpoint difference from the same field'),('d','moisture endpoint
            difference from the same field')],
133             ['Time-radius fields use all 10801 saved one-second rows and 129 radial nodes.',
134             'Difference panels use exactly seven half-hour points. Connecting segments guide the
            eye, with no fit.',
135             f'3 h endpoint differences: temperature {dt[-1]:.17g} degree_C; moisture {dc[-1]:.17g}
            kg/kg.'],

```

```

136         [f],receipt,110,[{'target':'3 h middle-plane endpoints','text':f'{dt[-1]:.4f} degree_C
        and {dc[-1]:.4f} kg/kg'}],
137         [('middle_field_TC','color'),('deltaT_surface_minus_center_C','y'),('deltaC_center_minu
        s_surface','y')])
138
139 def q3():
140     f='q3-2d-plot-data.npz';d=load(f);fid='fig-q3-main';h=d['times_s']/3600;critical=float(d['cri
        tical_s']/3600
141     r15=idx(d['radii_cm'],1.5)
142     series=[('全域最大值',d['Cmax'],DARK,'-'),('中面 r=1.5
        cm',d['moisture_mid'][:,r15],BLUE,'--'),
143             ('侧面中点',d['surface_mid_TC'][:,1],GRAY,'-'),('端面轴心',d['end_center_TC'][:,1],
        #ABB6BE,':')]
144     assert np.array_equal(d['Cmax'],d['center_TC'][:,1]) or
        np.allclose(d['Cmax'],d['center_TC'][:,1],rtol=0,atol=1e-14)
145     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=77,overrides=STYLE)
        as receipt:
146         fig,axs=plt.subplots(1,2,layout='constrained',gridspec_kw={'width_ratios':[1.08,1]})
147         for name,a,color,ls in series:axs[0].plot(h,a,label=name,color=color,ls=ls)
148         axs[0].axhline(.15,color=GRAY,lw=.75,ls='--')
149         axs[0].set(title=(a) 各位置的干燥进程',xlabel='时间 / h',ylabel='含水率 /
        (kg/kg)',xlim=(0,60),ylim=(0,2.62))
150         axs[0].legend(loc='upper
        right',frameon=False,handlelength=2.2);axs[0].set_xticks([0,12,24,36,48,60])
151         zoom=h>=50
152         axs[1].plot(h[zoom],d['Cmax'][zoom],color=DARK,label='全域最大值')
153         axs[1].axhline(.15,color=GRAY,lw=.75,ls='--',label='阈值 0.15')
154         axs[1].plot([h[-1]],d['Cmax'][-1],marker='o',markersize=3,color=BLUE)
155         axs[1].annotate(f'{critical:.4f} h',xy=(critical,.15),xytext=(50.2,.153),fontsize=8,
        arrowprops={'arrowstyle':'-','lw':.6,'color':GRAY})
156         axs[1].set(title='(b) 末期全域判据放大',xlabel='时间 / h',ylabel='全域最大含水率 /
        (kg/kg)',xlim=(50,58.2),ylim=(.149,.1595))
157         axs[1].set_xticks([50,52,54,56,58]);axs[1].set_yticks([.15,.153,.156,.159])
158         axs[1].legend(loc='upper right',frameon=False,handlelength=2)
159         for ax in axs:decorate(ax)
160         finish(fid,fig,'表面达标先于内部; 问题三终止时间由全域最大含水率越过阈值决定。',
161             [('a','distinct physical-location histories and global maximum'),('b','late global
        maximum with exact event annotation')],
162             ['Full histories use saved minute rows plus the strict terminal integer second.',
        'The r=1.5 cm middle-plane curve is an existing physically sampled source column; no
        new spatial interpolation.',
163             'Global maximum is taken from the independently provided whole-field maximum. It
        coincides with the center within 1e-14 in these saved rows.',
164             f'Critical time {float(d["critical_s"]):.17g} s; strict stop {int(d["strict_stop_s"])}
        s, max C {d["Cmax"][-1]:.17g}.',
165             'The second panel is an explicitly labeled enlargement of the global maximum, not a
        second independent experiment.'],

```

```

168         [f],receipt,77, [{'target': 'continuous threshold event', 'text': f'{critical:.4f} h'}],
169         [('Cmax', 'y'), ('moisture_mid', 'y'), ('surface_mid_TC', 'y'), ('end_center_TC', 'y')])
170
171 def q4():
172     f='q4-2d-plot-data.npz';d=load(f);fid='fig-q4-main'
173     h=d['times_s']/3600;stop=float(d['strict_stop_s']/3600
174     with publication_style('nature', language='zh-CN', width_mm=160, height_mm=116, overrides=STYLE)
175     as receipt:
176         fig=plt.figure(layout='constrained');grid=fig.add_gridspec(2,2)
177         tops=[fig.add_subplot(grid[0,k]) for k in range(2)];bottom=fig.add_subplot(grid[1,:])
178         for k,t in enumerate([21600,86400]):
179             j=idx(d['snapshot_times_s'],t);r=100*d['snapshot_R_m'][j]*d['x'];z=50*d['snapshot_L_m',
180             '][j]*d['eta']
181             ax=tops[k];im=ax.pcolormesh(faces(z),faces(r),d['snapshot_fields_TC'][j,:,:],cmap=C,
182             MAP,
183             norm=Normalize(0,2.55),shading='flat',rasterized=True)
184             ax.set(title=f'({{"ab"[k]}) {t/3600:g} h 半纵截面含水率',xlabel='距中面轴向距离 / cm',
185             ylabel='物理半径 / cm',xlim=(0,8.8),ylim=(0,1.45),facecolor='#E4E7E9')
186             ax.set_xticks([0,3,6,8.5]);ax.set_yticks([0,.5,1,1.4]);cb(fig,ax,im,'C /
187             (kg/kg)', [0,1,2,2.55])
188         # Mesh vertices use only the input radius-time geometry; node values are unchanged.
189         hf=faces(h);radius_faces=np.interp(hf,h,100*d['R_m']);xf=faces(d['x'])
190         XX=np.broadcast_to(hf,(len(xf),len(hf)));YY=xf[:,None]*radius_faces[None,:])
191         im=bottom.pcolormesh(XX,YY,d['middle_field_TC'][:, :, 1].T,cmap=CMAP,
192         norm=Normalize(0,2.55),shading='flat',rasterized=True)
193         bottom.plot(h,100*d['R_m'],color=GRAY,lw=.85)
194         bottom.set(title='(c) 当前物理半径内的中面场',xlabel='时间 / h',ylabel='物理半径 / cm',
195         xlim=(0,stop),ylim=(0,2.04),facecolor='#E4E7E9')
196         bottom.set_xticks([0,12,24,36,48]);bottom.set_yticks([0,1,1.2,2])
197         bottom.text(27,1.67,'灰区: 已在药材外部',color='#4C5963',fontsize=8,ha='center')
198         cb(fig,bottom,im,'C / (kg/kg)', [0,1,2,2.55])
199         finish(fid,fig,'第四问的二维收缩场与当前物理尺寸一致变化;被收缩边界排除的空间保持域外标记。',
200         [
201             ('a','actual shrinking-domain field at 6 h'),
202             ('b','actual shrinking-domain field at 24 h'),
203             ('c','middle-plane field with explicit moving physical boundary and exterior')],
204         ['Saved two-dimensional snapshots are exactly 6 and 24 h. Panels (a),(b) use identical
205         physical axes and moisture color limits.',
206         'Gray above/beyond the current boundary indicates exterior space, with no filled-in
207         moisture values.',
208         'Panel (c) displays every saved middle-plane moisture row on its current physical
209         radius. Time-face radius is linearly located between supplied geometry rows; field
210         values are never interpolated.',
211         'All three field panels use the same 0..2.55 kg/kg color scale. Spatial axis display
212         scales are independent and labeled in cm.'],
213         [f],receipt,116,[]),

```

```

203         (('snapshot_fields_TC','color'),('middle_field_TC','color'),('R_m','moving physical
            boundary'))))
204
205
206 def geometry():
207     f='q4-2d-plot-data.npz';g='q4-2d-geometry-control.npz';d=load(f);fixed=load(g);fid='fig-geome
            try-only'
208     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=65,overrides=STYLE)
            as receipt:
209         fig,ax=plt.subplots(layout='constrained')
210         ax.plot(fixed['times_s']/3600,fixed['Cmax'],label='固定尺寸（附录4物性）
            ',color=GRAY,ls='--')
211         ax.plot(d['times_s']/3600,d['Cmax'],label='等比例收缩（附录 4 物性）',color=BLUE)
212         ax.axhline(.15,color='#A1ABB2',lw=.7,ls=':',label='全域阈值 0.15')
213         ax.set(xlabel='时间 / h',ylabel='全域最大含水率 / (kg/kg)',xlim=(0,75),ylim=(0,2.65))
214         ax.set_xticks([0,12,24,36,48,60,72]);decorate(ax);ax.legend(loc='upper
            right',frameon=False)
215         ax.annotate(f'固定尺寸: 72 h 时为
            {fixed["Cmax"][-1]:.4f}',xy=(72,fixed['Cmax'][-1]),xytext=(42,.67),fontsize=8,
            arrowprops={'arrowstyle':'-','lw':.6,'color':GRAY})
216         ax.annotate(f'收缩: {float(d["critical_s"])/3600:.4f} h
            达到临界',xy=(float(d['critical_s'])/3600,.15),xytext=(18,1.03),fontsize=8,
            arrowprops={'arrowstyle':'-','lw':.6,'color':GRAY})
217         finish(fid,fig,'在附录4同物性、同边界和同初态下，等比例收缩算例先达标，固定尺寸算例在72
            h仍未达标。',
218             [('a','same-physics geometry comparison')],
219             ['Two Q4 cases share properties, initial state, boundary exchange and dimensional
            representation; only geometry changes.',
220             f'Fixed-geometry last saved value is {fixed["Cmax"][-1]:.17g} at 72 h; no later drying
            time is inferred.',
221             'The moving-domain curve ends at its recorded strict terminal
            second.'],[f,g],receipt,65,
222             [{'target':'last saved fixed-geometry row','text':f'72 h: {fixed["Cmax"][-1]:.4f}'},
223             {'target':'recorded moving-geometry event','text':f'{float(d["critical_s"])/3600:.4f}
            h'}],(['Cmax','y']))
224
225
226
227
228 def mass_diagnostic():
229     fs=['q3-2d-plot-data.npz','q4-2d-plot-data.npz'];ds=[load(f) for f in
            fs];fid='fig-mass-diagnostic'
230     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=70,overrides=STYLE)
            as receipt:
231         fig,axs=plt.subplots(1,2,layout='constrained',sharex=True,sharey=True)
232         for k,(d,title) in enumerate(zip(ds,['(a) 问题三: 固定尺寸','(b) 问题四: 等比例收缩'])):
233             h=d['times_s']/3600;ratio=1+d['density_dry_mass_relative_change'];ax=axs[k]
234             ax.plot(h,ratio,color=BLUE,ls='-' if k==0 else
                '--');ax.axhline(1,color=GRAY,ls=':',lw=.8)

```

```

235         ax.plot([h[-1]], [ratio[-1]], marker='o', color=BLUE, markersize=3)
236         ax.annotate(f'{ratio[-1]:.4f}', xy=(h[-1], ratio[-1]), xytext=(35, 1.75 if k==0 else
            .82), fontsize=8,
237             arrowprops={'arrowstyle': '-', 'lw': .6, 'color': GRAY})
238         ax.set(title=title, xlabel='时间 /
            h', xlim=(0, 60), ylim=(.4, 2.3)); ax.set_xticks([0, 12, 24, 36, 48, 60]); decorate(ax)
239         ax.set_yticks([.5, 1, 1.5, 2]); ax.text(5, 1.08, '守恒参照值 1', fontsize=7.8, color=GRAY)
240         axs[0].set_ylabel('按湿密度计算的干质量比')
241         finish(fid, fig, '按各问密度解释与所选几何计算的干质量比偏离1;
            各曲线分别检验对应条件组合的兼容性。',
242             [('a', 'Q3 fixed geometry and Q3 density interpretation'), ('b', 'Q4 isotropic geometry and
            Q4 density interpretation')],
243             ['Each ratio is exactly the saved density_dry_mass_relative_change plus one, with no
            refit or new integral approximation.',
244             'The two questions use their own property equations; the figure does not attribute
            their difference to geometry alone.',
245             'Shared axes permit reading deviation from the same dry-mass conservation reference
            value 1.',
246             f'Terminal ratios are {1+ds[0]["density_dry_mass_relative_change"][-1]:.17g} and
            {1+ds[1]["density_dry_mass_relative_change"][-1]:.17g}.'],
247             fs, receipt, 70, [{'target': 'each question terminal saved row', 'text': '2.1584 and 0.5357'}],
248             [('density_dry_mass_relative_change plus one', 'y')])
249
250 if __name__ == '__main__':
251     q1(); q2(); q3(); q4(); geometry(); mass_diagnostic()
252     dump(ROOT/'main-results-manifest.json', {'script_sha256': sha(__file__), 'figures': MANIFEST, 'new_
        _pde_runs': 0})
253     print(json.dumps({k: v['outputs']['pdf'] for k, v in MANIFEST.items()}, ensure_ascii=False))

```

F.56 build_structure_figures.py

类别：数据图形与示意图。原始代码共 233 行。

```

1  """Reproducible explanatory diagrams; no PDE solve and no fabricated field data."""
2  from pathlib import Path
3  import hashlib, importlib.util, json, os, subprocess, sys
4
5  ROOT = Path(__file__).resolve().parent
6  os.environ.setdefault('MPLCONFIGDIR', str(ROOT / '.mpl-cache'))
7  os.environ.setdefault('XDG_CACHE_HOME', str(ROOT / '.cache'))
8  SUPPORT = ROOT / 'support'
9  sys.path[:0] = [str(SUPPORT / 'lab-ultra-scientific-figure-maker/scripts'),
10                 str(SUPPORT / 'lab-ultra-scientific-visualization/scripts')]
11  from figure_library import publication_style
12  from figure_typography import inspect_figure_typography
13  from figure_export import export_figure
14  import numpy as np
15  import matplotlib
16  matplotlib.use('Agg')

```

```

17 import matplotlib.pyplot as plt
18 from matplotlib.patches import Rectangle, Circle, Wedge, FancyArrowPatch
19
20 def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()
21 def dump(p, obj): Path(p).write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str) +
    '\n')
22
23 SOURCES = [ROOT / 'source-data' / f for f in
24             ['approved-outline.md', 'baseline_engine.py', 'frozen_empirical_engine.py',
25              'q23-2d-main.json']]
26 CONFIG = json.loads(SOURCES[-1].read_text()['config'])
27 assert (CONFIG['nr'], CONFIG['nz'], CONFIG['radial_power'], CONFIG['axial_power']) == (128, 64,
    2., 3.)
28 BLUE, TEXT, GRAY, LIGHT = '#315A75', '#26313A', '#74818B', '#E9F0F4'
29 FONT = 'Arial, "Microsoft YaHei", sans-serif'
30
31 def contract(fid, claim, height, route, panels, transforms, source_list=SOURCES):
32     req = {'schema_version':'1.0.0', 'figure_id':fid, 'claim':claim,
33           'audience':'中文数学建模论文读者', 'medium':'A4 paper', 'final_width_mm':160,
34           'final_height_mm':height, 'language':'zh-CN',
35           'data_sources':[{ 'path':str(p.relative_to(ROOT)), 'sha256':sha(p)} for p in
36                           source_list],
37           'variables':[], 'uncertainty':{'kind':'none; explanatory construction, not observed or
38                                         simulated fields'},
39           'editable_format':'svg', 'claim_locked':True}
40     ir = {'schema_version':'1.0.0', 'figure_id':fid, 'claim':claim, 'route':route,
41          'recipe':'FigureSpec dependency graph' if route=='editable_vector' else 'axisymmetric
42                  node-centered control-volume construction',
43          'panels':[{ 'panel_id':k, 'purpose':v} for k,v in panels], 'encodings':[],
44          'annotations':[{ 'target':'source model and approved outline', 'text':s} for s in
45                          transforms],
46          'style_tokens':{'palette':[BLUE,TEXT,GRAY,LIGHT], 'base_font_pt':8.5, 'font_stack':FONT,
47                          'layout_owner':'FigureSpec coordinates' if route=='editable_vector' else
48                          'single Matplotlib axes in mm'},
49          'exports':['svg','pdf','png'], 'invariants':['data','units','claim','transform','model
50              topology','boundary geometry','no invented data'],
51          'provenance':{'renderer':'FigureSpec with librsvg' if route=='editable_vector' else
52                        'Matplotlib',
53                       'script_sha256':sha(__file__), 'source_sha256':{p.name:sha(p) for p in
54                               source_list},
55                       'transformations':transforms}}
56     dump(ROOT/f'{fid}.request.json', req); dump(ROOT/f'{fid}.ir.json', ir)
57     dump(ROOT/f'{fid}.profile.json', {'kind':'explanatory construction', 'observed_rows':None,
58                                       'geometry_and_dependencies_from':[p.name for p in source_list],
59                                       'transformations':transforms})
60
61     return ir
62

```

```

53
54 def workflow():
55     fid='fig-rewrite-flow'
56     claim='二维主模型独立给出四问结果；一维近似、数值检验、质量—
        几何条件研究及潜热配对回答不同检验问题。'
57     ir=contract(fid,claim,110,'editable_vector',
        [('main','二维题定求解与数值检验'),('conditional','质量几何与潜热条件研究')],
58         ['Solid arrows carry inputs/results. Dashed arrows indicate a matched comparison.',
59         'Conditional calculations do not replace the four primary outputs.',
60         'Primary geometry is fixed for Q1–3 and isotropic scaling driven by observed radius for Q4.',
61         'No sequential version history or empirical validation is implied.'])
62     def node(id,label,x,y,w,h,stroke=TEXT,fill='#FFFFFF'):
63         return
        dict(id=id,label=label,x=x,y=y,width=w,height=h,shape='rect',stroke=stroke,fill=fill)
64     spec={'title':claim,'description': '上部给出二维主模型、同条件一维近似及数值检验，下部区分质量
        一几何与潜热的条件研究。',
65         'canvas':{'width':800,'height':550,'width_mm':160},
66         'style':{'theme':'paper','font_family':FONT,'font_size':15,'bg_color':'#FFFFFF','palett
        e':[BLUE]},
67         'nodes':[
68             node('input','初始状态、分问物性与环境时序\n 第四问另用半径观测',400,42,720,60),
69             node('main','二维轴对称热湿模型\n侧面与端面分别交换\n前三问固定尺寸；
            第四问等比例收缩',258,152,438,82,BLUE),
70             node('oned','一维面积折算近似\n 同初态、物性与几何',658,152,220,65),
71             node('results','四问题定结果\n问题一、二：中面温湿分布\n问题三、四：
            全域达标时间',258,271,438,82,BLUE),
72             node('numerical','数值检验\n 网格、时间与离散收支',658,271,220,65),
73             node('diagnosis','质量—几何与能量一致性检验\n干质量历史、
            解析长度界限与相变能量收支',400,382,720,64),
74             node('kinematics','变长度条件研究\n 守恒输运与两种局部运动学',220,479,355,60),
75             node('latent','潜热条件研究\n 未标定表面关系内的组内配对',600,479,355,60),
76         ],
77         'edges':[
78             {'from':'input','to':'main','color':BLUE},
79             {'from':'input','to':'oned','color':GRAY},
80             {'from':'oned','to':'main','style':'dashed','color':GRAY,'label':'近似对照'},
81             {'from':'main','to':'results','color':BLUE},
82             {'from':'results','to':'numerical','color':GRAY,'label':'检验'},
83             {'from':'results','to':'diagnosis','color':BLUE},
84             {'from':'diagnosis','to':'kinematics','color':GRAY},
85             {'from':'diagnosis','to':'latent','color':GRAY},
86         ],
87         'labels':[{'text':'条件研究按各自假设解释，结论不回填二维题定结果','x':400,'y':540,'fon
            t_size':14,'anchor':'middle','color':TEXT}]]
88     dump(ROOT/f'{fid}.spec.json',spec)
89     renderer=SUPPORT/'figure_renderer.py'

```

```

90     out=subprocess.run([sys.executable,str(renderer),'validate',str(ROOT/f'{fid}.spec.json')],capture_output=True,text=True)
91     (ROOT/'review'/f'{fid}.spec-check.txt').write_text(out.stdout+out.stderr)
92     if out.returncode: raise RuntimeError(out.stdout+out.stderr)
93     subprocess.run([sys.executable,str(renderer),'render',str(ROOT/f'{fid}.spec.json'),'--output',str(ROOT/'review'/f'{fid}.figurespec.svg'),'--force'],check=True)
94     # Keep the validated FigureSpec as layout source. Matplotlib preserves verified
95     # regular Chinese and Latin font names in the final PDF, unlike Cairo aliases.
96     mspec=importlib.util.spec_from_file_location('figure_renderer',renderer)
97     mod=importlib.util.module_from_spec(mspec);mspec.loader.exec_module(mod)
98     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=110,
99         overrides={'font.size':8.5,'svg.fonttype':'none','pdf.fonttype':42,'figure.constrained_layout.use':False}) as materials:
100         fig=plt.figure();ax=fig.add_axes([0,0,1,1]);ax.set(xlim=(0,800),ylim=(550,0));ax.axis('off')
101         byid={n['id']:n for n in spec['nodes']}
102         for e in spec['edges']:
103             a,b=byid[e['from']],byid[e['to']]
104             p=mod.clip_to_shape(a['x'],a['y'],b['x'],b['y'],a['width'],a['height'],a['shape'])
105             q=mod.clip_to_shape(b['x'],b['y'],a['x'],a['y'],b['width'],b['height'],b['shape'])
106             ax.add_patch(FancyArrowPatch(p,q,arrowstyle='->',mutation_scale=9,lw=.8,color=e['color'],
107                                     linestyle='--' if e.get('style')=='dashed' else '-'))
108             if e.get('label'):
109                 ax.text((p[0]+q[0])/2,(p[1]+q[1])/2-11,e['label'],ha='center',va='bottom',fontsize=7.9,color=TEXT)
110         for n in spec['nodes']:
111             ax.add_patch(Rectangle((n['x']-n['width']/2,n['y']-n['height']/2),n['width'],n['height'],
112                                     facecolor=n['fill'],edgecolor=n['stroke'],lw=.8))
113             ax.text(n['x'],n['y'],n['label'],ha='center',va='center',fontsize=8.5,color=TEXT,linespacing=1.2)
114         for label in spec['labels']:
115             ax.text(label['x'],label['y'],label['text'],ha='center',va='center',fontsize=7.9,color=TEXT)
116         fig.canvas.draw()
117         ir['provenance']['materials']=materials
118         ir['provenance']['renderer']='Matplotlib export of validated FigureSpec nodes, labels and shape-clipped edges'
119         dump(ROOT/f'{fid}.typography.json',inspect_figure_typography(fig,final_width_mm=160))
120         export_figure(fig,ROOT/fid,formats=['svg','pdf','png'],dpi=450,bbox_inches=None,mkdir=True,overwrite=True,
121             write_manifest=True,provenance=ir['provenance'],accessibility={'title':claim,'desc':spec['description']})
122         plt.close(fig)
123         ir['provenance']['renderer_sha256']=sha(renderer)
124         dump(ROOT/f'{fid}.ir.json',ir)

```

```

125
126
127 def finite_volume():
128     fid='fig-fvm2d'
129     claim='节点中心有限体积在半纵截面上共享通量，旋转几何使径向侧壁面积与轴向环面面积具有不同权重。
130     transforms=[
131         'Displayed grid uses 6 radial and 5 axial intervals only to explain the construction;
132         exponents remain 2 and 3.',
133         'Coordinates are schematically scaled; the diagram is explicitly not to physical aspect
134         ratio.',
135         'Unknowns occupy all nodes including boundaries. Interior faces are adjacent-node
136         midpoints; outer faces are clipped to domain edges.',
137         'Boundary control volumes are truncated at the boundary. At  $r=0$ , the radial face has zero
138         physical area.',
139         'Main calculation uses 128 x 64 intervals, hence 129 x 65 nodes. No PDE state is
140         displayed.',
141         'Interior faces are shared once; opposite signs in neighboring control-volume balances
142         cancel their contribution.',
143         'Full-azimuth physical areas are  $Ar=2\pi r_{\text{face}}dz$ ,  $Az=\pi(r_{\text{plus}}^2-r_{\text{minus}}^2)$ ,  $V=Azdz$ .'
144     ]
145     ir=contract(fid,claim,112,'quantitative_code',[(('a','半纵截面与边界截断控制体'),('b','共享面
146     及旋转后的面积权重')],transforms)
147     nr,nz=6,5
148     r=1-(1-np.linspace(0,1,nr+1))**2
149     z=1-(1-np.linspace(0,1,nz+1))**3
150     rf=np.r_[0,(r[1:]+r[:-1])/2,1]; zf=np.r_[0,(z[1:]+z[:-1])/2,1]
151     np.savez(ROOT/'source-data/fvm-construction.npz',r=r,z=z,rf=rf,zf=zf)
152     # Independent geometric identities for every illustrative ring cell.
153     dr=np.diff(rf); dz=np.diff(zf)
154     az=np.pi*np.diff(rf**2)
155     vol=az[:,None]*dz[None,:]
156     assert np.isclose(vol.sum(),np.pi)
157     assert np.isclose(rf[1]-rf[0],(rf[1]-rf[0])/2)
158     assert np.isclose(r[-1]-rf[-2],(r[-1]-rf[-2])/2)
159     dump(ROOT/'review/fvm-geometric-check.json',{'total_half_domain_volume_for_R1_H1':float(vol.s_
160     um()),'exact':float(np.pi),
161         'boundary_faces':[float(rf[0]),float(rf[-1]),float(zf[0]),float(zf[-1])],
162         'main_intervals':[128,64], 'main_nodes':[129,65], 'illustrative_intervals':[nr,nz],
163         'code_source_sha256':sha(ROOT/'source-data/frozen_empirical_engine.py'),
164         'no_pde_run':True})
165     style={'font.size':8.5,'svg.fonttype':'none','pdf.fonttype':42,'mathtext.fontset':'stixsans',
166         'lines.linewidth':.8,'axes.linewidth':.65,'figure.constrained_layout.use':False}
167     with publication_style('nature',language='zh-CN',width_mm=160,height_mm=112,overrides=style)
168     as materials:
169         fig=plt.figure(); ax=fig.add_axes([0,0,1,1]); ax.set(xlim=(0,160),ylim=(0,112));
170         ax.axis('off')

```

```

160     def txt(x,y,t,size=8.5,ha='center',va='center',color=TEXT,**kw):
161         return ax.text(x,y,t,fontsize=size,ha=ha,va=va,color=color,**kw)
162     def arrow(x1,y1,x2,y2,color=BLUE,lw=.8,style='->',scale=8):
163         ax.add_patch(FancyArrowPatch((x1,y1),(x2,y2),arrowstyle=style,mutation_scale=scale,color=
164             color,lw=lw))
165     def line(x,y,color=GRAY,lw=.5,ls='-'):
166         ax.plot(x,y,color=color,lw=lw,ls=ls)
167     txt(35,106,'(a) 半纵截面与节点控制体',8.8)
168     txt(118,106,'(b) 共享通量与环形面积',8.8)
169     X0,Y0,W,H=13,39,49,49
170     XF=X0+W*r; Y=Y0+H*z; XF=X0+W*r; YF=Y0+H*z
171     for a in XF: line([a,a],[Y0,Y0+H],color='#CCD4DA',lw=.45)
172     for b in YF: line([X0,X0+W],[b,b],color='#CCD4DA',lw=.45)
173     # Selected interior and side-boundary control volumes.
174     i,j=2,1
175     ax.add_patch(Rectangle((XF[i],YF[j]),XF[i+1]-XF[i],YF[j+1]-YF[j],facecolor=LIGHT,edgecolor=
176         BLUE,lw=.9,zorder=2))
177     ax.add_patch(Rectangle((XF[-2],YF[j]),XF[-1]-XF[-2],YF[j+1]-YF[j],facecolor='#E2E6EA',edgecolor=
178         GRAY,lw=.9,zorder=2))
179     rr,zz=np.meshgrid(X,Y,indexing='ij');ax.scatter(rr,zz,s=3.5,c=GRAY,zorder=3)
180     ax.scatter([X[i]],[Y[j]],s=11,c=BLUE,zorder=4)
181     # Symmetry boundaries and physical exterior are distinct line styles.
182     line([X0,X0],[Y0,Y0+H],color=TEXT,lw=1,ls='--')
183     line([X0,X0+W],[Y0,Y0],color=TEXT,lw=1,ls='--')
184     line([X0+W,X0+W],[Y0,Y0+H],color=BLUE,lw=1)
185     line([X0,X0+W],[Y0+H,Y0+H],color=BLUE,lw=1)
186     for xx in [27,42,55]: arrow(xx,88,xx,93,scale=6)
187     for yy in [48,65,80]: arrow(62,yy,68,yy,scale=6)
188     txt(37,97,'端面交换',8.2);txt(70.5,74,'侧面交换',8.2,ha='left',rotation=90)
189     txt(5.5,64,'轴线对称',8.2,rotation=90)
190     txt(37,32,'中面对称',8.2)
191     txt(10,36,r'$\phi$',8,ha='right');txt(64,36,r'$R$',8)
192     txt(9,89,r'$L/2$',8,ha='right');txt(65,30,r'$r$',9)
193     txt(9,98,r'$z$',9)
194     arrow(13,35,64,35,color=GRAY,lw=.6,scale=6);arrow(9,39,9,95,color=GRAY,lw=.6,scale=6)
195     txt(14,21,'● 节点未知量',8.2,ha='left')
196     txt(14,15,'线框: 控制体边界',8.2,ha='left')
197     txt(14,8,'网格已稀疏示意, 长径比例未按实物绘制',7.5,ha='left')
198     # The right inset shows adjacent ring-cell meridional rectangles.
199     a,b,c,yd,yt=91,119,149,62,88
200     ax.add_patch(Rectangle((a,yd),b-a,yt-yd,facecolor=LIGHT,edgecolor=GRAY,lw=.8))
201     ax.add_patch(Rectangle((b,yd),c-b,yt-yd,facecolor='FFFFFF',edgecolor=GRAY,lw=.8))
202     line([b,b],[yd,yt],color=BLUE,lw=1.6)
203     ax.scatter([104,134],[75,75],s=12,color=TEXT,zorder=4)
204     txt(103,69,r'$(i,j)$',8.5); txt(134,69,r'$(i+1,j)$',8.5)
205     arrow(107,79,132,79,color=BLUE,scale=8)
206     txt(119,95,'同一共享面通量',8.2)
207     line([119,119],[91,88],color=BLUE,lw=.65)

```

```

205     txt(119,57,'相邻控制体中符号相反',8.1)
206     txt(91,61,r'$r_{i-1/2}$',8,ha='center',va='top')
207     txt(119,61,r'$r_{i+1/2}$',8,ha='center',va='top')
208     txt(151,87,r'$z_{j+1/2}$',8,ha='left')
209     txt(151,63,r'$z_{j-1/2}$',8,ha='left')
210     # Annular axial face: radii convey topology, no physical scale or field.
211     ax.add_patch(Wedge((98,40),8,0,360,width=8*(1-rf[i]/rf[i+1]),facecolor=LIGHT,edgecolor=BL,
212                        UE,lw=.8))
213     ax.add_patch(Circle((98,40),.45,color=GRAY))
214     txt(110,43,'绕轴旋转',8.2,ha='left')
215     txt(110,36,'轴向面为环形',8.2,ha='left')
216     txt(85,25,r'$A^{\{r\}_{i\backslash pm 1/2},j}=2\pi r_{i\backslash pm 1/2}\backslash,\Delta z_j$',8.7,ha='left')
217     txt(85,16,r'$A^{\{z\}_{i,j\backslash pm 1/2}}=\pi(r_{i+1/2}^{\{2\}}-r_{i-1/2}^{\{2\}})$',8.7,ha='left')
218     txt(85,7,r'$\Delta V_{ij}=A^{\{z\}_{i,j\backslash pm 1/2}}\backslash,\Delta z_j$',8.7,ha='left')
219     fig.canvas.draw()
220     ir['provenance']['materials']=materials
221     ir['provenance']['construction_data_sha256']=sha(R00T/'source-data/fvm-construction.npz')
222     dump(R00T/f'{fid}.ir.json',ir)
223     dump(R00T/f'{fid}.typography.json',inspect_figure_typography(fig,final_width_mm=160))
224     export_figure(fig,R00T/fid,formats=['svg','pdf','png'],dpi=450,bbox_inches=None,mkdir=True,
225                  e,overwrite=True,
226                  write_manifest=True,provenance=ir['provenance'],
227                  accessibility={'title':claim,'desc': '实际主网格为128×64个区间；
228                  图中减少区间数示意节点与面，边界节点的控制体截断于边界。
229                  右侧表示共享面相反符号及全周环形面积。'})
230     plt.close(fig)
231
232
233 if __name__=='__main__':
234     workflow(); finite_volume()
235     dump(R00T/'structure-figures-manifest.json',{'sources':{p.name:sha(p) for p in SOURCES},
236          'script_sha256':sha(__file__), 'artifacts':{p.name:sha(p) for p in R00T.glob('fig-*) if
237          p.is_file()}},
238          'scope':'two explanatory constructions; no PDE solve, no figure depicts an observed
239          field'})

```

F.57 build_comparisons.py

类别：数据图形与示意图。原始代码共 204 行。

```

1  """Two evidence-bound additions: analytic length test and latent grid comparison."""
2  from pathlib import Path
3  import csv
4  import hashlib
5  import json
6  import os
7  import shutil
8  import sys
9

```

```

10 ROOT = Path(__file__).resolve().parent
11 CASE = ROOT.parent
12 BASE = CASE.parent
13 os.environ.setdefault('MPLCONFIGDIR', str(ROOT / '.mpl-cache'))
14 os.environ.setdefault('XDG_CACHE_HOME', str(ROOT / '.cache'))
15 sys.path[:0] = [str(ROOT / 'support/lab-ultra-scientific-figure-maker/scripts'),
16                 str(ROOT / 'support/lab-ultra-scientific-visualization/scripts')]
17 from figure_library import publication_style
18 from figure_typography import inspect_figure_typography
19 from figure_export import export_figure
20 import matplotlib
21 matplotlib.use('Agg')
22 import matplotlib.pyplot as plt
23 import numpy as np
24
25 BLUE, GRAY, DARK, LIGHT = '#315A75', '#75838C', '#26313A', '#E7EDF1'
26
27
28 def sha(path):
29     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
30
31
32 def dump(path, obj):
33     Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str) + '\n')
34
35
36 DATA = ROOT / 'data'
37 DATA.mkdir(exist_ok=True)
38 source_specs = {
39     'variable-summary.json': BASE /
40         'model-system-20260912/variable-length/results/q4-robin-latent0-32x16/summary.json',
41     'variable-delivery.csv': BASE /
42         'model-system-20260912/variable-length/delivery/case-summary.csv',
43     'latent-pairs.csv': BASE / 'model-system-20260912/latent-branch/analysis/final/pairs.csv',
44     'latent-grids.csv': BASE / 'model-system-20260912/latent-branch/analysis/final/grids.csv',
45 }
46 bindings = []
47 for name, source in source_specs.items():
48     target = DATA / name
49     shutil.copy2(source, target)
50     bindings.append({'file': 'data/' + name, 'source': str(source.resolve()), 'sha256':
51                     sha(target)})
52 dump(DATA / 'source-bindings.json', bindings)
53 source_manuscript = CASE / 'inputs/manuscript-before.md'
54 source_text = source_manuscript.read_text()
55 assumptions = source_text.split('## 轴向尺寸约束\n', 1)[1].split('## 守恒输运与显焓方程', 1)[0]
56 (DATA / 'length-equations-and-conditions.md').write_text(assumptions)

```

```

54 bindings.append({'file': 'data/length-equations-and-conditions.md', 'source':
    str(source_manuscript),
55                 'source_sha256': sha(source_manuscript),
56                 'sha256': sha(DATA / 'length-equations-and-conditions.md'),
57                 'extraction': 'Section 9.2, unchanged equations 23 through 26 and their stated
    conditions'})
58 dump(DATA / 'source-bindings.json', bindings)
59
60
61 def contract(fid, claim, height, sources, panels, variables, transforms, profile):
62     data_sources = [{'path': str(p.relative_to(ROOT)), 'sha256': sha(p)} for p in sources]
63     request = {'schema_version': '1.0.0', 'figure_id': fid, 'claim': claim,
64               'audience': '中文数学建模论文读者', 'medium': 'A4 paper',
65               'final_width_mm': 160, 'final_height_mm': height, 'language': 'zh-CN',
66               'data_sources': data_sources, 'variables': variables,
67               'uncertainty': {'kind': 'none; necessary analytic range or deterministic grid
    differences, not confidence bounds'},
68               'editable_format': 'svg', 'claim_locked': True}
69     ir = {'schema_version': '1.0.0', 'figure_id': fid, 'claim': claim,
70           'route': 'quantitative_code', 'recipe': 'analytic line and conditional point comparison'
    if 'length' in fid else 'paired grid difference line plot',
71           'panels': [{'panel_id': chr(97 + i), 'purpose': text} for i, text in enumerate(panels)],
72           'encodings': [{'field': v['name'], 'channel': v['role'], 'scale': 'linear'} for v in
    variables],
73           'annotations': [{'target': 'documented transformation', 'text': t} for t in transforms],
74           'style_tokens': {'palette': [BLUE, GRAY, DARK, LIGHT], 'base_font_pt': 8.5,
    'layout_owner': 'Matplotlib constrained layout'},
75           'exports': ['svg', 'pdf', 'png'],
76           'invariants': ['data', 'units', 'claim', 'transform', 'uncertainty'],
77           'provenance': {'renderer': 'Matplotlib', 'script_sha256': sha(__file__),
    'transformations': transforms, 'data_sources': data_sources}}
78     dump(ROOT / (fid + '.request.json'), request)
79     dump(ROOT / (fid + '.ir.json'), ir)
80     dump(ROOT / (fid + '.profile.json'), dict(profile, data_sources=data_sources))
81     return ir
82
83
84 def export(fig, fid, ir, style):
85     ir['provenance']['materials'] = style
86     dump(ROOT / (fid + '.ir.json'), ir)
87     dump(ROOT / (fid + '.typography.json'), inspect_figure_typography(fig, final_width_mm=160))
88     export_figure(fig, ROOT / fid, formats=['svg', 'pdf', 'png'], mkdir=True,
89                  overwrite=True, dpi=220, bbox_inches=None, write_manifest=True,
90                  provenance=ir['provenance'], accessibility={'title': fid, 'desc': ir['claim']})
91     plt.close(fig)
92
93

```

```

94 def length_test():
95     summary = json.loads((DATA / 'variable-summary.json').read_text())
96     config = summary['config']
97     r0, l0, c0 = (config[k] for k in ['R0', 'length0', 'C0'])
98     radius = summary['R_at_strict_stop_m']
99     assert np.isclose(radius, .012, rtol=0, atol=1e-12)
100    md = np.pi * r0**2 * l0 * (760 + 90 * c0) / (1 + c0)
101    assert np.isclose(md, summary['dry_mass_kg'], rtol=1e-14, atol=0)
102    lower = md / (760 * np.pi * radius**2) * 100
103    upper = 1.15 * md / (773.5 * np.pi * radius**2) * 100
104    results = list(csv.DictReader((DATA / 'variable-delivery.csv').open(encoding='utf-8-sig')))
105    row = next(r for r in results if r['case'] == 'q4-robin-latent0-32x16')
106    length = float(row['length_at_stop_cm'])
107    assert np.isclose(length, summary['L_at_strict_stop_m'] * 100, atol=1e-10, rtol=0)
108    assert lower <= length < upper
109    c = np.linspace(0, c0, 401)
110    v = (1 + c) / (760 + 90 * c)
111    np.savetxt(DATA / 'volume-state-curve.csv', np.column_stack([c, v]), delimiter=',',
112                header='C_kg_kgdry,v_m3_kgdry', comments='')
113    records = [{ 'label': '等比例缩放', 'kind': 'geometric rule', 'L_cm': l0 * radius / r0 * 100},
114                { 'label': '保持初始长度', 'kind': 'geometric rule', 'L_cm': l0 * 100},
115                { 'label': '变长度映射 A', 'kind': 'conditional computed output', 'L_cm': length}]
116    dump(DATA / 'length-bound-check.json', { 'Md0_kg': md, 'Rstar_m': radius,
117        'lower_cm': lower, 'upper_cm_exclusive': upper, 'C_condition': '0 <= C < 0.15 throughout
118        material',
119        'records': records, 'scope': 'Necessary condition under stated density and dry-mass
120        interpretation; not observed deformation or confidence interval.'})
121    fid = 'fig-length-bound-test'
122    claim = 'At radius 1.20 cm, the stated wet-density law and dry-mass conservation require
123    length in a necessary range; isotropic and fixed-initial-length rules fall below it, while the
124    existing Map A output lies within it.'
125    ir = contract(fid, claim, 82, [DATA / 'variable-summary.json', DATA / 'variable-delivery.csv',
126    DATA / 'length-equations-and-conditions.md', DATA / 'volume-state-curve.csv', DATA /
127    'length-bound-check.json'],
128        ['单位干质量体积的单调关系', '相同终止半径下的长度必要条件'],
129        [{ 'name': 'C_kg_kgdry', 'unit': 'kg/kg dry', 'role': 'x'}, { 'name': 'v_m3_kgdry * 1000',
130        'unit': 'L/kg dry', 'role': 'y'},
131        { 'name': 'L_cm', 'unit': 'cm', 'role': 'x'}],
132        ['v(C)=(1+C)/(760+90C); direct evaluation on 401 points; no fitted values.',
133        'Bounds from Eq.26 at R=1.20cm; lower closed, upper open for strict threshold.',
134        'Geometric-rule lengths and conditional Map A output are explicitly distinguished; no PDE
135        rerun.'],
136        { 'source_kind': 'analytic evaluation and one archived conditional simulation', 'rows':
137        { 'curve': 401, 'lengths': 3},
138        'missingness': 0, 'paired': False, 'C_range': [0, c0], 'L_range_cm': [15., length]})
139    with publication_style('nature', language='zh-CN', width_mm=160, height_mm=82) as style:

```

```

131     fig, axes = plt.subplots(1, 2, layout='constrained', gridspec_kw={'width_ratios': [1,
132     1.2]})
133     ax = axes[0]
134     ax.plot(c, v * 1000, color=BLUE, lw=1.6)
135     ax.axvspan(0, .15, color=LIGHT, zorder=0)
136     ax.scatter([0, .15, c0], [(1+x)/(760+90*x)*1000 for x in [0,.15,c0]], color=BLUE, s=16,
137     zorder=3)
138     ax.annotate('初态: 2.55', (c0, v[-1]*1000), xytext=(-4, -17), textcoords='offset points',
139     ha='right', fontsize=8)
140     ax.annotate('达标范围\n0 ≤ C < 0.15', (.075, 1.42), xytext=(.65, 1.85), fontsize=8,
141     arrowprops={'arrowstyle': '-', 'color': GRAY, 'lw': .7})
142     ax.set(xlabel='干基含水率 / (kg/kg)', ylabel='单位干质量体积 / (L/kg)',
143     xlim=(-.05, 2.66), ylim=(1.15, 3.8), xticks=[0,.5,1,1.5,2,2.55],
144     title='(a) 密度关系对应的体积')
145     ax.grid(axis='y', color='#D6DBDE', lw=.5)
146     ax = axes[1]
147     ax.axvspan(lower, upper, color=LIGHT, zorder=0)
148     ax.axvline(lower, color=BLUE, lw=.85)
149     ax.axvline(upper, color=BLUE, ls='--', lw=.85)
150     ax.scatter([records[0]['L_cm'], records[1]['L_cm']], [2,1], marker='x', color=DARK, s=30,
151     lw=1.1, zorder=3)
152     ax.scatter([length], [0], color=BLUE, s=26, zorder=3)
153     for y, record in zip([2,1,0], records):
154         ax.annotate(f"{record['L_cm']:.3f}", (record['L_cm'], y), xytext=(-3 if y==1 else 0,
155         9),
156         textcoords='offset points', ha='right' if y==1 else 'center', fontsize=8)
157     ax.annotate(f'{lower:.3f} ≤ L < {upper:.3f}', ((lower+upper)/2, 2.75), ha='center',
158     fontsize=8, color=DARK)
159     ax.set(yticks=[2,1,0], yticklabels=[r['label'] for r in records], xlim=(13.5,30.8),
160     ylim=(-.6,3.15),
161     xticks=[15,20,25,30], xlabel='长度 / cm', title='(b) 半径 1.20 cm 时的检查')
162     ax.grid(axis='x', color='#D6DBDE', lw=.5)
163     for ax in axes:
164         ax.spines[['top','right']].set_visible(False)
165     export(fig, fid, ir, style)
166
167 def latent_grid():
168     rows = list(csv.DictReader((DATA / 'latent-grids.csv').open()))
169     pairs = list(csv.DictReader((DATA / 'latent-pairs.csv').open()))
170     assert len(rows) == 6 and len(pairs) == 9
171     series = []
172     for row in rows:
173         cb, latent = float(row['Ceq_reference']), int(row['latent'])
174         vals = np.array([float(row['t12_h']), float(row['t24_h']), float(row['t32_h'])])
175         for n, val in zip([12,24,32], vals):
176             p = next(p for p in pairs if float(p['Ceq_reference'])==cb and int(p['nr'])==n)

```

```

171         assert np.isclose(val, float(p['on_h' if latent else 'off_h']), rtol=0, atol=1e-12)
172         series.append({'Cb': cb, 'latent': latent, 'grid': [12, 24, 32], 'hours': vals.tolist(),
173                     'difference_from_own_32_min': ((vals - vals[-1]) * 60).tolist()})
174     dump(DATA / 'latent-grid-series.json', series)
175     fid = 'fig-latent-grid'
176     claim = 'Archived paired latent cases at all three reference equilibria show decreasing
differences under grid refinement; the finest grid is a comparison reference, not the exact
solution.'
177     ir = contract(fid, claim, 83, [DATA / 'latent-grids.csv', DATA / 'latent-pairs.csv', DATA / 'latent-grid-s,
eries.json'],
178                 ['六条条件算例相对于各自最细网格的时差'],
179                 [{'name': 'radial_elements', 'unit': 'count', 'role': 'x'},
180                 {'name': 'difference_from_own_32_min', 'unit': 'min', 'role': 'y'}],
181                 ['For each fixed Cb and latent flag, compute 60*(t_N - t_32); do not compare unrelated model
times.',
182                 'x values 12, 24, 32 are true radial element counts; axial counts are N/2.',
183                 'No asymptotic order or exact-error bound inferred from three meshes.'],
184                 {'source_kind': 'archived numerical grid
comparison', 'rows': 18, 'groups': 6, 'missingness': 0, 'paired': True,
185                 'range_difference_min': [0., max(max(s['difference_from_own_32_min']) for s in series)]})
186     styles = {'.03': (BLUE, 'o'), '.075': (GRAY, 's'), '.12': (DARK, '^')}
187     with publication_style('nature', language='zh-CN', width_mm=160, height_mm=83) as style:
188         fig, ax = plt.subplots(layout='constrained')
189         for s in series:
190             color, marker = styles[s['Cb']]
191             ax.plot(s['grid'], s['difference_from_own_32_min'], color=color, marker=marker,
192                   ls='-' if s['latent'] else '--', mfc=color if s['latent'] else 'white',
193                   ms=4, lw=1.1, label=f"{s['Cb']:.3f}, {'开启' if s['latent'] else '关闭'}")
194             ax.set(xlim=(11, 33), ylim=(-1, 53), xticks=[12, 24, 32], xticklabels=['12×6', '24×12', '32×16'],
195                   xlabel='径向 × 轴向单元数', ylabel='相对本组 32×16 结果的时差 / min')
196             ax.legend(ncol=3, loc='upper right', frameon=False, fontsize=8, title='参考平衡含水率 /
(kg/kg); 相变通道', title_fontsize=8)
197             ax.grid(axis='y', color='#D6DBDE', lw=.5)
198             ax.spines[['top', 'right']].set_visible(False)
199             export(fig, fid, ir, style)
200
201 if __name__ == '__main__':
202     length_test()
203     latent_grid()
204     print('Generated fig-length-bound-test and fig-latent-grid with frozen data and explicit
transformations.')

```

F.58 audit_figures.py

类别：数据图形与示意图。原始代码共 81 行。

```
1 """Post-render contracts, source hashes, fonts, dimensions and PDF text bounds."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import re
6 import subprocess
7 import sys
8 import xml.etree.ElementTree as ET
9
10 HERE = Path(__file__).resolve().parent
11 REV = HERE/'review'
12 REV.mkdir(exist_ok=True)
13 TOOLS = HERE/'support/lab-ultra-scientific-figure-maker/scripts'
14 sha = lambda p: hashlib.sha256(Path(p).read_bytes()).hexdigest()
15 def dump(path, value):
16     Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2)+'\n')
17 def run(args):
18     r=subprocess.run(args,capture_output=True,text=True)
19     assert r.returncode==0,(args,r.stdout,r.stderr)
20     return r.stdout
21
22 source_records=json.loads((HERE/'source-data/source-manifest.json').read_text())
23 for s in source_records:
24     assert sha(s['path'])==s['sha256'],s['path']
25     if s['local_copy']:
26         assert sha(HERE/'source-data'/s['local_copy'])==s['sha256']
27 reports={}
28 for fid,preview in [('fig-latent-thermal-response','thermal-pdf-review'),
29                     ('fig-latent-geometry-energy','geometry-energy-pdf-review')]:
30     contract=run([sys.executable,str(TOOLS/'validate_figure_contract.py'),
31                 str(HERE/f'{fid}.request.json'),str(HERE/f'{fid}.ir.json')])
32     (REV/f'{fid}.contract.txt').write_text(contract)
33     svg=run([sys.executable,str(TOOLS/'audit_svg.py'),str(HERE/f'{fid}.svg')])
34     (REV/f'{fid}.svg-audit.json').write_text(svg)
35     svg_obj=json.loads(svg);assert svg_obj['valid'] and not svg_obj['errors']
36     assert svg_obj['counts']['embedded_rasters']==0
37     fonts=run(['pdffonts',str(HERE/f'{fid}.pdf')])
38     (REV/f'{fid}.pdffonts.txt').write_text(fonts)
39     assert 'ArialMT' in fonts and 'MicrosoftYaHei' in fonts
40     assert 'Type 3' not in fonts and 'Black' not in fonts and 'Bold' not in fonts
41     for line in fonts.splitlines()[2:]:
42         assert 'yes yes yes' in line
43     info=run(['pdftools',str(HERE/f'{fid}.pdf')])
44     (REV/f'{fid}.pdftools.txt').write_text(info)
45     assert re.search(r'Pages:\s+\d+\b',info)
```

```

46 match=re.search(r'Page size:\s+([\d]+) x ([\d]+) pts',info)
47 size=[float(x)*25.4/72 for x in match.groups()]
48 req=json.loads((HERE/f'{fid}.request.json').read_text())
49 assert abs(size[0]-160)<.01 and abs(size[1]-req['final_height_mm'])<.01
50 for s in req['data_sources']:
51     assert sha(HERE/s['path'])==s['sha256']
52 bbox=REV/f'{fid}.bbox.html'
53 run(['pdftotext','-bbox',str(HERE/f'{fid}.pdf'),str(bbox)])
54 xml=ET.fromstring(bbox.read_text())
55 page=[e for e in xml.iter() if e.tag.endswith('page')][0]
56 width,height=float(page.attrib['width']),float(page.attrib['height'])
57 escaped=[]
58 words=[e for e in page.iter() if e.tag.endswith('word')]
59 for e in words:
60     a=e.attrib
61     if float(a['xMin'])<-.1 or float(a['yMin'])<-.1 or float(a['xMax'])>width+.1 or
        float(a['yMax'])>height+.1:
62         escaped.append(e.text)
63 assert not escaped,escaped
64 typography=json.loads((HERE/f'{fid}.typography.json').read_text())
65 assert typography['minimum_final_font_pt']>=8
66 assert not typography['warnings']
67 run(['pdftoppm','-f','1','-singlefile','-scale-to','1600','-png',
68     str(HERE/f'{fid}.pdf'),str(REV/preview)])
69 run(['pdftoppm','-f','1','-singlefile','-scale-to','1100','-gray','-png',
70     str(HERE/f'{fid}.pdf'),str(REV/f'{fid}-grayscale')])
71 reports[fid]={ 'contract_pass':True,'svg_audit':svg_obj,'size_mm':size,
72     'font_names':['ArialMT','MicrosoftYaHei'],'all_pdf_fonts_embedded':True,
73     'minimum_final_font_pt':typography['minimum_final_font_pt'],
74     'escaped_text':escaped,'source_hashes_valid':True,
75     'pdf_sha256':sha(HERE/f'{fid}.pdf'),'svg_sha256':sha(HERE/f'{fid}.svg'),
76     'png_sha256':sha(HERE/f'{fid}.png'),'actual_pdf_render_sha256':sha(REV/f'{previe_
77     w}.png'),'grayscale_render_sha256':sha(REV/f'{fid}-grayscale.png'),
78     'visual_review':'recorded separately after viewing actual PDF and grayscale
        renders'}
79 dump(REV/'mechanical-audit.json',reports)
80 print(json.dumps({k:{'contract':v['contract_pass'],'width_mm':v['size_mm'][0],
81     'min_font_pt':v['minimum_final_font_pt'],'all_vector':True} for k,v in
        reports.items()}),indent=2))

```

F.59 build_figures.py

类别：数据图形与示意图。原始代码共 200 行。

```
1  """Render two source-bound comparisons, without running or importing PDE code."""
2  from pathlib import Path
3  import hashlib
4  import json
5  import os
6  import platform
7  import sys
8
9  HERE = Path(__file__).resolve().parent
10 os.environ.setdefault('MPLCONFIGDIR', str(HERE/'.mpl-cache'))
11 os.environ.setdefault('XDG_CACHE_HOME', str(HERE/'.cache'))
12 SUPPORT = HERE/'support'
13 sys.path[:0] = [str(SUPPORT/'lab-ultra-scientific-figure-maker/scripts'),
14                 str(SUPPORT/'lab-ultra-scientific-visualization/scripts')]
15 from figure_library import publication_style
16 from figure_typography import inspect_figure_typography
17 from figure_export import export_figure
18 import numpy as np
19 import matplotlib
20 matplotlib.use('Agg')
21 import matplotlib.pyplot as plt
22 from matplotlib import font_manager
23
24 # Office's installed regular Microsoft YaHei face matches the existing paper
25 # figures. Register it without copying or modifying any system font.
26 CJK_FONT = Path('/Applications/Microsoft Word.app/Contents/Resources/DFonts/msyh.ttc')
27 assert CJK_FONT.exists()
28 font_manager.fontManager.addfont(str(CJK_FONT))
29
30 DATA = HERE/'source-data/latent-pair-extract.npz'
31 with np.load(DATA) as z:
32     d = {k: z[k] for k in z.files}
33 audit = json.loads((HERE/'source-data/extraction-audit.json').read_text())
34 sha = lambda p: hashlib.sha256(Path(p).read_bytes()).hexdigest()
35 def dump(path, value):
36     Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, default=str,
37                                     allow_nan=False)+'\n')
37
38 BLUE = '#244F73'
39 GRAY = '#7A8691'
40 DARK = '#28333D'
41 PAIRS = [(0, GRAY, '--', '关闭相变'), (1, BLUE, '-', '开启相变')]
42 STYLE = {'font.family': ['Arial', 'Microsoft YaHei'],
43          'font.size': 8.5, 'axes.labelsize': 8.5, 'axes.titlesize': 8.5,
44          'xtick.labelsize': 8, 'ytick.labelsize': 8, 'legend.fontsize': 8,
```

```

45     'axes.spines.top': False, 'axes.spines.right': False, 'axes.linewidth': .65,
46     'lines.linewidth': 1.25, 'grid.color': '#DDE2E6', 'grid.linewidth': .5,
47     'svg.fonttype': 'none', 'pdf.fonttype': 42, 'path.simplify': False,
48     'figure.constrained_layout.h_pad': .055, 'figure.constrained_layout.w_pad': .055}
49
50 def prepare(fid, claim, height, panels, variables, transformations, annotations):
51     sources = [{'path': 'source-data/latent-pair-extract.npz', 'sha256': sha(DATA)},
52                {'path': 'source-data/extraction-audit.json',
53                 'sha256': sha(HERE/'source-data/extraction-audit.json')}]
54     request = {'schema_version': '1.0.0', 'figure_id': fid, 'claim': claim,
55               'audience': '中文数学建模论文读者', 'medium': 'A4 paper',
56               'final_width_mm': 160, 'final_height_mm': height, 'language': 'zh-CN',
57               'data_sources': sources, 'variables': variables,
58               'uncertainty': {'kind': 'none; deterministic paired conditional outputs, not
59                               calibrated material evidence'},
60               'editable_format': 'svg', 'claim_locked': True}
61     ir = {'schema_version': '1.0.0', 'figure_id': fid, 'claim': claim,
62           'route': 'quantitative_code', 'recipe': 'bundled render_recipe line series; Matplotlib
63           constrained multi-panel layout',
64           'panels': [{'panel_id': chr(97+i), 'purpose': name} for i, name in enumerate(panels)],
65           'encodings': [{'field': x['name'], 'channel': x['role'], 'scale': 'linear',
66                          'source_layer': 'deterministic'} for x in variables],
67           'annotations': annotations, 'style_tokens': {'palette': 'blue-gray', 'base_font_pt': 8.5,
68               'line_semantics': 'latent off dashed gray; latent on solid blue; energy channels
69               additionally use distinct dashes'},
70           'exports': ['svg', 'pdf', 'png'], 'invariants':
71           ['data', 'units', 'claim', 'transform', 'uncertainty'],
72           'provenance': {'renderer': 'matplotlib', 'matplotlib_version': matplotlib.__version__,
73               'numpy_version': np.__version__, 'python_version': platform.python_version(),
74               'script_sha256': sha(__file__), 'data_sources': sources, 'transformations':
75               transformations,
76               'recipe_lookup': 'Loaded Lab-ultra material-library; find --tag line yielded no
77               external match; inspected bundled render_recipe.py draw_panel line route.',
78               'style_helper':
79               'support/lab-ultra-scientific-figure-maker/scripts/figure_library.py',
80               'style_helper_sha256':
81               sha(SUPPORT/'lab-ultra-scientific-figure-maker/scripts/figure_library.py')}}
73     profile = {'source_kind': 'frozen deterministic numerical results', 'paired': True,
74               'pair_count': 1, 'cases': 2, 'Ceq_reference_kgkg': .075, 'grid': [32,16],
75               'sample_counts': {'saved_field_per_case': 437, 'on_accepted_budget_records': 9474,
76               'known_initial_budget_anchor': 1},
77               'missing_policy': 'no imputation; fail on nonfinite input',
78               'same_radius_input': True, 'same_geometry_rule': True,
79               'length_is_observed': False, 'only_config_difference': ['latent'],
80               'transforms': transformations, 'fields': {k: {'shape': list(v.shape),
81               'dtype': str(v.dtype), 'missing': int(np.isnan(v).sum())},

```

```

82         'min': float(v.min()), 'max': float(v.max())} for k,v in d.items() if k !=
        'raw_on_budget_steps'}}
83     for suffix, obj in [('request',request),('ir',ir),('profile',profile)]:
84         dump(HERE/f'{fid}.{suffix}.json',obj)
85     return ir
86
87 def finish(fig, fid, ir, materials):
88     fig.canvas.draw()
89     ir['provenance']['materials'] = materials
90     dump(HERE/f'{fid}.ir.json', ir)
91     dump(HERE/f'{fid}.typography.json', inspect_figure_typography(fig, final_width_mm=160))
92     export_figure(fig, HERE/fid, formats=['svg','pdf','png'], dpi=350,
93                 bbox_inches=None, mkdir=True, overwrite=True, write_manifest=True,
94                 provenance=ir['provenance'], accessibility={'title':ir['claim'],
95                 'desc': '第四问未标定表面关系下的相变通道组内配对;
                        全部曲线由已保存的同源数值样本绘制。'})
96     plt.close(fig)
97
98     fid = 'fig-latent-thermal-response'
99     ir = prepare(fid,
100     '在第四问Cb=0.075、32×16的同边界条件配对中，相变能量通道改变前期中心及侧面中部温度，
    并延后全域含水率达标；数值属于未标定条件模型。',
101     111, ['前 12 小时中面轴心温度','前 12 小时侧面中部温度','0—72 小时全域最大含水率'],
102     [{'name':'latent0_time_h,latent1_time_h,environment_time_h','unit':'h','role':'x'},
103     {'name':'latent0_Tcenter_C,latent1_Tcenter_C,latent0_Tside_mid_C,latent1_Tside_mid_C,environm,
    ent_temperature_C','unit':'degC','role':'y'},
104     {'name':'latent0_Cmax,latent1_Cmax','unit':'kg/kg dry matter','role':'y'}],
105     ['Field series use all 437 saved samples per case; thermal panels explicitly restrict the
    displayed interval to 0–12 h. No smoothing or fitting.',
106     'Temperature recovered from H,C as 28+H/(1850+4000C); index 0 is r=z=0, side index 544 is
    r=R(t),z=0.',
107     '0–4 h environment line joins attachment records using their specified piecewise linear input;
    after 4 h the line uses the source-model final-hour mean 49.99893442622951 degC.',
108     'Cmax is the maximum over all saved nodes, not a prespecified point. Threshold-event
    annotations come from existing solver summary, not visual line intersections.',
109     'Both runs share R(t) and the same local map rule, but their solved L(t) differs.'],
110     [{'target':'existing off/on critical_h from summary','text':'49.09 h / 59.03 h'},
111     {'target':'maximum moisture criterion','text':'0.15 kg/kg threshold'}])
112 with publication_style('nature', language='zh-CN', width_mm=160, height_mm=111, overrides=STYLE)
    as materials:
113     fig = plt.figure(layout='constrained')
114     gs = fig.add_gridspec(2,2,height_ratios=[1,1.16])
115     axes = [fig.add_subplot(gs[0,0]),fig.add_subplot(gs[0,1]),fig.add_subplot(gs[1,:])]
116     for ax, key, title in [(axes[0],'Tcenter_C','(a) 中面轴心温度'),
117     (axes[1],'Tside_mid_C','(b) 侧面中部温度')]:
118         for a,col,ls,label in PAIRS:
119             sel=d[f'latent{a}_time_h']<=12

```

```

120         ax.plot(d[f'latent{a}_time_h'][sel],d[f'latent{a}_{key}'][sel],color=col,ls=ls,label=,
                label)
121     ax.plot(d['environment_time_h'],d['environment_temperature_C'],color=DARK,ls=':',lw=1,label=,
            el='环境输入')
122     ax.plot([4,12],[float(d['environment_tail_temperature_C'])]*2,color=DARK,ls=':',lw=1)
123     ax.set(xlim=(0,12),ylim=(24,52),xlabel='时间 / h',ylabel='温度 / °C',title=title)
124     ax.set_xticks([0,3,6,9,12]);ax.grid(axis='y')
125     axes[0].legend(loc='lower right',frameon=False,handlelength=2.4)
126     ax=axes[2]
127     for a,col,ls,label in PAIRS:
128         ax.plot(d[f'latent{a}_time_h'],d[f'latent{a}_Cmax'],color=col,ls=ls,label=label)
129     ax.axhline(.15,color=DARK,ls=':',lw=.9,label='全域阈值 0.15')
130     ax.set(xlim=(0,72),ylim=(0,2.65),xlabel='时间 / h',ylabel='全域最大含水率 /
            (kg/kg)',title='(c) 全域含水率响应')
131     ax.set_xticks([0,12,24,36,48,60,72]);ax.grid(axis='y')
132     ax.legend(loc='upper right',frameon=False,handlelength=2.4)
133     for a,textpos in [(0,(38,.68)),(1,(58,1.00))]:
134         t=float(d[f'latent{a}_critical_h'])
135         ax.annotate(f'{t:.2f} h',xy=(t,.15),xytext=textpos,fontsize=8,
136                 arrowprops={'arrowstyle':'-','color':GRAY,'lw':.7})
137     finish(fig,fid,ir,materials)
138
139 fid = 'fig-latent-geometry-energy'
140 ir = prepare(fid,
141     '同一第四问条件配对产生不同的未测长度历史; 含相变组长度峰值约45.12 cm。
    其累计对流供热与水携焓流出及材料显焓变化相配, 早期显焓变化为负。',
142     115, ['0—72小时两种能量设置的条件长度','含相变组0—
    72小时累计能量','同一材料显焓变化的最初半小时放大'],
143     [{'name':'latent0_time_h,latent1_time_h,energy_time_h','unit':'h','role':'x'},
144     {'name':'latent0_length_cm,latent1_length_cm','unit':'cm','role':'y'},
145     {'name':'energy_convection_in_kJ,energy_water_enthalpy_out_kJ,energy_storage_change_kJ','unit',
        '':'kJ','role':'y'}],
146     ['Length uses all 437 saved samples on 0–72 h; peak 45.12286956825604 cm at saved time 7.5 h is
    preserved. No monotonicity clamp or truncated peak.',
147     'Energy uses all 9474 accepted-step budget records plus exact known t=0 cumulative-integral/H
    anchor.',
148     'Specific energy channels in J/kg dry matter are multiplied by whole-body dry mass
    0.0875663642458338 kg and divided by 1000 to obtain kJ. Mirrored-domain factor is already in
    mass and boundary weights; no extra factor 2.',
149     'Qin=-Md*cum_convection_out/1000; Ewater=Md*cum_water_enthalpy_out/1000;
    deltaEH=Md*(mean_H-Hinitial)/1000, Hinitial=0.',
150     'Zero radiation channel verified. Qin-Ewater-deltaEH has maximum absolute residual 1.69915e-5
    kJ. Internal flux redistribution cancels in the whole-body account.',
151     'Water enthalpy export is signed upstream vapor enthalpy times net mass flux, including vapor
    sensible enthalpy; it is not an isolated latent-heat integral.',

```

```

152     'Panel c displays original accepted budget samples from the first 0.5 h of the storage series
      in panel b, using a labeled independent scale to expose the early negative value; no second
      y-axis, smoothing or normalization.'],
153     [{'target':'saved length peak latent1 at 7.5 h','text':'45.12 cm'},
154      {'target':'saved length peak latent0 at 6 h','text':'32.33 cm'},
155      {'target':'minimum energy_storage_change_kJ','text':'-0.97 kJ'}])
156 with publication_style('nature', language='zh-CN', width_mm=160, height_mm=115, overrides=STYLE)
      as materials:
157     fig=plt.figure(layout='constrained')
158     gs=fig.add_gridspec(2,2,height_ratios=[1,1.12])
159     axes=[fig.add_subplot(gs[0,:]),fig.add_subplot(gs[1,0]),fig.add_subplot(gs[1,1])]
160     ax=axes[0]
161     for a,col,ls,label in PAIRS:
162         ax.plot(d[f'latent{a}_time_h'],d[f'latent{a}_length_cm'],color=col,ls=ls,label=label)
163     ax.axhline(25,color=DARK,ls=':',lw=.8,label='初始长度 25 cm')
164     ax.set(xlim=(0,72),ylim=(23,48),xlabel='时间 / h',ylabel='条件计算长度 / cm',title='(a)
      相同半径输入下的长度响应')
165     ax.legend(loc='upper right',frameon=False,handlelength=2.4)
166     for a,pos in [(0,(13,26.6)),(1,(25,44))]:
167         v=d[f'latent{a}_length_cm'];j=int(np.argmax(v));t=d[f'latent{a}_time_h'][j]
168         ax.annotate(f'{v[j]:.2f} cm',xy=(t,v[j]),xytext=pos,fontsize=8,
169                    arrowprops={'arrowstyle':'-','color':GRAY,'lw':.7})
170     ax=axes[1]
171     for key,col,ls,label in [('energy_convection_in_kJ',BLUE,'-','对流供热'),
172                             ('energy_water_enthalpy_out_kJ',GRAY,'--','水携焓流出'),
173                             ('energy_storage_change_kJ',DARK,'-.','材料显焓变化')]:
174         ax.plot(d['energy_time_h'],d[key],color=col,ls=ls,label=label)
175     ax.set(xlim=(0,72),ylim=(-15,560),xlabel='时间 / h',ylabel='能量 / kJ',title='(b)
      含相变组累计能量')
176     ax.legend(loc='center right',frameon=False,handlelength=2.4,fontsize=8)
177     ax.set_yticks([0,200,400])
178     ax=axes[2]
179     early=d['energy_time_h']<=.5
180     assert d['energy_time_h'][early][-1]==.5
181     ax.plot(d['energy_time_h'][early],d['energy_storage_change_kJ'][early],color=DARK,ls='-.-')
182     ax.axhline(0,color=GRAY,ls=':',lw=.8)
183     ax.set(xlim=(0,.5),ylim=(-1.35,4.4),xlabel='时间 / h',ylabel='材料显焓变化 / kJ',title='(c)
      最初半小时的显焓变化')
184     ax.set_yticks([-1,0,2,4])
185     j=int(np.argmin(d['energy_storage_change_kJ']))
186     ax.annotate(f"{d['energy_storage_change_kJ'][j]:.2f} kJ",
187               xy=(d['energy_time_h'][j],d['energy_storage_change_kJ'][j]),xytext=(.20,-.90),
188               fontsize=8,arrowprops={'arrowstyle':'-','color':GRAY,'lw':.7})
189     for ax in axes:
190         ax.set_xticks([0,12,24,36,48,60,72] if ax is axes[0] else
191                      [0,24,48,72] if ax is axes[1] else [0,.1,.2,.3,.4,.5])
192     ax.grid(axis='y')

```

```

193     finish(fig,fid,ir,materials)
194
195 dump(HERE/'runtime.json', {'python':sys.version,'numpy':np.__version__,
196     'matplotlib':matplotlib.__version__, 'platform':platform.platform(),
197     'new_PDE_runs':0,'fitted_values':0,'smoothing':False,
198     'explicit_font_family':['Arial','Microsoft YaHei'],
199     'registered_existing_font':str(CJK_FONT),'registered_font_sha256':sha(CJK_FONT)})
200 print('Rendered exactly two figures at width 160 mm.')

```

F.60 prepare_data.py

类别：数据图形与示意图。原始代码共 230 行。

```

1  """Extract and audit existing latent branch evidence. Never imports a PDE solver."""
2  from pathlib import Path
3  import csv
4  import hashlib
5  import json
6  import shutil
7  import zipfile
8  import xml.etree.ElementTree as ET
9  import numpy as np
10
11  HERE = Path(__file__).resolve().parent
12  BASE = HERE.parents[1]
13  SOURCE = BASE / 'model-system-20260912/latent-branch'
14  DATA = HERE / 'source-data'
15  DATA.mkdir(parents=True, exist_ok=True)
16
17  def digest(path):
18      return hashlib.sha256(path.read_bytes()).hexdigest()
19
20  def dump(path, obj):
21      path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False) + '\n')
22
23  sources = []
24  def bind(rel, copy_as=None):
25      path = SOURCE / rel
26      sources.append({'path': str(path), 'sha256': digest(path), 'bytes': path.stat().st_size,
27                     'local_copy': copy_as})
28      if copy_as:
29          dest = DATA / copy_as
30          dest.parent.mkdir(parents=True, exist_ok=True)
31          shutil.copy2(path, dest)
32      return path
33
34  def rows(path):
35      with path.open() as f:
36          return list(csv.DictReader(f))

```

```

37
38 def read_numeric_sheet(path):
39     # Attachments contain a single sheet with a header followed by numeric cells.
40     # Read stored OOXML cell values only, without evaluating or inventing data.
41     with zipfile.ZipFile(path) as z:
42         sheets = [n for n in z.namelist() if n.startswith('xl/worksheets/sheet') and
43                    n.endswith('.xml')]
44         assert len(sheets) == 1
45         tree = ET.fromstring(z.read(sheets[0]))
46         ns = {'x': 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}
47         out = []
48         for row in tree.findall('.//x:sheetData/x:row', ns)[1:]:
49             cells = row.findall('x:c', ns)
50             out.append([float(c.find('x:v', ns).text) for c in cells])
51     return np.asarray(out, float)
52
53 csvs = {k: rows(bind(f'analysis/final/{k}.csv', f'{k}.csv'))
54          for k in ['pairs', 'grids', 'key-times', 'trajectories', 'cases']}
55 for rel in ['src/plot_branch.py', 'src/audit_branch.py', 'src/run_experiment.py',
56            'src/elongation_model.py', 'src/elongation_frozen/volume_fem.py',
57            'src/elongation_frozen/boundary_model.py',
58            'src/elongation_frozen/thermo_reference.py']:
59     bind(rel, rel)
60
61 arrays, summaries, audits, keyrows = {}, {}, {}, []
62 for alpha in (0, 1):
63     name = f'q4-latent{alpha}-ce075-32x16'
64     cr = [r for r in csvs['cases'] if r['case'] == name]
65     assert len(cr) == 1
66     cr = cr[0]
67     sp = bind(cr['source'], f'{name}-summary.json')
68     assert digest(sp) == cr['source_sha256']
69     s = json.loads(sp.read_text())
70     summaries[alpha] = s
71     cfg = s['config']
72     assert cfg['latent'] == alpha and cfg['nr'] == 32 and cfg['nz'] == 16
73     assert cfg['question'] == 4 and cfg['equilibrium_C50'] == .075
74     assert cfg['energy_gauge'] == cfg['desorption_B'] == cfg['emissivity'] == 0
75     fp = bind(str(Path(cr['source']).parent/'fields.npz'))
76     with np.load(fp) as a:
77         t, u, rec, m = a['times'], a['states_H_C'], a['records'], a['lumped_dry_mass']
78         b = a['budget_steps']
79         bc = {k: i for i, k in enumerate(s['budget_columns'])}
80         rc = {k: i for i, k in enumerate(a['record_columns'])}
81         C, H = u[:, :, 1], u[:, :, 0]
82         T = 28 + H/(1850 + 4000*C)
83         Md = float(m.sum())
84         Md0 = (760+90*2.55)/(1+2.55)*np.pi*.02**2*.25

```

```

84     assert np.isclose(Md, s['dry_mass_kg'], atol=1e-15, rtol=0)
85     assert np.isclose(Md, Md0, atol=1e-15, rtol=0)
86     assert t[0] == 0 and t[-1] == 259200 and np.all(np.diff(t) > 0)
87     assert np.array_equal(t, rec[:, rc['time_s']])
88     center, side = 0, cfg['nr']*(cfg['nz']+1)
89     # Flattening is radial-major. This is the current side at the midplane,
90     # not an end-face average, a surface average, or a fixed initial radius.
91     assert np.all(a['r_nodes_m'].reshape(len(t), -1)[: , center] == 0)
92     assert np.all(a['z_nodes_m'].reshape(len(t), -1)[: , center] == 0)
93     assert np.all(a['z_nodes_m'].reshape(len(t), -1)[: , side] == 0)
94     assert np.allclose(a['r_nodes_m'].reshape(len(t), -1)[: , side],
95                        rec[:, rc['radius_m']], atol=1e-15, rtol=0)
96     meanC, meanH = C@m/Md, H@m/Md
97     assert np.allclose(C.max(axis=1), rec[:, rc['Cmax']], atol=2e-12, rtol=0)
98     assert np.allclose(meanC, rec[:, rc['Cmean_dry']], atol=2e-12, rtol=0)
99     tr = [r for r in csvs['trajectories'] if r['case'] == name]
100    expected = {'t_h': t/3600, 'Cmax': C.max(axis=1), 'Cmean_dry': meanC,
101               'Tmean_dry_C': T@m/Md, 'Tcenter_C': T[:, center],
102               'Ccenter': C[:, center], 'L_cm': 100*rec[:, rc['length_m']],
103               'R_cm': 100*rec[:, rc['radius_m']]}
104    assert len(tr) == len(t)
105    csv_errors = {}
106    for k, v in expected.items():
107        vv = np.array([float(r[k]) for r in tr])
108        csv_errors[k] = float(np.max(abs(vv-v)))
109        assert np.allclose(vv, v, atol=2e-11, rtol=0), (name, k)
110    for r in [r for r in csvs['key-times'] if r['case'] == name]:
111        j = np.flatnonzero(t == float(r['t_h'])*3600)
112        assert len(j) == 1
113        j = int(j[0])
114        for k, val in [('Tcenter_C', T[j, center]), ('Tside_mid_C', T[j, side]),
115                   ('Ccenter', C[j, center]), ('Cside_mid', C[j, side])]:
116            assert abs(float(r[k])-val) < 2e-11, (name, k)
117        keyrows.append({k: (float(v) if k != 'case' else v) for k, v in r.items()})
118    initialH = float(meanH[0])
119    assert initialH == 0 and np.all(H[0] == 0)
120    massres = b[:, bc['mean_C']]-cfg['C0']+b[:, bc['cum_mass_out']]
121    energyres = b[:, bc['mean_H']]-initialH+b[:, bc['cum_total_energy_out']]
122    assert np.allclose(massres, b[:, bc['mass_residual']], atol=2e-12, rtol=0)
123    assert np.allclose(energyres, b[:, bc['enthalpy_residual']], atol=2e-8, rtol=0)
124    assert np.allclose(b[:, bc['cum_total_energy_out']],
125                       b[:, bc['cum_convection_out']]+b[:, bc['cum_radiation_out']]
126                       +b[:, bc['cum_water_enthalpy_out']], atol=2e-7, rtol=0)
127    assert np.all(b[:, bc['cum_radiation_out']] == 0)
128    assert np.allclose([meanH[-1], meanC[-1]], b[-1, 1:3], atol=2e-8, rtol=1e-13)
129    kmax = int(np.argmax(rec[:, rc['length_m']]))
130    assert abs(s['L_max_m']*100-float(cr['Lmax_cm'])) < 1e-12
131    pfx = f'latent{alpha}_

```

```

132     for k, v in {'time_h': t/3600, 'Tcenter_C': T[:, center], 'Tside_mid_C': T[:, side],
133                 'Cmax': C.max(axis=1), 'length_cm': 100*rec[:, rc['length_m']],
134                 'radius_cm': 100*rec[:, rc['radius_m']]}.items():
135         arrays[pfx+k] = v.copy()
136     arrays[pfx+'critical_h'] = np.array(s['critical_h'])
137     audits[name] = {'csv_matches_saved_fields': True, 'csv_max_abs_error': csv_errors,
138                   'saved_samples': len(t), 'budget_samples': len(b), 'dry_mass_kg': Md,
139                   'initial_mean_H_J_kgdry': initialH, 'center_node_index': center,
140                   'side_midplane_node_index': side, 'critical_h': s['critical_h'],
141                   'strict_stop_s': s['strict_stop_s'],
142                   'saved_length_peak_cm': float(100*rec[kmax, rc['length_m']]),
143                   'saved_length_peak_time_h': float(t[kmax]/3600),
144                   'accepted_step_length_peak_cm': float(s['L_max_m']*100),
145                   'budget_column_map': bc, 'record_column_map': rc}
146     if alpha == 1:
147         # Stored budget channels are specific to whole-body dry mass. The
148         # FEM uses 4*pi weights for the mirrored half: do not multiply by 2.
149         factor = Md/1000 # J/kg_dry * kg_dry -> J -> kJ
150         quantities = {
151             'energy_convection_in_kJ': -b[:, bc['cum_convection_out']]*factor,
152             'energy_water_enthalpy_out_kJ': b[:, bc['cum_water_enthalpy_out']]*factor,
153             'energy_storage_change_kJ': (b[:, bc['mean_H']]-initialH)*factor,
154             'energy_residual_kJ': energyres*factor}
155         # t=0 is an exact initial datum: H=0 and all cumulative integrals=0.
156         # The first budget record is an accepted step at positive t. Append
157         # this known endpoint explicitly; no field interpolant is generated.
158         arrays['energy_time_h'] = np.r_[0., b[:, bc['time_s']]/3600]
159         for k, v in quantities.items():
160             arrays[k] = np.r_[0., v]
161         arrays['raw_on_budget_steps'] = b.copy()
162         balance = quantities['energy_convection_in_kJ']-quantities['energy_water_enthalpy_out_
163         _kJ']-quantities['energy_storage_change_kJ']
164         assert np.max(abs(balance+quantities['energy_residual_kJ'])) < 2e-10
165         audits['energy'] = {'whole_body_dry_mass_kg': Md, 'specific_to_kJ_factor': factor,
166                           'all_channels_outward_positive_in_source': True,
167                           'convection_plotted_inward_positive': True,
168                           'zero_radiation_confirmed': True, 'initial_H_is_zero': True,
169                           'max_absolute_balance_error_kJ': float(np.max(abs(balance))),
170                           'min_storage_change_kJ': float(quantities['energy_storage_change_kJ'].min()),
171                           'min_storage_time_h': float(b[np.argmin(quantities['energy_storage_change_kJ']),
172                           0]/3600),
173                           'max_storage_change_kJ': float(quantities['energy_storage_change_kJ'].max()),
174                           'end_72h_kJ': {k: float(v[-1]) for k, v in quantities.items()},
175                           'water_enthalpy_definition': 'Signed integral of upstream vapor enthalpy times net
176                           outward water mass flux; includes latent and vapor sensible enthalpy. Not isolated
177                           latent heat.',
178                           'storage_definition': 'Whole-sample effective sensible enthalpy relative to initial
179                           28 degC, using H=(1850+4000C)(T-28).',

```

```

174         'initial_anchor': 'Exact zero-time initial H and cumulative-integral endpoint added
        before positive-time accepted budget records.}'

175
176 changes = [k for k in summaries[0]['config'] if summaries[0]['config'][k] !=
        summaries[1]['config'][k]]
177 assert changes == ['latent']
178 assert np.array_equal(arrays['latent0_time_h'], arrays['latent1_time_h'])
179 assert np.array_equal(arrays['latent0_radius_cm'], arrays['latent1_radius_cm'])
180 pair = [r for r in csvs['pairs'] if float(r['Ceq_reference']) == .075 and int(r['nr']) == 32]
181 assert len(pair) == 1
182 pair = pair[0]
183 for k, val in [('off_h', summaries[0]['critical_h']), ('on_h', summaries[1]['critical_h']),
184               ('delay_h', summaries[1]['critical_h']-summaries[0]['critical_h'])]:
185     assert abs(float(pair[k])-val) < 1e-12
186 for a in (0, 1):
187     gr = [r for r in csvs['grids'] if float(r['Ceq_reference']) == .075 and int(r['latent']) == a]
188     assert len(gr) == 1 and abs(float(gr[0]['t32_h'])-summaries[a]['critical_h']) < 1e-12
189     assert
        abs(float(gr[0]['change_24_to_32_minutes'])-60*(float(gr[0]['t32_h'])-float(gr[0]['t24_h'])))
        < 1e-10
190
191 for name in ['附件 1.xlsx', '附件 2.xlsx']:
192     p = bind('input/'+name, name)
193     v = read_numeric_sheet(p)
194     if name == '附件 1.xlsx':
195         assert v.shape == (241, 3) and v[0, 0] == 0 and v[-1, 0] == 14400
196         arrays['environment_time_h'], arrays['environment_temperature_C'],
        arrays['environment_water_index'] = v.T.copy()
197         arrays['environment_time_h'] /= 3600
198         tail = v[v[:, 0] >= 10800, 1:].mean(axis=0)
199         arrays['environment_tail_temperature_C'] = np.array(tail[0])
200         audits['environment'] = {'observed_rows': 241, 'observed_range_h': [0, 4], 'tail_rows': 61,
201                                'tail_T_C': float(tail[0]), 'tail_water_index': float(tail[1]),
202                                'extension': '0-4 h piecewise linear observations; t>4 h uses mean
        of samples t>=3 h, identical to source code.}'
203     else:
204         assert v.shape == (145, 2)
205         rr = np.interp(arrays['latent0_time_h']*3600, v[:, 0], v[:, 1])
206         assert np.allclose(rr, arrays['latent0_radius_cm'], atol=1e-13, rtol=0)
207         arrays['radius_observation_time_h'], arrays['radius_observation_cm'] = v.T.copy()
208         arrays['radius_observation_time_h'] /= 3600
209
210 for k, v in arrays.items():
211     assert np.all(np.isfinite(v)), k
212 np.savez_compressed(DATA/'latent-pair-extract.npz', **arrays)
213 audits['pair'] = {'only_configuration_difference': changes, 'same_saved_times': True,
214                  'same_prescribed_radius': True, 'pair_csv': pair,

```

```

215             'selection': 'Representative uncalibrated reference Cb=0.075, chosen by request,
                not ranked as best fit.')}
216 audits['sources'] = sources
217 audits['extraction_sha256'] = digest(DATA/'latent-pair-extract.npz')
218 audits['new_PDE_runs'] = 0
219 dump(DATA/'extraction-audit.json', audits)
220 dump(DATA/'key-times-selected.json', keyrows)
221 dump(DATA/'source-manifest.json', sources)
222
223 # Reuse frozen helper code from the completed Lab-ultra figure bundle. Its exact
224 # contents are shipped locally and hashed; no runtime solver is imported.
225 old = BASE/'outline-rewrite-20260913/figures/support'
226 shutil.copytree(old, HERE/'support', dirs_exist_ok=True)
227 dump(HERE/'support-manifest.json', [{'path': str(p.relative_to(HERE)), 'sha256': digest(p)}
228     for p in sorted((HERE/'support').rglob('*')) if p.is_file() and '__pycache__' not in p.parts])
229 print(json.dumps({'pair_config_difference': changes, 'samples_per_case': 437,
230                 'energy': audits['energy'], 'saved_peak_cm':
                    audits['q4-latent1-ce075-32x16']['saved_length_peak_cm']}, ensure_ascii=False))

```

F.61 build_figures.py

类别：数据图形与示意图。原始代码共 200 行。

```

1  """Compact native finite-volume diagram and lossless user workflow placement."""
2  from pathlib import Path
3  import os, sys, json, hashlib, shutil
4
5  ROOT = Path(__file__).resolve().parent
6  BASE = ROOT.parents[1]
7  OLD = BASE / 'outline-rewrite-20260913/figures'
8  HELPERS = ROOT / 'support'
9  os.environ['MPLCONFIGDIR'] = str(BASE / 'cylinder-hatching-20260913/evidence/.mpl-cache')
10 os.environ['XDG_CACHE_HOME'] = str(BASE / 'cylinder-hatching-20260913/evidence/.cache')
11 sys.path.insert(0, str(HELPERS))
12 import numpy as np
13 import matplotlib as mpl
14 mpl.use('Agg')
15 import matplotlib.pyplot as plt
16 from matplotlib.patches import Rectangle, Wedge, Circle, FancyArrowPatch, Polygon, Ellipse
17 import figure_typography as typography
18 from figure_export import export_figure
19 from PIL import Image
20 import fitz
21
22 typography.LATIN = ('Arial',)
23 typography.CJK_SC = ('Microsoft YaHei',)
24 RC, FONT_RECEIPT = typography.resolve_figure_fonts('zh-CN')
25 RC.update({'font.size':8.5, 'svg.fonttype':'none', 'pdf.fonttype':42,
26          'mathtext.fontset':'stixsans', 'hatch.linewidth':0.35,

```

```

27         'svg.hashsalt': 'fvm-compact-20260913'})
28 INK = '#24282B'
29 EDGE = '#30383D'
30 GRID = '#A4AAAF'
31 FILL = '#CDD1D4'
32 HATCH = '#8A9399'
33
34 def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()
35 def dump(p, x): Path(p).write_text(json.dumps(x,ensure_ascii=False,indent=2)+'\n')
36
37 def prepare_workflow():
38     source = Path('./assets/codex-clipboard-f5f1a1d0-0249-4493-a572-647492a33321.png')
39     target = ROOT / 'fig-user-flow.png'
40     shutil.copy2(source, target)
41     # Preserve every supplied pixel. Alpha composites naturally onto white paper.
42     im = Image.open(target)
43     doc = fitz.open()
44     width = 160/25.4*72
45     page = doc.new_page(width=width, height=width*im.height/im.width)
46     page.insert_image(page.rect, filename=str(target))
47     doc.save(ROOT/'fig-user-flow.pdf', deflate=True)
48     page.get_pixmap(matrix=fitz.Matrix(3,3)).save(ROOT/'fig-user-flow-paper-preview.png')
49     dump(ROOT/'fig-user-flow.receipt.json', {'source':str(source),'sha256':sha(source),
50         'copied_png_sha256':sha(target),'original_pixels_preserved':True,
51         'pdf_representation':'original PNG with original alpha, placed at 160 mm width on white
52         paper',
53         'source_dimensions_px':list(im.size),'effective_dpi':im.width/(160/25.4),
54         'native_text_editable':False,'text_and_edges':'user-supplied; not regenerated'})
55
56 def fvm():
57     fid = 'fig-fvm2d-compact'
58     for name in ['frozen_empirical_engine.py', 'q23-2d-main.json']:
59         (ROOT/'source-data').mkdir(exist_ok=True)
60         shutil.copy2(OLD/'source-data'/name, ROOT/'source-data'/name)
61     config = json.loads((ROOT/'source-data'/q23-2d-main.json').read_text()['config'])
62     assert (config['nr'],config['nz'],config['radial_power'],config['axial_power']) ==
63         (128,64,2.,3.)
64     # Fewer displayed intervals keep boundary nodes distinguishable at print size.
65     nr,nz = 5,4
66     r = 1-(1-np.linspace(0,1,nr+1))**2
67     z = 1-(1-np.linspace(0,1,nz+1))**3
68     rf = np.r_[0,(r[1:]+r[:-1])/2,1]
69     zf = np.r_[0,(z[1:]+z[:-1])/2,1]
70     az = np.pi*np.diff(rf**2)
71     volume = az[:,None]*np.diff(zf)[None,:]
```

```

72 claim = '节点控制体通过共享面交换通量；轴对称旋转产生径向圆柱侧面积、
    轴向环面面积和环形控制体体积。'
73 request = {'schema_version':'1.0.0','figure_id':fid,'claim':claim,
74             'audience':'数学建模论文读者','medium':'A4 paper','final_width_mm':160,
75             'final_height_mm':75,'language':'zh-CN','variables':[], 'uncertainty':None,
76             'data_sources': [{'path':f'source-data/{n}','sha256':sha(ROOT/'source-data'/n)}
77                             for n in ['frozen_empirical_engine.py','q23-2d-main.json','fvm-construction.npz']],
78             'editable_format':'svg','claim_locked':True}
79 ir = {'schema_version':'1.0.0','figure_id':fid,'claim':claim,'route':'quantitative_code',
80       'recipe':'native axisymmetric finite-volume construction',
81       'panels': [{'panel_id':'a','purpose':'nodes, interior and truncated boundary control
    volumes'},
82                  {'panel_id':'b','purpose':'radial shared-face flux'},
83                  {'panel_id':'c','purpose':'full-azimuth ring-cell face areas and volume'}],
84       'encodings': [{'field':'selected control volume','channel':'gray
    hatching','scale':'categorical'},
85                      {'field':'shared physical face','channel':'heavy line','scale':'categorical'}],
86       'annotations': [{'target':'diagram scale','text':'5x4 illustrative intervals; 128x64
    calculation intervals; not physical aspect ratio'}],
87       'style_tokens': {'palette': {'ink': INK, 'boundary': EDGE, 'grid': GRID, 'selected': FILL},
88                          'base_font_pt': 8.5, 'font_faces': FONT_RECEIPT['faces'],
89                          'outer_boundary_pt': 1.5, 'shared_face_pt': 2., 'grid_pt': .65},
90       'exports': ['svg', 'pdf', 'png'],
91       'invariants': ['data', 'units', 'claim', 'transform', 'boundary nodes and truncated cells',
92                      'one shared flux with opposite signs', 'full-azimuth areas', 'no PDE field
    invented'],
93       'provenance': {'source_sha256': sha(__file__), 'renderer': 'Matplotlib native vectors',
94                      'illustrative_grid': [nr, nz], 'actual_grid': [128, 64], 'radial_power': 2., 'axial_power': 3.,
95                      'shading': 'categorical identification of a control volume; not a material property or
    scalar field',
96                      'geometry': 'meridional and ring-cell constructions are schematically scaled',
97                      'library_lookup': 'svg_composition -> svg-utils reference; reuse native renderer for
    exact mathematical marks'}}
98 dump(ROOT/f'{fid}.request.json', request)
99 dump(ROOT/f'{fid}.ir.json', ir)
100 dump(ROOT/f'{fid}.profile.json', {'kind': 'explanatory construction', 'observed_rows': None,
101                                   'geometric_identity_volume_R1_H1': float(volume.sum()), 'expected': float(np.pi),
102                                   'all_boundary_nodes_present': True, 'boundary_faces': [0., 1., 0., 1.],
103                                   'changed_pde_parameters': False})
104 with mpl.rc_context(RC):
105     fig = plt.figure(figsize=(160/25.4, 75/25.4))
106     ax = fig.add_axes([0, 0, 1, 1]); ax.set(xlim=(0, 160), ylim=(0, 75)); ax.axis('off')
107     def txt(x, y, t, size=8.5, ha='center', va='center', **kw):
108         return ax.text(x, y, t, fontsize=size, ha=ha, va=va, color=INK, **kw)
109     def line(x, y, lw=.65, color=GRID, ls='-', zorder=2):
110         return ax.plot(x, y, lw=lw, color=color, ls=ls, zorder=zorder)[0]
111     def arrow(a, b, lw=1.05, style='->', scale=8, color=EDGE):

```

```

112         ax.add_patch(FancyArrowPatch(a,b,arrowstyle=style,mutation_scale=scale,lw=lw,color=co,
            lor,zorder=6))
113 def selected(x,y,w,h,lw=1.25):
114     ax.add_patch(Rectangle((x,y),w,h,facecolor=FILL,edgecolor=HATCH,hatch='////',lw=0,zor,
            der=2))
115     ax.add_patch(Rectangle((x,y),w,h,fill=False,ec=EDGE,lw=lw,zorder=3))
116     txt(28,71,'(a) 节点控制体',8.8)
117     txt(84,71,'(b) 径向共享面',8.8)
118     txt(135,71,'(c) 环形控制体',8.8)
119     # (a) No decorative container: real boundaries carry the visual weight.
120     x0,y0,w,h=10.,20.,40.,40.
121     X,Y=x0+w*r,y0+h*z; XF,YF=x0+w*rf,y0+h*zf
122     for xx in XF: line([xx,xx],[y0,y0+h])
123     for yy in YF: line([x0,x0+w],[yy,yy])
124     i,j=2,1
125     selected(XF[i],YF[j],XF[i+1]-XF[i],YF[j+1]-YF[j])
126     selected(XF[-2],YF[j],XF[-1]-XF[-2],YF[j+1]-YF[j],lw=1.)
127     xx,yy=np.meshgrid(X,Y,indexing='ij')
128     ax.scatter(xx,yy,s=2.1,c=INK,zorder=5)
129     ax.scatter([X[i]],[Y[j]],s=10,c=INK,zorder=6)
130     line([x0,x0],[y0,y0+h],1.25,EDGE,'--',4)
131     line([x0,x0+w],[y0,y0],1.25,EDGE,'--',4)
132     line([x0,x0+w],[y0+h,y0+h],1.5,EDGE,zorder=4)
133     line([x0+w,x0+w],[y0,y0+h],1.5,EDGE,zorder=4)
134     for xx in [21,34,45]:arrow((xx,60),(xx,63.8),scale=6)
135     for yy in [28,42,53]:arrow((50,yy),(54.7,yy),scale=6)
136     txt(30,65.8,'端面交换',8.2)
137     txt(57.2,41,'侧面交换',8.2,rotation=90)
138     txt(2.2,39,'轴线对称',8.2,rotation=90)
139     txt(30,14.1,'中面对称',8.2)
140     arrow((10,17.5),(51,17.5),lw=.8,scale=6)
141     arrow((6.5,20),(6.5,62),lw=.8,scale=6)
142     txt(6.5,64,'$z$',8.5)
143     txt(53,17.5,'$r$',8.5)
144     txt(7.5,18.5,'$0$',8,ha='right')
145     txt(52.4,21.5,'$R$',8,ha='left')
146     txt(5.9,59.5,'$L/2$',8,ha='right')
147     ax.scatter([10.5],[9],s=8,c=INK)
148     txt(13,9,'节点未知量',8,ha='left')
149     line([36,40],[9,9],1.2,EDGE)
150     txt(42,9,'控制体',8,ha='left')
151     # (b) One radial flux evaluated once, used with opposite signs.
152     a,b,c,yd,yt=65.,84.,104.,42.,60.
153     selected(a,yd,b-a,yt-yd)
154     ax.add_patch(Rectangle((b,yd),c-b,yt-yd,facecolor='white',edgecolor=EDGE,lw=1.25,zorder=2))
155     line([b,b],[yd,yt],2.,EDGE,zorder=5)
156     ax.scatter([74.5,94],[51,51],s=12,c=INK,zorder=6)
157     txt(74.5,46.5,'$(i,j)$',8.5)

```

```

158     txt(94,46.5,'$(i+1,j)$',8.5)
159     arrow((76.5,55),(93,55),lw=1.2,scale=8)
160     txt(84,65.8,'同一共享面通量',8.2)
161     line([84,84],[64,60],.8,EDGE)
162     txt(65,39.3,'$r_{i-1/2}$',8)
163     txt(84,39.3,'$r_{i+1/2}$',8)
164     txt(104.8,59.5,'$z_{j+1/2}$',8,ha='left')
165     txt(104.8,42.5,'$z_{j-1/2}$',8,ha='left')
166     txt(84,31.4,'左侧流出, 右侧流入',8.2)
167     txt(84,25.8,'相邻单元收支符号相反',8.2)
168     # (c) Rotation of the selected meridional cell: annular prism.
169     cx,cy,rx,ry,dz=134.,56.,10.,3.7,10.
170     ratio=rf[i]/rf[i+1]
171     tt=np.linspace(np.pi,2*np.pi,120)
172     top=np.c_[cx+rx*np.cos(tt),cy+ry*np.sin(tt)]
173     bottom=np.c_[cx+rx*np.cos(tt[:-1]),cy-dz+ry*np.sin(tt[:-1])]
174     ax.add_patch(Polygon(np.vstack([top,bottom]),facecolor='#BCC2C6',edgecolor=EDGE,lw=1.3,zorder=2))
175     # Native hatch is clipped to the front cylindrical wall.
176     wall=Polygon(np.vstack([top,bottom]),fill=False,ec=HATCH,hatch='////',lw=0,zorder=2.1)
177     ax.add_patch(wall)
178     ax.add_patch(Ellipse((cx,cy),2*rx,2*ry,facecolor=FILL,ec=EDGE,lw=1.3,zorder=3))
179     ax.add_patch(Ellipse((cx,cy),2*rx*ratio,2*ry*ratio,facecolor='white',ec=EDGE,lw=1.1,zorder=4))
180     txt(134,64.5,'绕轴旋转',8.2)
181     arrow((121,46),(121,56),lw=.8,style='<->',scale=6)
182     txt(118.8,51,r'$\Delta z_j$',8,ha='right')
183     line([142.5,147],[57,60],.8,EDGE)
184     txt(148,60,'$A^z$',8.5,ha='left')
185     line([142,147],[47,46],.8,EDGE)
186     txt(148,46,'$A^r$',8.5,ha='left')
187     txt(111,32,r'$A^r_{i\pm 1/2,j}=2\pi r_{i\pm 1/2}\backslash,\Delta z_j$',8.7,ha='left')
188     txt(111,23,r'$A^z_{i,j\pm 1/2}=\pi(r_{i+1/2}^2-r_{i-1/2}^2)$',8.7,ha='left')
189     txt(111,14,r'$\Delta V_{ij}=A^z_{i,j\pm 1/2}\backslash,\Delta z_j$',8.7,ha='left')
190     txt(3,3.5,'示意网格 5x4; 计算网格 128x64. 各构造均未按实物长径比例绘制。',7.5,ha='left')
191     fig.canvas.draw()
192     dump(ROOT/f'{fid}.typography.json',typography.inspect_figure_typography(fig,final_width_mm=160))
193     export_figure(fig,ROOT/fid,formats=['svg','pdf','png'],dpi=300,bbox_inches=None,mkdir=True,overwrite=True,
194                 write_manifest=True,provenance=ir['provenance'],accessibility={'title':claim,
195                 'desc': '实线为外部交换边界, 虚线为对称边界; 灰色排线标出控制体, 不表示温湿数值。'})
196     plt.close(fig)
197
198 if __name__=='__main__':
199     prepare_workflow()
200     fvm()

```

F.62 build_merged.py

类别：数据图形与示意图。原始代码共 127 行。

```
1 """Recompose existing numerical evidence; no PDE solve or parameter change."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import os
6 import sys
7
8 ROOT = Path(__file__).resolve().parent
9 os.environ.setdefault('MPLCONFIGDIR', str(ROOT / '.mpl-cache'))
10 os.environ.setdefault('XDG_CACHE_HOME', str(ROOT / '.cache'))
11 SUPPORT = ROOT / 'support'
12 sys.path[:0] = [str(SUPPORT / 'lab-ultra-scientific-figure-maker/scripts'),
13                 str(SUPPORT / 'lab-ultra-scientific-visualization/scripts')]
14 from figure_library import publication_style
15 from figure_typography import inspect_figure_typography
16 from figure_export import export_figure
17 import numpy as np
18 import matplotlib
19 matplotlib.use('Agg')
20 import matplotlib.pyplot as plt
21 from matplotlib.colors import LinearSegmentedColormap, Normalize
22
23 sha = lambda p: hashlib.sha256(Path(p).read_bytes()).hexdigest()
24 def dump(p, value):
25     Path(p).write_text(json.dumps(value, ensure_ascii=False, indent=2, default=str,
26                                   allow_nan=False) + '\n')
27
28 DATA = ROOT / 'data/fig-evolution.data.npz'
29 with np.load(DATA) as z:
30     d = {k: z[k] for k in z.files}
31     assert d['T_C'].shape == d['C'].shape == (129, 10801)
32     assert all(np.all(np.isfinite(x)) for x in d.values())
33     assert np.all(np.diff(d['time_h']) > 0) and np.all(np.diff(d['r_cm']) > 0)
34     ti = np.arange(0, 10801, 1800)
35     t = d['time_h'][ti]
36     delta_T = d['T_C'][-1, ti] - d['T_C'][0, ti]
37     delta_C = d['C'][0, ti] - d['C'][-1, ti]
38     assert np.array_equal(t, np.arange(7) * .5)
39     assert np.all(delta_T >= 0) and np.all(delta_C >= 0)
40     np.savetxt(ROOT / 'data/difference-points.csv', np.c_[t, delta_T, delta_C], delimiter=',',
41               header='time_h,surface_minus_center_T_C,center_minus_surface_C_kgkg', comments='',
42               fmt='%17g')
```

```

43 CLAIM = '问题二同一数值场在前三小时逐渐升温而仍保持水分梯度；差值曲线量化同一空间分布，
    不能把温度接近均匀视为全域烘干达标。'
44 TRANSFORMS = [
45     'Original immutable 10801 one-second fields on 129 nonuniform radial nodes; no fit, smoothing
        or PDE rerun.',
46     'Heatmaps use node-centered faces at neighboring-node midpoints, with physical radial
        coordinates in cm.',
47     'Difference curves select only times 0,.5,1,1.5,2,2.5,3 h from the same fields; lines join
        these seven points.',
48     'Temperature difference is surface minus center; moisture difference is center minus surface.',
49     'Heatmap data marks are rasterized at 600 dpi; text, lines, ticks and colorbar labels remain
        vector.',
50     'Temperature color limits 28..50 degC and moisture 0..2.55 kg/kg match the original figure;
        independent units are not normalized together.',
51 ]
52 req = {'schema_version': '1.0.0', 'figure_id': FID, 'claim': CLAIM,
53        'audience': '中文数学建模论文读者', 'medium': 'A4 paper; no external journal submission',
54        'final_width_mm': 160, 'final_height_mm': 105, 'language': 'zh-CN',
55        'data_sources': [{'path': 'data/fig-evolution.data.npz', 'sha256': sha(DATA)}],
56        'variables': [{'name': k, 'unit': unit, 'role': role} for k, unit, role in [
57            ('time_h', 'h', 'time'), ('r_cm', 'cm', 'radius'), ('T_C', 'degC', 'field'),
58            ('C', 'kg/kg dry matter', 'field'), ('delta_T', 'degC', 'difference'),
59            ('delta_C', 'kg/kg dry matter', 'difference')]],
60        'uncertainty': {'kind': 'none; deterministic saved outputs, not measured uncertainty'},
61        'editable_format': 'svg', 'claim_locked': True}
62 profile = {'fields': {k: {'shape': list(x.shape), 'dtype': str(x.dtype), 'missing':
        int(np.isnan(x).sum())},
63              'min': float(x.min()), 'max': float(x.max())} for k, x in d.items()),
64            'difference_points': 7, 'same_case_and_same_saved_fields': True,
65            'transformations': TRANSFORMS, 'computed_not_observed': True}
66 BLUE = '#244F73'
67 CMAP = LinearSegmentedColormap.from_list('paper_blue',
        ['#F4F7F9', '#D4E0E9', '#9AB8CF', '#587F9F', BLUE, '#12354F'])
68 style = {'font.size': 8.5, 'axes.labelsize': 8.5, 'axes.titlesize': 8.5,
69          'xtick.labelsize': 8, 'ytick.labelsize': 8, 'legend.fontsize': 8,
70          'axes.spines.top': False, 'axes.spines.right': False, 'axes.linewidth': .65,
71          'lines.linewidth': 1.2, 'grid.color': '#D9DEE3', 'grid.linewidth': .5,
72          'grid.alpha': .6, 'svg.fonttype': 'none', 'pdf.fonttype': 42,
73          'figure.constrained_layout.h_pad': .055, 'figure.constrained_layout.w_pad': .055}
74 faces = lambda x: np.r_[x[0], (x[:-1] + x[1:]) / 2, x[-1]]
75 with publication_style('nature', language='zh-CN', width_mm=160, height_mm=105, overrides=style)
    as materials:
76     ir = {'schema_version': '1.0.0', 'figure_id': FID, 'claim': CLAIM, 'route':
        'quantitative_code',
77          'recipe': 'existing Matplotlib heatmap and point-line recipes combined in a 2x2
        constrained layout',
78          'panels': [{'panel_id': k, 'purpose': purpose} for k, purpose in [

```

```

79         ('a', 'temperature distribution'), ('b', 'moisture distribution'),
80         ('c', 'surface minus center temperature'), ('d', 'center minus surface moisture')]],
81     'encodings': [{'field': k, 'channel': channel, 'scale': 'linear', 'source_layer':
'deterministic'}
82         for k, channel in [ ('time_h', 'x'), ('r_cm', 'y'), ('T_C', 'color'), ('C', 'color'),
            ('delta_T', 'y'), ('delta_C', 'y') ]],
83     'annotations': [{'target': '3 h last difference point', 'text': '0.1106 degC / 0.7815
kg/kg'}],
84     'style_tokens': {'palette': 'same sequential blue as preceding figure', 'base_font_pt':
8.5,
85         'layout_owner': 'matplotlib constrained layout'},
86     'exports': ['svg', 'pdf', 'png'], 'invariants':
['data', 'units', 'claim', 'transform', 'uncertainty'],
87     'provenance': {'renderer': 'matplotlib', 'version': matplotlib.__version__, 'materials':
materials,
88         'script_sha256': sha(__file__), 'data_sha256': sha(DATA),
            'transformations': TRANSFORMS,
89         'recipe_lookup': 'lab-ultra figure_library heatmap catalog and existing
frozen-data field/line implementations'}}
90 dump(ROOT / f'{FID}.request.json', req)
91 dump(ROOT / f'{FID}.profile.json', profile)
92 dump(ROOT / f'{FID}.ir.json', ir)
93 fig, axes = plt.subplots(2, 2, layout='constrained')
94 for ax, key, title, limits, label, ticks in [
95     (axes[0,0], 'T_C', '(a) 温度场', (28,50), 'T / °C', [28,34,40,46,50]),
96     (axes[0,1], 'C', '(b) 含水率场', (0,2.55), 'C / (kg/kg)', [0,.5,1,1.5,2,2.55]):
97     im = ax.pcolormesh(faces(d['time_h']), faces(d['r_cm']), d[key], cmap=CMAP,
98         norm=Normalize(*limits), shading='flat', rasterized=True)
99     ax.set(xlim=(0,3), ylim=(0,2), title=title, ylabel='半径 / cm')
100     ax.set_yticks([0,1,2])
101     cb = fig.colorbar(im, ax=ax, pad=.025, fraction=.035, aspect=18, ticks=ticks)
102     cb.set_label(label, fontsize=8)
103     cb.ax.tick_params(labelsize=7.5, length=2.5, width=.6)
104 for ax, values, title, ylabel in [
105     (axes[1,0], delta_T, '(c) 表面—中心温差', '温差 / °C'),
106     (axes[1,1], delta_C, '(d) 中心—表面含水率差', '含水率差 / (kg/kg)')]:
107     ax.plot(t, values, color=BLUE, marker='o', markersize=3)
108     ax.set(xlim=(0,3), ylim=(0,float(values.max())*1.16), title=title, ylabel=ylabel,
109         xlabel='时间 / h')
110     ax.grid(axis='y')
111 for ax in axes.flat:
112     ax.set_xticks([0,1,2,3])
113 axes[1,0].annotate(f'{delta_T[-1]:.4f} °C', xy=(3,delta_T[-1]), xytext=(2.0,1.3), fontsize=8,
114     arrowprops={'arrowstyle':'-', 'lw':.6, 'color':'#777777'})
115 axes[1,1].annotate(f'{delta_C[-1]:.4f} kg/kg', xy=(3,delta_C[-1]), xytext=(.65,.4),
116     arrowprops={'arrowstyle':'-', 'lw':.6, 'color':'#777777'})

```

```

116     fig.canvas.draw()
117     dump(ROOT / f'{FID}.typography.json', inspect_figure_typography(fig, final_width_mm=160))
118     export_figure(fig, ROOT / FID, formats=['svg', 'pdf', 'png'], dpi=600, bbox_inches=None,
119                 mkdir=True, overwrite=True, write_manifest=True, provenance=ir['provenance'],
120                 accessibility={'title': CLAIM, 'desc': '上排为同一模型温湿场，
                    下排为七个半小时时间间隔的端点差值；每列物理量相同，单位和色标独立。
                    全部来自自己保存的数值结果。'})
121     plt.close(fig)
122     dump(ROOT / 'merged-data-check.json', {'data_sha256':sha(DATA), 'time_h':t.tolist(),
123         'delta_T':delta_T.tolist(), 'delta_C':delta_C.tolist(),
124         'temperature_last_display':f'{delta_T[-1]:.4f}',
125         'moisture_last_display':f'{delta_C[-1]:.4f}',
126         'new_pde_runs':0, 'same_source_as_old_figures':True})
127     print(json.dumps({'figure':str(ROOT/f'{FID}.pdf'), 'width_mm':160, 'height_mm':105,
128         'delta_T_3h':float(delta_T[-1]),
129         'delta_C_3h':float(delta_C[-1])},ensure_ascii=False))

```

F.63 build_figures.py

类别：数据图形与示意图。原始代码共 259 行。

```

1  """Frozen-array paper figures. No PDE solve, parameter fitting, or model selection.
2
3  Run without arguments to extract current audited arrays then render. Use
4  --render-only to reproduce from the attached full-precision figure-data bundles.
5  """
6  from pathlib import Path
7  import os,sys,json,hashlib,argparse
8  ROOT=Path(__file__).resolve().parents[1]
9  os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'evidence/.mpl-cache'))
10 os.environ.setdefault('XDG_CACHE_HOME',str(ROOT/'evidence/.cache'))
11 SUPPORT=ROOT/'src/support'
12 sys.path[:0]=[str(SUPPORT/'lab-ultra-scientific-figure-maker/scripts'),str(SUPPORT/'lab-ultra-sc|
    ientific-visualization/scripts')]
13 from figure_library import publication_style
14 from figure_typography import inspect_figure_typography
15 from figure_export import export_figure
16 import numpy as np
17 from openpyxl import load_workbook
18 import matplotlib
19 matplotlib.use('Agg')
20 import matplotlib.pyplot as plt
21 from matplotlib.colors import LinearSegmentedColormap,Normalize
22 from matplotlib.ticker import MaxNLocator
23
24 MODEL=ROOT.parents[2]/'model-system-20260912'
25 BLUE='#244F73';GRAY='#777777';TEXT='#222222';LIGHT='#D9DEE3'
26 BLUESEQ=LinearSegmentedColormap.from_list('paper_blue', ['#F4F7F9', '#D4E0E9', '#9AB8CF', '#587F9F', ,
    BLUE, '#12354F'])

```

```

27 STYLE={'font.size':8.5,'axes.labelsize':8.5,'axes.titlesize':8.5,
28         'xtick.labelsize':8,'ytick.labelsize':8,'legend.fontsize':7.8,
29         'axes.spines.top':False,'axes.spines.right':False,'axes.linewidth':.65,
30         'lines.linewidth':1.2,'grid.color':LIGHT,'grid.linewidth':.5,'grid.alpha':.6,
31         'text.color':TEXT,'axes.labelcolor':TEXT,'xtick.color':TEXT,'ytick.color':TEXT,
32         'svg.fonttype':'none','pdf.fonttype':42,'figure.constrained_layout.h_pad':.05,
33         'figure.constrained_layout.w_pad':.05}
34 PLANS={
35     'fig-environment':{
36         'height_mm':60,'claim':'附件1给出0—4 h随时间变化的烘房温度和空气水分指标；
前期边界使用测量点间线性插值，不能以恒定环境替代。',
37         'panels':[{ 'panel_id':'a','purpose':'实测温度与线性插值'},{ 'panel_id':'b','purpose':'实测空气，
水分指标与线性插值'}],
38         'variables':[{ 'name':'time_h','unit':'h','role':'independent'},{ 'name':'T_air_C','unit':'degC',
', 'role':'measured forcing'},{ 'name':'w_air','unit':'kg/kg','role':'measured air moisture
indicator'}],
39         'transforms':['时间s除以3600；实测241点全部显示；连线是相邻观测点之间的线性插值；只画观测0—4
h，不画未测延拓。'],
40         'caption':'附件1中0—4 h的烘房环境。圆点为全部实测记录，实线为模型使用的逐段线性插值；
两列的纵轴量纲独立。4 h后的末一小时均值延拓属于另行声明的模型假设，不是图中实测数据。'],
41     'fig-evolution':{
42         'height_mm':90,'claim':'附录3的一维端面折算解在前3 h形成径向温湿梯度；温度较快接近环境，
而水分梯度持续存在，两种状态不可视作同步均匀化。',
43         'panels':[{ 'panel_id':'a','purpose':'0—3
h的径向温度时空分布'},{ 'panel_id':'b','purpose':'同一时空域的干基含水率分布'}],
44         'variables':[{ 'name':'time_h','unit':'h','role':'x'},{ 'name':'r_cm','unit':'cm','role':'y'},{
'name':'T_C','unit':'degC','role':'field'},{ 'name':'C','unit':'kg/kg dry
matter','role':'field'}],
45         'transforms':['读取Q23一维全部0—10800 s逐秒保存场；不抽样、不重新拟合；
控制体积色块按真实非均匀径向面位置绘制；温度色标28—50°C，含水率色标0—2.55，
颜色方案相同但数值归一化独立。','稠密定量色块以600 dpi栅格化进入SVG/PDF，文字、
坐标及色条文字保留矢量；栅格层是代码绘制的数值场而非生成图像。'],
46         'caption':'问题2的一维端面折算模型在0—3 h的温度与含水率时空分布。
绘图使用原始逐秒保存场及非均匀径向控制体积；两个面板采用相同深蓝单色序列、
各自独立且带单位的色标。不同色标不能作跨变量数值比较。'],
47     'fig-dimensional':{
48         'height_mm':100,'claim':'第四问6 h的二维场存在端面与侧面共同造成的空间差异；总达标时间仅差约11
min，不意味着一维折算和二维局部场完全相同。',
49         'panels':[{ 'panel_id':'a','purpose':'6 h的实际半长物理域含水率'},{ 'panel_id':'b','purpose':'
同一保存时刻与同一径向节点的一维减二维中面和轴向平均差'}],
50         'variables':[{ 'name':'z_cm','unit':'cm','role':'x
position'},{ 'name':'r_cm','unit':'cm','role':'radial position'},{ 'name':'C','unit':'kg/kg dry
matter','role':'field'},{ 'name':'C_1D-C_2D','unit':'kg/kg dry matter','role':'signed
difference'}],
51         'transforms':['严格选择双方已有的21600 s保存时刻；2D控制体积按此时R(t)、L(t)映射为实际cm，
横纵比例相同。','差值在两解相同的129个径向计算节点直接相减；二维轴向平均由原保存的体积权重得到，
不引入时间插值、拟合或新的平衡参数。'],

```

```

52     'caption': '第四问等比例收缩模型在6 h的空间对照。(a) 中面至一端、中心轴至侧面的实际物理域，
    横纵坐标使用相同比例；色块为二维含水率。(b) 在相同保存时刻、
    相同径向节点计算的一维解减二维中面值、以及一维解减二维轴向平均值。两条差值的含义不同：
    前者检验模板剖面近似，后者检验折算模型的平均意义。',
53     'fig-geometry-control': {
54         'height_mm': 85, 'claim': '保持附录4及其余条件相同时，固定尺寸二维对照在72 h仍未达标，
    等比例收缩解约51.09 h达标；两者按题给密度积分的干质量均未保持初值，几何收益不等于材料质量闭合。
    ',
55         'panels': [{ 'panel_id': 'a', 'purpose': '同附录4固定/等比例几何的Cmax轨迹与72
    h删失' }, { 'panel_id': 'b', 'purpose': '同两组场的真实密度解释下干质量漂移诊断' } ],
56         'variables': [{ 'name': 'time_h', 'unit': 'h', 'role': 'x' }, { 'name': 'Cmax', 'unit': 'kg/kg dry
    matter', 'role': 'criterion' }, { 'name': 'Md/Md0', 'unit': '1', 'role': 'diagnostic' } ],
57         'transforms': [ '只比较相同附录4、相同网格/边界/初值的两个算例；用各自实际保存时段，
    不延长已停止的收缩场曲线。', 'Md(t)=V(t) <rho(C)/(1+C)>，以各自初始Md归一化；
    这是未参与经验求解的质量诊断。', '固定对照72 h尚未达标，右端标记不是达标事件；没有预测72
    h之后的终止时间。' ],
58     'caption': '保持附录4物性、初值、二维网格和交换条件不变，仅改变几何的消融对照。(a)
    等比例收缩模型在51.09 h附近达标，固定尺寸模型在72 h仍有最大含水率0.2247；
    固定尺寸曲线右端表示观测计算窗口结束，不能视为达标时间。(b)
    按题给p(C)和当前几何积分的干质量与初值之比。偏离1表明该经验方程组与这些质量—
    几何解释不能自动实现严格干质量守恒。',
59     'fig-length-convergence': {
60         'height_mm': 100, 'claim': '质量—体积约束模型产生先伸长后回落的未实测长度，
    并可获得收敛的条件数值解；失水余额小和网格收敛不能证明该长度轨迹是真实材料行为。',
61         'panels': [{ 'panel_id': 'a', 'purpose': '等比例几何与Map
    A约束得到的长度历史' }, { 'panel_id': 'b', 'purpose': 'Map A数值失水余额而非再次重复经验干质量漂移' },
    { 'panel_id': 'c', 'purpose': '相同容差下12/24/32网格的达标时间' } ],
62         'variables': [{ 'name': 'time_h', 'unit': 'h', 'role': 'x' }, { 'name': 'length_cm', 'unit': 'cm', 'role': '
    model geometry' }, { 'name': 'relative_water_balance_residual', 'unit': '1', 'role': 'numerical
    residual' }, { 'name': 'mesh', 'unit': 'radial by axial elements', 'role': 'ordered numerical
    refinement' }, { 'name': 'critical_h', 'unit': 'h', 'role': 'numerical response' } ],
63         'transforms': [ 'Map A长度直接读取32×16保存轨迹；等比例长度只由附件2已知R(t)按L0R/R0计算到72 h，
    属于几何约束曲线，不外推含水率。', '失水余额取已有budget_steps的mass_residual除以初始C0；
    它检验水分账，不作为实测模型准确度。', '空间收敛只用12×6-tight、24×12、32×16，
    三组配置除nr/nz外完全一致；不混入更宽时间容差的12×6。' ],
64     'caption': '变长度条件模型的几何响应与数值核验。(a) Map A给出的长度先增至约31.93 cm后回落；
    等比例曲线由同一附件半径直接规定。两条长度均不是轴向实测。(b) Map A相对失水余额残差，
    不能替代材料验证。(c) 使用相同时间容差和最大步长的12×6、24×12、32×16网格，达标时间逐渐稳定。
    与等比例经验模型的差异同时包含守恒运算符、水显焓和运动学变化，不能单独归因于长度放开。',
65 }
66
67 def sha(p): return hashlib.sha256(Path(p).read_bytes()).hexdigest()
68 def dump(path, value): Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_na_
    n=False, default=str))
69 def read_sheet(path):
70     w = load_workbook(path, read_only=True, data_only=True); a = np.asarray(list(w.active.values)[1:], fl_
    oat); w.close(); return a

```

```

71 def read_npz(path):
72     with np.load(path) as z: return {k:z[k] for k in z.files}
73 def idx(a,t):
74     i=np.flatnonzero(np.isclose(a,t,atol=1e-8,rtol=0));assert len(i)==1,(t,len(i));return
        int(i[0])
75 def faces(nodes):return np.r_[nodes[0],(nodes[:-1]+nodes[1:])/2,nodes[-1]]
76
77 def extract(source):
78     sources={}
79     def bind(rel):
80         p=source/rel;sources[rel]={'path':rel,'absolute_path':str(p.resolve()),'sha256':sha(p),'b_
            ytes':p.stat().st_size};return p
81     def na(rel):return read_npz(bind(rel))
82     def js(rel):return json.loads(bind(rel).read_text())
83     env=read_sheet(bind('baseline/input/附件1.xlsx'));radius=read_sheet(bind('baseline/input/附件_
        2.xlsx'))
84     summ=js('baseline/numerical-summary.json')
85     q1=na('baseline/results/q1-1d-main.npz');q23=na('baseline/results/q23-1d-main.npz')
86     q41=na('baseline/results/q4-1d-main.npz');q42=na('baseline/results/q4-2d-main.npz')
87     fix=na('baseline/results/q4-2d-fixed-control.npz')
88     varroot='variable-length/results/'
89     names=['q4-robin-latent0-12x6-tight','q4-robin-latent0-24x12','q4-robin-latent0-32x16']
90     vd=[js(varroot+n+'/summary.json') for n in names]
91     var=na(varroot+names[-1]+'fields.npz')
92     for a in vd[:-1]:assert {k:v for k,v in a['config'].items() if k not in ('nr','nz')}=={k:v for
        k,v in vd[-1]['config'].items() if k not in ('nr','nz')}
93     def representative(a,ts):
94         out=[]
95         for t in ts:
96             i=idx(a['times_s'],t);out.append({'time_s':float(t),'time_h':t/3600,
97                 'center_T_C':float(a['temperature_mid_C'][i,0]),'surface_T_C':float(a['surface_mi_
                    d_TC'][i,0]),
98                 'center_C':float(a['moisture_mid'][i,0]),'surface_C':float(a['surface_mid_TC'][i,1]),
99                 'Cmean':float(a['Cmean'][i]),'Cmax':float(a['Cmax'][i])})
100     return out
101     i1=idx(q41['times_s'],21600);i2=idx(q42['times_s'],21600);si=idx(q42['snapshot_times_s'],2160
        0)
102     assert np.array_equal(q41['x'],q42['x']) and q41['R_m'][i1]==q42['R_m'][i2]
103     rcm=q42['R_m'][i2]*100*q42['x'];zcm=q42['L_m'][i2]*50*q42['eta']
104     ermid=q41['middle_field_TC'][i1,:,1]-q42['middle_field_TC'][i2,:,1]
105     ermean=q41['middle_field_TC'][i1,:,1]-q42['axialmean_field_TC'][i2,:,1]
106     dt=q23['times_s']<=10800
107     times=var['times'];records=var['records'];rcols=list(var['record_columns'])
108     budget=var['budget_steps'];bcols=list(var['budget_columns'])
109     lengths=records[:,rcols.index('length_m')];peak=int(np.argmax(lengths));stop=vd[-1]['strict_s_
        top_s']
110     residual=budget[:,bcols.index('mass_residual')]/vd[-1]['config']['C0']

```

```

111     dat={
112         'fig-environment':{'time_h':env[:,0]/3600,'T_air_C':env[:,1],'w_air':env[:,2]},
113         'fig-evolution':{'time_h':q23['times_s']/dt/3600,'r_cm':q23['x']*2,'T_C':q23['middle_field_
114             _TC']/dt[:,0].T,'C':q23['middle_field_TC']/dt[:,1].T},
115         'fig-dimensional':{'r_cm':rcm,'z_cm':zcm,'Cfield':q42['snapshot_fields_TC'][si[:,1],'erro
116             r_mid':ermid,'error_mean':ermean,'time_h':np.array(6.),'critical_difference_min':np.array(
117                 summ['dimensions'][2]['two_d_minus_one_d_time_minutes'])},
118         'fig-geometry-control':{'moving_time_h':q42['times_s']/3600,'moving_Cmax':q42['Cmax'],'movi
119             ng_Md_ratio':1+q42['density_dry_mass_relative_change'],
120         'fixed_time_h':fix['times_s']/3600,'fixed_Cmax':fix['Cmax'],'fixed_Md_ratio':1+fix['de
121             nsity_dry_mass_relative_change']},
122         'fig-length-convergence':{'map_time_h':times/3600,'map_length_cm':lengths*100,'aspect_time_
123             h':radius[:,0]/3600,'aspect_length_cm':25*radius[:,1]/2,
124         'balance_time_h':budget[:,bcols.index('time_s')]/3600,'mass_residual_relative':residua
125             l,'grid_nr':np.array([12,24,32]),'grid_nz':np.array([6,12,16]),'grid_critical_h':np.a
126             rray([d['critical_h'] for d in vd])}
127     }
128     deps={
129         'fig-environment':['baseline/input/附件 1.xlsx'],
130         'fig-evolution':['baseline/results/q23-1d-main.npz'],
131         'fig-dimensional':['baseline/results/q4-1d-main.npz','baseline/results/q4-2d-main.npz','bas
132             eline/numerical-summary.json'],
133         'fig-geometry-control':['baseline/results/q4-2d-main.npz','baseline/results/q4-2d-fixed-con
134             trol.npz','baseline/numerical-summary.json'],
135         'fig-length-convergence':['baseline/input/附件2.xlsx',varroot+names[-1]+'fields.npz']+var
136             root+n+'/summary.json' for n in names]
137     }
138     provenance={}
139     for fid,arrays in dat.items():
140         p=ROOT/'evidence'/f'{fid}.data.npz';np.savez_compressed(p,**arrays)
141         profile={'derived_fields':{},'paired':fid in ('fig-dimensional','fig-geometry-control'),
142             'transformations':PLANS[fid]['transforms'],'uncertainty':'deterministic conditional
143                 outputs; no statistical confidence interval'}
144         for key,a in arrays.items():
145             a=np.asarray(a);profile['derived_fields'][key]={'shape':list(a.shape),'dtype':str(a.d
146                 type),'missing':int(np.isnan(a).sum()),'min':float(np.nanmin(a)),'max':float(np.nanm
147                     ax(a))}
148         dump(ROOT/'evidence'/f'{fid}.profile.json',profile)
149         provenance[fid]={'data_file':str(p.relative_to(ROOT)),'data_sha256':sha(p),'sources':[sou
150             rces[k] for k in deps[fid]],'transformations':PLANS[fid]['transforms']}
151     metrics={'evidence_scope':'Computed values from the currently audited, frozen model system. No
152         fitting or new model choices. Empirical main models are not asserted to have true
153         dry-mass/full-energy closure.',
154         'source_root':str(source.resolve()),'all_input_files':list(sources.values()),
155         'q1_1d_representative':representative(q1,[100,300,600,900,1200,1500,1800]),
156         'q2_1d_representative':representative(q23,[1800,3600,5400,7200,9000,10800]),

```

```

140     'q3_q4_stopping':{k:{'critical_h':v['critical_h'],'strict_stop_s':v['strict_stop_s'],'stri
ct_stop_h':None if v['strict_stop_s'] is None else
v['strict_stop_s']/3600,'strict_stop_Cmax':v['strict_stop_Cmax']}} for k,v in
summ['main_cases'].items() if k.startswith(('q23','q4'))},
141     'dimensional_comparisons':summ['dimensions'],'baseline_grid_refinement':summ['grid_refinem
ent'],'baseline_time_refinement':summ['time_refinement'],
142     'q4_6h':{'R_cm':float(q42['R_m'][i2]*100),'L_cm':float(q42['L_m'][i2]*100),'Cmin':float(q4
2['Cmin'][i2]),'Cmax':float(q42['Cmax'][i2]),
143         'one_d_minus_midplane_max_abs_on_common_nodes':float(np.abs(ermid).max()),'one_d_minus
_axialmean_max_abs_on_common_nodes':float(np.abs(ermean).max()),
144         'comparison_time_s':21600,'time_interpolation_used':False,'radial_nodes_are_identical'
:True},
145     'fixed_geometry_control':summ['geometry_ablation'],
146     'density_mass_diagnostics':{k:{key:v[key] for key in ('density_dry_mass_initial_kg','densi
ty_dry_mass_max_abs_relative_change','density_dry_mass_final_relative_change')}} for k,v in
summ['main_cases'].items()},
147     'q4_Md_ratio_2d':{'min':float((1+q42['density_dry_mass_relative_change']).min()),'time_h_a
t_min':float(q42['times_s'][np.argmin(q42['density_dry_mass_relative_change'])]/3600),'at
_stop':float(1+q42['density_dry_mass_relative_change'][-1])},
148     'variable_length':{'case_id':names[-1],'config':vd[-1]['config'],'critical_h':vd[-1]['crit
ical_h'],'strict_stop_s':stop,'strict_stop_h':stop/3600,
149         'length_peak_cm':float(lengths[peak]*100),'length_peak_h':float(times[peak]/3600),'len
gth_at_stop_cm':float(vd[-1]['L_at_strict_stop_m']*100),
150         'length_72h_cm':float(lengths[-1]*100),'R_at_stop_cm':float(vd[-1]['R_at_strict_stop_m
']*100),
151         'relative_water_balance_residual_max':float(np.abs(residual).max()),'relative_enthalpy
_balance_residual_max':vd[-1]['enthalpy_balance_relative'],
152         'nodal_dry_mass_relative_error_max':vd[-1]['checks_max']['nodal_dry_mass_relative_erro
r'],
153         'density_interpolation_relative_error_max':vd[-1]['checks_max']['rho_mixture_relative_
error'],
154         'full_model_summary':vd[-1],
155         'spatial_refinement_common_tolerances':[{k:d[k] for k in
('critical_h','strict_stop_s','mass_balance_relative','enthalpy_balance_relative')}]},{
'case_id':n,'nr':d['config']['nr'],'nz':d['config']['nz'],'rtol':d['config']['rtol'],
'max_step_s':d['config']['max_step']} for n,d in zip(names,vd)},
156     'grid_24_to_32_change_h':vd[-1]['critical_h']-vd[-2]['critical_h'],'grid_24_to_32_chan
ge_min':60*(vd[-1]['critical_h']-vd[-2]['critical_h']),
157     'comparison_limit':'Changes versus affine empirical baseline include conservative
moisture operator, sensible water enthalpy and Map A kinematics. Not a length-only
effect; length is unobserved; no mechanical constitutive law identified.')}
158 dump(ROOT/'paper-metrics.json',metrics);dump(ROOT/'evidence/data-bindings.json',provenance)
159 dump(ROOT/'evidence/input-manifest.json',{'files':list(sources.values()),'source_root':str(so
urce.resolve()),'source_files_modified':False})
160
161 def contract(fid,material,bindings):
162     p=PLANS[fid];data=bindings[fid];claim=p['claim']

```

```

163     request={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'audience':'数学建模论文读者',
164             'medium':'A4 printed Chinese modeling paper',
165             'final_width_mm':160,'final_height_mm':p['height_mm'],'language':'zh-CN','data_sources':data[
166             'sources'],
167             'variables':p['variables'],'uncertainty':{'kind':'deterministic conditional calculations;
168             no confidence interval'},'editable_format':'svg','claim_locked':True}
169     ir={'schema_version':'1.0.0','figure_id':fid,'claim':claim,'route':'quantitative_code','recipe':
170     'native Matplotlib line/field composition; shared lab-ultra materials',
171     'panels':p['panels'],'encodings':[{ 'field':v['name'],'channel':('x' if v['role'] in
172     ('x','independent','x position') else 'color' if v['role']=='field' else
173     'y'),'scale':'linear','source_layer':'deterministic'} for v in p['variables']],
174     'annotations':[{ 'target':'supplied/calculated records','text':p['caption']}],
175     'style_tokens':{'palette':{'primary':BLUE,'secondary':GRAY,'sequential':'paper_blue,
176     monotone lightness'},'base_font_pt':8.5,'tick_font_pt':8,'layout_owner':'matplotlib
177     constrained layout'},
178     'exports':['svg','pdf','png'],'invariants':['data','units','claim','transform','uncertain
179     ty'],
180     'provenance':{'renderer':'matplotlib','matplotlib_version':matplotlib.__version__,'materi
181     als':material,'renderer_source_sha256':sha(__file__),
182     'source_files':data['sources'],'derived_data':{'path':data['data_file'],'sha256':data[
183     'data_sha256']},'transformations':data['transformations'],
184     'library_recipe_consulted':'lab-ultra-scientific-figure-maker/scripts/render_recipe.py
185     (line and heatmap); specialized field layout is native code'}}
186     dump(ROOT/f'{fid}.request.json',request);dump(ROOT/f'{fid}.ir.json',ir)
187
188     def finish(fig,fid,material,bindings):
189         fig.canvas.draw()
190         dump(ROOT/f'{fid}.typography.json',inspect_figure_typography(fig,final_width_mm=160))
191         export_figure(fig,ROOT/fid,formats=['svg','pdf','png'],dpi=600,bbox_inches=None,overwrite=True,
192         mkdir=True,
193         provenance={'materials':material,'data_binding':bindings[fid],'renderer_sha256':sha(__fil
194         e__)},write_manifest=True,
195         accessibility={'title':fid,'desc':PLANS[fid]['claim']})
196         plt.close(fig)
197
198     def panel(ax,title):
199         ax.set_title(title,loc='left',fontweight='normal',pad=6);ax.tick_params(length=3,width=.6);re
200         turn ax
201
202     def timeaxis(ax,limit):
203         ax.set_xlim(0,limit);ax.set_xlabel('时间 / h');ax.grid(axis='y');return ax
204
205     def render(fid,bindings):
206         data_path=ROOT/bindings[fid]['data_file'];assert sha(data_path)==bindings[fid]['data_sha256']
207         d=read_npz(data_path);p=PLANS[fid]
208         with publication_style('nature',language='zh-CN',width_mm=160,height_mm=p['height_mm'],overri
209         des=STYLE) as mat:
210             contract(fid,mat,bindings)

```

```

194     if fid=='fig-environment':
195         fig,axes=plt.subplots(1,2,layout='constrained')
196         for ax,key,title,ylabel in zip(axes,['T_air_C','w_air'],['(a) 烘房温度','(b)
            空气水分指标'],['温度 / °C','空气水分指标 / (kg/kg)']):
197             panel(ax,title);ax.plot(d['time_h'],d[key],color=BLUE,lw=1,label='线性插值',zorder=2)
198             ax.scatter(d['time_h'],d[key],color=GRAY,s=3,linewidths=0,label='实测点',zorder=3)
199             timeaxis(ax,4);ax.set_ylabel(ylabel);ax.set_xticks([0,1,2,3,4])
200             axes[0].set_ylim(25,52);axes[1].set_ylim(0,.055)
201             axes[0].legend(frameon=False,loc='lower right',markerscale=1.5)
202     elif fid=='fig-evolution':
203         fig,axes=plt.subplots(2,1,layout='constrained',sharex=True)
204         for ax,key,title,vmin,vmax,label,ticks in [
205             (axes[0],'T_C','(a) 温度: 先升温、再趋于均匀',28,50,'温度 / °C',[28,34,40,46,50]),
206             (axes[1],'C','(b) 含水率: 外层先干、内部保持较高水分',0,2.55,'含水率 /
                (kg/kg)',[0,.5,1,1.5,2,2.55])]:
207             panel(ax,title);im=ax.pcolormesh(faces(d['time_h']),faces(d['r_cm']),d[key],shadi_
                ng='flat',cmap=BLUESEQ,norm=Normalize(vmin,vmax),rasterized=True)
208             ax.set(xlim=(0,3),ylim=(0,2),ylabel='半径 /
                cm');ax.set_yticks([0,.5,1,1.5,2]);ax.set_xticks([0,.5,1,1.5,2,2.5,3])
209             cb=fig.colorbar(im,ax=ax,pad=.025,fraction=.025,aspect=15,ticks=ticks);cb.set_lab_
                el(label);cb.ax.tick_params(labelsize=8,length=2.5,width=.6)
210             axes[1].set_xlabel('时间 / h')
211     elif fid=='fig-dimensional':
212         fig=plt.figure(layout='constrained');gs=fig.add_gridspec(2,1,height_ratios=[1,1.5]);a_
            =fig.add_subplot(gs[0]);b=fig.add_subplot(gs[1])
213         panel(a,'(a) 二维含水率: 6 h, 中心中面至一端')
214         im=a.pcolormesh(faces(d['z_cm']),faces(d['r_cm']),d['Cfield'],shading='flat',cmap=BLU_
            ESEQ,norm=Normalize(0,2.55),rasterized=True)
215         a.set(xlim=(0,float(d['z_cm'][-1])),ylim=(0,float(d['r_cm'][-1])),xlabel='轴向坐标 /
            cm',ylabel='半径 / cm');a.set_aspect('equal',adjustable='box');a.set_xticks([0,2,4,6_
            ,8]);a.set_yticks([0,.5,1])
216         cb=fig.colorbar(im,ax=a,pad=.025,fraction=.025,aspect=12,ticks=[0,.5,1,1.5,2,2.55]);c_
            b.set_label('含水率 / (kg/kg)');cb.ax.tick_params(labelsize=8,length=2.5,width=.6)
217         panel(b,'(b) 同一径向节点的差值: 一维减二维')
218         b.plot(d['r_cm'],d['error_mid'],color=BLUE,label='相对二维中面',ls='-')
219         b.plot(d['r_cm'],d['error_mean'],color=GRAY,label='相对二维轴向平均',ls='--')
220         b.axhline(0,color='#999999',lw=.65,ls=':');b.set(xlabel='实际半径 /
            cm',ylabel='含水率差 / (kg/kg)',xlim=(0,float(d['r_cm'][-1])));b.grid(axis='y');b.le_
            gend(frameon=False,ncol=2,loc='lower left')
221     elif fid=='fig-geometry-control':
222         fig,axes=plt.subplots(1,2,layout='constrained')
223         for ax,key,title,ylabel in [(axes[0],'Cmax','(a) 相同物性下的几何对照','全域最大含水率
            / (kg/kg)'),(axes[1],'Md_ratio','(b) 干质量一致性诊断','由密度积分的干质量 / 初值')]:
224             panel(ax,title)
225             ax.plot(d['fixed_time_h'],d['fixed_'+key],color=GRAY,ls='--',label='固定尺寸')
226             ax.plot(d['moving_time_h'],d['moving_'+key],color=BLUE,ls='-',label='等比例收缩')

```

```

227         timeaxis(ax,72);ax.set_ylabel(ylabel);ax.set_xticks([0,24,48,72]);ax.legend(frame,
        on=False,loc='upper right' if key=='Cmax' else 'upper left')
228     axes[0].set_ylim(0,2.65);axes[0].axhline(.15,color='#999999',ls=':',lw=.8)
229     axes[0].plot([72],[d['fixed_Cmax'][-1]],marker='>',ms=5,mec=GRAY,mfc='white',clip_on=
        False)
230     axes[0].annotate('72 h 未达标',xy=(72,d['fixed_Cmax'][-1]),xytext=(34,.64),fontsize=8,
        ,color=TEXT,arrowprops={'arrowstyle':'-','color':GRAY,'lw':.65})
231     axes[1].axhline(1,color='#999999',ls=':',lw=.8);axes[1].set_ylim(0,3)
232     elif fid=='fig-length-convergence':
233         fig=plt.figure(layout='constrained');gs=fig.add_gridspec(2,2,height_ratios=[1,1]);a=f,
        ig.add_subplot(gs[0,:]);b=fig.add_subplot(gs[1,0]);c=fig.add_subplot(gs[1,1])
234         panel(a,'(a) 约束模型给出的长度: 没有轴向实测验证')
235         a.plot(d['map_time_h'],d['map_length_cm'],color=BLUE,label='变长度 (Map A) ')
236         a.plot(d['aspect_time_h'],d['aspect_length_cm'],color=GRAY,ls='--',label='等比例收缩')
237         a.axhline(25,color='#AAAAAA',ls=':',lw=.65);timeaxis(a,72);a.set_ylabel('长度 /
        cm');a.set_ylim(0,35);a.set_xticks([0,12,24,36,48,60,72]);a.legend(frameon=False,nco,
        l=2,loc='lower left')
238         panel(b,'(b) Map A 的失水余额残差')
239         b.plot(d['balance_time_h'],d['mass_residual_relative'],color=BLUE,lw=.9);b.axhline(0,
        color=GRAY,ls=':',lw=.65)
240         timeaxis(b,72);b.set_ylabel('相对残差');b.set_xticks([0,24,48,72]);b.set_ylim(-6e-8,1,
        e-8);b.set_yticks(np.array([-6,-4,-2,0])*1e-8)
241         b.ticklabel_format(axis='y',style='sci',scilimits=(0,0),useMathText=True)
242         panel(c,'(c) 相同容差下的网格收敛')
243         b.set_title('(b) Map A 的失水余额残差',loc='left',y=1.13,pad=6,fontweight='normal')
244         c.set_title('(c) 相同容差下的网格收敛',loc='left',y=1.13,pad=6,fontweight='normal')
245         c.plot(np.arange(3),d['grid_critical_h'],color=BLUE,marker='o',ms=4,lw=1)
246         c.set_xticks(np.arange(3),['12×6','24×12','32×16']);c.set_xlabel('网格 (径向 × 轴向)
        ');c.set_ylabel('达标时间 /
        h');c.set_ylim(47.8,48.7);c.set_xlim(-.35,2.35);c.grid(axis='y')
247         for i,y in enumerate(d['grid_critical_h']):c.annotate(f'{y:.4f}',(i,y),xytext=(0,7 if
        i!=2 else -13),textcoords='offset points',ha='center',fontsize=7.8)
248     else:raise ValueError(fid)
249     finish(fig,fid,mat,bindings)
250
251 def main():
252     p=argparse.ArgumentParser();p.add_argument('--source',type=Path,default=MODEL);p.add_argument,
        ('--render-only',action='store_true');p.add_argument('--figure',choices=list(PLANS));a=p.par,
        se_args()
253     (ROOT/'evidence').mkdir(parents=True,exist_ok=True)
254     if not a.render_only:extract(a.source.resolve())
255     bindings=json.loads((ROOT/'evidence/data-bindings.json').read_text())
256     for fid in ([a.figure] if a.figure else PLANS):render(fid,bindings);print(fid,flush=True)
257     dump(ROOT/'captions.json',{fid:{'claim':p['claim'],'caption':p['caption'],'size_mm':[160,p['h,
        eight_mm']], 'input_data':bindings[fid]} for fid,p in PLANS.items())

```

```

258     dump(ROOT/'evidence/render-code.json',{'source':'src/build_figures.py','sha256':sha(__file__),
        'data_mode':'frozen arrays
        only','model_code_modified':False,'python':sys.version,'numpy':np.__version__,'matplotlib':m,
        atplotlib.__version__})
259 if __name__=='__main__':main()

```

F.64 validate_figures.py

类别：数据图形与示意图。原始代码共 44 行。

```

1  """Read-only source, figure-contract, format, font, and geometry validation."""
2  from pathlib import Path
3  import json,hashlib,subprocess,sys
4  from PIL import Image
5  from pypdf import PdfReader
6  ROOT=Path(__file__).resolve().parents[1]
7  SKILL=Path('./support/skills/lab-ultra-scientific-figure-maker')
8
9  def h(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
10 def command(args):
11     p=subprocess.run(args,capture_output=True,text=True)
12     return {'command':args,'returncode':p.returncode,'stdout':p.stdout,'stderr':p.stderr}
13 def main():
14     binds=json.loads((ROOT/'evidence/data-bindings.json').read_text());reports=[]
15     manifest=json.loads((ROOT/'evidence/input-manifest.json').read_text())
16     changed=[]
17     for row in manifest['files']:
18         if h(row['absolute_path'])!=row['sha256']:changed.append(row['path'])
19     assert not changed,changed
20     for fid,bind in binds.items():
21         assert h(ROOT/bind['data_file'])==bind['data_sha256']
22         request=json.loads((ROOT/f'{fid}.request.json').read_text())
23         typ=json.loads((ROOT/f'{fid}.typography.json').read_text())
24         contract=command([sys.executable,str(SKILL/'scripts/validate_figure_contract.py'),str(ROOT/
            T/f'{fid}.request.json'),str(ROOT/f'{fid}.ir.json')])
25         svg=command([sys.executable,str(SKILL/'scripts/audit_svg.py'),str(ROOT/f'{fid}.svg')])
26         assert contract['returncode']==0 and svg['returncode']==0
27         reader=PdfReader(ROOT/f'{fid}.pdf');assert len(reader.pages)==1
28         page=reader.pages[0];mm=[float(page.mediabox.width)*25.4/72,float(page.mediabox.height)*2,
            5.4/72]
29         assert abs(mm[0]-160)<.01 and abs(mm[1]-request['final_height_mm'])<.01
30         with Image.open(ROOT/f'{fid}.png') as im:
31             rgba=im.convert('RGBA');extrema=rgba.getchannel('A').getextrema();assert
            extrema==(255,255)
32             png={'size_px':list(im.size),'mode':im.mode,'dpi':im.info.get('dpi'),'alpha_range':ex,
            tremas,
33             'effective_ppi_x':im.width/(160/25.4),'effective_ppi_y':im.height/(request['fina,
            l_height_mm']/25.4)}

```

```

34     fonts=command(['/opt/homebrew/bin/pdffonts',str(ROOT/f'{fid}.pdf')]);assert
        fonts['returncode']==0 and 'Type 3' not in fonts['stdout']
35     images=command(['/opt/homebrew/bin/pdfimages', '-list',str(ROOT/f'{fid}.pdf')]);assert
        images['returncode']==0
36     assert not typ['warnings'],typ['warnings'];assert typ['minimum_final_font_pt']>=7.5
37     reports.append({'figure_id':fid,'pdf_size_mm':mm,'png':png,'contract':contract,'svg_structu',
        ture':svg,'pdf_fonts':fonts,
38         'pdf_embedded_images':images,'typography':typ,'artifacts':{ext:h(ROOT/f'{fid}.{ext}')
        for ext in ('svg','pdf','png')},
39         'mechanical_checks':'passed','scientific_and_visual_review':'separate self-review
        required'})
40     result={'source_files_checked':len(manifest['files']),'source_files_changed':changed,'figures',
        ':reports',
41         'scope':'Mechanical checks do not establish material validity, unmeasured parameter
        identification, or publication acceptance.'}
42     (ROOT/'evidence/figure-checks.json').write_text(json.dumps(result,ensure_ascii=False,indent=2,
        ))
43     print(json.dumps({'source_files_changed':changed,'figures_passed':len(reports)}))
44 if __name__=='__main__':main()

```

F.65 extract_paper_data.py

类别：数据图形与示意图。原始代码共 40 行。

```

1  """Extract paper tables and named numerical claims from frozen model outputs."""
2  from pathlib import Path
3  import json, csv, hashlib
4  import numpy as np
5
6  ROOT=Path(__file__).resolve().parents[1]
7  SOURCE=ROOT.parent/'model-system-20260912'
8
9  def load(p):return json.loads(p.read_text())
10 def clean(v):
11     if isinstance(v,np.ndarray):return clean(v.tolist())
12     if isinstance(v,(np.floating,float)):return float(v) if np.isfinite(v) else None
13     if isinstance(v,np.integer):return int(v)
14     if isinstance(v,dict):return {k:clean(x) for k,x in v.items()}
15     if isinstance(v,(list,tuple)):return [clean(x) for x in v]
16     return v
17
18 def main():
19     paths=[SOURCE/'integration/paper-tables.json',SOURCE/'baseline/numerical-summary.json',SOURCE,
        /'variable-length/analysis/final-convergence.json',SOURCE/'latent-branch/analysis/final/pair',
        s.csv']
20     tables=load(paths[0]);audit=load(paths[1]);var=load(paths[2])
21     d={'tables':tables,'main':audit['main_cases'],'dimensions':audit['dimensions'],'grid':audit['',
        grid_refinement'],'time':audit['time_refinement'],'variable':var}
22     with paths[3].open(encoding='utf-8-sig') as f:d['latent_pairs']=list(csv.DictReader(f))

```

```

23     sourcevar=SOURCE/'variable-length/delivery/q4-robin-latent0-32x16'
24     paths += [sourcevar/'midplane.csv',sourcevar/'surface.csv']
25     with paths[-1].open(encoding='utf-8-sig') as f:s={float(x['time_s']):x for x in
        csv.DictReader(f)}
26     with paths[-2].open(encoding='utf-8-sig') as f:rows=list(csv.DictReader(f))
27     times=[*range(21600,172598,21600),172598]
28     mapped={}
29     for row in rows:
30         t=float(row['time_s'])
31         if t in times and float(row['radius_cm']) in
            [0,.5,1.,1.5]:mapped.setdefault(t,{})[float(row['radius_cm'])]=row
32     d['variable_table']=[{'time_h':t/3600,'C':[None if mapped[t][r]['C_kg_per_kgdry']=='' else
        float(mapped[t][r]['C_kg_per_kgdry']) for r in [0,.5,1.,1.5]]+[float(s[t]['side_midplane_C'])
        ],'R_cm':float(s[t]['current_radius_cm']),'L_cm':float(s[t]['length_cm'])} for t in times]
33     C0=2.55;R=.02;D0=7e-9*np.exp(-.89/C0);alpha=.36/(820*2600)
34     d['derived']={'initial_area_factor':1+.02/.25,'end_fraction_total':.02/((.25+.02),'q1_alpha_m2',
        s':alpha,'q1_D0_m2s':D0,'q1_alpha_over_D0':alpha/D0,'q1_radial_heat_scale_h':R*R/alpha/3600,
        'q1_radial_diffusion_scale_h':R*R/D0/3600,'q1_Bi_heat_folded':27*R/.36,'q1_Bi_mass_folded':8
        .64e-7*R/D0}
35     binding=[{'path':str(p.relative_to(SOURCE)),'sha256':hashlib.sha256(p.read_bytes()).hexdigest,
        ()} for p in paths]
36     (ROOT/'evidence/paper-data.json').write_text(json.dumps(clean(d),ensure_ascii=False,indent=2,
        allow_nan=False)+'\n')
37     (ROOT/'evidence/data-sources.json').write_text(json.dumps({'source_root':str(SOURCE),'files':
        binding,'operation':'Deterministic extraction only; no model fitting or PDE
        integration'},ensure_ascii=False,indent=2)+'\n')
38     print(json.dumps({'derived':clean(d['derived']),'variable_table_rows':len(d['variable_table'],
        )),ensure_ascii=False))
39
40 if __name__=='__main__':main()

```

F.66 enrich.py

类别：数据图形与示意图。原始代码共 72 行。

```

1 from pathlib import Path
2 import sys,json,re,hashlib
3 import numpy as np
4 import matplotlib
5 matplotlib.use('Agg')
6 import matplotlib.pyplot as plt
7 S=Path('./support/skills')
8 sys.path[:0]=[str(S/'lab-ultra-scientific-figure-maker/scripts'),str(S/'lab-ultra-scientific-vis
    ualization/scripts')]
9 from figure_library import publication_style
10 from figure_export import export_figure
11 P=Path(__file__).resolve().parents[1]; O=P/'figures/enriched';O.mkdir(exist_ok=True)
12 D=json.loads((P/'evidence/paper-data.json').read_text()); B='#244F73'; G='#8B949D'

```

```

13 src=P/'evidence/paper-data.json';
    sources=[{'path':str(src),'sha256':hashlib.sha256(src.read_bytes()).hexdigest()}]
14 add=[]
15 def finish(fig,id,claim,anchor,prose,caption,recipe='multi_panel_line'):
16     req=dict(schema_version='1.0.0',figure_id=id,claim=claim,audience='CUMCM judges',medium='A4
        paper',final_width_mm=158,language='zh-CN',data_sources=sources,variables=[dict(name='documented
        variables in axis labels',unit='see axes',role='computed or
        formula-derived')],uncertainty={'kind':'none; conditional deterministic
        results'},editable_format='svg',claim_locked=True)
17     ir=dict(schema_version='1.0.0',figure_id=id,claim=claim,route='quantitative_code',recipe=recipe,
        panels=[{'panel_id':str(i+1),'purpose':a.get_title()} for i,a in
        enumerate(fig.axes)],encodings=[{'field':'documented source arrays or supplied
        formulas','channel':'position','scale':'see labeled
        axes'}],annotations=[],style_tokens={'palette':'blue-gray','base_font_pt':9},exports=['svg','pdf',
        'png'],invariants=['data','units','claim','transform','uncertainty'],provenance={'renderer':
        'matplotlib','source_script':str(Path(__file__))})
18     for suffix,obj in [ ('request',req), ('ir',ir)]:
        (0/f'{id}.{suffix}.json').write_text(json.dumps(obj,ensure_ascii=False,indent=2))
19     export_figure(fig,0/id,formats=['svg','pdf','png'],dpi=260,overwrite=True,mkdir=True,provenance,
        ={'sources':sources,'materials':style},write_manifest=True,accessibility={'title':claim,'desc':
        caption});plt.close(fig)
20     add.append((anchor,f'\n\n{prose}\n\n!{{caption}}(figures/enriched/{id}.png){{#{id}
        width="100%"}\n\n'))
21 def sub(n=2,h=66):
22     f,a=plt.subplots(1,n,figsize=(158/25.4,h/25.4),layout='constrained');return f,np.atleast_1d(a)
23 with publication_style('nature',language='zh-CN',width_mm=158,height_mm=70) as style:
24     plt.rcParams.update({'font.size':9,'axes.labelsize':9,'axes.titlesize':9,'legend.fontsize':8,'a
        xes.spines.top':False,'axes.spines.right':False,'svg.fonttype':'none'})
25     f,a=sub(2,57); ends=2/(25+2)*100
26     a[0].barh(['初始表面积'],[100-ends],color=B);a[0].barh(['初始表面积'],[ends],left=100-ends,color
        =G);a[0].set(xlim=(0,100),xlabel='占总外表面积 / %',title='(a) 侧面与两端面')
27     a[0].text(35,0,'侧面 92.59%',ha='center',va='center',color='white');a[0].annotate('两端面
        7.41%',xy=(96,0),xytext=(65,.52),arrowprops={'arrowstyle':'-'},fontsize=8)
28     a[0].set_ylim(-.5,.65)
29     a[1].bar(['只用侧面','面积折算'],[1,1.08],color=[G,B],width=.5);a[1].set(ylim=(0,1.25),ylabel='
        相对侧面积',title='(b) 一维边界的面积因子')
30     for i,v in enumerate([1,1.08]):a[1].text(i,v+.025,f'{v:.2f}',ha='center')
31     finish(f,'fig-area','端面面积折算补偿交换总面积','## 方程、初值与边界条件','[图 X](#fig-area)
        将初始面积关系单独展开：两端面合计占总外表面积的 7.41%，相对于侧面则为 8%。因此折算因子为 1.08。
        该操作保留总交换面积，却没有恢复轴向温湿梯度；其局部近似误差仍需由二维计算检查。
        ','初始几何的面积分解与一维折算系数。面积由题给 R=2 cm、L=25 cm 直接计算，柱高不是拟合结果。')
32     c=np.linspace(.05,2.55,350);f,a=sub()
33     for t,col,ls in [(28,G,'--'),(50,B,'-')]:a[0].semilogy(c,2.4e-3*np.exp(-.45/c-3850/(t+273.15)),c,
        color=col,ls=ls,label=f'{t} °C')
34     a[0].set(xlabel='干基含水率 C / (kg/kg)',ylabel='D / (m²/s)，对数轴',title='(a) 附录 3
        的扩散系数');a[0].legend()

```

```

35 a[1].plot(c,.21+.38*c/(1+c),color=B,label='附录
3')a[1].plot(c,.12+.20*c/(1+c),color=G,ls='--',label='附录 4');a[1].set(xlabel='干基含水率 C /
(kg/kg)',ylabel='k / [W/(m·K)]',title='(b) 题给导热系数');a[1].legend()
36 finish(f,'fig-properties','题给扩散系数在低含水率时明显降低','## 第四问的移动区域','[图
X](#fig-properties) 把题给公式转成可读的状态曲线。相同温度下,低含水区域的扩散系数迅速减小;
相同含水率下,温度升高又提高扩散系数。因此表面先干后形成的低扩散区域会影响后续迁移,
不能用初始扩散系数代表整个过程。右图同时显示两套物性并不相同,跨问时长差异不能全部归于几何。
','题给物性函数的直接绘图。左图使用附录 3,温度为两条指定切片;右图比较附录 3、4。
曲线不是测量数据或新增拟合。')
37 f,a=sub();r=np.linspace(0,2,5)
38 for idx,col,ls in [(1,'#A2AAB1',':'),(3,'#647D91','--'),(6,B,'-')]:
39     row=D['tables']['ld-q1'][idx]
40     for j,key in enumerate(['T','C']):a[j].plot(r,row[key],marker='o',ms=3,color=col,ls=ls,label=f'_{
row["time_s"]:g} s')
41 for j in range(2):a[j].set(xlabel='半径 r / cm',ylabel=['温度 / °C','干基含水率 /
(kg/kg)'][j],title=['(a) 预热温度剖面','(b) 同时刻含水率剖面'][j]);a[j].legend()
42 finish(f,'fig-preheat','预热时表面响应先于中心','## 问题二:变物性下前三小时的演化','[图
X](#fig-preheat) 将表中的三个代表时刻按空间位置连起来。表面先升温、先失水,中心变化较慢;到 1800
s,中心含水率仍接近初值。热量进入与水分排出存在不同时间尺度,
所以预热温度上升不能直接当作干燥完成的证据。','问题一的径向剖面。圆点为表中五个输出位置,
线段只连接这些计算值,未增加空间采样或进行拟合。')
43 f,a=sub();rows=D['tables']['ld-q3'];tt=np.array([x['time_s']/3600 for x in
rows]);cc=np.array([x['C'] for x in rows])
44 for j in range(2):
45     for k,col,lab in [(0,B,'中心'),(2,'#647D91','r=1
cm'),(4,G,'表面')]:a[j].plot(tt,cc[:,k],marker='o',ms=3,color=col,label=lab)
46 a[j].axhline(.15,color='black',ls='--',lw=.8);a[j].set(xlabel='时间 / h',ylabel='干基含水率 /
(kg/kg)')
47 a[0].set(title='(a) 6 h
后的离散输出');a[0].legend();a[1].set(xlim=(30,58),ylim=(.045,.215),title='(b) 后期与达标阈值')
48 finish(f,'fig-threshold','表面达标早于全域达标','## 问题四:给定半径下的收缩计算','[图
X](#fig-threshold) 说明全域判据的必要性:表面含水率已低于 0.15 时,中心仍可能高于阈值。
后期曲线逐渐变平,中心少量剩余水分对应较长等待时间。图中离散点用于解释过程,
最终达标秒数仍由求解器的全场事件定位确定,不能从连接线估读。','问题三中心、
中间位置与表面的含水率演化及后期局部放大。点为已保存的表格输出,虚线为 0.15 阈值;
两面板纵轴范围不同。')
49 f,a=sub();dim=D['dimensions'];x=np.arange(3)
50 for k,col,off,lab in [('sampled_midplane_max_C_error',B,.17,'相对二维中面'),('sampled_axialmean,
_max_C_error',G,.17,'相对二维轴向平均')]:a[0].bar(x+off,[v[k] for v in
dim],width=.32,color=col,label=lab)
51 a[0].set(xticks=x,xticklabels=['问题一','问题二/三','问题四'],ylabel='采样最大含水率差 /
(kg/kg)',title='(a) 局部场的降维差异');a[0].legend(fontsize=7)
52 v=dim[1:];a[1].bar(['问题三','问题四'],[d['two_d_minus_one_d_time_minutes'] for d in
v],color=B,width=.5);a[1].set(ylabel='二维减一维达标时间 / min',title='(b) 总体达标时间差')
53 for i,d in enumerate(v):a[1].text(i,d['two_d_minus_one_d_time_minutes']+.4,f'{d["two_d_minus_one,
_d_time_minutes"]:.2f}',ha='center')

```

```

54 finish(f,'fig-dimension-summary','总体时间接近不代表局部场相同','## 只改变几何的收缩对照','[图
X](#fig-dimension-summary) 分别比较局部场和总体达标时间。问题三、四的二维时间分别比一维增加约
28.57、11.18 min, 但采样位置仍存在含水率差异。因此面积折算的适用性应按输出任务判断:
总时间接近可支持其简化用途, 却不足以替代端面附近的二维分布。','一维与二维在既有采样集上的比较。
左图为采样最大绝对差, 不是连续全域误差上界; 右图只比较同问、同物性下的维度差异。','grouped_bar')
55 f,a=sub(1,62);a=a[0];pairs=[x for x in D['latent_pairs'] if x['nr']==32']
56 for i,p in enumerate(pairs):
57     off,on=float(p['off_h']),float(p['on_h']);a.plot([off,on],[i,i],color=G,lw=2);a.scatter([off],
[i],color='white',edgecolor=B,s=40,zorder=3);a.scatter([on],[i],color=B,s=40,zorder=3);a.text(
(off+on)/2,i+.13,f'+{on-off:.2f} h',ha='center',fontsize=8)
58 a.set(yticks=range(3),yticklabels=[p['Ceq_reference'] for p in pairs],xlabel='条件模型达标时间 /
h',ylabel='参考平衡含水率 / (kg/kg)',ylim=(-.45,2.5),xlim=(45,73),title='空心点:
关闭相变通道 实心点: 开启相变通道')
59 finish(f,'fig-latent-pairs','潜热效应与未标定表面参数应分开解释','# 模型评价与结论','[图
X](#fig-latent-pairs) 中每一横线只连接同一表面参数下的配对结果,
因而线段长度可解释为该条件下相变通道的影响。三行之间的横向移动则说明绝对时长依赖表面关系。
这些点不构成置信区间, 也不能据此选择最接近某个预期答案的一组参数。','潜热条件实验的配对点图, 使用
32x16 网格。横线表示组内时长差, 没有统计误差条含义; 三个参考平衡点均未由题给材料数据标定。
','paired_dot')
60 # Additional direct visualization of the supplied radius implicit in original table.
61 f,a=sub(1,61);a=a[0];rows=D['tables']['1d-q4'];time=np.array([v['time_s']/3600 for v in
rows]);radius=np.array([v['radius_cm'] for v in
rows]);a.plot(time,radius,color=B,marker='o',ms=3,label='当前半径 (表格输出时刻)
');a.fill_between(time,radius,2,color='#ECEFF2',label='已不属于药材的原始区域');a.axhline(1.5,c
olor=G,ls='--',lw=1,label='固定位置 r=1.5 cm');a.set(xlabel='时间 / h',ylabel='物理半径 /
cm',ylim=(0,2.05),title='问题四输出窗口: 6 h 至达标');a.legend(fontsize=8,loc='lower left')
62 finish(f,'fig-domain-mask','收缩后外部位置必须与零含水率区分','# 二维轴对称模型与降维误差','[图
X](#fig-domain-mask) 将表 9 的空缺解释为几何事件。在所列输出时刻, 固定位置 r=1.5 cm
已超出当前半径, 因此没有对应的药材含水率。灰色区域表示原始圆柱中已不再被当前模型域占据的位置,
并不表示药材内部存在零含水率层。','问题四表格输出时刻的当前半径与域外位置。只展示已有 6 h
后的输出窗口, 不用这些点重建早期收缩曲线。','domain_mask')
63 s=(P/'manuscript-final-v4.md').read_text()
64 for anchor,block in add:
65     assert s.count(anchor)==1,(anchor,s.count(anchor));s=s.replace(anchor,block+anchor)
66 ids=re.findall(r'\{#(fig-[w-]+) width=',s);numbers={v:i+1 for i,v in enumerate(ids)}
67 s=re.sub(r'\[图\s*(?:\d+|X)\]\(#{fig-[w-]+})\)',lambda m:f'[图 {numbers[m[1]]}](#{m[1]})',s)
68 (P/'word-enriched/manuscript.md').write_text(s)
69 (O/'data-profile.json').write_text(json.dumps({'source':sources,'kind':'frozen deterministic
outputs and supplied analytic formulas','transforms':['seconds to hours','geometry area
percentages'],'discrete profiles connected without fitting','specified temperature slices of
supplied D(C,T)','existing paired times grouped by equilibrium
reference'],'missingness':'outside-domain values remain absent','uncertainty':'no fitted
parameters or confidence intervals added','figures':ids},ensure_ascii=False,indent=2))
70 build=(P/'word/build_word.py').read_text().replace("out=root/'word'", "out=root/'word-enriched'")
.replace("root/'manuscript-final-v4.md'", "root/'word-enriched/manuscript.md'").replace("交付/论
文Word版.docx", "交付/论文图文增强版.docx")
71 (P/'word-enriched/build_word.py').write_text(build)

```

```
72 print('total figures',len(ids))
```

F.67 round2-figures.py

类别：数据图形与示意图。原始代码共 43 行。

```
1 from pathlib import Path
2 import json,hashlib,re,sys
3 import numpy as np
4 import matplotlib
5 matplotlib.use('Agg')
6 import matplotlib.pyplot as plt
7 from matplotlib.patches import FancyArrowPatch
8 P=Path(__file__).resolve().parents[1];W=P/'three-rounds';O=P/'figures/three-rounds';O.mkdir(exis_
t_ok=True)
9 sys.path[:0]=[ './support/skills/lab-ultra-scientific-figure-maker/scripts','./support/skills/lab-
ultra-scientific-visualization/scripts']
10 from figure_library import publication_style
11 from figure_export import export_figure
12 from figure_typography import inspect_figure_typography
13 D=json.loads((P/'evidence/paper-data.json').read_text());B='#244F73';G='#7D8994'
14 sources=[{'path':str(p),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in
[P/'evidence/paper-data.json',P/'word-enriched/manuscript.md']]
15 changes=[]
16 def save(fig,id,claim,caption,anchor,prose,recipe,variables):
17     req=dict(schema_version='1.0.0',figure_id=id,claim=claim,audience='CUMCM readers',medium='A4
paper',final_width_mm=158,language='zh-CN',data_sources=sources,variables=variables,uncertainty_
={'kind':'none; deterministic values, not measured
confidence'},editable_format='svg',claim_locked=True)
18     ir=dict(schema_version='1.0.0',figure_id=id,claim=claim,route='quantitative_code',recipe=recipe,
,panels=[{'panel_id':str(i),'purpose':a.get_title()} for i,a in
enumerate(fig.axes)],encodings=[{'field':'supplied equations or frozen table
outputs','channel':'position','scale':'linear'}],annotations=[],style_tokens={'palette':'blue-g_
ray','base_font_pt':9},exports=['svg','pdf','png'],invariants=['data','units','claim','transfor_
m','uncertainty'],provenance={'renderer':'matplotlib','source_script':str(Path(__file__))})
19     for suffix,d in
[('request',req),('ir',ir),('typography',inspect_figure_typography(fig,final_width_mm=158))]:
(O/f'{id}.{suffix}.json').write_text(json.dumps(d,ensure_ascii=False,indent=2,default=str))
20     export_figure(fig,O/id,formats=['svg','pdf','png'],dpi=280,overwrite=True,mkdir=True,provenance_
={'sources':sources,'materials':style},write_manifest=True,accessibility={'title':claim,'desc':,
caption});plt.close(fig)
21     changes.append((anchor,f'\n\n{prose}\n\n!{{caption}}(figures/three-rounds/{id}.png){{#{id}
width="100%"}}\n\n'))
22     with publication_style('nature',language='zh-CN',width_mm=158,height_mm=68) as style:
23         plt.rcParams.update({'font.size':9,'axes.labelsize':9,'axes.titlesize':9,'legend.fontsize':8,'a_
xes.spines.top':False,'axes.spines.right':False,'svg.fonttype':'none'})
24         f,a=plt.subplots(figsize=(158/25.4,68/25.4));a.set(xlim=(0,10),ylim=(0,4));a.axis('off')
25         boxes=[(1.3,2,'温度场 T\ 热量传入与内部导热'),(8.6,2,'含水率场 C\ 内部扩散与表面交换')]
```

```

26 for x,y,label in boxes:a.text(x,y,label,ha='center',va='center',fontsize=9,bbox={'boxstyle':'rou
    nd,pad=.55','fc':'white','ec':B,'lw':1})
27 for start,end,rad in
    [(2.7,2.35),(7.1,2.35),-.45),((7.1,1.65),(2.7,1.65),-.45)]:a.add_patch(FancyArrowPatch(start,e
    nd,connectionstyle=f'arc3,rad={rad}',arrowstyle='->',mutation_scale=12,color=B,lw=1.2))
28 a.text(5,3.75,'温度改变扩散系数 D(C,T)',ha='center',va='center',fontsize=9)
29 a.text(5,.38,'含水率改变导热系数、密度和比热',ha='center',va='center',fontsize=9)
30 a.text(5,.02,'问题二至四: 每次非线性迭代按当前 T、C 更新物性',ha='center',fontsize=8,color=G)
31 # constrained layout owns placement
32 save(f,'fig-coupling-loop','变物性产生温度与水分的双向耦合','问题二至四的物性耦合关系。
    箭头表示方程中的参数依赖,不表示额外新增的热源或水源。该图对应基础热湿方程,潜热支线另行讨论。
    ','## 第四问的移动区域','[图 X](#fig-coupling-loop) 说明两条方程为何不能独立求解。
    温度影响水分扩散速度,而水分变化又通过导热系数和热容量影响温度响应;因此隐式时间步内也要更新物性,
    直至同一步的温湿状态共同收敛。问题一采用其专属物性,不将后续各问的全部依赖关系反向套用。
    ','dependency_diagram',[{'name':'T,C','unit':'°C, kg/kg','role':'coupled state'}])
33 f,a=plt.subplots(1,2,figsize=(158/25.4,65/25.4),layout='constrained');rows=D['tables']['1d-q2'],
    ;t=np.r_[0,[r['time_s']/3600 for r in rows]]
34 dt=np.r_[0,[r['T'][-1]-r['T'][0] for r in rows]];dc=np.r_[0,[r['C'][0]-r['C'][-1] for r in rows]]
35 for ax,v,label,title in zip(a,[dt,dc],['表面减中心温度 / °C','中心减表面含水率 / (kg/kg)'],[(a)
    温差先增后减','(b) 水分差异持续存在']):
36 ax.plot(t,v,'o-',color=B,ms=3);ax.set(xlabel='时间 /
    h',ylabel=label,title=title);ax.set_ylim(bottom=0);ax.grid(axis='y',color='#E3E7EA',lw=.5)
37 a[0].annotate(f'{dt[-1]:.4f}
    °C',xy=(3,dt[-1]),xytext=(1.9,.65),arrowprops={'arrowstyle':'-','color':G},fontsize=8)
38 a[1].annotate(f'{dc[-1]:.4f}
    kg/kg',xy=(3,dc[-1]),xytext=(1.35,.48),arrowprops={'arrowstyle':'-','color':G},fontsize=8)
39 save(f,'fig-gradient-lag','温差消退与含水率差消退不同步','问题二表面与中心的差值。
    初始点来自题设均匀初值,其余点来自表 6、7;线段仅连接半小时间隔的输出。
    两面板的差值方向和单位分别标注。','## 问题三: 全域达标时间','[图 X](#fig-gradient-lag)
    将前述空间分布压缩为两项可比较的过程指标。到 3 h, 温差已降至 0.1106 °C, 含水率差仍为 0.7815 kg/kg。
    两者并不同步消失,故问题三必须继续推进水分场并检查全域阈值,不能用温度接近均匀替代达标判断。
    ','paired_time_profiles',[{'name':'time_h','unit':'h','role':'x'},{'name':'surface_T-center_T',
    'unit':'°C','role':'response'},{'name':'center_C-surface_C','unit':'kg/kg','role':'response'}])
40 f,a=plt.subplots(1,2,figsize=(158/25.4,65/25.4),layout='constrained');rows=D['variable_table'];
    t=np.r_[0,[r['time_h'] for r in rows]];rr=np.r_[1,[r['R_cm']/2 for r in
    rows]];ll=np.r_[1,[r['L_cm']/25 for r in rows]]
41 a[0].plot(t,rr**2,'o-',color=B,ms=3,label='截面面积比');a[0].plot(t,ll,'s--',color=G,ms=3,label='
    长度比');a[0].axhline(1,color='black',lw=.6,ls=':');a[0].set(title='(a)
    径向缩小与轴向增加',xlabel='时间 /
    h',ylabel='相对初始值');a[0].legend(fontsize=7.5);a[0].set_ylim(0,1.5)
42 a[1].plot(t,rr**2*ll,'o-',color=B,ms=3);a[1].axhline(1,color='black',lw=.6,ls=':');a[1].set(tit
    le='(b) 总体积仍减小',xlabel='时间 / h',ylabel='体积比',ylim=(0,1.1))

```

```

43 save(f,'fig-volume-factors','变长度条件模型中伸长可以与总体积减小共存','变长度条件解的几何分解。
    由表 11 的半径与长度计算截面积比及体积比，补入题设初始点；线段只连接输出点。
    长度与总体积均为条件计算值，并非观测。','# 数值检验与潜热条件实验',[图 X](#fig-volume-factors)
    将体积变化分解为截面积与长度两个因子。条件解中长度虽超过初始值，截面积却缩小得更多，
    二者乘积仍小于 1。因此“轴向增加”与“总体积减少”在代数上可以同时成立；这解释了约束解的几何含义，
    但不能替代对真实变形方式的测量。','geometry_factorization',[{'name':'time_h','unit':'h','role':'',
    x'},{'name':'area_ratio,length_ratio,volume_ratio','unit':'1','role':'derived geometric
    ratio'}])

```

F.68 check_and_package.py

类别：数据图形与示意图。原始代码共 48 行。

```

1 from pathlib import Path
2 import json, hashlib, shutil, zipfile, fitz
3
4 root=Path(__file__).resolve().parent
5 t=root/'paper/typeset'
6 old=root.parent/'emphasis-20260913/paper/typeset-v1'
7 def nodes(x,kind):
8     result=[x] if isinstance(x,dict) and x.get('t')==kind else []
9     for v in x.values() if isinstance(x,dict) else x if isinstance(x,list) else []:
10         result += nodes(v,kind)
11     return result
12 a=json.loads((old/'input.ast.json').read_text())
13 b=json.loads((t/'input.ast.json').read_text())
14 report={}
15 for kind in ['Math','Table','Strong']:
16     assert nodes(a,kind)==nodes(b,kind),kind
17     report[kind]=len(nodes(b,kind))
18 doc=fitz.open(t/'paper.pdf')
19 report['pages']=len(doc)
20 report['pdf_sha256']=hashlib.sha256((t/'paper.pdf').read_bytes()).hexdigest()
21 pairs=root/'review/page-pairs';pairs.mkdir(exist_ok=True)
22 for start in range(0,len(doc),2):
23     sheet=fitz.open();page=sheet.new_page(width=850,height=615)
24     for off in range(min(2,len(doc)-start)):
25         page.insert_text((10+425*off,12),f'Page {start+off+1}',fontsize=9)
26         page.show_pdf_page(fitz.Rect(5+425*off,17,420+425*off,608),doc,start+off)
27     page.get_pixmap(matrix=fitz.Matrix(1.5,1.5)).save(pairs/f'pages-{start+1:02d}-{min(start+2,len(doc)):02d}.png')
28     sheet.close()
29 for p in doc:
30     if '图 3' in p.get_text().replace(' ','') and '物性耦合' in p.get_text():
31         p.get_pixmap(matrix=fitz.Matrix(1.6,1.6)).save(root/'review/coupling-in-paper.png')
32         report['coupling_page']=p.number+1
33 d=root/'delivery';md=d/'Markdown 稿件';md.mkdir(parents=True,exist_ok=True)
34 shutil.copy2(root/'paper/manuscript.md',md/'论文正文.md')
35 shutil.copy2(t/'paper.pdf',d/'药材烘干论文 _ 流程图更新版.pdf')

```

```

36 shutil.copy2(root/'paper/AI 工具使用详情.pdf',md/'AI 工具使用详情.pdf')
37 for asset in json.loads((t/'assets-manifest.json').read_text()['assets']):
38     src=root/'paper'/asset['source_reference'];dest=md/asset['source_reference']
39     dest.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(src,dest)
40     assert hashlib.sha256(dest.read_bytes()).hexdigest()==asset['source_sha256']
41     companion=src.with_suffix('.png')
42     if companion.exists() and companion!=src:shutil.copy2(companion,dest.with_suffix('.png'))
43 with zipfile.ZipFile(d/'Markdown 稿件与图片.zip','w',zipfile.ZIP_DEFLATED) as z:
44     for p in sorted(md.rglob('*')):
45         if p.is_file():z.write(p,p.relative_to(d))
46 with zipfile.ZipFile(d/'Markdown 稿件与图片.zip') as z:assert z.testzip() is None
47 (root/'review/preservation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2))
48 print(json.dumps(report,ensure_ascii=False))

```

F.69 figure_library.py

类别：图形公共工具。原始代码共 284 行。

公共工具：图形样式、导出或字体核查依赖，不参与物理方程。

```

1 """Find pinned figure recipes and apply the bundled publication materials.
2
3 Catalog commands use only the standard library. Applying styles requires
4 Matplotlib and the sibling lab-ultra-scientific-visualization skill, never vendor imports.
5 """
6
7 from __future__ import annotations
8
9 import argparse
10 import hashlib
11 import json
12 import math
13 import subprocess
14 import sys
15 from contextlib import contextmanager
16 from pathlib import Path
17
18 SKILL = Path(__file__).resolve().parents[1]
19 WORKSPACE = SKILL.parents[2]
20 ASSETS = SKILL / "assets"
21 UPSTREAM = ASSETS / "upstream"
22
23
24 def read_json(path):
25     return json.loads(Path(path).read_text(encoding="utf-8"))
26
27
28 def catalog():
29     return read_json(ASSETS / "library-catalog.json")["libraries"]
30

```

```

31
32 def find_libraries(tag=None, ecosystem=None):
33     return [
34         item
35         for item in catalog()
36         if (tag is None or tag in item["tags"])
37         and (ecosystem is None or ecosystem == item["ecosystem"])
38     ]
39
40
41 def sha256(path):
42     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
43
44
45 def resolve_library(library_id, workspace=WORKSPACE):
46     matches = [item for item in catalog() if item["id"] == library_id]
47     if not matches:
48         raise ValueError(f"unknown library: {library_id}")
49     item = matches[0]
50     repo = Path(workspace) / item["origin"]["local_path"]
51     if not (repo / ".git").is_dir():
52         raise ValueError(
53             f"source checkout missing: {repo}; use the locked origin to restore it"
54         )
55     result = subprocess.run(
56         ["git", "-C", str(repo), "rev-parse", "HEAD"],
57         capture_output=True,
58         text=True,
59         check=True,
60     )
61     if result.stdout.strip() != item["origin"]["commit"]:
62         raise ValueError(f"source commit drift: {library_id}")
63     dirty = subprocess.run(
64         ["git", "-C", str(repo), "status", "--porcelain"],
65         capture_output=True,
66         text=True,
67         check=True,
68     )
69     if dirty.stdout:
70         raise ValueError(f"source checkout has local changes: {library_id}")
71     paths = [repo / part for part in item["entrypoints"]]
72     for path in paths:
73         if not path.exists():
74             raise ValueError(f"recipe entrypoint missing: {path}")
75     return {
76         **item,
77         "resolved_paths": [str(p) for p in paths],
78         "runtime_status": "not_assumed_installed",

```

```

79     }
80
81
82 def verify_bundle():
83     manifest = read_json(UPSTREAM / "manifest.json")
84     for item in manifest["files"]:
85         path = SKILL / item["path"]
86         if not path.is_file() or sha256(path) != item["sha256"]:
87             raise ValueError(f"bundled material missing or changed: {item['path']}")
88     return {"valid": True, "files": len(manifest["files"])}
89
90
91 def profiles():
92     verify_bundle()
93     return read_json(ASSETS / "style-profiles.json")
94
95
96 def _positive(value, name):
97     if isinstance(value, bool) or not isinstance(value, (float, int)):
98         raise TypeError(f"{name} must be a positive finite number")
99     if not math.isfinite(value) or value <= 0:
100         raise ValueError(f"{name} must be a positive finite number")
101     return float(value)
102
103
104 def resolve_style(
105     profile="science",
106     *,
107     width_mm=None,
108     height_mm=None,
109     venue=None,
110     palette="okabe_ito_on_white",
111     overrides=None,
112     language=None,
113 ):
114     """Resolve one visual family, then explicit dimensions and caller overrides.
115
116     Export crop, layout ownership, and canvas size use named arguments instead
117     of upstream defaults. The returned rc dictionary can be used in rc_context.
118     """
119     import matplotlib as mpl
120
121     sys.path.insert(0, str(SKILL.parent / "lab-ultra-scientific-visualization" / "scripts"))
122     from style_presets import get_style
123
124     config = profiles()
125     if not isinstance(profile, str) or profile not in config["profiles"]:
126         raise ValueError(f"select one style from {list(config['profiles'])}")

```

```

127     selected = config["profiles"][profile]
128     rc = get_style("default", palette_name=palette)
129     colors = rc.pop("_palette_colors")
130     rc["axes.prop_cycle"] = mpl.cycler(color=colors)
131     # These settings are resolved once for the whole FigureRequest.
132     owned = {
133         "figure.figsize",
134         "figure.dpi",
135         "savefig.bbox",
136         "text.usetex",
137         "text.latex.preamble",
138         "axes.prop_cycle",
139         "savefig.transparent",
140         "figure.autolayout",
141         "figure.constrained_layout.use",
142     }
143     suppressed = set()
144     for rel in selected["files"]:
145         layer = dict(
146             mpl.rc_params_from_file(UPSTREAM / rel, use_default_template=False)
147         )
148         suppressed.update(owned.intersection(layer))
149         rc.update({key: value for key, value in layer.items() if key not in owned})
150     rc.update(selected.get("rc", {}))
151     rc.update(
152         {
153             "text.usetex": False,
154             "savefig.bbox": None,
155             "savefig.transparent": False,
156             "figure.autolayout": False,
157             "figure.constrained_layout.use": True,
158         }
159     )
160     size_source = "explicit_mm" if width_mm is not None else "general_150mm"
161     if venue is not None:
162         if venue not in config["venue_widths"]:
163             raise ValueError(f"unknown dated venue preset: {venue}")
164         if width_mm is None:
165             width_mm = config["venue_widths"][venue]["width_in"] * 25.4
166             size_source = venue
167     width = _positive(150.0 if width_mm is None else width_mm, "width_mm")
168     height = _positive(width * 0.66 if height_mm is None else height_mm, "height_mm")
169     rc["figure.figsize"] = [width / 25.4, height / 25.4]
170     overrides = dict(overrides or {})
171     for key in ("figure.figsize", "savefig.bbox"):
172         if key in overrides:
173             raise ValueError(
174                 f"set {key} through figure dimensions/export options, not style overrides"

```

```

175         )
176     typography = None
177     if (profile in {"nature", "cns"} or language is not None) and not any(
178         key in overrides for key in ("font.family", "font.sans-serif", "font.serif")
179     ):
180         from figure_typography import resolve_figure_fonts
181         font_rc, typography = resolve_figure_fonts(language or "en")
182         rc.update(font_rc)
183         if language and language.startswith("zh"):
184             rc.update({"font.size": 8.5, "axes.labelsize": 8.5, "axes.titlesize": 8.5,
185                       "xtick.labelsize": 8, "ytick.labelsize": 8, "legend.fontsize": 8})
186     rc.update(overrides)
187     # Mirrored ticks from an upstream style must not float on hidden spines.
188     # Explicit caller tick choices still win, including unusual borderless axes.
189     for tick, spine in (
190         ("xtick.top", "axes.spines.top"),
191         ("ytick.right", "axes.spines.right"),
192     ):
193         if tick not in overrides and not rc.get(spine, mpl.rcParamsDefault[spine]):
194             if rc.get(tick, mpl.rcParamsDefault[tick]):
195                 suppressed.add(tick)
196                 rc[tick] = False
197     if rc.get("figure.autolayout") and rc.get("figure.constrained_layout.use"):
198         raise ValueError(
199             "tight/autolayout and constrained layout cannot both own a figure"
200         )
201     # Validate once without changing global Matplotlib state.
202     validated = mpl.RcParams(rc)
203     manifest = read_json(UPSTREAM / "manifest.json")
204     receipt = {
205         "profile": profile,
206         "profile_upstream": selected["upstream_id"],
207         "palette": palette,
208         "width_mm": width,
209         "height_mm": height,
210         "dimension_source": size_source,
211         "ignored_upstream_defaults": sorted(suppressed),
212         "explicit_overrides": overrides,
213         "typography": typography or {"scope": "caller_or_serif_profile_owned; inspect actual
214                                     fonts"},
215         "material_manifest_sha256": sha256(UPSTREAM / "manifest.json"),
216         "resolver_sha256": sha256(__file__),
217         "sources": manifest["origins"],
218         "notice": "Visual starting point; venue presets are dated and do not certify submission
219                 compliance.",
220     }
221     return dict(validated), receipt

```

```

221
222 @contextmanager
223 def publication_style(profile="science", **kwargs):
224     import matplotlib as mpl
225
226     rc, receipt = resolve_style(profile, **kwargs)
227     with mpl.rc_context(rc):
228         yield receipt
229
230
231 def colormap(name="batlow", *, kind=None):
232     from matplotlib.colors import ListedColormap
233
234     config = profiles()["colormaps"]
235     if name not in config:
236         raise ValueError(f"unknown bundled colormap: {name}")
237     item = config[name]
238     if kind is not None and item["kind"] != kind:
239         raise ValueError(f"{name} is {item['kind']}, not {kind}")
240     rows = [
241         [float(x) for x in line.split()]
242         for line in (UPSTREAM / item["file"]).read_text().splitlines()
243         if line.strip()
244     ]
245     if len(rows) != 256 or any(
246         len(row) != 3 or any(not 0 <= x <= 1 for x in row) for row in rows
247     ):
248         raise ValueError(f"invalid RGB lookup table: {name}")
249     return ListedColormap(rows, name=f"cmcrameri_{name}").with_extremes(bad="#777777")
250
251
252 def main(argv=None):
253     parser = argparse.ArgumentParser(description=__doc__)
254     sub = parser.add_subparsers(dest="command", required=True)
255     find = sub.add_parser("find")
256     find.add_argument("--tag")
257     find.add_argument("--ecosystem")
258     resolve = sub.add_parser("resolve")
259     resolve.add_argument("library_id")
260     verify = sub.add_parser("verify")
261     verify.add_argument("--sources", action="store_true")
262     sub.add_parser("styles")
263     args = parser.parse_args(argv)
264     try:
265         if args.command == "find":
266             result = find_libraries(args.tag, args.ecosystem)
267         elif args.command == "resolve":
268             result = resolve_library(args.library_id)

```

```

269         elif args.command == "styles":
270             result = profiles()
271         else:
272             result = verify_bundle()
273         if args.sources:
274             result["verified_sources"] = len(
275                 [resolve_library(x["id"]) for x in catalog()]
276             )
277         print(json.dumps(result, ensure_ascii=False, indent=2))
278         return 0
279     except (OSError, ValueError, TypeError, subprocess.CalledProcessError) as exc:
280         parser.exit(2, f"error: {exc}\n")
281
282
283 if __name__ == "__main__":
284     raise SystemExit(main())

```

F.70 figure_typography.py

类别：图形公共工具。原始代码共 138 行。

公共工具：图形样式、导出或字体核查依赖，不参与物理方程。

```

1  """Resolve actual figure font faces; do not infer a face from a TTC filename.
2
3  No downloads, font extraction, global style changes or manuscript-font changes.
4  Matplotlib's public get_font/findfont APIs preserve collection face indices on
5  supported versions. Older backends must fail/choose another face, not claim SC
6  while actually rendering TC. Font availability is not an aesthetic certificate.
7  """
8  from __future__ import annotations
9
10 import hashlib
11 from pathlib import Path
12 import re
13
14 LATIN = ("Arial", "Helvetica", "Liberation Sans", "Nimbus Sans", "DejaVu Sans")
15 CJK_SC = ("Noto Sans CJK SC", "Source Han Sans SC", "Heiti SC", "Microsoft YaHei", "PingFang SC")
16
17
18 def font_identity(path):
19     from matplotlib import font_manager as fm
20
21     font = fm.get_font(path) # Do not str()/Path() a FontPath before font loading.
22     with Path(path).open("rb") as stream:
23         digest = hashlib.file_digest(stream, "sha256").hexdigest()
24     return {"path": str(path), "face_index": getattr(path, "face_index", 0),
25           "family": font.family_name, "style": font.style_name,
26           "postscript_name": font.postscript_name, "file_sha256": digest}
27

```

```

28
29 def _choose(families, sample):
30     from matplotlib import font_manager as fm
31
32     for family in families:
33         try:
34             path = fm.findfont(fm.FontProperties(family=[family], weight="normal", style="normal"),
35                                 fallback_to_default=False)
36             face = fm.get_font(path)
37             if face.family_name.casefold() != family.casefold():
38                 continue
39             if any(word in face.style_name.lower() for word in ("bold", "black", "heavy",
40 "italic", "oblique")):
41                 continue
42             if not all(ord(char) in face.get_charmap() for char in sample):
43                 continue
44             # This route exports TrueType PDF text. Avoid silently accepting a
45             # CFF/variable collection that this backend may mis-embed as Type 42.
46             from fontTools.ttLib import TTFont
47             with TTFont(str(path), fontNumber=getattr(path, "face_index", 0), lazy=True) as tt:
48                 if "glyf" not in tt:
49                     continue
50                 return font_identity(path)
51             except (OSError, ValueError, RuntimeError):
52                 continue
53         raise ValueError(f"No verified regular TrueType face for {families}; select a supported local
54 font explicitly")
55
56 def resolve_figure_fonts(language="en"):
57     """Return scoped rc defaults and an actual file/face receipt, not a font menu."""
58     if not isinstance(language, str) or language not in {"en", "en-US", "en-GB", "zh", "zh-CN",
59 "zh-Hans"}:
60         raise ValueError("Automatic typography supports English or simplified Chinese; use
61 explicit font overrides otherwise")
62     latin = _choose(LATIN, "0123456789xyz%-")
63     faces = {"latin": latin}
64     families = [latin["family"]]
65     if language.startswith("zh"):
66         cjk = _choose(CJK_SC, "  镜面厚度反射率月份输出")
67         faces["cjk"] = cjk
68         families.append(cjk["family"])
69     rc = {"font.family": families, "font.sans-serif": families,
70 "font.weight": "normal", "axes.labelweight": "normal", "axes.titleweight": "normal",
71 "mathtext.fontset": "custom", "mathtext.rm": latin["family"],
72 "mathtext.it": latin["family"] + ":italic", "mathtext.bf": latin["family"] + ":bold",
73 "mathtext.sf": latin["family"], "mathtext.fallback": "stix"}
74     return rc, {"language": language, "faces": faces,

```

```

72         "scope": "defaults_before_explicit_overrides; inspect final PDF",
73         "aesthetic_acceptance": "not_assessed"}
74
75
76 def inspect_figure_typography(fig, *, final_width_mm=None):
77     """Read-only text/face/scale screening; math and glyph appearance need PDF review."""
78     from matplotlib import font_manager as fm
79     from matplotlib.text import Text
80
81     fig.canvas.draw()
82     width = fig.get_figwidth() * 25.4
83     scale = 1 if final_width_mm is None else final_width_mm / width
84     if not 0 < scale < float("inf"):
85         raise ValueError("final_width_mm must be positive and finite")
86     faces, warnings, sizes = {}, [], []
87     for item in fig.findobj(Text):
88         text = item.get_text()
89         if not item.get_visible() or not text.strip():
90             continue
91         sizes.append(item.get_fontsize() * scale)
92         prop = item.get_fontproperties()
93         plain = re.sub(r"^\$.*\$", "", text)
94         if "$" in text and re.search(r"[\u4e00-\u9fff]", plain):
95             warnings.append(f"Mixed CJK/math Text may lose glyph fallback; use mixed_textbox:
96                             {text}")
97         if re.search(r"\b[A-Za-z]_[A-Za-z0-9]+\b", plain):
98             warnings.append(f"Literal subscript-like label: {text}")
99         paths = [prop.get_file()] if prop.get_file() else []
100        if not paths:
101            for family in prop.get_family():
102                candidate = prop.copy()
103                candidate.set_family([family])
104                try:
105                    paths.append(fm.findfont(candidate, fallback_to_default=False))
106                except ValueError:
107                    warnings.append(f"Unresolved requested font: {family}")
108        for path in paths:
109            key = (str(path), getattr(path, "face_index", 0))
110            if key not in faces:
111                faces[key] = font_identity(path)
112            if prop.get_file() and faces[key]["family"].endswith(" TC") and
113                re.search(r"[\u4e00-\u9fff]", plain):
114                warnings.append("File-bound TC face in Chinese labels; verify language/collection
115                                index")
116    return {"font_faces": list(faces.values()), "minimum_final_font_pt": min(sizes) if sizes else
117            None,
118            "warnings": sorted(set(warnings)), "math_glyphs_and_visual_quality":
119            "require_final_export_review"}

```

```

115
116
117 def mixed_textbox(text, *, fontsize=8.5, color="black"):
118     """An OffsetBox for inline CJK + math without sending CJK through MathText.
119
120     Place with AnnotationBbox (including leader arrows) or an anchored offset box.
121     Preserves the original text runs; does not interpret or rewrite mathematics.
122     """
123     from matplotlib.offsetbox import HPacker, TextArea, VPacker
124
125     if not isinstance(text, str) or not text.strip():
126         raise ValueError("text must be nonempty")
127     if text.count("$") % 2:
128         raise ValueError("unpaired math delimiters")
129     rows = []
130     for line in text.splitlines():
131         parts = re.split(r"(\${^}$*\$)", line)
132         if any(part.startswith("$") and re.search(r"[\u4e00-\u9fff]", part) for part in parts):
133             raise ValueError("keep Chinese prose outside math runs")
134         children = [TextArea(part, textprops={"fontsize": fontsize, "color": color})
135                     for part in parts if part]
136         if children:
137             rows.append(HPacker(children=children, align="baseline", pad=0, sep=0))
138     return rows[0] if len(rows) == 1 else VPacker(children=rows, align="left", pad=0, sep=2)

```

F.71 _common.py

类别：图形公共工具。原始代码共 136 行。

公共工具：图形样式、导出或字体核查依赖，不参与物理方程。

```

1  #!/usr/bin/env python3
2  """Shared, network-free safety helpers for visualization command-line tools."""
3
4  from __future__ import annotations
5
6  import json
7  import os
8  import stat
9  import tempfile
10 from pathlib import Path
11 from typing import Any
12
13 MAX_INPUT_BYTES = 200 * 1024 * 1024
14 MAX_REPORT_BYTES = 4 * 1024 * 1024
15
16
17 class CliError(ValueError):
18     """An expected command-line validation error."""
19

```

```

20
21 def checked_input_file(
22     value: str | os.PathLike[str],
23     *,
24     max_bytes: int = MAX_INPUT_BYTES,
25 ) -> Path:
26     """Return a bounded regular input file, rejecting symlinks."""
27     path = Path(value).expanduser()
28     if path.is_symlink():
29         raise CliError(f"input must not be a symlink: {path}")
30     try:
31         info = path.stat()
32     except OSError as exc:
33         raise CliError(f"cannot access input file {path}: {exc}") from exc
34     if not stat.S_ISREG(info.st_mode):
35         raise CliError(f"input is not a regular file: {path}")
36     if info.st_size > max_bytes:
37         raise CliError(
38             f"input is {info.st_size} bytes; limit is {max_bytes} bytes"
39         )
40     return path.resolve()
41
42
43 def checked_output_file(
44     value: str | os.PathLike[str],
45     *,
46     force: bool = False,
47     mkdir: bool = False,
48 ) -> Path:
49     """Validate an explicit output path without following a destination symlink."""
50     path = Path(value).expanduser()
51     if path.name in {"", ".", ".."}:
52         raise CliError("output must name a file")
53     if path.is_symlink():
54         raise CliError(f"output must not be a symlink: {path}")
55
56     parent = path.parent
57     if mkdir:
58         parent.mkdir(parents=True, exist_ok=True)
59     if not parent.exists() or not parent.is_dir():
60         raise CliError(f"output parent directory does not exist: {parent}")
61     if parent.is_symlink():
62         raise CliError(f"output parent must not be a symlink: {parent}")
63
64     if path.exists():
65         if not path.is_file():
66             raise CliError(f"output exists and is not a regular file: {path}")
67         if not force:

```

```

68         raise CliError(f"refusing to overwrite existing output: {path}")
69     return parent.resolve() / path.name
70
71
72 def atomic_write_bytes(path: Path, payload: bytes, *, force: bool = False) -> None:
73     """Write bytes through a same-directory temporary file."""
74     destination = checked_output_file(path, force=force)
75     descriptor, temporary_name = tempfile.mkstemp(
76         prefix=f".{destination.name}.",
77         suffix=".tmp",
78         dir=destination.parent,
79     )
80     temporary = Path(temporary_name)
81     try:
82         with os.fdopen(descriptor, "wb") as handle:
83             handle.write(payload)
84             handle.flush()
85             os.fsync(handle.fileno())
86             os.chmod(temporary, 0o600)
87             if destination.exists() and not force:
88                 raise CliError(f"refusing to overwrite existing output: {destination}")
89             os.replace(temporary, destination)
90     finally:
91         try:
92             temporary.unlink()
93         except FileNotFoundError:
94             pass
95
96
97 def emit_json(
98     document: dict[str, Any],
99     *,
100     output: str | os.PathLike[str] | None = None,
101     force: bool = False,
102 ) -> None:
103     """Print deterministic JSON or write it safely to an explicit file."""
104     payload = (
105         json.dumps(document, indent=2, sort_keys=True, ensure_ascii=False) + "\n"
106     ).encode("utf-8")
107     if len(payload) > MAX_REPORT_BYTES:
108         raise CliError(
109             f"report is {len(payload)} bytes; limit is {MAX_REPORT_BYTES} bytes"
110         )
111     if output is None:
112         print(payload.decode("utf-8"), end="")
113     return
114     atomic_write_bytes(Path(output), payload, force=force)
115

```

```

116
117 def positive_int(value: str) -> int:
118     """Argparse converter for strictly positive integers."""
119     try:
120         parsed = int(value)
121     except ValueError as exc:
122         raise CliError(f"expected an integer, got {value!r}") from exc
123     if parsed <= 0:
124         raise CliError("value must be greater than zero")
125     return parsed
126
127
128 def positive_float(value: str) -> float:
129     """Argparse converter for finite, strictly positive floats."""
130     try:
131         parsed = float(value)
132     except ValueError as exc:
133         raise CliError(f"expected a number, got {value!r}") from exc
134     if not (parsed > 0 and parsed < float("inf")):
135         raise CliError("value must be finite and greater than zero")
136     return parsed

```

F.72 figure_export.py

类别：图形公共工具。原始代码共 666 行。

公共工具：图形样式、导出或字体核查依赖，不参与物理方程。

```

1  #!/usr/bin/env python3
2  """Safe Matplotlib export helpers with explicit, auditable settings.
3
4  This module does not certify journal compliance. Publisher profiles are dated
5  planning snapshots and must be confirmed against the target journal.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import importlib.metadata
12 import json
13 import math
14 import os
15 import sys
16 import tempfile
17 import xml.etree.ElementTree as ET
18 from pathlib import Path
19 from typing import Any, Iterable
20
21 from _common import CliError, checked_output_file, emit_json, positive_float
22

```

```

23 SCHEMA_VERSION = "1.0"
24 SUPPORTED_FORMATS = {
25     "pdf",
26     "svg",
27     "eps",
28     "ps",
29     "png",
30     "tiff",
31     "jpeg",
32     "webp",
33 }
34 FORMAT_ALIASES = {"jpg": "jpeg", "tif": "tiff"}
35 VECTOR_FORMATS = {"pdf", "svg", "eps", "ps"}
36 RASTER_FORMATS = {"png", "tiff", "jpeg", "webp"}
37 METADATA_FORMATS = {"pdf", "svg", "png", "eps", "ps"}
38 MAX_FORMATS = 8
39
40
41 def _normalize_formats(formats: Iterable[str]) -> list[str]:
42     normalized: list[str] = []
43     for item in formats:
44         for value in item.split(","):
45             raw_name = value.strip().lower().lstrip(".")
46             name = FORMAT_ALIASES.get(raw_name, raw_name)
47             if not name:
48                 continue
49             if name not in SUPPORTED_FORMATS:
50                 raise CliError(
51                     f"unsupported format {name!r}; "
52                     f"available: {' ', ' '.join(sorted(SUPPORTED_FORMATS))}"
53                 )
54             if name not in normalized:
55                 normalized.append(name)
56     if not normalized:
57         raise CliError("at least one output format is required")
58     if len(normalized) > MAX_FORMATS:
59         raise CliError(f"at most {MAX_FORMATS} output formats are allowed")
60     return normalized
61
62
63 def _base_output_path(value: str | os.PathLike[str]) -> Path:
64     path = Path(value).expanduser()
65     if path.suffix.lower().lstrip(".") in SUPPORTED_FORMATS | set(FORMAT_ALIASES):
66         path = path.with_suffix("")
67     if not path.name:
68         raise CliError("output must include a base filename")
69     return path
70

```

```

71
72 def _font_rc(font_mode: str) -> dict[str, Any]:
73     if font_mode == "current":
74         return {}
75     if font_mode == "truetype":
76         return {
77             "pdf.fonttype": 42,
78             "ps.fonttype": 42,
79             # Editable SVG text; unlike PDF Type 42, this is not embedded.
80             "svg.fonttype": "none",
81         }
82     if font_mode == "paths":
83         return {
84             "pdf.fonttype": 3,
85             "ps.fonttype": 3,
86             "svg.fonttype": "path",
87         }
88     raise CliError("font_mode must be 'truetype', 'paths', or 'current'")
89
90
91 def _package_version(distribution: str) -> str | None:
92     try:
93         return importlib.metadata.version(distribution)
94     except importlib.metadata.PackageNotFoundError:
95         return None
96
97
98 def _atomic_savefig(
99     fig: Any,
100     destination: Path,
101     *,
102     format_name: str,
103     save_kwargs: dict[str, Any],
104     overwrite: bool,
105     accessibility: dict[str, str] | None = None,
106 ) -> None:
107     destination = checked_output_file(destination, force=overwrite)
108     descriptor, temporary_name = tempfile.mkstemp(
109         prefix=f"{destination.stem}.",
110         suffix=f"{format_name}",
111         dir=destination.parent,
112     )
113     os.close(descriptor)
114     temporary = Path(temporary_name)
115     try:
116         # Matplotlib expects to create the target on some backends.
117         temporary.unlink()
118         fig.savefig(temporary, format=format_name, **save_kwargs)

```

```

119         if not temporary.exists() or temporary.stat().st_size == 0:
120             raise CliError(f"Matplotlib produced no data for {destination}")
121         if format_name == "svg" and accessibility:
122             namespace = "http://www.w3.org/2000/svg"
123             ET.register_namespace("", namespace)
124             ET.register_namespace("xlink", "http://www.w3.org/1999/xlink")
125             tree = ET.parse(temporary)
126             root = tree.getroot()
127             for index, name in enumerate(("title", "desc")):
128                 element = root.find(f"{{{namespace}}}{name}")
129                 if element is None:
130                     element = ET.Element(f"{{{namespace}}}{name}")
131                     root.insert(index, element)
132                     element.text = accessibility[name]
133             tree.write(temporary, encoding="utf-8", xml_declaration=True)
134         if destination.exists() and not overwrite:
135             raise CliError(f"refusing to overwrite existing output: {destination}")
136         os.replace(temporary, destination)
137     except CliError:
138         raise
139     except Exception as exc:
140         raise CliError(
141             f"failed to export {destination.name} as {format_name}: {exc}"
142         ) from exc
143     finally:
144         try:
145             temporary.unlink()
146         except FileNotFoundError:
147             pass
148
149
150 def _validate_provenance(provenance: dict[str, Any] | None) -> dict[str, Any]:
151     if provenance is None:
152         return {}
153     if not isinstance(provenance, dict):
154         raise CliError("provenance must be a dictionary")
155     try:
156         encoded = json.dumps(provenance, sort_keys=True).encode("utf-8")
157     except (TypeError, ValueError) as exc:
158         raise CliError(f"provenance must be JSON serializable: {exc}") from exc
159     if len(encoded) > 1_000_000:
160         raise CliError("provenance metadata is limited to 1,000,000 bytes")
161     return provenance
162
163
164 def export_figure(
165     fig: Any,
166     filename: str | os.PathLike[str],

```

```

167     *,
168     formats: Iterable[str] = ("pdf", "png"),
169     dpi: float = 300,
170     transparent: bool = False,
171     bbox_inches: str | None = None,
172     pad_inches: float = 0.1,
173     facecolor: str = "white",
174     edgecolor: str = "none",
175     font_mode: str = "truetype",
176     overwrite: bool = False,
177     mkdir: bool = False,
178     metadata: dict[str, Any] | None = None,
179     provenance: dict[str, Any] | None = None,
180     write_manifest: bool = False,
181     savefig_kwargs: dict[str, Any] | None = None,
182     accessibility: dict[str, str] | None = None,
183 ) -> dict[str, Any]:
184     """Export a Matplotlib figure atomically with explicit format settings.
185
186     ``dpi`` is passed to vector backends too because it controls rasterized
187     artists and embedded raster images. ``bbox_inches=None`` preserves the
188     figure's physical page dimensions; use ``"tight"`` only when that change is
189     intentional.
190
191     if accessibility is not None and (
192         not isinstance(accessibility, dict)
193         or any(not isinstance(accessibility.get(k), str) or not accessibility[k].strip()
194               for k in ("title", "desc"))
195     ):
196         raise CliError("accessibility requires nonempty title and desc strings")
197     if not (float(dpi) > 0 and math.isfinite(float(dpi))):
198         raise CliError("dpi must be finite and greater than zero")
199     if not (float(pad_inches) >= 0 and math.isfinite(float(pad_inches))):
200         raise CliError("pad_inches must be finite and non-negative")
201     normalized_formats = _normalize_formats(formats)
202     base = _base_output_path(filename)
203     parent = base.parent
204     if mkdir:
205         parent.mkdir(parents=True, exist_ok=True)
206     if not parent.exists() or not parent.is_dir():
207         raise CliError(f"output parent directory does not exist: {parent}")
208     if parent.is_symlink():
209         raise CliError(f"output parent must not be a symlink: {parent}")
210     for format_name in normalized_formats:
211         checked_output_file(
212             parent / f"{base.name}.{format_name}",
213             force=overwrite,
214         )

```

```

215     if write_manifest:
216         checked_output_file(
217             parent / f"{base.name}.export.json",
218             force=overwrite,
219         )
220
221     try:
222         import matplotlib as mpl
223     except ImportError as exc:
224         raise CliError(
225             "Matplotlib is required for figure export; "
226             "run with --with 'matplotlib==3.11.1'"
227         ) from exc
228
229     provenance_document = _validate_provenance(provenance)
230     user_metadata = dict(metadata or {})
231     creator = user_metadata.setdefault(
232         "Creator",
233         f"lab-ultra-scientific-visualization figure_export.py; Matplotlib {mpl.__version__}",
234     )
235     if not isinstance(creator, str):
236         raise CliError("metadata Creator must be a string")
237     extra_kwargs = dict(savefig_kwargs or {})
238     if "format" in extra_kwargs:
239         raise CliError("savefig_kwargs must not override format")
240     protected = {
241         "dpi",
242         "transparent",
243         "bbox_inches",
244         "pad_inches",
245         "facecolor",
246         "edgecolor",
247         "metadata",
248     }
249     overlap = protected.intersection(extra_kwargs)
250     if overlap:
251         raise CliError(
252             "savefig_kwargs must not override explicit settings: "
253             + ", ".join(sorted(overlap))
254         )
255
256     figure_size = [float(value) for value in fig.get_size_inches()]
257     outputs: list[dict[str, Any]] = []
258     warnings_list: list[str] = []
259     export_rc = _font_rc(font_mode)
260     if bbox_inches is None:
261         # Prevent a user's global savefig.bbox="tight" from changing page size.
262         export_rc["savefig.bbox"] = None

```

```

263     with mpl.rc_context(export_rc):
264         for format_name in normalized_formats:
265             destination = parent / f"{base.name}.{format_name}"
266             checked_output_file(destination, force=overwrite)
267             if format_name in {"eps", "ps"}:
268                 format_metadata = {"Creator": user_metadata["Creator"]}
269             elif format_name in METADATA_FORMATS:
270                 format_metadata = user_metadata
271             else:
272                 format_metadata = None
273             if user_metadata and format_name not in METADATA_FORMATS:
274                 warnings_list.append(
275                     f"{format_name}: Matplotlib does not support savefig metadata "
276                     "for this format; metadata remains in the optional manifest."
277                 )
278             save_kwargs: dict[str, Any] = {
279                 "dpi": float(dpi),
280                 "transparent": transparent,
281                 "bbox_inches": bbox_inches,
282                 "pad_inches": float(pad_inches),
283                 "facecolor": "none" if transparent else facecolor,
284                 "edgecolor": "none" if transparent else edgecolor,
285                 **extra_kwargs,
286             }
287             if format_metadata is not None:
288                 save_kwargs["metadata"] = format_metadata
289             if format_name == "tiff":
290                 save_kwargs["pil_kwargs"] = {"compression": "tiff_lzw"}
291
292             _atomic_savefig(
293                 fig,
294                 destination,
295                 format_name=format_name,
296                 save_kwargs=save_kwargs,
297                 overwrite=overwrite,
298                 accessibility=accessibility,
299             )
300             outputs.append(
301                 {
302                     "path": str(destination.resolve()),
303                     "format": format_name,
304                     "kind": (
305                         "vector-container"
306                         if format_name in VECTOR_FORMATS
307                         else "raster"
308                     ),
309                     "size_bytes": destination.stat().st_size,
310                 }

```

```

311         )
312
313     report: dict[str, Any] = {
314         "schema_version": SCHEMA_VERSION,
315         "outputs": outputs,
316         "settings": {
317             "formats": normalized_formats,
318             "dpi": float(dpi),
319             "transparent": transparent,
320             "bbox_inches": bbox_inches,
321             "pad_inches": float(pad_inches),
322             "facecolor": facecolor,
323             "font_mode": font_mode,
324             "figure_size_inches": figure_size,
325         },
326         "versions": {
327             "matplotlib": mpl.__version__,
328             "pillow": _package_version("Pillow"),
329         },
330         "provenance": provenance_document,
331         "accessibility": accessibility,
332         "warnings": sorted(set(warnings_list)),
333         "notice": (
334             "Export completed with explicit settings. This report does not "
335             "establish visual, accessibility, scientific-integrity, or "
336             "journal-submission compliance."
337         ),
338     }
339
340     if write_manifest:
341         manifest_path = parent / f"{base.name}.export.json"
342         emit_json(report, output=manifest_path, force=overwrite)
343         report["manifest"] = str(manifest_path.resolve())
344     return report
345
346
347 def save_publication_figure(
348     fig: Any,
349     filename: str | os.PathLike[str],
350     formats: Iterable[str] = ("pdf", "png"),
351     dpi: float = 300,
352     transparent: bool = False,
353     bbox_inches: str | None = None,
354     pad_inches: float = 0.1,
355     facecolor: str = "white",
356     *,
357     overwrite: bool = False,
358     provenance: dict[str, Any] | None = None,

```

```

359     write_manifest: bool = False,
360     **kwargs: Any,
361 ) -> list[Path]:
362     """Compatibility wrapper returning saved paths.
363
364     Unlike the previous implementation, this function refuses implicit
365     overwrite, does not silently swallow export errors, does not cap vector
366     DPI, and preserves figure dimensions unless ``bbox_inches="tight"`` is
367     requested.
368     """
369     report = export_figure(
370         fig,
371         filename,
372         formats=formats,
373         dpi=dpi,
374         transparent=transparent,
375         bbox_inches=bbox_inches,
376         pad_inches=pad_inches,
377         facecolor=facecolor,
378         overwrite=overwrite,
379         provenance=provenance,
380         write_manifest=write_manifest,
381         savefig_kwargs=kwargs,
382     )
383     return [Path(item["path"]) for item in report["outputs"]]
384
385
386 def _load_profiles() -> dict[str, Any]:
387     profile_path = Path(__file__).resolve().parents[1] / "assets" / "publisher_profiles.json"
388     try:
389         document = json.loads(profile_path.read_text(encoding="utf-8"))
390     except (OSError, json.JSONDecodeError) as exc:
391         raise CliError(f"cannot load publisher profiles: {exc}") from exc
392     if not isinstance(document.get("profiles"), dict):
393         raise CliError("publisher profile asset is malformed")
394     return document
395
396
397 def check_figure_size(
398     fig: Any,
399     journal: str = "nature",
400     *,
401     tolerance_mm: float = 1.0,
402 ) -> dict[str, Any]:
403     """Compare figure dimensions with a dated profile without claiming compliance."""
404     if not (tolerance_mm >= 0 and math.isfinite(tolerance_mm)):
405         raise CliError("tolerance_mm must be finite and non-negative")
406     document = _load_profiles()

```

```

407     try:
408         profile = document["profiles"][journal]
409     except KeyError as exc:
410         raise CliError(
411             f"unknown profile {journal!r}; "
412             f"available: {'', ' '.join(sorted(document['profiles']))}"
413         ) from exc
414     width_inches, height_inches = (float(value) for value in fig.get_size_inches())
415     width_mm = width_inches * 25.4
416     height_mm = height_inches * 25.4
417     candidates = {
418         name: float(value)
419         for name, value in (profile.get("widths_mm") or {}).items()
420     }
421     nearest_name = None
422     nearest_width = None
423     difference = None
424     if candidates:
425         nearest_name, nearest_width = min(
426             candidates.items(), key=lambda item: abs(width_mm - item[1])
427         )
428         difference = abs(width_mm - nearest_width)
429     max_height = profile.get("max_height_mm")
430     return {
431         "schema_version": SCHEMA_VERSION,
432         "profile": journal,
433         "profile_label": profile["label"],
434         "profile_scope": profile["scope"],
435         "profile_accessed": document["accessed"],
436         "figure": {
437             "width_inches": width_inches,
438             "height_inches": height_inches,
439             "width_mm": width_mm,
440             "height_mm": height_mm,
441         },
442         "nearest_profile_width": {
443             "name": nearest_name,
444             "width_mm": nearest_width,
445             "difference_mm": difference,
446             "matches_tolerance": (
447                 difference is not None and difference <= tolerance_mm
448             ),
449         },
450         "height_screen": {
451             "maximum_mm": max_height,
452             "within_snapshot": (
453                 None if max_height is None else height_mm <= float(max_height)
454             ),

```

```

455         },
456         "notice": (
457             "A dimensional match to this dated snapshot is not a journal "
458             "compliance determination."
459         ),
460     }
461
462
463 def save_for_journal(
464     fig: Any,
465     filename: str | os.PathLike[str],
466     journal: str,
467     figure_type: str = "combination",
468     *,
469     confirm_profile: bool = False,
470     format_name: str | None = None,
471     dpi: float | None = None,
472     overwrite: bool = False,
473 ) -> list[Path]:
474     """Use a dated profile only after explicit caller confirmation.
475
476     This compatibility helper intentionally requires ``confirm_profile=True``.
477     Prefer ``export_plan.py`` and ``export_figure`` for an auditable workflow.
478     """
479     if not confirm_profile:
480         raise CliError(
481             "publisher rules are date-sensitive; run export_plan.py, verify the "
482             "target journal, then pass confirm_profile=True if this snapshot is "
483             "appropriate"
484         )
485     document = _load_profiles()
486     try:
487         profile = document["profiles"][journal]
488     except KeyError as exc:
489         raise CliError(f"unknown publisher profile: {journal!r}") from exc
490     profile_formats = profile.get("formats") or {}
491     allowed = profile_formats.get(figure_type) or []
492     selected = (
493         _normalize_formats([format_name])[0]
494         if format_name is not None
495         else next((name for name in allowed if name in SUPPORTED_FORMATS), None)
496     )
497     if selected is None:
498         raise CliError(
499             f"profile {journal!r} has no directly exportable {figure_type!r} "
500             "format; choose settings manually"
501         )
502     if selected not in allowed:

```

```

503         raise CliError(
504             f"format {selected!r} is not recorded for {journal} "
505             f"{figure_type}; recorded formats: {'', '.join(allowed)}"
506         )
507     if dpi is None:
508         raise CliError(
509             "provide an explicit dpi after reviewing embedded raster content; "
510             "the compatibility helper does not choose one automatically"
511         )
512     dpi_profile = (profile.get("raster_dpi") or {}).get(figure_type) or {}
513     if dpi_profile.get("min") is not None and dpi < float(dpi_profile["min"]):
514         raise CliError(f"dpi is below the profile snapshot minimum: {dpi_profile}")
515     if (
516         dpi_profile.get("min_exclusive") is not None
517         and dpi <= float(dpi_profile["min_exclusive"])
518     ):
519         raise CliError(
520             f"dpi does not exceed the profile snapshot threshold: {dpi_profile}"
521         )
522     if dpi_profile.get("max") is not None and dpi > float(dpi_profile["max"]):
523         raise CliError(f"dpi is above the profile snapshot maximum: {dpi_profile}")
524     return save_publication_figure(
525         fig,
526         filename,
527         formats=[selected],
528         dpi=dpi,
529         overwrite=overwrite,
530         provenance={
531             "publisher_profile": journal,
532             "profile_accessed": document["accessed"],
533             "profile_scope": profile["scope"],
534             "caller_confirmed_snapshot": True,
535         },
536         write_manifest=True,
537     )
538
539
540 def verify_font_embedding(pdf_path: str | os.PathLike[str]) -> dict[str, Any]:
541     """Return a conservative PDF font-resource report using the metadata tool."""
542     try:
543         from image_metadata import inspect_file
544     except ImportError as exc:
545         raise CliError(f"cannot import image_metadata.py: {exc}") from exc
546     report = inspect_file(pdf_path)
547     if report["metadata"]["format"] != "PDF":
548         raise CliError("font embedding inspection requires a PDF input")
549     return {
550         "path": report["input"]["path"],

```

```

551         "font_resources": report["metadata"].get("font_resources"),
552         "notice": (
553             "Inspection covers declared first-page font resources only; "
554             "publisher preflight may apply additional checks."
555         ),
556     }
557
558
559 def _demo_figure() -> Any:
560     try:
561         import matplotlib.pyplot as plt
562     except ImportError as exc:
563         raise CliError(
564             "Matplotlib is required for --demo; "
565             "run with --with 'matplotlib==3.11.1'"
566         ) from exc
567     x_values = [index / 20.0 for index in range(121)]
568     first = [math.sin(value) for value in x_values]
569     second = [math.cos(value) for value in x_values]
570     fig, ax = plt.subplots(figsize=(3.5, 2.5), layout="constrained")
571     ax.plot(x_values, first, label="sin(x)", marker="o", markevery=20)
572     ax.plot(
573         x_values,
574         second,
575         label="cos(x)",
576         linestyle="--",
577         marker="s",
578         markevery=20,
579     )
580     ax.axhline(0, color="0.35", linewidth=0.7)
581     ax.set(xlabel="Angle (radians)", ylabel="Amplitude (unitless)")
582     ax.legend()
583     return fig
584
585
586 def build_parser() -> argparse.ArgumentParser:
587     parser = argparse.ArgumentParser(
588         description=(
589             "Render a deterministic Matplotlib export smoke-test. Programmatic "
590             "helpers in this module support explicit, safe figure export."
591         )
592     )
593     parser.add_argument(
594         "--demo",
595         metavar="BASE_PATH",
596         help="render a deterministic demo to this base path",
597     )
598     parser.add_argument(

```

```

599     "--formats",
600     default="pdf,png",
601     help="comma-separated formats (default: pdf,png)",
602 )
603 parser.add_argument("--dpi", type=positive_float, default=300.0)
604 parser.add_argument(
605     "--font-mode",
606     choices=("truetype", "paths", "current"),
607     default="truetype",
608 )
609 parser.add_argument("--transparent", action="store_true")
610 parser.add_argument(
611     "--tight",
612     action="store_true",
613     help="use bbox_inches='tight' (changes physical page dimensions)",
614 )
615 parser.add_argument("--manifest", action="store_true")
616 parser.add_argument("--force", action="store_true")
617 parser.add_argument(
618     "--style",
619     choices=("default", "nature", "science", "cell", "minimal", "presentation"),
620     default="default",
621 )
622 parser.add_argument("--palette", default="okabe_ito_on_white")
623 return parser
624
625
626 def main(argv: list[str] | None = None) -> int:
627     parser = build_parser()
628     try:
629         args = parser.parse_args(argv)
630         if not args.demo:
631             parser.error("--demo BASE_PATH is required")
632         try:
633             from style_presets import style_context
634         except ImportError as exc:
635             raise CliError(f"cannot import style_presets.py: {exc}") from exc
636         with style_context(args.style, palette_name=args.palette):
637             fig = _demo_figure()
638             try:
639                 report = export_figure(
640                     fig,
641                     args.demo,
642                     formats=args.formats.split(","),
643                     dpi=args.dpi,
644                     transparent=args.transparent,
645                     bbox_inches="tight" if args.tight else None,
646                     font_mode=args.font_mode,

```

```

647         overwrite=args.force,
648         provenance={
649             "purpose": "deterministic export smoke test",
650             "raw_data": "analytical sin/cos values generated in code",
651             "transformations": ["none"],
652         },
653         write_manifest=args.manifest,
654     )
655     finally:
656         import matplotlib.pyplot as plt
657
658         plt.close(fig)
659         emit_json(report)
660         return 0
661     except CliError as exc:
662         parser.exit(2, f"error: {exc}\n")
663
664
665 if __name__ == "__main__":
666     sys.exit(main())

```